! > 4 5 @ 6 0 = 8 5

Предисловие

Информационные технологии являются составной частью научного направления «Информатика» и базируются на ее достижениях. Информатизация как процесс перехода к информационному обществу сопровождается возникновением новых и интенсивным развитием существующих информационных технологий. Информация превращается в коммерческий ресурс, способствуя получению прибыли при внедрении информационных технологий во многие сферы человеческой деятельности. Возникают информационная экономика, новая информационная инфраструктура промышленности и социальной сферы, формируется информационная культура. Достижения информатики могут позитивно воздействовать на развитие общества при приоритетном развитии образования. Существует мнение, что преодоление современного кризиса мировой образовательной системы возможно на основе информатизации.

Зарубежный и российский опыт внедрения информационных технологий подтверждает высокую экономическую эффективность для многих сфер применения.

В своем становлении любая отрасль, в том числе и информационная, проходила стадии от кустарного ремесленного производства к производству, основанному на высоких технологиях.

Информационные технологии обеспечивают переход от рутинных к промышленным методам и средствам работы с информацией в различных сферах человеческой деятельности, обеспечивая рациональное и эффективное ее использование.

В данном курсе с позиций системного подхода, теории информации, теории моделирования, искусственного интеллекта и других прикладных наук информатики предлагается подход к рассмотрению информационных технологий как науки о промышленных способах ее переработки, преобразования и использования.

Необходимость создания данного курса связана с широким развитием информационных технологий и использованием их в различных предметных областях. Овладение основами информационных технологий позволяет принципиально на новых позициях решать проблемы использования информационных ресурсов. Однако содержание, модели, методы и средства современных информационных технологий слабо раскрыты в технической литературе, доступ к которой ограничен. Авторами предпринята попытка системного

изложения вопросов теории и практического применения информационных технологий в различных приложениях с использованием современных компьютерных средств.

Учебник включает семь тем, объединяющих 29 пунктов.

В первой теме последовательно рассматриваются понятия, виды и свойства информации, даются фундаментальные определения информации, ее количественные и качественные оценки, ставится проблема превращения информации в ресурс. Информационные технологии рассматриваются авторами как система. Определяются основные понятия и задачи информационной технологии, приводятся этапы эволюции. Во второй теме раскрываются базовые информационные процессы, входящие в состав информационных технологий. Для каждого из рассматриваемых процессов, таких как извлечение информации, транспортирование, обработка, хранение, представление и использование информации, дается подробная характеристика с раскрытием моделей и современного состояния. Непосредственно содержанию информационных технологий посвящена третья тема курса, в ней детально раскрываются базовые информационные технологии, к которым отнесены: мультимедиатехнологии, геоинформационные, технологии защиты информации, *CASE*-технологии, телекоммуникационные, технологии искусственного интеллекта, технологии программирования, облачные технологии, технологии больших данных. В четвертой теме рассматривается создание искусственного интеллекта, его парадигма и эволюция. Вводятся понятия нейрокомпьютинг и машинное обучение.

В курсе авторами приводится анализ прикладных информационных технологий. Ввиду многообразия предметных технологий в основном рассмотрены технологии корпоративного управления, которые инвариантны к сфере применения. Особое внимание уделяется современным и перспективным типовым стандартным технологическим и инструментальным средствам, которые полезно применять в различных сферах деятельности. Непрерывное совершенствование существующих и возникновение новых информационных технологий невозможно без развития их инструментальной базы. В курсе дается анализ и приводятся рекомендации по использованию программных, технических и методических средств информационных технологий. Это позволит сориентироваться на сложившемся рынке вычислительной и сетевой продукции с учетом изложенных в книге мировых стандартов, а также самостоятельно выявить тенденции развития и перспективы информационных технологий и информатики в целом. Курс посвящен информационной технологии построения систем, что особо актуально для формирования профессионалов-разработчиков. Приводятся основы системного подхода, интерпретированные к задачам построения инфор-

мационных систем. Раскрываются стадии их разработки. Дается методика построения информационных систем с оценкой их качества.

Курс предназначен для студентов средних специальных учебных заведений, обучающихся по укрупненной группе специальностей «Информатика и вычислительная техника». По своему содержанию курс является вводным ко всем дисциплинам профессионального цикла и направлен на формирование единой концепции информационных технологий, которые раскрываются в других дисциплинах.

Тема 1
ВОЗНИКНОВЕНИЕ И ЭТАПЫ СТАНОВЛЕНИЯ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

1.1. Понятие информации, виды информации

1.1.1. Понятие информации

Процесс движения мирового сообщества сопровождается чередой различных социальных и промышленных революций, что наглядно видно на рис. 1.1 [1]. Каждый такой переход помимо основательного изменения политического и экономического устройства приводил к значительному прорыву в индустриальном развитии.

На рубеже 1970-х гг. достижения науки привели к созданию высокотехнологичного промышленного производства. Внедрение ЭВМ и сетей передачи данных способствовали революционным процессам в области информационных технологий. В процессе своего развития человечество в любой сфере деятельности последовательно проходило стадии от ручного кустарного труда до высокотехнологичного промышленного производства. В первую очередь усилия были направлены на облегчение физического труда, а информационная сфера долгие годы была уделом умственного труда человека и с каждым годом требовала большего количества трудовых ресурсов. Революционные процессы в области информатизации позволили перейти на промышленный уровень технологий и инструментальных средств.

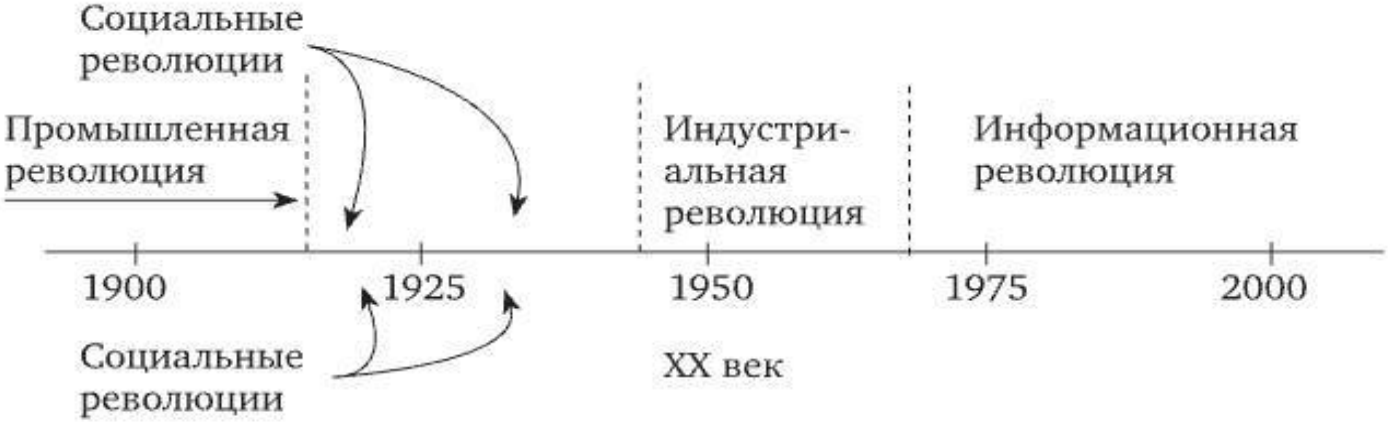


Рис. 1.1. **Последовательность развития мирового сообщества**

Если перейти от развития мирового сообщества к техническим проблемам, то в настоящее время развитие техники и технологии

принято делить на четыре этапа, которые называют также технологическими укладами, переход между которыми связывают с появлением принципиально новых, революционных для своего времени технологий. Смена технологических укладов приводит к резким скачкам производительности и росту экономики. Обычно, эти четыре этапа называют индустриальными революциями.

Первую промышленную революцию (конец XVIII в. — начало XIX в.) связывают с переходом от аграрной экономики к промышленному производству за счет изобретения паровой энергии, механических устройств, развития металлургии.

Вторую промышленную революцию (вторая половина XIX в. — начало XX в.) связывают с активным использованием электрической энергии и с переходом к поточному производству и разделению труда.

Третья промышленная революция (с 1970 г.) — применение в производстве информационных систем, обеспечивших интенсивную автоматизацию и роботизацию производственных процессов.

Четвертую промышленную революцию связывают с переходом на полностью автоматизированное цифровое производство, управляемое интеллектуальными системами в режиме реального времени в постоянном взаимодействии с внешней средой, выходящее за границы одного предприятия, с перспективой объединения в глобальную промышленную сеть вещей и услуг.

В узком смысле Индустрия 4.0 (*Industrie* 4.0) — это название одного из 10 проектов государственной *Hi-Tech* стратегии Германии до 2020 года, описывающего концепцию умного производства (*Smart Manufacturing*) на базе глобальной промышленной сети Интернета вещей и услуг (*Internet of Things and Services*). Следует заметить, что аналогичные программы имеются во многих других странах.

В широком смысле, Индустрия 4.0 характеризует текущий тренд развития автоматизации и обмена данными, который включает в себя киберфизические системы (КФС), Интернет вещей (*IoT*) и облачные вычисления. Индустрия 4.0 представляет собой новый уровень организации производства и управления цепочкой создания стоимости на протяжении всего жизненного цикла выпускаемой продукции.

В качестве основных составных частей Индустрии 4.0 выступают следующие элементы (рис. 1.2).

IoT, искусственный интеллект, машинное обучение и робототехника, облачные вычисления, *Big Data*, аддитивное производство, кибербезопасность, моделирование, дополненная реальность: многие из этих элементов уже давно и успешно применяются на практике, но в данном случае речь идет об их объединении в систему. Именно объединение их в одну целостную систему позволяет развить кон-

цепцию «Индустрии 4.0» и обеспечить новый уровень эффективности производства и дополнительный доход за счет использования цифровых технологий, формирования сетевого взаимодействия поставщиков и партнеров, а также реализации инновационных бизнес-моделей.

Рис. 1.2. **Элементы Индустрии 4.0**

На смену индустриальному этапу развития общества пришла новая эволюционная фаза, фаза информатизации и соответствующая общественно-экономическая формация — информационное общество, при котором наиболее эффективное и динамичное его развитие возможно на основе максимально полного использования имеющихся информационных ресурсов и средств их обработки, составляющих основу соответствующих информационных пространств. Главным ресурсом ускоренного развития современного информационного общества становятся знания, а главным механизмом развития — цифровая экономика, основанная на знаниях. Главными технологиями цифровой экономики становятся новые информационно-коммуникационные технологии (ИКТ), которые уже фактически являются технологиями общего назначения, также как технологии производства тепла и электроэнергии. В новых условиях значительно возрастает значимость информационных процессов, обеспечивающих материальные производства.

Главной компонентой цифрового производства и в целом цифровой экономики становятся разнообразные классы КФС. В перспективных КФС наряду с функциями позиционирования, контроля и диагностики также будут реализованы функции автоматического составления отчетов о состоянии соответствующей подсистемы

контролируемого оборудования, в том числе: данных о всех возникающих неисправностях; об остатке ресурса изнашиваемых деталей; о ресурсе расходных материалов; загрузке оборудования и режиме его эксплуатации. Производственные КФС включают в себя интеллектуальные производственные мощности и складские системы, которые способны взаимодействовать в электронном виде и за счет этого могут быть интегрированы на основе ИКТ на протяжении всего производственного процесса от размещения заказа до производства, маркетинга и исходящей логистики. Указанные возможности открывают широчайшие перспективы по автоматизации и интеллектуализации не только самого цифрового производства продукции, но и ее обслуживания и эксплуатации во время послепродажного функционирования.

Ключевым элементом предлагаемого электронного обслуживания (*e-maintenance*) будет являться базирующееся на веб-технологиях дистанционное администрирование, мониторинг, тестирование, диагностика, прогнозирование состояния эксплуатируемых изделий, реконфигурация их структур, в случае возникновения аварийных и нештатных ситуаций и отсутствия необходимых резервов.

Особый класс КФС представлен формируемым в настоящее время понятием промышленного интернета, включающим интеллектуальные сетевые объекты, облачные, граничные и туманные вычислительные платформы, обеспечивающие сбор, анализ, обмен данными в реальном времени в промышленной среде для повышения производительности труда, уменьшения себестоимости производимой продукции и сокращения расходов ресурсов. Проведенный анализ основных тенденций, связанных с созданием и развитием промышленного интернета, показал, что его основное предназначение связано с реализацией перспективных технологий, способствующих широкомасштабному развитию клиент–ориентированных, мелкосерийных и гибких производств, главным отличием которых от крупных поточных линий является непостоянство (нестационарность) производственных процессов, а вместе с ними характеристик соответствующих информационных потоков и вычислительных процессов. Так, например, становится возможным перенос ряда функций обработки данных ближе к месту их возникновения (к границе с физическим миром), что позволяет снизить сетевые задержки и нагрузки на каналы связи, а также снизить вычислительную нагрузку на ресурсы облачных вычислений. Но при этом возникают проблемы, вызванные низкой производительностью таких «умных» устройств и их энергопотреблением при автономной работе. Таким образом, для эффективного управления описанным выше типом производства необходимо создать новую архитектуру, модели, методы, алгоритмы и программные комплексы управления соответствующей КФС.

Эталонная архитектурная модель Индустрия 4.0, сокращенно *RAMI* 4.0 (*Reference Architecture Model Industrie* 4.0), показанная на рис 1.3, состоит из трехмерной системы координат, которая описывает все важнейшие аспекты Индустрии 4.0. Таким образом, сложные взаимосвязи могут быть разбиты на более мелкие и простые кластеры [2, 3].

Рис. 1.3. **Эталонная архитектурная модель Индустрия 4.0**

1. Ось «Уровни иерархии». На правой горизонтальной оси указаны уровни иерархии из стандарта *IEC* 62264, серии международных стандартов для корпоративных ИТ и систем управления. Эти уровни иерархии представляют различные функциональные возможности на заводах или объектах.
2. Ось «Жизненный цикл и Поток создания ценности» Левая горизонтальная ось представляет жизненный цикл объектов и продуктов, основанный на стандарте *IEC* 62890 для управления жизненным циклом. Кроме того, проводится различие между «типами» и «экземплярами». «Тип» становится «экземпляром», когда проектирование и прототипирование завершены и производится фактический продукт.
3. Ось «Слои» Шесть слоев на вертикальной оси служат для описания декомпозиции машины на ее свойства, структурированные слой за слоем, т. е. виртуальное отображение машины. Такие представления происходят из информационных и коммуникационных технологий, где свойства сложных систем обычно разбиваются на уровни.

В рамках этих трех осей могут быть сопоставлены все важнейшие аспекты Индустрии 4.0, что позволяет классифицировать такие объекты, как машины, в соответствии с моделью. Таким образом, очень гибкие концепции Индустрии 4.0 могут быть описаны и реализованы с использованием *RAMI* 4.0.

Эталонная архитектурная модель позволяет поэтапно переходить из настоящего в мир Индустрии 4.0. Модель объединяет различные точки зрения пользователей и обеспечивает общее понимание

технологий Индустрии 4.0. С помощью *RAMI* 4.0 требования секторов – от автоматизации производства и машиностроения до технологического проектирования могут быть рассмотрены в отраслевых ассоциациях и комитетах по стандартизации. Таким образом, *RAMI* 4.0 обеспечивает общее понимание стандартов и вариантов использования. *RAMI* 4.0 можно рассматривать как своего рода дорожную карту по переходу к Индустрии 4.0.

Отличительной чертой человеческого общества является то, что в течение длительного времени основным предметом труда оставались материальные объекты. Воздействуя на них, человек добывал себе средства к существованию, и на протяжении многих веков решалась задача усиления мускульных возможностей человека с помощью различных инструментов, агрегатов и машин. На это была направлена механизация производства, которая стала интенсивно внедряться в начале XX в. Развитие человеческого общества практически на всех этапах проходило на основе технического прогресса — это и овладение огнем, и использование паровых машин, и проникновение в тайны атомной энергии и т. п. Повышению производительности труда способствовала автоматизация. В процессе формирования трудовых коллективов возникла необходимость обмена знаниями. Первоначально знания передавались устно из поколения в поколение; появление письменности позволило по-новому показать накопленные знания — представить их в виде информации. Предполагают, что между первыми инструментами обработки материальных объектов и средствами отображения информационных образов существовал временной интервал около миллиона лет. Таким образом, появление информации является естественным следствием развития человеческого общества.

В настоящее время информация является одним из самых дорогих видов ресурсов. Это проявляется в тенденции стремительного перекачивания трудовых ресурсов из сферы материального производства в информационную. Например, в США в конце XIX в. свыше 95 % трудоспособного населения было занято физическим трудом, только менее 5 % — работой по обработке информации. Сегодня мы наблюдаем картину соотношения трудовых ресурсов с точностью до наоборот. В 1940-х гг. экстенсивный фактор как средство преодоления разрыва между потребностями и возможностями обработки информации себя исчерпал. Это явилось толчком к созданию новых средств обработки информации — ЭВМ — и переходу к интенсивному развитию информационной индустрии и, как следствие, к становлению научного направления — *теории информации*.

Теория информации быстро и прочно внедряется в самые различные области человеческого знания: физику, химию, биологию, медицину, философию, лингвистику, педагогику, экономику, логику, технические науки, эстетику и др.

ями, входящими в состав какого-либо формализованного языка, и их интерпретациями (истолкованиями) в терминах той системы понятий и представлений, формализацией которого служит данный язык. В более узком смысле под семантикой подразумевают совокупность правил соответствия между формальными выражениями и их интерпретацией. Под *семиотикой* понимают комплекс научных теорий, изучающих свойства знаковых систем, т. е. систем конкретных или абстрактных объектов, с каждым из которых сопоставлено некоторое значение. Примерами знаковых систем являются естественные языки, а также искусственные языки, такие, как алгоритмические языки, языки программирования, информационные языки и др.

Семантическая мера информации. Для измерения смыслового содержания информации, т. е. ее количества на семантическом уровне, наибольшее признание получила *тезаурусная мера* (предложена Ю. И. Шрейдером), которая связывает семантические свойства информации со способностью пользователя принимать поступившее сообщение. Для этого используется понятие тезаурус пользователя [4].

Тезаурус — это совокупность сведений, которыми располагает пользователь или система.

В зависимости от соотношений между смысловым содержанием информации S и тезаурусом пользователя S_p изменяется количество семантической информации I_c, воспринимаемой пользователем и включаемой им в дальнейшем в свой тезаурус. Характер такой зависимости показан на рис. 1.7. Рассмотрим два предельных случая, когда количество семантической информации I_c равно 0:

- при $S_p \to 0$ пользователь не воспринимает, не понимает поступающую информацию;
- при $S_p \to 1$ пользователь все знает, и поступающая информация ему не нужна.

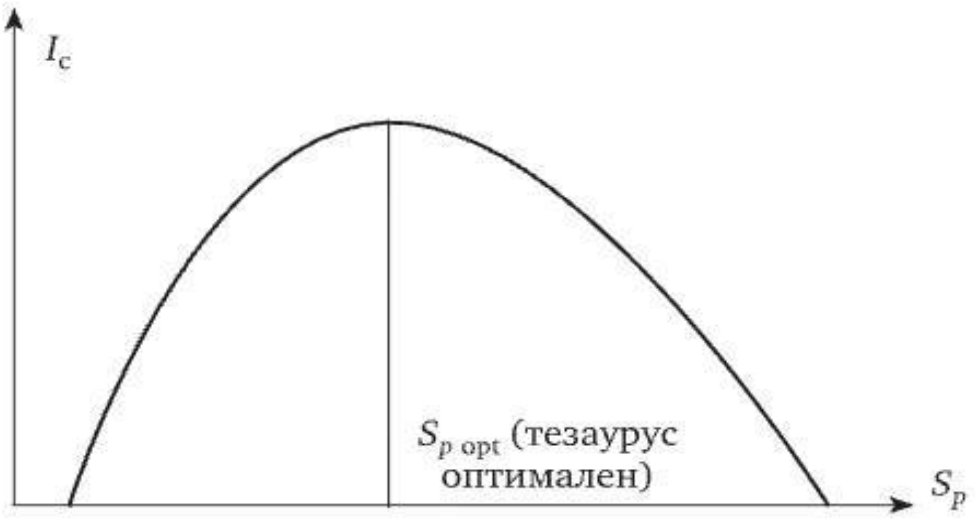


Рис. 1.7. **Зависимость количества семантической информации, воспринимаемой потребителем, от его тезауруса**

Максимальное количество семантической информации I_c потребитель приобретает при согласовании ее смыслового содержания S со своим тезаурусом S_p ($S_p = S_{p\,opt}$), когда поступающая информация понятна пользователю и несет ему ранее не известные (отсутствующие в его тезаурусе) сведения. Следовательно, количество семантической информации в сообщении, количество новых знаний, получаемых пользователем, является величиной относительной. Одно и то же сообщение может иметь смысловое содержание для компетентного пользователя и быть бессмысленным для пользователя некомпетентного. Относительной мерой количества семантической информации может служить коэффициент содержательности С, рассмотренный выше.

Прагматический (аксиологический) подход к информации базируется на анализе ее ценности, с точки зрения потребителя. Например, информация, имеющая несомненную ценность для биолога, будет иметь ценность, близкую к нулевой, для программиста. Ценность информации связывают со временем, поскольку с течением времени она стареет и ценность ее, а следовательно, и «количество» уменьшаются. Таким образом, прагматический подход оценивает содержательный аспект информации. Он имеет особое значение при использовании информации для управления, поскольку ее количество тесно связано с эффективностью управления в системе.

Прагматическая мера информации определяет полезность информации (ценность) для достижения пользователем поставленной цели. Эта мера — также величина относительная, обусловленная особенностями использования этой информации в той или иной системе.

Ценность информации целесообразно измерять в тех же самых единицах (или близких к ним), в которых измеряется целевая функция.

Алгоритмический подход связан с желанием внедрения универсальной меры информации. Количественная характеристика, отражающая сложность (размер) программы и позволяющая произвести какое-либо сообщение, была предложена А. Н. Колмогоровым.

Так как существуют разные способы задания и реализации алгоритма с использованием различных вычислительных машин и языков программирования, то для определенности задается некоторая конкретная машина, например *машина Тьюринга*. В этом случае в качестве количественной характеристики сообщения можно взять минимальное число внутренних состояний машины, требующихся для воспроизведения данного сообщения.

Разные подходы к оценке количества информации заставляют, с одной стороны, использовать разнотипные единицы информации для характеристики различных информационных процессов, а с другой — увязывать эти единицы между собой как на логическом, так и на физическом уровнях. Например, процесс передачи

информации, измеряемой в одних единицах, сопрягается с процессом хранения информации, где она измеряется в других единицах, и т. д., а поэтому выбор единицы информации является весьма актуальной задачей.

В табл. 1.3 сопоставлены введенные меры информации.

Таблица 1.3

Сопоставление мер информации

Мера информации	Единицы измерения	Примеры (для компьютерной области)
Синтаксическая: — объемный подход; — вероятностный, шенноновский подход	Степень уменьшения неопределенности. Единицы представления информации	Вероятность события (бит, байт. Кбайт и т. д.)
Семантическая	Тезаурус. Экономические показатели	Пакет прикладных программ, персональный компьютер, компьютерные сети и т. д.
		Рентабельность, производительность, коэффициент амортизации и т. д.
Прагматическая	Ценность использования	Емкость памяти, производительность компьютера, скорость передачи данных и т. д. Денежное выражение. Время обработки информации и принятия решений
Алгоритмическая	Сложность (размер) программы	Минимальное число внутренних состояний ЭВМ

1.4. Превращение информации в ресурс

Ресурсное обеспечение любого вида деятельности составляют финансы, материальные ресурсы, штаты и информационные ресурсы.

Если первые три вида ресурсов можно рассматривать обособленно, то информационные ресурсы тесно взаимосвязаны с каждым из них и по уровню иерархии стоят выше, так как используются при управлении остальными.

Информацию как вид ресурса можно создавать, передавать, искать, принимать, копировать (в той или иной форме), обрабатывать, разрушать. Информационные образы могут создаваться в самых разнообразных формах: в форме световых, звуковых, или радиоволн, электрического тока или напряжения, магнитных по-

лей, знаков на бумажных носителях. Важность информации как экономической категории составляет одну из главнейших характеристик «постиндустриальной» эпохи.

Информационный ресурс — концентрация имеющихся фактов, документов, данных и знаний, отражающих реальное изменяющееся во времени состояние общества, используемых при подготовке кадров, в научных исследованиях и материальном производстве [5].

Факты — результат наблюдения за состоянием предметной области.

Документы — часть информации, определенным образом структурированная и занесенная на бумажный носитель.

Данные — вид информации, отличающийся высокой степенью форматированности, в отличие от более свободных структур, характерных для речевой, текстовой и визуальной информации.

Знания — итог теоретической и практической деятельности человека, отражающий накопление предыдущего опыта и отличающийся высокой степенью структуризации.

В знаниях можно выделить три основные составные части:

- *декларативные* (факторальные), представляющие общее описание объекта, что не позволяет их использовать без предварительной структуризации в конкретной предметной области;
- *понятийные* (системные), содержащие помимо первой части взаимосвязи между понятиями и свойства понятий;
- *процедурные* (алгоритмические), позволяющие получить алгоритм решения.

Долгие годы информационный ресурс рассматривался как вспомогательный в развитии финансовых и материальных ресурсов. Однако в последние десятилетия все более явной становится тенденция к превращению информационного ресурса в главный фактор экономического развития. Это проявляется в следующих направлениях:

- информационный ресурс становится самостоятельным фактором экономики, выступая как товар, включая и информационные услуги;
- информационный ресурс становится основным компонентом материальных ресурсов (доля затрат на информацию в себестоимости продукции сравнивается или превышает остальные статьи затрат);
- возрастание роли информационных ресурсов в средствах труда, что проявляется в автоматизации процессов материального производства и концентрации трудовых усилий в сфере интеллектуального производства.

Таким образом, четко прослеживается тенденция в сторону интеллектуального информационного производства. Так, один из ведущих производителей в сфере инструментальных средств вычисли-

тельной техники корпорация *IBM* в последние годы сделала резкий крен в сторону когнитивных наук и направленности в сторону создания средств аналитики больших данных. В рамках этой инициативы с бюджетом в 17 млн евро, финансируемой Евросоюзом и названной *RESERVOIR — Resources and Services Virtualization without Barriers* (виртуализация ресурсов и сервисов без ограничений), будут изучены возможности разработки и управления ИТ-сервисами в масштабах административных доменов (групп серверов, маршрутизаторов и сетей, управляемых одной организацией), информационно-технологических платформ и географических регионов. Цель этого проекта, охватывающего сферу «*cloud computing* (облачные решения)» как новой парадигмы вычислений, — разработка технологий для поддержки онлайновой экономики, основанной на сервисах, где ресурсы и сервисы инициализируются и управляются в непрерывном и «прозрачном» для конечных пользователей режиме. Эксперты любой компании могут иметь доступ для анализа ситуации, после этого специалисты предоставляют свои рекомендации о путях более эффективного использования ресурсов и возможностей, показывают новые направления [6].

Функционирование информационного производства не возможно без исходного сырья — информации и накопленных знаний. Как вид ресурса информация обладает рядом специфических особенностей, отличающих ее от традиционных факторов производства — природных, трудовых и финансовых ресурсов. Наиболее значимыми свойствами информации являются:

- неисчерпаемость, обеспечивающая возможность их многоразового и многоцелевого использования, неотчуждение при обмене или продаже;
- самовозрастание (постоянный рост объема) в процессе информационного производства;
- неопределенность оценки ее полезности при первоначальном использовании информации;
- отсутствие зависимости между исходным объемом данных и знаний и характеристиками нового информационного продукта;
- высокая мобильность в пространстве и во времени без потери актуальности;
- многоцелевое использование в различных предметных областях;
- неразрывность информационного пространства, заключающаяся в стирании границ (политических, национальных, ведомственных и других) в распространении информации и знаний;
- изменчивость состава вследствие изменений информационных потребностей общества и развития информационного рынка;
- сложность выделения активной и пассивной части информационных ресурсов из-за различных темпов старения информации;

• отсутствие прямой зависимости между временем создания информации и ее полезностью (ценностью).

Повышение значимости и повсеместное распространение в качестве экономических ресурсов информации и знаний ведут не только к разнообразным положительным эффектам, частности, к экономии ресурсов, снижению нагрузки на окружающую среду, расширению возможностей людей. Существуют и различные проблемы, свойственные экономике, в которой информация и знания становятся важными ресурсами. Так, ускорение темпов научно-технических прогрессов приводит к усилению давления на общество, поскольку социальные, равно как и экономические институты, не успевают адаптироваться к изменениям. Информационная нагрузка на людей может оказывать деструктивное влияние на них, тем более что происходит все более жесткое и целенаправленное использование методов информационного воздействия.

1.5. Определение и задачи информационной технологии

Теоретической базой для информационных технологий является информатика.

Целью информатики является изучение структуры и общих свойств научной информации с выявлением закономерностей процессов коммуникации.

В современном понимании **информатика** — это область науки и техники, изучающая информационные процессы и методы их автоматизации. Пользователю она предоставляет методологические основы построения информационной модели объекта. Примером такой модели является концептуальная модель, которая отражает реальное содержание конкретной предметной области.

В информатике можно выделить три уровня. Физический (нижний) уровень представляет собой средства вычислительной техники и техники связи. Их развитие оказывает решающее влияние на возможности и направление использования информатики. Логический (средний) уровень составляют информационные технологии. Прикладной (верхний) уровень определяет идеологию применения информационных технологий для проектирования различных систем.

Информатика как научное направление имеет ряд определений. Это объясняется тем, что основным объектом изучения информатики является информация, точного определения которой нет до настоящего времени.

Теория информации, кибернетика и синергетика внесли значительный вклад в развитие информатики, однако не в состоянии

описать и дать научное объяснение разнообразным информационным процессам, имеющим место в природе и обществе. Новое научное направление — *инфодинамика* — связывает воедино массу, энергию и негэнтропию.

Поскольку однозначного понимания научного направления «информатика» не существует, целесообразно говорить не об истории, а об ее задачах на современном этапе. Так как информация является отражением, то в информационном обществе мы имеем дело с приближенными моделями реального мира. В связи с этим главной задачей информатики должно быть методологическое обоснование построения информационной модели объекта, явления, процесса. Использование этой модели для целенаправленной деятельности в любых сферах человеческого общества осуществляется на основе реализации информационных процессов и соответствующих им технологий.

Таким образом, для современного состояния информационных технологий необходим переход от информационного описания предметной области к представлению на уровне данных, осуществляемый на основе декомпозиции, абстракции, агрегирования.

Декомпозиция — это разбиение системы (программы, задачи) на компоненты, объединение которых позволяет решить данную задачу.

Абстракция позволяет правильно выбрать нужные компоненты для декомпозиции и представляет собой эффективный способ декомпозиции, осуществляемый посредством изменения списка декомпозиции.

Процесс абстракции может быть рассмотрен как некоторое обобщение. Он позволяет забыть о различиях и рассматривать предметы и явления так, как если бы они были эквивалентны. Так как выделение общего у процессов и явлений есть основа классификации, то иерархия абстракций представляет собой фактически схему классификации.

Выделяют ***два основных способа абстрагирования***:

- *абстракция через параметризацию* — выделение формальных параметров с возможностью их замены на фактические в различных контекстах. Выделение формальных параметров позволяет абстрагироваться от конкретного приложения и базируется на общности определенных свойств конкретных приложений;
- *абстракция через спецификацию* — позволяет абстрагироваться от внутренней структуры до уровня знания свойств внешних проявлений (результата). Внешние свойства компонента указываются путем описания внешних связей, требований и эффектов.

Внешние связи — это связи различной природы данного компонента с окружением.

Требования (*requires*) — это условия, которые должны быть выполнены для правильного использования компонента.

Эффекты (*effects*) — это условия, которым удовлетворяют внешние проявления (результаты) компонента.

С точки зрения конкретных приложений выделяют следующие ***виды абстракций***:

- *процедурная абстракция* (*ПА*) — позволяет расширить некоторую виртуальную машину новой операцией;
- *абстракция данных* (*АД*) — состоит из набора объектов и набора операций, характеризующих поведение этих объектов;
- *абстракция через итерацию* (*АИ*) — дает возможность не рассматривать информацию, не имеющую прямого отношения к управляющему потоку или циклу.

При построении модели данных предметной области наряду с естественным процессом декомпозиции используется и агрегирование. Это связано с необходимостью интеграции информационных ресурсов в силу их разнородности для ряда предметных областей.

Агрегирование — процесс объединения предметов в некоторую группу как в целях классификации, так и для обеспечения взаимодействия компонентов информационной системы.

В настоящее время при проектировании информационных систем используется два подхода: функционально-структурный и объектно-ориентированный.

- *Функционально-структурный подход* (*структурный*) использует принцип алгоритмической декомпозиции с выделением функциональных элементов предметной области и установлением строгого порядка выполняемых действий. Недостатком данного способа является неизбежность продвижения информации в одну сторону («вниз по течению»), что в случае ошибки при проектировании приводит к деформированию системы.

По мере постепенного превращения компьютеров из счетных устройств в универсальные машины для обработки данных, примерно после 1970 г., стали появляться новые термины: данные как продукты (*data product*); инструменты для работы с данными (*data tool*); приложения, реализуемые посредством соответствующей организации (*data application*); наука о данных (*data science*); ученые, работающие с данными (*data scientist*), и даже журналисты, которые доносят сведения, содержащиеся в данных, до широкой публики (*data journalist*).

- *Объектно-ориентированный подход* основан на декомпозиции объектов предметной области, при этом статическая структура системы описывается в терминах объектов и связей между ними. В объектно-

ориентированном подходе выделяют классы объектов. Каждый класс содержит однородные объекты. Иерархическая декомпозиция системы представляется в виде иерархии классов объектов, а функционирование системы — в виде взаимодействия объектов, обменивающихся сообщениями. Объектно-ориентированный подход позволяет осуществить параллельность и автономность разработки отдельных компонент ИС.

Технология (от др.-греч. τέχνη — искусство, мастерство, умение; λόγος — мысль, причина; методика, способ производства) — в широком смысле — совокупность методов, процессов и материалов, используемых в какой-либо отрасли деятельности, а также научное описание способов технического производства; в узком — комплекс организационных мер, операций и приемов, направленных на изготовление, обслуживание, ремонт и (или) эксплуатацию изделия с номинальным качеством и оптимальными затратами и обусловленных текущим уровнем развития науки, техники и общества в целом.

При этом:

- под термином «*изделие*» следует понимать любой конечный продукт труда (материальный, интеллектуальный, моральный, политический и т. п.);
- под термином «*номинальное качество*» следует понимать качество, прогнозируемое или заранее заданное, например, оговоренное техническим заданием и согласованное техническим предложением.

Современные технологии основаны на достижениях научно-технического прогресса и ориентированы на производство продукта: материальная технология создает материальный продукт, информационная технология (далее — ИТ) — информационный продукт. Технология — это также научная дисциплина, разрабатывающая и совершенствующая способы и инструменты производства. Технологией, или *технологическим процессом*, часто называют также сами операции добычи, транспортировки и переработки, которые являются основой производственного процесса.

Таким образом, термин «технология» имеет множество толкований. В широком смысле под технологией понимают науку о законах производства материальных благ, вкладывая в нее три основные части: идеологию, т. е. принципы производства; орудия труда, т. е. станки, машины, агрегаты; кадры, владеющие профессиональными навыками. Эти составляющие называют соответственно: *информационной, инструментальной* и *социальной*. Для конкретного производства технологию понимают в узком смысле как совокупность приемов и методов, определяющих последовательность действий для реализации производственного процесса. *Уровень технологий* связан с научно-техническим прогрессом общества и влияет на его социальную структуру, культуру и идеологию.

Для любой технологии могут быть выделены цель, предмет и средства. *Целью технологии* в промышленном производстве является повышение качества продукции, сокращение сроков ее изготовления и снижение себестоимости.

Методология любой технологии включает в себя: декомпозицию производственного процесса на отдельные взаимосвязанные и подчиненные составляющие (стадии, этапы, фазы, операции); реализацию определенной последовательности выполнения операций, фаз, этапов и стадий производственного процесса в соответствии с целью технологии; технологическую документацию, формализующую выполнение всех составляющих.

Производство информации направлено на целесообразное использование информационных ресурсов и снабжение ими всех элементов организационной структуры и реализуется путем создания информационной системы. Информационные ресурсы являются исходным «сырьем» для системы управления любой организационной структуры, конечным продуктом является принятое решение. Принятие решения в большинстве случаев осуществляется в условиях недостатка информации, поэтому степень использования информационных ресурсов во многом определяет эффективность работы организации.

В своем становлении любая отрасль, в том числе и информационная, проходила стадии от кустарного ремесленного производства к производству, основанному на высоких технологиях.

Информационные технологии обеспечивают переход от рутинных к промышленным методам и средствам работы с информацией в различных сферах человеческой деятельности, обеспечивая рациональное и эффективное ее использование.

В развитии технологии выделяют два принципиально разных этапа: один этап характеризуется непрерывным совершенствованием установившейся базисной технологии и достижением верхнего предельного уровня, когда дальнейшее улучшение является неоправданным из-за больших экономических вложений; другой этап отличается отказом от существующей технологии и переходом к принципиально другой, развивающейся по законам первого этапа.

Информационная технология — совокупность методов и способов получения, обработки, представления информации, направленных на изменение ее состояния, свойств, формы, содержания и осуществляемых в интересах пользователей.

Можно выделить три уровня рассмотрения информационных технологий:

1-й уровень — ***теоретический***. Основная задача — создание комплекса взаимосвязанных моделей информационных процессов, совместимых параметрически и критериально.

2-й уровень — ***исследовательский***. Основная задача — разработка методов, позволяющих автоматизированно конструировать оптимальные конкретные информационные технологии.

3-й уровень — ***прикладной***, который целесообразно разделить на две страты: инструментальную и предметную:

- *инструментальная страта* (аналог — оборудование, станки, инструмент) определяет пути и средства реализации информационных технологий, которые можно разделить на методические, информационные, математические, алгоритмические, технические и программные;
- *предметная страта* связана со спецификой конкретной предметной области и находит отражение в специализированных информационных технологиях, например, организационное управление, управление технологическими процессами, автоматизированное проектирование, обучение и др.

Основным критерием оценки информационных технологий является их эффективность, особенно экономическая эффективность. Традиционный расчет прибыли производится с учетом исчисляемых расходов и доходов.

Зачастую степень эффективности определяют исходя из того, насколько выгодны решения, принимаемые с точки зрения функционирования систем и устройств, государства, права и бизнеса, образования, культуры и т. д. При этом не самым главным критерием является расчет финансовых вложений. Так, например, если информационные службы высокоэффективны, то не только бизнес, но культура и образование выигрывают от их деятельности. В этом случае весьма затруднительно обосновать необходимые капиталовложения в информационные технологии. Здесь не работают расчеты, сделанные только с учетом экономической эффективности, а обычно учитывается, каких результатов можно достигнуть при создании новой системы или модернизации существующей. Успех применения информационной технологии может определяться эффективностью решения основных задач. Некоторые специалисты считают, что обоснование полезности — это искусство маркетинга. Для этого предлагается использовать разработанный Робертом Бенсоном метод «информационной экономики». Такой метод рассматривается как надежный способ анализа экономической эффективности, позволяющий учитывать качественные выгоды, величина которых определяется методом финансового прогноза с учетом возможных рисков.

Успешное внедрение информационных технологий связано с возможностью их типизации. Конкретная информационная технология обладает комплексным составом компонентов, поэтому целесообразно определить структуру и состав информационной технологии (рис. 1.8).

Технологический процесс — часть информационного процесса, содержащая действия (физические, механические и др.) по изменению состояния информации.

Рис. 1.8. **Состав информационной технологии**

Информационные технологии базируются на информации о предметной области, информационных процессах и являются основой реализации процессов информатизации и построения информационных систем на основе инструментального обеспечения (*hardware + software*), математического обеспечения (*braineware*) и методического обеспечения (*orgware*).

Конкретная информационная технология определяется в результате компиляции и синтеза базовых технологических операций, специализированных технологий и средств реализации.

Информационная технология базируется на реализации информационных процессов, разнообразие которых требует выделения базовых, характерных для любой информационной технологии.

Базовый технологический процесс основан на использовании стандартных моделей и инструментальных средств и может быть использован в качестве составной части информационной технологии. К их числу можно отнести: операции извлечения, транспортировки, хранения, обработки и представления информации.

Среди базовых технологических процессов выделим:

- извлечение информации;
- транспортирование информации;
- обработки информации;
- хранение информации;
- представление и использование информации.

Процесс извлечения информации связан с переходом от реального представления предметной области к его описанию в формальном виде и данных, которые отражают это представление.

Процесс транспортировки осуществляет передачу информации на расстояние для ускоренного обмена и организации быстрого доступа к ней, используя при этом различные способы преобразования.

Процесс обработки информации состоит в получении одних «информационных объектов» из других «информационных объектов» путем выполнения некоторых алгоритмов; является одной из основных операций, выполняемых над информацией и главным средством увеличения ее объема и разнообразия.

Процесс хранения связан с необходимостью накопления и долговременного хранения данных с обеспечением их актуальности, целостности, безопасности, доступности.

Процесс представления и ***использования информации*** направлен на решение задачи доступа к информации в удобной для пользователя форме.

Базовые технологические процессы строятся на основе базовых технологических операций, но кроме этого включают ряд специфических моделей и инструментальных средств. Этот вид технологий ориентирован на решение определенного класса задач и используется в конкретных технологиях в виде отдельной компоненты. Среди них можно выделить:

- облачные технологии;
- мультимедиатехнологии;
- геоинформационные технологии;
- технологии защиты информации;
- *CASE*-технологии;
- телекоммуникационные технологии;
- технологии искусственного интеллекта.

Специфика конкретной предметной области находит отражение в специализированных прикладных информационных технологиях, например, организационном управлении, управлении технологическими процессами, автоматизированном проектировании, обучении и др. Среди них наиболее продвинутыми являются следующие информационные технологии:

- организационного управления (корпоративные информационные технологии);
- в промышленности и экономике;
- образовании;
- автоматизированного проектирования.

Аналогом инструментальной базы (оборудование, станки, инструмент) являются *средства реализации информационных технологий*, которые можно разделить на методические, информационные, математические, алгоритмические, технические и программные.

***CASE*-технология** (*Computer Aided Software Engineering* — компьютерное автоматизированное проектирование программного обеспечения) является своеобразной «технологической оснасткой», позволяющей осуществить автоматизированное проектирование информационных технологий.

Методические средства определяют требования при разработке, внедрении и эксплуатации информационных технологий, обеспечивая информационную, программную и техническую совместимость. Наиболее важными из них являются требования по стандартизации.

Информационные средства обеспечивают эффективное представление предметной области, к их числу относятся *информационные модели*, системы классификации и кодирования информации (общероссийские, отраслевые) и др.

Математические средства включают в себя модели решения функциональных задач и модели организации информационных процессов, обеспечивающие эффективное принятие решения. Математические средства автоматически переходят в алгоритмические, обеспечивающие их реализацию.

Технические и ***программные средства*** задают уровень реализации информационных технологий, как при их создании, так и при их реализации.

Таким образом, конкретная информационная технология определяется в результате компиляции и синтеза базовых технологических операций, «отраслевых технологий» и средств реализации.

Эволюция информационных технологий наиболее ярко прослеживается на процессах хранения, транспортирования и обработки информации.

Отправной точкой для информационных технологий является 1900 г. До этого времени господствовала ручная обработка информации, для которой характерны:

- последовательная смена носителей информации (глиняные таблички, берестяные грамоты, папирус, пергамент, бумага);
- эволюция способов записи информации (рукопись, машинопись, печать), способов представления хранения (сочинения, книги, бумажные и печатные издания) и форм хранения (архивы, библиотеки).

Выделим шесть основных временных поколений [7] становления информационных технологий (рис 1.9), характеризуемых типом носителя, способами получения, передачи, хранения и обработки информации.

Первое поколение (1900—1955) связано с двоичным представлением информации. Основной носитель информации — перфокарта. Для записи данных на карты, обработки (сортировки и составления таблиц) использовалось электромеханическое оборудование. Дальнейшему развитию препятствовали серьезные недостатки:

- громоздкость оборудования;
- проблемы хранения (1 гигабайт информации, представленной в виде перфокарт весит 22 т);
- невозможность оперативной обработки.

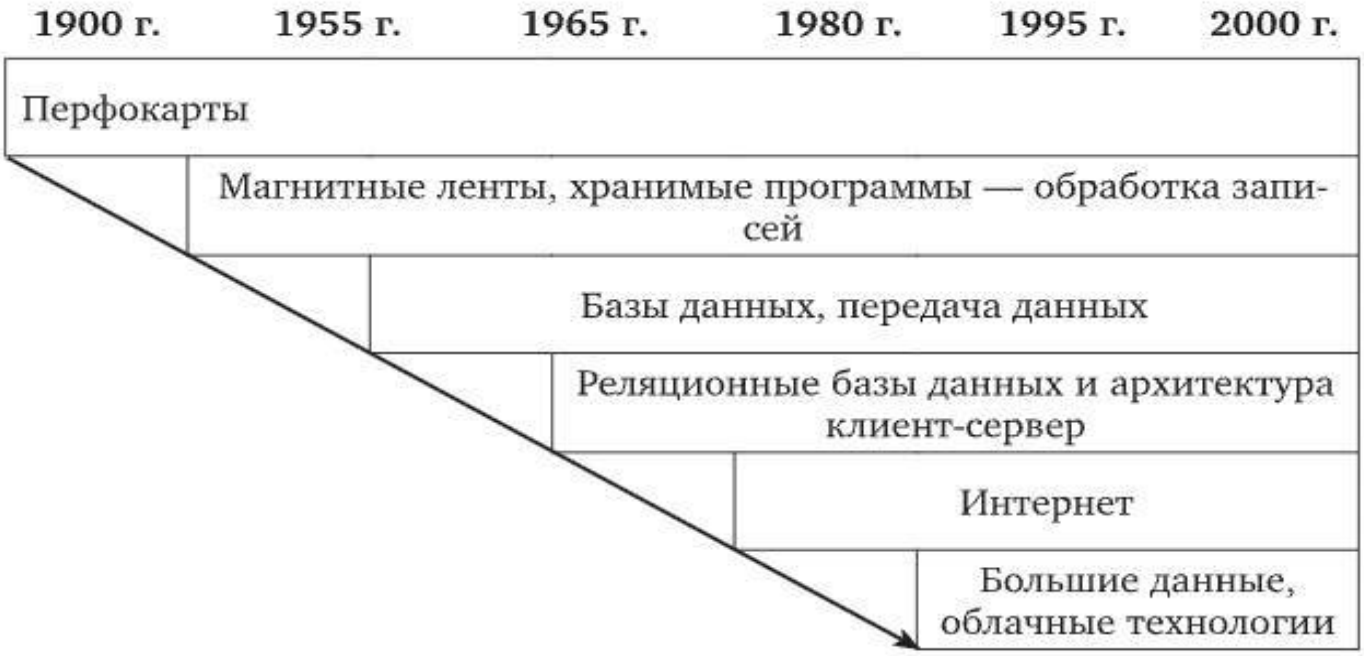


Рис. 1.9. **Временны́е фазы развития управления данными**

Второе поколение (1955—1980) сопряжено со следующими факторами:

- появлением в качестве носителя магнитных лент, позволяющих хранить большие объемы информации (одна магнитная лента заменяла более десяти тысяч перфокарт);
- разработкой ЭВМ с программной поддержкой, обеспечивающей более высокое быстродействие (обработка сотни записей в секунду);

Основное внимание в этот период времени было уделено программному обеспечению, направленному на удобство и эффективность программирования. Наиболее характерным для программного обеспечения этого времени являлось следующее:

- обработка записей на основе файловой организации данных;
- разработка типовых программ для последовательного чтения нескольких входных файлов и производства на выходе новых файлов;
- создание языков программирования (например, *COBOL*), ориентированных на записи последовательных задач;
- в рамках операционных систем разработка языка управления заданиями и планировщика заданий для управления потоком работ;
- внедрение пакетной обработки заданий.

Отметим недостатки, сдерживающие дальнейшее развитие информационных технологий:

- отсутствие оперативного доступа к данным;
- недостаточное быстродействие;

- невозможность распознавания ошибки до обработки основного файла.

Третье поколение (1965—1980) базировалось на внедрении технологий оперативного доступа к данным в интерактивном режиме, основанном на использовании систем баз данных и систем передачи данных.

Определяющим на этом этапе явилось появление персональных компьютеров, что дало толчок реализации интерактивного обмена данными, обеспечивающего доступ к любому элементу данных за доли секунды.

Хотя оперативные базы данных хранились на магнитных дисках или барабанах, доступ к любому элементу данных обеспечивался за доли секунды. Внедрение технологий баз данных шло по пути от индексно-последовательной организации записей к иерархическим и сетевым моделям, от простого поиска данных к прямому поиску по номеру записи или ассоциативному поиску по ключу.

Сетевые (и иерархические) модели данных получили широкое развитие, что выразилось в создании международной организации сопровождения сетевых баз данных *CODASYL*.

Однако ряд недостатков препятствовал дальнейшему развитию данного направления:

- низкий уровень навигационного программного интерфейса;
- сложность проектирования баз данных;
- недостаточный уровень типизации и универсальности проектных решений.

Четвертое поколение (1980—1995) определяется бурным внедрением реляционных баз данных и архитектурой «клиент — сервер». Привлекательность реляционной модели заключается в использовании четкого математического аппарата реляционной алгебры, что позволило:

- осуществить единообразное представление предметной области;
- создать унифицированный язык описания данных и манипулирования данными.

Работы в области стандартизации и унификации породили язык, названный *SQL* (англ. *Structured Query Language* — язык структурированных запросов) и присутствующий во всех реляционных системах управления данными.

Реляционная модель обладает следующими преимуществами:

- высокая степень продуктивности и простоты использования;
- совместимость с архитектурой «клиент — сервер»;
- возможность использования параллельной обработки;
- применение графического пользовательского интерфейса.

Однако при значительных объемах данных начинают проявляться недостатки реляционных баз данных. К этим недостаткам относятся сложность структуры, вызванная необходимостью проведения нор-

мализации; низкая производительность из-за поиска по ключу, что в три — пять раз увеличивает количество операций доступа; ограниченный набор типов данных; представление данных только в виде двумерных таблиц и невозможность реализации таблиц с нелинейной структурой; невозможность послойного рассмотрения данных (например, работающие — в одном слое, научные сотрудники и преподаватели — в другом, подчиненном слое); нестыковка с принципами все более широко применяемого объектно-ориентированного подхода; невозможность задать для определенного типа данных набор операторов-методов, которые приходится вводить в конкретном приложении; возникновение конфузии — утраты при многочисленных обновлениях третьей (а порой — и второй) нормальной формы; сложность совмещения с другой парадигмой хранилищ данных.

Одним из способов устранения указанных недостатков является построение объектно-реляционных (ОРБД) и объектно-ориентированных (ООБД) баз данных.

В качестве примера можно упомянуть два крупных проекта, характеризующие современные возможности технологий баз данных [7]. Первый из них — система *Earth Observation System/Data Information System* (*EOS/DIS*), разработанная агентством *NASA* для хранения всех спутниковых данных, которые начали поступать со спутников серии «Миссия к планете Земля» в 1977 г. Объем базы данных составляет несколько десятков петабайт. Другим примером базы данных является во многом стихийно развивающаяся всемирная библиотека. Реализация этих проектов стала возможной благодаря внедрению мультимедийных баз данных, что связано с переходом от традиционных баз данных, хранящих числа и символы к базам со сложными данными (звук, изображение, текст и др.).

Пятое поколение (1995—2000) связано с широким распространением Интернета (англ. *Internet* — всемирная система объединенных компьютерных сетей для хранения и передачи информации).

Основной причиной быстрого развития Интернета стал бесплатный и открытый доступ к основным документам, особенно к спецификациям протоколов.

В 1995 г. было сформулировано окончательное понятие термина «Интернет».

Под Интернетом понимается глобальная информационная система, обладающая следующими признаками:

- логическая связанность с помощью глобального уникального адресного пространства на основе протокола Интернета (IP) или его последующих расширений/дополнений;
- поддержка передачи данных на основе стандартных протоколов *Transmission Control Protocol/Internet Protocol* (*TCP/IP*) или его последующих расширений/дополнений и/или других IP-совместимых протоколов;

- общая доступность на любом уровне услуг верхнего уровня, строящихся на основе связи и связанной инфраструктуры.

Очень часто Интернет рассматривается как развитие традиционной телефонии, хотя на самом деле является ветвью развития информационных технологий. Интернет оказался очень жизнеспособной конструкцией, легко адаптируемой ко всем новшествам информационных технологий. От первоначального использования компьютера в режиме разделения времени Интернет органично впитал в себя локальные сети, архитектуру «клиент —сервер», одноранговые сети и сетевые компьютеры.

Иногда термин «Интернет» путают с «Интранет» (англ. *Intranet* — интрасеть), который в отличие от первого является внутренней частной ведомственной сетью, построенной на использовании IP.

Интернет широко внедрился во все сферы нашей жизни. Ярким примером служит цифровая экономика, которая базируется на достижениях Интернета. Сам Интернет достиг коммерческого успеха с многомиллиардным объемом.

Шестое поколение (2000 г. — по настоящее время) связано с возникновением двух проблем на рубеже двух веков:

1. Что делать с огромными объемами накопленных данных?
2. Где найти ресурсы для решения непрерывно возрастающего количества задач?

Решение этих проблем стало возможным благодаря трем новым направлениям информационных технологий: большие данные, облачные вычисления и искусственный интеллект.

Большие данные (англ. *big data*) — совокупность математических методов, инструментальных средств и методов обработки больших объемов разнообразных (структурированных и неструктурированных) данных для получения информации, необходимой для принятия эффективных решений в различных областях деятельности.

Это направление является альтернативой сформировавшимся в конце 2000-х гг. традиционным базам данных и решениям класса *Business Intelligenc* (*BI*). Назначение *BI* — обработка данных для принятия решений в области бизнеса, включая следующие этапы:

1) информационный поиск;
2) аналитическая обработка в реальном времени (OLAP);
3) инструменты предупреждения об отклонениях от ожидаемых показателей;
4) бизнес-аналитика;
5) бизнес-отчетность.

Облачные вычисления (англ. *cloud computing*) — технология, обеспечивающая комфортный удаленный доступ по согласованному требованию с минимальными эксплуатационными затратами к общему фонду конфигурируемых инструментальных средств (сетям передачи данных, серверам, устройствам хранения данных, приложениям и сервисам).

Услуги, предоставляемые пользователям в *cloud computing*, принято называть словом *aaS* (*as a Service* — «как сервис» или «в виде сервиса»). Перечень услуг весьма разнообразен:

- *Storage-as-a-Service* («хранение как сервис»);
- *Database-as-a-Service* («база данных как сервис»);
- *Information-as-a-Service* («информация как сервис»);
- *Process-as-a-Service* («управление процессом как сервис»);
- *Platform-as-a-Service* («платформа как сервис»);
- *Integration-as-a-Service* («интеграция как сервис»);
- *Security-as-a-Service* («безопасность как сервис»);
- *Management/Governace-as-a-Service* («администрирование и управление как сервис»);
- *Infrastructure-as-a-Service* («инфраструктура как сервис»);
- *Testing-as-a-Service* («тестирование как сервис»).

Дальнейшее развитие информационных технологий привело к созданию некоторой информационной среды в киберпространстве, получившей название искусственный интеллект. Программное обеспечение для определения, поиска и визуализации оперативно доступной информации — ключ к созданию и доступу к такой информации. Основные задачи, которые в настоящее время решаются:

- определение моделей данных для новых типов (например, пространственных, темпоральных, графических) и их интеграция с традиционными системами баз данных;
- масштабирование баз данных по размеру (до петабайт), пространственному размещению (распределенные) и многообразию (неоднородные);
- автоматическое обнаружение тенденций данных, структур и аномалий (добывание данных, анализ данных);
- интеграция (комбинирование) данных из нескольких источников;
- создание сценариев и управление потоком работ (процессом) и данными в организациях;
- автоматизация проектирования и администрирования базами данных.

Само понятие ИИ появилось в середине 1950-х гг. прошлого века как раздел кибернетики и базировалось на предположении, что любое свойства естественного интеллекта, в частности, способность к обучению можно точно описать и смоделировать его на ЭВМ. При этом отмечалось, что при решении творческих задач должны быть использованы методы, которые не наблюдаются у человека, если это необходимо для решения конкретных проблем.

Такая трактовка понятия ИИ привела к тому, что до настоящего времени имеются разные точки зрения на ИИ. В качестве традиционных можно назвать биологическую и кибернетическую точки зрения (табл. 1.4).

Таблица 1.4

Искусственный интеллект в различных областях

Кибернетический кластер	Биология и психология
Общая теория систем и системный анализ	Нейрокомпьютеры и нейросети
Исследование операций	Когнитивная психология
Анализ данных и принятие решений	Нейробиология
Теория управления	Когнитивные исследования
Оптимизация	Нейробиология
Теория информации	Исследования мозга
Теория связи	Философия сознания
Другие науки	Биология и биоинспирированные алгоритмы

Следует заметить, что исследователи в области нейробиологии, психологии и философии сознания использовали термин «интеллект» как синоним термина «человеческий интеллект» задолго до появления кибернетики и информационных технологий.

В последние годы сформировалась еще одна точка зрения на ИИ, которую можно определить как бизнес-ориентированную или инженерную точки зрения, в соответствии с которой ИИ является средством построения ИИС для решения практических задач средствами программной инженерии (рис. 1.10).

Рис. 1.10. **Составные части ИИ**

ИИ является междисциплинарной наукой, в которой используется практически весь спектр кибернетики, математики, статистики, компьютерных наук, коммуникаций, психологии, физиологии, биологии, нейробиологии, философии сознания, информатики мозга, социальных наук и др.

С другой стороны, все эти науки в разной форме используют понятия и/или результаты ИИ, и, что важно, у этих наук и у ИИ много общих терминов, которые звучат одинаково. Однако содержание этих терминов, как правило, различно, причем наибольшие различия имеют место именно в трактовке смысла базового термина «интеллект». По этим причинам специалисты этих наук со своих позиций трактуют и суть ИИ как науки, причем необязательно в контексте информатики. В настоящее время эта неопределенность в трактовке имеет хождение и в среде специалистов, которые занимаются исследованиями и разработками ИИ и приложений в области информатики [2, 10, 11].

В бизнес-сообществе ИИ определяют как «способность цифрового компьютера или робота с компьютерным управлением выполнять задачи, обычно связанные с разумными существами». Системы ИИ практически с момента появления стали позиционировать как ИС, ориентированные на работу со знаниями. Исследования и разработки направлены на решение трех основных задач: получение знаний, представление и преобразование знаний и использование знаний для решения задач.

Тема 2
БАЗОВЫЕ ИНФОРМАЦИОННЫЕ ПРОЦЕССЫ, ИХ ХАРАКТЕРИСТИКА И МОДЕЛИ

2.1. Извлечение информации

Источниками информации могут являться данные, знания, документы. Источниками *данных* в любой предметной области являются объекты и их свойства, процессы и функции, выполняемые этими объектами или для них. Любая предметная область рассматривается в виде трех **представлений**: реального, формального и информационного.

Непосредственно в процессе извлечения информации можно выделить следующие **фазы** [8]:

- ***накопление*** — системное или бессистемное (стихийное) накопление информации в рамках предметной области;
- ***структурирование*** — выделение основных понятий, выработка структуры представления информации, обладающей максимальной наглядностью, простотой изменения и дополнения;
- ***формализация*** — представление структурированной информации в форматах машинной обработки, т. е. на языках описания данных и знаний;
- ***обслуживание*** — корректировка формализованных данных и знаний (добавление, обновление), удаление устаревшей информации, фильтрация данных и знаний для поиска информации, необходимой пользователям.

По аналогии с добычей полезных ископаемых процесс извлечения информации направлен на получение наибольшей ее концентрации. В связи с этим процесс извлечения можно представить как ее прохождение через трехслойный фильтр, в котором осуществляется оценка ***синтаксической*** (правильность представления), ***семантической*** (смысловой), ***прагматической*** (потребительской) ***ценностей***.

При извлечении информации важное место занимают различные **формы и методы исследования данных**:

- нахождение ассоциаций, связанных с привязкой к какому-либо событию;

- нахождение последовательностей событий во времени;
- нахождение скрытых закономерностей по наборам данных путем определения причинно-следственных связей между значениями определенных косвенных параметров исследуемого объекта (ситуации, процесса);
- оценка важности (влияния) параметров на события и ситуации;
- классифицирование (распознавание), осуществляемое путем поиска критериев, по которым можно было бы относить объект (события, ситуации, процессы) к той или иной классификационной категории;
- кластеризация, основанная на группировании объектов по каким-либо признакам;
- прогнозирование событий и ситуаций.

Следует упомянуть неоднородность (разнородность) информационных ресурсов, характерную для многих предметных областей. Одним из путей решения данной проблемы является объектно-ориентированный подход, наиболее распространенный в настоящее время. Кратко рассмотрим его основные положения.

Декомпозиция на основе объектно-ориентированного подхода основана на выделении следующих основных понятий: объект, класс, экземпляр.

Объект — это абстракция множества предметов реального мира, обладающих одинаковыми характеристиками и законами поведения. Объект характеризует собой типичный неопределенный элемент такого множества. Основной характеристикой объекта является состав его атрибутов (свойств).

Атрибуты — это специальные объекты, посредством которых можно задать правила описания свойств других объектов.

Экземпляр объекта — это конкретный определенный элемент множества. Например, объектом может являться государственный номер автомобиля, а экземпляром этого объекта — конкретный номер — К 173 ПА.

Класс — это множество предметов реального мира, связанных общностью структуры и поведением. **Элемент класса** — это конкретный элемент данного множества. Например, класс регистрационных номеров автомобиля.

Предмет — конкретный предмет реального мира.

Обобщая эти определения, можно сказать, что объект — это типичный представитель класса, а термины «экземпляр объекта» и «элемент класса» равнозначны. На рис. 2.1 показаны отношения между классами, объектами и предметами реального мира.

Важная особенность объектно-ориентированного подхода связана с понятием *инкапсуляции*, обозначающим сокрытие данных и методов (действий с объектом) в качестве собственных ресурсов объекта.

Понятия полиморфизма и наследования определяют эволюцию объектно-ориентированной системы, что подразумевает определение новых классов объектов на основе базовых классов.

Полиморфизм интерпретируется как способность объекта принадлежать более чем одному типу.

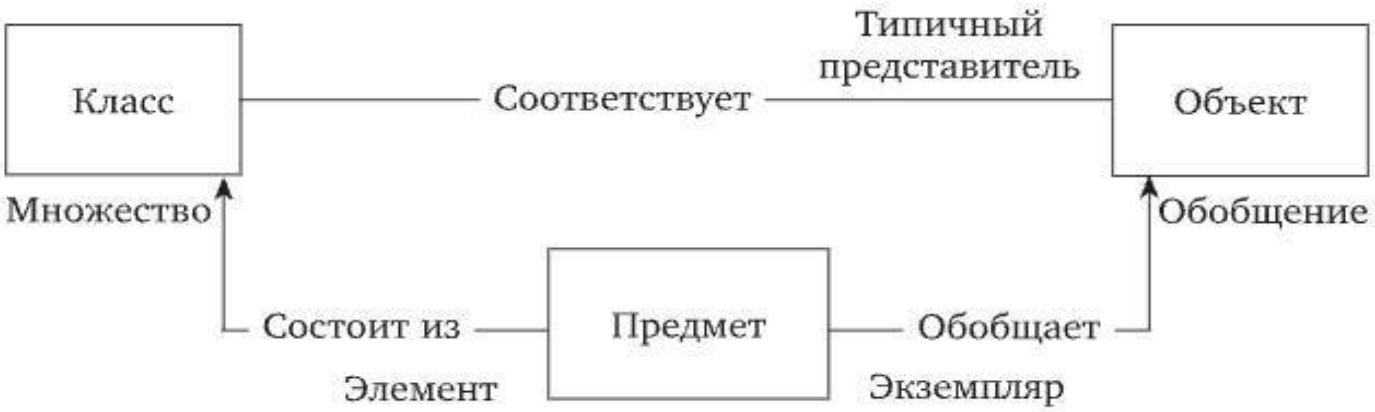


Рис. 2.1. **Отношения между классами, объектами и предметами реального мира**

Наследование выражает возможность определения новых классов на основе существующих с возможностью добавления или переопределения данных и методов.

Для уменьшения избыточности используется процесс обогащения информации, например, при хранении в компьютере списка сотрудников организации иногда достаточно использовать первые 3—4 буквы их фамилий.

Среди **методов обогащения информации** различают структурное, статистическое, семантическое и прагматическое обогащения.

Структурное обогащение предполагает изменение параметров сообщения, отображающего информацию в зависимости от частотного спектра исследуемого процесса, скорости обслуживания источников информации и требуемой точности.

При ***статистическом обогащении*** осуществляют накопление статистических данных, обработку выборок из генеральных совокупностей накопленных данных.

Семантическое обогащение означает минимизацию логической формы, исчислений и высказываний, выделение и классификацию понятий, содержания информации, переход от частных понятий к более общим. В итоге семантического обогащения удается обобщенно представить обрабатываемую либо передаваемую информацию и устранить логическую противоречивость в ней.

Прагматическое обогащение является важной ступенью при использовании информации для принятия решения, при котором из полученной информации отбирается наиболее ценная, отвечающая целям и задачам пользователя.

Развитие методов и средств извлечения информации направлено в сторону стандартизации и унификации. Характерным примером является создание и внедрение технологий *Data Mining* и *Text Mining*.

Data Mining (в буквальном переводе с английского — «добыча данных») — это направление в информационных технологиях, которое связано с автоматизированным извлечением знаний (неявным образом присутствующих в обрабатываемой информации) и базируется на интеллектуальном анализе данных.

В основе современной технологии *Data Mining* лежит концепция шаблонов, отражающих различные фрагменты взаимоотношений в данных. Важное свойство методов *Data Mining* — нетривиальность обнаруживаемых шаблонов, которые должны отражать неочевидные, ранее неизвестные регулярности в данных, составляющие так называемые скрытые знания (*hidden knowledge*), и изменения клиентуры, выявления мошенничества с кредитными карточками.

Методы *Data Mining* позволяют выделить следующие **типы закономерностей**:

- последовательность (например, после события А в течение определенного интервала времени с большой вероятностью следует событие Б);
- связь между событиями (например, события А и Б с большой вероятностью осуществляются одновременно);
- классификация (объекты относятся к одной из групп с относительно постоянными характеристиками);
- кластеризация отличается от классификации тем, что сами группы заранее не задаются и выделяются непосредственно в процессе анализа;
- прогноз — построение временных рядов, отражающих динамику поведения целевых показателей.

Text Mining является разновидностью *Data Mining*, ориентированной на обработку текстовой информации и широко применяемой для мониторинга ресурсов Интернета. Задача *Text Mining* — проанализировать не синтаксис, а семантику значения текстов, выбрать из него информацию, наиболее значимую для пользователя (есть тесная связь с контент-анализом). Обычно выделяют такие приложения *Text Mining*:

- реферирование текстов на естественном языке;
- классификацию (тематическое индексирование) текстовых документов;
- кластеризацию текстовых документов и их фрагментов;
- построение онтологии текстового документа (основных терминов и связей между ними), например, семантической сети;
- визуализация полученных знаний.

Среди основных поставщиков информации сети Интернет выделяют следующие: информационные агентства, Web-ресурсы СМИ,

сайты юридических и физических лиц, познавательные сайты (библиотеки, справочники, ссылки на другие сайты, каталоги и т. п.), сайты для обмена информацией (чаты, телеконференции, социальные сети и т. п.).

Непрерывный рост объема информации в сети Интернет приводит к необходимости решения двух взаимосвязанных задач: анализа и поиска нужной информации. Проблемы поиска информации в сети Интернет связаны со следующими факторами:

- недостаточно развитые стандарты представления информации (различные форматы представления, слабоструктурированность информации);
- разнообразие языков представления информации и необходимость учета языковых особенностей;
- быстрый рост объемов хранимой и передаваемой информации;
- высокие требования к скорости поиска информации;
- несовершенство навигационных методов.

Существует широкий спектр методов поиска информации в сети Интернет на основе информационно-поисковых систем (ИПС), которые, однако, можно разделить на два основных класса (рис. 2.2): поисковые машины и поисковые каталоги [8]. Рассмотрим каждый из этих классов в отдельности.

Поисковые машины в свою очередь подразделяют на две группы: серверные и поисковые машины.

Серверная поисковая машина является программно-аппаратным комплексом, предназначенным для одновременного обслуживания множества клиентов. Основные преимущества: малое время отклика и обработки запроса; высокая производительность.

Отличительной особенностью серверных поисковых машин является ориентация на использование современных методов поиска информации и высокие аппаратные характеристики сервера.

Клиентская поисковая машина по своей сути является интеллектуальным агентом, работающим на стороне клиента. Отличительными особенностями по сравнению с серверным вариантом являются узкая направленность и поиск информации в определенном секторе Интернета; более низкое быстродействие; гибкость в настройке. Адаптация к требованиям поиска особенно проявляется в использовании нейросетевых и других перспективных алгоритмов. Другим направлением повышением качества поиска является возможность отправления наиболее популярных ссылок клиента на основной поисковой сервер.

Поисковые машины обеспечивают автоматическую индексацию большого количества документов, но не обладают развитыми средствами искусственного интеллекта для экспертной оценки смыслового содержания информации. Этим обусловлена низкая

релевантность ответа поисковых систем (релевантность — степень адекватности результатов поиска запросу пользователя). Решение данной проблемы заключается в применении прогрессивных методов искусственного интеллекта для обработки и анализа текстовой информации.

Поисковые каталоги ресурсов представляют собой иерархически организованные наборы резюме содержания информационных ресурсов. Каталоги позволяют пользователю, двигаясь от общих понятий к более узким, найти ссылку на сайт с интересующей их информацией. Преимущество таких систем перед поисковыми машинами заключается в том, что база данных каталогов наполняется людьми, что приводит к высокой релевантности расположенных в них ссылок. Существуют каталоги двух типов: универсальные и тематические. Как показывает практика, хорошие тематические каталоги содержат больше информации по своей тематике, чем универсальные. Однако информационная полезность таких каталогов, как правило, ограничена небольшим количеством проиндексированных документов, большими затратами средств на поддержание актуальности базы проиндексированных документов и, следовательно, низкой оперативностью ее обновления.

Объем базы данных каталогов сравнительно невелик, скорость пополнения базы на порядок ниже, чем у автоматизированных поисковых машин. Также ниже и скорость поиска по каталогу.

Для повышения эффективности поиска информации используют гибридные структуры, объединяющие в одно целое поисковые машины и каталоги. За счет этого удается повысить скорость поиска поисковых машин и повысить точность (релевантность) каталогов.

Рис. 2.2. **Классификация информационно-поисковых систем**

Поисковые машины обеспечивают автоматическую индексацию большого количества документов, но не обладают развитыми средствами искусственного интеллекта для экспертной оценки смыслового содержания информации. Этим обусловлена низкая релевантность ответа поисковых систем (*релевантность* — степень

адекватности результатов поиска запросу пользователя). Решение данной проблемы заключается в применении прогрессивных методов искусственного интеллекта для обработки и анализа текстовой информации.

Поисковые каталоги ресурсов представляют собой иерархически организованные наборы резюме содержания информационных ресурсов. Каталоги позволяют пользователю, спускаясь от общих понятий к более узким, найти ссылку на сайт с интересующей их информацией. Преимущество таких систем перед поисковыми машинами заключается в том, что база данных каталогов наполняется людьми, что приводит к высокой релевантности расположенных в них ссылок. Существуют каталоги двух типов: *универсальные* и *тематические*. Как показывает практика, хорошие тематические каталоги содержат больше информации по своей тематике, чем универсальные. Однако информационная полезность таких каталогов, как правило, ограничена небольшим количеством проиндексированных документов, большими затратами средств на поддержание актуальности базы проиндексированных документов и, следовательно, низкой оперативностью ее обновления.

Объем базы данных каталогов сравнительно невелик, скорость пополнения базы на порядок ниже, чем у автоматизированных поисковых машин. Также ниже и скорость поиска по каталогу.

Одним из способов устранения вышеизложенных недостатков каталогов, а также поисковых машин является их объединение в общую структуру. Это позволяет придать такой гибридной структуре скорость поиска поисковых машин в сочетании с точностью (релевантностью) каталогов.

Рассмотрим процесс формирования информационных ресурсов и их представление в ИПС. Общеизвестно, что документальным массивом ИПС *Internet* является все множество документов шести основных типов: *WWW*-страницы, *Gopher*-файлы, документы *Wais*, записи архивов *FTP*, новости *Usenet*, статьи почтовых списков рассылки. Все это — довольно разнородная информация, которая представлена в виде различных, никак несогласованных друг с другом форматов данных. Здесь есть и текстовая информация, и графическая информация, и аудиоинформация и вообще все, что есть в указанных выше хранилищах. Естественно, встает вопрос, как информационно-поисковая система должна со всем этим работать.

Первоочередной задачей для информационно-поисковой системы является формирование идентификационного признака информационного ресурса на основе его ключевых слов. Этот процесс называется индексированием и его целью является сопоставление каждого информационного ресурса с некоторым множеством индексов (рубрик, индексационных терминов, ключевых слов, дескрипторов, полей и т. д.)

В простейшем случае к индексированию относят создание инвертированного списка, в котором каждому термину индексирования ставится в соответствие список документов, в которых он встречается.

Одним из наиболее важных факторов, влияющих на качество поиска, является **метод внутреннего представления** документов в поисковой машине. В традиционных системах есть понятие **поискового образа документа** (далее — ПОД), который его заменяет и используется при поиске вместо реального. Поисковый образ является результатом применения некоторой модели информационного массива документов к реальному массиву.

Обычно поиск информации в документах происходит путем сравнения терминов этих документов с терминами из запроса пользователя. В этих методах есть два существенных недостатка. Во-первых, обычно имеется много способов выражения данного понятия (с помощью синонимов), поэтому относящиеся к делу документы могут быть отвергнуты. Во-вторых, многие слова имеют множественное значение (полисемия), поэтому в результате работы программы могут быть получены ненужные документы. Эти два недостатка приводят к тому, что методы, основанные на сравнении терминов, оказываются неприемлемыми для поиска ответа на запрос пользователя. Более эффективный подход должен позволять пользователю получать информацию, учитывая смысл конкретного документа.

Наиболее распространенными из таких методов индексирования являются: лексическое, векторное, вероятностное, скрытое семантическое. Рассмотрим преимущества, недостатки и особенности применения каждого из методов.

Лексическое индексирование — реализовано как инвертированный файл, в котором каждому термину (ключевому слову) из словаря предметной области соответствует список документов, в которых этот термин встречается.

Запросы пользователей формируются на основе аппарата булевой алгебры и представляют собой логические выражения, в которых ключевые слова соединены операторами AND, NOT или ANDNOT.

Несмотря на хорошее распространение данный тип индексирования имеет существенные недостатки:

- аппарат булевой алгебры с трудом воспринимается рядовым пользователем;
- в отличие от экспертов пользователю трудно сформулировать запрос на основе ключевых слов, особенно в том случае, если одинаковые слова с разным смыслом;
- результат поиска является полностью бесконтекстным, что сильно понижает его релевантность.

Векторное индексирование базируется на соответствии каждому документу частотного спектра слов и вектора в лексическом про-

странстве. Частотная характеристика запроса представляется как вектор в том же пространстве, и по степени близости (расстоянию или углу между векторами) определяются наиболее релевантные документы.

Таким образом, каждому документу ставится в соответствие вектор с размерностью, равной числу терминов, которыми можно воспользоваться при поиске. В самом простом случае при булевой векторной модели элемент вектора равен 1 или 0 (отсутствие или наличие в ПОДе документа). В более сложных моделях терминам присваивается «вес», который отражает соответствие данного термина документу. Эта модель в силу своей релевантности наиболее популярна, в том числе в информационно-поисковых системах Интернета.

Вероятностное индексирование использует сопоставление каждого слова и его веса в документе. Это обеспечивает высокое качество поиска в сравнении с лексическим и векторным индексированием, но приводит к значительным издержкам при формировании ПОДа документа.

Семантическое индексирование состоит в установлении семантических отношений между составляющими высказываний в основном тексте, что позволяет представить содержание документа в виде совокупности упомянутых в нем фактов. В основе лежат **модели семантического анализа** (*MSA — Models of Semantic Analysis*), в которых решается задача сравнения терминов с использованием статистически полученных смысловых параметров вместо отдельных слов. Основные вычислительные преимущества *MSA* над другими методами заключаются в другом подходе к добавлению информации в базу данных. Основная идея в *MSA* заключается в том, чтобы явно смоделировать взаимосвязи между терминами (через двустороннее ортогональное разложение) и использовать его, чтобы улучшить возможности поисковой системы.

Скрытое семантическое индексирование. Математический аппарат данного метода базируется на экономном сингулярном разложении матриц, которое позволяет выявить скрытые семантические связи при обработке большого массива документов. Данный подход позволяет реализовать методологию автоматической классификации документа, когда проводится исследование слов во всей совокупности документов и производится подсчет одинаковых характеристик для каждого документа или термина.

Эффективность поиска в данном метода намного выше лексического или векторного индексирования, но его реализация требует увеличения вычислительных возможностей сервера.

Латентно-семантическое индексирование использует принципы факторного анализа, в частности выявление латентных связей изучаемых явлений при помощи статистической обработки

больших массивов текстов. Оно часто используется для выявления главных факторов из массива информационных данных. Разновидностью является вероятностное латентно-семантическое индексирование.

Развитие методов поиска информации сопровождалось попытками внедрения интеллектуального анализа данных, направленного на совершенствование поиска информации по смысловому содержанию запроса, а не по совпадению терминов. Наибольший прогресс в этом направлении достигнут при использовании моделей семантического анализа (*MSA*). Применение *MSA* основано на анализе структуры текста, которая в явном виде не прослеживается, на основе аппарата матричного исчисления. В результате грамматического анализа текста формируется матрица терминов и документов. Анализ осуществляется с использованием сингулярного разложения (англ. *singular value decomposition, SVD*), заключающегося в разложении прямоугольной вещественной матрицы. Преимуществом сингулярного разложения является наглядная геометрическая интерпретация и наличие встроенных численных алгоритмов во многих математических пакетах, например в системе *MATLAB*. В результате разложения исходной матрицы мы получаем произведение трех матриц: ортогональной, средней и еще одной ортогональной. Средняя матрица на главной диагонали содержит сингулярные числа, наибольшие значения которых определяют результаты поиска документов. Как показывает практика, сингулярные числа являются более надежными показателями смысла, чем отдельные термины.

Основное преимущество *MSA* заключается в возможности накопления результатов грамматического анализа текстов в специально создаваемой базе данных, что позволяет сократить объем вычислений.

Новой парадигмой построения поисковых механизмов является применение **систем нейронных сетей** (*Neural Network System*) и **онтологий** для поиска документов по запросам пользователей в коллекциях и при объединении результатов поиска серверами запросов. При ***кластеризации*** локальной коллекции в фоновом режиме профайлы, представляющие локальную коллекцию, становятся входными данными для нейронной сети. Нейронная сеть затем строит кластерное дерево: несколько кластеров верхнего уровня, группу субкластеров для каждого из кластеров верхнего уровня и так далее до отдельных документов. Для осуществления поиска в коллекции посредством запросов каждый кластер представлен документом, наиболее близко находящимся к центроиду кластера (*cluster centroid*) в векторном пространстве профайлов. Таким образом, профайл заданного запроса необходимо сравнить только с профайлами кластерного центроида. Это в значительной степени увеличивает скорость обработки запросов. Кроме этого, нейронные

сети могут использоваться для кластеризации результатов поиска, поступивших в ответ на запрос от нескольких серверов. Главная задача кластеризации — выдать пользователю репрезентативный набор результатов, если общее результирующее число документов превышает «порог», заданный пользователем.

Известно **два типа архитектуры нейронных сетей**: *Radius-based Competitive Learning* (далее — *RCL*) — основанная на радиусе, обучающаяся, соревновательного типа), и ее иерархическое расширение, называемое *Hierarchical Radius-based Competitive Learning* (далее — *HRCL*), иерархическая, основанная на радиусе, обучающаяся, соревновательного типа). В их основе лежит идея о том, что после каждого введения в систему входного вектора все нейроны упорядочиваются по их расстоянию к точке текущего ввода. Самый близкий к текущему вводу *RCL* нейрон становится победителем, кроме этого, *RCL* адаптирует все остальные нейроны из нейронного набора.

Высокая производительность и универсальность подсистемы нейронных сетей дает все основания для предположения о том, что она будет играть значительно большую роль в поисковых системах.

В настоящий момент наметилась тенденция к стандартизации описания структурированных, неструктурированных и полуструктурированных текстов при помощи *XML*-технологии, что позволяет наметить пути к созданию единой технологии их обработки.

Представление данных как *XML*-документов является естественным, поскольку они получаются из реальных документов. Представлять данные как документы привычнее и понятнее, чем представлять их как реляционные таблицы. Реляционная таблица, в лучшем случае — отдельный фрагмент документа. Неестественность табличного представления легко прочувствовать вначале при проектировании реляционных баз данных, когда из набора имеющихся документов происходит вычленение сущностей, и затем при подготовке отчета, когда из этих же сущностей вновь создаются документы. Манипулировать данными с использованием такой естественной для человека (но логически избыточной!) сущности, как «связь» также привычнее и понятнее, чем со ссылочными ключами, которые в реальных документах встречаются редко.

IBM разрабатывает базирующуюся на *XML*-систему поиска данных — *Архитектуру управления неструктурированной информацией* (далее — *UIMA*), которая, как предполагается, значительно расширит возможности средств поиска, применяемых в базах данных. По замыслу *UIMA* — это нечто, что становится частью базы данных или, скорее, тем, к чему базы данных обращаются, при этом появляется возможность «обдумывать» что-нибудь почти непрерывно. Это значительно изменит автоматизированные или человеко-машинные системы. Например, предполагается, что станет реальностью автоматический перевод с языка на язык и работа с естественными языками.

В основе *UIMA* лежит теория сочетания гипотез (*Combination Hypothesis*), которая утверждает, что в ближайшем будущем появится возможность объединить статистическое обучение машины — вроде того, которое использует поисковый сайт *Google* для интеллектуального ранжирования данных, — синтетический искусственный интеллект и другие методы. Между тем *XML* обеспечивает простой способ обмена данными и их классификации, что облегчает использование искусственного интеллекта в вычислительной среде. По мнению представителей *IBM*, благодаря появлению *XML* за ближайшие три года индустрия баз данных изменится сильнее, чем за предыдущие 20 лет. По сути, искусственный интеллект будет функционировать как фильтр. Датчики собирают информацию о внешнем мире и передают ее в компьютер, который выполняет надлежащие действия, беспокоя владельца лишь в случае крайней необходимости. Если нужно найти что-то в *Web*, человек делает запрос, а компьютер помогает ему уточнить его таким образом, чтобы вышло не 14 страниц списка потенциальных *Web*-сайтов, а только требуемая информация. В такой ситуации ключевой проблемой является задача быстрого и максимально эффективного поиска, т. е. такого поиска информации, который позволит за минимальное время найти по запросу пользователя наиболее релевантные (подходящие) ресурсы. В настоящее время для решения этой проблемы пытаются применить механизм онтологий.

Онтологии используются для систематизации данных на корпоративном портале для индексации и удобного поиска — несмотря на то, что многие крупные организации имеют собственную таксономию для организации внутренней информации, этого обычно недостаточно. Простая классификация сильно ограничивает возможности поиска и индексации, поскольку многие документы могут подпадать под разные категории, поэтому поиск по различным критериям будет намного эффективней, чем обычный поиск по ключевым словам.

Семантическая сеть (*Semantic web* согласно терминологии консорциума Всемирной паутины *World Wide Web Consortium W3C*) — воплощает развитие концепции существующей глобальной сети, направленной на создание условий для эффективной работы пользователей. Решая задачу придания информации точного смысла, в своем развитии глобальная система активно использовала:

- язык *XML* для описания собственной структуры документов;
- язык *RDF* (*Resource Definition Framework*) в качестве удобной среды формализации метаданных и сведений о контексте;
- *DTD* (*Document Type Definition*, формальное описание структуры *XML*-документов) в стандарте *XML* и схем *XML* (*XML Schema*) для обмена данными между сторонами, которые заранее договорились о значении определений и терминов.

Следует отметить что *RDF* (создан консорциумом *W3C*) является подмножеством языка *XML* и имеет собственный язык *RDF Schema*

для описания структуры документов. Практика применения перечисленных выше инструментальных средств выявила ряд серьезных недостатков.

Так *RDF* оперирует лишь понятиями связей примитивных сущностей (например, «объект А владеет субъектом В») и по своей сути относится к низкоуровневым языкам описания метаданных. Помимо недостатков семантического характера он сложен для решения задач в глобальной сети. В то же время *XML* ввиду своей общности не может быть использован для какого-либо конкретного применения и для развития семантической сети его возможностей недостаточно.

Отсутствие семантики в *DTD* и *RDF Schema* серьезно ограничивает надежность выполнения задачи поиска и объединения данных при использовании новых *XML*-словарей. Например, элемент *<Rate>*, встретившийся в разных документах, может означать курс валюты, оценку за экзамен, стоимость товара или другую цифровую оценку. Без дополнительной информации поисковый агент не сможет со стопроцентной вероятностью реализовать запрос пользователя.

Для осуществления точного *Web*-поиска необходимо было разработать универсальную унифицированную методику описания предметных областей. Данная задача была решена с использованием языка онтологий, базирующимся на общем наборе терминов и связей между ними. Практическим итогом такой работы было создание языка *OWL* (*Ontology Web Language*), разработанного при содействии консорциума *W3C* и послужившего основой интернет-стандартов.

Новый язык способствовал развитию автоматизированных инструментов для глобальной сети нового поколения в качестве интеллектуальных программных агентов управления знаниями. Использование онтологий особенно эффективно в приложениях-агентах, осуществляющих поиск и объединение информации из различных источников и сред, в которых один и тот же термин может означать разные вещи.

Практически любой пользователь сети хотя бы раз сталкивался с ситуацией, когда при поиске интересующей его информации он помимо прочего получал от поисковой машины множество бесполезных ссылок. Поскольку поиск информации осуществляется вне контекста, никакие уточнения запросов не смогут надежно найти именно то, что нужно. Для качественного осуществления поиска пользователю необходимо понимать все тонкости предметной области, включая ее лексику, термины, определения, иерархии сущностей — одним словом, досконально знать онтологию. Иногда, даже если пользователь является экспертом предметной области, может быть найдена лишняя информация. А что делать обычным пользователям? Язык *OWL* призван упростить процесс поиска, возложив необходимость знания предметной области и описание контекста поиска полностью на авторов документа и систему поиска, причем

передача этих функций авторам документа должна быть незаметна для пользователя.

2.2. Транспортирование информации

Основным физическим способом реализации операции транспортировки является использование локальных сетей и сетей передачи данных. При разработке и использовании сетей для обеспечения совместимости используется ряд стандартов, объединенных в семиуровневую модель открытых систем, принятую во всем мире и определяющую правила взаимодействия компонентов Сети на данном уровне (протокол уровня) и правила взаимодействия компонентов различных уровней (межуровневый интерфейс) [8, 9]. Международные стандарты в области сетевого информационного обмена нашли отражение в эталонной семиуровневой модели, известной как модель *Open System Interconnection* (далее — *OSI*) (связь открытых систем) (рис. 2.3). Данная модель разработана международной организацией по стандартизации (*International Standards Organization — ISO*). Большинство производителей сетевых программно-аппаратных средств стремятся придерживаться модели *OSI*. Но в целом добиться полной совместимости пока не удается.

Рис. 2.3. **Связь открытых систем**

Физический уровень. Он реализует физическое управление и относится к физической цепи, по которой передаются биты информации, например, телефонной. На этом уровне модель *OSI* определяет физические, электрические, функциональные и процедурные характеристики цепей связи, а также требования к сетевым адаптерам и модемам.

Канальный уровень. Этот уровень осуществляет управление звеном сети (каналом) и относится к пересылке блоков (совокуп-

ности битов) по физическому звену. Осуществляет такие процедуры управления, как определение начала и конца блока, обнаружение ошибок передачи, адресация сообщений и др. Канальный уровень определяет правила совместного использования сетевых аппаратных средств компьютерами сети.

Сетевой уровень. Относится к виртуальной (воображаемой) цепи, которая не обязана существовать физически. С помощью интерфейса, обеспечиваемого этим уровнем, удается «спрятать» сложности управления передачей на физическом уровне. Программные средства данного уровня обеспечивают определение маршрута передачи пакетов в сети. Маршрутизаторы, обеспечивающие поиск оптимального маршрута на основе анализа адресной информации, функционируют на сетевом уровне модели *OSI*. В качестве простейшего маршрутизирующего устройства между сегментами сети или различными локальными сетями может выступать и устройство, функционирующее на более низком канальном уровне модели *OSI*, называемое мостом.

Транспортный уровень. Первые три уровня образуют общую сеть, которую коллективно могут использовать многие пользователи. На транспортном уровне контролируется очередность пакетов сообщений и их принадлежность. Таким образом, в процессе обмена между компьютерами поддерживается виртуальная связь, аналогичная телефонной коммутации.

Сеансовый уровень. Из-за обилия способов взаимодействия между пользователями в некоторых случаях трудно организовать процесс взаимодействия между ними. Для устранения этих трудностей на данном уровне координируются и стандартизируются процессы установления сеанса, управления передачей и приемом пакетов сообщений, завершения сеанса. На сеансовом уровне между компьютерами устанавливается и завершается виртуальная связь по такому же принципу, как при голосовой телефонной связи.

Управление представлением. Программные средства этого уровня выполняют преобразования данных из внутреннего формата передающего компьютера во внутренний формат компьютера получателя, если эти форматы отличаются друг от друга (например, *IBM PC* и *DEC*). Данный уровень включает функции, относящиеся к используемому набору символов, кодированию данных и способам представления данных на экранах дисплеев или печати. Помимо конвертирования форматов на данном уровне осуществляется сжатие передаваемых данных и их распаковка.

Прикладной уровень. Относится к функциям, которые обеспечивают поддержку пользователю на более высоком прикладном и системном уровне, например:

- организация доступа к общим сетевым ресурсам — информации, дисковой памяти, программным приложениям, внешним устройствам (принтерам, стримерам и др.);

- общее управление сетью (управление конфигурацией, разграничение доступа к общим ресурсам сети, восстановление работоспособности после сбоев и отказов, управление производительностью);
- передача электронных сообщений, включая электронную почту;
- организация электронных конференций;
- диалоговые функции высокого уровня.

Модель *OSI* представляет собой стандартизированный каркас и общие рекомендации, требования же к конкретным компонентам сетевого программного обеспечения задаются протоколами.

Протокол является стандартом в области сетевого программного обеспечения и определяет совокупность функциональных и эксплуатационных требований к какому-либо его компоненту, которых придерживаются производители этого компонента. Требования протокола могут отличаться от требований эталонной модели *OSI*.

Международный институт инженеров по электротехнике и радиоэлектронике (*Institute of Electrical and Electronics Engineers — IEEE*) разработал стандарты для протоколов передачи данных в локальных сетях. Эти стандарты, которые описывают методы доступа к сетевым каналам данных, получили название *IEEE* 802.

Протоколы сетевого взаимодействия можно классифицировать по степени близости к физической среде передачи данных:

- *протоколы нижнего уровня*, распространяемые на канальный и физический уровни модели *OSI*;
- *протоколы среднего уровня*, распространяемые на сетевой, транспортный и сеансовый уровни *OSI*;
- *протоколы верхнего уровня*, распространяемые на уровень представления и прикладной уровень модели *OSI*.

Каждая из реализаций протоколов вышестоящих уровней использует в процессе своей работы реализации протоколов нижестоящих уровней.

Протоколы нижнего уровня OSI соответствуют уровню сетевых аппаратных средств и нижнему уровню сетевого программного обеспечения. Среди наиболее распространенных стандартов данного уровня выделим следующие [10].

- стандарт *NDIS* (*Network Driver Interface Specification* — спецификация интерфейса сетевых драйверов), разработанный совместно фирмами *Microsoft* и *3Com*;
- стандарт *ODI* (*Open Datalink Interface* — открытый интерфейс связи), разработанный совместно фирмами *Novell* и *Apple Computer*.

Данные стандарты позволяют реализовывать протоколы среднего уровня независимо от сетевых аппаратных средств и обеспечивают совместное функционирование разнотипных протоколов среднего уровня. Универсальный интерфейс канального уровня представлен

на рис. 2.4. Производители сетевых аппаратных средств, как правило, разрабатывают драйверы, удовлетворяющие обоим стандартам.

Рис. 2.4. **Универсальный интерфейс канального уровня**

Драйвер сетевого адаптера является последним программным компонентом перед физическим уровнем модели *OSI* и называется подуровнем управления доступом к среде *MAC* (*Media Access Control*). Подуровень *MAC* ориентирован на выполнение таких функций, как непосредственное управление доступом к передающей среде, проверке пакетов сообщений на наличие ошибок.

Подуровень *LLC* (*Logical Linc Control*) считается независимым от особенностей физической передающей среды и используемых методов доступа к каналам передачи данных. Стандарты по разработке интерфейсов для связи реализаций протоколов среднего уровня модели *OSI* с драйверами сетевых аппаратных средств как раз относятся прежде всего к подуровню *LLC*.

Протоколы среднего уровня распространяются на сетевой, транспортный и сеансовый уровни эталонной модели. По типу межкомпьютерного обмена эти протоколы можно классифицировать следующим образом:

- сеансовые протоколы (протоколы виртуального соединения);
- дейтаграммные протоколы.

Сеансовые протоколы определяют организацию передачи информации между компьютерами по так называемому виртуальному каналу в три этапа:

- установление виртуального канала (установка сеанса);
- реализация непосредственного обмена информацией;
- уничтожение виртуального канала (разъединение).

В сеансовых протоколах порядок следования пакетов при передаче соответствует их исходному порядку в сообщении, а передача осуществляется с подтверждением доставки, а в случае потери отправленных пакетов они передаются повторно.

При использовании *дейтаграммных протоколов* пакеты сообщений передаются так называемыми дейтаграммами независимо

друг от друга, поэтому порядок доставки пакетов каждого сообщения может не соответствовать их исходному порядку в сообщении. При этом пакеты сообщений передаются без подтверждения.

Таким образом, с точки зрения достоверности, сеансовые протоколы являются более предпочтительными, зато скорость передачи при использовании дейтаграммных протоколов гораздо выше.

Любой протокол среднего уровня предусматривает следующие этапы реализации межкомпьютерного обмена:

- инициализация связи;
- непосредственный информационный обмен;
- завершение обмена.

Наиболее часто используемыми наборами протоколов среднего уровня являются следующие:

- набор протоколов *SPX/IPX*, используемый в локальных сетях, функционирующих под управлением сетевой операционной системы *NetWare*;
- протоколы *NetBIOS* и *NetBEUI*, поддерживаемые большинством сетевых операционных систем и используемых только в локальных сетях;
- протоколы *TCP/IP*, являющиеся стандартом для глобальной сети Интернет, используемые в локальных сетях и поддерживаемые большинством сетевых операционных систем.

Набор протоколов *SPX/IPX* используется в сетевой операционной системе *NetWare* фирмы *Novell*.

Протокол *IPX* (*Internetwork Packet Exchange* — межсетевой обмен пакетами) является дейтаграммным протоколом и соответствует сетевому уровню эталонной модели. Применяется для выполнения функций адресации при обмене пакетами сообщений.

Протокол *SPX* (*Sequenced Packet Exchange* — последовательный обмен пакетами) является сеансовым протоколом и соответствует транспортному и сеансовому уровням эталонной модели. По степени близости к самому низкому уровню эталонной модели протокол *SPX* находится над протоколом *IPX* и использует этот протокол.

Драйвер, реализующий протокол *SPX* использует в процессе своей работы драйвер, реализующий протокол *IPX*. Протокол *IPX* является более быстродействующим, чем протокол *SPX*.

Важным недостатком протоколов *SPX* и *IPX* является несовместимость с протоколами *TCP/IP*, используемыми в глобальной сети Интернет. Для подключения локальной сети *NetWare* к Интернет используется один из следующих способов:

- непосредственная инсталляция на каждом сетевом компьютере драйверов, реализующих набор протоколов *TCP/IP*;
- подключение локальной сети к Интернет через шлюз *IPX-IP*.

Протоколы *NetBIOS* и *NetBEUI* разработаны фирмой *IBM* и предназначены только для локальных компьютерных сетей.

Протокол *NetBIOS* (*Network Basic Input/Output System* — базовая система ввода-вывода) соответствует сетевому, транспортному и сеансовому уровням эталонной модели. Реализация данного протокола обеспечивает прикладной интерфейс, используемый для создания сетевых программных приложений.

Протокол *NetBEUI* (*Extended User Interface NetBIOS* — расширенный пользовательский интерфейс *NetBIOS*) является модификацией предыдущего протокола и распространяется только на сетевой и транспортный уровни.

Реализации протоколов *NetBIOS* и *NetBEUI* обеспечивают решение следующих задач: поддержка имен, поддержка сеансового и дейтаграммного взаимодействия, получение информации о состоянии Сети.

Достоинства протоколов *NetBIOS* и *NetBEUI*: удобная адресация, высокая производительность, самонастройка и хорошая защита от ошибок, экономное использование оперативной памяти.

Недостатки *NetBIOS* и *NetBEUI* связаны с отношением к глобальным сетям: отсутствие поддержки функций маршрутизации и низкая производительность.

Семейство протоколов *TCP/IP* было разработано для объединения различных компьютерных сетей в одну глобальную сеть, получившую название *Internet*.

Семейство протоколов *TCP/IP* включает протоколы, относящиеся как к средним, так и другим уровням модели *OSI*:

- прикладной уровень и уровень представления — протокол передачи файлов (*FTP*), протоколы электронной почты (*SMTP*, *POP3*, *IMAP4*), протоколы удаленного доступа (*SLIP*, *PPP*, *Telnet*), протокол сетевой файловой системы (*NPS*), протокол управления сетями (*SNMP*), протокол передачи гипертекста (*HTPP*) и др.;
- сеансовый и транспортные уровни — протоколы *TCP* и *UDP*;
- сетевой уровень — протоколы *IP*, *ICMP*, IGMP;
- канальный уровень — протоколы *ARP*, *RARP*.

Дейтаграммный протокол *IP* (*Internet Protocol*) является основным для сетевого уровня и обеспечивает маршрутизацию передаваемых пакетов сообщений.

Протокол *ICMP* (*Internet Control Message Protocol*) отвечает за обмен сообщениями об ошибках и другой важной информацией с программными средствами сетевого уровня на другом компьютере, маршрутизаторе или шлюзе.

Протокол *IGMP* (*Internet Management Protocol*) используется для отправки *IP*-пакетов множеству компьютеров в Сети.

Протокол *TCP* (*Transmission Control Protocol*) является протоколом сетевого уровня и обеспечивает надежную передачу данных между двумя компьютерами путем организации виртуального канала обмена и использования его для передачи больших массивов данных.

Протокол *UDP* (*User Datagram Protocol*) реализует гораздо более простой сервис передачи, обеспечивая надежную доставку данных без установления логического соединения.

Протоколы верхнего уровня соответствуют уровню пользователей и прикладных программ и распространяются на уровень представления и прикладной уровень эталонной модели сетевого взаимодействия. Наиболее распространенными являются следующие высокоуровневые протоколы:

- перенаправления запросов и обмена сообщениями (*SMB*, *NCP*);
- управления сетями (*SNMP*);
- сетевой файловой системы (*NFS*);
- вызова удаленных процедур (*RPC*);
- повышающие эффективность использования протоколов *TCP/IP* среднего уровня (*DNS*, *DHSP*);
- удаленного доступа к компьютерным ресурсам (*SLIP*, *PPP*, *Telnet*);
- передачи файлов (*FTP*);
- передачи гипертекста (*HTTP*);
- электронной почты (*SMTP*, *POP3*, *IMAP4*);
- организации электронных конференций и системы новостей (*NNTP*).

Протокол *SMB* (*Server Message Blocks* — блоки серверных сообщений), разработанный совместно корпорациями *Microsoft*, *Intel* и *IBM*, используется в сетевых операционных системах *Windows NT*, *Lan Manager*, *LAN Server*. Данный протокол определяет серии команд, используемых для передачи информации между сетевыми компьютерами.

Протокол *NCP* (*NetWare Core Protocol* — протокол ядра *NetWare*) разработан фирмой *Novell* и используется в сетевых ОС *NetWare*.

Протокол *SNMP* (*Simple Network Management Protocol* — простой протокол управления сетью) осуществляет гибкое и полное управление сетью, предполагая выполнение администратором следующих функций: управление конфигурацией, управление доступом к общим сетевым ресурсам, управление производительностью, управление подготовкой к восстановлению, управление восстановлением. При этом любая из функций управления должна обеспечивать решение трех базовых задач:

- получение информации о состоянии управляемого объекта;
- анализ полученной информации и выработку управляющих воздействий;
- передачу управляющих воздействий на исполнение.

Протокол *NFS* (*Network File System* — сетевая файловая система) предназначен для предоставления универсального интерфейса работы с файлами для различных операционных систем, сетевых архитектур и протоколов среднего уровня.

Протокол *RPC* (*Remote Procedure Call* — сервис вызова удаленных процедур) предназначен для организации межпрограммных взаимодействий для сети «клиент-сервер» и обеспечивает связь между процессами-клиентами и процессами-серверами, функционирующими на разных компьютерах сети.

Протокол *DNS* (*Domain Name System* — система доменных имен) предназначен для установления соответствия между смысловыми символьными именами и *IP*-адресами компьютеров.

Протокол *DHCP* (*Dynamic Host Configuration Protocol* — протокол динамической конфигурации компьютеров) позволяет автоматически назначать *IP*-адреса подключаемых к сети компьютеров и изменять их при перемещении из одной подсети в другую.

Протокол *SLIP* (*Serial Line Internet Protocol* — протокол *Internet* последовательного соединения) обеспечивает работу протоколов *TCP/IP* при коммутируемом телефонном соединении.

Протокол *PPP* (*Point-to-Point Protocol* — протокол «точка-точка») обеспечивает установление соединения и реализацию непосредственного обмена информацией, а также по сравнению со *SLIP* позволяет решать следующие задачи:

- конфигурация и проверка качества связи;
- подтверждение подлинности (аутентификация) удаленного пользователя;
- динамическое присвоение адресов *IP* и управление этими адресами;
- обнаружение и коррекция ошибок и др.

Протокол *PPTP* (*Point-to-Point Tunntling Protocol* — туннельный протокол «точка-точка») ориентирован на поддержку мультипротокольных виртуальных частных сетей (*Virtual Private Networks* — *VPN*), предоставляя возможность удаленным пользователям иметь безопасный доступ к корпоративным сетям по Интернету.

Протокол *Telnet* является общепризнанным стандартом удаленного дистанционного управления в Интернете, позволяющий в режиме командной строки запускать и выполнять программы на компьютере, с которым установлено удаленное соединение.

Основное требование к компьютерной сети — это выполнение сетью того набора услуг, для которого она предназначена. К таким услугам могут относиться:

- доступ к файловым архивам;
- доступ к страницам веб-сайтов;
- обмен с использованием электронной почты;
- интерактивный обмен с помощью *IP*-телефонии;
- потоковое видео и т. д.

Все остальные требования (а это могут быть: производительность, надежность, совместимость, управляемость, защищенность,

расширяемость и масштабируемость) связаны с качеством выполнения основной задачи.

Хотя важны все перечисленные требования, в понятие «качество обслуживания» (*Quality of Service — QoS*) для компьютерной сети часто включают только две важнейшие характеристики — производительность и надежность.

Рассмотрим эти параметры.

Производительность. К основным характеристикам производительности относятся:

- время реакции;
- скорость передачи данных;
- задержка передачи и ее вариация.

Время реакции определяют как интервал времени между возникновением запроса пользователя к какой-либо сетевой службе и получением ответа на этот запрос.

Обычно это время складывается из нескольких составляющих (рис. 2.5):

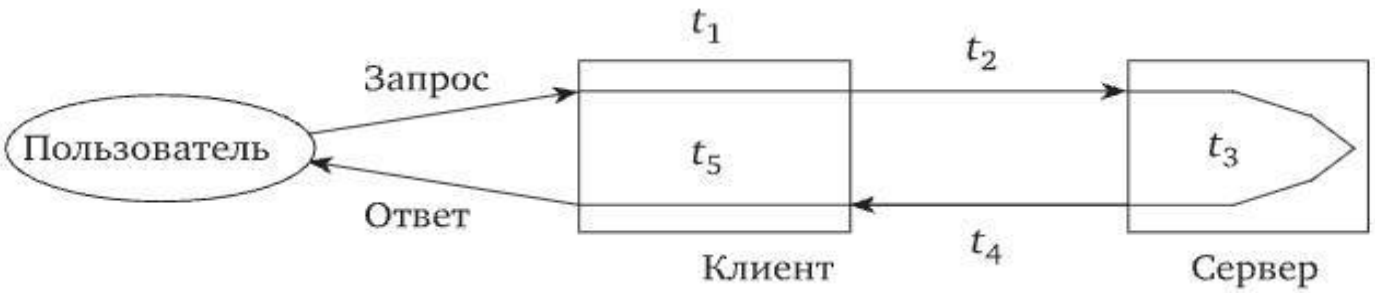


Рис. 2.5. **Обмен «Пользователь — Клиент — Сервер»**

t_1 — время подготовки запроса на пользовательском компьютере;
t_2 — время передачи между клиентом и сервером;
t_3 — время обработки запроса на сервере; t_4 — время доставки ответа с сервера на клиентский компьютер; t_5 — время обработки ответа на этом компьютере

С точки зрения работы сети здесь важнейшими составляющими являются t_2 и t_4 — т. е. задержки на передачу по сети.

Скорость передачи данных — это объем данных, передаваемый в единицу времени. Используется также понятие пропускной способности — это скорость передачи пакетов между узлами сети через различные коммуникационные устройства.

Скорость передачи измеряется:

- в битах в секунду;
- в пакетах в секунду.

Различают три скорости:

1) *средняя скорость* — берется достаточно длительный промежуток времени (час, сутки) и общий объем переданных данных делится на время;

2) *мгновенная скорость* — для усреднения выбирается очень маленький промежуток времени — например, 10 мс или 1 с.

3) *максимальная скорость* — это максимальная мгновенная скорость, зафиксированная за время наблюдения.

При проектировании сети чаще всего пользуются параметрами средней и максимальной скорости.

Иногда оперируют и общей пропускной способностью, рассматриваемой как максимальное количество информации, передаваемой между всеми узлами сети в единицу времени.

Задержка передачи — это время между моментом поступления данных на вход какого-то сетевого устройства или части сети и моментом появления этих данных на выходе, т. е.

$$t_{\text{задержки}} = t_{\text{вых}} - t_{\text{вх}},$$

где $t_{\text{задержки}}$ — задержка передачи, $t_{\text{вых}}$ — момент появления данных на выходе, $t_{\text{вх}}$ — момент поступления данных на вход.

Отличием этого параметра от времени реакции является то, что в $t_{\text{задержки}}$ никогда не включается время на обработку в конечных узлах сети.

Обычно рассматривают:

- максимальную задержку;
- вариацию задержки.

Надежность и безопасность. Для обычных технических средств используют следующие показатели надежности:

- среднее время наработки на отказ;
- вероятность отказа;
- интенсивность отказов.

Однако эти характеристики пригодны для оценки только простых устройств, которые могут находиться в двух состояниях — работоспособном и отказа.

Сложные системы из многих элементов могут иметь и промежуточные состояния. Поэтому для оценки надежности таких сложных систем, как сети, применяют другой набор характеристик:

1) *готовность* (*или коэффициент готовности*) (*Availability*). Это доля времени, в течение которого система может быть использована. Для увеличения коэффициента готовности в состав системы включаются резервные элементы;

2) *сохранность данных* (*и их защита от искажений*);

3) *согласованность данных* (*их непротиворечивость*) требуется, например, когда несколько копий данных хранятся на разных файловых серверах;

4) *вероятность доставки пакета узлу назначения без искажений*. Может также применяться параметр — *вероятность потери пакета* (например, из-за переполнения буфера маршрутизатора, отсутствия работоспособного пути в сети, поражения пакета ошибками). Этот параметр может представляться в виде:

- вероятности искажения отдельного бита передаваемых данных,
- отношения числа потерянных пакетов к общему числу передаваемых пакетов;

5) *безопасность* (*Security*). Это способность системы защитить данные от несанкционированного доступа. Сюда относятся — защита каналов, защита компьютеров, защита от взлома паролей и т. д.;

6) *отказоустойчивость* (*Fault tolerance*). В сетях под этим понимают способность системы скрывать от пользователя факт отказа отдельных элементов. Например, если копии данных хранятся на нескольких файловых серверах, пользователь может и не заметить факт отказа одного их них. В этом случае говорят о деградации системы, так как при отказе одного сервера увеличивается время доступа к базе данных из-за уменьшения степени — распараллеливания запросов;

7) специфичными для сетей являются параметры расширяемости и масштабируемости:

- *расширяемость* (*Extensibility*). Это возможность легко добавлять в сеть новые элементы (пользователей, компьютеры, приложения, службы), наращивать длину сегментов и заменять аппаратуру более мощной;
- *масштабируемость* (*Scalability*). Этот параметр означает, что в сети возможно наращивание количества узлов и протяженности связей в очень широких пределах. При этом производительность сети не ухудшается.

Часто термины «расширяемость» и «масштабируемость» используются как синонимы. Однако если взять, к примеру, сеть *Ethernet*, то можно говорить о хорошей расширяемости (количество компьютеров на сегменте можно увеличить до 100), но при этом резко снижается производительность сети, т. е. это указывает на плохую масштабируемость.

8) *прозрачность* (*Transparency*). Прозрачность сети достигается в том случае, когда для пользователя сеть представляется не как множество компьютеров, связанных сложной системой каналов, а как единая вычислительная машина с системой разделения времени.

Символом прозрачности считают принцип: «Сеть — это компьютер».

Прозрачность может достигаться на двух уровнях — пользователя и программиста:

- *на уровне пользователя* — для работы в сети используются те же команды и привычные процедуры, что и для работы с локальными ресурсами;
- *на уровне программиста* — приложению для доступа к удаленным ресурсам требуются те же вызовы, что и для локальных ресурсов.

Сеть должна скрывать различия операционных систем и компьютеров. Можно одинаково обращаться к ресурсам на компьютере с *ОС Macintoch*, *Windows* или *Unix*. От пользователя не требуется знания места расположения ресурса. Ресурсы должны свободно перемещаться с одного компьютера на другой без изменения их имен;

9) *поддержка разных видов трафика*. Наряду с традиционным трафиком передачи данных все увеличивается доля мультимедийного трафика — передаваемых в цифровой форме речи и изображения.

Особенность мультимедийного трафика — это жесткие требования к синхронизации передаваемых данных. При запаздывании сообщений будут наблюдаться искажения.

Необходимость передачи мультимедийного трафика требует внесения изменений, как в протоколы, так и в оборудование.

В сети в общем случае должны сосуществовать два вида трафика:

- традиционный компьютерный (пульсирующий);
- мультимедийный (синхронный).

Это является сложной задачей и ближе всего к ее решению подошли сети *ATM* (англ. *Asynchronous Transfer Mode* — асинхронный способ передачи данных);

10) *управляемость* — это возможность централизованно контролировать состояние основных элементов сети, выявлять и устранять неисправности, выполнять анализ производительности и планировать развитие сети.

В этой области еще очень много нерешенных проблем. В основном существующие системы не управляют сетью, а лишь осуществляют наблюдение за ее работой;

11) *совместимость*, или *интегрируемость* означает, что сеть способна включать в себя разнообразное ПО и аппаратное обеспечение. Иными словами, сеть может быть неоднородной или гетерогенной. Еще такие сети называют интегрированными.

В них могут сосуществовать различные *ОС* (англ. *Operating System*, *OS* — операционная система), стеки протоколов, аппаратные средства и приложения от разных производителей.

Основной путь обеспечения совместимости — это использование открытых стандартов и спецификаций.

Традиционные сети обеспечивают сервис, который получил название «*Best effort*» — с максимальными усилиями. Это означает, что сеть не дает никаких гарантий на обслуживание.

Примеры: сети *Ethernet*, *Token Ring*, *IP*, *X*25.

При обработке очередей используется обычно алгоритм *FIFO*, а при переполнении буфера — пакеты отбрасываются.

В настоящее время самый распространенный путь обеспечения *QoS* — это постоянное наращивание пропускной способности сети, т. е. стремление постоянно иметь избыточную пропускную способность.

Встроенные механизмы *QoS* пока применяются только в таких сетях, как ATM и *Frame Relay*.

Типы QoS. *Сервис Best Effort* — с максимальными усилиями. Это фактически отсутствие *QoS*. Обслуживание производится без каких-либо гарантий. Пример: *IP*-сети и *Ethernet* с принципом *FIFO*.

Сервис с предпочтением (называют также «мягким» сервисом *QoS*). Здесь некоторые виды трафика обслуживаются лучше остальных. Но характеристики обслуживания точно неизвестны — они зависят от характеристик трафика. Например, при высокой интенсивности высокоприоритетного трафика может совсем прекратиться обслуживание трафика с низким приоритетом.

Гарантированный сервис (его называют также «жестким» или «истинным» сервисом *QoS*). Различным типам трафика даются статистические гарантии. Обычно этот тип *QoS* основан на предварительном резервировании сетевых ресурсов для каждого из потоков, получивших гарантии обслуживания. Однако и эти гарантии носят статистический характер. Например, с вероятностью 0,999 задержка пакета не должна превышать 100 мс.

При этом производится контроль интенсивности входных потоков — чтобы это значение не превышало заранее оговоренную величину. Такой тип *QoS* применяется обычно для обслуживания тех приложений, для которых нужны гарантии пропускной способности и (или) задержек. Например, это может быть трафик видеоконференции или трафик, поступающий от измерительных систем реального времени.

Рассмотрим **новые базовые сети и их роль в обеспечении качества обслуживания**.

Эти сети являются основой вторичных сетей — компьютерных и телефонных. Как было показано выше, обеспечение *QoS* зависит от имеющегося резерва пропускных способностей, т. е. от производительности первичной сети. Без развития первичных сетей невозможен прогресс сетевых технологий.

Такие сети называют также *опорными* и *первичными*.

Современные сети основаны на коммутации каналов. Для создания абонентского канала коммутаторы первичных сетей поддерживают один из методов мультиплексирования и коммутации.

В настоящее время для мультиплексирования абонентских каналов используются:

- техника частотного мультиплексирования (*Frequency Division Multiplexing*, ***FDM***);
- мультиплексирование с разделением времени (*Time Division Multiplexing*, ***TDM***);
- мультиплексирование по длине волны (*Wave Division Multiplexing*, ***WDM***).

Частотное мультиплексирование. Этот метод применяется в основном в телефонных сетях, где речевой телефонный канал имеет спектр 300 – 3400 Гц (т. е. на его передачу необходима пропускная способность 3100 Гц). Кабельные же системы между телефонными коммутаторами имеют пропускную способность в сотни мегагерц. Для передачи производится модуляция высокочастотного сигнала низкочастотным. Таким образом, спектр модулированного сигнала переносится в другой частотный диапазон. Высокочастотный сигнал делится на полосы по 4000 Гц (3100 + 900 — страховой промежуток). В сетях на основе *FDM*-коммутации принято несколько уровней уплотненных каналов. Это базовая группа (12 абонентских каналов), супергруппа (60 абонентских каналов) и главная группа (600 абонентских каналов), которая имеет полосу пропускания 2520 КГц (564 – 3084 КГц).

Мультиплексирование по длине волны. Первичные сети с мультиплексированием по длине волны (*WDM* и *DWDM*) используют тот же принцип частотного разделения, но информационным сигналом в них является не электрический ток, а свет. Используется инфракрасный диапазон с длинами волн от 850 до 1565 нм, что соответствует частотам 196—350 ТГц. В магистральном канале обычно мультиплексируются достаточно много спектральных каналов: 16, 32, 40, 80 или 160. Если используется 16 и более каналов, такая техника часто называется плотной, т. е. *Dense WDM* или *DWDM*. Внутри спектрального канала данные могут кодироваться как дискретным, так и аналоговым способом.

Коммутация каналов на основе разделения времени. Как уже упоминалось, эта техника носит название «Мультиплексирование с разделением времени» (*Time Division Multiplexing*, *TDM*). Основой этой технологии являются каналы T1/E1, которые были предложены для передачи вызовов между телефонными станциями (АТС). Это дуплексные цифровые каналы.

Для передачи используются две пары витых проводников (по паре в каждую сторону). В 1990-е гг. эти каналы стали очень популярны в качестве средства подключения абонентов (небольших фирм, корпоративных сетей) к сети Интернет.

На базе внедренных в телефонии каналов T1 к настоящему времени сформировалось два поколения таких цифровых базовых сетей:

1. Технология *плезиохронной цифровой передачи* (*Plesiochronous Digital Hierarchy*, *PDH*). «Плезо» означает «почти», т. е. это почти синхронная передача.

2. *Синхронная цифровая иерархия* (*Sinchronous Digital Hierarchy, SDH*). В США технология *SDH* называется *SONET*.

Недостатки технологии *PDH*:

1. Сложность мультиплексирования и демультиплексирования данных. Например, для извлечения одного канала из потока Т3 надо демультиплексировать канал до Т2, затем — до Т1, а уже затем демультиплексировать канал Т1.
2. Отсутствие встроенных средств контроля и управления сетью. Нет и процедур поддержки отказоустойчивости.
3. Слишком низкие (по понятиям современных сетей) скорости передачи. Иерархия скоростей Е1 заканчивается на 139 Мбит/с, а современные оптоволоконные каналы позволяют передавать со скоростью в десятки гигабит в секунду.

Эти недостатки были устранены в сетях *SDH*, которые и стали в настоящее время одним из тех типов сетей, который позволяет удовлетворить требования к качеству обслуживания.

Сети *SDH*. Технология синхронной цифровой иерархии разработана для создания надежных транспортных каналов, позволяющих гибко формировать цифровые каналы в широком диапазоне скоростей — от единиц мегабит в секунду до десятков гигабит в секунду.

Основная область применения — первичные сети операторов связи.

Эти сети относятся к классу полупостоянных сетей с коммутацией каналов — формирование канала происходит по инициативе оператора связи. Поэтому здесь чаще всего вместо термина «коммутация» используют термин «кросс-коннект» (*cross-connect*). Используется мультиплексирование с разделением времени *TDM*. Информация адресуется путем относительного временного положения внутри составного кадра. Обычно эти сети используются для объединения большого числа более низкоскоростных каналов *PDH*. На рис. 2.6 приведен пример структуры сети *SDH*.

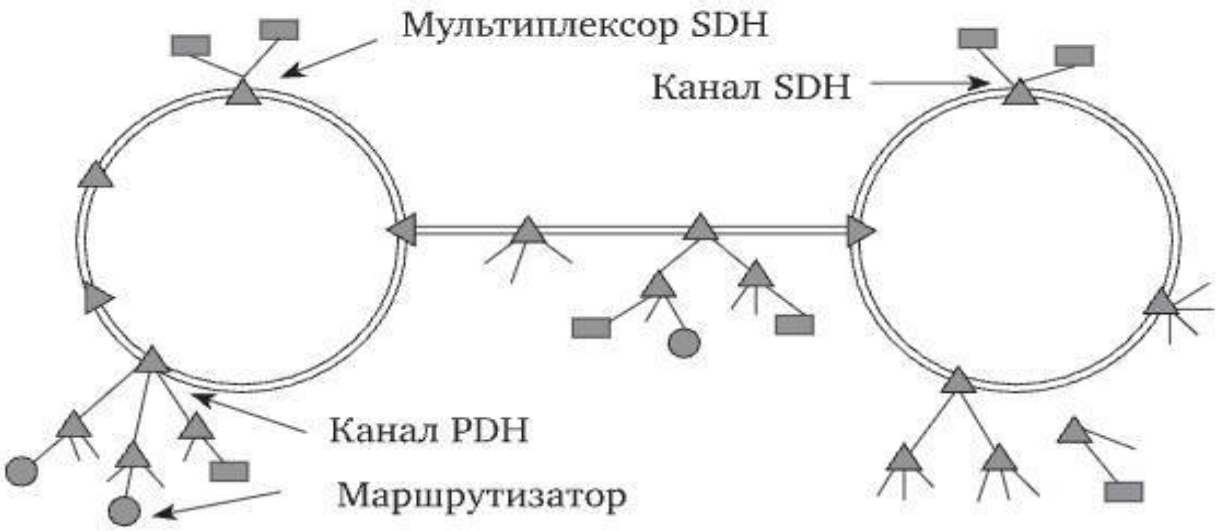


Рис. 2.6. **Пример структуры сети SDH**

Достоинства сетей *SDH*:

1) гибкая иерархическая система мультиплексирования цифровых потоков с различными скоростями. Возможность ввода и вывода пользовательской информации без демультиплексирования потока в целом;

2) отказоустойчивость сети. Использование резервных маршрутов и резервного оборудования. Переход на резервный путь обычно требует не более 50 мс;

3) мониторинг и управление сетью на основе той информации, которая встроена в заголовки кадров.

Высокое качество транспортного обслуживания для трафика любого вида — голосового, видео и компьютерного. Техника мультиплексирования *TDM*, лежащая в основе *SDH*, обеспечивает трафику каждого абонента гарантированную пропускную способность, а также низкий и фиксированный уровень задержек.

Сети *SDH* составляют сегодня фундамент практически всех крупных телекоммуникационных сетей — региональных, национальных и международных.

Эти сети легко интегрируются с сетями *DWDM*, обеспечивая передачу информации по оптическим магистралям со скоростями сотни гигабит в секунду за счет мультиплексирования с разделением по длине волны (рис. 2.7).

В *DWDM*-сетях сети *SDH* играют роль сетей доступа, т. е. ту же роль, что играют по отношению к ним сети *PDH*.

Типовые топологии. Чаще всего используются — кольца, линейные цепи и ячеистая топология, близкая к полносвязной.

Сети *DWDM*. Вторым типом современных базовых сетей, которые позволяют обеспечить поддержку служб, имеющих требуемое качество обслуживания, являются сети *DWDM*.

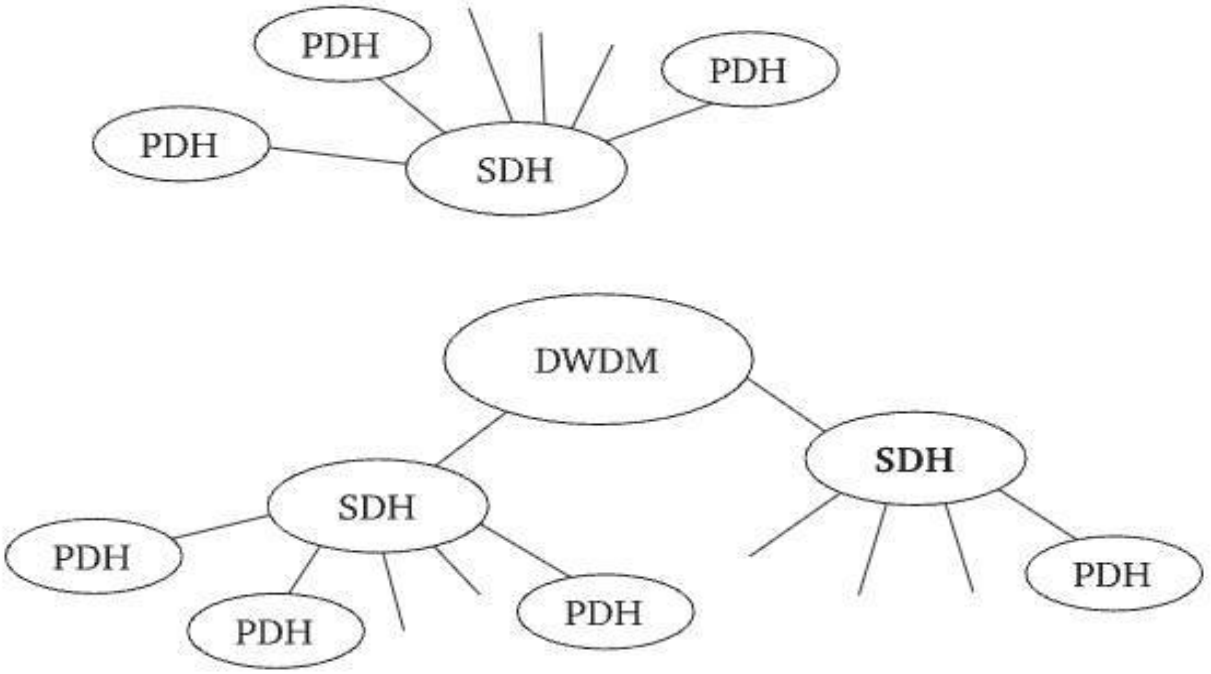


Рис. 2.7. **Иерархия базовых сетей и сетей доступа**

Технология плотного волнового (спектрального) мультиплексирования (*Dense Wave Division Multiplexing*, *DWDM*) предназначена для создания оптических магистралей нового поколения, работающих на мультимегабитных и терабитных скоростях.

Информация в оптическом волокне передается одновременно большим количеством световых волн.

Сети *DWDM* работают по принципу коммутации каналов — при этом каждая световая волна представляет собой отдельный спектральный канал.

Каждая волна несет свою собственную информацию, при этом оборудование *DWDM* не занимается непосредственно проблемами передачи данных на каждой волне, т. е. способом кодирования информации и протоколом ее передачи.

Устройства *DWDM* занимаются только объединением различных волн в одном световом пучке, а также выделением из общего сигнала информации каждого спектрального канала.

Оборудование *DWDM* позволяет передавать по одному оптическому каналу 32 и более волн различной длины в окне прозрачности 1550 нм. При этом каждая волна может переносить информацию со скоростью до 10 Гбит/с (при применении протоколов STM-64 или 10GE). Ведутся работы по повышению этой скорости до 40—80 Гбит/с.

Мультиплексирование *DWDM* называется «плотным», так как в нем используется существенно меньшее (чем у предшествующей технологии *WDM*) расстояние между длинами волн.

В рекомендации *G*.692 определен частотный план с разнесением частот на 100 ГГц ($\Delta\lambda$ = 0,8 нм). Для передачи используется 41 волна (от 1528 нм (191 ТГц) — до 1560 нм (192 ТГц)).

Определен также частотный план с разнесением на 50 ГГц ($\Delta\lambda$ = = 0,4 нм), что позволяет передавать в этом диапазоне 81 волну.

Имеются экспериментальные образцы с разнесением на 25 ГГц. Такая технология называется *High Dence WDM* (далее —*HDWDM*).

Как показано на рис. 2.8, необходимо обеспечить высокую точность частоты, чтобы не допустить перекрытия спектра каналов.

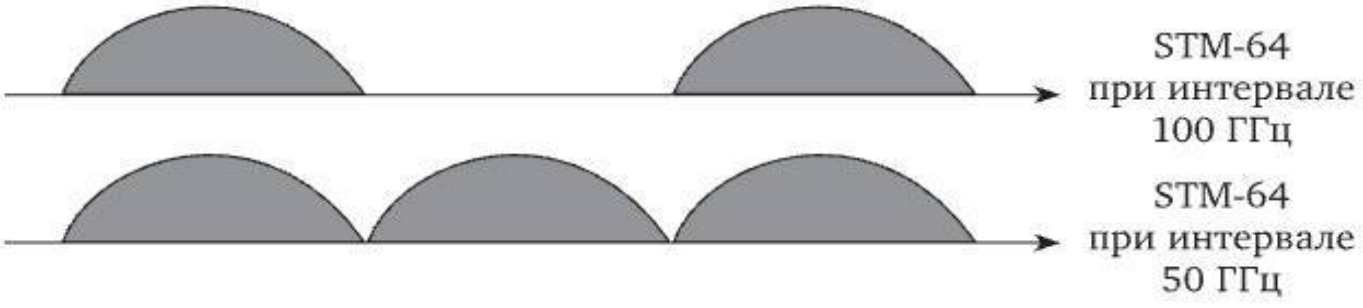


Рис. 2.8. **Интервалы между каналами**

***Преимущества технологии DWDM*:**

1. Повышение коэффициента использования частотного диапазона оптоволокна.

2. Отличная масштабируемость — достаточно просто добавить новые спектральные каналы.

3. Экономическая эффективность — не требуется электрическая регенерация на длинных маршрутах.

4. Независимость от протокола передачи — через магистраль *DWDM* можно передавать трафик сетей любого типа.

5. Независимость спектральных каналов друг от друга.

6. Совместимость с технологией *SDH*. Мультиплексоры *DWDM* оснащаются интерфейсами *STM-N*, способными принимать и передавать данные мультиплексоров *SDH*.

7. Совместимость с технологиями *Ethernet* — *Gigabit Ethernet* и 10*GE*.

8. Стандартизация на уровне Международного союза по телекоммуникациям *ITU-T*.

2.3. Обработка информации

Обработка информации состоит в получении одних «информационных объектов» из других «информационных объектов» путем выполнения некоторых алгоритмов и является одной из основных операций, выполняемых над информацией и главным средством увеличения ее объема и разнообразия.

На самом верхнем уровне можно выделить числовую и нечисловую обработку. В указанные виды обработки вкладывается различная трактовка содержания понятия «данные». При числовой обработке используются такие объекты, как переменные, векторы, матрицы, многомерные массивы, константы и т. д. При нечисловой обработке объектами могут быть файлы, записи, поля, иерархии, сети, отношения и т. д. Другое отличие заключается в том, что при числовой обработке содержание данных не имеет большого значения, в то время как при нечисловой обработке нас интересуют непосредственные сведения об объектах, а не их совокупность в целом.

2.3.1. Режимы обработки данных

При проектировании технологических процессов ориентируются на режимы их реализации. Режим реализации технологии зависит от объемно-временных особенностей решаемых задач: периодичности и срочности, требований к быстроте обработки сообщений, а также от режимных возможностей технических средств, и в первую очередь ЭВМ. Существуют: пакетный режим; режим реального масштаба времени; режим разделения времени; регламентный режим; запросный; диалоговый; телеобработки; интерактивный; однопрограммный; многопрограммный (мультиобработка).

Пакетный режим. При использовании этого режима пользователь не имеет непосредственного общения с ЭВМ. Сбор и регистра-

ция информации, ввод и обработка не совпадают по времени. Вначале пользователь собирает информацию, формируя ее в пакеты в соответствии с видом задач или каким-то другим признаком. (Как правило, это задачи неоперативного характера с долговременным сроком действия результатов решения.) После завершения приема информации производится ее ввод и обработка, т. е. происходит задержка обработки. Этот режим используется, как правило, при централизованном способе обработки информации.

Диалоговый режим (запросный) ***режим***, при котором существует возможность пользователя непосредственно взаимодействовать с вычислительной системой в процессе работы пользователя. Программы обработки данных находятся в памяти ЭВМ постоянно, если ЭВМ доступна в любое время или в течение определенного промежутка времени, когда ЭВМ доступна пользователю. Взаимодействие пользователя с вычислительной системой в виде диалога может быть многоаспектным и определяться различными факторами: языком общения, активной или пассивной ролью пользователя; кто является инициатором диалога — пользователь или ЭВМ; временем ответа; структурой диалога и т. д. Если инициатором диалога является пользователь, то он должен обладать знаниями по работе с процедурами, форматами данных и т. п. Если инициатор — ЭВМ, то машина сама сообщает на каждом шаге, что нужно делать с разнообразными возможностями выбора. Этот метод работы называется «выбором меню». Он обеспечивает поддержку действий пользователя и предписывает их последовательность. При этом от пользователя требуется меньшая подготовленность.

Диалоговый режим требует определенного уровня технической оснащенности пользователя, т. е. наличие терминала или ПЭВМ (персональных электронных вычислительных машин), связанных с центральной вычислительной системой каналами связи. Этот режим используется для доступа к информации, вычислительным или программным ресурсам. Возможность работы в диалоговом режиме может быть ограничена во времени начала и конца работы, а может быть и неограниченной.

Иногда различают диалоговый и запросный режимы, тогда под запросным понимается одноразовое обращение к системе, после которого она выдает ответ и отключается, а под диалоговым — режим, при котором система после запроса выдает ответ и ждет дальнейших действий пользователя.

Режим реального масштаба времени. Означает способность вычислительной системы взаимодействовать с контролируемыми или управляемыми процессами в темпе протекания этих процессов. Время реакции ЭВМ должно удовлетворять темпу контролируемого процесса или требованиям пользователей и иметь минимальную задержку. Как правило, этот режим используется при децентрализованной и распределенной обработке данных.

Режим телеобработки дает возможность удаленному пользователю взаимодействовать с вычислительной системой.

Интерактивный режим предполагает возможность двустороннего взаимодействия пользователя с системой, т. е. у пользователя есть возможность воздействия на процесс обработки данных.

Режим разделения времени предполагает способность системы выделять свои ресурсы группе пользователей поочередно. Вычислительная система настолько быстро обслуживает каждого пользователя, что создается впечатление одновременной работы нескольких пользователей. Такая возможность достигается за счет соответствующего программного обеспечения.

Однопрограммный и многопрограммный режимы характеризуют возможность системы работать одновременно по одной или нескольким программам.

Регламентный режим характеризуется определенностью во времени отдельных задач пользователя. Например, получение результатных сводок по окончании месяца, расчет ведомостей начисления зарплаты к определенным датам и т. д. Сроки решения устанавливаются заранее по регламенту в противоположность к произвольным запросам.

2.3.2. Способы обработки данных

Различаются следующие способы обработки данных: централизованный, децентрализованный, распределенный и интегрированный.

Централизованная обработка. При этом способе пользователь доставляет на ВЦ (вычислительный центр) исходную информацию и получают результаты обработки в виде результативных документов. Особенностью такого способа обработки являются сложность и трудоемкость налаживания быстрой, бесперебойной связи, большая загруженность ВЦ информацией (так как велик ее объем), регламентацией сроков выполнения операций, организация безопасности системы от возможного несанкционированного доступа.

Децентрализованная обработка. Этот способ связан с появлением ПЭВМ, дающих возможность автоматизировать конкретное рабочие место.

Распределенный способ обработки данных основан на распределении функций обработки между различными ЭВМ, включенными в сеть. Этот способ может быть реализован двумя путями: *первый* предполагает установку ЭВМ в каждом узле сети (или на каждом уровне системы), при этом обработка данных осуществляется одной или несколькими ЭВМ в зависимости от реальных возможностей системы и ее потребностей на текущий момент времени. *Второй путь* — размещение большого числа различных процессоров внутри одной системы. Такой путь применяется в системах обработки

банковской и финансовой информации, там, где необходима сеть обработки данных (филиалы, отделения и т. д.). Преимущества распределенного способа: возможность обрабатывать в заданные сроки любой объем данных; высокая степень надежности, так как при отказе одного технического средства есть возможность моментальной замены его на другой; сокращение времени и затрат на передачу данных; повышение гибкости систем, упрощение разработки и эксплуатации программного обеспечения и т. д. Распределенный способ основывается на комплексе специализированных процессоров, т. е. каждая ЭВМ предназначена для решения определенных задач, или задач своего уровня.

Интегрированный способ обработки информации. Он предусматривает создание информационной модели управляемого объекта, т. е. создание распределенной базы данных. Такой способ обеспечивает максимальное удобство для пользователя. С одной стороны, базы данных предусматривают коллективное пользование и централизованное управление. С другой стороны, объем информации, разнообразие решаемых задач требуют распределения базы данных. Технология интегрированной обработки информации позволяет улучшить качество, достоверность и скорость обработки, так как обработка производится на основе единого информационного массива, однократно введенного в ЭВМ. Особенностью этого способа является отделение технологически и по времени процедуры обработки от процедур сбора, подготовки и ввода данных.

С точки зрения реализации, на основе современных достижений вычислительной техники выделяют следующие **виды обработки информации**:

- последовательная обработка, применяемая в традиционной фоннеймановской архитектуре ЭВМ, располагающей одним процессором;
- параллельная обработка, характеризующаяся наличием нескольких процессоров в ЭВМ;
- конвейерная обработка, связанная с использованием в архитектуре ЭВМ одних и тех же ресурсов для решения разных задач, причем если эти задачи тождественны, то это последовательный конвейер, если задачи одинаковые — векторный конвейер.

Принято относить существующие архитектуры ЭВМ с точки зрения обработки информации к одному из следующих классов [28].

Архитектуры с одиночным потоком команд и одиночным потоком данных (*Single Instruction, Single Data, SISD*). К этому классу относятся традиционные фоннеймановские однопроцессорные системы, где имеется центральный процессор, работающий с парами «атрибут — значение».

Архитектуры с одиночным потоком команд и множественным потоком данных (*Single Instruction, Multiple Data, SIMD*). Особенно-

стью данного класса является наличие одного (центрального) контроллера, управляющего рядом одинаковых процессоров. В зависимости от возможностей контроллера и процессорных элементов, числа процессоров, организации режима поиска и характеристик маршрутных и выравнивающих сетей выделяют:

- матричные процессоры, используемые для решения векторных и матричных задач;
- ассоциативные процессоры, применяемые для решения нечисловых задач и использующие память, в которой можно обращаться непосредственно к информации, хранящейся в ней;
- процессорные ансамбли, применяемые для числовой и нечисловой обработки;
- конвейерные и векторные процессоры.

Архитектуры с множественным потоком команд и одиночным потоком данных (далее — *MISD — Multiple Instruction stream, Single Data stream*). К этому классу могут быть отнесены конвейерные процессоры.

Архитектуры с множественным потоком команд и множественным потоком данных (далее — *MIMD — Multiple Instructions, Multiple Data*). К этому классу могут быть отнесены следующие конфигурации: мультипроцессорные системы, системы с мультобработкой, вычислительные системы из многих машин, вычислительные сети.

Основные процедуры обработки данных представлены на рис. 2.9 [8].

Рис. 2.9. **Основные процедуры обработки данных**

Создание данных как процесс обработки предусматривает их образование в результате выполнения некоторого алгоритма и дальнейшее использование для преобразований на более высоком уровне.

Модификация данных связана с отображением изменений в реальной предметной области, осуществляемых путем включения новых данных и удалением ненужных.

Контроль, безопасность и целостность направлены на адекватное отображение реального состояния предметной области в информационной модели и обеспечивают защиту информации от несанкционированного доступа (безопасность) и от сбоев и повреждений технических и программных средств.

Поиск информации, хранимой в памяти компьютера, осуществляется как самостоятельное действие при выполнении ответов на различные запросы, так и вспомогательная операция при обработке информации.

Поддержка принятия решения является наиболее важным действием, выполняемым при обработке информации. Широкая альтернатива принимаемых решений приводит к необходимости использования разнообразных математических моделей [10, 11].

Создание документов, сводок, отчетов заключается в преобразовании информации в формы, пригодные для чтения как человеком, так и компьютером. С этим действием связаны и такие операции, как обработка, считывание, сканирование и сортировка документов.

Преобразование информации осуществляет ее перевод из одной формы представления или существования в другую и определяется потребностями, возникающими в процессе реализации информационных технологий.

Реализация всех действий, выполняемых в процессе обработки информации, осуществляется с помощью разнообразных программных средств.

Наиболее распространенной областью применения технологической операции обработки информации является принятие решений.

В зависимости от степени информированности о состоянии управляемого процесса, полноты и точности моделей объекта и системы управления, взаимодействия с окружающей средой, процесс принятия решения протекает в следующих различных условиях.

1. **Принятие решений в условиях определенности.** В этой задаче модели объекта и системы управления считаются заданными, а влияние внешней среды — несущественным. Поэтому между выбранной стратегией использования ресурсов и конечным результатом существует однозначная связь, откуда следует, что в условиях определенности достаточно использовать решающее правило для оценки полезности вариантов решений, принимая в качестве оптимального то, которое приводит к наибольшему эффекту. Если таких стратегий несколько, то все они считаются эквивалентными. Для поиска решений в условиях определенности используют методы математического программирования.

2. **Принятие решений в условиях риска.** В отличие от предыдущего случая, для принятия решений в условиях риска необходимо учитывать влияние внешней среды, которое не поддается точному прогнозу, а известно только вероятностное распределение ее состояний. В этих условиях использование одной и той же стратегии может привести к различным исходам, вероятности появления которых считаются заданными или могут быть определены. Оценка и выбор стратегий проводят с помощью решающего правила, учитывающего вероятность достижения конечного результата.

3. **Принятие решений в условиях неопределенности.** Как и в предыдущей задаче, между выбором стратегии и конечным результатом отсутствует однозначная связь. Кроме того, неизвестны также значения вероятностей появления конечных результатов, которые либо не могут быть определены, либо не имеют в контексте содержательного смысла. Каждой паре «стратегия — конечный результат» ставится в соответствие некоторая внешняя оценка в виде выигрыша. Наиболее распространенным является использование критерия получения максимального гарантированного выигрыша.

4. **Принятие решений в условиях многокритериальности.** В любой из перечисленных выше задач многокритериальность возникает в случае наличия нескольких самостоятельных, не сводимых одна к другой целей. Наличие большого количества решений усложняет оценку и выбор оптимальной стратегии. Одним из возможных путей решения является использование методов моделирования.

Решение задач с помощью искусственного интеллекта (см. параграф 3.6) заключается в сокращении перебора вариантов при поиске решения, при этом программы реализуют те же принципы, которыми пользуется в процессе мышления человек.

Процесс выработки решения на основе первичных данных, схема которого представлена на рис. 2.10, можно разбить на два этапа: выработка допустимых вариантов решений путем математической формализации с использованием разнообразных моделей и выбор оптимального решения на основе субъективных факторов.

Информационные потребности лиц, принимающих решение, во многих случаях ориентированы на интегральные технико-экономические показатели, которые могут быть получены в результате обработки первичных данных, отражающих текущую деятельность предприятия. Анализируя функциональные взаимосвязи между итоговыми и первичными данными, можно построить так называемую информационную схему, которая отражает процессы агрегирования информации. Первичные данные, как правило, чрезвычайно разнообразны, интенсивность их поступления высока, а общий объем на интересующем интервале велик. Однако состав интегральных показателей относительно мал, а требуемый период их актуализа-

ции может быть значительно ниже периода изменения первичных данных — аргументов.

Рис. 2.10. **Процесс выработки решения на основе первичных данных**

Для поддержки принятия решений обязательным является наличие следующих компонент:

- обобщающий анализ;
- прогнозирование;
- ситуационное моделирование.

Система поддержки принятия решений (далее — СППР) может быть представлена как автоматизированная интерактивная человеко-машинная система поддержки принятия решений на основе использования данных и моделей. СППР предназначены для поддержки управленческих решений и интуитивного подхода к решению управленческих задач. Для этого они обеспечивают пользователей необходимой информацией, генерируют, оценивают и предлагают несколько вариантов. Системы поддержки принятия решений входят в состав практически любой современной информационной системы. Понятие информационной системы (далее — ИС) и понятие

СППР являются взаимодополняющими. В соответствии с характером обработки информации в ИС на различных уровнях управления заданной системой (оперативном, тактическом и стратегическом) выделяются следующие типы информационных подсистем:

- системы обработки данных (*Electronic Data Processing, EDP*);
- информационная система управления (*Management Information System, MIS*);
- система поддержки принятия решений (*Decision Support System, DSS*).

Для задач СППР свойственны недостаточность имеющейся информации, ее противоречивость и нечеткость, преобладание качественных оценок целей и ограничений, слабая формализованность алгоритмов решения. В качестве инструментов обобщения чаще всего используются средства составления аналитических отчетов произвольной формы, методы статистического анализа, экспертных оценок и систем, математического и имитационного моделирования.

Таким образом, СППР — это автоматизированная система, используемая для различных видов деятельности при принятии решений в ситуациях, где невозможно или нежелательно иметь автоматическую систему, полностью выполняющую весь процесс решения, вследствие слабой структурированности или не структурированности решаемых проблем.

Такая СППР может состоять из нескольких подсистем, реализующих следующие семь основных функций СППР:

1) оценка обстановки (ситуации), выбор критериев и оценка их относительной важности;
2) генерация возможных решений (сценариев действий);
3) оценка решений (сценариев действий) и выбор лучшего;
4) обмен информацией об обстановке принимаемых решений и согласование групповых решений (в тех случаях, когда это возможно);
5) моделирование принимаемых решений (в тех случаях, когда это возможно);
6) динамический компьютерный анализ возможных последствий принимаемых решений;
7) сбор данных о результатах реализации принятых решений и оценка результатов.

СППР зачастую используются как интегрированные подсистемы в составе сложных комплексов управления и контроля. Для подобных интегрированных СППР можно выделить следующие основные функции:

- обеспечение ЛПР (лицо, принимающее решения) информацией для процесса принятия решения, включая ее предварительную обработку;
- организационно-методическое обеспечение процесса принятия решений;

- моделирование последствий принятия решений;
- экспертные функции: выдача рекомендаций и обоснований;
- обеспечение согласованности решений, принимаемых в группах.

Модель процесса принятия решений человеком включает три основные ступени: интеллектуальную ступень, конструирование и выбор. Термин «поддержка» подразумевает различные шаги и задачи на каждом этапе процесса принятия решений.

СППР в зависимости от данных, с которыми они работают, можно разделить:

- на оперативные, предназначенные для немедленного реагирования на текущую ситуацию;
- стратегические, основанные на анализе большого количества информации из разных источников с привлечением сведений, содержащихся в системах, аккумулирующих опыт решения проблем.

Оперативные СППР получили название *информационных систем руководства* (*Executive Information Systems*) (далее — ИСР). По сути, они представляют собой конечные наборы отчетов, построенные на основании данных из информационной системы предприятия или *OLTP*-системы. Для ИСР характерны следующие основные черты:

- отчеты, как правило, базируются на стандартных для организации запросах; число последних относительно невелико;
- ИСР представляет отчеты в максимально удобном виде, включающем наряду с таблицами деловую графику, мультимедийные возможности и т. п.;
- как правило, ИСР ориентированы на конкретный вертикальный рынок, например финансы, маркетинг, управление ресурсами.

Стратегические СППР предполагают достаточно глубокую проработку данных, специально преобразованных так, чтобы их было удобно использовать в ходе процесса принятия решений. Неотъемлемым компонентом СППР этого уровня являются правила принятия решений, которые на основе агрегированных данных подсказывают менеджерскому составу выводы и придают системе черты искусственного интеллекта. Такого рода системы создаются только в том случае, если структура бизнеса уже достаточно определена и имеются основания для обобщения и анализа не только данных, но и процессов их обработки.

Большая часть используемых сегодня СППР разработана для генерации и оценки альтернатив посредством анализа «что-если» и «поиска цели» на этапах конструирования и выбора. *Финансовые* модели служат для поддержки и планирования, рассчитывая последствия запланированных действий на основе оценки прибылей. *Репрезентативные* модели оценивают последствия действий (принятия решений) на основе комплекса моделей включая все имитационные модели. *Оптимизирующие* модели находят оптимальные

решения. *Предлагающие* модели дают специальное решение для четко структурированных задач. Такие системы выполняют механические вычисления и не оставляют места управленческому суждению.

Управление данными и моделями в СППР практически неразделимо, поэтому многие исследователи фокусируют внимание на обеих сторонах вопроса — на управлении и данными, и моделями. Данные представляют собой фактические результаты наблюдения за физическими явлениями, например, это могут быть размеры суточного выпуска продукции, объем дневных продаж и уровень запасов продукта А. База данных содержит совокупность взаимосвязанных файлов.

Управление данными в СППР — это необходимая функция, используемая, главным образом, на интеллектуальной стадии процесса принятия решений, но недостаточная для поддержки этапов конструирования и выбора альтернатив. Для поддержки этих этапов СППР должны обеспечить выполнение следующих операций: проекцию, дедукцию, анализ, генерацию альтернатив, сравнение альтернатив, оптимизацию и имитацию. При выполнении этих задач СППР используют различные типы моделей из областей знаний об управлении и исследовании операций. Эти модели включают линейное программирование, целочисленное программирование, сетевые модели, программирование целей, имитационные и статистические модели и электронные таблицы.

Другой важной, развивающейся в последнее время разновидностью СППР являются *системы поддержки принятия решений, основанные на базе знаний* (далее — СППРБЗ), которые представляют собой гибрид СППР и ЭС (экспертная система) и предназначены для решения широкого круга организационных задач. В интегрировании СППР и ЭС выделяются два основных подхода: *экспертные системы поддержки* (далее — ЭСП) и *интеллектуальные системы поддержки* (далее — ИСП). Основные различия между этими системами заключается в следующем — ЭСП предназначены для замены живого эксперта машинным экспертом, а задачи ИСП заключаются в поддержке знаний отдельных пользователей и групп. Широкий ряд управленческих задач реального мира легче поддается решению, если используются как количественные, так и качественные данные. Новая интегрированная система (ЭСП или ИСП) может помогать, используя при этом знания и опыт ключевых фигур в организации. Узким местом при разработке систем, основанных на знаниях, таких как ЭСП например, является задача приобретения знаний; этот процесс состоит из представления знаний, проверки, механизма построения логических выводов, механизмов объяснения и управления.

Множество новых средств и технологий способно расширить возможности СППР, среди них важное место занимает интеллек-

туальный анализ данных (далее — ИАД), обычно определяемый как метод поддержки принятия решений, основанный на анализе зависимостей между данными [23]. Существует два подхода автоматизации поиска зависимостей между данными. В первом случае, используемом в традиционные технологии анализа, пользователь сам выдвигает гипотезы относительно зависимостей между данными. Гипотеза приводит к построению отчета, анализ отчета — к выдвижению новой гипотезы и т. д. Описанная методика действует и в том случае, когда пользователь применяет такие развитые средства, как *OLAP* (*On-line Analytical Processing*), поскольку процесс поиска по-прежнему полностью контролируется человеком. Во многих системах ИАД в этом процессе автоматизирована проверка достоверности гипотез, что позволяет оценить вероятность тех или иных зависимостей в базе данных. Второй подход основан на автоматическом поиске зависимости между данными. Растущее количество продуктов, реализующих автоматический поиск зависимостей, говорит об интересе производителей и потребителей к системам именно такого типа.

Процессы ИАД подразделяются на три большие группы: поиск зависимостей (*discovery*), прогнозирование (*predictive modelling*) и анализ аномалий (*forensic analysis*). В первом случае при просмотре базы данных осуществляется автоматическое выявление зависимостей. Проблемой здесь является отбор действительно важных зависимостей из огромного числа существующих в базах данных (далее — БД). Прогнозирование основано на том, что пользователь предъявляет системе записи с незаполненными полями и запрашивает недостающие значения. Система сама анализирует содержимое базы и делает правдоподобное предсказание относительно этих значений. Анализ аномалий заключается в процессе поиска подозрительных данных, сильно отклоняющихся от устойчивых зависимостей.

Необходимость оперативной обработки информации стремительно возрастающих объемов информации вызвало к жизни появление и активное развитие технологии интеллектуального анализа данных (ИАД, или *data mining*). Теоретической базой ИАД являются методов математической статистики и искусственного интеллекта. Сложность решаемых задач потребовала создания специализированных инструментальных средств, ориентированных на конечного пользователя и предназначенных для решения как типовых, так и специфических задач в различных предметных областях.

Области применения ИАД: научные исследования, образование, статистика, здравоохранение, производство, финансы, правоохранительная и военная деятельность.

Разнообразие решаемых задач вынуждают разработчиков создавать исследовательские ИАД, предназначенные для работы с новыми типами проблем, и прикладные ИАД, ориентированные на решение

типовых задач. Каждый из классов систем может быть ориентирован как на специалистов, так и непрограммирующих пользователей.

К сожалению, универсальные средства ИАД довольно сложны и дороги, поэтому они не могут широко применяться в рамках интегрированных систем, ориентированных на конечного пользователя. В основу технологии ИАД положен не один, а несколько принципиально различных подходов (табл. 2.1), причем использование некоторых из них невозможно без специальной подготовки [12]. Выбор подхода нередко требует привлечения специалиста по ИАД.

Таблица 2.1

Основные технологии интеллектуального анализа данных

Технология	Область применения	Недостатки
Правила вывода	Данные связаны отношениями, представимыми в виде правил «если — то»	Потеря наглядности при большом количестве правил, не всегда удается выделить отношения «если — то»
Нейронные сети	Работа с нелинейными зависимостями, зашумленными и неполными данными	Модель типа «черный ящик» не позволяет объяснить выявленные знания, при этом данные обязательно должны быть преобразованы к числовому виду
Нечеткая логика	Ранжировка данных по степени близости к желаемым результатам; нечеткий поиск в базах данных	Ввиду новизны технологии в настоящее время известно ограниченное число специализированных приложений
Визуализация	Многомерное графическое представление данных, пользователю самому представляется возможность выявления закономерностей отношений между данными	Зависимость интерпретация модели от аналитика
Статистика	Научные и инженерные приложения, наличие большое числа алгоритмов и опыта их применения	Крен в сторону проверки гипотез, чем выявления новых закономерностей в данных
К — ближайший сосед	Выявление кластеров, обработка целостных источников данных	Требует больших объемов памяти, проблемы с чувствительностью
Интегрированные технологии	Выбор подходов, адекватных задачам, или их сравнение	Сложность средств поддержки; высокая стоимость. Для каждой отдельно взятой технологии не всегда реализуется наилучшее решение

Это наблюдение легло в основу подхода к поиску и выборке данных, называемого *OLAP*. Общеизвестно, что *OLAP*-системы построены на двух базовых принципах:

- все данные, необходимые для принятия решений, предварительно агрегированы на всех соответствующих уровнях и организованы так, чтобы обеспечить максимально быстрый доступ к ним;
- язык манипулирования данными основан на использовании бизнес-понятий.

Так как в основе *OLAP* лежит понятие гиперкуба, или многомерного куба данных, в ячейках которого хранятся анализируемые (числовые) данные, то дальнейшее усложнение модели данных может идти по нескольким направлениям:

- увеличение числа измерений — например, данные о статьях не только по месяцам и годам, но и по темам. В этом случае куб становится трехмерным;
- усложнение содержимого ячейки — в этом случае в ячейке будет несколько значений признаков;
- введение иерархии в пределах одного измерения — общее понятие «*время*» естественным образом связано с иерархией значений: год состоит из кварталов, квартал из месяцев и т. д.

Речь пока идет не о физической структуре хранения, а лишь о логической модели данных. Другими словами, определяется лишь пользовательский интерфейс модели данных. В рамках этого интерфейса должны быть введены следующие базовые операции:

- поворот;
- проекция (при проекции значения в ячейках, лежащих на оси проекции, суммируются по некоторому предопределенному закону);
- раскрытие (*drill-down*) — одно из значений измерения заменяется совокупностью значений из следующего уровня иерархии измерения; соответственно, заменяются значения в ячейках гиперкуба;
- свертка (*roll-up/drill-up*) — операция, обратная раскрытию;
- сечение (*slice-and-dice*).

В зависимости от ответа на вопрос, существует ли гиперкуб как отдельная физическая структура или лишь как виртуальная модель данных, различают системы *MOLAP* (*Multidimensional OLAP*) и *ROLAP* (*Relational OLAP*). В первых гиперкуб реализуется как отдельная база данных специальной нереляционной структуры, обеспечивающая максимально эффективный по скорости доступ к данным, но требующая дополнительного ресурса памяти. *MOLAP*-системы весьма чувствительны к объемам хранимых данных. Поэтому данные из хранилища сначала помещаются в специальную многомерную базу (*Multidimensional Data Base*, *MDB*), а затем эффективно обрабатываются *OLAP*-сервером.

Реализация процесса принятия решений заключается в построении информационных приложений. Выделим в информационном

приложении типовые функциональные компоненты, достаточные для формирования любого приложения на основе БД.

PS (*Presentation Services*) — средства представления. Обеспечиваются устройствами, принимающими ввод от пользователя и отображающими то, что сообщает ему компонент логики представления *PL*, плюс соответствующая программная поддержка. Может быть текстовым терминалом или *X*-терминалом, а также персональным компьютером или рабочей станцией в режиме программной эмуляции терминала или X-терминала.

PL (*Presentation Logic*) — логика представления. Управляет взаимодействием между пользователем и ЭВМ. Обрабатывает действия пользователя по выбору альтернативы меню, по нажатию кнопки или выборе элемента из списка.

BL (*Business or Application Logic*) — прикладная логика. Набор правил для принятия решений, вычислений и операций, которые должно выполнить приложение.

DL (*Data Logic*) — логика управления данными. Операции с базой данных (*SQL*-операторы *SELECT*, *UPDATE* и *INSERT*), которые нужно выполнить для реализации прикладной логики управления данными.

DS (*Data Services*) — операции с базой данных. Действия систем управления базами данных (далее — СУБД), вызываемые для выполнения логики управления данными, такие как манипулирование данными, определения данных, фиксация или откат транзакций и т. п. СУБД обычно компилирует *SQL*-приложения.

FS (*File Services*) — файловые операции. Дисковые операции чтения и записи данных для СУБД и других компонент. Обычно являются функциями ОС.

Среди средств разработки информационных приложений можно выделить следующие основные группы:

- традиционные системы программирования;
- инструменты для создания файл-серверных приложений;
- средства разработки приложений «клиент — сервер»;
- средства автоматизации делопроизводства и документооборота;
- средства разработки *Internet/Intranet*-приложений;
- средства автоматизации проектирования приложений.

2.4. Хранение информации

Хранение и накопление являются одними из основных действий, осуществляемых над информацией и главным средством обеспечения ее доступности в течение некоторого промежутка времени. В настоящее время определяющими направлениями реализации этой операции является концепция базы данных, склада (хранилища) данных [9, 13].

База данных может быть определена как совокупность взаимосвязанных данных, используемых несколькими пользователями

и хранящихся с регулируемой избыточностью. Хранимые данные не зависят от программ пользователей, для модификации и внесения изменений применяется общий управляющий метод.

Система баз данных — совокупность управляющей системы, прикладного программного обеспечения, базы данных, операционной системы и технических средств, обеспечивающих информационное обслуживание пользователей.

Хранилище данных (склад данных, информационное хранилище) (далее — ХД) (*Data Warehouse*) — это база данных, хранящая данные, агрегированные по многим измерениям. Основные отличия ХД от БД: агрегирование данных; данные из ХД никогда не удаляются; пополнение ХД происходит на периодической основе; автоматическое формирование новых агрегатов данных, зависящих от старых; доступ к ХД осуществляется на основе многомерного куба или гиперкуба.

Альтернативой хранилищу данных является концепция ***витрин данных*** (*Data Mart*). **Витрины данных** — множество тематических БД, содержащих информацию, относящуюся к отдельным информационным аспектам предметной области.

Еще одним важным направлением развития баз данных являются репозитарии. Репозитарий в упрощенном виде можно рассматривать просто как базу данных, предназначенную для хранения не пользовательских, а системных данных. Технология репозитариев проистекает от словарей данных, которые по мере обогащения новыми функциями и возможностями приобретали черты инструмента для управления метаданными.

Каждый из участников действия имеет свое представление об информации:

- пользователь;
- группа пользователей;
- «физическая память».

По отношению к пользователям принято использовать ***трехуровневое представление для описания предметной области***: концептуальное, логическое и внутреннее (физическое) (рис. 2.11).

Концептуальный уровень связан с частным представлением данных группы пользователей в виде внешней схемы, объединяемых общностью используемой информации. Каждый конкретный пользователь работает с частью БД и представляет ее в виде внешней модели. Характеризуется разнообразием используемых моделей (модель «сущность — связь» (*ER*-модель, модель Чена), бинарные и инфологические модели, семантические сети). На рис. 2.12 представлен фрагмент предметной базы данных «сбыт» и одно из возможных его концептуальных представлений, которое отражает не только объекты, и их свойства, но и взаимосвязи между ними.

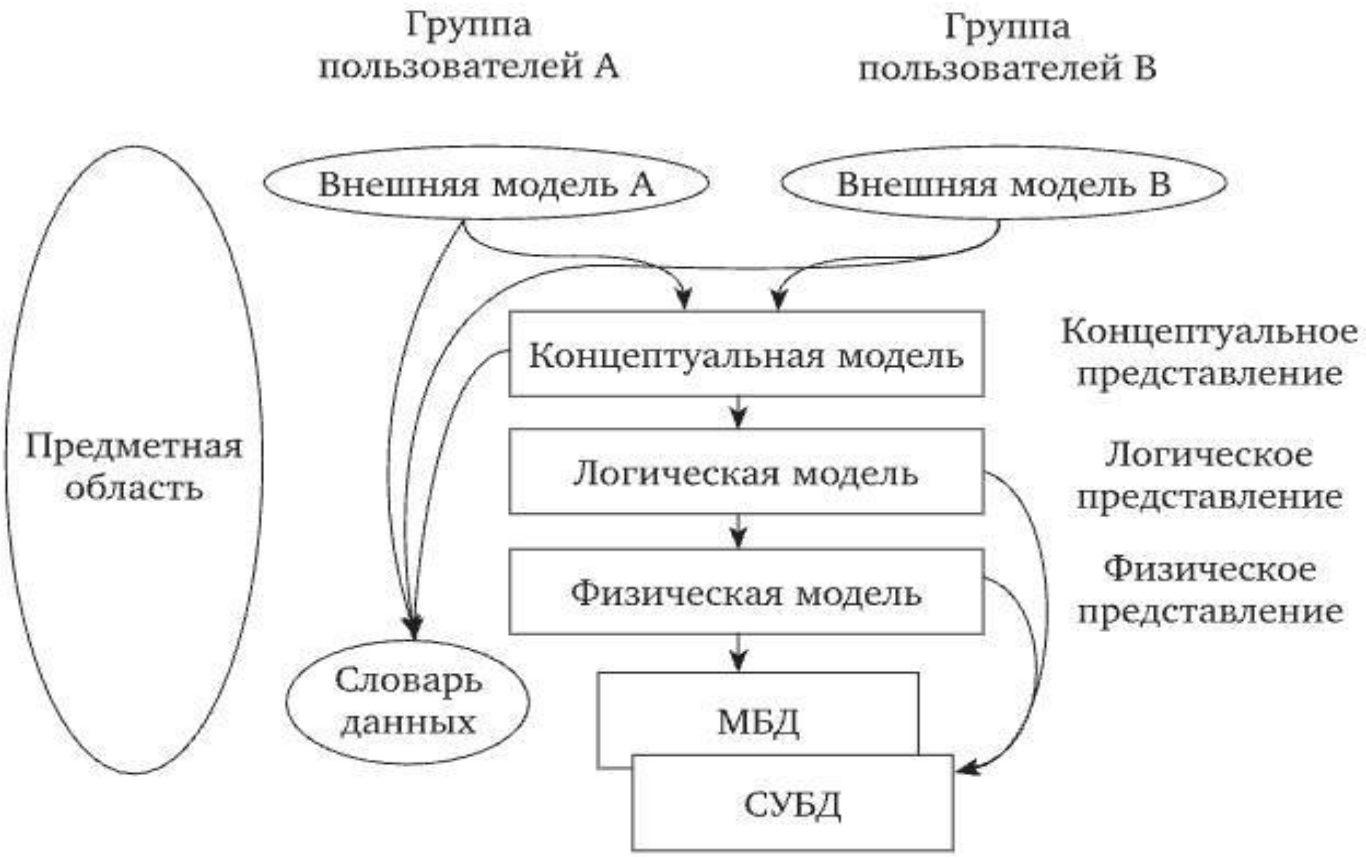


Рис. 2.11. **Описания предметной области**

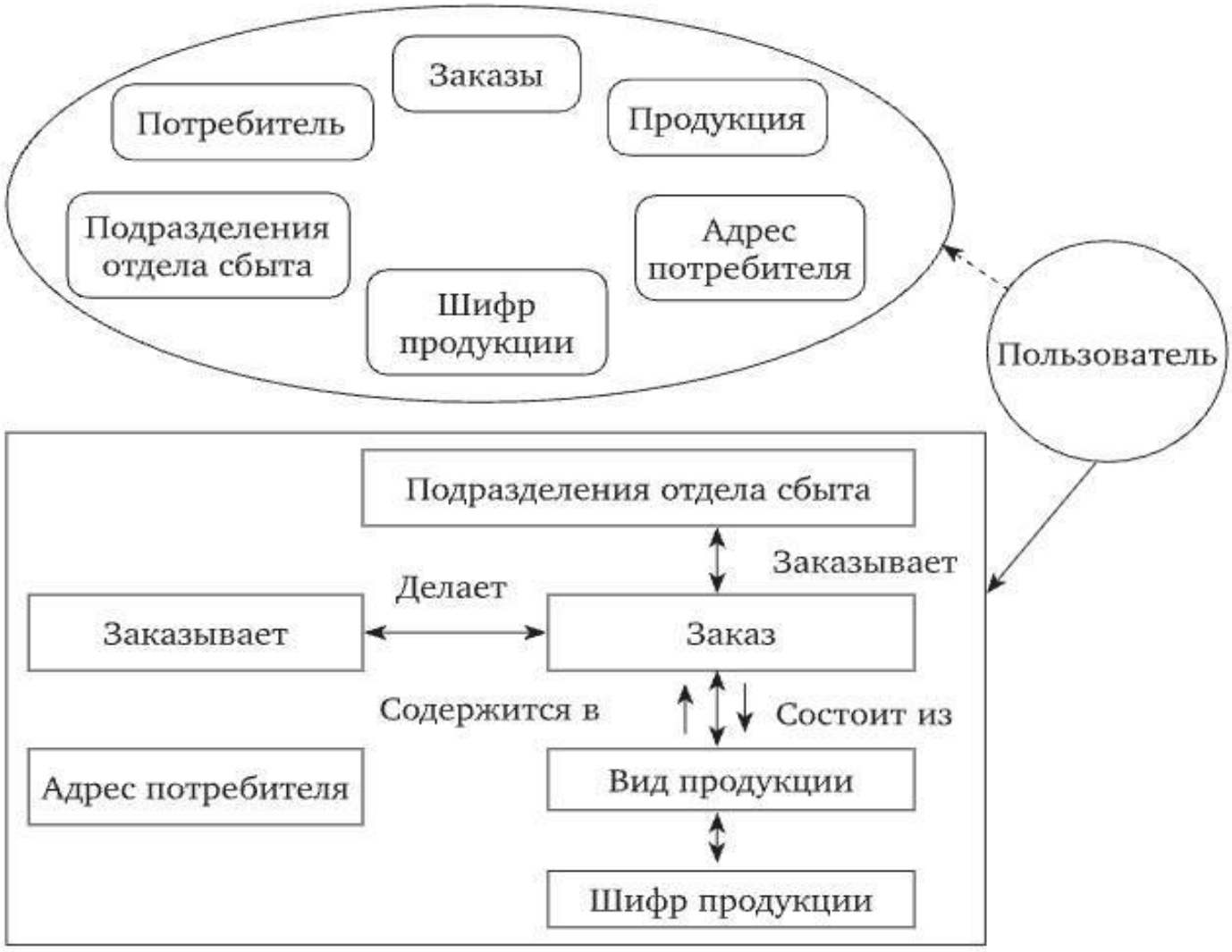


Рис. 2.12. **Фрагмент предметной базы данных «сбыт» и одно из возможных его концептуальных представлений**

Логический уровень является обобщенным представлением данных всех пользователей в абстрактной форме. Используется ***три классических вида моделей***: иерархические, сетевые и реляционные.

Сетевая модель является моделью объектов-связей, допускающей только бинарные связи «многие к одному» и использует для описания модель ориентированных графов.

Иерархическая модель является разновидностью сетевой, являющейся совокупностью деревьев (лесом).

Реляционная модель использует представление данных в виде таблиц (реляций), в основе лежит математическое понятие теоретико-множественного отношения, базируется на реляционной алгебре и теории отношений.

Представление предметной базы данных «сбыт» на логическом уровне для различных моделей представлено на рис. 2.13.

Дальнейшим развитием логического представления данных является переход к объектно-ориентированным моделям данных, что связано с процессом «перекачки» в них огромных объемов информации, которая в настоящее время хранится преимущественно в реляционных базах данных.

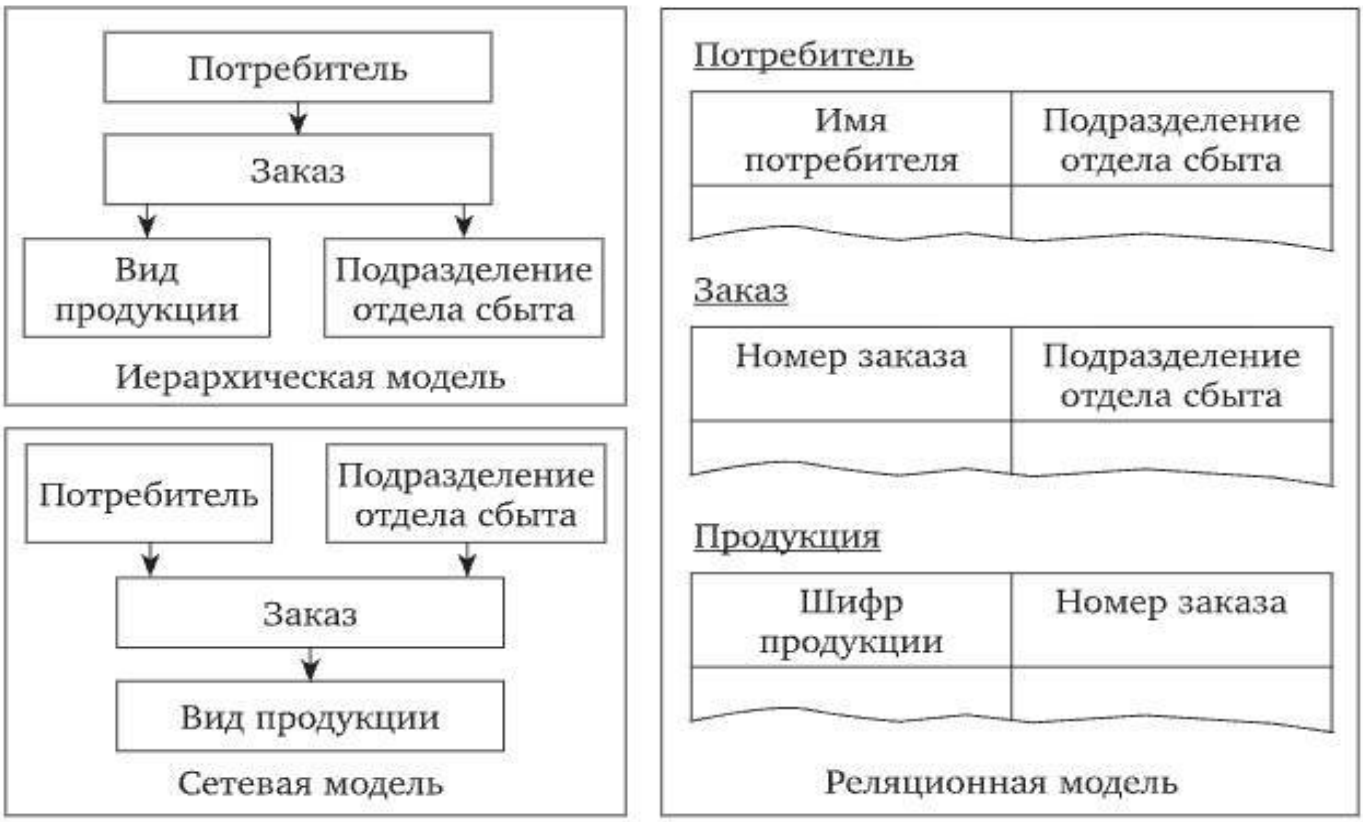


Рис. 2.13. **Представление предметной базы данных «сбыт» на логическом уровне для различных моделей**

Суть *объектно-ориентированной базы данных* (далее — ООБД) определяется объектно-ориентированным подходом.

Объектно-ориентированный подход подразумевает объектно-ориентированное проектирование и объектно-ориентированное программирование.

Объектно-ориентированное программирование — методология программирования, основанная на представлении программ в виде связанной совокупности объектов, каждый из которых является экземпляром определенного класса, а классы образуют иерархию по наследованию.

Объектно-ориентированное проектирование предполагает не только деление (декомпозицию) базы знаний или базы данных на составные части, но и рассмотрение общей этапности реализации БД, выбор инструмента реализации с учетом оговоренных в техническом задании вариантов реализации.

Свойства реляционных, сетевых, иерархических, объектно-иерархических и объектно-реляционных моделей данных (многомерные модели считаем разновидностью иерархических моделей данных) возможно систематизировать и сравнить, как это сделано в табл. 2.2.

Физический (***внутренний***) ***уровень*** связан со способом фактического хранения данных в физической памяти ЭВМ. Во многом определяется конкретным методом управления. Основными компонентами физического уровня являются хранимые записи, объединяемые в блоки; указатели, необходимые для поиска данных; данные переполнения; промежутки между блоками; служебная информация.

Таблица 2.2

Сравнительная характеристика моделей БД

Вид модели	Достоинства	Недостатки
Иерархическая	Простота понимания. Высокое быстродействие при совпадении структур базы данных и запроса	Отношения М : М могут быть реализованы только искусственно. Могут быть избыточные данные. Усложняются операции включения и удаления. Удаление исходных сегментов приводит к удалению порожденных сегментов. Процедурный характер построения структуры БД и манипулирования данными. Доступ к любому порожденному сегменту возможен только через корневой сегмент. Сильная зависимость логической и физической моделей. Ограниченный набор структур запроса. Невозможность реализации таблиц с нелинейной структурой

Окончание табл. 2.2

Вид модели	Достоинства	Недостатки
Сетевая	Сохранение информации при уничтожении записи-владельца. Более богатая структура запросов Меньшая зависимость логической и физической моделей. Возможность реализации таблиц с нелинейной структурой	Отношения М : М могут быть реализованы только искусственно. Необходимость программисту знать логическую структуру БД. Процедурный характер построения структуры БД и манипулирования данными. Возможная потеря независимости данных при реорганизации БД
Реляционная	Произвольная структура запроса. Простота работы и отражения представлений пользователя. Отделение физической модели от логической и логической от концептуальной. Хорошая теоретическая проработка	Отношения М : М могут быть реализованы только искусственно. Необходимость нормализации данных. Возможность логических ошибок при нормализации и реализации. Невозможность реализации таблиц с нелинейной структурой
Объектно-ориентированная	Неограниченный набор типов данных. Возможность реализации таблиц с нелинейной структурой.	Сложность освоения модели из-за сложности структуры БД. Нечеткий язык программирования.
	Послойное представление данных. Высокая скорость работы из-за отсутствия ключа. Ненужность нормализации. Легкая расширяемость структуры и ее гибкость. Повторное использование типов данных и компонент. Реализация отношений М : М	Недостаточная защита данных. Нечетко проработанный одновременный доступ. Плохая обозримость структуры

По наиболее характерным признакам БД можно классифицировать следующим образом.

По способу хранения информации:

- интегрированные;
- распределенные.

По типу пользователя:

- монопользовательские;
- многопользовательские.

По характеру использования данных:

- прикладные;
- предметные.

В настоящее время при проектировании БД используют два подхода. Первый из них основан на стабильности данных, что обеспечивает наибольшую гибкость и адаптируемость к используемым приложениям. Применение такого подхода целесообразно в тех случаях, когда не предъявляются жесткие требования к эффективности функционирования (объем памяти и время поиска), существует большое количество разнообразных задач с изменяемыми и непредсказуемыми запросами.

Другой подход базируется на стабильности процедур запросов к БД и является предпочтительным при жестких требованиях к эффективности функционирования, особенно это касается быстродействия.

Другим важным аспектом проектирования БД является проблема интеграции и распределения данных. Господствовавшая до недавнего времени концепция интеграции данных при резком увеличении их объема оказалась несостоятельной. Этот факт, а также увеличение объемов памяти внешних запоминающих устройств при ее удешевлении, широкое внедрение сетей передачи данных способствовало внедрению распределенных БД. Распределение данных по месту их использования может осуществляться различными способами.

1. *Копируемые данные*. Одинаковые копии данных хранятся в различных местах использования, так как это дешевле передачи данных. Модификация данных контролируется централизованно.
2. *Подмножество данных*. Группы данных, совместимые с исходной базой данных, хранятся отдельно для местной обработки.
3. *Реорганизованные данные*. Данные в системе интегрируются при передаче на более высокий уровень.
4. *Секционированные данные*. На различных объектах используются одинаковые структуры, но хранятся разные данные.
5. *Данные с отдельной подсхемой*. На различных объектах используются различные структуры данных, объединяемые в интегрированную систему.
6. *Несовместимые данные*. Независимые базы данных, спроектированные без координации, требующие объединения.

Важное влияние на процесс создания БД оказывает внутреннее содержание информации. Существует два направления:

- прикладные БД, ориентированные на конкретные приложения, например, может быть создана БД для учета и контроля поступления материалов;

- предметные БД, ориентированные на конкретный класс данных, например, предметная БД «Материалы», которая может быть использована для различных приложений.

Для работы с БД используется специальный обобщенный инструментарий в виде СУБД (МБД), предназначенный для управления БД и обеспечения интерфейса пользователя.

Управление всеми процессами в БД, в том числе обеспечение интерфейса пользователя, осуществляет специальный инструментарий в виде системы управления базой данных (СУБД) или машины базы данных (МБД).

Основные стандарты СУБД:

- независимость данных на концептуальном, логическом, физическом уровнях;
- универсальность (по отношению к концептуальному и логическому уровню, типу ЭВМ);
- совместимость, неизбыточность;
- безопасность и целостность данных;
- актуальность и управляемость.

Существует два основных направления реализации СУБД: программное и аппаратное.

Программная реализация (в дальнейшем — СУБД) представляет собой набор программных модулей, работает под управлением конкретной ОС и выполняет следующие функции:

- описание данных на концептуальном и логическом уровнях;
- загрузку данных;
- хранение данных;
- поиск и ответ на запрос (транзакцию);
- внесение изменений;
- обеспечение безопасности и целостности.

Обеспечивает пользователя следующими языковыми средствами:

- язык описания данных (далее — ЯОД);
- язык манипулирования данными (далее — ЯМД);
- прикладной (встроенный) язык данных (далее — ПЯД, ВЯД).

Аппаратная реализация предусматривает использование так называемых *машин баз данных* (далее — МБД). Их появление вызвано возросшими объемами информации и требованиями к скорости доступа. Слово «машина» в термине МБД означает вспомогательный периферийный процессор. Термин «компьютер БД» — автономный процессор баз данных или процессор, поддерживающий СУБД. Основные направления МБД:

- параллельная обработка;
- распределенная логика;
- ассоциативные ЗУ (запоминающие устройства);
- конвейерные ЗУ;
- фильтры данных и др.

На рис. 2.14 представлена совокупность процедур проектирования БД, которые можно объединить в четыре этапа.

Рис. 2.14. **Совокупность процедур проектирования БД**

На этапе формулирования и анализа требований устанавливаются цели организации, определяются требования к БД. Эти требования документируются в форме доступной конечному пользователю и проектировщику БД. Обычно при этом используется методика интервьюирования персонала различных уровней управления.

Этап концептуального проектирования заключается в описании и синтезе информационных требований пользователей в первоначальный проект БД. Результатом этого этапа является высокоуров-

невое представление информационных требований пользователей на основе различных подходов.

В процессе логического проектирования высокоуровневое представление данных преобразуется в структуре используемой СУБД. Полученная логическая структура БД может быть оценена количественно с помощью различных характеристик (число обращений к логическим записям, объем данных в каждом приложении, общий объем данных и т. д.). На основе этих оценок логическая структура может быть усовершенствована с целью достижения большей эффективности.

На этапе физического проектирования решаются вопросы, связанные с производительностью системы, определяются структуры хранения данных и методы доступа.

Весь процесс проектирования БД является итеративным, при этом каждый этап рассматривается как совокупность итеративных процедур, в результате выполнения которых получают соответствующую модель.

Взаимодействие между этапами проектирования и словарной системой необходимо рассматривать отдельно. Процедуры проектирования могут использоваться независимо в случае отсутствия словарной системы. Сама словарная система может рассматриваться как элемент автоматизации проектирования.

Этап расчленения БД связан с разбиением ее на разделы и синтезом различных приложений на основе модели. Основными факторами, определяющими методику расчленения, помимо указанных на рис. 2.14, являются: размер каждого раздела (допустимые размеры); модели и частоты использования приложений; структурная совместимость; факторы производительности БД. Связь между разделом БД и приложениями характеризуется идентификатором типа приложения, идентификатором узла сети, частотой использования приложения и его моделью.

Модели приложений могут быть классифицированы следующим образом:

1) приложения, использующие единственный файл;
2) приложения, использующие несколько файлов, в том числе:
   - допускающие независимую параллельную обработку,
   - допускающие синхронизированную обработку.

Сложность реализации этапа размещения БД определяется многовариантностью. Поэтому на практике рекомендуется в первую очередь рассмотреть возможность использования определенных допущений, упрощающих функции СУБД, например, допустимость временного рассогласования БД, осуществление процедуры обновления БД из одного узла и др. Такие допущения оказывают большое влияние на выбор СУБД и рассматриваемую фазу проектирования.

Средства проектирования и оценочные критерии используются на всех стадиях разработки. Любой метод проектирования (ана-

литический, эвристический, процедурный), реализованный в виде программы, становится инструментальным средством проектирования, практически не подверженным влиянию стиля проектирования.

В настоящее время неопределенность при выборе критериев являются наиболее слабым местом в проектировании БД. Это связано с трудностью описания и идентификации бесконечного числа альтернативных решений. При этом следует иметь в виду, что существует много признаков оптимальности, являющихся неизмеримыми свойствами, которые трудно выразить в количественном представлении или в виде целевой функции. Поэтому оценочные критерии принято делить на количественные и качественные. Наиболее часто используемые ***критерии оценки БД***, сгруппированные в такие категории, представлены ниже.

Количественные критерии: время ответа на запрос, стоимость модификации, стоимость памяти, время на создание, стоимость на реорганизацию.

Качественные критерии: гибкость, адаптивность, доступность для новых пользователей, совместимость с другими системами, возможность конвертирования в другую вычислительную среду, возможность восстановления, возможность распределения и расширения.

Трудность в оценке проектных решений связана также с различной чувствительностью и временем действия критериев. Например, критерий эффективности обычно является краткосрочным и чрезвычайно чувствительным к проводимыми изменениям, а такие понятия, как адаптируемость и конвертируемость, проявляются на длительных временных интервалах и менее чувствительны к воздействию внешней среды.

Предназначение склада данных — информационная поддержка принятия решений, а не оперативная обработка данных. Потому база данных и склад данных не являются одинаковыми понятиями. Архитектура ХД представлена на рис. 2.15.

Основные принципы организации ХД [39].

1. *Предметная ориентация*. В оперативной базе данных обычно поддерживается несколько предметных областей, каждая из которых может послужить источником данных для ХД. Например, для магазина, торгующего видео- и музыкальной продукцией, интерес представляют следующие предметные области: клиенты, видео-, аудиокассеты и *CD*-диски, сотрудники, поставщики. Явно прослеживается аналогия между предметными областями ХД и классами объектов в объектно-ориентированных базах данных. Это говорит о возможности применения методов проектирования, применяемых в объектно-ориентированных СУБД.

2. *Средства интеграции*. Приведение разных представлений одних и тех же сущностей к некоторому общему типу.

Рис. 2.15. **Архитектура ХД**

3. *Постоянство данных*. В ХД не поддерживаются операции модификации в смысле традиционных баз данных. В ХД поддерживается модель «массовых загрузок» данных, производимых в заданные моменты времени по установленным правилам, в отличие от традиционной модели индивидуальных модификаций.

4. *Хронология данных*. Благодаря средствам интеграции реализуется определенный хронологический временной аспект, присущий содержимому ХД.

Основные функции репозитариев:

- парадигма включения/выключения и некоторые формальные процедуры для объектов;
- поддержка множественных версий объектов и процедуры управления конфигурациями для объектов;
- способность оповещения инструментальных и рабочих систем об интересующих их событиях;
- управление контекстом и разные способы обзора объектов репозитария;
- возможность определения потоков работ.

Рассмотрим кратко основные направления научных исследований в области баз данных:

- развитие теории реляционных баз данных;
- моделирование данных и разработка конкретных моделей разнообразного назначения;
- отображение моделей данных, направленных на создание методов преобразования моделей данных и конструирования коммутативных отображений, разработку архитектурных аспектов ото-

бражения моделей данных и спецификаций определения отображений для конкретных моделей данных;

- СУБД с мультимодельным внешним уровнем, обеспечивающие возможности отображения широко распространенных моделей;
- разработка, выбор и оценка методов доступа;
- самоописываемые базы данных, позволяющие применять единые методы доступа для данных и метаданных;
- управление конкурентным доступом;
- системы программирования баз данных и знаний, которые обеспечивали бы единую эффективную среду как для разработки приложений, так и для управления данными;
- машины баз данных;
- дедуктивные базы данных, основанные на применении аппарата математической логики и средств логического программирования в области с данных;
- пространственно-временные базы данных;
- интеграция неоднородных информационных ресурсов.

Серьезные недостатки реляционной модели данных привели к необходимости поиска других моделей. Такой прогрессивной и перспективной моделью данных является объектно-ориентированная модель данных.

В ней собственно БД, интерфейс пользователя и алгоритм приложения построены с использованием объектно-ориентированного подхода.

Автор реляционной модели данных Э. Ф. Кодд первоначально сформулировал 12 требований к БД, чтобы она могла называться реляционной[1]. В дальнейшем этот перечень увеличился до 333 требований. Им, несмотря на широкое распространение реляционных баз данных, не удовлетворяет ни одна из известных СУБД.

Более того, при значительных объемах данных начинают проявляться недостатки реляционных баз данных. К этим недостаткам относятся: сложность структуры, вызванная необходимостью проведения нормализации; низкая производительность из-за поиска по ключу, что в 3—5 раз увеличивает количество операций доступа; ограниченный набор типов данных; представление данных только в виде двумерных таблиц и невозможность реализации таблиц с нелинейной структурой; невозможность послойного рассмотрения данных (например, *работающие* — в одном слое, *научные сотрудники и преподаватели* — в другом, подчиненном слое); нестыковка с принципами все более широко применяемого объектно-ориентированного подхода; невозможность задать для определенного типа данных набор операторов-методов, которые приходится вводить в конкрет-

---

[1] *Codd E. F.* Extending the Database Relational Model to Capture More Meaning. ACM Transactions on Database Systems, Vol. 4. № 4. December 1979.

ном приложении; возникновение конфузии — утраты при многочисленных обновлениях третьей (а порой — и второй) нормальной формы; сложность совмещения с другой парадигмой хранилищ данных.

Суть объектно-ориентированной БД определяется особенностями объектной модели. Выделяется несколько специфических понятий. Данные называют *свойствами*, а алгоритмы — *методами*. Доступ к классу осуществляется через свойства — в статическом режиме написания и отладки программы или через методы — в процессе выполнения (работы) программы.

Начало работы элемента класса задается с помощью специальных внутренних (например, нажатие кнопки) или внешних (вызов из другой подпрограммы) сигналов, называемых *событием*.

Программную реализацию класса называют компонентой. Реализация компоненты в некоторой программе получила название объекта.

В объектно-ориентированном программировании используются три основные принципа: инкапсуляция, наследование, полиморфизм.

1) *инкапсуляция* — выделение класса с доступом к нему через свойства или методы;

2) *наследование* — трансформация класса путем изменения свойств и методов с помощью методов, называемых конструктором и деструктором. Все классы (компоненты) строятся по иерархическому принципу с происхождением от некоторого исходного класса;

3) *полиморфизм* позволяет использовать метод с одним именем как в базовом, так и в производных классах. Дело в том, что количество классов значительно: уже сейчас оно приближается к 100 и продолжает расти. Если для производных классов применять для однотипных, «похожих» методов (например, начертание квадрата и окружности), разные имена, пользователю, особенно начинающему, будет сложно освоиться с таким разнообразием имен.

Приведем основные положения ООБД, базирующиеся на объектно-ориентированном подходе.

1. В качестве значения столбца может быть использован произвольный кортеж. Иными словами, столбец может являться как бы некоторой другой таблицей. Таким образом, создается возможность реализации таблиц с нелинейной структурой.
2. Процедура манипуляции данными позволяют присоединять процедуры, определенные значениями столбцов.
3. Данные столбцов могут наследоваться.
4. Элементами отношений могут служить не только отдельные элементы, как в реляционных БД, но и множества.
5. Формируются классы данных, которые организуют столбцы в иерархию.

Базовым языком ООБД чаще всего является *C++*. Для работы с такими ООБД разрабатывается новый вариант языка программирования *SQL*, получивший название *SQL3* и содержащий внутри себя

в качестве частного случая реляционную модель (*SQL2*). Если *SQL2* определяет семь способов связывания со стандартными языками программирования, в *SQL3* это количество предполагается увеличить.

В технологии разработки ООБД конкурирует два направления:

1) *Distributed Object Linking and Embedding* (*OLE*) фирмы *Microsoft*;

2) *Common Object Request Broker Architecture* (*CORBA*) группы *OBDMG*, поддерживаемая фирмами *IBM, Novell, DEC*, с ориентацией на все платформы. В рамках этого направления выделены и сформированы указанные ранее язык определения объектов *Object Definition Language* (*ODL*); объектный язык запроса *Object Query Language* (*OQL*); язык определения интерфейсов *Interface Definition Language* (*IDL*).

Во втором направлении обычно выделяют [44] два стандарта управления БД.

1) *ODL/OQL*, в котором объекты и методы формируются обычно с помощью языка программирования *C*;

2) язык программирования *SQL3*.

Необходимо отметить, что к объектно-ориентированным возможно отнести только те базы данных, у которых все структурные элементы реализации построены с использованием объектно-ориентированного подхода. Если хотя бы один структурный элемент реализации не использует объектно-ориентированный подход, говорят об *объектно-реляционной базе данных* (далее — *ОРБД*).

Таким образом, чтобы воспользоваться объектно-ориентированным подходом в построении собственно БД, необходимо:

- провести инкапсуляцию данных, т. е. выделить классы и объекты;
- определить возможные виды структуры реализуемых таблиц;
- создать наследование классов данных;
- обеспечить полиморфизм.

Реализация даже первой позиции неоднозначна и различна, например, в ООСУБД (объектно-ориентированная СУБД) и ОРСУБД (объектно-реляционная СУБД). Имеется некоторое отличие даже в рамках различных ООСУБД.

По сравнению с реляционными БД ООБД обладают следующими преимуществами.

1. Лучшие возможности моделирования систем из блоков, обладающих произвольными связями.

2. Легкая расширяемость структуры за счет создания новых типов данных (свойств), наследования, установления новых связей и корректировки методов.

3. Возможность использования рекурсивных методов при навигационном методе доступа к большим объемам данных.

4. Повышение производительности в 10—30 раз.

5. Более широкая сфера применения (например, использование в мультимедиасистемах).

Преимущества ООБД [13] приведут, видимо, к очень широкому их распространению. Однако прежде следует решить ряд задач по устранению недостатков ООБД: создать гибкую структуру БД; построить четкий язык программирования; отработать синтаксис разбора запросов, в том числе — вложенных; определить несколько методов доступа к данным; отработать вопросы одновременного доступа (разрешение конфликтов при множественном наследии); определить сложный перебор данных; отработать защиту и восстановление данных; уточнить семантику (действия) операторов при динамических изменениях; встроить изменение атрибутов дочерних объектов.

Переход к объектно-ориентированным моделям данных связан с процессом «перекачки» в них огромных объемов информации, которая в настоящее время хранится преимущественно в реляционных базах данных. Чтобы упростить этот процесс, сформировали объектно-реляционную модель данных, в которой выделяются две разновидности — гибридные и расширенные.

В гибридных ОРБД объектно-ориентированный подход используется в создании интерфейса пользователя и алгоритма приложения. В то же время система таблиц формируется в рамках реляционной модели данных.

В расширенных ОРБД объектно-ориентированный подход используется прежде всего при построении системы таблиц. Для этого разработана модификация языка *SQL2*, получившая название языка программирования *SQL3*.

«Промежуточной» моделью данных между реляционными и объектно-ориентированными базами данных является объектно-реляционная модель (ОРБД).

Ее появление вызвано двумя причинами.

1. Сложностью построения новой модели данных «с листа». Удобнее это делать на основе имеющихся проверенных разработок.
2. Учетом преемственности с широко используемыми реляционными моделями, которые нельзя мгновенно заменить на объектно-ориентированные БД.

Различают, как отмечалось ранее, две разновидности ОРБД — *гибридные и расширенные.*

1. *Гибридные* [13]. Интерфейс пользователя и алгоритм приложения выполнены с учетом объектно-ориентированного подхода, тогда как собственно БД является реляционной. Примерами могут служить СУБД *Paradox* и *InterBase* в рамках программного продукта *Delphi*. В каком-то смысле гибридной можно считать СУБД *Access* при использовании языка программирования *Visual Basic for Application* (*VBA*).
2. *Расширенные* (*постреляционные*). В них предполагается объектно-ориентированное построение собственно базы данных путем использования известных и введения новых типов данных, связанных между собой. Эта связь чаще всего осуществляется созданием

методов с помощью триггеров и хранимых процедур. В расширенных объектно-реляционной модели допускается, в отличие от реляционной модели данных, не атомарность данных в поле. В таких полях может располагаться другая таблица или массив. К подобным СУБД относятся *Informix Universal Server*, *Oracle* 8, *UniSQL*. В таких СУБД широко используется язык *SQL3*.

Заметим, что постреляционными называют БД, в которых частично проведена денормализация данных, при этом допускается неатомарность записей. Рассмотрим более подробно обе разновидности.

Чтобы понять перспективы развития ОРБД, следует оценить их достоинства и недостатки.

К достоинствам следует отнести:

- устранение ряда недостатков реляционных БД;
- повторное использование компонентов;
- использование накопленных знаний по реляционным БД.

К недостаткам ОРБД возможно отнести:

- усложнение структуры БД и частичная утрата простой обозримости результатов, как в реляционных БД;
- сложность построения абстрактных типов данных и методов, связывающих типы в иерархию;
- менее широкий набор типов связей, определяемых языком программирования *SQL*, чем в ООБД;
- менее продуманный, отлаженный и стандартизованный набор типов данных, чем в ООБД.

ОРБД, по-видимому, будут существовать еще достаточно долго, чему есть, по меньшей мере, два объяснения:

1) быстрое накопление с помощью ОРБД опыта, который можно использовать при создании ООСУБД;

2) необходимость иметь средства для постепенного эволюционного «перевода» многочисленных реляционных БД в разряд ООБД, за которыми, видимо, будущее.

В последнее время ***распределенные базы данных*** (далее — РБД) находят все более широкое применение в связи с массовым распространением «сетевых» технологий.

Теория создания, использования и функционирование РБД имеет свои особенности по сравнению с централизованными БД.

Быстрое распространение сетей передачи данных, резкое увеличение объема внешней памяти персонального компьютера (далее — ПК) при ее удешевлении в 1980-е гг., развитие возможностей персональных компьютеров, которые по своим характеристикам в 1990-е гг. уже превосходили суперЭВМ 1980-х гг., создали необходимую базу для реализации и широкого внедрения РБД.

К достоинствам РБД относятся:

- соответствие структуры РБД структуре организаций;
- гибкое взаимодействие локальных БД;

- широкие возможности централизации узлов;
- непосредственный доступ к информации, снижение стоимости передач (за счет уплотнения и концентрации данных);
- высокие системные характеристики (малое время отклика за счет распараллеливания процессов, высокая надежность);
- модульная реализация взаимодействия, расширения аппаратных средств, возможность использования объектно-ориентированного подхода в программировании;
- возможность распределения файлов в соответствии с их активностью;
- независимые разработки локальных БД через стандартный интерфейс.

Вместе с тем РБД обладают более сложной структурой, что вызывает появление дополнительных проблем (избыточность, несогласованность данных по времени, согласование процессов обновления и запросов, использования телекоммуникационных ресурсов, учет работы дополнительно подсоединенных локальных БД, стандартизация общего интерфейса, усложнение защиты данных) согласования работы элементов.

Серьезные проблемы возникают при интеграции в рамках РБД однородных (гомогенных) локальных БД с одинаковыми, чаще всего — реляционными моделями данных.Проблемы значительно усложняются, если локальные БД построены с использованием различных моделей данных (неоднородные, гетерогенные РБД).

По результатам внедрения первых РБД возникло ряд проблем, в частности с синхронизацией данных при модификации, что привело к необходимости теоретической проработки процессов распределения.

Распределенная база данных — система логически интегрированных и территориально РБД, языковых, программных, технических и организационных средств, предназначенных для создания, ведения и обработки информации.

Это означает, что информация физически хранится на разных ЭВМ, связанных сетью передачи данных. Любой узел (участок) может выполнять приложение и участвовать в работе, по крайней мере, одного приложения.

Большинство требований, предъявляемых к РБД, аналогично требованиям к централизованным БД, но их реализация имеет свою специфику, рассматриваемую ниже. В частности, в РБД иногда полезна избыточность.

Дополнительными специфическими требованиями являются:

1) ЯОД в рамках схемы должен быть один для всех локальных БД;
2) доступ должен быть коллективным к любой области РБД с соответствующей защитой информации;
3) подсхемы должны быть определены в месте сосредоточения алгоритмов (приложений, процессов) пользователя;
4) степень централизации должна быть разумной;

5) необходим сбор и обработка информации об эффективности функционирования РБД.

В дальнейшем К. Дейт сформулировал 12 правил для РБД.

1. Локальная автономность.
2. Отсутствие опоры на центральный узел.
3. Непрерывное функционирование (развитие) РБД.
4. Независимость РБД от расположения локальных БД.
5. Независимость от фрагментации данных.
6. Независимость от репликации (дублирования) данных.
7. Обработка распределенных запросов.
8. Обработка распределенных транзакций.
9. Независимость от типа оборудования.
10. Независимость от операционной системы.
11. Независимость от сетевой архитектуры.
12. Независимость от типа СУБД.

В дальнейшем рассмотрении РБД выделим:

- общие вопросы (состав, работа РБД);
- теоретические вопросы РБД, в которых по-прежнему выделим три составляющих (создание, использование и функционирование РБД).

Схема РБД может быть представлена в виде, показанном на рис. 2.16.

В ней выделяют пользовательский, глобальный (концептуальный), фрагментарный (логический) и распределенный (локальный) уровни представления данных, определяющие сетевую СУБД.

Сравнительные характеристики стратегий хранения приведены в табл. 2.3. На ее основе может быть построен простейший алгоритм выбора стратегии, показанный на рис. 2.17.

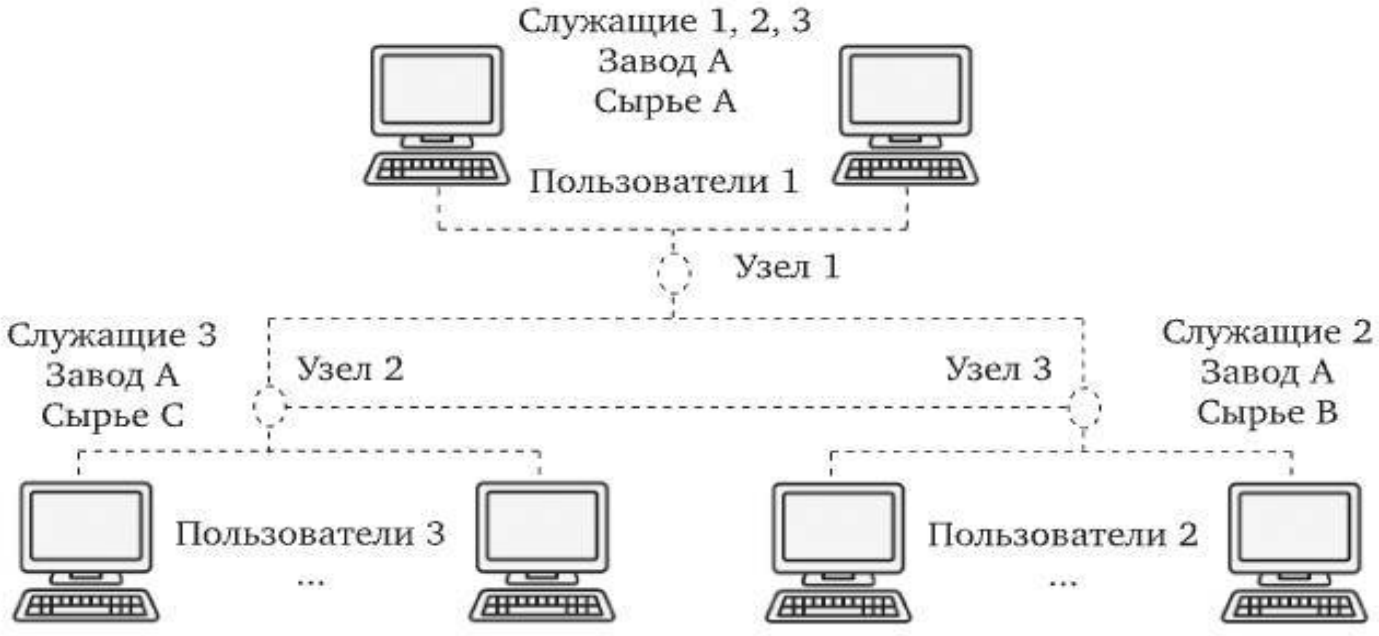


Рис. 2.16. **Схема распределенной БД**

Отметим, что в обычной сети имеет место равноправие компьютеров, что может вызвать дополнительные осложнения в части доступа к данным в процедурах обновления и запросов.

В связи с этим часто используют архитектуру «клиент — сервер» (см. параграф 3.5) структуру локальной сети, в которой применено распределенное управление сервером и рабочими станциями (клиентами) для максимально эффективного использования вычислительной мощности.

Таблица 2.3

Стратегии хранения данных

Название	Суть стратегии	Достоинство	Недостатки
Централизация (в том числе технология «клиент — сервер»)	Единственная копия в одном узле	Простота структуры	Скорость обработки ограничена одним узлом. Ограниченный доступ. Малая надежность. Долговременная память определяет объем БД
Локализация (расчленение)	Единственная копия, расчленение по узлам (полная копия БД не допускается)	Объем БД определяется памятью сети. Снижение стоимости РБД. Время отклика при параллельной обработке уменьшается. Малая чувствительность к узким местам. Повышенная надежность при высокой локализации данных	Запрос может быть по всем узлам. Доступ хуже, чем при централизации. Рекомендации применения. Долговременная память ограничена по сравнению с объемом БД. Должна быть повышена эффективность функционирования при высокой степени локализации

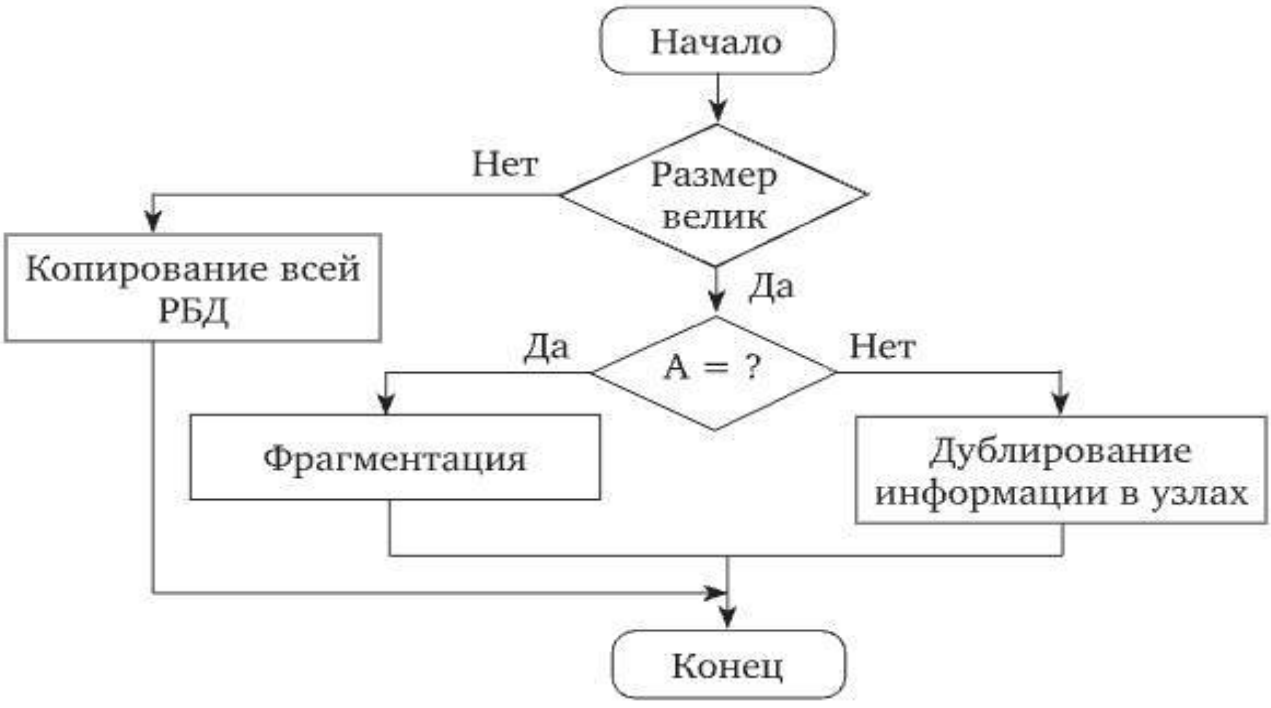


Рис. 2.17. **Алгоритм выбора стратегии хранения данных.**
A — запрос локален

Работа в архитектуре «клиент — сервер» может поддерживаться и с помощью схемы *Open DataBase Connectivity* (*ODBC*), как показано на рис. 2.18.

Рис. 2.18. **ODBC в архитектуре «клиент — сервер»**

Сеть образуется в этом случае путем соединения серверов. Такое соединение обеспечивается или средствами СУБД (*SQL Server*) или мониторами транзакций (*TUXEDO*).

2.5. Представление и использование информации

В условиях использования информационных технологий функции распределены между человеком и техническими устройствами. При анализе деятельности человека наибольшее значение имеют эргономические (инженерно-психологические) и психологические (социально-психологические) факторы [14].

Эргономические факторы позволяют, во-первых, определить рациональный набор функций человека, во-вторых, обеспечить рациональное сопряжение человека с техническими средствами и информационной средой.

Психологические факторы имеют большое значение, так как внедрение информационных технологий в корне изменяет деятельность человека. Наряду с положительными моментами, связанными с рационализацией деятельности, предоставлением новых возможностей, возникают и негативные явления. Это может быть вызвано различными факторами: психологическим барьером, усложнением функций, другими субъективными факторами (условиями и организацией труда, уровнем заработной платы, результативностью труда, изменением квалификации).

При работе в среде информационных технологий человек воспринимает не сам объект, а некоторую его обобщенную информационную модель, что накладывает особые требования на совместимость пользователя с различными компонентами информационных технологий.

Важным признаком, который необходимо учитывать при разработке и внедрении информационных технологий является отношение человека к информации. Оно может быть пассивным, когда пользователю предоставляется информация по жесткому алгоритму, и активным, когда пользователь создает необходимые ему данные.

Основной задачей операции представления информации пользователю является создание эффективного интерфейса в системе «человек — компьютер». При этом осуществляется преобразование информации в форму, удобную для восприятия пользователя.

Интерфейс должен обеспечивать:

- *наглядность* отображения информации; приближенность к естественному языку, естественным знаковым системам;
- возможность отображения *различной* — как фактографической, так и документальной информации (т. е. текста и мультимедиа содержимого);
- возможность работы с *максимально* доступным множеством источников данных *без потери гибкости*, причем с условием регулярности элементов управления; независимость от архитектуры системы и организации сетевых ресурсов.

Пользователи или потребители информации — это животный и растительный мир, люди и технические устройства. При этом *конечный пользователь* (англ. «*End user*») — это пользователь, не работающий непосредственно с системой, но применяющий результат ее функционирования.

Интерфейс пользователя — это совокупность правил, методов и программно-аппаратных средств, обеспечивающих взаимодействие пользователя с компьютером. Пользовательский интерфейс часто понимают только как внешний вид программы. В действительности интерфейс пользователя включает в себя все аспекты, оказывающие влияние на взаимодействие пользователя и системы.

Экран. В настоящее время сформировались следующие основные *режимы представления* и *управления информацией* на экране, которым соответствуют определенные сценарии диалога *человек-ЭВМ*: режим командной строки; режим форматированного экрана; режим меню.

Интерфейсы информационных систем условно можно разделить на три группы:

- текстовые (текст-ориентированные);
- смешанные (псевдографические);
- графические.
- *WEB*-интерфейсы;
- объектно-ориентированные интерфейсы.

Рассмотрим ретроспективу подходов к построению пользовательского интерфейса.

В качестве примера текстовых (текст-ориентированных) интерфейсов приведем интерфейс командной строки *DOS* или *shell*-интерпретатор *UNIX*. Пользователь взаимодействует с ВС (вычислительной системой) с помощью клавиатуры, набирая специальные команды. Для задания различных опций служат параметры. Система как ответ на действия пользователя выдает сообщения или результат выполнения введенной команды в текстовом виде. Курсор может иметь вид мигающего прямоугольника или черточки, обозначающей место ввода. В таком режиме можно одновременно взаимодействовать лишь с одной программой. Управлять взаимодействием программ можно только с командной строки, причем проверить результат — по окончании работы.

В *смешанных* (*псевдографических*) *интерфейсах* различают понятия «оконный» и «графический» интерфейсы. Первый базируется на принципе разделения реального окна монитора (или виртуального *desktop*'а намного большего размера, чем физический дисплей) на прямоугольные области, внутри каждой из которых определенная программа направляет свой вывод и откуда получает команды.

Термин «*графический*» означает использование оконного графического интерфейса (каждое окно отображает графический интерфейс); полноэкранного режима (выполняется только одна программа, осуществляющая вывод в графическом режиме). Другими словами, оконный — не обязательно графический, а графический — не всегда оконный.

Псевдографическими обозначают *интерфейсы* с графическими интерфейсными элементами, например кнопками, индикаторами процесса выполнения, реализуемыми с помощью псевдографики набора *ANSI*, например, оболочка *FAR*. Псевдографический интерфейс можно отнести к промежуточному между чисто командным интерфейсом и графическим.

К *графическим интерфейсам* относят все оконные графические системы *Windows*, оболочки для *UNIX* — *KDE*, *GNOM*, *CDE*, *X-Window*, *Photon* из *ОС QNX*, *Aqua* из *MacOS X*. Графическими они называются потому, что все элементы пользовательского интерфейса, как и сами данные в окнах, отображаются в графическом режиме с помощью 256 цветов, 16-битной или 32-битной глубины цветового буфера. Это позволяет сформировать привлекательные с точки зрения пользователя окна, кнопки, пиктограммы, ползунки, индикаторы.

Понятие окна — общее для всех этих систем. **Окно** — прямоугольная область экрана, куда программа выводит свои данные и откуда получает команды.

Среди существующих вариантов интерфейса в системе «человек — компьютер» можно выделить два основных типа: на основе

меню («смотри и выбирай») и на основе языка команд («вспоминай и набирай»).

Интерфейсы типа меню облегчают взаимодействие пользователя с компьютером, так как не требуют предварительного изучения языка общения с системой. На каждом шаге диалога пользователю предъявляются все возможные в данный момент команды в виде наборов пунктов меню, из которого пользователь должен выбрать нужный. Такой способ общения удобен для начинающих и непрофессиональных пользователей.

Интерфейс на основе языка команд требует знания пользователем синтаксиса языка общения с компьютером. Достоинством командного языка является его гибкость и мощность.

Указанные два способа реализации интерфейса представляют собой крайние случаи, между которыми возможно существование различных промежуточных вариантов. Составные части интерфейса представлены на рис. 2.19.

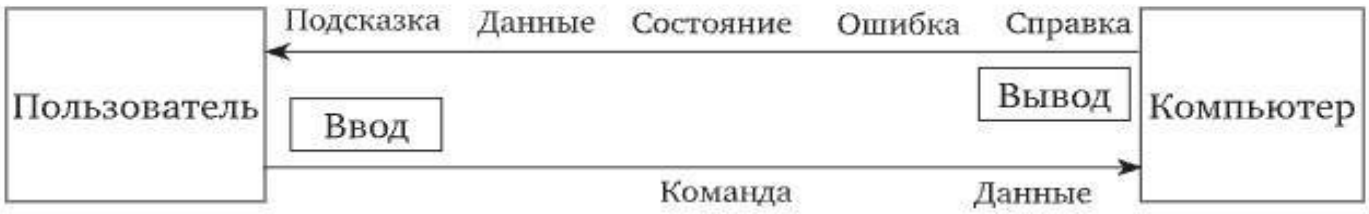


Рис. 2.19. **Составные части интерфейса**

Технология представления информации должна давать дополнительные возможности для понимания данных пользователями, поэтому целесообразно использование графики, диаграмм, карт.

Пользовательский интерфейс целесообразно строить на основе концептуальной модели предметной области, которая представляется совокупностью взаимосвязанных объектов со своей структурой. Однако доступ к объектам и их экземплярам возможен только через систему окон различных типов. Ряд окон связан с конкретным объектом. В соответствии с этим предложением в сценарии работы пользователя при информационном наполнении понятий предметной области выделяем две фазы:

- выбор окон;
- работа с окнами.

Для упрощения работы окна можно группировать в соответствии с функциональными потребностями. С этой целью вводится механизм разделов, который предоставляет возможность создания иерархии функционально ориентированных разделов, в каждый из которых включается необходимый набор других разделов и окон. Посредством спецификации окон для каждого из объектов возможно указать допустимые режимы работы с экземплярами и состав видимых атрибутов с режимами работы с ними. Возможно отобрать несколько разделов и несколько окон в них одновременно.

Таким образом, фаза выбора объектов должна поддерживаться следующими функциями:

- работа с общим каталогом окон в главном разделе;
- создание нового раздела;
- удаление раздела;
- редактирование описания раздела;
- передача определений и окон между разделами;
- движение по иерархии разделов;
- отбор разделов для работы;
- отбор окон для работы.

Позиции окон могут быть связаны с другими окнами через соответствующие команды из типового набора. По существу спецификация окон задает сценарий работы с экземплярами.

Окно — средство взаимосвязи пользователя с системой. Окно представляется как специальный объект. Проектирование пользовательского интерфейса представляет собой процесс спецификации окон.

Примером оконного интерфейса является интерфейс *MS Windows*, использующий метафору рабочего стола и включающий ряд понятий, близких к естественным (окна, кнопки, меню и т. д.).

Пользователь информационной системы большей частью вынужден использовать данные из самых разных источников: файлов, баз данных, электронных таблиц, электронной почты и т. д. При этом данные имеют самую различную форму: текст, таблицы, графику, аудио- и видеоданные и др. В связи с этим возникает проблема интеграции источников информации, заключающаяся в том, что, во-первых, пользователю должны представляться не данные, а информация в форме максимально удобной для восприятия, во-вторых, он должен использовать единственный универсальный интерфейс, позволяющий единообразно работать с подготовленной информацией. Пассивные пользователи, называемые иногда потребителями, обладают рядом специфических качеств, связанных с отсутствием времени, желания и квалификации для более глубокого изучения используемых инструментальных средств. В этом случае алгоритм общения с системой должен быть предельно простым. Другая часть пользователей требует предоставления достаточно широкого круга средств активного влияния на выполняемые информационные процессы.

Этим требованиям удовлетворяет *Web*-технология. Развитие средств вычислительной техники привело к ситуации, когда вместо традиционных параметров — производительность, пропускная способность, объем памяти, — узким местом стал интерфейс с пользователем. Первым шагом на пути преодоления кризисной ситуации стала концепция гипертекста, впервые предложенная Теодором Хольмом Нельсоном.

По своей сути **гипертекст** — это обычный текст, содержащий ссылки на собственные фрагменты и другие тексты (рис. 2.20).

Аналогом гипертекста можно считать книгу, оглавление которой по своей сути представляет ссылки на главы, разделы, страницы. Внутри книги содержатся ссылки на другие источники. Дальнейшее развитие гипертекст получил с появлением сети Интернет, когда появилась возможность размещать тексты на различных, территориально удаленных компьютерах. При этом требовалось дальнейшее совершенствование интерфейса, так как имеющийся не позволял представить разнообразную информацию разной природы, был ограничен и затруднен для восприятия, отсутствовал доступ множества потребителей к единому массиву структурированной информации. В результате была предложена и реализована концепция навигатора *Web*. *Web*-сервер выступает в качестве информационного концентратора, получающего информацию из разных источников и в однородном виде представляющем пользователю. Средства *Web* обеспечивают также представление информации с нужной степенью детализации с помощью *Web*-навигатора. Таким образом, *Web* — это инфраструктурный интерфейс для пользователей различных уровней.

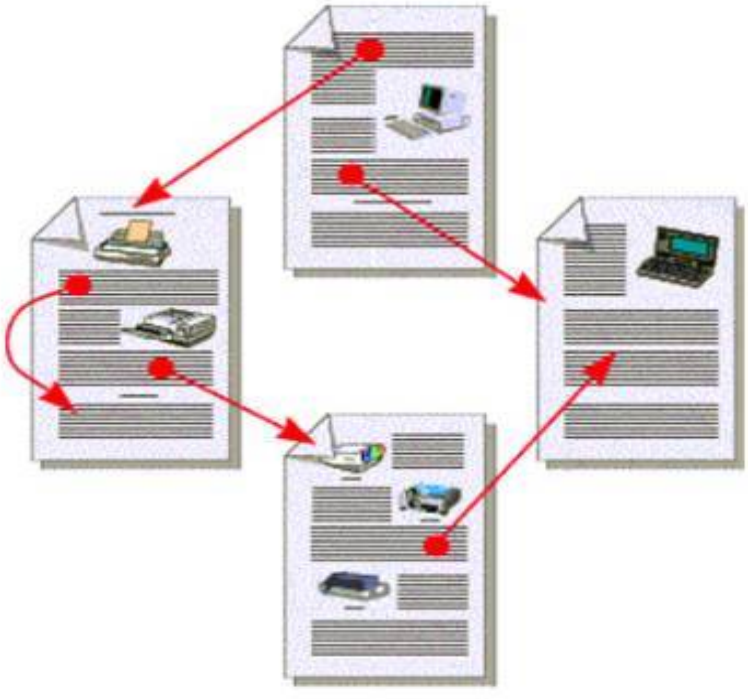


Рис. 2.20. **Гипертекст**

Несомненным преимуществом *Web*-технологии является удобная форма предоставления информационных услуг потребителям, определяемая как концепция публикаций информации и имеющая следующие особенности:

- информация предоставляется потребителю в виде публикаций;
- публикация может объединять информационные источники различной природы и географического расположения;

- изменения в информационных источниках мгновенно отражаются в публикациях;
- в публикациях могут содержаться ссылки на другие публикации без ограничения на местоположение и источники последних (гипертекстовые ссылки);
- потребительские качества публикаций соответствуют современным стандартам мультимедиа (доступны текст, графика, звук, видео, анимация);
- публикатор не заботится о процессе доставки информации к потребителю;
- количество потенциальных потребителей информации практически не ограничено;
- публикации отражают текущую информацию, время запаздывания определяется исключительно скоростью подготовки электронного документа;
- информация, предоставленная в публикации, легкодоступна, благодаря гипертекстовым ссылкам и средствам контекстного поиска;
- информация легко усваивается потребителем благодаря широкому спектру изобразительных возможностей, предоставляемых *Web*-технологией;
- технология не предъявляет особых требований к типам и источникам информации;
- технология допускает масштабируемые решения: увеличение числа одновременно обслуживаемых потребителей не требует радикальной перестройки системы.

В настоящее время сформировались и на уровне стандартов утвердились следующие ***стили пользовательских интерфейсов***.

1) *пользовательский интерфейс* (*Graphical User Interface*, *GUI*) определяется как стиль взаимодействия «пользователь — компьютер», в котором применяются четыре фундаментальных элемента: окна, пиктограммы, меню и указатели. Иногда *GUI*-интерфейс называют *WIMP*-интерфейсом (*Windows* — окна, *Icons* — пиктограммы, *Menus* — меню и *Pointers* — указатели);

2) *пользовательский Web-интерфейс* (*WUI-интерфейс*). Базовый *WUI*-стиль (*Web User Interface*) весьма схож с меню иерархической структурой, которые пользователи знают по опыту работы в средах с графическим интерфейсом за исключением более наглядного представления и использования гиперссылок. Необходимая навигация выполняется в рамках одного или нескольких приложений с использованием текстовых или визуальных гиперссылок. В зависимости от структуры гиперссылок приложения навигация в пределах *WUI*-интерфейса приводит к отображению *Web*-страниц в иерархии приложения — по одной за раз — в линейном или нелинейном стиле внутри одного *GUI*-окна. Во многих отношениях *WUI*-ориентированные приложения — это «шаг назад в будущее» — или,

может быть, нечто худшее, учитывая объемы электронных документов и других материалов в *Web*;

3) *объектно-ориентированный пользовательский интерфейс.* Проектирование программных объектов дает возможность предоставить в распоряжение пользователя приложение, обладающее объектно-ориентированным стилем ПИ (прикладного интерфейса) и (или) объектно-ориентированной внутренней структурой (реализацией). Многие объектно-ориентированные свойства реального мира находят отражение во внешнем виде, поведении, требованиях к взаимодействию и функциональных возможностях. Компьютеризованное усовершенствование или дополнение объектов реального мира, если только оно плохо спроектировано или реализовано, неочевидно для конечного пользователя и не в состоянии преодолеть его устоявшиеся знания и восприятие. Представленные в явном виде при проектировании обозначения классов объектов, иерархии классов и наследование посредством иерархии классов остаются прозрачными для пользователя.

Объектно-ориентированный прикладной ПИ должен обладать следующими свойствами:

- обеспечивать непосредственное манипулирование (перетаскивать любые объекты куда угодно);
- обеспечивать непосредственный ввод данных (записывать любую информацию);
- обеспечивать контекстную зависимость от объектов (всплывающие (контекстные) меню, справки, согласованность и т. д.).

Хороший прикладной объектно-ориентированный ПИ прост в использовании; это значит, что его механизмы пользовательского интерфейса прозрачны.

Тема 3
БАЗОВЫЕ ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ

3.1. Мультимедиатехнологии

Мультимедиа (англ. *multimedia* от лат. *multum* — много и *media*, *medium* — средоточие, средства) — это комплекс аппаратных и программных средств, позволяющих пользователю работать в диалоговом режиме с разнородными данными (графикой, текстом, звуком, видео и анимацией), организованными в виде единой информационной среды».

Таким образом, мультимедиа объединяет несколько типов разнородных данных (текст, звук, видео, графическое изображение и анимацию) в единое целое. И это понятие само по себе имеет три стороны:

- мультимедиа как идея, т. е. новый подход к хранению информации различного типа в единой цифровой форме;
- мультимедиа как оборудование для обработки и хранения информации, без него мультимедиа-идею реализовать невозможно;
- программное обеспечение, позволяющее объединить четыре элемента информации в законченное мультимедиаприложение.

Одним из важных факторов мультимедиа является *интерактивность* — свойство реагировать на действия пользователей, в том числе и управлять пользователем.

В настоящее время мультимедиа является активно развивающейся областью информационных технологий. В этом направлении активно работает значительное число крупных и мелких фирм, технических университетов и студий (в частности *IBM*, *Apple*, *Motorola*, *Philips*, *Sony*, *Intel* и др.). Области использования чрезвычайно многообразны: интерактивные обучающие и информационные системы, автоматизированное проектирование, развлечение и др.

Три основные принципа мультимедиа:

- представление информации с помощью комбинации множества воспринимаемых человеком сред;
- наличие нескольких сюжетных линий в содержании продукта (в том числе и выстраиваемых самим пользователем на основе «свободного поиска» в рамках предложенной в содержании продукта информации);
- художественный дизайн интерфейса и средств навигации.

Основными характерными особенностями этих технологий являются:

- объединение многокомпонентной информационной среды (текста, звука, графики, фото, видео) в однородном цифровом представлении;
- обеспечение надежного (отсутствие искажений при копировании) и долговечного хранения (гарантийный срок хранения — десятки лет) больших объемов информации;
- простота переработки информации от рутинных до творческих операций.

Несомненным достоинством и особенностью технологии являются следующие возможности мультимедиа, которые активно используются в представлении информации:

- хранение большого объема самой разной информации на одном носителе;
- увеличение (детализация) на экране изображения или его наиболее интересных фрагментов, иногда в двадцатикратном увеличении (режим «лупа») при сохранении качества изображения. Это особенно важно для презентации произведений искусства и уникальных исторических документов;
- сравнение изображения и обработки его разнообразными программными средствами с научно-исследовательскими или познавательными целями;
- выделение в сопровождающем текстовом или другом визуальном материале «горячих слов (областей)», по которым осуществляется немедленное получение справочной или любой другой пояснительной (в том числе визуальной) информации (технологии гипертекста и гипермедиа);
- осуществление непрерывного музыкального или любого другого аудиосопровождения, соответствующего статичному или динамичному визуальному ряду;
- использование видеофрагментов из фильмов, видеозаписей и т. д., функции «стоп-кадра», покадрового «пролистывания» видеозаписи;
- включение в содержание диска баз данных, методик обработки образов, анимации (к примеру, сопровождение рассказа о композиции картины графической анимационной демонстрацией геометрических построений ее композиции) и т. д.;
- подключение к глобальной сети Интернет;
- работа с различными приложениями (текстовыми, графическими и звуковыми редакторами, картографической информацией);
- автоматический просмотр всего содержания продукта («слайд-шоу») или создание анимированного и озвученного «путеводителя-гида» по продукту («говорящей и показывающей инструк-

ции пользователя»), включение в состав продукта игровых компонентов с информационными составляющими;

- «свободная» навигация по информации и выходу в основное меню (укрупненное содержание), на полное оглавление или вовсе из программы в любой точке продукта.

Таким образом, *мультимедийный продукт* — наиболее эффективная форма подачи информации в среде компьютерных информационных технологий. Он позволяет собрать воедино огромные и разрозненные объемы информации, дает возможность с помощью интерактивного взаимодействия выбирать интересующие в данный момент информационные блоки, значительно повышая эффективность восприятия информации.

Достигнутый технологический базис основан на использовании нового стандарта оптического носителя *DVD* (*Digital Versalite/Video Disk*), имеющего емкость порядка единиц и десятков Гбайт и заменяющего все предыдущие: *CD-ROM*, *Video CD*, *CDaudio*. Использование *DVD* позволило реализовать концепцию однородности цифровой информации. Одно устройство заменяет аудиоплейер, видеомагнитофон, *CD-ROM* дисковод, слайдер и др. В плане представления информации оптический носитель *DVD* приближает ее к уровню виртуальной реальности.

Многокомпонентную мультимедиасреду целесообразно разделить на три группы: аудиоряд, видеоряд, текстовая информация.

Аудиоряд может включать речь, музыку, эффекты (звуки типа шум, гром, скрип и т. д., объединяемые обозначением *WAVE* (волна)) [42]. Главной проблемой при использовании этой группы мультисреды является информационная емкость. Для записи одной минуты *WAVE*-звука высшего качества необходима память порядка 10 Мбайт, поэтому стандартный объем *CD* (до 640 Мбайт) позволяет записать не более часа *WAVE*. Для решения этой проблемы используются методы компрессии звуковой информации.

Другим направлением является использование в мультисреде звуков (одноголосая и многоголосая музыка, вплоть до оркестра, звуковые эффекты) *Musical Instrument Digitale Interface* (*MIDI*). В данном случае звуки музыкальных инструментов, звуковые эффекты синтезируются программно-управляемыми электронными синтезаторами. Коррекция и цифровая запись *MIDI*-звуков осуществляется с помощью музыкальных редакторов (программ-секвенсоров). Главным преимуществом *MIDI* является малый объем требуемой памяти — 1 мин *MIDI*-звука занимает в среднем 10 Кбайт.

Видеоряд характеризуется по сравнению с аудиорядом большим количеством элементов. Выделяют статический и динамический видеоряды.

Статический видеоряд включает графику (рисунки, интерьеры, поверхности, символы в графическом режиме) и фото (фотографии и сканированные изображения).

Динамический видеоряд представляет последовательность статических элементов (кадров). Можно выделить три типовых группы:

- обычное видео (*life video*) — последовательность фотографий (около 24 кадров в секунду);
- квазивидео — разреженная последовательность фотографий (6—12 кадров в секунду);
- анимация — последовательность рисованных изображений.

Первая проблема при реализации видеорядов — разрешающая способность экрана и количество цветов. Выделяют три направления:

- стандарт *VGA* дает разрешение 640 × 480 пикселей (точек) на экране при 16 цветах или 320 × 200 пикселей при 256 цветах;
- стандарт *SVGA* (видеопамять 512 К, 8 бит/пиксель) дает разрешение 640 × 480 при 256 цветах;
- 24-битные видеоадаптеры (видеопамять 2 Мбайт, 24 бит/пиксель) позволяют использовать 16 млн цветов.

Вторая проблема — объем памяти. Для статических изображений один полный экран требует следующих объемов памяти:

- в режиме 640 × 480, 16 цветов — 150 Кбайт;
- в режиме 320 × 200, 256 цветов — 62,5 Кбайт;
- в режиме 640 × 480, 256 цветов — 300 Кбайт.

Такие значительные объемы при реализации аудио- и видеорядов определяют высокие требования к носителю информации, видеопамяти и скорости передачи информации.

При размещении ***текстовой информации*** на *CD-ROM* нет никаких сложностей и ограничений ввиду большого информационного объема оптического диска.

Области применения мультимедиа приложений:

- обучение с использованием компьютерных технологий (научно-просветительская или образовательная сфера);
- видеоэнциклопедии, интерактивные путеводители, тренажеры, ситуационно-ролевые игры и др.;
- информационная и рекламная служба;
- популяризаторская и развлекательная сферы;
- интернет-вещание. Спектр возможных применений — от просмотра заказной телепередачи, выбора нужной книги до участия в мультимедиаконференциях. Такие разработки получили название *Information Highway*;
- развлечения, игры, системы виртуальной реальности;
- презентационная (витринная реклама), СМИ;
- творчество;
- военные технологии;
- промышленность и техника (сенсорные экраны);
- торговля;
- мультимедийные информационные системы («мультимедиа киоски»), выдающие по запросу пользователя наглядную информацию;

- научные исследования — электронные архивы и библиотеки;
- медицина: базы знаний, методики операций, каталоги лекарств и т. п.;
- искусственный интеллект — внедрение элементов искусственного интеллекта в системе мультимедиа. Они обладают способностью «чувствовать» среду общения, адаптироваться к ней и оптимизировать процесс общения с пользователем: они подстраиваются под читателей, анализируют круг их интересов, помнят вопросы, вызывающие затруднения и могут сами предложить дополнительную или разъясняющую информацию;
- системы распознавания речи, понимающие естественный язык, еще более расширяют диапазон взаимодействия с компьютером.

Технологию мультимедиа составляют специальные аппаратные и программные средства.

Для построения мультимедиасистемы необходима дополнительная аппаратная поддержка: аналого-цифровые и цифроаналоговые преобразователи для перевода аналоговых аудио- и видеосигналов в цифровой эквивалент и обратно, видеопроцессоры для преобразования обычных телевизионных сигналов к виду, воспроизводимому электронно-лучевой трубкой дисплея, декодеры для взаимного преобразования телевизионных стандартов, специальные интегральные схемы для сжатия данных в файлы допустимых размеров и т. д.

Аппаратные средства мультимедиа:

- средства звукозаписи (звуковые платы, микрофоны);
- средства звуковоспроизведения (усилитель, колонки, акустические системы, наушники и гарнитуры);
- манипуляторы (компьютерные мыши, джойстики, миди-клавиатуры);
- средства «виртуальной реальности» (перчатки, очки, шлемы виртуальной реальности, используемые в играх);
- носители информации (*CD*, *DVD* и *HDD*);
- средства передачи (мини-видеокамеры, цифровые фотоаппараты);
- средства записи (приводы CD/*DVD-ROM*, *CDRW*/*DVD+RW*, *TV*- и *FM*-тюнеры);
- средства обработки изображения (платы видеомонтажа, клавиатуры, графические акселераторы).
- компьютер, телевизор, средства для получения и удобного восприятия информации и др.

С точки зрения технических средств на рынке представлены как полностью укомплектованные мультимедиакомпьютеры, так и отдельные комплектующие и подсистемы (*Multimedia Upgrade Kit*), включающие в себя звуковые карты, приводы компакт-дисков, джойстики, микрофоны, акустические системы.

Программные средства мультимедиа складываются из трех компонентов:

1) системные программные средства;
2) инструментальные программные средства;
3) прикладные программные средства.

Для персональных компьютеров класса *IBM PC* утвержден специальный стандарт MPC, определяющий минимальную конфигурацию аппаратных средств для воспроизведения мультимедиа-продуктов. Для оптических дисков *CD-ROM* разработан международный стандарт (*ISO* 9660).

3.2. Геоинформационные технологии

Геоинформационные технологии можно определить как совокупность программно-технологических, методических средств получения новых видов информации об окружающем мире. Они предназначены для повышения эффективности: процессов управления, хранения и представления информации, обработки и поддержки принятия решений. Это заключается во внедрении геоинформационных технологий в науку, производство, образование и применение в практической деятельности получаемой информации об окружающей реальности.

Геоинформационные технологии являются новыми информационными технологиями, направленными на достижение различных целей, включая информатизацию производственно-управленческих процессов. Особенностью геоинформационных систем (далее — ГИС) является то, что как информационные системы они являются результатом эволюции этих систем и поэтому включают в себя основы построения и функционирования информационных систем. ГИС как система включает множество взаимосвязанных элементов, каждый из которых связан прямо или косвенно с каждым другим элементом, а два любые подмножества этого множества не могут быть независимыми, не нарушая целостность, единство системы.

Еще одной особенностью ГИС является то, что она является интегрированной информационной системой. Интегрированные системы построены на принципах интеграции технологий различных систем. Они зачастую применяются настолько в разных областях, что их название часто не определяет все их возможности и функции. По этой причине не следует связывать ГИС с решением задач только геодезии или географии. «Гео» в названии геоинформационных систем и технологий определяет объект исследований, а не предметную область использования этих систем.

Интеграция ГИС с другими информационными системами порождает их многоаспектность. В ГИС осуществляется комплексная

обработка информации от сбора данных до ее хранения, обновления и представления, поэтому следует рассмотреть ГИС с различных позиций.

Как *системы управления* ГИС предназначены для обеспечения процесса принятия решений по оптимальному управлению землями и ресурсами, городским хозяйством, организации транспорта и розничной торговли, использованию океанов или других пространственных объектов. В отличие от информационных систем, в ГИС появляется множество новых технологий пространственного анализа данных, объединенных с технологиями электронного офиса и оптимизации решений на этой основе. В силу этого ГИС является эффективным методом преобразования и синтеза разнообразных данных для задач управления.

Как *геосистемы* ГИС интегрируют технологии сбора информации таких систем, как: географические информационные системы, системы картографической информации, автоматизированные системы картографирования, автоматизированные фотограмметрические системы, земельные информационные системы, автоматизированные кадастровые системы и т. п.

Как *системы баз данных* ГИС характерны широким набором данных, собираемых с помощью разных методов и технологий. При этом следует подчеркнуть, что они объединяют в себе возможности текстовых и графических баз данных.

Как *системы моделирования* ГИС использует максимальное количество методов и процессов моделирования, применяемых в других информационных системах и в первую очередь в САПР.

Как *системы получения проектных решений* ГИС во многом используют концепции и методы автоматизированного проектирования и решают ряд специальных проектных задач, которые в типовом автоматизированном проектировании не встречаются.

Как *системы представления информации* ГИС являются развитием автоматизированных систем документационного обеспечения с использованием современных технологий мультимедиа. Они обладают средствами деловой графики и статистического анализа и дополнительно к этому средствами тематического картографирования. Именно эффективность последнего обеспечивает разнообразное решение задач в разных отраслях при использовании интеграции данных на основе картографической информации.

Как *прикладные системы* ГИС не имеют себе равных по широте, так как применяются в транспорте, навигации, геологии, географии, военном деле, топографии, экономике, экологии и т. д.

Как *системы массового использования* ГИС позволяют использовать картографическую информацию на уровне деловой графики, что делает их доступными любому школьнику или бизнесмену, а не только специалисту географу. Именно поэтому принятие мно-

гих решений на основе ГИС-технологий не сводится к созданию карт, а лишь использует картографические данные.

Организация данных в ГИС. Тематические данные хранятся в ГИС в виде таблиц, поэтому проблем с их хранением и организацией в базах данных не возникает. Наибольшие проблемы представляет хранение и визуализация графических данных.

Основным классом данных ГИС являются координатные данные, содержащие геометрическую информацию и отражающие пространственный аспект. Основные типы координатных данных: точка (узлы, вершины), линия (незамкнутая), контур (замкнутая линия), полигон (ареал, район). На практике для построения реальных объектов используют большее число данных (например, висячий узел, псевдоузел, нормальный узел, покрытие, слой и др.). На рис. 3.1 показаны основные из рассмотренных элементов координатных данных [15].

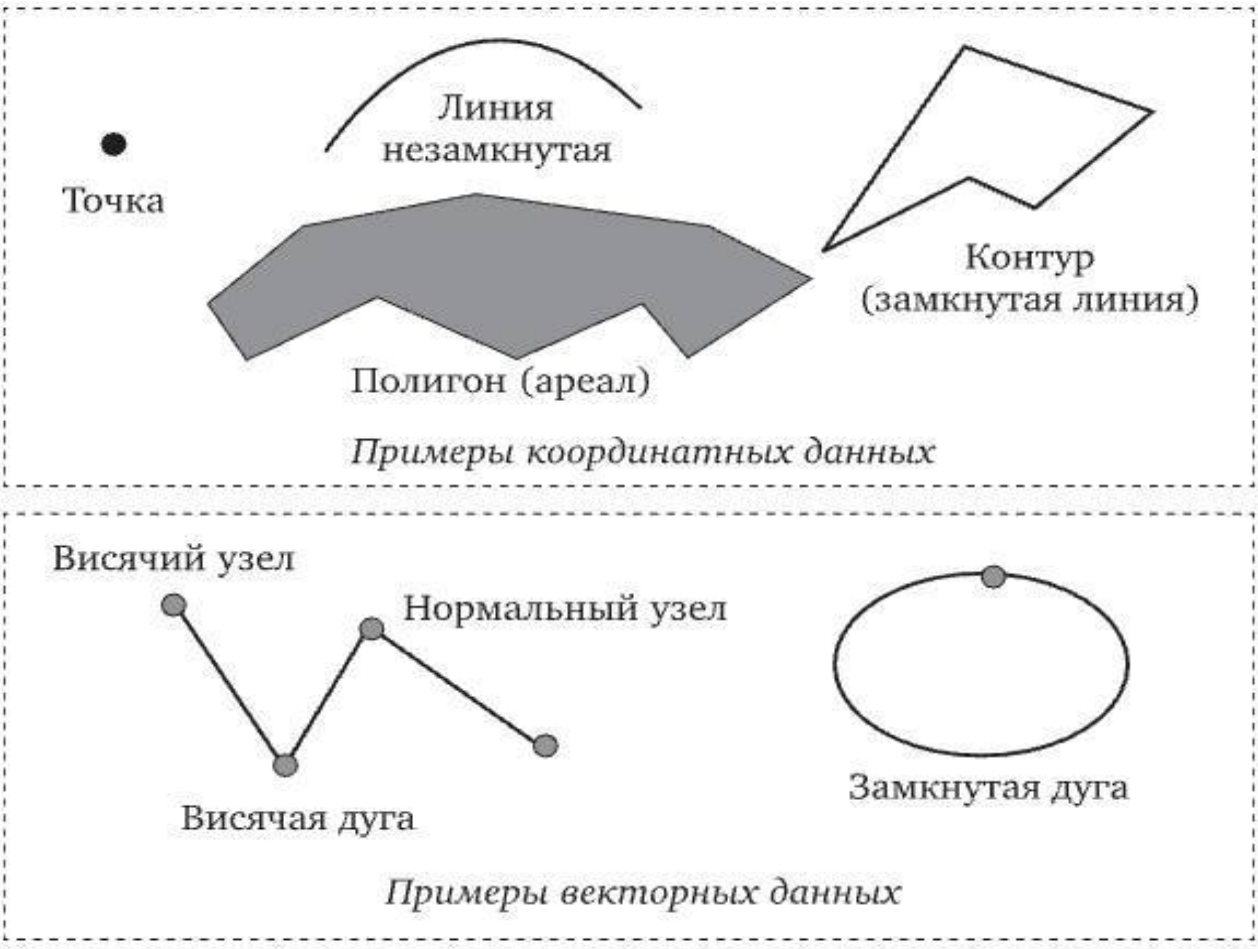


Рис. 3.1. **Основные элементы координатных данных**

Рассмотренные типы данных имеют большее количество разнообразных связей, которые можно условно разделить на три группы:

- взаимосвязи для построения сложных объектов из простых элементов;
- взаимосвязи, вычисляемые по координатам объектов;
- взаимосвязи, определяемые с помощью специального описания и семантики при вводе данных.

В общем случае модели пространственных (координатных) данных могут иметь векторное или растровое (ячеистое) представление, содержать или не содержать топологические характеристики. Этот подход позволяет классифицировать модели по трем типам: растровая модель; векторная нетопологическая модель; векторная топологическая модель. Все эти модели взаимно преобразуемы. Тем не менее при получении каждой из них необходимо учитывать их особенности. В ГИС форме представления координатных данных соответствуют два основных подкласса моделей: *векторные* и *растровые* (*ячеистые* или *мозаичные*). Возможен класс моделей, которые содержат характеристики как векторов, так и мозаик. Они называются *гибридными моделями*.

Графическое представление какой-либо ситуации на экране компьютера подразумевает отображение на экране различных графических образов. Сформированный графический образ на экране ЭВМ состоит из двух различных с точки зрения среды хранения частей — графической «подложки» или графического фона и других графических объектов. По отношению к этим другим графическим образам «образ-подложка» является «площадным», или пространственным двухмерным изображением. Основной проблемой при реализации геоинформационных приложений является трудность формализованного описания конкретной предметной области и ее отображения на электронной карте.

Таким образом, геоинформационные технологии предназначены для широкого внедрения в практику методов и средств информационного взаимодействия над пространственно-временными данными, представляемыми в виде системы электронных карт, и предметно-ориентированных сред обработки разнородной информации для различных категорий пользователей.

Рассмотрим более подробно основные графические модели.

Векторные модели широко применяются в ГИС. Они строятся на векторах, занимающих часть пространства, в отличие от занимающих все пространство растровых моделей. Это определяет их основное преимущество — требование на порядки меньшей памяти для хранения и меньших затрат времени на обработку и представление, а главное — более высокая точность позиционирования и представления данных. При построении векторных моделей объекты создаются путем соединения точек прямыми линиями, дугами окружностей, полилиниями. Площадные объекты — ареалы задаются наборами линий.

Векторные модели используются преимущественно в транспортных, коммунальных, маркетинговых приложениях ГИС. Системы ГИС, работающие в основном с векторными моделями, получили название векторных ГИС. В реальных ГИС имеют дело не с абстрактными линиями и точками, а с объектами, содержащими ли-

нии и ареалы, занимающими пространственное положение, а также со сложными взаимосвязями между ними. Поэтому полная векторная модель данных ГИС отображает пространственные данные как совокупность следующих основных частей: геометрические (метрические) объекты (точки, линии и полигоны); атрибуты — признаки, связанные с объектами; связи между объектами. Векторные модели (объектов) используют в качестве элементарной модели последовательность координат, образующих линию. Линией называют границу, сегмент, цепь или дугу. Основные типы координатных данных в классе векторных моделей определяются через базовый элемент линия следующим образом. Точка определяется как выродившаяся линия нулевой длины, линия — как линия конечной длины, а площадь представляется последовательностью связанных между собой отрезков. Каждый участок линии может являться границей для двух ареалов либо двух пересечений (узлов). Отрезок общей границы между двумя пересечениями (узлами) имеет разные названия, которые являются синонимами в предметной области ГИС. Специалисты по теории графов предпочитают слову «линия» термин «ребро», а для обозначения пересечения употребляют термин «вершина». Национальным стандартом США официально санкционирован термин «цепь». В некоторых системах (*Arclnfo*, *GeoDraw*) используется термин «дуга». В отличие от обычных векторов в геометрии, дуги имеют свои атрибуты. Атрибуты дуг обозначают полигоны по обе стороны от них. По отношению к последовательному кодированию дуги эти полигоны именуются левый и правый. Понятие дуги (цепи, ребра) является фундаментальным для векторных ГИС.

Векторные модели получают разными способами. Один из наиболее распространенных — векторизация сканированных (растровых) изображений.

Векторизация — процедура выделения векторных объектов с растрового изображения и получение их в векторном формате. Для векторизации необходимо высокое качество (отчетливые линии и контуры) растровых образов. Чтобы обеспечить требуемую четкость линий иногда приходится заниматься улучшением качества изображения.

При векторизации возможны ошибки, исправление которых осуществляется в два этапа:

1) корректировка растрового изображения до его векторизации;
2) корректировка векторных объектов.

Векторные модели с помощью дискретных наборов данных отображают непрерывные объекты или явления. Следовательно, можно говорить о векторной дискретизации. При этом векторное представление позволяет отразить большую пространственную изменчивость для одних районов, чем для других, по сравнению с растровым представлением, что обусловлено более четким показом

границ и их меньшей зависимостью от исходного образа (изображения), чем при растровом отображении. Это типично для социальных, экономических, демографических явлений, изменчивость которых в ряде районов более интенсивна.

Некоторые объекты являются векторными по определению, например границы соответствующего земельного участка, границы районов и т. д. Поэтому векторные модели обычно используют для сбора данных координатной геометрии (топографические записи), данных об административно-правовых границах и т. п.

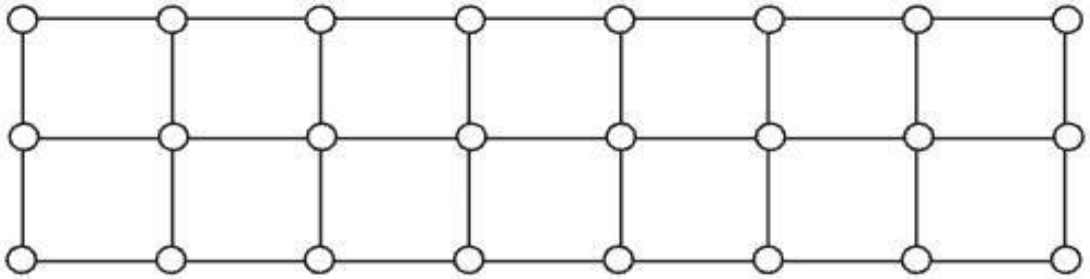
Особенности векторных моделей: в векторных форматах набор данных определен объектами базы данных. Векторная модель может организовывать пространство в любой последовательности и дает «произвольный доступ» к данным. В ней легче осуществляются операции с линейными и точечными объектами, например, анализ сети — разработка маршрутов движения по сети дорог, замена условных обозначений. В растровых форматах точечный объект должен занимать целую ячейку. Это создает ряд трудностей, связанных с соотношением размеров растра и размера объекта.

Что касается точности векторных данных, то здесь можно говорить о преимуществе векторных моделей перед растровыми, так векторные данные могут кодироваться с любой мыслимой степенью точности, которая ограничивается лишь возможностями метода внутреннего представления координат. Обычно для представления векторных данных используется 8 или 16 десятичных знаков (одинарная или двойная точность). Только некоторые классы данных, получаемых в процессе измерений, соответствуют точности векторных данных: это данные, полученные точной съемкой (координатная геометрия); карты небольших участков, составленные по топографическим координатам и политические границы, определенные точной съемкой.

Не все природные явления имеют характерные четкие границы, которые можно представить в виде математически определенных линий. Это обусловлено динамикой явлений или способами сбора пространственной информации. Почвы, типы растительности, склоны, место обитания диких животных — все эти объекты не имеют четких границ. Обычно линии на карте имеют толщину 0,4 мм и, как часто считается, отражают неопределенность положения объекта. В растровой системе эта неопределенность задается размером ячейки. Поэтому следует помнить, что в ГИС действительное представление о точности дают размер растровой ячейки и неопределенность положения векторного объекта, а не точность координат. Для анализа связей в векторных моделях необходимо рассмотреть их топологические свойства, т. е. рассмотреть топологические модели, которые являются разновидностью векторных моделей данных.

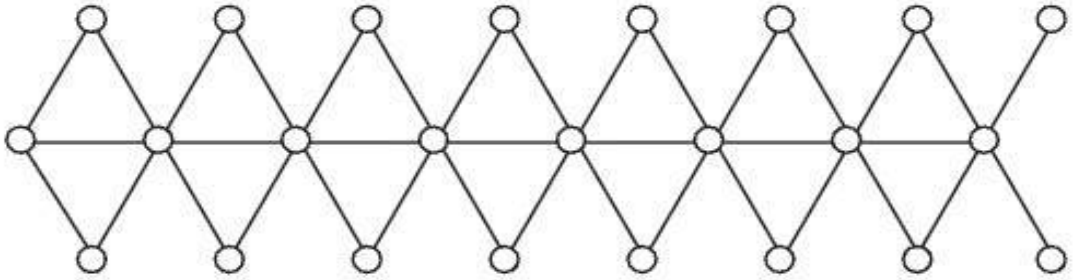
В ***растровых моделях*** дискретизация осуществляется наиболее простым способом — весь объект (исследуемая территория) ото-

бражается в пространственные ячейки, образующие регулярную сеть. Каждой ячейке растровой модели соответствует одинаковый по размерам, но разный по характеристикам (цвет, плотность) участок поверхности. Ячейка модели характеризуется одним значением, являющейся средней характеристикой участка поверхности. Эта процедура называется *пикселизацией*. Растровые модели делятся на *регулярные*, *нерегулярные* и *вложенные* (рекурсивные или иерархические) мозаики. Плоские регулярные мозаики бывают трех типов: квадрат (рис. 3.2), треугольник и шестиугольник (рис. 3.3).



Регулярная прямоугольная решетка

Рис. 3.2. **Мозаика-квадрат**



Регулярная треугольная решетка

Рис. 3.3. **Мозаика-треугольник и шестиугольник**

Квадратная форма удобна при обработке больших объемов информации, треугольная — для создания сферических поверхностей. В качестве нерегулярных мозаик используют треугольные сети неправильной формы (*Triangulated Irregular Network — TIN*) и полигоны Тиссена (рис. 3.4). Они удобны для создания цифровых моделей отметок местности по заданному набору точек.

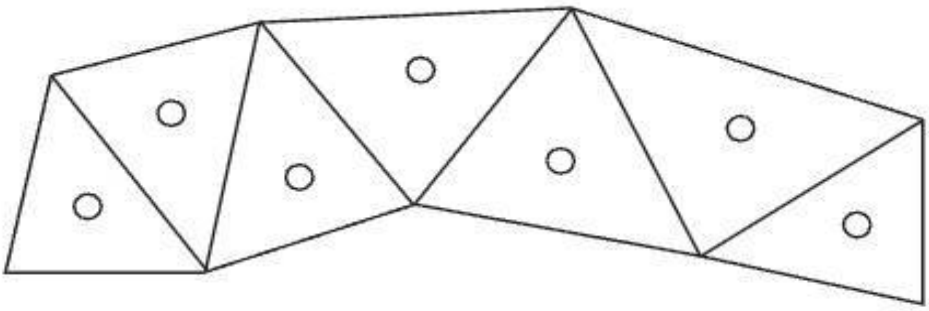


Рис. 3.4. **Полигоны Тиссена**

Таким образом, векторная модель содержит информацию о местоположении объекта, а растровая — о том, что расположено в той

или иной точке объекта. Векторные модели относятся к бинарным или квазибинарным.

Если векторная модель дает информацию о том, где расположен тот или иной объект, то растровая — информацию о том, что расположено в той или иной точке территории. Это определяет основное назначение растровых моделей — непрерывное отображение поверхности. В растровых моделях в качестве атомарной модели используют двухмерный элемент пространства — пиксель (ячейка). Упорядоченная совокупность атомарных моделей образует растр, который, в свою очередь, является моделью карты или геообъекта. Векторные модели относятся к бинарным или квазибинарным. Растровые позволяют отображать полутона и цветовые оттенки. Как правило, каждый элемент растра или каждая ячейка должны иметь лишь одно значение плотности или цвета. Это применимо не для всех случаев. Например, когда граница двух типов покрытий может проходить через центр элемента растра, элементу дается значение, характеризующее большую часть ячейки или ее центральную точку.

Ряд систем позволяет иметь несколько значений для одного элемента растра. Для растровых моделей существует ряд характеристик: разрешение, значение, ориентация, зоны, положение.

Разрешение — минимальный линейный размер наименьшего участка отображаемого пространства (поверхности), отображаемый одним пикселем. Пиксели обычно представляют собой прямоугольники или квадраты, реже используются треугольники и шестиугольники. Более высоким разрешением обладает растр с меньшим размером ячеек. Высокое разрешение подразумевает обилие деталей, множество ячеек, минимальный размер ячеек.

Значение — элемент информации, хранящийся в элементе растра (пикселе). Поскольку при обработке применяют типизированные данные, т. е. необходимость определить типы значений растровой модели. Тип значений в ячейках растра определяется как реальным явлением, так и особенностями ГИС. В частности, в разных системах можно использовать разные классы значений: целые числа, действительные (десятичные) значения, буквенные значения. Целые числа могут служить характеристиками оптической плотности или кодами, указывающими на позицию в прилагаемой таблице или легенде. Например, возможна следующая легенда, указывающая наименование класса почв: 0 — пустой класс, 1 — суглинистые, 2 — песчаные, 3 — щебнистые и т. п.

Ориентация — угол между направлением на север и положением колонок растра.

Зона растровой модели включает соседствующие друг с другом ячейки, имеющие одинаковое значение. Зоной могут быть отдельные объекты, природные явления, ареалы типов почв, элементы гидрографии и т. п. Для указания всех зон с одним и тем же значением используют понятие «класс зон». Естественно, что не во всех слоях

изображения могут присутствовать зоны. Основные характеристики зоны — ее значение и положение.

Буферная зона — зона, границы которой удалены на известное расстояние от любого объекта на карте. Буферные зоны различной ширины могут быть созданы вокруг выбранных объектов на базе таблиц сопряженных характеристик.

Положение обычно задается упорядоченной парой координат (номер строки и номер столбца), которые однозначно определяют положение каждого элемента отображаемого пространства в растре. Проводя сравнение векторных и растровых моделей, отметим удобство векторных для организации и работы со взаимосвязями объектов. Тем не менее, используя простые приемы, например, включая взаимосвязи в таблицы атрибутов, можно организовать взаимосвязи и в растровых системах.

Необходимо остановиться на вопросах *точности* отображения в растровых моделях. В растровых форматах в большинстве случаев неясно, относятся ли координаты к центральной точке пикселя или к одному из его углов. Поэтому точность привязки элемента растра определяют как 1/2 ширины и высоты ячейки.

Растровые модели имеют следующие *достоинства*:

- растр не требует предварительного знакомства с явлениями, данные собираются с равномерно расположенной сети точек, что позволяет в дальнейшем на основе статистических методов обработки получать объективные характеристики исследуемых объектов. Благодаря этому растровые модели могут использоваться для изучения новых явлений, о которых не накоплен материал. В силу простоты этот способ получил наибольшее распространение;
- растровые данные проще для обработки по параллельным алгоритмам и этим обеспечивают более высокое быстродействие по сравнению с векторными;
- некоторые задачи, например создание буферной зоны, намного проще решать в растровом виде;
- многие растровые модели позволяют вводить векторные данные, в то время как обратная процедура весьма затруднительна для векторных моделей;
- процессы растеризации намного проще алгоритмически, чем процессы векторизации, которые зачастую требуют экспертных решений.

Цифровая карта может быть организована в виде множества слоев (покрытий или карт подложек). Слои в ГИС представляют набор цифровых картографических моделей, построенных на основе объединения (типизации) пространственных объектов, имеющих общие функциональные признаки. Совокупность слоев образует интегрированную основу графической части ГИС. Пример слоев интегрированной ГИС представлен на рис. 3.5.

Рис. 3.5. **Пример слоев интегрированной ГИС**

Важным моментом при проектировании ГИС является **размерность модели**. Применяют двухмерные модели координат (2*D*) и трехмерные (3*D*). Двухмерные модели используются при построении карт, а трехмерные — при моделировании геологических процессов, проектировании инженерных сооружений (плотин, водохранилищ, карьеров и др.), моделировании потоков газов и жидкостей.

Существует два типа трехмерных моделей:

1) псевдотрехмерные, когда фиксируется третья координата;
2) истинное трехмерное представление.

Большинство современных ГИС осуществляют комплексную обработку информации:

- сбор первичных данных;
- накопление и хранение информации;
- различные виды моделирования (семантическое, имитационное, геометрическое, эвристическое);
- автоматизированное проектирование;
- документационное обеспечение.

Множество задач, возникающих в жизни, привело к созданию различных ГИС, которые могут ***классифицироваться по следующим признакам***:

1) по *функциональным возможностям*:

- полнофункциональные ГИС общего назначения,
- специализированные ГИС ориентированы на решение конкретной задачи в какой-либо предметной области,
- информационно-справочные системы для домашнего и информационно-справочного пользования.

Функциональные возможности ГИС определяются также *архитектурным принципом* их построения:

- закрытые системы — не имеют возможностей расширения, они способны выполнять только тот набор функций, который однозначно определен на момент покупки,
- открытые системы отличаются легкостью приспособления, возможностями расширения, так как могут быть достроены самим пользователем при помощи специального аппарата (встроенных языков программирования);

2) *пространственному (территориальному) охвату*:

- глобальные (планетарные),
- общенациональные,
- региональные,
- локальные (в том числе муниципальные);

3) *проблемно-тематической ориентации*:

- общегеографические,
- экологические и природопользовательские,
- отраслевые (водных ресурсов, лесопользования, геологические, туризма и т. д.);

4) *способу организации географических данных*:

- векторные,
- растровые,
- векторно-растровые ГИС.

В качестве ***источников данных*** для формирования ГИС служат:

- *картографические материалы* (топографические и общегеографические карты, карты административно-территориального деления, кадастровые планы и др.). Сведения, получаемые с карт, имеют территориальную привязку, поэтому их удобно использовать в качестве базового слоя ГИС. Если нет цифровых карт на исследуемую территорию, тогда графические оригиналы карт преобразуются в цифровой вид;
- *данные дистанционного зондирования* (далее — ДДЗ) все шире используются для формирования баз данных ГИС. К ДДЗ прежде всего относят материалы, получаемые с космических носителей. Для дистанционного зондирования применяют разнообразные технологии получения изображений и передачи их на Землю, носители съемочной аппаратуры (космические аппараты и спутники) размещают на разных орбитах, оснащают разной аппаратурой. Благодаря этому получают снимки, отличающиеся разным уровнем обзорности и детальности отображения объектов природной среды в разных диапазонах спектра (видимый и ближний инфракрасный, тепловой инфракрасный и радиодиапазон). Все это обуславливает широкий спектр экологических задач, решаемых с применением ДДЗ. К методам дистанционного зондирования относятся аэро- и наземные съемки и другие неконтактные методы, например гидроакустические съемки рельефа морского дна. Материалы таких

съемок обеспечивают получение как количественной, так и качественной информации о различных объектах природной среды;

- *материалы полевых изысканий территорий* включают данные топографических, инженерно-геодезических изысканий, кадастровой съемки, геодезические измерения природных объектов, выполняемые нивелирами, теодолитами, электронными тахеометрами, *GPS*-приемниками, а также результаты обследования территорий с применением геоботанических и других методов, например, исследования по перемещению животных, анализ почв и др.;
- *статистические данные* содержат данные государственных статистических служб по самым разным отраслям народного хозяйства, а также данные стационарных измерительных постов наблюдений (гидрологические и метеорологические данные, сведения о загрязнении окружающей среды и т. д.));
- *литературные данные* (справочные издания, книги, монографии и статьи, содержащие разнообразные сведения по отдельным типам географических объектов).

В ГИС редко используется только один вид данных, чаще всего это сочетание разнообразных данных на какую-либо территорию.

Основные области использования ГИС:

- электронные карты;
- городское хозяйство;
- государственный земельный кадастр;
- экология;
- дистанционное зондирование;
- экономика;
- специальные системы военного назначения.

На практике наиболее хорошо себя зарекомендовали для работы с мелкомасштабными «природными» картами (геология, сельское хозяйство, навигация, экология и т. п.) такие ГИС, как *ArcInfo* и *ArcView GIS*. Обе системы разработаны американской компанией *ESRI* (www.esri.com, www.dataplus.ru) и весьма распространены в мире.

Из относительно простых западных ГИС, которые начинали свою родословную с анализа территорий в объеме, необходимом для бизнеса и относительно простых применений, можно назвать систему *MapInfo*, которая также распространена в мире весьма широко. Эта система очень быстро прогрессирует и сегодня может составить конкуренцию самым развитым ГИС.

Корпорацией *Intergraph* (www.intergraph.com) поставляется ГИС *MGE*, базирующаяся на основе AutoCAD-подобной системы *MicroStation*, выпускаемой в свою очередь компанией *Bently*. Система *MGE* представляет собой целое семейство различных программных продуктов, помогающих решать набольшее множество задач, существующих в области геоинформатики.

Все указанные продукты имеют и интернет-ГИС-серверы, позволяющие публиковать цифровые карты в Интернете. Правда, приходится говорить только о вьюерах, поскольку обеспечить сегодня редактирование топологических карт со стороны удаленного клиента Интернета нельзя по причине недостаточной развитости как ГИС-, так и интернет-технологий.

Буквально недавно вышла на рынок ГИС и *Microsoft*, подтвердив тем самым, что ГИС станет в ближайшем будущем такой системой, которую должен иметь на своем компьютере всякий мало-мальски уважающий себя пользователь, как он имеет сегодня у себя *Excel* или *Word*. *Microsoft* выпустила продукт *MapPoint* (*Microsoft MapPoint* 2000 *Business Mapping Software*), который войдет в состав *Office* 2000. Эта компонента офисного продукта будет ориентирована в основном на бизнес-планирование и анализ.

Повторением концепции *ArcInfo*, но сильно уступающей последней по функциональной полноте является отечественная система *GeoDraw*, разработанная в ЦГИ ИГРАН (г. Москва). Возможности ее ограничены сегодня в основном мелкомасштабными картами. С нашей точки зрения, значительно «сильнее» здесь выглядит «старейшина» отечественной геоинформатики — ГИС *Sinteks ABRIS*. В последней хорошо представлены функции по анализу пространственной информации.

В геологии сильны позиции ГИС ПАРК (Ланэко, г. Москва), в которой также реализованы уникальные методы моделирования соответствующих процессов.

Наиболее «продвинутыми» в области представления и дежурства крупномасштабных насыщенных карт городов и генеральных планов крупных предприятий можно считать две отечественные системы: *GeoCosm* (ГЕОИД, г. Геленджик) и «ИнГео» (ЦСИ «Интегро», г. Уфа, www.integro.ru). Эти системы — одни из самых молодых и потому разрабатывались сразу с использованием самых современных технологий. А систему «ИнГео» разрабатывали даже не столько геодезисты, сколько специалисты, относящие себя к профессионалам в области имитационного моделирования и кадастровых систем.

В целом в России едва ли не в каждой организации создают свою ГИС. Однако, этот процесс — весьма непростой, и вероятность его завершения неудачно несравненно более высока, чем вероятность беспроблемной реализации, не говоря уже о возможности выхода коммерческого продукта, допускающего отчуждение от разработчиков.

3.3. Технологии защиты информации

Наряду с позитивным влиянием на все стороны человеческой деятельности широкое внедрение информационных технологий

привело к появлению новых угроз безопасности людей. Это связано с тем обстоятельством, что информация, создаваемая, хранимая и обрабатываемая средствами вычислительной техники, стала определять действия большей части людей и технических систем. В связи с этим резко возросло количество предпосылок к нанесению ущерба, связанных с хищением информации, так как воздействовать на любую систему (социальную, биологическую или техническую) с целью ее уничтожения, снижения эффективности функционирования или воровства ее ресурсов (денег, товаров, оборудования) возможно только в том случае, когда известна информация о ее структуре и принципах функционирования.

Все ***виды информационных угроз*** можно разделить на две большие группы:

- отказы и нарушения работоспособности программных и технических средств;
- преднамеренные угрозы, заранее планируемые злоумышленниками для нанесения вреда.

Выделяют следующие основные ***группы причин сбоев и отказов в работе компьютерных систем***:

- нарушения физической и логической целостности хранящихся в оперативной и внешней памяти структур данных, возникающие по причине старения или преждевременного износа их носителей;
- нарушения, возникающие в работе аппаратных средств из-за их старения или преждевременного износа;
- нарушения физической и логической целостности хранящихся в оперативной и внешней памяти структур данных, возникающие по причине некорректного использования компьютерных ресурсов;
- нарушения, возникающие в работе аппаратных средств из-за неправильного использования или повреждения, в том числе из-за неправильного использования программных средств;
- не устраненные ошибки в программных средствах, не выявленные в процессе отладки и испытаний, а также оставшиеся в аппаратных средствах после их разработки.

Помимо естественных способов выявления и своевременного устранения указанных выше причин, используют следующие ***специальные способы защиты информации от нарушений работоспособности компьютерных систем***:

- внесение структурной, временной, информационной и функциональной избыточности компьютерных ресурсов,
- защита от некорректного использования ресурсов компьютерной системы;
- выявление и своевременное устранение ошибок на этапах разработки программно-аппаратных средств.

Структурная избыточность компьютерных ресурсов достигается за счет резервирования аппаратных компонентов и машинных

носителей данных, организации замены отказавших и своевременного пополнения резервных компонентов. Структурная избыточность составляет основу остальных видов избыточности.

Внесение *информационной избыточности* выполняется путем периодического или постоянного (фонового) резервирования данных на основных и резервных носителях. Зарезервированные данные обеспечивают восстановление случайно или преднамеренно уничтоженной и искаженной информации. Для восстановления работоспособности компьютерной системы после появления устойчивого отказа кроме резервирования обычных данных должна заблаговременно резервироваться и системная информация, а также подготавливаться программные средства восстановления.

Функциональная избыточность компьютерных ресурсов достигается дублированием функций или внесением дополнительных функций в программно-аппаратные ресурсы вычислительной системы для повышения ее защищенности от сбоев и отказов, например периодическое тестирование и восстановление, а также самотестирование и самовосстановление компонентов компьютерной системы.

Защита от некорректного использования информационных ресурсов заключается в корректном функционировании программного обеспечения с позиции использования ресурсов вычислительной системы. Программа может четко и своевременно выполнять свои функции, но некорректно использовать компьютерные ресурсы из-за отсутствия всех необходимых функций (например, изолирование участков оперативной памяти для операционной системы и прикладных программ, защита системных областей на внешних носителях, поддержка целостности и непротиворечивости данных).

Выявление и устранение ошибок при разработке программно-аппаратных средств достигается путем качественного выполнения базовых стадий разработки на основе системного анализа концепции, проектирования и реализации проекта.

Однако основным видом угроз целостности и конфиденциальности информации являются преднамеренные угрозы, заранее планируемые злоумышленниками для нанесения вреда, которые можно разделить на две группы:

1) угрозы, реализация которых выполняется при постоянном участии человека;

2) угрозы, реализация которых после разработки злоумышленником соответствующих компьютерных программ выполняется этими программами без непосредственного участия человека.

Задачи по защите от угроз каждого вида одинаковы:

- запрещение несанкционированного доступа к ресурсам вычислительных систем;
- невозможность несанкционированного использования компьютерных ресурсов при осуществлении доступа;

- своевременное обнаружение факта несанкционированных действий, устранение их причин и последствий.

Основным способом запрещения несанкционированного доступа к ресурсам вычислительных систем является подтверждение подлинности пользователей и разграничение их доступа к информационным ресурсам, включающего следующие этапы:

- идентификация;
- установление подлинности (аутентификация);
- определение полномочий для последующего контроля и разграничения доступа к компьютерным ресурсам.

Идентификация необходима для указания компьютерной системе уникального идентификатора обращающегося к ней пользователя. Идентификатор может представлять собой любую последовательность символов и должен быть заранее зарегистрирован в системе администратора службы безопасности. В процессе регистрации заносится следующая информация:

- фамилия, имя, отчество (при необходимости другие характеристики пользователя);
- уникальный идентификатор пользователя;
- имя процедуры установления подлинности;
- эталонная информация для подтверждения подлинности (например, пароль);
- ограничения на используемую эталонную информацию (например, время действия пароля);
- полномочия пользователя по доступу к компьютерным ресурсам.

Установление подлинности (аутентификация) заключается в проверке истинности полномочий пользователя.

Общая схема идентификации и установления подлинности пользователя представлена на рис. 3.6.

Для особо надежного опознания для идентификации используются технические средства, определяющие индивидуальные характеристики человека (голос, отпечатки пальцев, структура значка). Однако такие методы требуют значительных затрат и поэтому используются редко. Наиболее массово используемыми являются парольные методы проверки подлинности пользователей, которые можно разделить на две группы:

- простые пароли;
- динамически изменяющиеся пароли.

Простой пароль не изменяется от сеанса к сеансу в течение установленного времени его существования.

Во втором случае пароль изменяется по правилам, определяемым используемым методом. Выделяют следующие методы реализации динамически изменяющихся паролей:

- методы модификации простых паролей, например, случайная выборка символов пароля и одноразовое использование паролей;

- метод «запрос — ответ», основанный на предъявлении пользователю случайно выбираемых запросов из имеющегося массива;
- функциональные методы, основанные на использовании некоторой функции F с динамически изменяющимися параметрами (дата, время, день недели и др.), с помощью которой определяется пароль.

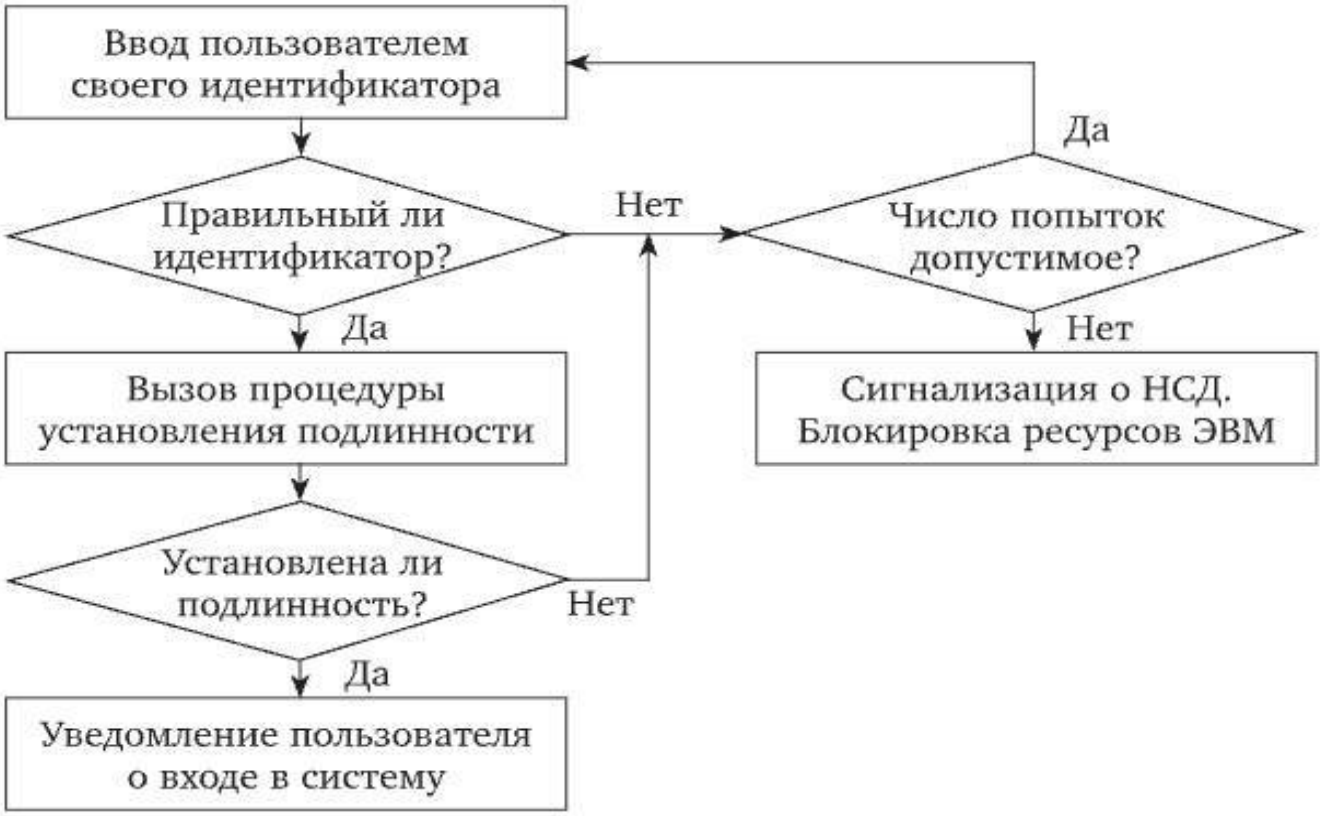


Рис. 3.6. **Схема идентификации и установления подлинности пользователя**

Для защиты от несанкционированного входа в компьютерную систему используются как общесистемные, так и специализированные программные средства защиты.

После идентификации и аутентификации пользователя система защиты должна определить его полномочия для последующего контроля санкционированного доступа к компьютерным ресурсам (разграничение доступа). В качестве *компьютерных ресурсов* рассматриваются:

- программы;
- внешняя память (файлы, каталоги, логические диски);
- информация, разграниченная по категориям в базах данных;
- оперативная память;
- время (приоритет) использования процессора;
- порты ввода-вывода;
- внешние устройства.

Различают следующие *виды прав пользователей по доступу к ресурсам*:

- всеобщее право (полное предоставление ресурса);

- функциональное или частичное право;
- временное право.

Наиболее распространенными способами разграничения доступа являются:

- разграничение доступа по спискам (пользователей или ресурсов);
- использование матрицы установления полномочий (строки матрицы — идентификаторы пользователей, столбцы — ресурсы компьютерной системы);
- разграничение доступа по уровням секретности и категориям (например, общий доступ, конфиденциально, секретно);
- парольное разграничение доступа.

Защита информации от исследования и копирования предполагает криптографическое закрытие защищаемых от хищения данных. Задачей криптографии является обратимое преобразование некоторого понятного исходного текста (открытого текста) в кажущуюся случайной последовательность некоторых знаков, часто называемых шифротекстом или криптограммой. В шифре выделяют два основных элемента — *алгоритм* и *ключ*. Алгоритм шифрования представляет собой последовательность преобразований обрабатываемых данных, зависящих от ключа шифрования. Ключ задает значения некоторых параметров алгоритма шифрования, обеспечивающих шифрование и дешифрование информации. В криптографической системе информация I и ключ K являются входными данными для шифрования (рис. 3.7) и дешифрования (рис. 3.8) информации. При похищении информации необходимо знать ключ и алгоритм шифрования.

По способу использования ключей различают два типа криптографических систем: симметрические и асимметрические.

Рис. 3.7. **Процесс шифрования**

Рис. 3.8. **Процесс дешифрования**

В *симметрических* (*одноключевых*) криптографических системах ключи шифрования и дешифрования либо одинаковы, либо легко выводятся один из другого.

В *асимметрических* (*двухключевых*, или системах с открытым ключом) криптографических системах ключи шифрования и дешифрования различаются таким образом, что с помощью вычислений нельзя вывести один ключ из другого.

Скорость шифрования в двухключевых криптографических системах намного ниже, чем в одноключевых. Поэтому асимметрические системы используют в двух случаях:

- для шифрования секретных ключей, распределенных между пользователями вычислительной сети;
- формирования цифровой подписи.

Одним из сдерживающих факторов массового применения методов шифрования является потребление значительных временных ресурсов при программной реализации большинства хорошо известных шифров (*DES*, *FEAL*, *REDOC*, *IDEA*, ГОСТ).

Одной из основных угроз хищения информации является угроза доступа к остаточным данным в оперативной и внешней памяти компьютера. Под ***остаточной информацией*** понимаются данные, оставшиеся в освободившихся участках оперативной и внешней памяти:

- оставшиеся после удаления файлов пользователя;
- оставшиеся после удаления временных файлов без ведома пользователя;
- находящиеся в неиспользуемых хвостовых частях последних кластеров, занимаемых файлами;
- оставшиеся в кластерах, освобожденных после уменьшения размеров файлов;
- оставшиеся после форматирования дисков.

Основным способом защиты от доступа к конфиденциальным остаточным данным является своевременное уничтожение данных в следующих областях памяти компьютера:

- в рабочих областях оперативной и внешней памяти, выделенных пользователю, после окончания им сеанса работы;
- в местах расположения файлов после выдачи запросов на их удаление.

Уничтожение остаточных данных может быть реализовано либо средствами операционных сред, либо с помощью специализированных программ. Использование специализированных программ (автономных или в составе системы защиты) обеспечивает гарантированное уничтожение информации.

Подсистема защиты от компьютерных вирусов (специально разработанных программ для выполнения несанкционированных действий) является одним из основных компонентов системы защиты информации и процесса ее обработки в вычислительных системах.

Выделяют три уровня защиты от компьютерных вирусов:

- защита от проникновения в вычислительную систему вирусов известных типов;

- углубленный анализ на наличие вирусов известных и неизвестных типов, преодолевших первый уровень защиты;
- защита от деструктивных действий и размножения вирусов, преодолевших первые два уровня.

Поиск и обезвреживание вирусов осуществляется как автономными *антивирусными программными средствами* (сканеры), так и в рамках *комплексных систем защиты информации*.

Среди транзитных сканеров, которые загружаются в оперативную память, наибольшей популярностью в нашей стране пользуются антивирусные программы *Aidstest* Дмитрия Лозинского и *DrWeb* Игоря Данилова. Эти программы просты в использовании и для детального ознакомления с руководством по каждой из них следует прочитать файл, поставляемый вместе с антивирусным средством.

Широкое внедрение в повседневную практику компьютерных сетей, их открытость, масштабность делают проблему защиты информации исключительно сложной. Выделяют две базовые подзадачи:

- обеспечение безопасности обработки и хранения информации в каждом из компьютеров, входящих в сеть;
- защита информации, передаваемой между компьютерами сети.

Решение первой задачи основано на многоуровневой защите автономных компьютерных ресурсов от несанкционированных и некорректных действий пользователей и программ, рассмотренных выше.

Безопасность информации при сетевом обмене данными требует также обеспечения их конфиденциальности и подлинности. Защита информации в процессе передачи достигается на основе защиты каналов передачи данных, а также криптографического закрытия передаваемых сообщений. В идеальном случае защита каналов передачи данных должна обеспечивать их защиту, как от нарушений работоспособности, так и несанкционированных действий (например, подключения к линиям связи). По причине большой протяженности каналов связи, а также возможной доступности их отдельных участков (например, при беспроводной связи) защита каналов передачи данных от несанкционированных действий экономически неэффективна, а в ряде случаев невозможна. Поэтому реально защита каналов передачи данных строится на основе защиты нарушений их работоспособности. На рис. 3.9 представлены цели и способы защиты передаваемых данных.

Ввиду актуальности проблемы защиты информации рынок насыщен значительным количеством средств защиты информации, которые можно классифицировать следующим образом:

- средства защиты от несанкционированного доступа (далее — НСД): средства авторизации, мандатное управление доступом, избирательное управление доступом, управление доступом на основе ролей, журналирование (так же называется аудит);

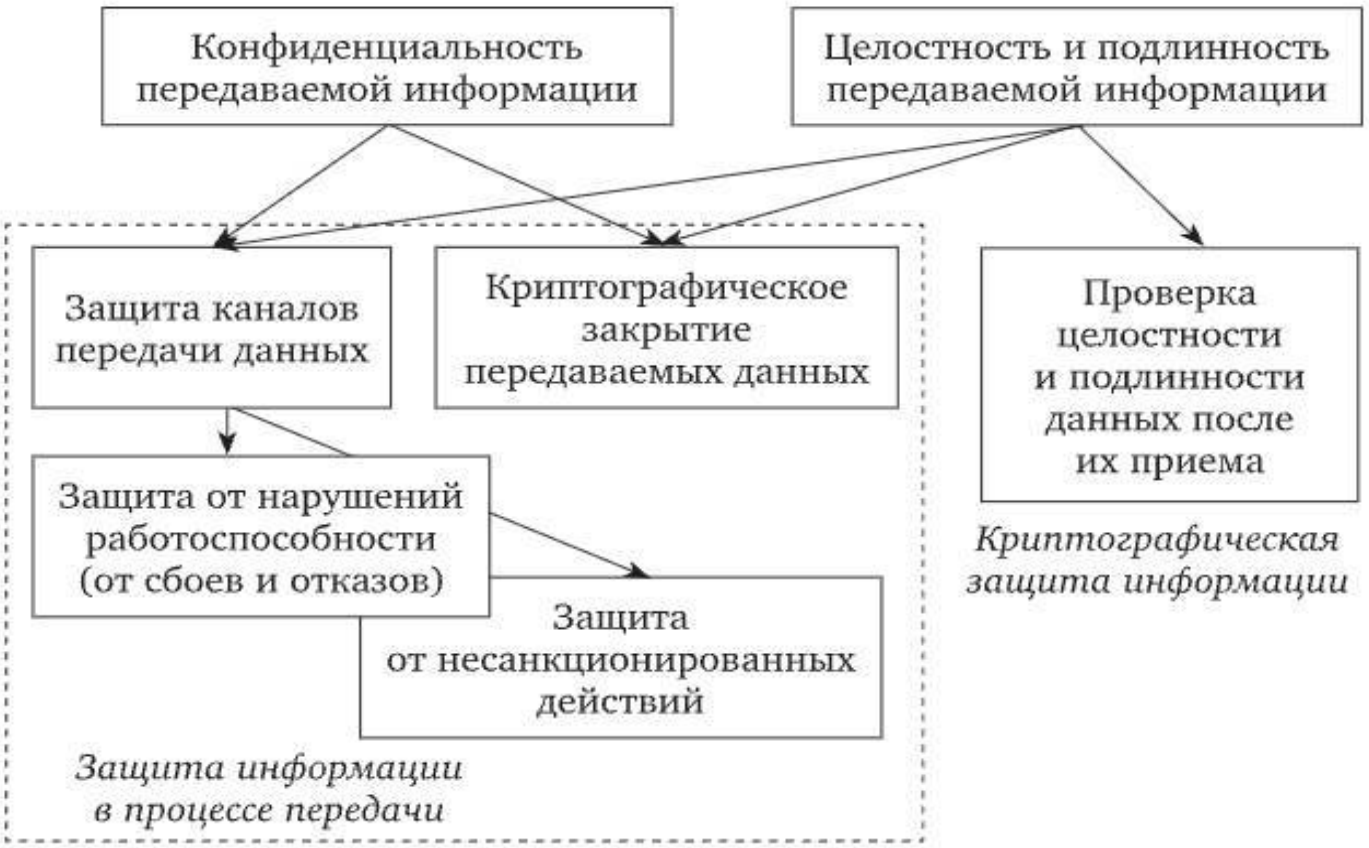


Рис. 3.9. **Цели и способы защиты передаваемых данных**

- системы анализа и моделирования информационных потоков (*CASE*-системы);
- системы мониторинга сетей: системы обнаружения и предотвращения вторжений (*IDS/IPS*), системы предотвращения утечек конфиденциальной информации (*DLP*-системы).
- анализаторы протоколов;
- антивирусные средства;
- межсетевые экраны;
- криптографические средства: шифрование, цифровая подпись;
- системы резервного копирования;
- системы бесперебойного питания: источники бесперебойного питания, резервирование нагрузки, генераторы напряжения;
- системы аутентификации: пароль, ключ доступа (физический или электронный), сертификат, биометрия;
- средства предотвращения взлома корпусов и краж оборудования;
- средства гарантированного уничтожения информации;
- средства контроля доступа в помещения;
- инструментальные средства анализа систем защиты: мониторинговый программный продукт.

3.4. CASE-технологии

Тенденции развития современных информационных технологий приводят к постоянному возрастанию сложности ИС, создаваемых

в различных областях экономики. Современные крупные проекты ИС характеризуются, как правило, следующими особенностями:

- сложность описания (достаточно большое количество функций, процессов, элементов данных и сложные взаимосвязи между ними), требующая тщательного моделирования и анализа данных и процессов;
- наличие совокупности тесно взаимодействующих компонентов (подсистем), имеющих свои локальные задачи и цели функционирования (например, традиционных приложений, связанных с обработкой транзакций и решением регламентных задач, и приложений аналитической обработки (поддержки принятия решений), использующих нерегламентированные запросы к данным большого объема);
- отсутствие прямых аналогов, ограничивающее возможность использования каких-либо типовых проектных решений и прикладных систем;
- необходимость интеграции существующих и вновь разрабатываемых приложений;
- функционирование в неоднородной среде на нескольких аппаратных платформах;
- разобщенность и разнородность отдельных групп разработчиков по уровню квалификации и сложившимся традициям использования тех или иных инструментальных средств;
- существенная временная протяженность проекта, обусловленная, с одной стороны, ограниченными возможностями коллектива разработчиков, а с другой — масштабами организации-заказчика и различной степенью готовности отдельных ее подразделений к внедрению ИС.

Перечисленные факторы способствовали появлению программно-технологических средств специального класса — *CASE*-средств, реализующих *CASE*-технологию создания и сопровождения ИС.

Термин «*CASE*» (*Computer Aided Software Engineering*) используется в настоящее время в весьма широком смысле. Первоначальное значение термина «*CASE*», ограниченное вопросами автоматизации разработки только лишь ПО, в настоящее время приобрело новый смысл, охватывающий процесс разработки сложных ИС в целом. Теперь под термином «*CASE*-средства» понимаются программные средства, поддерживающие процессы создания и сопровождения ИС, включая анализ и формулировку требований, проектирование прикладного ПО (приложений) и баз данных, генерацию кода, тестирование, документирование, обеспечение качества, конфигурационное управление и управление проектом, а также другие процессы. CASE-средства вместе с системным ПО и техническими средствами образуют полную среду разработки ИС.

Появлению *CASE*-технологии и *CASE*-средств предшествовали исследования в области методологии программирования. Программи-

рование обрело черты системного подхода с разработкой и внедрением языков высокого уровня, методов структурного и модульного программирования, языков проектирования и средств их поддержки, формальных и неформальных языков описаний системных требований и спецификаций и т. д. Кроме того, появлению *CASE*-технологии способствовали и такие факторы, как:

- подготовка аналитиков и программистов, восприимчивых к концепциям модульного и структурного программирования;
- широкое внедрение и постоянный рост производительности компьютеров, позволившие использовать эффективные графические средства и автоматизировать большинство этапов проектирования;
- внедрение сетевой технологии, предоставившей возможность объединения усилий отдельных исполнителей в единый процесс проектирования путем использования разделяемой базы данных, содержащей необходимую информацию о проекте.

CASE-технология представляет собой методологию проектирования ИС, а также набор инструментальных средств, позволяющих в наглядной форме моделировать предметную область, анализировать эту модель на всех этапах разработки и сопровождения ИС и разрабатывать приложения в соответствии с информационными потребностями пользователей. Большинство существующих *CASE*-средств основано на методологиях структурного (в основном) или объектно-ориентированного анализа и проектирования, использующих спецификации в виде диаграмм или текстов для описания внешних требований, связей между моделями системы, динамики поведения системы и архитектуры программных средств [16].

При использовании методологий структурного анализа появился ряд ограничений (сложность понимания, большая трудоемкость и стоимость использования, неудобство внесения изменений в проектные спецификации и т. д.). С самого начала *CASE*-технологии развивались с целью преодоления этих ограничений путем автоматизации процессов анализа и интеграции поддерживающих средств. Они обладают достоинствами и возможностями, перечисленными ниже.

1. ***Единый графический язык.*** *CASE*-технологии обеспечивают всех участников проекта, включая заказчиков, единым строгим, наглядным и интуитивно понятным графическим языком, позволяющим получать обозримые компоненты с простой и ясной структурой. При этом программы представляются двумерными схемами (которые проще в использовании, чем многостраничные описания), позволяющими заказчику участвовать в процессе разработки, а разработчикам — общаться с экспертами предметной области, разделять деятельность системных аналитиков, проектировщиков и программистов, облегчая им защиту проекта перед руководством, а также обеспечивая легкость сопровождения и внесения изменений в систему.

2. ***Единая БД проекта.*** Основа *CASE*-технологии — использование базы данных проекта (репозитория) для хранения всей информации о проекте, которая может разделяться между разработчиками в соответствии с их правами доступа. Содержимое репозитория включает не только информационные объекты различных типов, но и отношения между их компонентами, а также правила использования или обработки этих компонентов. Репозиторий может хранить свыше 100 типов объектов: структурные диаграммы, определения экранов и меню, проекты отчетов, описания данных, логика обработки, модели данных, их организации и обработки, исходные коды, элементы данных и т. п.

3. ***Интеграция средств.*** На основе репозитория осуществляется интеграция *CASE*-средств и разделение системной информации между разработчиками. При этом возможности репозитория обеспечивают несколько уровней интеграции: общий пользовательский интерфейс по всем средствам, передачу данных между средствами, интеграцию этапов разработки через единую систему представления фаз жизненного цикла, передачу данных и средств между различными платформами.

4. ***Поддержка коллективной разработки и управления проектом.*** *CASE*-технология поддерживает групповую работу над проектом, обеспечивая возможность работы в сети, экспорт-импорт любых фрагментов проекта для их развития и (или) модификации, а также планирование, контроль, руководство и взаимодействие, т. е. функции, необходимые в процессе разработки и сопровождения проектов. Эти функции также реализуются на основе репозитория. В частности, через репозиторий может осуществляться контроль безопасности (ограничения и привилегии доступа), контроль версий и изменений и др.

5. ***Макетирование.*** *CASE*-технология дает возможность быстро строить макеты (прототипы) будущей системы, что позволяет заказчику на ранних этапах разработки оценить, насколько она приемлема для будущих пользователей и устраивает его.

6. ***Генерация документации.*** Вся документация по проекту генерируется автоматически на базе репозитория (как правило, в соответствии с требованиями действующих стандартов). Несомненное достоинство *CASE*-технологии заключается в том, что документация всегда отвечает текущему состоянию дел, поскольку любые изменения в проекте автоматически отражаются в репозитории (известно, что при традиционных подходах к разработке ПО документация в лучшем случае запаздывает, а ряд модификаций вообще не находит в ней отражения).

7. ***Верификация проекта.*** *CASE*-технология обеспечивает автоматическую верификацию и контроль проекта на полноту и состоятельность на ранних этапах разработки, что влияет на успех разра-

ботки в целом — по статистическим данным анализа пяти крупных проектов фирмы *TRW* (США) ошибки проектирования и кодирования составляют соответственно 64 % и 32 % от общего числа ошибок, а ошибки проектирования в 100 раз труднее обнаружить на этапе сопровождения ПО, чем на этапе анализа требований.

8. ***Автоматическая генерация объектного кода.*** Генерация программ в машинном коде осуществляется на основе репозитория и позволяет автоматически построить до 85—90 % объектного кода или текстов на языках высокого уровня.

9. ***Сопровождение и реинжиниринг.*** Сопровождение системы в рамках *CASE*-технологии характеризуется сопровождением проекта, а не программных кодов. Средства реинжиниринга и обратного инжиниринга позволяют создавать модель системы из ее кодов и интегрировать полученные модели в проект, автоматически обновлять документацию при изменении кодов и т. п.

При использовании *CASE*-технологий изменяются все фазы жизненного цикла ИС, причем наибольшие изменения касаются фаз анализа и проектирования. В табл. 3.1 приведены основные изменения жизненного цикла ИС при использовании *CASE*-технологий по сравнению с традиционной технологией разработки.

Таблица 3.1

Сравнительная характеристика основных изменений жизненного цикла ИС

Традиционная технология разработки	**Разработка с помощью *CASE*-технологий**
Основные усилия — на кодирование и тестирование	Основные усилия — на анализ и проектирование
«Бумажные» спецификации	Быстрое итеративное макетирование
Ручное кодирование	Автоматическая генерация машинного кода
Тестирование ПО	Автоматический контроль проекта
Сопровождение программного кода	Сопровождение проекта

В табл. 3.2 приведены оценки трудозатрат по фазам жизненного цикла программного обеспечения (ПО) в зависимости от используемой технологии разработки.

Перейдем к характеристике современных *CASE*-систем, которые охватывают обширную область поддержки многочисленных технологий проектирования ИС: от простых средств анализа и документирования до полномасштабных средств автоматизации, покрывающих весь жизненный цикл ПО.

Перейдем к характеристике современных *CASE*-систем, которые охватывают обширную область поддержки многочисленных технологий проектирования ИС: от простых средств анализа и докумен-

тирования до полномасштабных средств автоматизации, покрывающих весь жизненный цикл ПО [16].

Таблица 3.2

Оценка трудозатрат по фазам жизненного цикла ПО в зависимости от используемой технологии разработки, %

Технология разработки	Анализ	Проектирование	Программирование	Тестирование
Традиционная	20	15	20	45
Структурная методология (вручную)	30	30	15	25
CASE-технология	40	40	5	15

В разряд *CASE*-средств попадают как относительно дешевые системы для персональных компьютеров с весьма ограниченными возможностями, так и дорогостоящие системы для неоднородных вычислительных платформ и операционных сред. Так, современный рынок программных средств насчитывает около 300 различных *CASE*-средств, наиболее мощные из которых так или иначе используются практически всеми ведущими западными фирмами.

Полный комплекс *CASE*-средств, обеспечивающий поддержку жизненного цикла ПО, содержит следующие ***компоненты***:

- репозиторий, являющийся основой *CASE*-средства. Он должен обеспечивать хранение версий проекта и его отдельных компонентов, синхронизацию поступления информации от различных разработчиков при групповой разработке, контроль метаданных на полноту и непротиворечивость;
- графические средства анализа и проектирования, обеспечивающие создание и редактирование иерархически связанных диаграмм (потоков данных, «сущность-связь» и др.), образующих модели ИС;
- средства разработки приложений, включая языки 4*GL* и генераторы кодов;
- средства конфигурационного управления;
- средства документирования;
- средства тестирования;
- средства управления проектом;
- средства реинжиниринга.

Все современные *CASE*-средства могут быть классифицированы по следующим ***признакам***:

- функциональной ориентации;
- применяемым методологиям и моделям систем и БД;
- степени интегрированности с СУБД;
- доступным платформам.

Классификация по функциональной ориентации представляет наибольший интерес и в основном совпадает с компонентным составом *CASE*-средств и включает следующие ***основные типы***:

- средства анализа (*Upper CASE*), предназначенные для построения и анализа моделей предметной области (*Design/IDEF*, *BPwin*);
- средства анализа и проектирования (*Middle CASE*), поддерживающие наиболее распространенные методологии проектирования и использующиеся для создания проектных спецификаций (*Vantage Team Builder*, *Designer/*2000, *Silverrun*, *PRO-IV*, *CASE.*Аналитик). Выходом таких средств являются спецификации компонентов и интерфейсов системы, архитектуры системы, алгоритмов и структур данных;
- средства проектирования БД, обеспечивающие моделирование данных и генерацию схем баз данных (как правило, на языке *SQL*) для наиболее распространенных СУБД. К ним относятся *ERwin*, *S-Designor* и *DataBase Designer* (*ORACLE*). Средства проектирования баз данных имеются также в составе *CASE*-средств *Vantage Team Builder*, *Designer/*2000, *Silverrun* и *PRO-*IV;
- средства разработки приложений. К ним относятся средства *4GL* (*Uniface*, *JAM*, *PowerBuilder*, *Developer/*2000, *New Era*, *SQLWindows*, *Delphi* и др.) и генераторы кодов, входящие в состав *Vantage Team Builder*, *PRO-*IV и частично — в *Silverrun*;
- средства реинжиниринга, обеспечивающие анализ программных кодов и схем баз данных и формирование на их основе различных моделей и проектных спецификаций. Средства анализа схем БД и формирования *ERD* входят в состав *Vantage Team Builder*, *PRO-*IV, *Silverrun*, *Designer/*2000, *ERwin* и *S-Designor*. В области анализа программных кодов наибольшее распространение получают объектно-ориентированные *CASE*-средства, обеспечивающие реинжиниринг программ на языке *C++* (*Rational Rose*, *Object Team*).

Российский рынок программного обеспечения располагает следующими наиболее развитыми ***CASE-средствами***:

- *Vantage Team Builder* (*Westmount I-CASE*);
- *Designer/*2000;
- *Silverrun*;
- *ERwin+BPwin*;
- *S-Designor*;
- *CASE.*Аналитик;
- *Rational Rose*.

Кроме того, на рынке постоянно появляются как новые для отечественных пользователей системы, так и новые версии и модификации перечисленных систем. Наибольший интерес представляет *CASE*-средство фирмы *Rational Software Corporation* (США) *Rational Rose*, которое предназначено для автоматизации этапов анализа и проектирования ПО, а также для генерации кодов на различных

языках и выпуска проектной документации. *Rational Rose* использует синтез-методологию объектно-ориентированного анализа и проектирования, основанную на подходах трех ведущих специалистов в данной области: Буча, Рамбо и Джекобсона. Разработанная ими универсальная нотация для моделирования объектов (*Unified Modeling Language — UML*) претендует на роль стандарта в области объектно-ориентированного анализа и проектирования.

Ориентация на объектно-ориентированные методы объясняется следующими причинами:

- возможность сборки программной системы из готовых повторно используемых компонентов;
- возможность накопления проектных решений в виде библиотек классов на основе механизмов наследования;
- простота внесения изменений в проекты за счет инкапсуляции данных в объектах;
- быстрая адаптация приложений к изменяющимся условиям за счет использования свойств наследования и полиформизма;
- возможность организации параллельной работы аналитиков, проектировщиков и программистов.

Концепция объектно-ориентированного подхода и концепция распределенных вычислений стали базой для создания консорциума *Object Management Group* (далее — *OMG*), членами которой являются более 500 ведущих компьютерных компаний (*Sun*, *DEC*, *IBM*, *HP*, *Motorola* и др.). Основным направлением деятельности консорциума является разработка спецификаций и стандартов для создания распределенных объектных систем в разнородных средах. Базисом стали спецификации под названием *Object Management Architecture* (далее — *OMA*).

OMA состоит из четырех основных компонентов, представляющих спецификации различных уровней поддержки приложений (рис. 3.10):

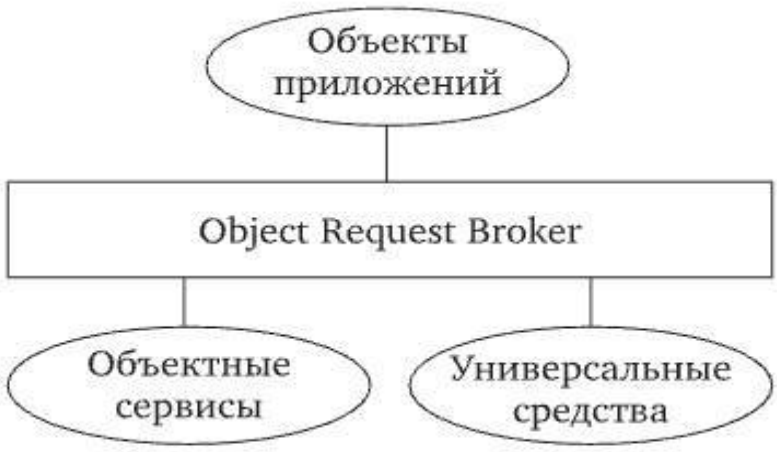


Рис. 3.10. **Спецификация OMA**

- архитектура брокера запросов объектов (*CORBA*) определяет механизмы взаимодействия объектов в разнородной сети;

- объектные сервисы (*Object Services*) являются основными системными сервисами, используемыми разработчиками для создания приложений;
- универсальные средства (*Common Facilities*) являются высокоуровневыми системными сервисами, ориентированными на поддержку пользовательских приложений (электронная почта, средства печати и др.);
- прикладные объекты (*Application Object*) предназначены для решения конкретных прикладных задач.

Исходя из основных положений объектно-ориентированного подхода рассмотрим концепцию идеального объектно-ориентированного *CASE*-средства.

Существует несколько объектно-ориентированных методов, авторами наиболее распространенных из них являются Г. Буч, Д. Рамбо, И. Джекобсон. В настоящее время наблюдается процесс сближения объектно-ориентированных методов. В частности, указанные выше авторы создали и выпустили несколько версий унифицированного метода *UML* (*Unified Modeling Language* — Унифицированный язык моделирования).

Классическая постановка задачи разработки программной системы (инжиниринг) представляет собой спиральный цикл итеративного чередования этапов объектно-ориентированного анализа, проектирования и реализации (программирования).

В реальной практике в большинстве случаев имеется предыстория в виде совокупности разработанных и внедренных программ, которые целесообразно использовать при разработке новой системы. Процесс проектирования в таком случае основан на реинжиниринге программных кодов, при котором путем анализа текстов программ восстанавливается исходная модель программной системы.

Современные *CASE*-средства поддерживают процессы инжиниринга и автоматизированного реинжиниринга.

Идеальное объектно-ориентированное *CASE*-средство (рис. 3.11) должно содержать четыре основных блока: анализ, проектирование, разработка и инфраструктура [17].

Основные требования к блоку ***анализа***:

- возможность выбора выводимой на экран информации из всей совокупности данных, описывающих модели;
- согласованность диаграмм при хранении их в репозитарии;
- внесение комментариев в диаграммы и соответствующую документацию для фиксации проектных решений;
- возможность динамического моделирования в терминах событий;
- поддержка нескольких нотаций (хотя бы три нотации Г. Буча, И. Джекобсона и *OMT* (*Object Model Template* — шаблон объектной модели).

Рис. 3.11. **Идеальное объектно-ориентированное CASE-средство**

Основные требования к блоку ***проектирования***:

- поддержка всего процесса проектирования приложения;
- возможность работы с библиотеками, средства поиска и выбора;
- возможность разработки пользовательского интерфейса;
- поддержка стандартов *OLE*, *ActiveX* и доступ к библиотекам *HTML* или *Java*;
- поддержка разработки распределенных или двух- и трехзвенных клиент-серверных систем (работа с *CORBA*, *DCOM*, *Internet*).

Основные требования к блоку ***реализации***:

- генерация кода полностью из диаграмм;
- возможность доработки приложений в клиент-серверных *CASE*-средствах типа *Power Builder*;
- реинжиниринг кодов и внесение соответствующих изменений в модель системы;
- наличие средств контроля, которые позволяют выявлять несоответствие между диаграммами и генерируемыми кодами и обнаруживать ошибки как на стадии проектирования, так и на стадии реализации.

Основные требования к блоку ***инфраструктуры***:

- наличие репозитория на основе базы данных, отвечающего за генерацию кода, реинжиниринг, отображение кода на диаграммах, а также обеспечивающего соответствие между моделями и программными кодами;
- обеспечение командной работы (многопользовательской работы и управление версиями) и реинжиниринга.

Сравнительный анализ *CASE*-систем показывает, что на сегодняшний день одним из наиболее приближенных к идеальному варианту *CASE*-средств является семейство *Rational Rose* фирмы *Rational Software Corporation*. Следует отметить, что именно здесь работают авторы унифицированного языка моделирования — Г. Буч, Д. Рамбо и И. Джекобсон, под руководством которых ведется разработка нового *CASE*-средства, поддерживающего *UML*.

Конкретный вариант *Rational Rose* определяется языком, на котором генерируются коды программ (*C++*, *Smalltalk*, *PowerBuilder*, *Ada*, *SQLWindows* и *ObjectPro*). Основной вариант — Rational Rose/C++ — позволяет разрабатывать проектную документацию в виде диаграмм и спецификаций, а также генерировать программные коды на C++.

Кроме того, Rational Rose содержит средства реинжиниринга программ, обеспечивающие повторное использование программных компонент в новых проектах.

В основе работы *Rational Rose* лежит построение различного рода диаграмм и спецификаций, определяющих логическую и физическую структуры модели, ее статические и динамические аспекты. В их число входят диаграммы классов, состояний, сценариев, мо-

дулей, процессов. *Rational Rose* функционирует на различных платформах: *IBM PC* (в среде *Windows*), *Sun SPARC stations* (*UNIX*, *Solaris*, *SunOS*), *Hewlett-Packard* (*HP UX*), *IBM RS/6000* (*AIX*).

Выделим ***основные критерии оценки и выбора CASE-средств***.

1. Функциональные характеристики:
   - среда функционирования: проектная среда, программное обеспечение/технические средства, технологическая среда;
   - функции, ориентированные на фазы жизненного цикла: моделирование, реализация, тестирование;
   - общие функции: документирование, управление конфигурацией, управление проектом.
2. Надежность.
3. Простота использования.
4. Эффективность.
5. Сопровождаемость.
6. Переносимость.
7. Общие критерии (стоимость, затраты, эффект внедрения, характеристики поставщика).

Данные критерии подробно изложены в стандартах *IEEE Std* 1348-1995. *IEEE recommended Practice for the Adoption of Computer-Aided Software Engineering* (*CASE*) Tools и *IEEE Std* 1209-1992 *Recommended Practice for the Evaluation and Selection of CASE Tools.*

3.5. Телекоммуникационные технологии

Спектр телекоммуникационных технологий чрезвычайно широк, что во многом объясняется высоким уровнем стандартизации и унификации, способствовавшим их распространению. Из всего многообразия остановимся на телекоммуникационных технологиях, обеспечивающий удаленный доступ и распределенную обработку.

Задача разработчиков распределенных систем — спроектировать программное и аппаратное обеспечение так, чтобы предоставить все необходимые характеристики распределенной системы. А для этого требуется знать преимущества и недостатки различных архитектур распределенных систем. Выделяется три типа архитектур распределенных систем [18] по реализации механизма управления доступом:

- прямое управление доступом;
- мандатное управление доступом;
- ролевое управление доступом.

При использовании прямого управления все субъекты и объекты определены так, как это было описано выше применительно к эталонной модели. При использовании прямого управления субъекты, которые являются владельцами объектом, могут разрешать или запрещать

доступ к этим другим субъектам. Принципиально объекты и субъекты могут меняться ролями. Данный подход используется наиболее часто, хотя и не обеспечивает высоких показателей безопасности.

При использовании мандатного управления доступом все объекты и субъекты относятся к определенному уровню. При использовании данного подхода субъекты не могут получить доступ к объектам, если им не разрешено работать на данном уровне.

Идея ролевого управления состоит в том, что определяется набор ролей, которые могут соответствовать, например, служебным обязанностям сотрудника или функциям, реализуемым подсистемой. Права доступа определяются, в частности, ролями, закрепленными за субъектом.

Ролевое управление доступом является наиболее гибким и простым с точки зрения администрирования.

В рамках конкретных систем возможно совместное использование рассмотренных подходов.

Выделяют следующие виды распределенных архитектур ИС:

- архитектура «файл — сервер»;
- двухзвенная архитектура «клиент — сервер»;
- многозвенная архитектура «клиент — сервер»;
- архитектура веб-приложений;
- сервис-ориентированная архитектура.

Рассмотрим каждую из этих архитектур.

Архитектура «файл — сервер». Файл-серверные приложения — приложения, схожие по своей структуре с локальными приложениями и использующие сетевой ресурс для хранения программы и данных.

Классическое представление информационной системы в архитектуре «файл — сервер» представлено на рис. 3.12.

Организация информационных систем на основе использования выделенных файл-серверов все еще является распространенной в связи с наличием большого количества персональных компьютеров разного уровня развитости и сравнительной дешевизны связывания *PC* в локальные сети. Конечно, основным достоинством данной архитектуры является простота организации. Проектировщики и разработчики информационной системы находятся в привычных и комфортных условиях *IBM PC* в среде *MS-DOS*, *Windows* или какого-либо облегченного варианта *Windows Server*. Имеются удобные и развитые средства разработки графического пользовательского интерфейса, простые в использовании средства разработки систем баз данных и (или) СУБД.

Достоинства такой архитектуры:

- многопользовательский режим работы с данными;
- удобство централизованного управления доступом;
- низкая стоимость разработки;

- высокая скорость разработки;
- невысокая стоимость обновления и изменения ПО.

Недостатки:

- проблемы многопользовательской работы с данными: последовательный доступ, отсутствие гарантии целостности;
- низкая производительность (зависит от производительности сети, сервера, клиента);
- плохая возможность подключения новых клиентов;
- ненадежность системы.

Рис. 3.12. **Классическое представление архитектуры «файл — сервер»**

Архитектура «клиент — сервер» (*Client-server*) — вычислительная или сетевая архитектура, в которой задания или сетевая нагрузка распределены между поставщиками услуг (сервисов), называемых серверами, и заказчиками услуг, называемых клиентами. Нередко клиенты и серверы взаимодействуют через компьютерную сеть и могут быть как различными физическими устройствами, так и программным обеспечением.

Первоначально системы такого уровня базировались на классической двухуровневой клиент-серверной архитектуре (*Two-tier architecture*). Под клиент-серверным приложением в этом случае понимается информационная система, основанная на использовании *серверов баз данных*.

Схематически такую архитектуру можно представить, как показано на рис. 3.13.

На стороне клиента выполняется код приложения, в который обязательно входят компоненты, поддерживающие интерфейс с конечным пользователем, производящие отчеты, выполняющие другие специфичные для приложения функции.

Рис. 3.13. **Классическое представление архитектуры «клиент — сервер»**

Клиентская часть приложения взаимодействует с клиентской частью программного обеспечения управления базами данных, которая фактически является индивидуальным представителем СУБД для приложения.

Заметим, что интерфейс между клиентской частью приложения и клиентской частью сервера баз данных, как правило, основан на ис-пользовании языка *SQL*. Поэтому такие функции, как, например, предварительная обработка форм, предназначенных для запросов к базе данных, или формирование результирующих отчетов, выполняются в коде приложения.

Наконец, клиентская часть сервера баз данных, используя средства сетевого доступа, обращается к серверу баз данных, передавая ему текст оператора языка *SQL*.

Посмотрим теперь, что же происходит на стороне сервера баз данных. В продуктах практически всех компаний сервер получает от клиента текст оператора на языке *SQL*:

- сервер производит компиляцию полученного оператора.
- далее (если компиляция завершилась успешно) происходит выполнение оператора.

Преимуществами данной архитектуры являются:

1) возможность, в большинстве случаев, распределить функции вычислительной системы между несколькими независимыми компьютерами в сети;

2) все данные хранятся на сервере, который, как правило, защищен гораздо лучше большинства клиентов, а также на сервере проще обеспечить контроль полномочий, чтобы разрешать доступ к данным только клиентам с соответствующими правами доступа;

3) поддержка многопользовательской работы;

4) гарантия целостности данных.

Недостатки:

1) неработоспособность сервера может сделать неработоспособной всю вычислительную сеть;

2) администрирование данной системы требует квалифицированного профессионала;

3) высокая стоимость оборудования;

4) бизнес-логика приложений осталась в клиентском ПО.

При проектировании информационной системы, основанной на архитектуре «клиент — сервер», большее внимание следует обращать на грамотность общих решений. Технические средства пилотной версии могут быть минимальными (например, в качестве аппаратной основы *сервера баз данных* может использоваться одна из рабочих станций). После создания пилотной версии нужно провести дополнительную исследовательскую работу, чтобы выяснить узкие места системы. Только после этого необходимо принимать решение о выборе аппаратуры сервера, которая будет использоваться на практике.

Увеличение масштабов информационной системы не порождает принципиальных проблем. Обычным решением является замена аппаратуры сервера (и, может быть, аппаратуры рабочих станций, если требуется переход к локальному кэшированию баз данных). В любом случае практически не затрагивается прикладная часть информационной системы.

Также данный вид архитектуры называют *архитектурой с «толстым» клиентом*.

Многозвенная архитектура. В случае большого числа пользователей возникают проблемы своевременной и синхронной замены версий клиентских приложений на рабочих станциях. Такие проблемы решаются в рамках многозвенной архитектуры (рис. 3.14). Часть общих приложений переносится на специально выделенный сервер приложений. Тем самым понижаются требования к ресурсам рабочих станций, которые будут называться «тонкими» клиентами. Данный способ организации вычислительного процесса является разновидностью архитектуры «клиент — сервер».

Использование многозвенной архитектуры может быть рекомендовано также в случае, если некоторая программа требует для своей

работы много ресурсов, то может оказаться дешевле построить тонкую сеть с одним очень мощным сервером, чем использовать несколько мощных клиентных рабочих станций. Особенно это имеет значение, если данной программой пользуются не постоянно, а время от времени. На рис. 3.15 представлена архитектура многозвенного приложения.

Рис. 3.14. **Многозвенная архитектура**

Разумное сочетание производительности сервера приложений и производительности рабочих станций позволят построить сеть, более дешевую при установке и эксплуатации.

Рис. 3.15. **Архитектура многозвенного приложения**

Архитектура веб-приложений. Эта архитектура широко применяется в настоящее время и носит также название *архитектуры веб-сервисов.*

Веб-сервис — это приложение, доступное через Интернет и предоставляющее некоторые услуги, форма которых не зависит от поставщика (так как используется универсальный формат данных — *XML*) и платформы функционирования.

Сервис-ориентированная архитектура также является одной из технологий реализации информационных систем (подробнее см. в подпараграфе 6.2.9).

В данное время существует три различные технологии, поддерживающие концепцию распределенных объектных систем. Это технологии *EJB*, *CORBA* и *DCOM*.

В основе веб-сервисов лежат открытые стандарты и протоколы: *SOAP*, *UDDI* и *WSDL*.

1. *SOAP* (*Simple Object Access Protocol*), разработанный консорциумом *W3C*, определяет формат запросов к веб-сервисам. Сообщения между веб-сервисом и его пользователем пакуются в так называемые *SOAP*-конверты (*SOAP envelopes*, иногда их еще называют *XML*-конвертами). Само сообщение может содержать либо запрос на осуществление какого-либо действия, либо ответ — результат выполнения этого действия.

2. *WSDL* (*Web Service Description Language*). Интерфейс веб-сервиса описывается в WSDL-документах (а *WSDL* — это подмножество *XML*). Перед развертыванием службы разработчик составляет ее описание на языке *WSDL*, указывает адрес веб-сервиса, поддерживаемые протоколы, перечень допустимых операций, форматы запросов и ответов.

3. *UDDI* (*Universal Description, Discovery and Integration*) — протокол поиска веб-сервисов в *Internet* (http://www.uddi.org/). Представляет собой бизнес-реестр, в котором провайдеры веб-сервисов регистрируют службы, а разработчики находят необходимые сервисы для включения в свои приложения.

***Enterprise JavaBeans* (*EJB*).** Основная идея, лежавшая в разработке технологии *EJB*, — создать такую инфраструктуру для компонент, чтобы они могли бы легко «вставляться» («*plug in*») и удаляться из серверов, тем самым увеличивая или снижая функциональность сервера. Технология *EJB* похожа на технологию *JavaBeans* в том смысле, что она использует ту же самую идею (а именно, создание новой компоненты из уже существующих, готовых и настраиваемых компонент, аналогично *RAD*-системам), но во всем остальном *EJB* — совершенно иная технология.

Достоинства EJB:

1) быстрое и простое создание;
2) *Java*-оптимизация;
3) кросс-платформенность;
4) динамическая загрузка компонент-переходников;
5) возможность передачи объектов по значению;
6) встроенная безопасность.

Недостатки EJB:

1) поддержка только одного языка — *Java*;
2) трудность интегрирования с существующими приложениями;
3) плохая масштабируемость;
4) производительность;
5) отсутствие международной стандартизации.

Благодаря своей легко используемой *Java*-модели *EJB* является самым простым и самым быстрым способом создания распределенных систем. *EJB* — хороший выбор для создания *RAD*-компонент и небольших приложений на языке *Java*. Конечно, *EJB* не такая мощная технология, как *DCOM* или *CORBA*. Тем самым, роль *RMI* (англ. *Remote Method Invocation* — программный интерфейс вызова удаленных методов в языке *Java*) в создании больших, масштабируемых промышленных систем, снижается.

Distributed Component Object Model (далее — *DCOM*) (распределенная компонентная объектная модель) — программная архитектура, разработанная компанией *Microsoft* для распределения приложений между несколькими компьютерами в сети. Программный компонент на одной из машин может использовать *DCOM* для передачи сообщения (его называют удаленным вызовом процедуры) к компоненту на другой машине. *DCOM* автоматически устанавливает соединение, передает сообщение и возвращает ответ удаленного компонента.

Для того чтобы различные фрагменты сложного приложения могли работать вместе через Интернет, необходимо обеспечить между ними надежные и защищенные соединения, а также создать специальную систему, которая направляет программный трафик.

Для решения этой задачи компания *Microsoft* создала *DCOM*, которая встраивается в операционные системы *Windows NT* 4.0 и *Windows* 98 и выше.

Достоинства DCOM:

1) независимость от языка;
2) динамический/статический вызов;
3) динамическое нахождение объектов;
4) масштабируемость;
5) открытый стандарт (контроль со стороны *TOG*);
6) множественность *Windows*-программистов;

Недостатки DCOM:

1) сложность реализации;
2) зависимость от платформы;
3) нет именования через *URL*;
4) нет проверки безопасности на уровне выполнении *ActiveX* компонент;
5) отсутствие альтернативных разработчиков.

DCOM является лишь частным решением проблемы распределенных объектных систем. Он хорошо подходит для *Microsoft*-ориентированных сред. Как только в системе возникает необходимость работать с архитектурой, отличной от *Windows*, *DCOM* перестает быть оптимальным решением проблемы. Конечно, вскоре это положение может измениться, так как *Microsoft* стремится перенести *DCOM* и на другие платформы. Например, фирмой *Software*

AG уже выпущена версия *DCOM* для *Solaris UNIX* и планируется выпуск версий и для других версий *UNIX*. Но все-таки на сегодняшний день *DCOM* хорош лишь в качестве решения для систем, ориентированных исключительно на продукты *Microsoft*. Большие нарекания вызывает также отсутствие безопасности при исполнении *ActiveX* компонент, что может привести к неприятным последствиям.

Common Object Request Broker Architecture (далее — *CORBA*). В конце 1980-х — начале 1990-х гг. многие ведущие фирмы-разработчики были заняты поиском технологий, которые принесли бы ощутимую пользу на все более изменчивом рынке компьютерных разработок. В качестве такой технологии была определена область распределенных компьютерных систем. Необходимо было разработать единообразную архитектуру, которая позволяла бы осуществлять повторное использование и интеграцию кода, что было особенно важно для разработчиков.

Поэтому в мае 1989 г. была сформирована *OMG*. Как уже отмечалось, сегодня *OMG* насчитывает более 700 членов (в *OMG* входят практически все крупнейшие производители ПО, за исключением *Microsoft*).

Задачей консорциума *OMG* является определение набора спецификаций, позволяющих строить интероперабельные информационные системы. Спецификация *OMG — CORBA* — является индустриальным стандартом, описывающим высокоуровневые средства поддерживания взаимодействия объектов в распределенных гетерогенных средах.

CORBA специфицирует инфраструктуру взаимодействия компонент (объектов) на представительском уровне и уровне приложений модели *OSI*. Она позволяет рассматривать все приложения в распределенной системе как объекты. Причем объекты могут одновременно играть роль и клиента, и сервера: роль клиента, если объект является инициатором вызова метода у другого объекта; роль сервера, если другой объект вызывает на нем какой-нибудь метод. Объекты-серверы обычно называют «реализацией объектов». Практика показывает, что большинство объектов одновременно исполняют роль и клиентов, и серверов, попеременно вызывая методы на других объектах и отвечая на вызове извне. При использовании *CORBA* появляется возможность строить гораздо более гибкие системы, чем системы «клиент — сервер», основанные на двухуровневой и трехуровневой архитектуре.

Достоинства CORBA:

1) платформенная независимость;
2) языковая независимость;
3) динамические вызовы;
4) динамическое обнаружение объектов;
5) масштабируемость;

6) *CORBA*-сервисы;

7) широкая индустриальная поддержка.

Недостатки CORBA:

1) нет передачи параметров «по значению»;

2) отсутствует динамическая загрузка компонент-переходников;

3) нет именования через *URL*.

К основным достоинствам *CORBA* можно отнести межъязыковую и межплатформенную поддержку. Хотя *CORBA*-сервисы и отнесены к достоинствам технологии *CORBA*, их в равной степени можно одновременно отнести и к недостаткам *CORBA*, ввиду практически полного отсутствия их реализации.

Интернет (*Internet*). Это бурно разросшаяся совокупность компьютерных сетей, опутывающих земной шар, связывающих правительственные, военные, образовательные и коммерческие институты, а также отдельных граждан.

Как и многие другие великие идеи, «сеть сетей» возникла из проекта, который предназначался совершенно для других целей: из сети *ARPAnet*, разработанной и созданной в 1969 г. по заказу Агентства передовых исследовательских проектов (*Advanced Research Project Agency — ARPA*) Министерства обороны США. *ARPAnet* была сетью, объединяющей учебные заведения, военных и военных подрядчиков; она была создана для помощи исследователям в обмене информацией, а также (что было одной из главных целей) для изучения, каким образом поддерживать связь в случае ядерного нападения.

В модели *ARPAnet* между компьютером-источником и компьютером-адресатом всегда существует связь. Сама сеть считается ненадежной; любой ее отрезок может в любой момент исчезнуть (после бомбежки или в результате неисправности на кабеле). Сеть была построена так, чтобы потребность в информации от компьютеров-клиентов была минимальной. Для пересылки сообщения по сети компьютер должен был просто помещать данные в конверт, называемый «пакетом межсетевого протокола» (*IP, Internet Protocol*), правильно «адресовать» такие пакеты. Взаимодействующие между собой компьютеры (а не только сама сеть) также несли ответственность за обеспечение передачи данных. Основополагающий принцип заключался в том, что каждый компьютер в сети мог общаться в качестве узла с любым другим компьютером с широким выбором компьютерных услуг, ресурсов, информации. Комплекс сетевых соглашений и общедоступных инструментов «сети сетей» разработан с целью создания одной большой сети, в которой компьютеры, соединенные воедино, взаимодействуют имея множество различных программных и аппаратных платформ.

В настоящее время направление развития Интернета в основном определяет «Общество *Internet*», или *Internet Society* (*ISOC*). *ISOC* —

это организация на общественных началах, целью которой является содействие глобальному информационному обмену через Интернет. Она назначает совет старейшин *Internet Architecture Board* (далее — *IAB*), который отвечает за техническое руководство и ориентацию Интернета (в основном это стандартизация и адресация в Интернет). Пользователи Интернета выражают свои мнения на заседаниях инженерной комиссии *Internet Engineering Task Force* (*IETF*). *IETF* — еще один общественный орган; он собирается регулярно для обсуждения текущих технических и организационных проблем Интернета.

Финансовая основа Интернета заключается в том, что каждый платит за свою часть. Представители отдельных сетей собираются и решают, как соединяться и как финансировать эти взаимные соединения. Учебное заведение или коммерческое объединение платят за подключение к региональной сети, которая в свою очередь платит за доступ к Интернету поставщику на уровне государства. Таким образом, каждое подключение к Интернету кем-то оплачивается.

Рассмотрим кратко *основные компоненты* Интернета.

World Wide Web (*WWW*, просто *Web*, Всемирная паутина) представляет совокупность серверов *Web* (веб-серверов), на которых хранятся данные, реализованные в виде текстовых и (или) графических страниц с гипертекстовыми ссылками на другие страницы или веб-серверы. Если ссылка заинтересовала пользователя, то он может перейти на нужную страницу независимо от ее местонахождения, вернуться на предыдущую просмотренную, поставить закладку. В этом заключается основное преимущество *WWW*. Пользователя не интересует, как организовано и где находится огромное структурированное хранилище данных. Графическое представление подключения различных серверов представляет собой сложную невидимую электронную паутину.

Серверы Web — специальные компьютеры, осуществляющие хранение страниц с информацией и обработку запросов от других машин. Пользователь, попадая на какой-нибудь сервер *Web*, получает страницу с данными. На компьютере пользователя специальная программа (*браузер*) преобразует полученный документ в удобный для просмотра и чтения вид, отображаемый на экране. Серверы *Web* устанавливаются, как правило, в фирмах и организациях, желающих распространить свою информацию среди многих пользователей, и отличаются специфичностью информации. Организация и сопровождение собственного сервера требует значительных затрат. Поэтому в *WWW* встречаются «разделяемые» (*shared*) серверы, на которых публикуют свои данные различные пользователи и организации. Это самый дешевый способ опубликования своей информации для обозрения. Такие серверы зачастую представляют своеобразные информационные свалки.

Серверы FTP представляют собой хранилища различных файлов и программ, хранящихся в виде архивов. На этих серверах может храниться как полезная информация (дешевые условно-бесплатные утилиты, программы, картинки), так и информация сомнительного характера, например, порнографическая.

Электронная почта является неотъемлемой частью Интернета и одной из самых полезных вещей. С ее помощью можно посылать и получать любую корреспонденцию (письма, статьи, деловые бумаги и др.). Время пересылки зависит от объема, обычно занимает минуты, иногда часы. Каждый абонент электронной почты имеет свой уникальный адрес. Надо отметить, что подключение к электронной почте может быть организовано и без подключения к Интернету. Необходимый интерфейс пользователя реализуется с помощью браузера, который, получив от него запрос с интернет-адресом, преобразовывает его в электронный формат и посылает на определенный сервер. В случае корректности запроса он достигает веб-сервера, и последний посылает пользователю в ответ информацию, хранящуюся по заданному адресу. Браузер, получив информацию, делает ее читабельной и отображает на экране. Современные браузеры имеют также встроенную программу для электронной почты.

Подсоединение к Интернету для каждого конкретного пользователя может быть реализовано различными способами: от полного подсоединения по *локальной вычислительной сети* (далее — ЛВС) до доступа к другому компьютеру для работы с разделением и использованием программного пакета эмуляции терминала.

Диапазон предлагаемых интернет-услуг достаточно широк. Можно воспользоваться электронной почтой, электронными досками объявлений, пересылкой файлов, удаленным доступом, каталогизирующими программами и т. д. Для получения полного набора услуг у пользователя должно быть подсоединение по протоколу *TCP/IP*. Это необходимо для того, чтобы компьютер пользователя был частью сети и мог устанавливать контакт с любой сервисной программой, имеющейся в Интернете.

Фактически выход в Интернет может быть реализован несколькими видами подключений:

- доступ по выделенному каналу;
- доступ по *ISDN* (*Integrated Services Digital Network* — цифровая сеть с интегрированными услугами);
- доступ по коммутируемым линиям;
- с использованием протоколов *SLIP* и *PPP*.

Корпорациям и большим организациям лучше всего использовать *доступ по выделенному каналу*. В этом случае предоставляется наиболее полно использовать все средства Интернета. Поставщик сетевых услуг при этом сдает в аренду выделенную телефонную линию с указанной скоростью передачи и устанавливает специаль-

ный компьютер-маршрутизатор для приема и передачи сообщений от телекоммуникационного узла организации. Это дорогостоящее подключение. Однако, установив такое соединение, каждый компьютер ЛВС организации является полноценным членом Интернета и может выполнять любую сетевую функцию.

ISDN — это использование цифровой телефонной линии, соединяющей домашний компьютер или офис с коммутатором телефонной компании. Преимущество *ISDN* заключается в том, что предоставляется возможность доступа с очень высокими скоростями при относительно низкой стоимости. При этом сервис Интернет предоставляется таким же, как и по коммутируемым линиям. Услуги телефонных компаний, предоставляющих сервис *ISDN*, доступны не на всей территории России.

Доступ по коммутируемым линиям — наиболее простой и дешевый способ получения доступа к сети (*Dial-up Access*). В этом случае пользователь приобретает права доступа к компьютеру, который подсоединен к Интернету (хост-компьютеру или узлу Интернета). Войдя по телефонной линии (при этом используется модем и программное обеспечение для работы в коммутируемом режиме) с помощью эмулятора терминала в удаленную систему, необходимо в ней зарегистрироваться и далее уже можно пользоваться всеми ресурсами Интернета, предоставленными удаленной системе. Пользователь в таком режиме арендует дисковое пространство и вычислительные ресурсы удаленной системы. Если требуется сохранить важное сообщение электронной почты или другие данные, то это можно сделать в удаленной системе, но не на диске пользовательского компьютера: сначала нужно записать файл на диск удаленной системы, а затем с помощью программы передачи данных перенести этот файл на свой компьютер. При таком доступе пользователь не может работать с прикладными программами, для которых нужен графический дисплей, так как в такой конфигурации компьютер, подсоединенный к Интернету, не имеет возможности передать графическую информацию на компьютер пользователя.

При дополнительных финансовых затратах и в коммутируемом режиме можно получить полный доступ к Интернету. Это достигается *применением протоколов SLIP и PPP*. Один называется «межсетевой протокол последовательного канала» (*Serial Line Internet Protocol* (далее — *SLIP*), а другой — «протокол точка-точка» (*Point-to-Point Protocol* (далее — *PPP*). Одно из главных достоинств *SLIP* и *PPP* состоит в том, что они обеспечивают полноценное соединение с Интернетом. Пользовательский компьютер не использует какую-то систему как «точку доступа», а непосредственно подключается к Интернету. Но для подключения средних и больших сетей к Интернету эти протоколы не подходят, поскольку их быстродействия недостаточно для одновременной связи со многими пользователями.

Современные сети создаются по многоуровневому принципу. Передача сообщений в виде последовательности битов начинается на уровне линий связи и аппаратуры, причем линий связи не всегда высокого качества. Затем добавляется уровень базового программного обеспечения, управляющего работой аппаратуры. Следующий уровень программного обеспечения позволяет наделить базовые программные средства дополнительными необходимыми возможностями. Расширение необходимых функциональных возможностей сети путем добавления уровня за уровнем приводит к тому, что пользователь в конце концов получает по-настоящему дружественный и полезный инструментарий.

Аналогом информационной модели Интернета можно назвать почтовое ведомство, представляющее собой сеть с коммутацией пакетов. Там корреспонденция конкретного пользователя смешивается с другими письмами, отправляется в ближайшее почтовое отделение, где сортируется и направляется в другие почтовые отделения до тех пор, пока не достигнет адресата.

Для передачи данных в Интернет используются интернет-протокол (*IP*) и протокол управления передачей (*TCP*).

С помощью интернет-протокола (*IP*) обеспечивается доставка данных из одного пункта в другой. Различные участки Интернета связываются с помощью системы компьютеров (называемых маршрутизаторами), соединяющих между собой сети. Это могут быть сети *Ethernet*, сети с маркерным доступом, телефонные линии. Правила, по которым информация переходит из одной сети в другую, называются протоколами. Межсетевой протокол (*IP*) отвечает за адресацию, т. е. гарантирует, что маршрутизатор знает, что делать с данными пользователя, когда они поступят. Некоторая адресная информация приводится в начале каждого пользовательского сообщения. Она дает сети достаточно сведений для доставки пакета данных, так как каждый компьютер в Internet имеет свой уникальный адрес.

Для более надежной передачи больших объемов информации служит протокол управления передачей (*TCP*). Информация, которую пользователь хочет передать, *TCP* разбивает на порции. Каждая порция нумеруется, у нее подсчитывается контрольная сумма, чтобы можно было на приемной стороне проверить, вся ли информация получена правильно, а также расположить данные в правильном порядке. На каждую порцию добавляется информация протокола *IP*, таким образом получается пакет данных в Интернете, составленный по правилам *TCP/IP*.

По мере развития Интернета и увеличения числа компьютерных узлов, сортирующих информацию, в сети была разработана *доменная система имен — DNS*, и способ адресации — способ адресации по доменному принципу. *DNS* иногда еще называют региональной системой наименований.

Доменная система имен — это метод назначения имен путем передачи сетевым группам ответственности за их подмножество имен. Каждый уровень этой системы называется доменом. Домены в именах отделяются друг от друга точками: *inr.msk.su*. В имени может быть различное количество доменов, но практически их не больше пяти. По мере движения по доменам слева направо в имени, количество имен, входящих в соответствующую группу возрастает.

Все компьютеры Интернета способны пользоваться доменной системой. Работающий в сети компьютер всегда знает свой собственный сетевой адрес. Когда используется доменное имя, например, *mx.ihep.su*, компьютер преобразовывает его в числовой адрес. Для этого он начинает запрашивать помощь у *DNS*-серверов. Это узлы, рабочие машины, обладающие соответствующей базой данных, в число обязанностей которых входит обслуживание такого рода запросов. *DNS*-сервер начинает обработку имени с правого его конца и двигается по нему влево, т. е. сначала производится поиск адреса в самой большой группе (домене), потом постепенно сужает поиск. Но для начала опрашивается на предмет наличия нужной информации местный узел. Если местный сервер адрес не знает, он связывается с корневым сервером. Это сервер, который знает адреса серверов имен высшего уровня (самых правых в имени), здесь это уровень государства (ранга домена *su*). У него запрашивается адрес компьютера, ответственного за зону *su*. Местный *DNS*-сервер связывается с этим более общим сервером и запрашивает у него адрес сервера, ответственного за домен *ihep.su*. Теперь уже запрашивается этот сервер и у него выясняется адрес рабочей машины *mx*.

Дальнейшим развитием Интернета явилась ***Intranet*-технология.** *Intranet* (*интранет*) представляет собой технологию управления корпоративными коммуникациями, в отличие от Интернета, являющейся технологией глобальных коммуникаций.

В телекоммуникационных технологиях выделяют три уровня реализации: аппаратный, программный и информационный. С этой точки зрения интранет отличается от Интернета только информационными аспектами, где выделяются три уровня: универсальный язык представления корпоративных знаний, модели представления, фактические знания.

Архитектура интранета явилась естественным развитием информационных систем: от систем с централизованной архитектурой, через системы «клиент — сервер» к интранету.

Идея централизованной архитектуры была классически реализована в мейнфреймах, отличительной чертой которых было концентрация вычислительных ресурсов в едином комплексе, где осуществлялось хранение и обработка огромных массивов информации.

Достоинства систем «клиент — сервер»:

1) простота администрирования,
2) защита информации.

С появлением персональных компьютеров появилась возможность переноса части информационной системы непосредственно на рабочее место. Таким образом, возникла необходимость построения распределенной информационной системы. Этим целям соответствует архитектура «клиент — сервер», основанная на модели взаимодействия компьютеров и программ в сети, рассмотренная выше.

Однако системам «клиент — сервер» присущ ряд серьезных *недостатков*:

1) трудность администрирования, вследствие территориальной разобщенности и неоднородности компьютеров на рабочих местах;

2) недостаточная степень защиты информации от несанкционированных действий;

3) закрытый протокол для общения клиентов и сервера, специфичный для данной информационной системы.

Как следствие указанных недостатков была разработана архитектура систем интранет, сконцентрировавших и объединивших в себе лучшие качества централизованных систем и традиционных систем «клиент — сервер» (рис. 3.16).

Рис. 3.16. **Архитектура систем «Intranet»**

Вся информационная система находится на центральном компьютере. На рабочих местах находятся простейшие устройства доступа (навигаторы), предоставляющие возможность управления процессами в информационной системе. Все процессы осуществляются на центральной ЭВМ, с которым устройство доступа общается посредством простого протокола, путем передачи экранов и кодов нажатых клавиш на пульте.

Основные *достоинства* систем *Intranet*:

- на сервере вырабатывается информация (а не данные) в форме удобной для представления пользователю;

- использование для обмена информацией между клиентом и сервером протокола открытого типа;
- концентрация прикладной системы на сервере, на клиентах размещается только программа-навигатор;
- облегченное централизованное управление серверной частью и рабочими местами;
- унифицированность интерфейса, не зависящего от программного обеспечения, используемого пользователем (операционная система, СУБД и др.).

Важным *преимуществом* интранета является открытость технологии. Существующее программное обеспечение, основанное на закрытых технологиях, когда решения, разработанные одной фирмой для одного приложения, кажутся более функциональными и удобными, однако резко ограничивают возможности развития информационных систем.

В настоящее время в интранете широко используются открытые стандарты по следующим направлениям:

- управление сетевыми ресурсами (*SMTP*, *IMAP*, *MIME*);
- телеконференции (*NNTP*);
- информационный сервис (*HTPP*, *HTML*);
- справочная служба (*LDAP*);
- программирование (*Java*).

Тенденции дальнейшего развития интранета:

- интеллектуальный сетевой поиск;
- высокая интерактивность навигаторов за счет применения *Java*-технологии;
- сетевые компьютеры;
- превращение интерфейса навигатора в универсальный интерфейс с компьютером.

Сервис-ориентированная архитектура (*Service-Oriented Architecture*, *SOA*) — это подход к разработке ИС, основанный на использовании распределенных, слабо связанных компонентов, в качестве которых выступают веб-сервисы или службы (*service*), взаимодействующие по единому протоколу (более подробно рассмотрены в параграфе 6.2).

3.6. Технологии Интернета вещей

3.6.1. Интернет вещей (IoT)

Идея Интернета вещей (*Internet of Things*, *IoT*) сама по себе достаточно тривиальна и состоит в том, что все окружающие предметы и устройства имеют уникальные идентификаторы и могут быть снабжены датчиками (сенсорами) и исполнительными устройствами (актуаторами). Все эти устройства соединены между собой с помощью сети, в качестве которой может выступать Интернет. В этом

случае все устройства *IoT* становятся полноправными хостами. Тогда появляется возможность отслеживать эти объекты и их параметры в пространстве и во времени и управлять ими.

Следует заметить, что в настоящее время нет общепринятого четкого определения этого понятия. Обычно термин *IoT* определяют следующим образом: «*Интернет вещей* — это концепция развития технологий сети Интернет в сторону автоматизации, и исключения участия человека из большинства процессов работы *IT*-инфраструктуры. Интернет–вещи посредством обмена информацией между различными датчиками и сенсорами должны полностью автоматизировать процессы управления группами устройств» [26, 27].

Это очень общее определение, но для этого есть веские причины. Дело в том, что в настоящее время *IoT* представляет собой парадигму, которая подобно зонтику накрывает большое число более мелких парадигм, платформ и протоколов.

Самые первые интернет-вещи появились еще в конце 1980-х гг. прошлого века, но только в XXI в. в связи с бурным развитием инфокоммуникационных технологий сформировалась сама концепция *IoT*.

Наиболее важными отличиями *IoT* от интернета людей являются следующие моменты:

— фокус не на человеке, а на вещах;

— *IoT* включает в свой состав большее число подключенных объектов;

— объекты могут иметь существенно меньшие размеры и использовать низкоскоростные каналы связи;

— фокус на считывании информации, а не на коммуникациях.

Первые *IoT*-системы представляли собой системы управления удаленными физическими объектами, в качестве которых часто выступали чайники, пылесосы и т. п. Достаточно быстро пришло осознание того, что в качестве «вещей» может выступать все что угодно: промышленное оборудование, разного рода виртуальные объекты, и что для построения сети, которая соединяет «вещи» могут использоваться не только такие же, как у людей каналы доступа к Интернету, но и различные протоколы взаимодействия между собой.

IoT не исключает участие человека, поскольку он не во всех случаях полностью автоматизирует вещи, а чаще всего просто предоставляет человеку доступ к вещам. В *IoT* каждая вещь имеет свой уникальный идентификатор, которые совместно образуют континуум вещей, способных взаимодействовать друг с другом, создавая временные или постоянные сети. Так вещи могут принимать участие в процессе их перемещения, делясь сведениями о текущей геолокации, что позволяет полностью автоматизировать процесс логистики. Имея встроенный интеллект, вещи могут менять свои свойства и адаптироваться к окружающей среде, в том числе для

уменьшения энергопотребления. *IoT* позволяет создавать комбинацию из интеллектуальных устройств, объединенных сетями связи, и людей.

Сферы применения. Начиная с момента появления в конце прошлого века *IoT* постоянно развивалась. Расширение шло как в плане совершенствования характеристик, так и в плане расширения сферы применения. В качестве движущей силы развития *IoT* выступает, в первую очередь, прогресс в области инфокоммуникационных технологий, увеличение доступных вычислительных мощностей и широкополосных каналов связи при постоянном снижении их стоимости. Это приводит, с одной стороны, к увеличению объемов данных, поступающих из внешнего мира, а с другой стороны позволяет использовать мощные алгоритмы обработки данных. Эти факторы приводят к необходимости и возможности использовать новые архитектурные решения.

На сегодняшний день очень сложно перечислить все сферы применения *IoT*. Можно указать следующие наиболее популярные области их применения:

1. Умный дом. Главные задачи — Безопасность, интеллектуальный дверной звонок, управление телевизором и кухней, автоматические системы полива и освещения, сигнализация пожара, газа, утечки воды, температуры дома.
2. Умный город. Главные задачи — управление и регулирование автомобильного движения, ночное / дневное уличное освещение, уведомление об опасности для пешеходов, определение нестандартных и опасных ситуаций в городе.
3. Оптимизация использования ресурсов в доме, городе, стране. Освещение, потребление электричества, отопления, оптимизация и прогнозирование использования, например, топлива на электростанциях.
4. Погода и стихийные бедствия. Метеорологическая информация, сейсмическая активность, противопожарный контроль. Прогнозирование погодных данных.
5. Оптимизация перевозок, доставок, хранения и сортировки. Многие компании использует решение для построения оптимальных транспортных маршрутов, для управления потоком грузов в терминалах хранения и сортировки грузов в крупных аэропортах.
6. Промышленный мониторинг. Контроль, управление конвейерными линиями, управление роботами, сортировка товаров, сырья и тестирование готовой продукции.
7. Встроенные системы управления транспортными средствами. Сложные механизмы, высокотехнологичные устройства, такие как современные автомобили, самолеты и пр. Автоматизированная система управления, защита от угона, контроль агрегатов системы. Распознавание лица и тела водителя для предотвращения сна, по-

тери внимания. Прогнозирование обслуживания и замены компонентов системы.

При этом список областей применения постоянно расширяется. Обобщенная идея *IoT* показана в рис. 3.17.

Рис 3.17. **Идея Интернета вещей**

С *IoT* связано множество стандартов, относящихся к разным составляющим *IoT*. Официальное определение Интернета вещей приведено в Рекомендации МСЭ-Т, согласно которому *IoT* — глобальная инфраструктура информационного общества, обеспечивающая передовые услуги за счет организации связи между вещами (физическими или виртуальными) на основе существующих и развивающихся совместимых информационных и коммуникационных технологий.

Под «вещами» (*things*) здесь понимается физический объект (физическая вещь) или объект виртуального (информационного) мира, которые могут быть идентифицированы и объединены через коммуникационные сети. Часто вместо термина «вещи» использует термин «сущность».

Схема отображения физических и виртуальных вещей представлена на рис. 3.18. Из рисунка следует, что виртуальные вещи могут существовать без их физических воплощений, в то время как физическим объектам/вещам обязательно соответствует минимум один виртуальный объект. При этом ведущую роль играют именно устройства, которые могут собирать различную информацию и распространять ее по коммуникационным сетям различными способами: через шлюзы и через сеть; без шлюзов, но через сеть; напрямую между собой.

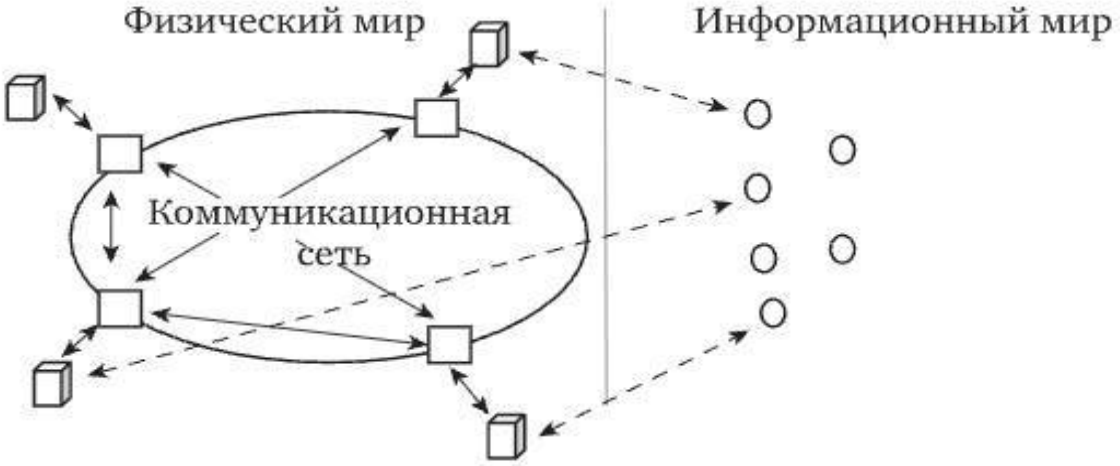


Рис. 3.18. **Физический и виртуальный миры**

— Физический объект; — Шлюз; — Виртуальный объект

Следует различать понятия «Интернет вещей» и «интернет-вещь». Под интернет–вещью понимается любое устройство, которое имеет доступ к сети Интернет с целью передачи или запроса каких-либо данных, имеет конкретный адрес в глобальной сети или идентификатор, по которому можно осуществить обратную связь с вещью, имеет интерфейс для взаимодействия с пользователем.

Каждый узел сети интернет-вещей предоставляет свой сервис, оказывая определенный набор услуг поставки (обработки) данных. В то же время узел такой сети может принимать команды от любого другого узла. Это означает, что все интернет-вещи могут взаимодействовать друг с другом и решать совместные вычислительные задачи. Интернет–вещи могут образовывать локальные сети, объединенные какой-либо одной зоной обслуживания или функцией.

Архитектура IoT. *IoT* является многоуровневой иерархической гетерогенной киберфизической системой и с точки зрения архитектуры верхнего уровня может быть отнесена к системам, реализующим иерархический стиль. При этом сами уровни могут быть выделены разными способами. К настоящему времени известно большое число разнообразных эталонных архитектур, ориентированных на разные подклассы *IoT*, которые отличаются числом уровней и способами распределения работ между отдельными уровнями.

Реализация Интернета вещей включает в себя четыре основных строительных блока. К ним относятся вещи, шлюзы, сетевая инфраструктура и облачная инфраструктура (табл. 3.3).

Для удобства понимания реализации *IoT* и бизнес процессов (БП) вышеперечисленные четыре элемента можно сгруппированы в три компонента. Теперь три основных компонента Интернета вещей включают в себя: вещи с подключенными к сети сенсорами и актуаторами, хранилища данных, информации, знаний, а также разного рода процессоры, реализующие обработку (табл. 3.4).

Таблица 3.3

Основные элементы *IoT* и их назначение

Элемент *IoT*	Устройство *IoT*	Реализуемая функциональность
Вещи	Сенсоры, актуаторы	Позволяет взаимодействовать с внешним миром, собирать данные объектов внешнего мира без какого-либо взаимодействия с человеком
Шлюзы	—	Действует как промежуточный элемент и обеспечивает надежную связь между вещами и облачной инфраструктурой. Он также обеспечивает безопасность и управляемость при работе с потоками данных
Сетевая инфраструктура	Роутеры, коммутаторы, повторители	Это позволяет контролировать поток данных от объектов к облачной инфраструктуре
Облачная инфраструктура	Виртуальные сервера, виртуальные серверы, озера данных	Это позволяет использовать для реализации аналитики

Таблица 3.4

Компоненты *IoT* и их функциональность

Компонент	Реализуемая функциональность
Вещи с подключенными к сети сенсорами и актуаторами	Собирает информацию об объектах или вещах
Хранилища	Хранит собранную информацию в различных формах: сырые данные, текст, видео, изображения, знания, модели и т. п.
Процессоры	Реализует разного рода обработку, в частности, формирует разное представление собранных данных с соответствии с запросами пользователей

Архитектурные слои. Функциональность описанных выше модулей может быть распределена по слоям разными способами. Способ разбиения *IoT*-системы на слои во многом зависит от области ее применения, размеров и требований многих других факторов. Известно достаточно много эталонных архитектур *IoT*, которые различаются в основном числом слоев. Известные эталонные архитектуры могут содержать от трех до шести слоев (рис. 3.19)

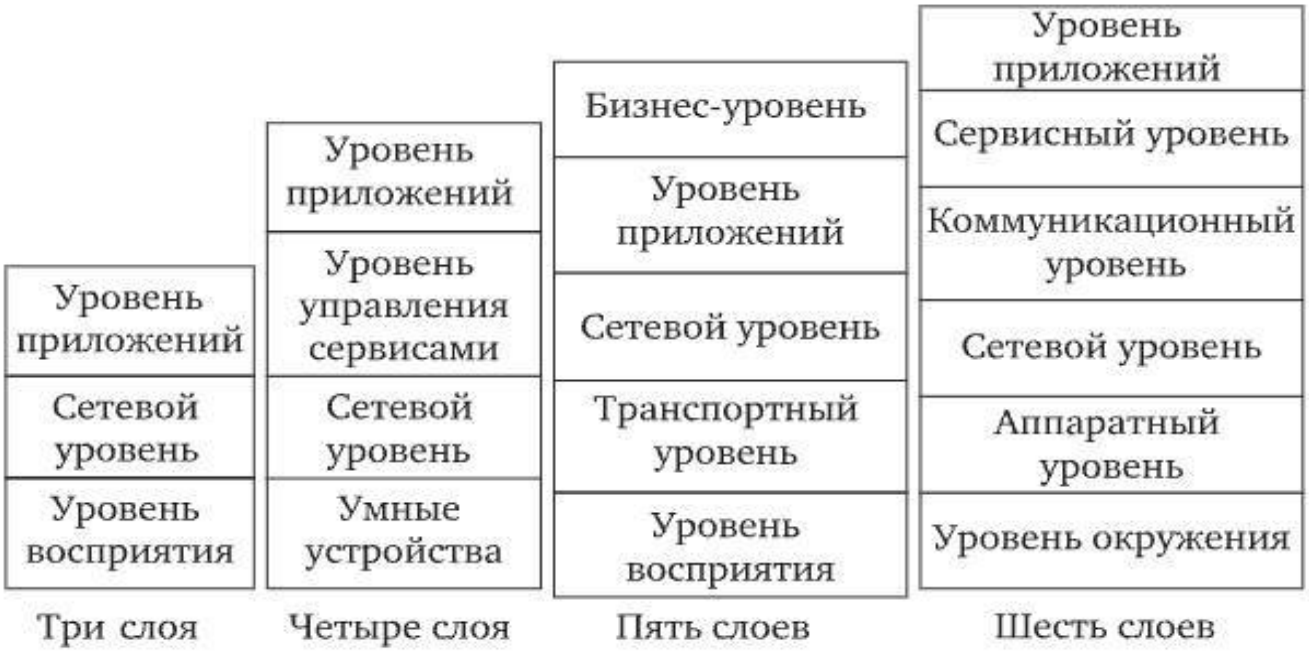


Рис. 3.19. **Уровневые архитектуры**

Кратко опишем первые три из них.

Трехслойную архитектуру можно рассматривать как базовую эталонную архитектуру *IoT* (табл. 3.5). Это наиболее простая с точки зрения реализации архитектура. В ней присутствуют только три уровня: уровень восприятия (*Perception Layer*), сетевой уровень (*Network Layer*) и уровень приложений (*Application Layer*). Данная архитектура достаточно хорошо работает для простых систем, в которых требуется минимальная обработка: например, для дачных систем, где требуется следить за температурой и при ее понижении закрывать форточки и регулярно включать на определенное время капельный полив.

Таблица 3.5

Трехслойная архитектура IoT и поуровневая функциональность

Слой	Реализуемая функциональность
Слой восприятия	Сбор информации об окружающей среде
Сетевой слой	Возможность передачи и обработки информации с подключением к Интернету различных устройств
Слой приложений	Предоставление пользователю конкретных сервисов

Четырехслойная архитектура существует как некоторый компромисс между простейшей трехслойной и многослойными архитектурами. Данная архитектура включает следующие четыре слоя: слой сенсоров и умных устройств (*Smart Device / Sensor Layer*), слой шлюзов и сетей (*Gateways and Networks Layer*), слой управления сервисами (*Management Service Layer*) и слой приложений (*Application*

Layer). Данная архитектура может использоваться во многих типах *IoT*-систем. Первоначально она была ориентирована на такие системы как умный дом, умное здание и умный город.

Таблица 3.6

Четырехслойная архитектура *IoT* и поуровневая функциональность

Слой	Реализуемая функциональность
Слой сенсоров и умных устройств	Сбор и предварительная обработка информации
Шлюзы и сети	Надежная и высокопроизводительная проводная или беспроводная сетевая инфраструктура, используемая в качестве транспортного средства
Слой управления сервисами	Обработка информации с помощью аналитики, средств контроля безопасности, моделирования процессов и управления устройствами
Слой приложений	Предоставление пользователю конкретных сервисов

Для систем средней сложности трехслойной архитектуры явно недостаточно. В этом случае обычно используют *пятислойную* архитектуру. Имеются разные варианты пятиуровневой эталонной архитектуры *IoT*.

Пятислойная архитектура [28] аналогична трехслойной архитектуре, но с добавлением дополнительных двух слоев. В трехслойную модель добавляется два дополнительных слоя: транспортный слой (*Transport Layer*) и бизнес слой (*Business Layer*).

Роль и функционирование каждого слоя показаны в табл. 3.7. Данная архитектура ориентирована на системы, основными пользователями которой являются бизнес-аналитики.

Таблица 3.7

Пятислойная архитектура IoT и поуровневая функциональность

Слой	Реализуемая функциональность
Слой восприятия	Сбор информации об окружающей среде
Транспортный слой	Передача информации с нижнего уровня, полученной с помощью датчиков, на уровень обработки или наоборот через беспроводную связь, *3G*, локальную сеть, *Bluetooth*, *RFID*
Сетевой слой	Возможность передачи и обработки информации с подключением к Интернету различных устройств
Слой приложений	Предоставление пользователю конкретных сервисов.
Бизнес–слой	Управление всей *IoT*–системой, включая приложения на основе бизнес–моделей.

Известны *IoT*–архитектуры с большим числом слоев, которые могут включать до 7 слоев [28].

Более подробное рассмотрение четырехслойной архитектуры. На рис. 3.20 показано, какие технологии могут использоваться на каждом из четырех уровней. Из рис. 3.20 видно, что в каждой отдельной *IoT*-системе может использоваться большое число разнообразных технологий.

Функциональность каждого слоя описана ниже.

Самый нижний слой состоит из интеллектуальных (*smart*) объектов, интегрированных с датчиками. Датчики обеспечивают взаимосвязь физического и цифрового миров, позволяя собирать и обрабатывать информацию в режиме реального времени. Существуют различные типы датчиков для различных целей. Датчики способны производить измерения, такие как: температура, качество воздуха, скорость, влажность, давление, расход, движение, электричество и т. д. В некоторых случаях они также могут обладать памятью, позволяющей им записывать определенное количество измерений. Датчик может измерять физическое свойство и преобразовывать его в сигнал, который может быть понят прибором. Датчики сгруппированы в соответствии с их уникальным назначением: датчики окружающей среды, датчики тела, датчики бытовой техники, датчики телеметрии транспортных средств и т. д.

Рис. 3.20. **Архитектура IoT**

Большинство датчиков требуют подключения к сенсорным шлюзам. Это может быть в виде локальной сети (*LAN*), такой как подключение *Ethernet* или *Wi-Fi*, или сетей более низкого уровня, таких как *ZigBee*, *Bluetooth*.

Для датчиков, которым не требуется подключение к хабу датчиков, для их подключения к внутренним серверам/приложениям можно использовать *GSM*, *GPRS* или *LTE*. Датчики с низким энергопотреблением и низкой скоростью передачи данных обычно образуют беспроводные сенсорные сети (*Wireless Sensor Networks*, *WSN*). *WSN* позволяют объединить достаточно большое количество сенсорных узлов, сохраняя при этом достаточное время автономной работы и покрывая большие площади.

Большой объем данных, создаваемых на первом уровне *IoT* многочисленными миниатюрными сенсорами, требует надежной и высокопроизводительной проводной или беспроводной сетевой инфраструктуры в качестве транспортной среды. Для реализации широкого спектра сервисов и приложений в *IoT* необходимо обеспечить совместную работу множества сетей, построенных на базе различных технологий, и протоколов доступа в гетерогенной конфигурации. Эти сети должны обеспечивать требуемые значения качества передачи информации, прежде всего по задержке, пропускной способности и безопасности. Данный уровень состоит из конвергентной сетевой инфраструктуры, которая создается путем интеграции разнородных сетей в единую сетевую платформу.

Конвергентный абстрактный сетевой уровень в *IoT* позволяет через соответствующие шлюзы нескольким пользователям использовать ресурсы в одной сети независимо и совместно без ущерба для конфиденциальности, безопасности и производительности.

Способы взаимодействия с интернет-вещами. В Интернете вещей шлюз используется не только для прямой связи интернет-вещей с пользователем, но и при использовании централизованного сервера. Существует несколько способов организации шлюзов.

Первый способ заключается в использовании компьютеров, которые имеют точку доступа к глобальной сети Интернет, и каждая из объединяемых сетей подключена к такому компьютеру. Основными недостатками такого подхода являются стоимость и громоздкость.

Второй способ заключается в использовании устройства-шлюза, позволяющего соединить сенсорную сеть с ближайшей проводной сетью, имеющей выход в Интернет. Такой проводной сетью, как правило, является *Ethernet*. Устройство имеет в себе приемопередатчик, совместимый с объединяемой сенсорной сетью, порт для подключения к *Ethernet* и микроконтроллер, выполняющий функции преобразования пакетов одной сети в формат другой. Такой способ отличается меньшей стоимостью, чем первый, при этом размер устройства небольшой, но оно нуждается в относительно высоком энергопотреблении.

Третий способ заключается в использовании устройства-шлюза, которое является полностью автономным и само предоставляет точку доступа к сети Интернет. Это возможно при использовании беспроводных технологий передачи данных. Устройство состоит

из одного приемопередатчика, совместимого с сенсорной сетью и второго — совместимого с той или иной глобальной беспроводной сетью, в область действия которой попадает сенсорная сеть. Такими сетями могут служить *GSM* или *WiMAX*. Использование сети GSM является более экономичным в плане энергопотребления.

Существуют также шлюзы, предоставляющие доступ сенсорным сетям к ближайшим сетям *Wi-Fi* для поиска точки доступа к сети Интернет.

Таким образом, если необходимо организовать полностью автономную территориально-распределенную сенсорную сеть, то следует использовать третий способ. Если же сенсорная сеть используется как часть какой-либо крупной проводной сети, то нет необходимости в ее полной автономности и возможно использование первых двух способов.

Сервисный уровень содержит набор сервисов, призванных автоматизировать технологические и бизнес операции в *IoT*: поддержки операционной и бизнес-деятельности, различной аналитической обработки информации (статистической, интеллектуального анализа данных и текстов, прогностической аналитики и др.), хранения данных, обеспечения информационной безопасности, управления бизнес-правилами, управления бизнес-процессами и др.

На четвертом уровне архитектуры *IoT* существуют различные типы приложений для соответствующих промышленных секторов и сфер деятельности (энергетика, транспорт, торговля, медицина, образование и др.). Приложения могут быть «вертикальными», когда они являются специфическими для конкретной отрасли промышленности, а также «горизонтальными», (например, управление автопарком, отслеживание активов и др.), которые могут использоваться в различных секторах экономики.

3.6.2. Промышленный интернет вещей.

Промышленный интернет вещей (*IIoT*) можно рассматривать как платформу для построения Индустрии 4.0. *IIoT* представляют собой индустриальную сервисную экосистему, построенную на базе *IoT* инфраструктуры, способную гибко взаимодействовать с промышленным оборудованием и поддерживать производственные бизнес–процессы и адаптироваться к изменениям производственных процессов. В основе *IIoT* лежит идея объединения механизмов межмашинного (*M2M*) и анализа данных, которые поступают от промышленного оборудования.

IIoT должен обладать шестью базовыми возможностями:

1) интеллектуальное восприятие;
2) способность взаимодействовать с самым широким кругом устройств;
3) реализовывать сложные процедуры управления;
4) работать с цифровыми моделями;

5) выполнять в реальном времени анализ данных;

6) реализовывать в реальном времени процедуры оптимизации как структур, так и бизнес–процессов.

Наиболее важным для *IIoT*-систем представляется возможность обрабатывать и «осмысливать» огромные объемы данных, которые поступают от производственных подсистем, логистики, подразделений, связанных с продажами. Это могут быть данные от *RFID*, аппаратуры, информация о складских запасах, информация о персонале и многое другое.

Для *IIoT* характерен очень высокий уровень гетерогенности структуры и разнообразие БП, как следствие использования большого числа разнообразных протоколов и стандартов для поддержания связей между оборудованием и людьми.

Цифровое моделирование является средством представления производственных ресурсов и создаваемых изделий в цифровой форме, что открывает широкие возможности по моделированию бизнес-процессов производства и управлению самим производством с использованием моделей.

Использование *IIoT* предполагает возможность обрабатывать в реальном времени в цифровом пространстве весь объем данных, поступающих от всех ресурсов, используемых в производственном процессе и на основе анализа этих данных выдавать управляющие воздействия для внешних физических сущностей.

Используемый *IIoT* должен обладать способностью к самоанализу и обучению. Это должна быть открытая система, которую можно постоянно совершенствовать и развивать, как в плане оптимизации структуры и (или) масштабирования, так и в плане постоянного совершенствования БП.

Использование *IIoT*, обладающих перечисленными выше свойствами, должно привести к кардинальным изменениям в самом подходе к организации процесса производства, в основу которого предлагается положить такие технологии как адаптивное управление оборудованием, прогнозное техническое обслуживание оборудования, цифровые двойники, 3D–печать и др. Производственные системы, построенные на платформе *IIoT* также называют киберфизическими производственными системами. Одним из серъезных вызовов при создании киберфизических производственных систем является стандартизация, поскольку уровень их гетерогенности крайне высок. Можно считать, что *IIoT* является базовой платформой для четвертой промышленной революции.

Следует заметить, что понятие *IIoT* во многом совпадает с понятием «Умное производство (*Smart Manufacturing*)». Однако это понятие является более широким, поскольку оно включает в себя само производственное оборудование, персонал, производственные процессы и конечную продукцию. *IIoT* можно рассматривать как платформу, на которой строится подсистема управления умным произ-

водством. *IIoT* также можно рассматривать как двигатель цифровых трансформаций общества и производства.

С технической точки зрения, *IIoT* представляет собой множество взаимосвязанных датчиков, приборов и другим устройств, объединенных в сеть вместе с промышленными приложениями компьютеров, включая производство и управление энергопотреблением. Эта связь позволяет осуществлять сбор, обмен и анализ данных, потенциально способствуя повышению производительности и эффективности, а также другим экономическим выгодам. *IIoT* — это эволюция распределенной системы управления, которая обеспечивает более высокую степень автоматизации за счет использования облачных и туманных вычислений для контроля и оптимизации БП.

Современные IIoT строятся на платформах IoT по принципу туманных вычислений.

Эталонная архитектура *IIoT* определена в стандарте, разработанном Консорциумом промышленного интернета в 2019 году — «Эталонная архитектура промышленного интернета» [29]. В состав консорциума входят такие компании как: *Toshiba*, *IBM*, *General Electric*, *SAP*, *Fujitsu*, *Oracle* и ряд других. Данный документ предоставляет рекомендации для архитекторов ИС, технологов, менеджеров разного уровня и пользователей для согласования их усилий по созданию производственных систем нового поколения с принципиально новыми функциональными возможностями.

В рамках эталонной архитектуры *IIoT* определяются четыре уровня, которые соответствуют четырем точкам зрения (рис. 3.21):

— извлечение прибыли;
— использование ресурсов;
— функционирование;
— реализация.

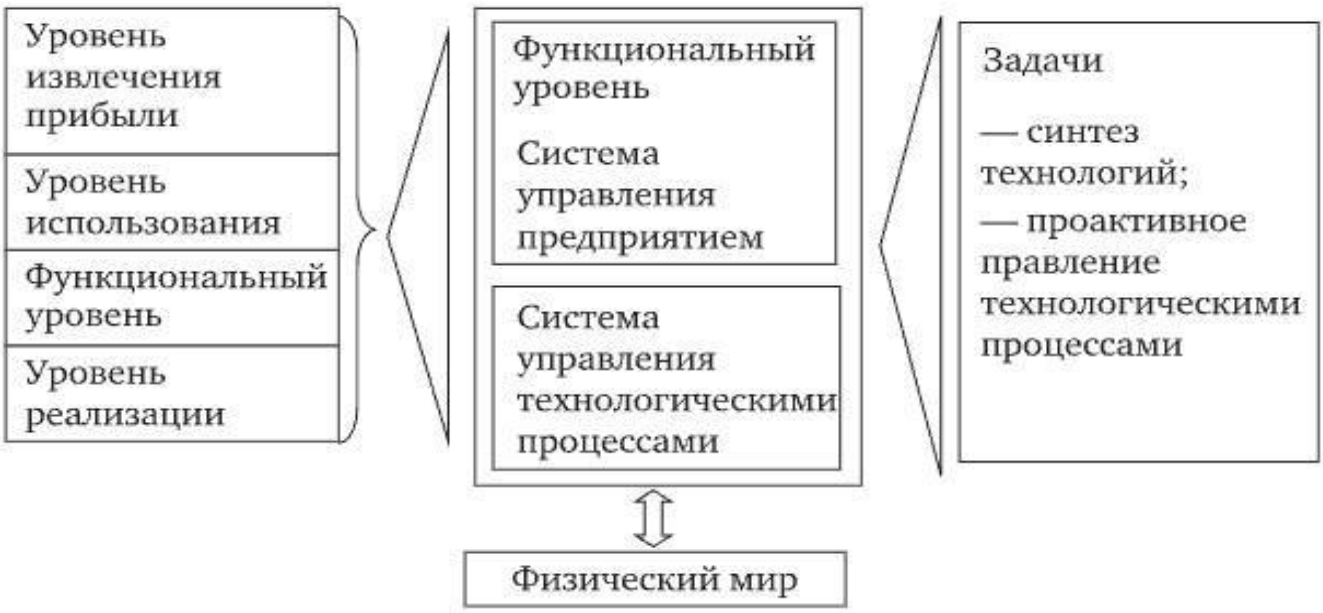


Рис. 3.21. **Место задач управления информационными процессами в эталонной архитектуре IIoT**

На верхнем уровне (уровне извлечения прибыли) важной функцией с точки зрения проблем управления производственными процессами является формирование критериев качества функционирования, а также учет экономической эффективности, которые в дальнейшем должны учитываться при организации работы всего предприятия на основе Промышленного интернета вещей.

На уровне использования ресурсов формируются представления о том, какие специалисты и какое оборудование будет использовано в технологических и управленческих процессах промышленного предприятия.

Следует заметить, что верхние два уровня отсутствуют в стеках протоколов *IoT*.

Следующие два уровня рассматривают IIoT с точки зрения БП и реализации.

Уровень функционирования *IIoT* сконцентрирован на функциональных компонентах, их взаимосвязи, а также их связи с внешними системами. Фактическое исполнение этих компонентов описывается на уровне реализации.

На функциональном уровне проблемы построения промышленного интернета делятся на пять областей:

— системы управления технологическими процессами;
— системы управления информационными процессами;
— системы обработки данных и информации;
— прикладные системы;
— системы управления предприятием.

К настоящему времени в мировой практике достигнуты значительные результаты в четырех из этих направлений. Автоматизированные системы управления технологическими процессами выполняют короткий цикл управления, заключающийся в получении данных о физическом мире с помощью сенсоров, применении правил и операций их обработки и формировании управления для воздействия на физический мир. Такие системы всегда располагаются максимально близко к физическим производственным процессам, вследствие чего могут быть значительно территориально распределены и не согласованы на уровне целей функционирования всего предприятия (уровень бизнеса).

Системы обработки данных и информации являются связующим звеном между всеми уровнями промышленного интернета. Наиболее кратко можно охарактеризовать цель этой системы как совокупность операций обработки данных, направленных на согласование и обеспечение их высокого качества для всех элементов промышленного интернета.

Прикладные системы функционируют на высоком уровне, доводя до остальных уровней результаты работы систем управления предприятием.

Последняя, но не менее важная область функционального уровня *IIoT* — системы управления информационными процессами. В их задачи входит обеспечение работоспособности, мониторинг, управление и оптимизация функционирования распределенных автоматизированных систем управления технологическими процессами. Если традиционные системы управления производством фокусируются на конкретных технологических процессах одного предприятия, то системы управления информационными процессами в промышленном интернете функционируют уровнем выше, организуя совместную работу различных участников производства: от заказчиков до множества совместных предприятий. Ниже представлено краткое описание функций систем управления информационными процессами.

1. Функция подготовки и развертывания сервисов выполняет настройку, регистрацию и отслеживание работоспособности сервисов, их вывода из эксплуатации. Эти функции должны обеспечивать масштабирование, а также удаленное и безопасное и взаимодействие сервисов в оперативном режиме в условиях жестких динамичных ограничений и возмущающих воздействий, характерных для промышленного контекста.
2. Функция координации работы локальных (унаследованных) автоматизированных системам управления производством состоит в работе с набором программно-аппаратных адаптеров, позволяющих доводить управляющие воздействия верхнего уровня до систем управления производством, и, наоборот, получать от них обратную связь.
3. Функция мониторинга и диагностики позволяет обнаруживать и прогнозировать возникновение проблем в системах промышленного интернета, диагностировать реальную причину проблемы и предупреждать об отклонениях в их работе.
4. Функция прогнозирования состояния ресурсов позволяет, опираясь на модели их функционирования и накопленные данные, выявлять потенциальные проблемы до их возникновения и предоставлять рекомендации по их устранению. Данная функция является важной частью управления ресурсами *IIoT*.
5. Функция оптимизации предназначена для повышения надежности и производительности предприятия.
6. Функция оптимизации предназначена для повышения надежности и производительности предприятия, уменьшения суммарного потребления энергии за счет рационального использования ресурсов. Эти функции должны быть максимально автоматизированы для сокращения доли ошибок, связанных с человеческим фактором при работе с большими объемами данных и информации.

Последний (нижний) уровень эталонной архитектуры промышленного интернета представлен уровнем реализации, описывающим технические и технологические решения, которые могут быть

применены для выполнения функций всех других уровней системы. Далее представлен обзор современных программно-аппаратных решений и технических архитектур на их основе.

Цифровые двойники (Digital twins)

Первое использование концепции близнецов (twins) можно проследить в проекте «Аполлон», в рамках которого НАСА построило несколько космических аппаратов. Корабль на земле был точной копией корабля в космосе и использовалось в учебных целях, а также для имитации решений в критических ситуациях. Подобный подход также использовался в авиастроении, где близнецы использовались для испытаний (управление полетом, гидравлика).

Digital Twin Model (*DTM*) была впервые представлена в 2002 году как концепция управления жизненным циклом продукта (*Product Life Cycle Management, PLM*) без указания названия модели. Модель вскоре была названа, первоначально Зеркальной Моделью Пространств (*MSM*), но позже была изменена на Модель Зеркального Отображения Информации (*IMM*).

Хотя эта концепция и не называлась *DTM*, она включала в себя все, что содержал цифровой близнец и определялась как виртуальная сущность, которая содержит три основные части:

а) физическую сущность, находящуюся в физическом пространстве;

б) виртуальную сущность, находящуюся в виртуальном пространстве;

в) соединения в форме данных и информации, которые связывают виртуальную и реальную сущности в единую систему [30].

Модель, наконец, была названа *DT* [31]. В последние годы наблюдается устойчивая тенденция к повышению интереса практического использования концепции *DT* и расширению сферы ее применения.

В основе современной концепции *DT* лежит идея модельного сопровождения системы на всех этапах ее жизненного цикла.

В настоящее время известно несколько определений понятия *DT*. Чаще всего его определяют, как виртуальное представление объекта любой природы, которое двунаправленно связано с двойником. *DT* можно рассматривать как набор моделей для поддержки всех заинтересованных сторон в течение всего жизненного цикла системы или объекта.

В этом определении важными являются два момента. Во-первых, с помощью цифрового двойника может быть представлен не только физический объект, но и БП, а также система любой физической природы, включая людей. Следует также заметить, что с помощью *DT* может быть представлена и программная система. Вторым важным моментом является то, что сама моделируемая система может управлять состоянием *DT*. Это может быть полезно, когда модели-

руемая система, например, изменила свое состояние. В этом случае *DT* также изменяет состояние.

Известно и еще одно определение, в соответствии с которым, *DT* определяется как набор данных, информации и знаний, которые представляют структуру, поведение и контекст наблюдаемой и управляемой системы, а *DT* предлагает интерфейс, позволяющий получать эти данные, информацию и знания о прошлой и нынешней работе наблюдаемой и управляемой системы и делать прогноз о будущем состоянии наблюдаемой и управляемой системы.

В этом определении важными является следующие моменты:

— *DT* является в общем случае многоуровневой и многослойной системой, которая может включать в себя другие модели;

— на разных этапах жизненного цикла используются разные модели;

— *DT* могут строиться с использованием разных методологий программирования (распределенные системы объектов, компонентные технологии *SOA*, *Multi-Agent Systems* (*MAS*);

— *DT* могут строиться в терминах знаний;

— для построения *DT*, соответствующим разным подсистемам одной системы, могут использоваться различные технологии и платформы.

Известны и другие определения *DT*, которые отражают несколько отличные точки зрения на *DT*. Наличие достаточно большого числа определений понятия *DT* указывает на то, что это многоаспектное понятие.

В зависимости от специфики конкретной решаемой с помощью *DT* задачи и роли конкретного заинтересованного лица (*SH*) можно выделить следующие основные точки зрения на DT:

1) *DT* — это модель (система моделей), сопровождающая систему на протяжении всего времени ее существования.

2) *DT* — это порт доступа к другим системам и мирам, в частности, физическим системам.

3) *DT* — это интеллектуальный контроллер некоторой сущности.

4) *DT* — это архитектурный стиль.

5) *DT* — это сама система управления заказами (*Order Management System*, *OMS*).

В первом случае речь идет о реализации концепции цифровой нити [32].

Во втором случае *DT* рассматривается как средство построения системы систем (*System of Systems*, *SoS*). Каждая система, входящая в состав *SoS*, может иметь произвольное число *DT*, которые описывают систему на языке, понятном другим системам, входящим в состав *SoS*. В этом случае *DT* можно рассматривать как сервис, доступный другим системам, входящим в состав *SoS*.

В третьем случае *DT* рассматривается как средство реализации механизма адаптивности. *DT* управляет реконфигурацией наблюдаемой и управляемой системы.

В четвертом случае *DT* рассматривается с точки зрения программной архитектуры. DT можно рассматривать как одну из возможных реализаций архитектурного стиля виртуальной машины. Однако его не следует объединять с другими реализациями, такими как промежуточное ПО, интерпретаторы и др. [13].

В пятом случае система, состоящая из реконфигураций наблюдаемой и управляемой системы и *DT*, рассматривается как единая система, доступ к которой осуществляется через *DT*. Это пользовательская точка зрения.

Эти точки зрения можно рассматривать как архитектурные точки зрения.

Возможны разные подходы к использованию *DT* в реальных системах. Чаще всего встречаются два типа центров данных (ЦД).

ЦД первого типа являются полным функциональным аналогом наблюдаемой и управляемой системы и реализуют все сервисы, которые предусмотрены в системе. Такой подход позволяет заменять физические системы на их модели: например, на этапе проектирования, отладки, испытаний.

ЦД второго типа является некоторой частной моделью наблюдаемой и управляемой системы, которая строится для решения конкретной задачи в интересах конкретного пользователя. Например, такой подход можно использовать для сбора данных и представления результатов в виде модели.

С понятием *DT* тесным образом связано понятие цифровой тени [28]. Применительно к наблюдаемой и управляемой системе с вариабельной структурой и поведением понятие цифровой тени можно определить как моментальный снимок, отображающий состояние наблюдаемой и управляемой системы на конкретный момент времени. Множество цифровых теней, относящихся к отдельным упорядоченным во времени моментам времени, образуют теневой след. В отличие от *DT*, цифровая тень является пассивной, и она только отображает текущее состояние наблюдаемой и управляемой системы.

DT может быть реализован разными способами. Он может быть либо встроенным в саму управляемую систему, либо реализованным как отдельный компонент.

Цифровые нити (Digital threads)

Идея цифровой нити в определенном смысле обобщает идею *DT* и состоит в следующем. *DT* является моделью, которая используется на этапе эксплуатации. С помощью множества цифровых теней можно отследить поведение наблюдаемой и управляемой системы

во времени. Идея цифровой нити состоит в том, чтобы отследить состояние системы на всех этапах жизненного цикла, начиная от момента появления идеи создания системы до момента окончания ее использования.

Цифровую нить обычно определяют как систему моделей, которые описывают систему на протяжении всего ее жизненного цикла.

Таким образом идея состоит в том, чтобы построить некоторый континиум моделей, которые описывают систему на протяжении всего времени жизненного цикла, включая проектирование, производство, эксплуатацию и модернизацию.

В этом контексте DT — это одна из моделей, которая используется на этапе эксплуатации. В этом случае процесс проектирования включает два параллельных процесса: процесс разработки системы и процесс разработки ее моделей, в качестве которых выступает DT. Следует заметить, что на разных этапах используется разный набор моделей. Например, на этапе производства КФС используются модели оборудования и модели технологического процесса.

Цифровые нити можно рассматривать как процесс оцифровки и отслеживания жизненного цикла продукта. Цифровая нить соединяет все различные возможности цифрового двойника с конструкциями деталей, требованиями и программным обеспечением, которые входят в продукт, представленный цифровым двойником.

Можно сказать, что цифровая нить и полная отслеживаемость продукта на протяжении всего жизненного цикла являются синонимами. В этом случае можно говорить, что цифровая нить является механизмом управления жизненного цикла изделия.

3.7. Технологии программирования

Развитие технологии программирования происходит на фоне возрастающих требований к качеству ПО, что определяется увеличением сложности и ответственности выполняемых им задач. Если на начальном этапе создание качественного ПО во многом зависело от опыта и квалификации разработчика, то на современном этапе акцент смещается в сторону инструмента, которым он пользуется, и средств автоматизации.

На всем протяжении развития технологией программирования разработчики стремились к тому, чтобы по некоторому неформальному описанию объекта программирования автоматически генерировался бы текст синтаксически и семантически корректной программы. Однако на данном этапе процесс далек от такого идеального варианта.

В настоящее время любые технологии разработки ПО представляют процесс, состоящий из ряда последовательных преобразова-

ний одного описания решаемой задачи в другое, начиная от постановки задачи и заканчивая программой, реализованной в кодах конкретной ЭВМ.

Рассмотрим краткую ретроспективу развития методов и средств разработки ПО [21, 22].

Установка ключевых переключателей на передней панели вычислительных устройств — предпосылка первых программ.

Машинный язык — возможность задавать команды, оперируя с ячейками памяти, полностью используя возможности компьютера. Однако использование большинства ЭВМ на уровне машинного языка затруднительно, особенно это касается ввода-вывода. Поэтому от его использования пришлось отказаться.

Специализированные или проблемно- (предметно-)ориентированные языки — построены для решения конкретных задач, упрощают процесс разработки программных продуктов в выбранной предметной области, но в то же время для новых языков характерна проблема преемственности, а для специализированных — ограниченности области применения. На этом этапе следует выделить развитие таких языков программирования, как КОБОЛ, СИМУЛА, СИМСКРИПТ, АСПИД, АНАЛИТИК и т. д.

Постепенное усложнение решаемых задач, а вместе с ними кодов и объемов разрабатываемых программных средств, заставило критически переосмыслить сложившийся к концу 1960-х гг. стиль и технику программирования. В связи с чем начиная с 1970-х гг. усилиями рабочей группы Технического комитета по программированию Международной федерации по обработке информации (*International Federation for Information Processing — IFIP* — ИФИП) формируются основы методологии и теории программирования и утверждается *принцип структурного программирования* как наиболее производительного и привлекательного стиля программирования Развитие концепции *структуризации* привело к осознанию необходимости структуризации данных и определило появление в языках программирования *механизмов абстрагирования типов данных* (Клу, Модула и др.). Последнее легло в основу *техники модульного программирования*, содержательной основой которой является интерпретация типа как алгебры над множеством объектов и множеством операций, а модуля — как программного эквивалента абстрактного типа. В модульном программировании акцент делается на разбиение программы на модули таким образом, чтобы данные (обрабатываемые модулем) были спрятаны в нем. Эта доктрина, известная как «принцип ограничения доступа к данным», в значительной степени повысила модифицируемость и эффективность порождаемого кода.

Развитие техники модульного программирования привела к появлению *объектно-ориентированного стиля программирования*,

который во многом унифицировал процесс создания ПО. К достоинствам этого метода относится:

- более полная реализация технологии структурного программирования;
- упрощение процесса создания сложных иерархических систем;
- возможность создания пользовательских библиотек объектов в различных областях применения.

Параллельно с развитием процедурного стиля программирования в начале 1970-х гг. появляются непроцедурные *языки логического программирования и программирования искусственного интеллекта* (*LISP*, Пролог, РЕФАЛ, ПРИЗ). Программа в таких языках представляет собой совокупность *правил* (определяющих отношения между объектами) и *цели* (запроса). Процесс выполнения программы трактуется как процесс установления общезначимости логической формулы по правилам, установленным семантикой того или иного языка. Результат вычислений является побочным продуктом процедуры вывода. Такой метод являет собой полную противоположность программирования на каком-либо из процедурных языков. Сегодня Пролог — язык, предназначенный для программирования приложений, использующих средства и методы искусственного интеллекта, создания экспертных систем и представления знаний.

Программный продукт является результатом некоего технологического процесса, включающего следующие основные этапы:

- спецификация;
- проектирование;
- кодирование;
- отладка;
- документирование;
- тестирование;
- сопровождение;
- реинжиниринг.

Спецификация. Формирование спецификации предназначено для определения сервисов, которыми будет обладать проектируемое ПО, а также ограничений, накладываемых на функциональные возможности и разработку программной системы. Этот процесс в настоящее время обычно называют «разработка требований» (*requirements engineering*). Разработка требований часто является критическим этапом в создании ПО, поскольку ошибки, допущенные на этом этапе, ведут к возникновению проблем на этапах проектирования и разработки.

Процесс разработки требований включает четыре основных этапа.

1. ***Предварительные исследования.*** Оценивается степень удовлетворенности пользователей существующими программными продуктами и аппаратными средствами, а также экономическая

эффективность будущей системы и возможность уложиться в существующие бюджетные ограничения при ее разработке. Этот этап должен быть по возможности коротким и дешевым.

2. ***Формирование и анализ требований.*** Формируются системные требования путем изучения существующих аналогичных сметем, обсуждения будущей системы с потенциальными пользователями и заказчиками, анализа задач, которые должна решать система, и т. п. Этот этап может включать разработку нескольких моделей системы и ее прототипов, что помогает сформировать функциональные требования к системе.

3. ***Специфицирование требований.*** Осуществляется перевод всей совокупности информации, собранной на предыдущем этапе, в документ, определяющий множество требований. Этот документ обычно содержит два типа требований: пользовательские — обобщенные представления заказчиков и конечных пользователей о системе; системные — детальное описание функциональных показателей системы.

4. ***Утверждение требований.*** Проверяется выполнимость, согласованность и полнота множества требований. В процессе формирования ограничений неизбежно возникновение каких-либо ошибок. На этом этапе они должны быть по возможности выявлены и устранены.

Проектирование. Целью проектирования является определение внутренних свойств системы и детализации ее внешних (видимых) свойств на основе выданных заказчиком требований к ПО (исходные условия задачи). Эти требования подвергаются анализу. Первоначально программа рассматривается как черный ящик. Ход процесса проектирования и его результаты зависят не только от состава требований, но и выбранной модели процесса, опыта проектировщика.

В зависимости от класса создаваемого ПО, процесс проектирования может обеспечиваться как «ручным» проектированием, так и различными средствами его автоматизации. Проектированию обычно подлежат:

- архитектура ПО;
- устройство компонентов (модулей) ПО;
- пользовательские интерфейсы.

В ходе проектирования архитектором или опытным программистом создается проектная документация, включающая текстовые описания, диаграммы, модели будущей программы.

Проектирование **ПО** — этап жизненного цикла ПО, во время которого исследуется структура и взаимосвязи элементов разрабатываемой системы. Результатом этого этапа является проект, содержащий достаточное количество информации для реализации системы.

Жизненный цикл программного обеспечения — непрерывный процесс, который начинается с принятия решения о необходимости создания ПО и заканчивается при полном изъятии его из эксплуатации.

Результат процесса проектирования — *дизайн*. Рассматриваемое как процесс проектирование есть инженерная деятельность в рамках жизненного цикла (в данном контексте — программного обеспечения), в которой надлежащим образом анализируются требования для создания описания внутренней структуры ПО, являющейся основой для конструирования программного обеспечения как такового. Программный дизайн (как результат деятельности по проектированию) должен описывать архитектуру программного обеспечения, т. е. представлять декомпозицию программной системы в виде организованной структуры компонент и интерфейсов между компонентами. Важнейшей характеристикой готовности дизайна является тот уровень детализации компонентов, который позволяет заняться их конструированием. Термины «дизайн» и «архитектура» могут использоваться взаимозаменяемым образом, но чаще говорят о дизайне как о целостном взгляде на архитектуру системы.

Проектирование играет важную роль в процессах жизненного цикла создания программного обеспечения (*Software Development Life Cycle*), например, *IEEE* и *ISO/IEC* (ГОСТ Р ИСО.МЭК) 12207. Проектирование программных систем можно рассматривать как деятельность, результат которой состоит из двух составных частей:

- архитектурный или высокоуровневый дизайн (*Software Architectural Design, Top-Level Design*) — описание высокоуровневой структуры и организации компонентов системы;
- детализированная архитектура (*Software Detailed Design*) — описывающая каждый компонент в том объеме, который необходим для конструирования.

Кодирование — процесс написания программного кода, скриптов с целью реализации определенного алгоритма на определенном языке программирования.

Разработка программ начинается с постановки технического задания. От того, насколько грамотным и точным оно будет, во многом зависит продуктивный результат. Требования к будущему ПО должны быть тщательно проанализированы и точно сформулированы. На этом этапе главная задача — собрать как можно больше информации, для этого стоит пообщаться и с руководством, и с теми, кто будет непосредственно пользоваться софтом. Итогом этого этапа должно стать техническое задание, детально описывающее действия программного продукта и ожидаемых от нее результатов.

Программирование — процесс и искусство создания компьютерных программ и (или) программного обеспечения с помощью языков программирования. Программирование сочетает в себе элементы искусства, фундаментальных наук (прежде всего информатика и математика), инженерии, спорта и ремесла.

Кодирование программного обеспечения — собственно разработка программы или написание кода программы. Используемые технологии различны, и у каждого специалиста в этой сфере есть свои предпочтения. На данном этапе осуществляется создание ПО с учетом выставленного технического задания. Некоторые путают такие понятия, как программирование и непосредственно кодирование. Кодирование является лишь частью программирования наряду с анализом, проектированием, компиляцией, тестированием и отладкой, сопровождением. В узком смысле слова программирование рассматривается как кодирование алгоритмов на заданном языке программирования. Под программированием также может пониматься разработка логической схемы для программируемой логической интегральной схемы, а также процесс записи информации в ПЗУ. В более широком смысле программирование — процесс создания программ, т. е. разработка программного обеспечения.

Процесс кодирования информации может производиться либо ручным, либо автоматическим способом.

Отладка — это процесс локализации и исправления ошибок, обнаруженных при тестировании программного обеспечения [23].

Локализацией называют процесс определения оператора программы, выполнение которого вызвало нарушение нормального вычислительного процесса. Для исправления ошибки необходимо определить ее причину, т. е. определить оператор или фрагмент, содержащие ошибку. Причины ошибок могут быть как очевидными, так и очень глубоко скрытыми. В целом сложность отладки обусловлена следующими причинами:

- требует от программиста глубоких знаний специфики управления используемыми техническими средствами, операционной системы, среды и языка программирования, реализуемых процессов, природы и специфики различных ошибок, методик отладки и соответствующих программных средств;
- психологически дискомфортна, так как необходимо искать собственные ошибки и, как правило, в условиях ограниченного времени;
- возможно взаимовлияние ошибок в разных частях программы, например, за счет затирания области памяти одного модуля другим из-за ошибок адресации;
- отсутствуют четко сформулированные методики отладки.

Различают следующие виды ошибок:

- *синтаксические ошибки* — ошибки, фиксируемые компилятором (транслятором, интерпретатором) при выполнении синтаксического и частично семантического анализа программы;
- *ошибки компоновки* — ошибки, обнаруженные компоновщиком (редактором связей) при объединении модулей программы;
- *ошибки выполнения* — ошибки, обнаруженные операционной системой, аппаратными средствами или пользователем при выполнении программы.

Синтаксические ошибки относят к группе самых простых, так как синтаксис языка, как правило, строго формализован, и ошибки сопровождаются развернутым комментарием с указанием ее местоположения. Определение причин таких ошибок, как правило, труда не составляет, и даже при нечетком знании правил языка за несколько прогонов удается удалить все ошибки данного типа. Следует иметь в виду, что чем лучше формализованы правила синтаксиса языка, тем больше ошибок из общего количества может обнаруживать компилятор и, соответственно, меньше ошибок будет обнаруживаться на следующих этапах. В связи с этим говорят о языках программирования с защищенным синтаксисом и с незащищенным синтаксисом.

Ошибки компоновки, как следует из названия, связаны с проблемами, обнаруженными при разрешении внешних ссылок. Например, предусмотрено обращение к подпрограмме другого модуля, а при объединении модулей данная подпрограмма не найдена или не стыкуются списки параметров. В большинстве случаев ошибки такого рода также удается быстро локализовать и устранить.

Ошибки выполнения относятся к самой непредсказуемой группе. Прежде всего они могут иметь разную природу и, соответственно, по-разному проявляться. Часть ошибок обнаруживается и документируется операционной системой. Выделяют четыре способа проявления таких ошибок:

- появление сообщения об ошибке, зафиксированной схемами контроля выполнения машинных команд, например, при переполнении разрядной сетки, ситуации «деление на ноль», нарушении адресации и т. п.;
- появление сообщения об ошибке, обнаруженной операционной системой, например, нарушении защиты памяти, при попытке записи на устройства, защищенные от записи, отсутствии файла с заданным именем и т. п.;
- «зависание» компьютера, как простое, когда удается завершить программу без перезагрузки операционной системы, так и «тяжелое», когда для продолжения работы необходима перезагрузка;
- несовпадение полученных результатов с ожидаемыми.

Причины ошибок выполнения очень разнообразны, а потому и локализация может оказаться крайне сложной. Все возможные причины ошибок можно разделить на следующие группы:

- неверное определение исходных данных (ошибки передачи, преобразования, перезаписи, неправильные данные);
- логические ошибки (проектирования, кодирования);
- накопление погрешностей результатов вычислений (при некорректном использовании приближенных методов вычислений).

Отладка программы в любом случае предполагает обдумывание и логическое осмысление всей имеющейся информации об ошибке. Большинство ошибок можно обнаружить по косвенным признакам посредством тщательного анализа текстов программ и результатов тестирования без получения дополнительной информации. При этом используют различные методы.

Метод ручного тестирования — самый простой и естественный способ. При обнаружении ошибки необходимо выполнить тестируемую программу вручную, используя тестовый набор, при работе с которыми была обнаружена ошибка. Метод очень эффективен, но не применим для больших программ, программ со сложными вычислениями и в тех случаях, когда ошибка связана с неверным представлением программиста о выполнении некоторых операций. Данный метод часто используют как составную часть других методов отладки.

Метод индукции основан на тщательном анализе симптомов ошибки, которые могут проявляться как неверные результаты вычислений или как сообщение об ошибке. Если компьютер просто «зависает», то фрагмент проявления ошибки вычисляют, исходя из последних полученных результатов и действий пользователя. Полученную таким образом информацию организуют и тщательно изучают, просматривая соответствующий фрагмент программы. В результате этих действий выдвигают гипотезы об ошибках, каждую из которых проверяют. Если гипотеза верна, то детализируют информацию об ошибке, иначе — выдвигают другую гипотезу.

Метод дедукции основан на формировании множества причин, которые могли бы вызвать данное проявление ошибки. Затем, анализируя причины, исключают те, которые противоречат имеющимся данным. Если все причины исключены, то следует выполнить дополнительное тестирование исследуемого фрагмента. В противном случае — наиболее вероятную гипотезу пытаются доказать. Если гипотеза объясняет полученные признаки ошибки, то ошибка найдена, иначе — проверяют следующую причину.

Метод обратного прослеживания эффективен для небольших программ. Начинают с точки вывода неправильного результата. Для этой точки строится гипотеза о значениях основных переменных, которые могли бы привести к получению имеющегося результата.

Далее, исходя из этой гипотезы, делают предложения о значениях переменных в предыдущей точке. Процесс продолжают, пока не обнаружат причину ошибки.

Отладочный вывод — метод, требующий включения в программу дополнительного отладочного вывода в *узловых* точках. Узловыми считают точки алгоритма, в которых основные переменные программы меняют свои значения. Данный метод не очень эффективен и в настоящее время практически не используется, так как в сложных случаях в процессе отладки может потребоваться вывод большого количества значений многих переменных, которые выводятся при каждом изменении. Кроме того, внесение в программы дополнительных операторов может привести к изменению проявления ошибки.

Большинство современных сред программирования (*Delphi, Builder C++, Visual Studio* и т. д.) включают средства отладки, которые обеспечивают максимально эффективную отладку. Они позволяют:

- выполнять программу по шагам, причем как с заходом в подпрограммы, так и выполняя их целиком;
- предусматривать точки останова;
- выполнять программу до оператора, указанного курсором;
- отображать содержимое любых переменных при пошаговом выполнении;
- отслеживать поток сообщений и т. п.

При отладке программ иногда используют специальные программы — *отладчики*, которые позволяют выполнить любой фрагмент программы в пошаговом режиме и проверить содержимое интересующих программиста переменных. Как правило, такие отладчики позволяют отлаживать программу только в машинных командах, представленных в 16-ричном коде.

Суммируя все сказанное выше, можно предложить следующую ***методику отладки программного обеспечения***, написанного на универсальных языках программирования:

1 этап — изучение проявления ошибки — если выдано какое-либо сообщение или выданы неправильные или неполные результаты, то необходимо их изучить и попытаться понять, какая ошибка могла так проявиться. При этом используют индуктивные и дедуктивные методы отладки. В результате выдвигают версии о характере ошибки, которые необходимо проверить. Для этого можно применить методы и средства получения дополнительной информации об ошибке. Если ошибка не найдена или система просто «зависла», переходят ко второму этапу.

2 этап — локализация ошибки — определение конкретного фрагмента, при выполнении которого произошло отклонение от предполагаемого вычислительного процесса. Локализация может выполняться:

- путем отсечения частей программы, причем, если при отсечении некоторой части программы ошибка пропадает, то это может означать как то, что ошибка связана с этой частью, так и то, что внесенное изменение изменило проявление ошибки;
- с использованием отладочных средств, позволяющих выполнить интересующий нас фрагмент программы в пошаговом режиме и получить дополнительную информацию о месте проявления и характере ошибки, например, уточнить содержимое указанных переменных.

При этом если были получены неправильные результаты, то в пошаговом режиме проверяют ключевые точки процесса формирования данного результата. Как подчеркивалось выше, ошибка не обязательно допущена в том месте, где она проявилась. Если в конкретном случае это так, то переходят к следующему этапу.

3 этап — определение причины ошибки — изучение результатов второго этапа и формирование версий возможных причин ошибки. Эти версии необходимо проверить, возможно, используя отладочные средства для просмотра последовательности операторов или значений переменных.

4 этап — исправление ошибки — внесение соответствующих изменений во все операторы, совместное выполнение которых привело к ошибке.

5 этап — повторное тестирование — повторение всех тестов с начала, так как при исправлении обнаруженных ошибок часто вносят в программу новые.

Документирование — это важная часть в разработке программного обеспечения, но часто ей уделяется недостаточно внимания. Существует четыре основных типа документации на ПО [24, 25]:

- архитектурная/проектная — обзор программного обеспечения, включающий описание рабочей среды и принципов, которые должны быть использованы при создании ПО;
- техническая — документация на код, алгоритмы, интерфейсы, *API*;
- пользовательская — руководства для конечных пользователей, администраторов системы и другого персонала;
- маркетинговая.

Архитектурная/проектная документация. Проектная документация обычно описывает продукт в общих чертах. Не описывая того, как что-либо будет использоваться, она, скорее, отвечает на вопрос: «почему именно так?». Например, в проектном документе программист может описать обоснование того, почему структуры данных организованы именно таким образом. Описываются причины, почему какой-либо фрагмент сконструирован определенным образом, выделяются паттерны, в некоторых случаях могут быть даны предложения по улучшению ПО в дальнейшем.

Техническая документация. При создании программы одного лишь кода, как правило, недостаточно. Должен быть предоставлен некоторый дополнительный материал, описывающий различные аспекты реализации кода. Такая документация часто включается непосредственно в исходный код или предоставляется вместе с ним. Подобная документация имеет сильно выраженный технических характер и в основном используется для определения и описания *API*, структур данных и алгоритмов. Часто при составлении технической документации используются автоматизированные средства — генераторы документации, такие как *Doxygen*, *javadoc*, *NDoc* и др. Они получают информацию из специальным образом оформленных комментариев в исходном коде и создают справочные руководства в каком-либо формате, например, в виде текста или *HTML*. Использование генераторов документации и документирующих комментариев многими программистами признается удобным средством по различным причинам. В частности, при таком подходе документация является частью исходного кода, и одни и те же инструменты могут использоваться для сборки программы и одновременной сборки документации к ней. Это также упрощает поддержку документации в актуальном состоянии.

Пользовательская документация. В отличие от технической документации, сфокусированной на коде и том, как он работает, пользовательская документация описывает лишь то, как использовать программу. В случае если продуктом является программная библиотека, пользовательская документация и документация на код становятся очень близкими, почти эквивалентными понятиями. Но в общем случае, это не так. Обычно пользовательская документация представляет собой руководство пользователя, которое описывает каждую функцию программы, а также шаги, которые нужно выполнить для использования этой функции. Хорошая пользовательская документация идет еще дальше и предоставляет инструкции о том, что делать в случае возникновения проблем. Очень важно, чтобы документация не вводила в заблуждение и была актуальной. Руководство должно иметь четкую структуру; очень полезно, если имеется сквозной предметный указатель. Логическая связность и простота также имеют большое значение. Существует три подхода к организации пользовательской документации. Вводное руководство (англ. *tutorial*), наиболее полезное для новых пользователей, последовательно проводит по ряду шагов, служащих для выполнения каких-либо типичных задач. Тематический подход, при котором каждая глава руководства посвящена какой-то отдельной теме, больше подходит для совершенствующихся пользователей. В последнем, третьем, подходе команды или задачи организованы в виде алфавитного справочника — часто это положительно воспринимается продвинутыми пользователями, хорошо знающими,

что они ищут. Жалобы пользователей обычно относятся к тому, что документация охватывает только один из этих подходов, и поэтому хорошо подходит лишь для одного класса пользователей. Во многих случаях разработчики программного продукта ограничивают набор пользовательской документации лишь встроенной системой помощи (англ. *online help*), содержащей справочную информацию о командах или пунктах меню. Работа по обучению новых пользователей и поддержке совершенствующихся пользователей перекладывается на частных издателей, часто оказывающих значительную помощь разработчикам.

Маркетинговая документация. Для многих приложений необходимо располагать рядом рекламные материалы, чтобы заинтересовать людей, обратив их внимание на продукт. Такая форма документации имеет целью: подогреть интерес к продукту у потенциальных пользователей, информировать их о том, что именно делает продукт, чтобы их ожидания совпадали с тем, что они получат, объяснить положение продукта по сравнению с конкурирующими решениями. Одна из хороших маркетинговых практик — предоставление слогана — простой запоминающейся фразы, иллюстрирующей то, что мы хотим донести до пользователя, а также характеризующей ощущение, которое создает продукт. Часто бывает так, что коробка продукта и другие маркетинговые материалы дают более ясную картину о возможностях и способах использования программы, чем все остальное.

Самым главным документом является *Техническое задание* (далее — ТЗ), в котором описываются цели и задачи работы, заказчик и исполнители, технические требования, сроки и этапы, требования секретности, форс-мажорные обстоятельства и правила предъявления результатов. Технические требования, в свою очередь, делятся на функциональные, экономические, требования по надежности, эффективности, защищенности от несанкционированного доступа и др.

Следующим по важности документом является *Программа и методика испытаний* (далее — ПМИ). Структурно ПМИ подобна ТЗ — практически для каждого пункта ТЗ в ПМИ говорится, как этот пункт будет проверяться. Способы проверки могут быть самыми разными — от пропуска специального теста до изучения исходных текстов программы, но они должны быть предусмотрены заранее.

Руководство системного программиста на современном чисто русском языке называется руководством по инсталляции. В нем описывается порядок установки системы на ЭВМ, как проверить корректность поставленной системы, как вносить изменения и т. п. Обычно это простой короткий документ; в противном случае система, наверное, плохая, сделанная непрофессионалами.

Руководство оператора (пользователя) — это основной документ, описывающий, как пользоваться системой. В хорошем руководстве

сначала описывается идея системы, основные функции и как ими пользоваться, а уже потом идет описание всех клавиш и меню.

Руководство программиста. Часто это самый объемный документ, описывающий внутреннюю организацию программы. Обычно этот документ идет в паре с документом «текст программы». Руководство программиста дает заказчику возможность дописать какие-то новые фрагменты программы или переделать старые. В современной литературе этот документ называется *Software Development Kit* (далее — *SDK*). Продукт, снабженный *SDK*, может стоить на порядок дороже, чем такой же продукт без оного, так что можно понять, насколько трудно создать действительно полезный *SDK*. Сейчас не очень принято продавать исходные тексты программ — проблемы с интеллектуальной собственностью, даже при наличии *SDK* трудно «влезть» в чужую программу. Поэтому большое распространение получили *Application Program Interface* (далее — *API*). Программа передается только в виде *DLL* (библиотека двоичных кодов), но известно, как обратиться к каждой функции из других программ, т. е. известно имя точки входа, количество, типы и значения параметров. Наличие множества *API*, конечно, хуже, чем наличие исходных текстов (например, нельзя переделать что-то в середине функции), зато много проще в использовании. С другой стороны, все большую популярность приобретает *Free Software Foundation* (*далее — FSF*). Основателем этого движения был Ричард Столман, который забил тревогу по поводу попыток крупных фирм запатентовать многие основные алгоритмы и программы: «Дойдет до того, что они запатентуют понятия «цикл» и «подпрограмма», что мы будем тогда делать?» *FSF* представляет собой собрание программ в исходных текстах. Любой программист может свободно использовать их в своих целях, но все добавления и улучшения, которые он сделал, тоже следует положить в *FSF*. Таким образом, *FSF* представляет собой одно из самых больших доступных хранилищ программ.

Остальные документы, перечисленные в единой системе программной документации (ЕСПД), носят формальный или необязательный характер.

Тестирование. К сожалению, наличие ошибок считается нормальным явлением, реальную рабочую программу невозможно создать безошибочно с первой попытки. Во время тестирования наиболее важны: умение тестировщика найти в программе слабые места и умение программиста исправить ошибочную часть кода, не затронув при этом рабочие участки. Наиболее важные участки многократно проверяются различными методами. Программа полного тестирования может помочь обнаружить проблемы до того, как они проявят себя в эксплуатируемом программном продукте. Кроме того, соответствующая документация необходима тем, кто проводит тестирование, чтобы было понятно, что нужно проверять. Поэтому, такие правила долж-

ны начинаться с требований по тестированию и документированию. Формулировка может выглядеть следующим образом. Все собственные разработки должны быть протестированы и задокументированы перед сдачей их в промышленную эксплуатацию.

Важно отметить, что эта формулировка устанавливает требование, но не тип тестирования или формат документации. Данное предложение необходимо включить в правила для всего программного обеспечения и в процедуры.

Многие организации, занимающиеся созданием программного обеспечения, до 50 % средств, выделенных на разработку программ, тратят на тестирование, что составляет миллиарды долларов по всему миру в целом. И все же, несмотря на громадные капиталовложения, знаний о сути тестирования явно не хватает и большинство программных продуктов неприемлемо ненадежно даже после «основательного тестирования».

Многими тестирование трактуется как процесс, подтверждающий правильность программы и демонстрирующий, что ошибок в программе нет. Такое определение логически некорректно. Тестирование надо рассматривать, как процесс выполнения программы с намерением найти ошибки.

Психологические эксперименты показывают, что большинство людей, поставив цель (например, показать, что ошибок нет), ориентируется в своей деятельности на достижение этой цели [26]. Тестовик подсознательно не позволит себе действовать против цели, т. е. подготовить тест, который выявил бы одну из оставшихся в программе ошибок. Поскольку мы все признаем, что совершенство в проектировании и кодировании любой программы недостижимо и поэтому каждая программа содержит некоторое количество ошибок, самым плодотворным применением тестирования будет найти некоторые из них. Если мы хотим добиться этого и избежать психологического барьера, мешающего нам действовать против поставленной цели, наша цель должна состоять в том, чтобы найти как можно больше ошибок. Сформулируем основополагающий вывод: если ваша цель — показать отсутствие ошибок, вы их найдете не слишком много; если же ваша цель — показать наличие ошибок, вы найдете значительную их часть.

Надежность невозможно внести в программу в результате тестирования, она определяется правильностью этапов проектирования. Наилучшее решение проблемы надежности — с самого начала не допускать ошибок в программе. Однако вероятность того, что удастся безупречно спроектировать большую программу, бесконечно мала. Роль тестирования состоит как раз в том, чтобы определить местонахождение немногочисленных ошибок, оставшихся в хорошо спроектированной программе. Попытки с помощью тестирования достичь надежности плохо спроектированной программы совершенно бесплодны.

Рассмотрим классификацию различных форм тестирования.

Тестирование (*testing*), как мы уже выяснили, — процесс выполнения программы (или части программы) с намерением (или целью) найти ошибки.

Доказательство (*proof*) — попытка найти ошибки в программе безотносительно к внешней для программы среде. Большинство методов доказательства предполагает формулировку утверждений о поведении программы и затем вывод и доказательство математических теорем о правильности программы. Доказательства могут рассматриваться как форма тестирования, хотя они и не предполагают прямого выполнения программы. Многие исследователи считают доказательство альтернативой тестированию — взгляд во многом ошибочный; более подробно это обсуждается в [27].

Контроль (*verification*) — попытка найти ошибки, выполняя программу в тестовой, или моделируемой, среде.

Испытание (*validation*) — попытка найти ошибки, выполняя программу в заданной реальной среде.

Аттестация (*certification*) — авторитетное подтверждение правильности программы, аналогичное аттестации электротехнического оборудования *Underwriters Laboratories*. При тестировании с целью аттестации выполняется сравнение с некоторым заранее определенным стандартом.

Отладка (*debugging*) не является разновидностью тестирования. Хотя слова «отладка» и «тестирование» часто используются как синонимы, под ними подразумеваются разные виды деятельности. Тестирование — деятельность, направленная на обнаружение ошибок; отладка направлена на установление точной природы известной ошибки, а затем — на исправление этой ошибки. Эти два вида деятельности связаны — результаты тестирования являются исходными данными для отладки.

Тестирование модуля, или *автономное тестирование* (*module testing*, *unit testing*), — контроль отдельного программного модуля, обычно в изолированной среде (т. е. изолированно от всех остальных модулей). Тестирование модуля иногда включает также математическое доказательство.

Тестирование сопряжения (*integration testing*) — контроль сопряжения между частями системы (модулями, компонентами, подсистемами).

Тестирование внешних функций (*external function testing*) — контроль внешнего поведения системы, определенного внешними спецификациями.

Комплексное тестирование (*system testing*) — контроль и (или) испытание системы по отношению к исходным целям. Комплексное

тестирование является процессом контроля, если оно выполняется в моделируемой среде, и процессом испытания, если выполняется в среде реальной, жизненной.

Тестирование приемлемости (*acceptance testing*) — проверка соответствия программы требованиям пользователя.

Тестирование настройки (*installation testing*) — проверка соответствия каждого конкретного варианта установки системы с целью выявить любые ошибки, возникшие в процессе настройки системы.

Тестирование ПО охватывает целый ряд видов деятельности, весьма аналогичной последовательности процессов разработки программного обеспечения. Сюда входят постановка задачи для теста, проектирование, написание тестов, тестирование тестов и, наконец, выполнение тестов и изучение результатов тестирования. Решающую роль играет проектирование теста.

Различают тестирование по типам:

- черный ящик (без просмотра исходного текста);
- белый ящик (с изучением исходного текста);

Сторонники первого подхода проектирует свои тесты, исследуя внешние спецификации или спецификации сопряжения программы или модуля, которые он тестирует. Позиция их такова: «Нас не интересует, как выглядит эта программа и выполнил ли я все команды или все пути. Мы будем удовлетворены, если программа будет вести себя так, как указано в спецификациях». Их идеал — проверить все возможные комбинации и значения на входе.

Приверженцы второго подхода проектируют свои тесты, изучают логику программы. Они начинают с того, что стремятся подготовить достаточное число тестов для того, чтобы каждая команда была выполнена, по крайней мере, один раз. Если они немного более искушены, то проектируют тесты так, чтобы каждая команда условного перехода выполнялась в каждом направлении хотя бы раз. Их идеал — проверить каждый путь, каждую ветвь алгоритма. При этом их совсем (или почти совсем) не интересуют спецификации.

Ни один из этих подходов не является хорошей стратегией. К сожалению, они страдают тем недостатком, что в идеале совершенно неосуществимы из-за огромного числа тестов.

Поскольку исчерпывающее тестирование невозможно, мы должны ограничиться чем-то меньшим.

По объему можно выделить следующие тесты:

- маленький тест, типа печати «*hello, world*», чтобы понять, есть ли вообще о чем говорить;
- получасовое тестирование (американцы говорят «Тест на одну сигарету»), обычно проверяют по одному тесту на каждую функцию программы перед серьезным тестированием;
- модульное тестирование;
- комплексное тестирование.

Каждый тест должен давать максимальную отдачу по сравнению с нашими затратами. Эта отдача измеряется вероятностью того, что тест выявит не обнаруженную прежде ошибку. Затраты измеряются временем и стоимостью подготовки, выполнения и проверки результатов теста. Считая, что затраты ограничены бюджетом и графиком, можно утверждать, что искусство тестирования, по существу, представляет собой искусство отбора тестов с максимальной отдачей. Более того, каждый тест должен быть представителем некоторого класса входных значений, так чтобы его правильное выполнение создавало у нас некоторую убежденность в том, что для определенного класса входных данных программа будет выполняться правильно. Это обычно требует некоторого знания алгоритма и структуры программы, и мы, таким образом, смещаемся к правому концу спектра.

Сопровождение — процесс изменения приложения после поставки с целью исправления ошибок, повышения производительности или для адаптации к изменившимся условиям.

Начинается этап сопровождения, который длится, пока программа живет. В процессе эксплуатации возникают новые требования, всплывают ошибки, объемы обрабатываемых данных растут, соответственно, растут и требования к эффективности. Основными задачами сопровождения являются:

- исправление ошибок;
- регулярное проведение замеров производительности (профилирование);
- улучшение «времяпожирающих» мест в программе (еще говорят «бутылочное горлышко»);
- улучшение документации;
- расширение функциональности;
- принятие решения о прекращении эксплуатации программы или ее реинжиниринге (переводе программы на другую платформу).

Очень трудно сопровождать программу, в процессе разработки которой о сопровождении никто не думал. Такие простые вещи, как разумные комментарии, выбор идентификаторов и ступенчатое расположение строк исходного текста, заметно облегчают сопровождение. Только на этапе разработки можно предусмотреть средства для профилирования, снятия трасс, отладочных печатей и т. п.

Сопровождение — это особое искусство. Старые программы (*legacy* — унаследованные) обычно состоят из сотен или даже тысяч модулей общим объемом в миллионы строк исходного кода. Вряд ли кто-то возьмется все прочитать, понять и запомнить. Обычно сопровождающий программист, исправляя ошибку, лишь примерно знает, где ее искать. Более того, после нахождения неправильного фраг-

мента не всегда ясно, как повлияет исправление на другие участки кода. Часто бывает, что, исправив одну ошибку, получаешь две других.

Реинжиниринг. Особый интерес вызывает вопрос, когда же следует прекращать сопровождение [27]. На этот вопрос отвечает диаграмма оценки качества ПО (рис. 3.22).

Здесь по горизонтали растет важность программы для бизнеса компании, а по вертикали — сложность внесения исправлений. Так вот, если исправлять несложно и важность — небольшая, то надо просто сопровождать, исправляя ошибки (*corrective fixing*), если же программа важна для компании, то можно и улучшать ее (*adaptive fixing*). Если исправлять сложно, а важность — небольшая, то легче программу выкинуть, если же программа важна, содержит определенные знания бизнес-процессов, но сопровождать ее трудно, например, потому что авторы программы покинули компанию или компания вынуждена сменить платформу, то программу следует подвергнуть реинжинирингу.

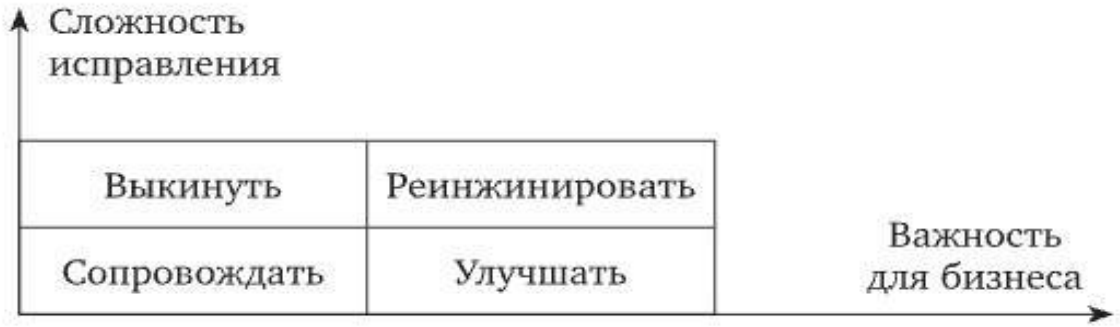


Рис. 3.22. **Диаграмма оценки качества ПО**

Попросту говоря, *реинжиниринг* — это перевод программы со старых языков (Кобол, ПЛ1, Адабас-Натурал и т. п.) на новые, такие как *Java*, *Visual Basic*, *C++*. На самом деле, все гораздо сложнее. Смена операционной системы, используемой базы данных, стиля организации диалога с пользователем — каждая из этих задач весьма нетривиальна. Если же добавить естественное желание реструктурировать программу, выкинуть отмершие части, перейти к объектно-ориентированной организации программы и т. д., да еще вспомнить, что все эти задачи требуют перебора и анализа путей в графе управления программы, число которых растет экспоненциально относительно числа вершин, то задача вообще выглядит нерешаемой.

Относительные затраты ресурсов на создание ПО по основным этапам: спецификацию (10 %), проектирование (10 %), кодирование (10 %), отладку (20 %) и сопровождение (50 %).

Систематический, обоснованный подход к выбору и применению технологий создания программного обеспечения может сократить время и повысить качество разработки программного обеспечения, обеспечить высокую степень его независимости от конкретных разработчиков, а также снизить затраты на разработку и сопровождение программного обеспечения.

Рассмотрим существующие технологии создания ПО.

Восходящее программирование (*программирование «снизу вверх»*) — методика разработки программ, при которой крупные блоки собираются из ранее созданных мелких блоков. Восходящее программирование начинается с разработки ключевых процедур и подпрограмм, которые затем постоянно модифицируются.

Метод расширения ядра — метод восходящего программирования, при котором основное внимание уделяется выявлению множества вспомогательных модулей, а не определению функции всей программы в целом.

Диаграмма функционального моделирования (*Structured analysis and design technique — SADT*) — инструмент разработки функциональных спецификаций в виде диаграмм, фрагментов текста и глоссария, связанных перекрестными ссылками. В состав диаграммы входят:

- блоки, изображающие активность моделируемой системы;
- дуги, связывающие блоки вместе и изображающие взаимодействия и взаимосвязи между ними.

Место соединения дуги с блоком определяет тип интерфейса:

- управляющая информация входит в блок сверху;
- входная информация, подвергающаяся обработке, показывается с левой стороны блока;
- выходная информация показывается с правой стороны;
- механизм, осуществляющий операцию, представляется дугой, входящей в блок снизу.

Императивное программирование — технология, характеризующаяся принципом последовательного изменения состояния вычислителя пошаговым образом. При этом управление изменениями полностью определено и полностью контролируемо.

Компонентное сборочное программирование — объектно-ориентированное сборочное программирование, основанное на распространении классов в бинарном виде и предоставлении доступа к методам класса через строго определенные интерфейсы.

Компонентное сборочное программирование поддерживают технологические подходы *COM*, *CORBA*, *.Net*.

Компьютерный дарвинизм — подход к разработке программных систем, основанный на принципе восходящей разработки при интенсивном тестировании. Подход состоит из трех основных процессов: макетирования, тестирования и отладки.

Логическое программирование — программирование в терминах фактов и правил вывода с использованием языка, основанного на формальных исчислениях.

Модульное программирование — метод разработки программ, предполагающий разбиение программы на независимые модули. Считается, что:

- оптимальный по размерам модуль целиком помещается на экране дисплея;
- разделение большой программы на модули облегчает ее разработку, отладку и сопровождение.

Сборочное программирование — технология, при которой программа собирается посредством повторного использования уже известных фрагментов программ.

Модульное сборочное программирование — разновидность сборочного программирования, основанная на процедурах и функциях методологии структурного императивного программирования.

Нисходящее программирование (*программирование «сверху вниз»*) — методика разработки программ, при которой разработка начинается с определения целей решения проблемы, после чего идет последовательная детализация, заканчивающаяся детальной программой.

Объектно-ориентированное программирование — технология программирования, при которой программа рассматривается как набор дискретных объектов, содержащих, в свою очередь, наборы структур данных и процедур, взаимодействующих с другими объектами.

Объектно-ориентированное сборочное программирование — разновидность сборочного программирования:

- основанная на методологии объектно-ориентированного программирования;
- предполагающая распространение библиотек классов в виде исходного кода или упаковку классов в динамически компонуемую библиотеку.

Синтезирующее программирование — программирование, предполагающее синтез программы по ее спецификации.

Структурное программирование — методология и технология разработки программных комплексов, основанная на принципах:

- программирования «сверху-вниз»;
- модульного программирования.

При этом логика алгоритма и программы должны использовать три основные структуры: последовательное выполнение, ветвление и повторение.

Инструментарий технологии программирования — программные продукты, предназначенные для поддержки технологии программирования. К ним относятся рассмотренные ранее (параграф 3.4) *CASE*-технологии.

3.8. Большие данные

Стремительное развитие инструментальных средств создало условия для накопления и хранения больших объемов данных, однако технологии их использования отстают от потребностей пользователей.

Большие данные (*Big Data*) в информационных технологиях означают серию подходов, инструментов и методов обработки структурированных и неструктурированных данных огромных объемов и значительного многообразия для получения воспринимаемых пользователем результатов [40–42].

Большие данные принято описывать в терминах четырех *V*: *volume* (объем), *velocity* (скорость), *variety* (разнообразие) и *verasity* (достоверность). Обычно эти характеристики перечисляют в произвольном порядке и считают, что все они одинаково важные. Иногда к этим характеристикам добавляют *value* (ценность), *variability* (переменчивость) и др. Наиболее важным является то, что определяющей характеристикой для больших данных является не только их физический объем, но и другие категории, существенные для представления о сложности задачи обработки и анализа данных.

1. Объем. Данные, генерируемые и обрабатываемые с помощью больших данных, постоянно растут все возрастающими темпами. Объем растет в геометрической прогрессии благодаря тому факту, что предприятия постоянно собирают данные для создания более качественных и масштабных бизнес–решений. Этот постоянно растущий объем данных требует широко масштабируемых и надежных хранилищ. Основными источниками, способствующими такому огромному росту объема, являются разного рода сенсоры, входящие в состав *IoT*, которые могут генерировать огромные потоки данных: видео–контент, социальные сети, транзакции в точках продаж, онлайн-банкинг.

2. Скорость. С резким увеличением объема данных скорость, с которой они генерируются, также возросла. Термин «скорость» относится не только к скорости генерации данных, но также и к скорости их обработки и анализа. В эпоху больших данных огромное количество данных генерируется с высокой скоростью, и иногда эти данные поступают так быстро, что их становится даже трудно собрать, помимо того, что еще необходимо их проанализировать.

3. Разнообразие. Разнообразие относится к форматам и типам данных, поддерживаемых большими данными. Данные могут поступать в структурированном, полуструктурированном и неструктурированном формате. Структурированные данные относятся к данным, обрабатываемым традиционными системами управления базами данных, где данные организованы в виде таблиц, таких как данные о сотрудниках, данные банковских клиентов и т. д. Полуструктурированные данные представляют собой комбинацию структурированных и неструктурированных данных, таких как *XML*. Данные *XML* являются полуструктурированными, поскольку они не соответствуют формальной модели данных (таблице), связанной с традиционной базой данных. Чаще всего они содержат теги для организации поля внутри данных. Неструктурированные данные относятся к данным

без определенной структуры, таким как сообщения электронной почты, фотографии, веб–страницы и т. п.

4. Достоверность. Данные могут поступать из самых разных источников, они могут быть неполными, зашумленными, преднамеренно или непреднамеренно искаженными, или просто недостоверными.

Источники больших данных. Большое разнообразие данных, поступающих из разных источников, является одним из ключевых свойств феномена больших данных. Поэтому полезно понять, как генерируются данные в различных средах и сценариях, прежде чем рассматривать, что следует делать с этими данными и как разработать наилучшую возможную архитектуру для достижения этой цели.

Можно выделить следующие основные предметные домены в которых появляется проблема обработки больших массивов неоднородных данных: научные исследования, *Internet*, *IoT*, *IIoT*.

1. Научные исследования. Научные данные являются традиционным источником больших данных. В качестве основных источников можно назвать биомедицинские системы, физические и астрофизические данные, данные об окружающей среде.

2. Интернет. Интернет, несомненно, является крупнейшей базой данных, когда-либо созданной людьми. Хотя с помощью этой среды доступен достаточно большой объем хорошо описанных, очищенных и структурированных наборов данных, однако большинство информационных ресурсов носят неоднозначный, неструктурированный, неполный или даже ошибочный характер и могут рассматриваться как неструктурированные данные.

3. Интернет вещей. *IoT* является одной из самых быстрорастущих классов *IT*-систем. С ростом числа электронных схем, встроенных в физические элементы, и увеличением их вычислительной мощности и возможностей, естественным направлением развития является их подключение и обеспечение крупномасштабной совместимости и обмена данными. Большой объем данных, поступающих от систем, построенных по принципу *IoT*, связан с двумя моментами. С одной стороны, число датчиков может быть очень велико, а с другой многие «вещи», такие как камеры видеонаблюдения, дроны, генерируют потоки данных, которые часто требуется обрабатывать в реальном времени.

4. Промышленный интернет вещей (*IIoT*). На современных производствах машины в основном соединены, создавая совместный организм, что увеличивает общую сложность системы и требует расширенных возможностей автоматизированного планирования, мониторинга, диагностики и восстановления. Такие среды относятся к категории *CPS*. Сквозная инженерная интеграция представляет собой путь создания ценности, связанный с продуктом, начиная

от представления потребителю физических производственных ресурсов, совокупности ресурсов, продуктов, деталей и операций.

По мере того как на производстве будет устанавливаться все больше и больше датчиков, это число будет только расти.

Прочие источники. К прочим типам ИС, которые можно рассматривать как источники больших данных можно отнести банковские системы, телекоммуникационные системы, системы административного назначения и многие другие.

На практике данных действительно становится все больше, но проблема вызвана не столько обрушившимися в невероятном количестве данными, сколько неспособностью старыми методами справиться с новыми объемами. Способность порождать данные оказалась сильнее, чем способность их перерабатывать. Причина возникновения такого дисбаланса связана с недостаточным вниманием к отслеживанию связей цепочки «данные — информация — знание». В современной теории информации недостаточное внимание уделено раскрытию понятий «данные» и «информация», отдается предпочтение технологии работы с данными. Данные должны обрабатываться для получения информации, необходимой пользователю для превращения в знание.

Принято делить подходы к *Big Data* на три группы: быстрые данные (*Fast Data*), их объем измеряется терабайтами; большая аналитика (*Big Analytics*) — петабайтные данные и глубокое проникновение (*Deep Insight*) — экзабайты, зеттабайты. Группы различаются между собой не только оперируемыми объемами данных, но и качеством решения по их обработке.

Обработка для *Fast Data* не предполагает получения новых знаний, ее результаты соотносятся с априорными знаниями и позволяют судить о том, как протекают те или иные процессы, она позволяет лучше и детальнее увидеть происходящее, подтвердить или отвергнуть какие-то гипотезы. Небольшая часть из существующих сейчас технологий подходит для решения задач *Fast Data*, среди них некоторые технологии работы с хранилищами (продукты *Teradata*, *Netezza*, *Greenplum*, СУБД типа *Vertica* и *Kdb*). Скорость работы этих технологий должна возрастать синхронно с ростом объемов данных.

Задачи, решаемые средствами *Big Analytics*, заметно отличаются, причем не только количественно, но и качественно, а соответствующие технологии должны помогать в получении новых знаний — они служат для преобразования зафиксированной в данных информации в новое знание. Однако на этом среднем уровне не предполагается наличие искусственного интеллекта при выборе решений или каких-либо автономных действий аналитической системы — она строится по принципу «обучения с учителем». Иначе говоря, весь ее аналитический потенциал закладывается в нее

в процессе обучения. Классическими представителями такой аналитики являются продукты *MATLAB*, *SAS*, *Revolution R*, *Apache Hive*, *SciPy Apache* и *Mahout*.

Высший уровень *Deep Insight* предполагает обучение без учителя (*unsupervised learning*) и использование современных методов аналитики, а также различные способы визуализации. На этом уровне возможно обнаружение знаний и закономерностей, априорно неизвестных.

С течением времени компьютерные приложения становятся все ближе к реальному миру во всем его многообразии, отсюда рост объемов входных данных и отсюда же потребность в их аналитике, причем в режиме, максимально приближенном к реальному времени. Конвергенция этих двух тенденций привела к возникновению направления аналитики больших данных (*Big Data Analytics*).

Потенциальные практические сферы применения «больших данных» отличаются широким разнообразием: здравоохранение и медицина, государственные услуги, розничная торговля, различные производственные отрасли, телекоммуникационная отрасль, информационно–технологическая отрасль, финансы и страхование, энергетика, маркетинг и риск–менеджмент, издательское дело, медиа и развлечения, биотехнологии, транспорт и логистика, коммунальные услуги. Со временем это разнообразие будет только нарастать. Применительно к коммерческим структурам «большие данные» призваны стать дополнительным источником конкурентных преимуществ.

Методы анализа больших данных. Рост объемов входных данных и потребность в их аналитике, причем в режиме, максимально приближенном к реальному времени, привели к возникновению направления аналитики больших данных (*Big Data Analytics*). Обработка больших данных решает следующие типовые задачи.

1. Поиск. В условиях экспоненциального роста объемов хранилищ данных всех видов цифрового контента объем поиска может варьироваться в зависимости от приложения, например, веб-поиск, корпоративный поиск и т. д. Чтобы облегчить эффективный поиск информации, поисковой системе, возможно, придется выполнять другие задачи, кроме непосредственного поиска, например кластеризацию документов, фильтрацию спама и т. д.

2. Системы прогнозирования. Прогнозирование — это классическая задача науки о данных, которая сводится к прогнозированию будущего на основе исторических данных и любой другой доступной информации. Большинство методов прогнозирования сосредоточены на данных временных рядов, таких как цены на акции, температура, загрузка системы и т. д.

Чаще всего для решения задач предсказания используются временные ряды. Типичные данные временных рядов реальной жизни являются результатом различных факторов. На тренд более высоко-

го уровня обычно влияют множественные циклические и разовые события.

3. Большие данные в социальных сетях. Механизмы работы с большими данными могут быть использованы для понимания человеческого поведения путем измерения социальных явлений. Приложения варьируются от анализа настроения, до анализа отношения потенциальных покупателей к таким типовым задачам, как анализ сетевых взаимодействий, обнаружение сообществ, анализ текста, анализ распространения информации, слияние информации.

4. Обнаружение аномалий и мошенничества. В последние годы обнаружению аномалий уделяется значительное внимание как со стороны научных исследований, так и со стороны промышленности. Это связано с несколькими областями, в которых генерируются очень большие наборы данных, а также с растущей сложностью систем, что исключает возможность предварительного перечисления всех ненормальных сценариев.

Примеры приложений включают мониторинг сетевого трафика (обнаружение вторжений, обнаружение ботнетов), обнаружение мошенничества, управление сенсорной сетью, медицинскую диагностику и многие другие.

Мониторинг и управление крупномасштабными распределенными системами. К данному классу может быть отнесено достаточно много разных систем, в частности, энергетические системы, телекоммуникационные системы, системы кабельного телевидения и многие другие.

Обработка видеопотоков. Системы видеонаблюдения и машинного зрения используются в таких CPS, в частности *IoT*, *IIoT*-системах. Видеонаблюдение является одной из наиболее трудоемких вычислительных задач из вышеперечисленных. Оно не только предполагает на интенсивный ввод мультимедийных данных, но и требует извлечения семантики и шаблонов из потока видео–данных.

Умные здания. Сценарий для систем больших данных Интернета вещей. Современные коммерческие здания имеют ряд установок, начиная от электричества, водоснабжения, кондиционирования воздуха, лифтов и заканчивая сетью Ethernet, все взаимосвязано и взаимозависимо. Можно определить ряд основных задач для интеллектуальных зданий, которые требуют объединения данных из нескольких подсистем, таких как энергоэффективность, безопасность, управление в чрезвычайных ситуациях и т. д.

Способы хранения больших данных. Модель хранения данных является ключевым элементом любой системы, ориентированной на работу с большими данными. Для решения задач, связанных с обработкой данных, за последние несколько десятилетий были разработаны три основные модели хранения данных: хранилища на основе блоков, хранилища на основе файлов и хранилища на основе объектов.

Хранилище на уровне блоков — это одна из самых классических моделей хранения данных. Традиционная блочная система хранения данных представлена сервером, использующим стандартные отраслевые механизмы подключения. В принципе, хранилище на уровне блоков можно рассматривать как жесткий диск на сервере, за исключением того, что жесткий диск может быть установлен в удаленном корпусе и доступен с помощью *Fibre Channel* или *iSCSI*.

Упрощенная модель блочного хранилища показана на рис. 3.23. В нем основные данные хранятся в виде блоков, которые обычно имеют фиксированный размер, при этом дополнительная информация (метаданные) отсутствует. Для доступа к каждому блоку используется уникальный идентификатор. Идентификатор сопоставлен с точным местоположением фактических блоков. Традиционно блочное хранилище использует такие низкоуровневые протоколы как *SCSI*, *iSCSI*, *ATA* и *SATA* [80].

С развитием распределенных вычислений и увеличением объема хранимых данных стали использовать блочные хранилища, состоящие из нескольких узлов (рис. 3.24). В этом случае блочное хранилище включает в себя блочный сервер и группы узлов.

Рис. 3.23. **Блочное хранилище**

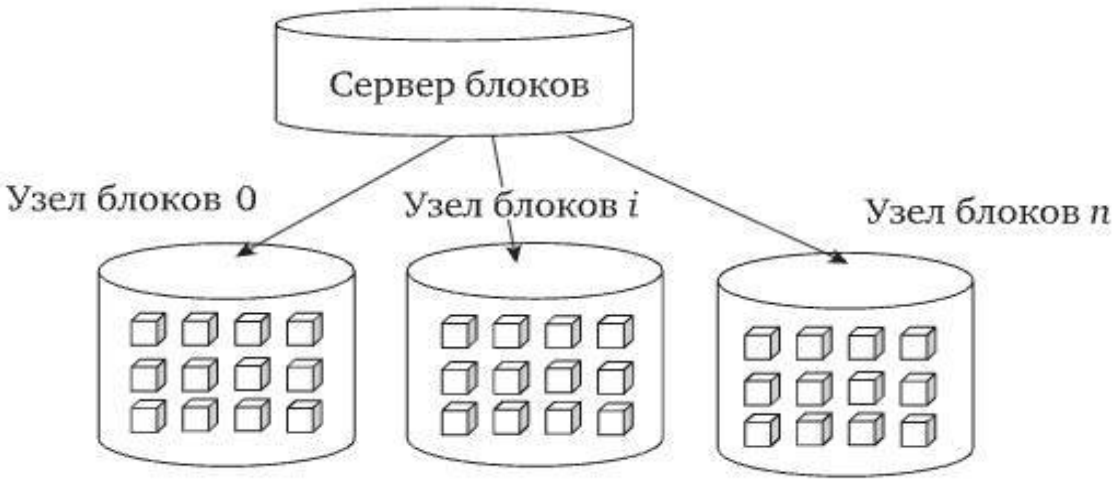


Рис. 3.24. **Распределенное блочное хранилище**

Сервер блоков в этом случае отвечает за реализацию механизмов сопоставления или индексации идентификаторов блоков с фактическими блоками данных в узлах блоков. Узлы блоков, в свою очередь,

отвечают за хранение данных в разделах фиксированного размера, каждый из которых рассматривается как блок.

Хранилище на основе файлов наследует традиционную архитектуру файловой системы и рассматривает данные как файлы, которые организованы в виде иерархической структуры. Это наиболее распространенная модель хранения данных, которая относительно проста в реализации и использовании. В сценариях с большими данными файловая система хранения может быть построена на какой–либо другой низкоуровневой абстракции, такой как блочная и объектно-ориентированная модель с целью повышения ее производительности и масштабируемости.

Типовая структура файлового хранилища показана на рис. 3.25. Файлы образуют древовидную структуру. При работе с большими данными используются распределенные файловые системы.

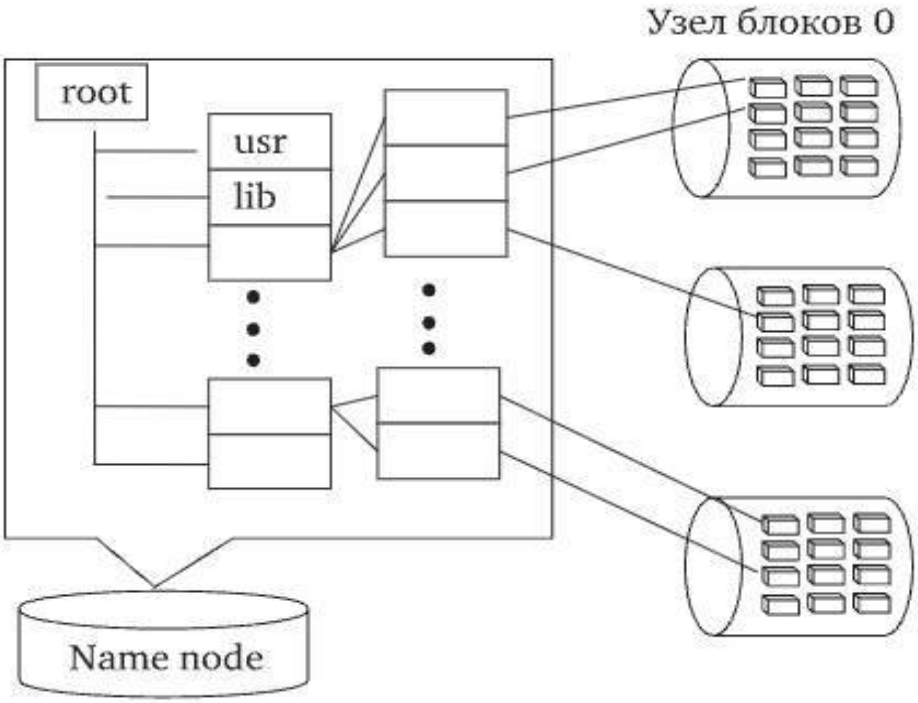


Рис. 3.25. **Типовая структура файлового хранилища**

На рис. 3.26 показана типовая структура распределенной файловой системы, которая обычно содержит один или несколько узлов имен и набор узлов данных. Узел имени отвечает за поддержание иерархии записей файлов для всей системы, в то время как узлы данных отвечают за сохранение данных файла.

В файловой системе пользователю необходимо знать о пространствах имен и пути для доступа к сохраненным файлам.

Для обмена файлами между системами путь или пространство имен файла будет включать три основные части: протокол, имя домена и путь к файлу. Например, при работе с файловой системой *Hadoop* (*Hadoop File System*, *HDFS*), которая будет рассмотрена ниже, путь к файлу может быть указан как:

```
[hdfs://][Адрес сервера:Порт сервера]/[Путь к файлу]
```

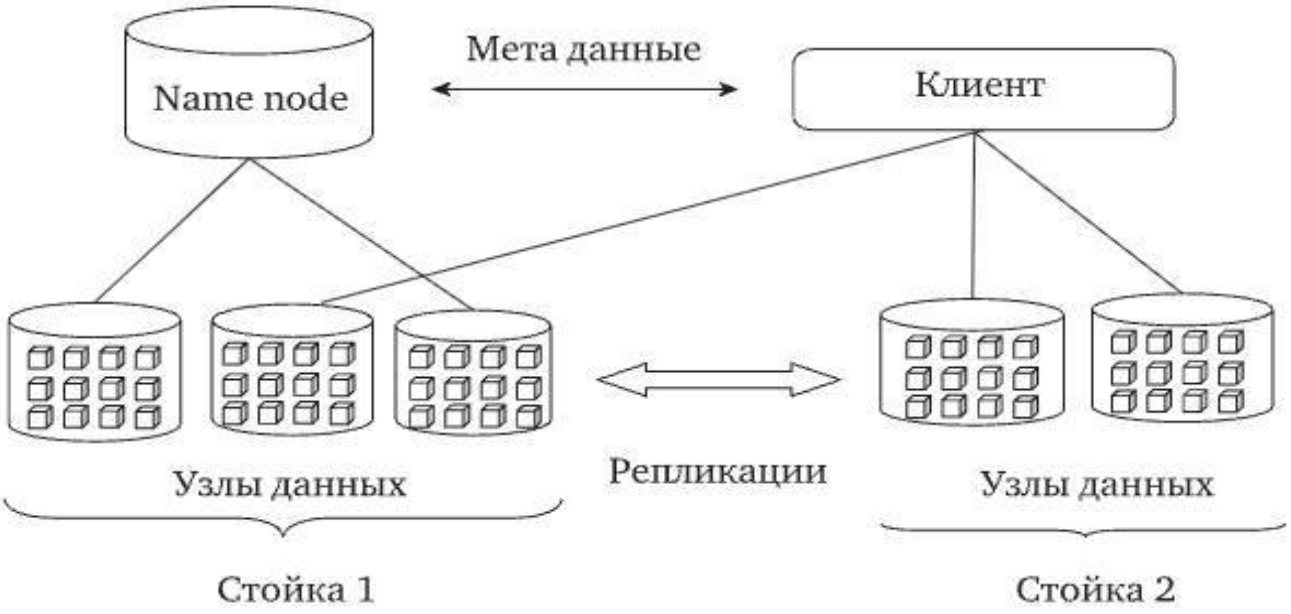


Рис. 3.26. **Распределенная файловая система Hadoop**

Объектно-ориентированная модель хранения данных. В объектно-ориентированной модели хранения данные управляются как объекты (рис. 3.27). Каждый объект включает в себя сами данные, некоторые метаданные, атрибуты и глобально уникальный идентификатор объекта (*OID*). Объектно-ориентированная модель хранения абстрагирует нижние уровни хранилища от администраторов и приложений. Системы хранения объектов могут быть реализованы на разных уровнях, в том числе на уровне устройства, системном уровне и уровне интерфейса.

Данные отображаются и управляются как объекты, которые включают дополнительные описательные метаданные, которые можно использовать для лучшей индексации или управления. Метаданные могут быть любыми, начиная от свойств безопасности, конфиденциальности и аутентификации и заканчивая информацией, связанной с любыми приложениями.

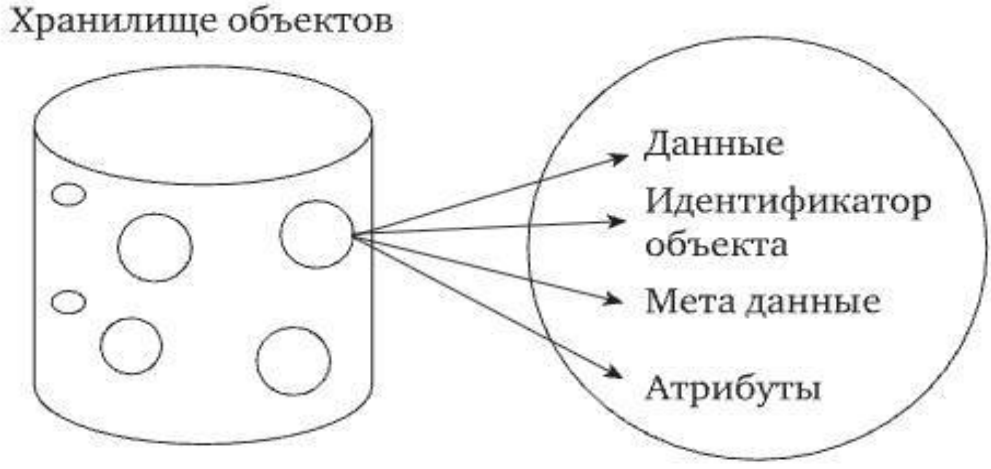


Рис. 3.27. **Обобщенная структура объектно-ориентированного хранилища данных**

Типовая структура объектно-ориентированного хранилища данных показана на рис. 3.28. Как видно из рисунка, система хранения

на основе объектов обычно использует плоское пространство имен, в котором идентификатор данных и их местоположение обычно сохраняются в виде пар ключ–значение на сервере объектов. В принципе, сервер объектов обеспечивает адресацию, не зависящую от местоположения, и постоянную задержку поиска для чтения каждого объекта. Кроме того, метаданные данных отделены от данных и также поддерживаются в виде объектов на сервере метаданных. В результате, обеспечивается стандартный и более простой способ обработки, анализа и манипулирования метаданными, не затрагивая сами данные.

Благодаря плоской архитектуре объектно-ориентированные хранилища достаточно просто масштабировать, добавляя в систему дополнительные узлы хранения. Кроме того, добавленное хранилище может быть автоматически расширено по мере увеличения емкости, доступной для всех пользователей. Опираясь на контейнер объектов и поддерживаемые метаданные, он также способен предоставлять гораздо более гибкие и детализированные политики в отношении данных на разных уровнях.

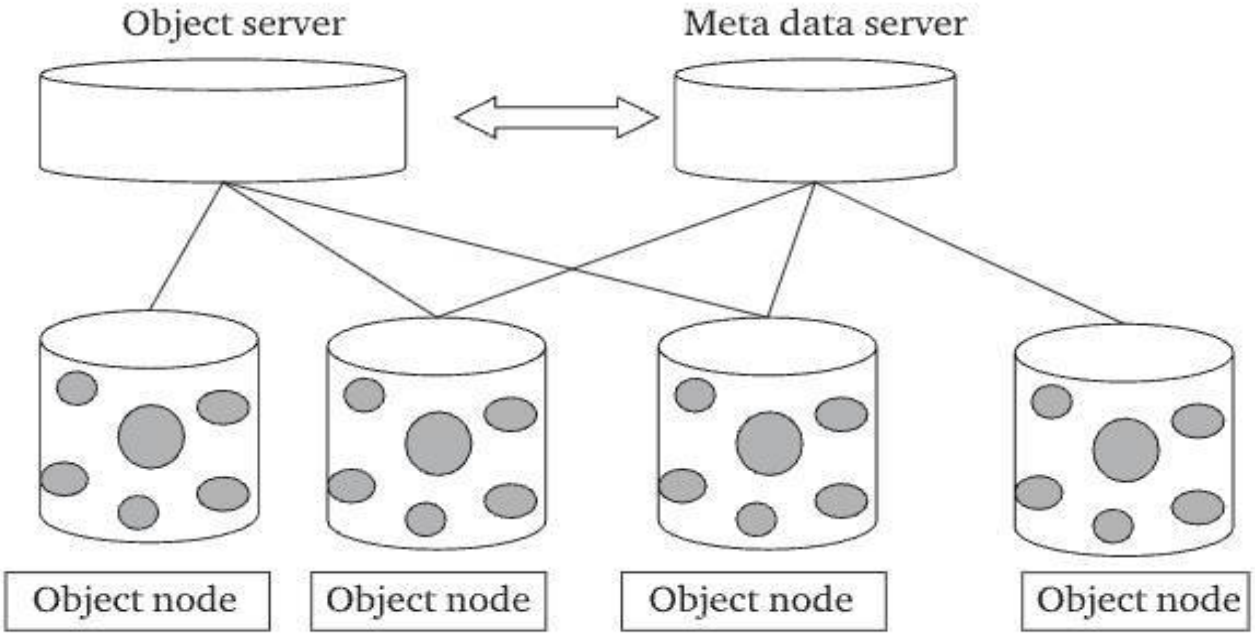


Рис. 3.28. **Детализированная структура объектно-ориентированного хранилища данных**

Известно достаточно много реализаций объектно–ориентированных хранилищ. Наиболее известной является система *Amazon S3* (*Simple Storage Service*) [18], которая представляет собой облачную систему хранения объектов, которая широко используется для онлайн–резервного копирования и архивирования данных и прикладных программ.

Сравнение различных типов хранилищ. Блочные хранилища отличаются гибкостью, универсальностью и простотой. В системе хранения на уровне блоков создаются необработанные тома хранилища (состоящие из набора блоков), а затем серверная система подключается к этим томам и использует их в качестве отдельных накопителей. Это делает хранилище на основе блоков пригодным практиче-

ски для любых приложений, включая хранилище файлов, хранилище баз данных, тома файловой системы виртуальных машин и многое другое. Однако, блочное хранилище сложно в управлении и непросто в использовании из-за недостатка информации такой как метаданные, логическая семантика и взаимосвязь между блоками данных по сравнению с другими моделями хранения, такими как хранилище на основе файлов и хранилище на основе объектов.

Хранилище файлов легко в управлении и внедрении. Это также дешевле в использовании, чем блочное хранилище. Однако из-за иерархической структуры файловое хранилище менее масштабируемо, а когда количество файлов становится очень большим, то задержки в доступе к данным существенно возрастают.

Объектно-ориентированные хранилища легко масштабировать и ими просто управлять, добавляя дополнительные узлы. Плоская организация пространства имен данных в сочетании с расширяемыми функциями метаданных способствует такой простоте использования. Объектные хранилища обычно используются для хранения крупномасштабных неструктурированных данных, таких как фотографии, музыкальные файлы, видео и т. п.

Хранилище объектов обычно не обеспечивает возможность постепенного редактирования одной части файла (как это делают блочное хранилище и хранилище файлов). Объектами необходимо управлять как единым целым, требуя доступа ко всему объекту, обновления, а затем перезаписи в физическое хранилище.

Методы анализа больших данных. Рост объемов входных данных и потребность в их аналитике, причем в режиме, максимально приближенном к реальному времени привели к возникновению направления аналитика больших данных (*Big Data Analytics*).

На текущий момент большие данные опираются на многочисленные методы анализа (табл. 3.8), создававшиеся на протяжении десятилетий во многих областях информационных технологий (искусственный интеллект, анализ данных, распознавание и др.). В процессе развития технологии больших данных будут созданы новые методы анализа или же существующие методы будут видоизменяться.

Аппаратное и программное обеспечение больших данных. Выше отмечалось, что появление больших данных обусловлено бурным развитием информационно–телекоммуникационных технологий, что сопровождалось в том числе количественным наращиванием объемов данных, их качественным усложнением, стремительным нарастанием разнообразия. В какой-то момент стало очевидным, что устоявшиеся технологические решения не в состоянии справиться с новым вызовом вовлечения накапливаемых массивов данных в экономическую деятельность. Пренебрежительное отношение к таким данным ведет к потерям, которых хотелось бы избежать. Для решения этой задачи нужны новые технологические подходы.

Таблица 3.8

Методы анализа больших данных

Метод	Краткая характеристика
А/Б тестирование (*A/B Testing*)	Метод, предусматривающий сравнение реакций на контрольный (А) и тестируемый (Б) варианты некоторых стимулов. Тестируемых вариантов может быть несколько. На основании сравнения реакций выбирается наилучшее для данной цели решение
Анализ с использованием ассоциативных правил (*Association Rule Learning*)	Совокупность методов для выявления характерных взаимосвязей (ассоциативных правил) между переменными в больших базах данных. Такие методы представляют собой набор алгоритмов для генерации и тестирования возможных правил. Одно из применений метода ассоциативных правил — анализ рыночной корзины (*Market Basket Analysis*), применение которого позволяет, например, установить, что покупатель товара А с высокой вероятностью купит и товар Б. Это дает возможность продавцам принимать соответствующие маркетинговые решения
Классификация (*Classification*)	Совокупность методов рубрикации новых данных на основе рубрикации тренировочных данных (*Training Data Set*). Эти методы относят к группе методов управляемого (направляемого) обучения (*Supervised Learning*), так как изначально наличествующие тренировочные данные направляют процесс обучения (см. далее)
Кластерный анализ (*Cluster Analysis*)	Статистический метод классификации объектов, предполагающий разбиение всей совокупности объектов на более мелкие группы (кластеры), объединяющие похожие между собой объекты и в то же время сильно отличающиеся от объектов других групп. Характеристики, на основе которых данный объект относится к тому или иному кластеру, заблаговременно неизвестны. Этот метод относят к числу методов ненаправляемого обучения (*Unsupervised Learning*)
Краудсорсинг (*Crowdsourcing*)	Метод сбора данных, предоставляемых большими группами людей («толпой») в ответ на открытый запрос на такие данные. Как правило, осуществляется с использованием сетевых методов социального взаимодействия
Интеграция данных (*Data Fusion and Data Integration*)	Совокупность методов интеграции и анализа данных, полученных из множественных источников, что позволяет делать более точные выводы по сравнению с ситуацией, когда такие данные используется раздельно
Глубинный анализ данных (*Data Mining*)	К числу методов, применяемых для глубинного анализа данных можно отнести, например, анализ с использованием ассоциативных правил, кластерный анализ, классификацию, регрессионный анализ и др.

Продолжение табл. 3.8

Метод	Краткая характеристика
Ансамблевое обучение (*Ensemble Learning*)	Подразумевает использование множественных прогностических моделей (*Predictive Models*), каждая из которых разработана с использованием статистических методов или машинного обучения для получения более адекватных прогнозов
Генетические (эволюционные) алгоритмы (*Genetic Algorithms*)	Метод оптимизации, вдохновленный изучением процессов естественной эволюции, которая, как известно, предполагает выживание наиболее приспособленных особей. В этом методе потенциальные решения интерпретируются как «хромосомы», которые могут объединяться и мутировать. «Хромосомы» приводятся в соприкосновении с условиями «окружающей среды». «Выжившие» «хромосомы» рассматриваются в качестве наилучшего решения. Такого рода методы наиболее ценны при решении нелинейных задач
Машинное обучение (*Machine Learning*)	Подраздел компьютерной науки (относимый к области искусственного интеллекта), занимающийся разработкой алгоритмов, позволяющих компьютерам изменять свое поведение на основе поступающих эмпирических данных. Важнейшее направление исследований — автоматическое распознавание сложных устойчивых моделей и принятие адекватных решений на основе данных
Обработка естественных языков (*Natural Language Processing*)	Составная часть машинного обучения, подразумевающая совместное использование компьютерных технологий и лингвистики для создания алгоритмов, позволяющих анализировать естественные (человеческие) языки
Искусственные нейросети (*Neural Networks*)	Модели вычислений, предназначенные для распознавания устойчивых моделей в массивах данных. Их создание вдохновлено структурой и работой биологических нейросетей
Сетевой анализ (*Network Analysis*)	Совокупность методов, применяемых для описания взаимоотношений между отдельными узлами, объединенных в граф или сеть
Оптимизация (*Optimization*)	Совокупность численных методов, применяемых для изменения (*redesign*) сложных систем и процессов с целью улучшения их результатов работы, определяемых на основе одного или нескольких объективных численных показателей (например, затраты, скорость, надежность и др.)
Распознавание устойчивых моделей (*Pattern Recognition*)	Совокупность методов машинного обучения, которые в соответствии с конкретным алгоритмом присваивают некоторой входной величине некоторую выходную величину — метку (*label*). Классификация относится к числу таких методов

Окончание табл. 3.8

Метод	Краткая характеристика
Прогностическое моделирование (*Predictive Modeling*)	Совокупность методов, подразумевающих создание или выбор математических моделей для предсказания вероятности некоторого результата (параметра выхода)
Регрессионный анализ (*Regression*)	Совокупность статистических методов для определения изменений зависимой величины вследствие изменений одной или нескольких независимых величин
Анализ настроений (мнений) (*Sentiment Analysis*)	Применение методов обработки естественных языков и других аналитических методов для выявления и извлечения из анализируемого текста субъективной информации, характеризующей настроения, мнения, отношение людей к проблеме
Статистическая обработка сигналов (*Signal Processing*)	Эти методы заимствованы из электротехники и прикладной математики. Изначально они применялись для анализа дискретных и непрерывных сигналов. В современном анализе данных применяются для статистического вычленения из данных полезного сигнала и шума
Пространственный анализ (*Spatial Analysis*)	Совокупность методов анализа топологических, геометрических или географических свойств объектов, содержащихся в данных
Статистика (*Statistics*)	Наука о сборе, структурировании и интерпретации данных, включая планирование опросов (*design of surveys*) и планирование экспериментов (*design of experiments*)
Направляемое обучение (*Supervised Learning*)	Совокупность методов машинного обучения, позволяющих делать выводы о наличии некоторых функций или отношений на основе тренировочных данных. Пример — метод классификации
Имитационное моделирование (*Simulation*)	Создание моделей поведения сложных систем, применяемых для прогнозирования возможных будущих состояний и (или) сценарного планирования
Анализ временных рядов (*Time Series Analysis*)	Совокупность методов для анализа последовательностей данных, характеризующих изменение некоторых параметров объекта во времени, с целью содержательной характеризации такого объекта
Ненаправляемое обучение (*Unsupervised Learning*)	Совокупность методов машинного обучения, нацеленных на выявление скрытых взаимосвязей между объектами за счет анализа охарактеризованных (*unlabeled*) данных
Визуализация (*Visualization*)	Методы создания изображений, диаграмм, анимации, используемых для коммуникаций, улучшения понимания самих данных и результатов их анализа

Ввиду разнообразия рассмотрим одну из технологий — *Hadoop* — стоит затронуть хотя бы на самом общем уровне, так как на текущий момент она находит наибольшее распространение и наиболее обсуждаема в сообществе специалистов.

Hadoop представляет собой свободно распространяемый набор утилит, библиотек и программный каркас для разработки и выполнения распределенных программ, работающих на кластерах из сотен и тысяч узлов. Отличительные особенности *Hadoop* следующие:

- доступность: *Hadoop* работает на больших кластерах общераспространенных (*commodity*) компьютеров или в облачных сервисах, таких как *Elastic Compute Cloud* (*EC2*) компании *Amazon*;
- надежность: так как пакет *Hadoop* ориентирован на работу на обычных, общедоступных компьютерах, его архитектура предусматривает возможность частых сбоев и позволяет успешно справляться с большинством из них;
- масштабируемость: *Hadoop* предусматривает возможность очень простого расширения вычислительных мощностей посредством добавления в кластер дополнительных узлов — общедоступных компьютеров, которых в одном кластере могут быть сотни;
- простота: *Hadoop* предоставляет пользователям возможность быстрого написания эффективного программного кода.

Совокупность этих свойств делает *Hadoop* востребованной технологией в самых различных секторах и применениях. Доступность и простота позволяют быстро и дешево создавать кластеры *Hadoop* даже учащимся колледжей. Надежность и масштабируемость делают *Hadoop* привлекательным даже в самых крупных и технологически требовательных проектах и компаниях.

Ключевые элементы *Hadoop* — распределенная файловая система *Hadoop Distributed File System* (далее — *HDFS*) и *MapReduce* — технология распределенных параллельных вычислений над большими наборами данных в компьютерных кластерах. Отметим лишь наиболее принципиальные особенности этих технологических элементов [43].

Распределенная файловая система *HDFS* функционирует таким образом, что если требуется обеспечить работу вычислительной системы с очень большим блоком данных, например, несколько терабайт, то этот блок разбивается на более мелкие блоки (обычно 64 Мегабайт), которые распределяются по многочисленным компьютерам, составляющим кластер *Hadoop*. Затем *HDFS* обеспечивает параллельную работу вычислительной системы с каждым малым блоком. В результате кластер простых общедоступных компьютеров справляется с большой вычислительной задачей намного быстрее, чем одиночный сервер с самыми передовыми техническими характеристиками. К тому же затраты на создание кластера будут меньшими по сравнению с затратами на высокопроизводительные серверы.

Другая техническая особенность *Hadoop* заключена в самой философии работы с данными. Традиционно в работе с данными используется подход, предусматривающий неоднократную передачу данных между клиентами и сервером. Этот подход вполне оправдан, когда объемы данных сравнительно невелики. Однако в больших данных мы имеем дело с большими объемами данных по определению. Перемещение таких данных становится очень затратной затеей. Поэтому философия взаимодействия данных и вычислительных программ меняется на прямо противоположную: данные остаются там, где они изначально сохранены в рамках кластера *Hadoop*, а пересылаются от клиента к серверу вычислительные программы *MapReduce*, размеры которых обычно очень малы (килобайты).

Еще одна важная особенность *Hadoop* касается принципов построения баз данных. В большинстве сегодняшних приложений используются реляционные базы данных, в качестве ключевого элемента которых выступает технология *SQL* (*Structured Query Language*). Ключевое слово в этом наименовании — *Structured* (структурированный). Оно «выдает» тот факт, что реляционные базы данных и *SQL* ориентированы на работу с хорошо структурированными данными, подразумевающую тщательный заблаговременный выбор полей и таблиц, в которых данные будут храниться, установление четких взаимосвязей между ними и т. п.

Но задачи больших данных подразумевают работу не только с хорошо структурированными данными, но и с полу- и неструктурированными данными. Применение технологии реляционных баз данных и *SQL* в таких случаях было бы либо сопряжено со значительными затратами, либо вообще невозможно. Исходные данные могут поступать в *Hadoop* в любой форме, однако в какой-то момент они преобразуются в пары «ключ — значение». В качестве технологии формирования и обработки запросов к данным, структурированными по принципу «ключ — значение», вместо *SQL* используются программы *MapReduce*, которые предъявляют принципиально иные (существенно менее строгие) требования к структурированности данных и обеспечивают принципиально иной уровень работы с ними.

Разумеется, здесь приведены лишь наиболее примечательные особенности применяемого в задачах больших данных аппаратного и программного обеспечения. Более детальное изложение данного вопроса выходит за рамки данного курса. Всем, кто заинтересован более глубоким изучением технологии *Hadoop*, можно лишь порекомендовать обратиться к специальным источникам.

Гигантские объемы в сочетании с высокой скоростью, отличающие *Big Data Analytics* от других приложений, требуют соответствующих компьютеров, и сегодня практически все основные производители предлагают специализированные программно–аппаратные

системы: *SAP HANA*, *Oracle Big Data Appliance* и *Oracle Exalytics Business Intelligence Machine*, *Teradata Extreme Performance Appliance*, *NetApp E-Series Storage Technology*, *IBM Netezza Data Appliance*, *EMC Greenplum*, *Vertica Analytics Platform* на базе *HP Converged Infrastructure*. Помимо этого, в игру вступило множество небольших и начинающих компаний: *Cloudera*, *DataStax*, *Northscale*, *Splunk*, *Palantir*, *Factual*, *Kognitio*, *Datameer*, *TellApart*, *Paraccel*, *Hortonworks*.

3.9. Технологии распределенных вычислений

3.9.1. Грид

Классическим определением термина «грид» является следующее: «Грид — согласованная, открытая и стандартизованная среда, которая обеспечивает гибкое, безопасное, скоординированное разделение (общий доступ) ресурсов в рамках виртуальной организации». При этом под виртуальной организацией понимается сообщество пользователей, объединенных едиными научными, инженерными задачами, подчиняющихся некоторым заданным правилам. Классическое определение было дано столь широко, что его трактуют весьма свободно.

Более точным является определение: Грид — как географически распределенная инфраструктура, объединяющая множество разнотипных ресурсов, таких как компьютеры, хранилища и базы данных, сети, доступ к которым пользователь может получить из любой точки, независимо от места их расположения. Термин «грид» в смысле одной из технологий распределенных вычислений появился в середине 1990-х гг. и происходит от английского «*grid*». Слово переводится на русский язык как «решетка» или «электрическая сеть».

Концепция грид предполагает создание компьютерной инфраструктуры нового типа, обеспечивающей глобальную интеграцию информационных и вычислительных ресурсов на основе промежуточного ПО, которое предоставляет пользователю набор стандартизированных сервисов для обеспечения надежного, совместимого, дешевого и всепроникающего доступа к географически распределенным вычислительным и информационным ресурсам (отдельным компьютерам, кластерам и суперкомпьютерным центрам, хранилищам информации, сетям, научному инструментарию и т. д.).

Авторы идеи грид сформулировали три критерия принадлежности системы к грид-системам [44]:

- координированное использование ресурсов при отсутствии централизованного управления (в противном случае это не грид, а кластер);
- использование стандартных, открытых, универсальных протоколов и интерфейсов;
- новый уровень качества обслуживания.

Не следует смешивать технологию грид с технологией параллельных вычислений. В ее задачи входит лишь координация с использованием существующих технологий (*MPI, PVM*).

По своему назначению грид принято делить на:

1) вычислительный Грид (*computational Grid*);

2) Грид для хранения больших массивов информации (*data Grid*);

3) приборный Грид (*device Grid*), предназначенный для обслуживания сложных технических систем;

4) Грид доступа (*access Grid*), ориентированный на организацию взаимодействия групп пользователей в рамках грид;

5) семантический Грид (*semantic Grid*), являющийся некоторым гибридом семантического веба и грида, и ориентированный, в первую очередь, на поддержку концепции электронной науки (*e-science*), например, подбор исходных данных и вычислительных ресурсов для решения крупной научной проблемы.

Архитектура грид представляет собой архитектуру взаимодействующих протоколов, сервисов и интерфейсов.

Архитектура протоколов грид разделена на уровни (рис. 3.29) и может быть представлена в виде стека (набор уровней и слоев) протоколов. Эта структура аналогична модели *OSI* (модель взаимодействия отрытых систем).

Рис. 3.29. **Стек протоколов грид**

Стек протоколов включает 5 уровней:

1. Аппаратный уровень (*Fabric Layer*) имеет в себе протоколы, по которым соответствующие сервисы непосредственно работают с ресурсами.

2. Связывающий уровень (*Connectivity Layer*) составляет протоколы, которые обеспечивают обмен данными между компонентами базового уровня, а также протоколы идентификации и аутентификации.

3. Ресурсный уровень (*Resource Layer*) — это ядро многоуровневой системы, протоколы которого взаимодействуют с ресурсами,

используя унифицированный интерфейс, не различая архитектурные особенности конкретного ресурса.

4. Коллективный уровень (*Collective Layer*) отвечает за координацию использования имеющихся ресурсов.

5. Прикладной уровень (*Application Layer*) описывает пользовательские приложения, работающие в среде виртуальной организации. Приложения функционируют, используя протоколы, определенные на нижележащих уровнях.

В качестве примера реализации грид рассмотрим более подробно ннструментальные средства грид (*Globus Toolkit*), разработанные в рамках проекта Глобус (*Globus Project*).

Средства *Globus Toolkit* включают в себя следующие основные компоненты.

GRAM (*Globus Resource Allocation Manager*), ответственный за создание/удаление процессов. Этот компонент *Globus Toolkit* устанавливается на вычислительном узле грид-системы, в качестве которого может выступать как рабочая станция, так и вычислительный кластер. Пользователи формируют запросы к *GRAM* на специальном языке *RSL* (*Resource Specification Language*).

MDS (*Monitoring and Discovery Service*) обеспечивает способы представления информации о грид-системе, такой, как данные о конфигурации или состоянии как всей системы, так и отдельных ее ресурсов. Логически система организована в виде дерева, и доступ к ней осуществляется по стандартному протоколу *LDAP* (*Lightweight Directory Access Protocol*).

GASS (*Global Access to Secondary Storage*) обеспечивает возможность хранения массивов данных в распределенном окружении и доступ к этим данным.

GSI (*Globus Security Infrastructure*) обеспечивает защиту, включающую шифрование данных, а также аутентификацию и авторизацию с использованием механизма цифровых сертификатов *X*.509 [45].

Globus Toolkit получил широкое распространение, так как был первым полноценным набором инструментальных средств для разработок в области технологии грид и стал стандартом де-факто.

3.9.2. Облачные вычисления

Идея облачных вычислений состоит в предоставлении пользователям удаленного доступа к сервисам, вычислительным ресурсам и приложениям через Интернет. Набор ресурсов, предоставляемых конечному пользователю, может динамически масштабироваться в сторону увеличения или уменьшения в зависимости от нагрузки.

При этом плата с пользователей облачных платформ может взиматься в зависимости от уровня потребления, а тарификация возможна по времени работы приложений потребителя, по объему обрабатываемых данных и количеству транзакций над ними, по се-

тевому трафику. Такой подход обычно называют вычислениями по запросу (*on demand computing*).

Облака, как правило, строятся на базе крупных центров обработки данных, которые ориентированы на внешнего потребителя. При этом центры создают иллюзию единого облачного ресурса, в то время как на самом деле может быть использовано много географически распределенных ресурсов. Это обеспечивает пользователю независимость от местоположения. Сервисы, которые работают в облаке, отличаются от традиционного программного обеспечения своей конструкцией и реализацией. Облачные приложения могут разрабатываться и развертываться быстрее и в меньшей степени зависеть от изменчивости среды.

Появление современных облачных вычислений обычно относят к 2005 году и связывают выходом на рынок бизнес–решение под названием «*Amazon Web Services*» от компании *Amazon*. В 2009 году было запущено приложение *Google Apps*, что стало следующим важным шагом к популяризации и осмыслению облачных вычислений.

Согласно документу IEEE, опубликованному в 2008 году, облачные вычисления обычно определяют, как парадигму, в рамках которой информация постоянно хранится на серверах в Интернете и временно кэшируется на клиентской стороне, например, на персональных компьютерах, игровых приставках, ноутбуках, смартфонах и т. д. и предоставляется конечному пользователю в форме сервисов.

Национальный институт стандартов и технологий США (*NIST*) определяет облачные вычисления (*Cloud Computing*) как «модель обеспечения повсеместного сетевого доступа по требованию к совместно используемому пулу конфигурируемых вычислительных ресурсов, которые можно быстро предоставить и внедрить с минимумом административных усилий или взаимодействия с сервис-провайдером» [46].

Облачные вычисления по замыслу создателей должны реализовывать самообслуживание по требованию, обеспечивать широкополосный сетевой доступ, поддерживать пул ресурсов, обеспечивать возможность быстрой перенастройки или расширения и иметь измеряемое обслуживание. Облачные провайдеры обычно поддерживают целый ряд сервисов.

На рис. 3.30 показаны типовые модели облачных вычислений. Модель облачных вычислений можно представить в виде виртуальной девятислойной машины, которая отображается на две другие виртуальные машины, в качестве которых выступают облачная структура, реализованная на стороне провайдера и пользовательская структура. Виртуальная машина облачных вычислений включает следующие слои: сетевой слой, хранилища данных, серверный слой, слой виртуализации, операционная система, промежуточное ПО, среда выполнения, данные и сами приложения. Отдельные слои связаны

между собой интерфейсами. Интерфейсы определенного уровня могут быть доступны пользователю в форме сервисов.

С точки зрения распределения функций между облаком, которое находится на стороне провайдера и конечным пользователем, можно выделить четыре базовых варианта.

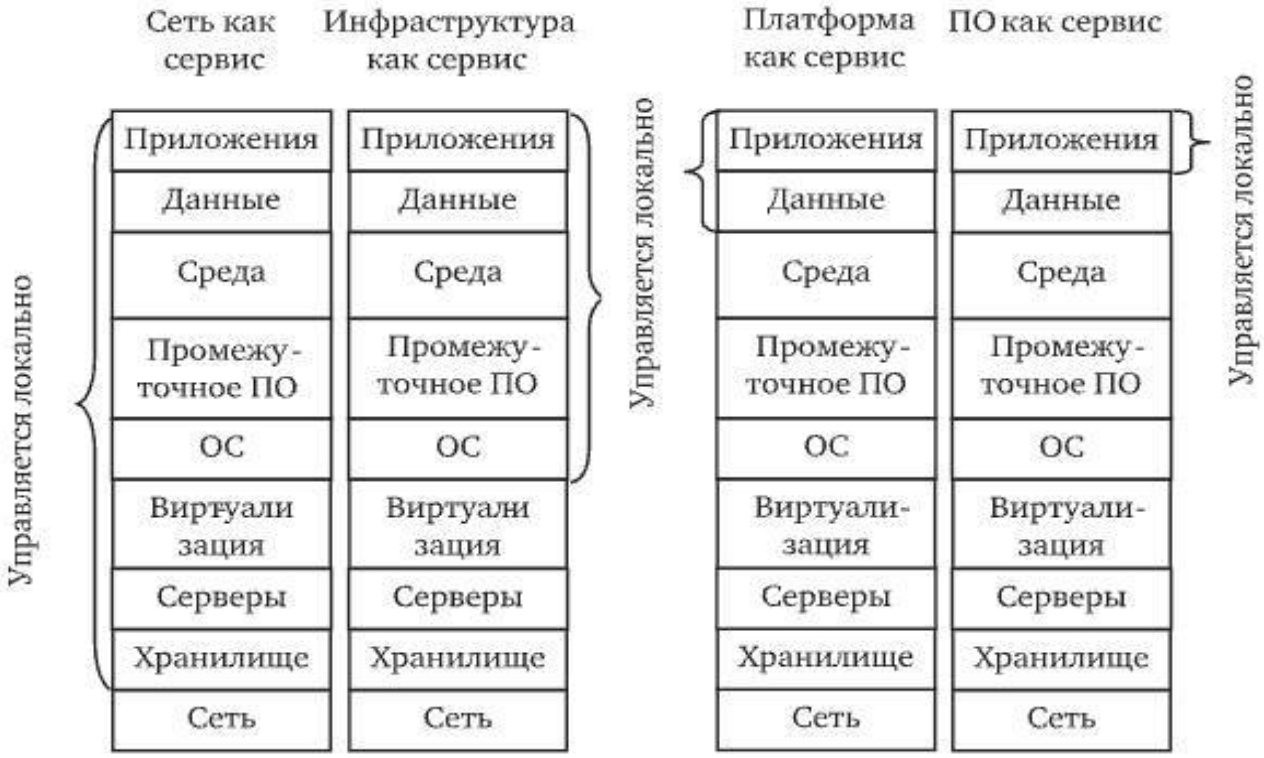


Рис. 3.30. **Типовые модели облачных вычислений**

В качестве основных обычно выделяют четыре модели: сеть как сервис (*Network As A Service*, *NaaS*), инфраструктура как сервис (*Infrastructure As A Service*, *IaaS*), платформа как сервис (*Platform As A Service*, *PaaS*) и программное обеспечение как сервис (*Software A Service*, *SaaS*).

NaaS предоставляет конечному пользователю защищенную сетевую инфраструктуру и сервисы поддержки сетевых взаимодействий. Идея состоит в том, чтобы вместо того, чтобы строить корпоративную сеть можно просто реализовать требуемую сетевую структуру в облаке на стороне провайдера.

IaaS была изначальной концепцией облачных сервисов. В этой модели провайдер создает масштабируемые эмуляторы аппаратуры в облаке и предоставляет сервисы, позволяющие пользователю запускать собственные виртуальные машины, что обеспечивает максимальную гибкость при развертывании, однако использование этой модели требует больших усилий со стороны пользователя в плане построения КИС. В этом случае пользователь освобождается от необходимости приобретать собственное серверное оборудование. В *IaaS* пользователю предоставляется возможность самостоятельно управлять ресурсами обработки, хранения, сетями, например, он (она) может устанавливать и запускать произвольное программ-

ное обеспечение, такое как ОС, платформенное и прикладное ПО, контролировать ОС, виртуальные системы хранения данных и установленные приложения. Управление и контроль основной физической и виртуальной инфраструктурой облака, в том числе сетями, серверами осуществляется провайдером. Основное достоинство такого подхода состоит в том, что отпадает необходимость в приобретении физического оборудования, организации его обслуживания.

PaaS использует базовое оборудование и программные средства нижнего уровня, предоставляемые облаком. В этом случае конечный пользователь просто использует расположенное на стороне провайдера аппаратное обеспечение центра обработки данных, операционную систему, промежуточное ПО и различные базы данных для размещения своих приложений или сервисов. Промежуточное ПО может включать, в частности базы данных. Разница между *PaaS* и *IaaS* заключается в том, что при использовании *PaaS* конечный пользователь получает доступ к масштабируемой инфраструктуре, проверенному промежуточному ПО и операционной системе. Это могут быть, в частности, контейнеры типа *Docker*. *PaaS* — это модель предоставления облачных сервисов, при которой пользователь получает доступ к использованию информационно-технологических платформ, включая ОС, СУБД, промежуточному ПО, инструментальным средствам разработки и тестирования, размещенным у облачного провайдера. В этой модели вся информационно-технологическая инфраструктура полностью управляется провайдером, который также определяется состав доступных для потребителей видов платформ и состав управляемых параметров платформ, а пользователь может использовать платформы, создавать их виртуальные экземпляры, устанавливать, разрабатывать, тестировать, эксплуатировать на них прикладное программное обеспечение, при этом динамически изменяя количество потребляемых вычислительных ресурсов.

SaaS является основой облачных вычислений. У провайдера обычно имеется набор е приложений и сервисов, которые предлагаются конечным пользователям с помощью таких клиентов, как мобильные устройства, тонкие клиенты или фреймворки в других облаках. С точки зрения пользователя, виртуальный *SaaS* фактически работает на клиенте пользователя. Эта абстракция программного обеспечения позволила отрасли добиться значительного роста в облачном сервисе. Примером *SaaS* может служить облачная реализация *Microsoft Office*. *SaaS* — это модель использования бизнес–приложений в качестве Интернет-сервисов. *SaaS*–приложения работают на сервере провайдера. Пользователь не покупает, а арендует приложение и работает с ним через браузер. В качестве преимуществ *SaaS* по сравнению с традиционным программным обеспечением обычно указывают следующие: низкая стоимость владения, более короткие сроки внедрения, возможность оперативно и часто бесплатно про-

тестировать приложение, обеспечение поддержки и обновления системы, возможность работать через Интернет, что очень удобно для географически распределенных компаний и удаленных сотрудников, минимальные требования к производительности компьютера пользователя, кроссплатформенность. Основные недостатки — это риски, связанные передачей коммерческих данных стороннему провайдеру, ограниченная скорость работы приложения и возможности сбоев в связи с необходимостью работать через Интернет.

Приведенный список сервисов далеко не полный. В настоящее время провайдеры предлагают очень много разнообразных облачных сервисов, для обозначения всего множества сервисов используют термин «Все как сервис» (*XaaS*).

Границы управляемости. Использование различных типов облачных сервисов в разной мере ограничивают свободу пользователя. Отличия в границах управляемости для различных типов облачных сервисов показаны в табл. 3.9.

Из табл. 3.9 видно, что при развертывании собственной *IT*-инфраструктуры пользователь может управлять всеми ее компонентами — от сетевых ресурсов до выполняющихся приложений. При использовании модели *IaaS* пользователь может контролировать данные, приложения и промежуточное ПО, а при использовании модели *PaaS*, все компоненты платформы предоставляются пользователю как сервисы с ограниченными возможностями для управления ими. Таким образом, пользователь получает в свое распоряжение оптимально сконфигурированную платформу, не требующую дополнительных настроек.

Таблица 3.9

Отличия в границах управляемости для различных типов облачных сервисов

—	**Собственная *IT*–инфраструктура**	*SaaS*	*IaaS*	*PaaS*
Приложения	+	+	+	–
Данные	+	+	+	–
Промежуточное ПО	+	+	–	–
ОС	+	–	–	–
Виртуализация	+	–	–	–
Серверы	+	–	–	–
Сетевая инфраструктура	+	–	–	–

Модели развертывания. Можно выделить четыре основные модели развертывания облачных сервисов.

Публичное облако (*public cloud*) представляет собой инфраструктуру, предназначенную для свободного использования и может на-

ходиться в собственности, управлении и эксплуатации коммерческих, научных и правительственных организаций (или какой-либо их комбинации). Владельцем облака является поставщик сервисов.

Общественное облако (*community cloud*) может находиться в совместной собственности, управлении и эксплуатации одной или более из организаций сообщества или третьей стороны. Физически оно может существовать как внутри, так и вне юрисдикции владельца.

Частное облако (*private cloud*) ориентировано на использование в рамках одной организации, но может включать несколько групп пользователей, например, подразделений одной организации. Частное облако может находиться в собственности, управлении и эксплуатации как самой организации, так и третьей стороны (или какой-либо их комбинации). Физически оно может существовать как внутри, так и вне юрисдикции владельца.

Гибридное облако (*hybrid cloud*) представляет собой комбинацию публичных, общественных или частных облачных инфраструктур, остающихся уникальными объектами, но связанных между собой стандартизованными или частными технологиями передачи данных и приложений.

Характеристики. К основным характеристикам облачных платформ обычно относят: масштабируемость, эластичность, многофункциональность, самообслуживание по требованию и биллинг.

Масштабируемость можно определить как способность выдерживать растущие нагрузки и обрабатывать большие объемы данных. Быстрая и надежная работа, исключающая отказы в обслуживании, задержки в ответах от системы и сбои, позволяет повысить удовлетворенность и лояльность пользователей.

Эластичность можно определить как способность реагировать на изменяющиеся условия ведения бизнеса. Например, сложившаяся рыночная конъюнктура и действия конкурентов могут потребовать быстро внедрить новый сервис, выполнив полный цикл планирования, проектирования и разработки ИС. Эластичность позволяет оперативно наращивать мощность облачной инфраструктуры, при минимальных инвестициях в оборудование и ПО. Эластичность тесно связана с масштабируемостью приложений, так как решает задачу моментального изменения количества выделяемых для работы вычислительных ресурсов.

Многофункциональность можно рассматривать как один из способов снижения расходов за счет максимального использования общих ресурсов для обслуживания разных организаций и различных групп пользователей. Многофункциональность может быть особенно привлекательна для разработчиков приложений, так как позволяет снизить собственные расходы на оплату ресурсов облачной платформы и максимально использовать доступные вычислительные ресурсы.

Самообслуживание по требованию можно определить как наличие у пользователя возможности самостоятельно определять и изменять параметры сервисов, такие как скорость доступа и объем хранимых данных без взаимодействия с провайдером сервисов.

Биллинг обычно определяют, как комплекс процессов и решений, ответственных за сбор информации об использовании сервисов, их тарификацию, выставление счетов абонентам. Провайдер автоматически исчисляет потребленные ресурсы на определенном уровне абстракции, например, объем хранимых данных, пропускную способность, количество пользователей, количество транзакций, и на основе этих данных оценивает объем предоставленных потребителям услуг.

Существующие облачные платформы. На рынке сегодня существует много как коммерческих, так и открытых платформ, на основе которых организации создают свои инфраструктуры и предлагают средства для их управления, в частности, предоставляют комплексы для превращения имеющихся ресурсов в облака.

Публичные облака. Из наиболее известных облачных решений можно отметить следующие платформы и компании:

Amazon Web Services (*AWS*). Особенностью является то, что *AWS* — это, инфраструктурный сервис (*IaaS*), который предоставляет максимум свободы разработчикам в выборе платформы и среде разработки. Однако, проблема состоит в том, что многое приходится делать вручную. Лучше всего *AWS* подходит для построения *SaaS*-сервисов.

Windows Azure. Ориентирована на разработчиков и компании, работающих с технологиями от *Microsoft*, хотя и поддерживает *PHP*, *MySQL*, *Ruby onRails*, Python, Java, Основным преимуществом *Azure* перед *Amazon Web Services* является высокий уровень автоматизации, позволяющий разработчику думать только о своем приложении, а не об инфраструктуре. Следует отметить, что *Windows Azure* поддерживает стандарты *SOAP* и *REST*.

Google App Engine (*GAE*). Ориентирован на создание *SaaS* сервисов для малого бизнеса. В основном, он поддерживает *Python* и *Java*. Платформа *GAE* предоставляет ограниченные бесплатные ресурсы (дисковое пространство и трафик), что удобно для образовательных учреждений и стартапов.

VMWare vCloud. Не является самодостаточной облачной платформой. Это промежуточное ПО, которое несколько партнеров *VMWare* предоставляют поверх своей серверной инфраструктуры. Поскольку *VMWare* является мировым лидером на рынке систем виртуализации, то главным преимуществом этой платформы является поддержка виртуальных образов приложений, что позволяет быстро и просто переносить локальные бизнес–приложения на облачную платформу без проблем, связанных с переносом сопутствующей ИТ-инфраструктуры.

IBM Cloud. Преимущественно ориентирована на крупные компании и ресурсоемкие процессы: разработка и тестирование ПО, хранение и аналитическая обработка огромных массивов данных.

Частные облака. Существует не только множество провайдеров, предоставляющих ресурсы облачных систем, но и масса различных приложений для создания собственных облаков, такие как *Eucalyptus, Open Nebula, Open Stack* [47].

Eucalyptus. (*Elastic Utility Computing Architecture for Linking Your Programs To Useful Systems*). *Eucalyptus* позиционируется как открытое (*opensource*) решение для организации доступа к вычислительным ресурсам. Также предоставляется возможность балансировки нагрузки и динамического масштабирования системы. *Eucalyptus* ориентирован на реализацию «облачных» технологий на кластерах. Интерфейс *Eucalyptus* совместим с интерфейсами облаков *Amazon*. Инфраструктура разработана для поддержки множественных клиентских интерфейсов. *Eucalyptus* реализуется на платформе *Linux* с использованием, как правило, доступных инструментариев и базовых веб–сервисных технологий.

Eucalyptus реализует схему *IAAS*, а именно «нижний» уровень облака, позволяющий запускать виртуальные машины по запросу вышестоящих приложений. Возможности *Eucalyptus* во многом позаимствованы из *Amazon*, поэтому *Eucalyptus* с некими ограничениями и допущениями можно рассматривать как открытую альтернативу сервисам *Amazon*. Пример иерархической структуры *Eucalyptus* изображен на рис. 3.31.

Рис. 3.31. **Eucalyptus. Пример иерархической структуры**

NodeController (Контроллер узла, КУ) управляет запуском, работой и остановкой виртуальных машин на подведомственном узле. Узел — это компьютер, на который установлен гипервизор, обеспечивающий работу виртуальных машин (в терминологии Eucalyptus).

Cluster Controller (Контроллер кластера, КК) управляет подведомственными контроллерами узлов (КУ). Собирает информацию о загруженности узлов и принимает решения, на каких узлах будет выполнен запуск виртуальных машин.

Образы виртуальных машин хранятся в хранилище, в качестве которого используется *Walrus* (морж) — сервис, подобный сервису *Amazon S3* (*Amazon Simple Storage Server* — простое хранилище файлов от *Amazon*). Если говорить кратко, то это просто хранилище в Интернете.

CloudController (Контроллер облака) представляет собой точку входа, на которую со стороны конечного пользователя или вышестоящего приложения поступают запросы на запуск виртуальных машин, а со стороны контроллеров кластеров поступают данные о загруженности отдельных узлов облака.

Гибридное облако — это сочетание, по крайней мере, одного частного облака и одной облачной инфраструктуры общего пользования, создающее среду, которая обеспечивает прозрачный доступ к гибридному облаку и может динамически наращиваться для управления неравномерной нагрузкой. Гибридные облака становятся все более популярны, поскольку позволяют организации создать лучшие условия для реализации своей бизнес-модели. При этом усиливается внутренний контроль приложений, которые предприятие не хочет выводить за пределы межсетевого экрана, но в то же время остается возможность использовать облачные вычисления, когда они соответствуют потребностям организации.

Схема гибридного облака показана на рис. 3.32. Работу приложений организации обеспечивают два публичных и одно частное облако с применением базовой модели *SaaS*.

Гибридные облака чаще всего используются в случаях, когда критически важные приложения необходимо оставить внутри организации. Остальные используют облако в качестве временного рабочего пространства или резервного ресурса при внезапных пиковых нагрузках.

SaaS, *IaaS* и *PaaS* — это только основные модели облачных сервисов. В первую очередь, в рамках гибридных облаков предлагается большое число других моделей. Рассмотрим некоторые из них.

Преимущества облачных вычислений.

1. Пользователь оплачивает услугу только тогда, когда она ему необходима, а самое главное он платит только за то, что использует.
2. Облачные технологии позволяют экономить на приобретении, поддержке, модернизации ПО и оборудовании.

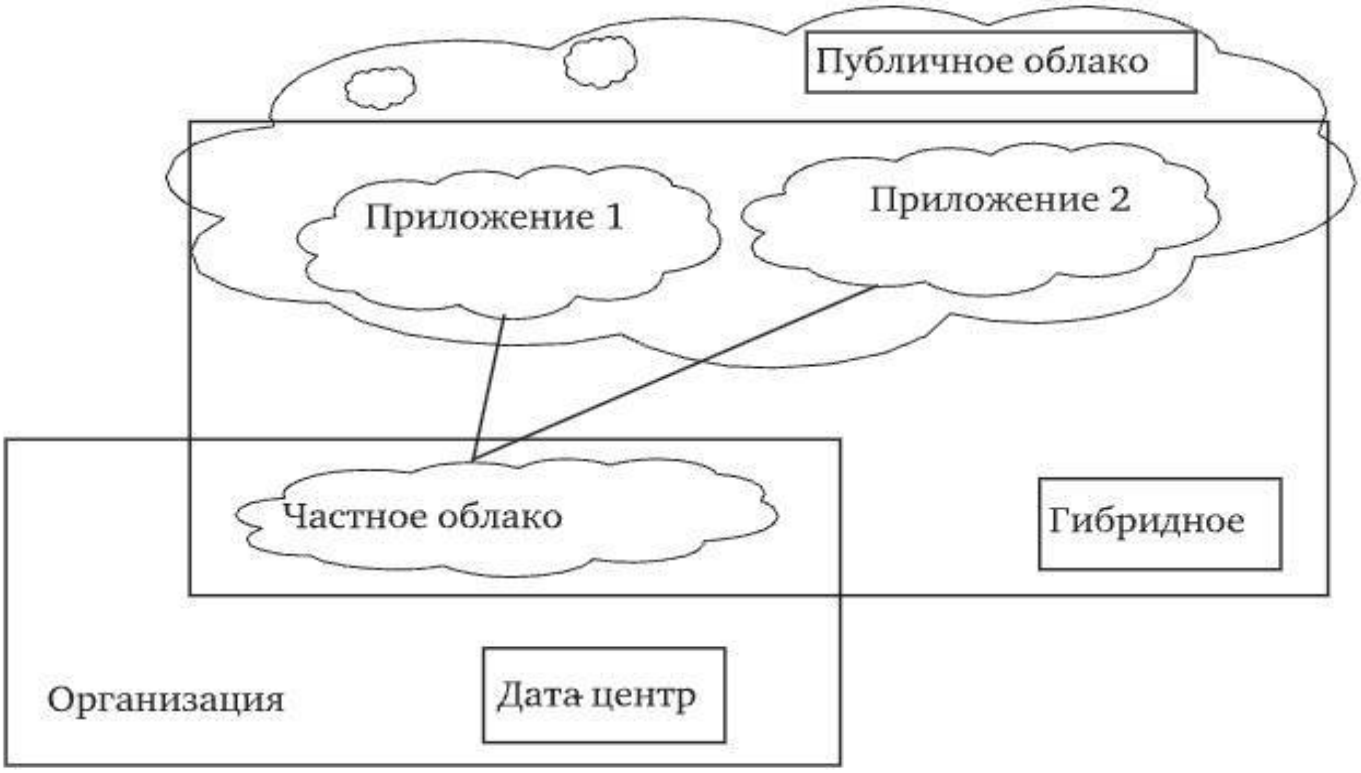


Рис. 3.32. **Гибридное облако**

3. Масштабируемость, отказоустойчивость и безопасность — автоматическое выделение и освобождение необходимых ресурсов в зависимости от потребностей приложения. Техническое обслуживание и обновление ПО производит провайдер услуг.

4. Удаленный доступ к данным в облаке — работать можно из любой точки на планете, где есть доступ в Интернет.

Недостатки облачных вычислений.

1. Пользователь не является владельцем и не имеет доступа к внутренней облачной инфраструктуре. Сохранность пользовательских данных сильно зависит от компании провайдера.

2. Для получения качественных услуг пользователю необходимо иметь надежный и быстрый доступ в Интернет.

3. Отсутствие общепринятых стандартов в направлении безопасности облачных технологий.

3.9.3. Туманные и граничные вычисления

Ограничения *IoT* на базе облачных вычислений *Fog and Edge Computing*. Интернет вещей (*IoT*) [48] представляет собой комплексную среду, которая объединяет большое количество разнородных физических объектов или вещей, таких как приборы, сооружения, животные, транспортные средства, фермы, фабрики и т. д. с Интернетом, с целью повышения эффективности таких приложений, как логистика, производство, сельское хозяйство, городские вычисления, домашняя автоматизация, окружающая среда для оказания помощи и различные вездесущие вычислительные приложения в режиме реального времени.

В большинстве своем классические *IoT* системы являются облачно–ориентированными. В них физические объекты представлены в виде веб-ресурсов, управляемых серверами в Интернете [48]. По сути, для подключения физических объектов к сети, система будет использовать различные интерфейсные устройства, такие как проводные или беспроводные датчики, исполнительные механизмы и считывающие устройства для взаимодействия с ними. Кроме того, интерфейсные устройства обеспечивают подключение к Интернету через промежуточные шлюзы, такие как маршрутизаторы, коммутаторы, базовые станций сотовой связи и т. д. В целом, *IoT*-система включает в себя три основных элемента: встроенные системы, промежуточное программное обеспечение и облачные сервисы, где встроенные системы обеспечивают доступ к данным, промежуточное программное обеспечение соединяет разнородные встроенные системы интерфейсных устройств с облаком и, наконец, облако предоставляет сервисы хранения, обработки и управления.

Это достаточно распространенный подход к построению *IoT*-систем, использование которого сталкивается с целым рядом проблем.

Хотя данная модель и является распространенным подходом к внедрению систем Интернета вещей, она сталкивается с растущими проблемами в своей области. В частности, появляются проблемы, связанные с пропускной способностью, задержкой, непрерывностью, ограничением ресурсов и безопасностью, которые принято называть *BLURS* (*bandwidth*, *latency*, *uninterrupted*, *resource-constraint*, *security* [48].

Пропускная способность. Современные КФС используют все большее число датчиков, которые выдают все больший объем информации и со временем каналы связи начинают не справляться с нагрузкой. Например, движущийся автомобиль может генерировать десятки мегабайт данных в секунду с информацией о его маршруте, скорости, состоянии автомобиля, состоянии водителя, окружающей среде, погоде и т. д. Если речь идет о самоуправляемом транспортном средстве, то это гигабайты данных в секунду из-за необходимости потоковой передачи видео в реальном времени. Поэтому полностью пересылать все эти данные в облако оказывается просто невозможным.

Задержка. При использовании облака часто оказывается трудным обеспечить время отклика даже в пределах десятков миллисекунд. В частности, промышленные системы, автономные автомобили, приложения виртуальной и дополненной реальности, приложения для финансовой торговли в реальном времени, приложения для здравоохранения и ухода за пожилыми людьми не могут позволить себе работать с такими задержками.

Непрерывность. Большое расстояние между облаком и оконечным устройством может привести к проблемам, связанным с не-

стабильной и прерывистой работой сети связи. Например, подключенное к облаку беспилотное транспортное средство не сможет функционировать должным образом из-за отключения, произошедшего на промежуточном узле между транспортным средством и удаленным облаком.

Ограниченные ресурсы. Как правило, многие оконечные устройства ограничены в ресурсах и не могут выполнять сложные вычислительные задачи, и, следовательно, они должны непрерывно передавать данные для обработки в облако. Однако, это не лучшее решение для систем, имеющих батарейное питание.

Безопасность. Большое количество оконечных устройств с ограничениями по вычислительной мощности могут не иметь достаточно ресурсов для защиты от атак. В частности, оконечные устройства, которые используют удаленное облако для обновления своего программного обеспечения, могут достаточно легко стать целями злоумышленников, поскольку злоумышленники способны выполнять вредоносные действия в пограничной сети, где расположены оконечные устройства, а облако не имеет полного контроля над этой сетью. Кроме того, злоумышленник может также повредить или контролировать оконечное устройство и отправлять от его имени ложные данные в облако.

Основные понятия и определения. Возникает закономерный вопрос: каким образом могут быть решены эти проблемы? Очевидно, что для того, чтобы решить эти проблемы требуется приблизить вычислительные ресурсы к источникам данных. Однако, с точки зрения программирования, наличие единого виртуального ресурса, реализованного в виде облачного сервиса, очень привлекательно. В последнее десятилетие было предпринято несколько попыток решить проблему в рамках облачного подхода, однако нельзя назвать эти подходы успешными.

В результате работы *Open Fog consortium*, в которую входят такие фирмы и университеты как: *ARM, AT&T, Cisco, Dell, Fog Horn Systems, Fujitsu, Hitachi, Foxconn, Indian Institute of Technology, Intel, Microsoft, Mitsubishi Electric Corporation, Princeton University* и еще несколько десятков других организаций остановились на решении, предложенном фирмой *Cisco*, которое получило название «туманные вычисления» (*fog computing*). Эта организация в 2017 году разработала эталонную архитектуру туманных вычислений, на базе которой годом позже был создан международный стандарт *IEEE* 1934 [49]. В соответствии со стандартом основным назначением туманных вычислений является обеспечение недостающего звена в континууме «облако-вещь».

Модель туманных вычислений включает в себя широкий спектр оборудования и сетей. В целом это концептуальная модель, которая учитывает все возможности расширения облака за счет включения

некоторой промежуточной сети. С точки зрения пользователя, туманная система представляет собой сервисную экосистему.

Туманные архитектуры выборочно перемещают вычисления, хранение, связь, управление и принятие решений ближе к границе сети, туда, где генерируются данные, чтобы устранить ограничения по *BLURS*-характеристикам для обеспечения критически важных вариантов использования с высокой плотностью данных.

В упомянутом стандарте туманные вычисления определяются следующим образом:

туманные вычисления — это горизонтальная архитектура системного уровня, которая распределяет вычислительные, запоминающие, управляющие и сетевые функции ближе к пользователям по континууму «облако-вещь».

По мнению авторов стандарта, туманные вычисления — это расширение традиционной модели облачных вычислений, в которой реализации архитектуры могут располагаться на нескольких уровнях топологии сети. Ключевой идеей является то, что все преимущества облачных технологий должны быть сохранены с помощью этих расширений. К этим преимуществам относятся возможности использования таких механизмов как контейнеризация, виртуализация, оркестровка, управляемость и эффективность. Во многих случаях туманные вычисления работают с облаком.

В качестве основных характеристик туманных вычислений выступают безопасность, масштабируемость, открытость, автономность, надежность, доступность и удобство обслуживания, гибкость, иерархичность и программируемость.

Близким по значению к термину «туманные вычисления» является термин «граничные вычисления» (*edge computing*).

Граничные вычисления. Согласно определению Гартнера, *граничные вычисления* — «это подвид распределенных вычислений, в котором обработка информации происходит в непосредственной близости к месту, где данные были получены и будут потребляться» [50]. Это основное отличие граничных вычислений от облачных вычислений, при которых информация собирается и обрабатывается в публичных или частных дата–центрах. Основным отличием от локальных вычислений является то, что обычно *edge computing* — это часть большой системы, которая включает в себя сбор статистики, централизованное управление и удаленное обновление приложений на граничных устройствах.

Различия между туманными и граничными вычислениями. Это достаточно часто задаваемый вопрос, на который разные специалисты отвечают по-разному. Авторы стандарта [49] считают, что ключевое отличие состоит в том, что туманные вычисления работают с облаком, в то время как граничные вычисления предполагают выход за пределы облака.

Существует точка зрения, в соответствии с которой туманные вычисления используют ресурсы, применяемые в различных устройствах, например в умном доме может быть много умных вещей, имеющих встроенные контроллеры, на которых можно организовать вычисления, объединив их в сеть. Можно, напротив, поставив в доме мини-дата-центр, выполнять в нем все вычисления и использовать его как хранилище данных.

Однако, разница между этими подходами не слишком велика, поскольку реализуется одна и та же идея, которая состоит в том, что вычисления следует выполнять в непосредственной близости от источников данных, поэтому эти подходы будут рассматриваться как единый подход — гранично-туманные вычисления.

Иерархическая модель гранично-туманных вычислений. Иерархическая модель гранично-туманных вычислений показана на рис. 3.33. Данная модель, фактически, является моделью размещения по уровням (*deployment model*). Серверы развертываются на трех граничных уровнях или границах (*edges*): внутреннем, среднем и внешнем [51].

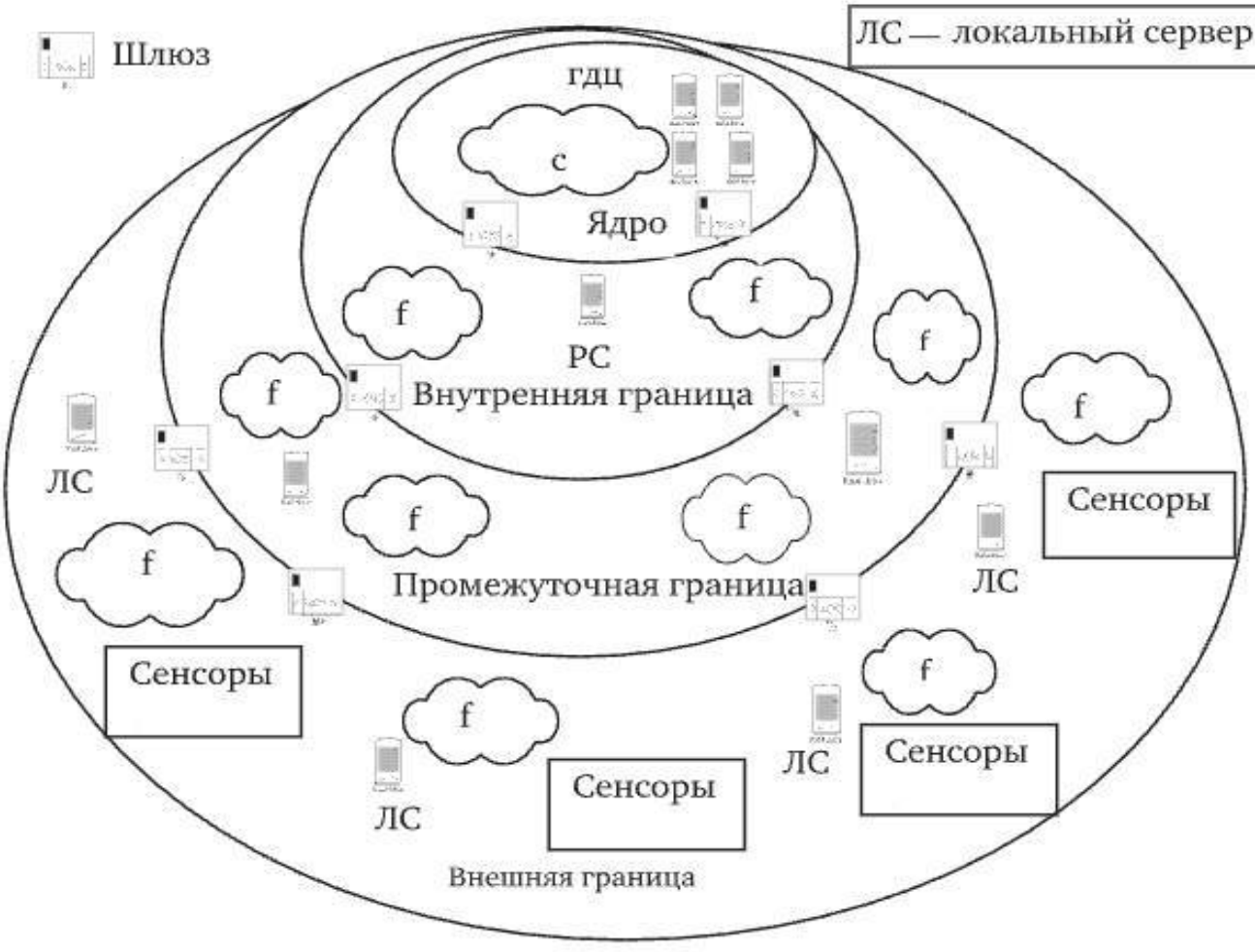


Рис. 3.33. **Иерархическая модель гранично-туманных вычислений**

Внутренний слой. Внутренняя граница (также известная как ближняя граница) соответствует глобальной сети фирмы, интернет–провайдеров, центра обработки данных. Этот уровень можно

рассматривать как уровень, на котором работают провайдеры, которые традиционно предлагали только основную инфраструктуру для подключения локальных сетей к Интернету. Тем не менее, возрастающие потребности в повышении качества обслуживания привели к созданию механизма географически распределенного кэширования. Например, многие интернет-провайдеры (например, *AT&T*, *Vodafone*, *Deutsche Telekom* и др.) знают, что многим местным предприятиям требуется облако с низкой задержкой, и они предлагают локальное облако внутри страны. Основываясь на эталонной архитектуре туманных вычислений облачные центры обработки данных на основе глобальной сети можно рассматривать как туман внутреннего слоя.

Средний слой — это то, что соответствует интуитивному пониманию туманных вычислений. Этот слой состоит из сетей двух типов: локальных сетей и сотовой сети. Обычно в ней присутствует маршрутизатор, который обеспечивает доступ к облачным сервисам внутреннего слоя. Сервера располагаются таким образом, чтобы они находились на расстоянии одного перехода (*hope*) между устройством Интернета вещей и сервером.

Такой подход также известен как локальное облако, локальный центр обработки данных или облако. На серверах данного уровня активно используются механизмы виртуализации и контейнеризации.

На этом уровне самым активным образом используются сотовые сети. В целом большинство развитых городов имеют широкий охват сотовых сетей, предоставляемых многочисленными типами базовых станций, которые являются идеальными объектами для использования в качестве придорожных узлов туманных вычислений для различных случаев использования мобильного Интернета вещей, таких как подключенные транспортные средства, мобильное здравоохранение и виртуальная или дополненная реальность, которые требуют быстрый процесс и реагирование на поток данных в реальном времени.

Внешний слой. Также известен как крайний слой, дальний край или туман (*mist*). Представляет собой интерфейсную часть сети Интернета вещей, которая состоит из трех типов устройств: оконечные устройства, интегрированные устройства и IP-шлюзы.

Оконечные устройства, такие как датчики или исполнительные механизмы, обычно управляются микроконтроллерами, которые имеют ограниченную вычислительную мощность и память. Например, популярный однокристальный *микроконтроллер Atmel ATmega328*, который является процессором *Arduino Uno Rev3* [52], работает на частоте 20 МГц и имеет всего 32 КБ флэш–памяти. Обычно администраторы Интернета вещей не ожидают развертывания сложных задач на устройствах такого типа. Однако, благода-

ря возможности программирования на местах современных беспроводных датчиков и исполнительных механизмов система Интернета вещей всегда может динамически и удаленно обновлять или перенастраивать программный код устройств.

Интегрированные устройства — это встроенные в оборудование микропроцессоры, которые могут обладать достаточно приличными вычислительными возможностями. Они также имеют встроенные средства сетевого взаимодействия, такие как *Wi-Fi* и *Bluetooth*.

Эти устройства управляются процессорами, которые обладают приличной вычислительной мощностью.

Кроме того, интегрированные устройства имеют множество встроенных возможностей в сети (например, подключения *Wi-Fi* и *Bluetooth*), встроенные датчики, такие как гироскоп, ускорители. Примерами таких устройств могут служить смартфоны и планшеты, которые работают под управлением полноценных операционных систем.

Оконечные устройства, работающие во внешнем слое, не используют *IP*-протокол, в частности, по причине ограничений на потребляемую мощность. Устройства внешнего слоя используют очень простые протоколы, такие как *Bluetooth* с низким энергопотреблением, *ZigBee* или *Z-Wave*, которые не могут работать напрямую с *IP* сети.

Шлюз можно построить на базе одноплатного компьютера, например, такого как *Raspberry Pi* или любого другого одноплатного компьютера.

Модель *Raspberry Pi 4 Model B* обеспечивает производительность настольного компьютера, сравнимую с системами ПК начального уровня x86. Ключевые характеристики этого продукта включают высокопроизводительный 64-разрядный четырехядерный *RISC* процессор *ARM Cortex-A72*, поддержку двух дисплеев с разрешением до 4K через пару портов micro–HDMI, аппаратное декодирование видео до 4Kp60, беспроводную локальную сеть 2,4 / 5,0 ГГц, *Bluetooth* 5.0, *Gigabit Ethernet*, USB 3.0, а в качестве жесткого диска использует *SD*-карты. Кроме этого, имеется большое число модулей расширения. Основной операционной системой является ОС *Raspbian*, построенная на базе популярной ОС *Debian GNU/Linux*. Эта модель появилась в 2019 году. Плата по размеру сопоставима с кредитной карточкой. Очевидно, что, соединив несколько плат через *Gigabit Ethernet*, можно построить достаточно мощный кластер.

На туманном узле, построенном на базе одноплатного компьютера можно легко разместить среду виртуализации, такую как *Docker Containers Engine*, или построить локальную облачную среду. Для отладки к *HDMI*-порту можно подключить дисплей, а к USB-портам — клавиатуру и мышь, и превратить одноплатный компьютер в полноценный ПК.

Туманные узлы. Средний слой, который реализует туманные вычисления, можно представить себе состоящим из множества узлов, которые далее будем называть туманно–граничными узлами (*Fog Edge Nodes, FEN*). Обобщенная структура такого узла показана на рис. 3.34.

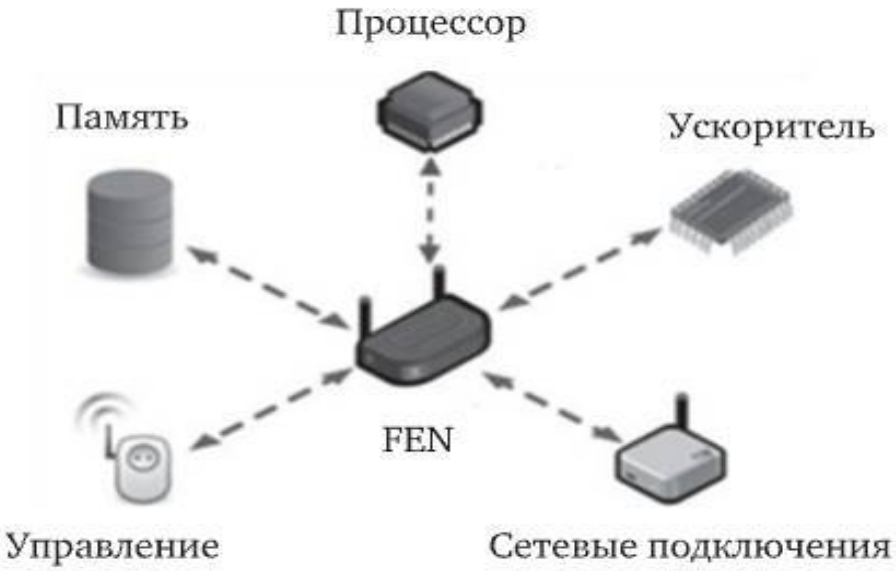


Рис. 3.34. **Обобщенная структура FEN**

FEN обычно рассматривают, принимая во внимания следующие архитектурно значимые характеристики: задержка, эффективность, безопасность, возможность масштабирования, когнитивность, гибкость.

Задержка. Уменьшение задержек при реакции на определенные внешние события. Основным источником получения выигрыша можно считать уменьшение времени пересылок. Основным источником получения выигрыша можно считать уменьшение времени пересылок в случае, когда альтернативой является обработка события в облаке, и времени вычислений, если в качестве альтернативы рассматривается обработка события в *IoT*-контроллере.

Эффективность. FEN повышает эффективность с точки зрения увеличения производительности и снижения ненужных затрат. Например, применяя FEN в системе здравоохранения или ухода за пожилыми людьми, многие задачи можно решать, используя серверы туманного слоя для обработки сенсорных данных. В идеале, поскольку процесс происходит рядом с источником данных, система может генерировать результат намного быстрее. Кроме того, можно снизить затраты за счет уменьшения объема интернет–трафика.

Безопасность. FEN поддерживает дополнительную защиту устройств Интернета вещей для обеспечения безопасности и надежности транзакций. Например, современные беспроводные датчики, развернутые на открытом воздухе, часто требуют удаленного обновления исходного кода через беспроводные сети для решения проблем, связанных с безопасностью. Однако из-за различных динамических факторов окружающей среды, таких как нестабильный уровень сигнала, перебои, ограниченная пропускная способность

и т. д., удаленный сервер может столкнуться с трудностями при быстром выполнении обновления и, следовательно, повышает вероятность кибератаки. С другой стороны, если доступна туманная инфраструктура, то обновления можно вносить намного быстрее. Кроме того, можно оперативно реагировать на попытки атак.

Когнитивные возможности. Туманная структура может накапливать и использовать информацию о поведении своих клиентов и использовать ее, чтобы принимать решения о том, где и когда развертывать модули, реализующие функции вычисления, хранения и управления. Это позволяет реализовывать такие механизмы как самоадаптация, самоорганизация, самовосстановление и др. В перспективе это позволяет перейти от пассивных оконечных устройств к интеллектуальным датчикам, которые могут непрерывно работать и реагировать на требования клиентов, не полагаясь на решение из удаленного облака.

Гибкость. Использование туманных архитектур позволяет повысить уровень гибкости систем, в частности, за счет того, что решение о реконфигурации может приниматься «по месту», что позволяет ускорять развертывание крупномасштабных инфраструктур Интернета вещей.

Рассмотрим более подробно за счет чего можно получить перечисленные выше преимущества. В каждом *FEN* реализуются пять базовых механизмов: хранение данных, вычисление, ускорение вычислений и сети, поддержание сетевых соединений, управление.

Хранение. С точки зрения хранения информации, *FEN* реализует функции, преимущественно связанные с кэшированием и временным хранением информации с целью повышения производительности при решении задач, связанных с доставкой контента. Например, беспилотные автомобили могут получать свежую информацию о состоянии внешнего мира, собранную другими автомобилями.

Вычисление. FEN реализует доступ к вычислительным ресурсам в двух формах. Чаще всего это сервисы типа инфраструктура или платформа как сервис (*I/PaaS*) и программное обеспечение как сервис (*SaaS*). Как правило, при построении *FEN* используются виртуальные машины с гипервизором или механизмы контейнеров, которые позволяют развертывать необходимое программное обеспечение в изолированной среде. Сам характер обработки может быть разным. Это может быть простая обработка по запросу клиента, а может быть и более сложная обработка, при которой клиенту возвращается некоторый ответ, представленный в форме знания.

Ускорение. В *FEN* используются два основных подхода к увеличению производительности: ускорение вычислений и увеличение скорости работы сети.

Одним из действенных механизмов является использование технологии сетевой виртуализации, которая позволяет *FEN* параллель-

но управлять несколькими таблицами маршрутизации и реализовывать программно-определяемую сеть. Таким образом, клиенты узлов FEC могут настраивать маршрутизацию в индивидуальном порядке для своих приложений, чтобы достичь оптимальной скорости передачи данных по сети.

Повысить производительность *FEN* можно несколькими способами. Во-первых, можно просто использовать более мощный процессор. Во-вторых, можно использовать графический или сигнальный сопроцессор. В-третьих, можно использовать вентильные матрицы (*FPGA*). В частности, использование графических процессоров для реализации сложных алгоритмов является распространенным подходом в облачных вычислениях. Поэтому можно предположить, что поставщики *FEN* могут также предоставлять оборудование, содержащее независимые графические процессоры средней или высокой производительности. Программируемые логические интегральные системы (ПЛИС) уже довольно давно используют для реконфигурации датчиков во время выполнения. Кроме того, по сравнению с графическими процессорами, *FPGA* потенциально может быть более энергоэффективным подходом для обеспечения необходимого ускорения, основанного на предоставлении клиентам возможности настраивать свой индивидуальный код на *FEN*.

Поддержание сетевых соединений. Сетевое взаимодействие *FEN* включает в себя вертикальные и горизонтальные связи. Вертикальная сеть соединяет объекты и облако, которое используют *IP* сети; в то время как горизонтальная сеть в зависимости от комплектации может использовать самые разные протоколы.

Узлы FEC обеспечивают вертикальные сетевые соединения с использованием стандартных сетевых протоколов *IP*, таких как сокеты *TCP/UDP*, *HTTP*, *CoAP* или очереди сообщений (*AMQP*; *ISO/IEC* 19464), *MQTT* (*ISO/IEC PRF* 20922) и т. д.

Необходимость поддержки горизонтальных сетевых взаимодействий основывается на различных требованиях к оптимизации, таких как энергоэффективность или эффективность сетевой передачи. Системы Интернета вещей часто используют разнородные экономически эффективные сетевые подходы. В частности, умный дом, умные заводы и подключенные транспортные средства обычно используют *Bluetooth*, *ZigBee* и *Z-Wave* на устройствах Интернета вещей и подключают их к сетевому шлюзу для обеспечения подключения между устройствами и внутренним облаком. В общем, устройства сетевого шлюза являются идеальными объектами для размещения серверов туманного уровня, поскольку они имеют возможность подключения к устройствам Интернета вещей.

Управление. На туманном уровне может реализовываться четыре основных механизма управления: развертывание, управление эле-

ментами нижележащего уровня, управление внешними взаимодействиями и безопасность:

1. Управление развертыванием позволяет клиентам динамически выполнять настраиваемое развертывание программного обеспечения. Кроме того, клиенты могут настраивать *FEN* для управления тем, какую программу должен выполнять узел и когда он должен ее выполнять. Кроме того, клиентам может предоставляться сервис получения информации о полной топологии сети, что позволяет им управлять перемещениями приложений с одного узла *FEN* на другой. Кроме того, клиенты также могут управлять несколькими *FEN* для достижения оптимальной производительности своих приложений.

2. Управление элементами нижележащего уровня предполагает, что управление устройствами ведется не напрямую из облака, а используются некоторые механизмы типа политик, т. е. в *FEN* из облака поступают указания что нужно сделать, а *FEN* решает, как это должно быть сделано.

3. Управление внешними взаимодействиями предполагает организацию взаимодействия с внешними сущностями, которые могут принадлежать другим организациям и использовать собственные протоколы.

4. Контроль безопасности является одним из основных требований, предъявляемым к FEN, и предполагает реализацию таких функций как аутентификация, авторизация, идентификация и защита виртуализированной среды выполнения, размещенных на узлах FEN.

Эталонная архитектура туманных вычислений (*Open Fog Reference Architecture*). Данная эталонная архитектура была предложена *Open Fog* консорциумом в 2017 году. Она определяет архитектурные точки зрения, архитектурные виды и архитектурные перспективы (табл. 3.10).

Таблица 3.10

Туманные вычисления. Архитектурные точки зрения

<table>
<tr><td rowspan="11">Производительность</td><td colspan="4">Сервисы прикладного уровня</td><td rowspan="11">Безопасность</td><td rowspan="11">Управляемость</td><td rowspan="11">Данные. Аналитика. Управление</td><td rowspan="11">Бизнес–интересы</td></tr>
<tr><td colspan="4">Сервисы низкого уровня</td></tr>
<tr><td colspan="4">Управление туманной структурой</td></tr>
<tr><td colspan="4">Виртуализация аппаратуры</td></tr>
<tr><td colspan="4">Управление узлами</td></tr>
<tr><td colspan="4">Безопасность узлов</td></tr>
<tr><td>Сеть</td><td>Ускорители</td><td>Вычислители</td><td>Память</td></tr>
<tr><td colspan="4">Промежуточное ПО</td></tr>
<tr><td colspan="4">Сенсоры и актуаторы</td></tr>
</table>

В рамках предложенной эталонной архитектуры выделяется три архитектурных вида: программное обеспечение, система и узел.

Программный вид представлен тремя верхними слоями: сервисы прикладного уровня, сервисы поддержки прикладных сервисов и систему управления узлами.

Системный вид представлен средними уровнями, которые включают аппаратную платформу и ее виртуализацию.

Вид узла представлен в двух нижних слоях, показанных в табл. 3.10, и включает в себя датчики, приводы и управление, а также уровень абстракции протоколов.

Эталонная архитектура туманных вычислений включает в себя следующие архитектурные перспективы.

Производительность. Низкая задержка является одной из основных причин использования туманных архитектур. Существует множество требований и конструктивных соображений для различных заинтересованных сторон, чтобы обеспечить их выполнение. Это включает в себя критичные по времени вычисления, чувствительные ко времени сети, сетевые протоколы времени и т. д. Это сквозная проблема, поскольку она влияет на систему и сценарий развертывания.

Безопасность. Безопасность имеет решающее значение для успеха всех сценариев развертывания туманных вычислений. Если аппаратный уровень можно считать достаточно защищенным, то на программном уровне имеется достаточно много проблем с безопасностью.

Управляемость. Управление всеми аспектами развертывания туманной структуры является критически важным аспектом на всех уровнях иерархии туманных вычислений.

Аналитика и управление. Для обеспечения автономности туманных узлов требуется локальная аналитика данных в сочетании с контролем. Управление должно быть разнесено по уровням.

ИТ-бизнес и кроссуровневые приложения. Туманная среда обычно представляет собой сервисную экосистему, для построения которой используются оборудование и программные компоненты от разных производителей. При этом требуется обеспечить возможность миграции и корректного функционирования всей системы. Кроме того, должна обеспечиваться возможность миграции приложений между слоями.

Пример использования туманных вычислений. Кластеры дронов.

В последние несколько лет беспилотные летательные аппараты (дроны) получили широкое распространение и широкое применение в различных областях, таких как военное дело, сельское хозяйство, торговля и др. Все большее число дронов используются в составе кластеров (групп) реализующих роевое поведение. Ожи-

дается, что к 2025 году беспилотные летательные аппараты будут обладать полной способностью к автономной групповой работе в составе кластеров.

Кластеры дронов можно рассматривать как одну из реализаций концепции облачно–граничных вычислений.

По сравнению с обычными сетями граничных вычислений с поддержкой одного или нескольких дронов, туманные сети поддержки кластера (роя) дронов могут намного более эффективно решать такие задачи, как доставка грузов, мониторинг земли, точное сельское хозяйство, крупномасштабные военные операции и многие другие.

Этот подход называют интернет дронов (*Internet of Drones*, *IoD*). *IoD* обладает рядом несомненных достоинств, основными из которых являются следующие:

— использование кластера дронов позволяет распределять вычислительные ресурсы между большим количеством дронов, т. е. распараллелить процессы;

— повышенная надежность связи, которая может быть достигнута благодаря гибкому антенному соединению нескольких дронов с одной антенной для формирования виртуальной системы с несколькими антеннами, с помощью которой качество приема сигнала и передачи информации может быть значительно улучшено при той же или даже меньшей мощности используемого приемно-передающего оборудования;

— может быть достигнуто существенное повышение отказоустойчивости, например, если несколько дронов покидают кластер, то работы могут быть перераспределены между оставшимися дронами.

Эти преимущества можно получить в случае, если дроны смогут общаться не только с наземной станцией, но и между собой.

Возможные сценарии применения. Ниже рассмотрены наиболее типичные применения кластеров дронов. Следует заметить, что этот список далеко не полный.

Мониторинг Земли. Использование традиционных механизмов мониторинга, основанных на сетях стационарных датчиков для слежения за состоянием таких объектов как вулканы, динамика моря, а также измерение температуры атмосферы, уровней загрязняющих веществ, выбросов углерода в воздух и т. д. Как правило, являются дорогостоящими или трудоемкими из-за отсутствия поблизости адекватных средств хранения и обработки информации. Эффективной альтернативой может быть развертывание кластера дронов, оснащенных бортовыми датчиками и блоками обработки, что позволит своевременно и экономически эффективно выполнять вышеуказанные задачи за счет автономного полета по заданной траектории.

Военные применения. На поле боя рой дронов, охватывающий большую площадь, может собирать и обрабатывать массивные

разведданные в режиме реального времени, а затем передавать их на близлежащие боевые станции, значительно повышая их осведомленность о ситуации на поле боя. Например, кластер дронов можно использовать, чтобы помочь обнаружить скрытых врагов в уличных боях и найти вражеские корабли вне поля зрения в морских сражениях в режиме реального времени. Кластер дронов может сыграть важную роль в этих сценариях.

Борьба со стихийными бедствиями. В случае стихийного бедствия крайне важно провести спасательные работы в первые несколько часов. Отсутствие надежной связи и основных средств неотложной медицинской помощи, несомненно, окажет негативное влияние на жизнь и имущество людей. Поэтому ситуация, как правило, требует, чтобы спасательные силы оказали немедленную помощь. В этом случае кластер дронов, способных выполнять вычисления непосредственно на борту, может сыграть очень важную роль. Дроны могут помочь оценить риски и повреждения, а также обеспечить связь с изолированными людьми и доставить грузы.

Доставка грузов. На базе кластера дронов можно организовать сервис быстрой и дешевой доставки грузов. Этот подход привлекает активное внимание таких компаний, как Amazon и Google.

Точное земледелие (*Precision agriculture*). Точное земледелие [53] — это «комплексная высокотехнологичная система сельскохозяйственного менеджмента, включающая в себя технологии глобального позиционирования, географические информационные системы, технологии оценки урожайности, технологию переменного нормирования, технологии дистанционного зондирования земли и решения технологии “Интернета вещей”». Кластерам дронов в системе точного земледелия отводится важная роль. Это может быть мониторинг состояния здоровья растений, распыление пестицидов на пораженные растения, что сопряжено с риском для здоровья людей.

Типовые архитектурные решения. Если рассматривать кластер дронов как элемент КФС, реализующий парадигму граничных вычислений, то такие КФС можно классифицировать в соответствии с табл. 3.11.

С точки зрения назначения — это могут быть системы мониторинга окружающей среды, системы военного назначения, системы сельскохозяйственного назначения, системы, используемые в условиях чрезвычайных ситуаций, системы поддержки точного земледелия и др.

Кластер дронов может быть самого разного размера: от единиц до тысяч дронов.

Структура кластера может быть однородной или неоднородной. Чаще всего используются иерархические структуры. Это может быть одноуровневая структура, когда кластер дронов принимает решения автономно. При двухуровневой организации кластер дронов

напрямую взаимодействует с наземной станцией или станциями. При трехуровневой организации для взаимодействия с наземной станцией используется отдельный, обычно более крупный дрон.

Таблица 3.11

Классификация архитектурных решений КФС, использующих кластеры дронов

Назначение	Число дронов	Структура	Вычислительные возможности дрона	Коммуникационные возможности дрона	Использование роевых алгоритмов
Мониторинг окружающей среды, военные применения, сельское хозяйство	Малое, среднее, большое	Одно–, двух–, трехуровневая	Незначительные, средние, значительные	*D2S*, *D2S+D2D*	Не используют, могут взаимодействовать по типу *D2D* или ориентированы на использование роевых алгоритмов

Дрон, входящий в состав кластера, может обладать разными вычислительными и коммуникационными возможностями. У простейших дронов на борту может находиться контроллер, обладающий минимальными вычислительными возможностями. В крупных дронах на борту может находиться контроллер, способный выполнять функции туманного узла.

С точки зрения коммуникационных возможностей дроны могут иметь возможность реализовывать разные типы взаимодействия, в частности взаимодействия типа дрон–наземная станция (*D2S*) и/или дрон-дрон (*D2D*).

Реализация дронами механизма роевого интеллекта напрямую не связана со способностью реализовывать взаимодействие типа *D2D*. Дроны могут взаимодействовать друг с другом, но не использовать роевые алгоритмы и, напротив, роевые алгоритмы могут реализовываться в наземных станциях на основе информации, полученной от дронов.

Можно выделить следующие типовые подходы к построению КФС, использующих кластеры дронов:

— система строится по принципу классического *IoT* (рис. 3.35);

— система строится по принципу туманно-граничных вычислений (рис. 3.36).

Дрон только формирует поток данных (чаще видеоданных), а обработка идет на наземной станции. Связь может идти либо напрямую по принципу *D2S* (рис. 3.35), либо через более крупный дрон (рис. 3.36). Если на борту дрона можно установить достаточно мощный вычислитель, то обработку или существенную часть обработки

можно организовать по месту. Это позволяет уменьшить задержки и снизить нагрузку на каналы связи. Основные ограничения на вычислительные мощности на борту дрона связаны, с потребляемой мощностью и весогабаритными ограничениями. В этом случае наземная станция выступает в роли сервера, обслуживающего заявки дрона. Дрон обычно работает с ближайшей наземной станцией.

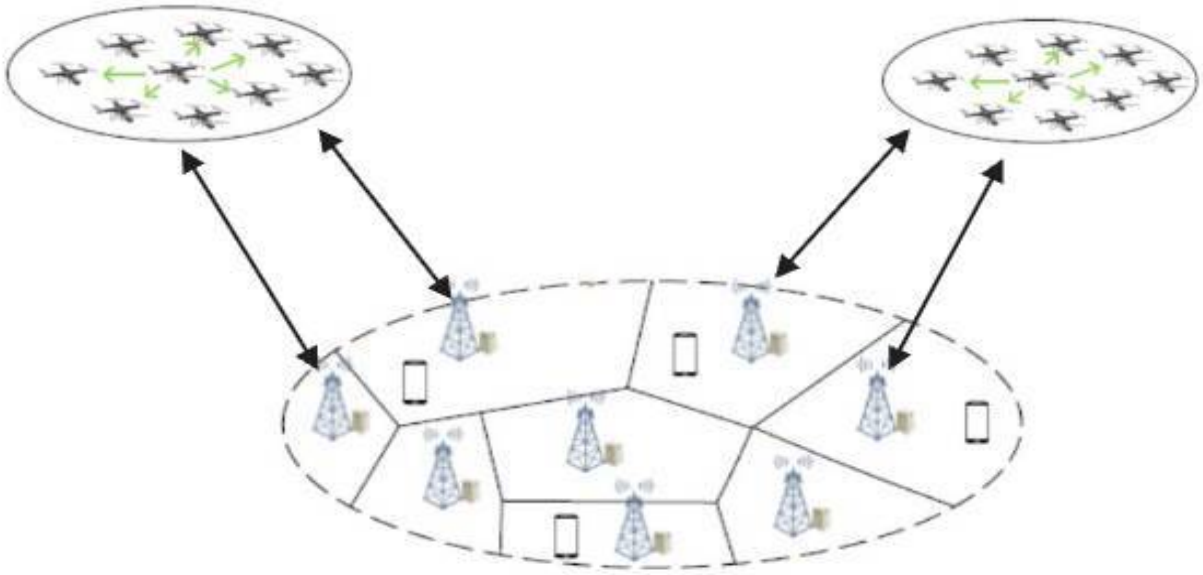


Рис. 3.35. **КФС по принципу классического IoT**

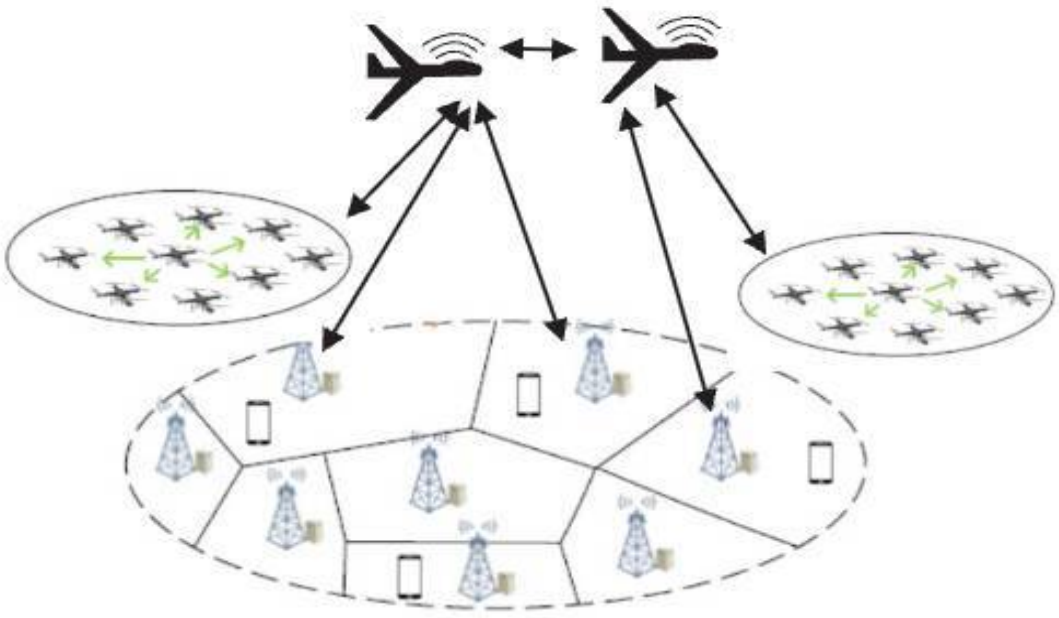


Рис. 3.36. **ДКФС по принципу туманно-граничных вычислений**

Дроны могут взаимодействовать между собой для решения разного рода задач, таких как доставка данных, построение карты местности, оповещение о событиях и др.

В частности, для первой парадигмы, для удовлетворения требований к обслуживанию со сверхнизкой задержкой можно использовать возможности наземных систем мобильной связи. В [54] описана система управления кластером дронов, использующая 5G–сети для построения «летающей» сети (*Flying Ad-Hoc Network*,

FANET), состоящей из дронов, оснащенных средствами поддержки граничных вычислений. В соответствии с предлагаемой структурой большие объемы данных, создаваемые наземными устройствами, расположенными в географических районах, удаленных от структурированной базовой сети, могут быть оперативно переправлены к стационарным серверам, на которых осуществляется накопление и обработка данных.

Для решения задачи, связанной с минимизацией энергопотребления, вычислительные элементы каждого дрона могут быть адаптивно включены и выключены в зависимости от уровня активности контролируемой области. Кроме того, если один из дронов перегружен, он может с некоторой вероятностью переложить работу на близлежащие недогруженные дроны, чтобы сэкономить свою энергию и, следовательно, продлить продолжительность полета кластеров дронов.

Данный вариант предполагает, что в районе действия кластера дронов работает мобильная связь.

Однако, если речь идет об устранении чрезвычайных ситуаций, то мобильная связь часто отсутствует. В этом случае сами дроны могут использоваться в качестве маршрутизаторов. В [55] для решения этой проблемы предлагается архитектура дрона, ориентированная на максимально быструю передачу важных и срочных сообщений в центр управления, в которой для передачи используется только два прыжка (*hope*) [56]. Во время первого прыжка небольшие дроны (аппараты с вращающимся крылом) в кластере передают обнаруженные данные большому беспилотному летательному аппарату с неподвижным крылом по беспроводной восходящей линии связи. Беспилотный летательный аппарат с неподвижным крылом сжимает полученные данные с помощью встроенного сервера граничных вычислений, а затем пересылает результаты в центр управления по каналу связи во время второго перехода. Использование крупногабаритного самолета в качестве воздушной вычислительной платформы является многообещающей альтернативой, особенно в районах, которые не охвачены наземными сетями. Система с такой архитектурой показана на рис. 3.36.

Протоколы маршрутизации для роя беспилотных летательных аппаратов. Протокол маршрутизации играет жизненно важную роль в обеспечении надежной связи между дронами в составе кластера. Однако, из-за особенностей беспилотных летательных аппаратов, таких как высокая скорость, прерывистое воздушное сообщение, разнообразные требования к качеству обслуживания и ограниченная энергия, традиционные протоколы маршрутизации не могут использоваться для связи внутри роя или между роем и наземной станцией. Как указывалось выше, кластер дронов может быть организован как FANET, которая имеет много общего с тради-

ционной специальной сетью. Таким образом, традиционные протоколы маршрутизации могут быть использованы в качестве основы для предоставления таблиц маршрутизации для роя беспилотных летательных аппаратов в FANET.

3.10. Виртуальная реальность

Виртуальная реальность (*VR*) — это созданный техническими средствами мир, передаваемый человеку через его ощущения: зрение, слух, обоняние, осязание и другие. Виртуальная реальность имитирует как воздействие, так и реакции на воздействие.

Дополненная реальность (*AR*) — технология, которая дополняет реальный мир, добавляя любые сенсорные данные. Несмотря на название, эти технологии могут как привносить в реальный мир виртуальные данные, так и устранять из него объекты.

Технологии дополненной и виртуальной реальности используются в образовании и медицине, на их базе разрабатываются обучающие программы и тренажеры. В связи с вышеизложенным актуален вопрос о влиянии, которое эти технологии могут оказать на наш мир. Существуют следующие типы *VR*.

***VR* с эффектом полного погружения.** Этот тип подразумевает наличие трех факторов:

1) правдоподобная симуляция мира с высокой степенью детализации;

2) высокопроизводительный компьютер, способный распознавать действия пользователя и реагировать на них в режиме реального времени;

3) специальное оборудование, соединенное с компьютером, которое обеспечивает эффект погружения в процессе исследования среды.

***VR* без погружения.** К типу «без погружения» относятся симуляции с качественным изображением, звуком и контроллерами, в идеале транслируемые на широкоформатный экран. Также в эту категорию попадают такие проекты, как археологические 3D–реконструкции древних поселений или модели зданий, которые архитекторы создают для демонстрации своей работы клиенту. Эти примеры не отвечают стандартам *VR* в полной мере, но позволяют прочувствовать моделируемый мир на несколько уровней глубже, чем другие средства мультимедиа, а потому причисляются к виртуальной реальности.

***VR* с совместной инфраструктурой.** Сюда можно отнести «виртуальные миры» вроде *Second Life* и *Minecraft*. Единственное свойство, которого им не хватает — создание эффекта присутствия: такие миры не обеспечивают полного погружения. Тем не менее, в виртуальных мирах хорошо прописано взаимодействие с другими пользователями, чего часто не хватает продуктам «настоящей» виртуальной реальности.

VR на базе интернет-технологий. К ним относится прежде всего язык *Virtual Reality Markup Language*, аналогичный *HTML*. Сейчас эта технология считается устаревшей, но не исключено, что в будущем виртуальная реальность будет создаваться в том числе и с использованием интернет-технологий.

Виртуальная реальность в полной мере формируется оборудованием. Без специальных технических устройств ни создание, ни использование виртуальных и компьютерных сред невозможно. На рынке сейчас представлены *VR*-гаджеты всех форм и размеров: от картонных очков до огромных шлемов.

Все *VR*-устройства разделяются на три вида: проводные, беспроводные и мобильные. Все типы обладают общими характеристиками: глубиной погружения, удобством ношения и типом подачи контента.

Глубина погружения складывается из следующих аспектов.

- Позиционный трекинг. Точное определение положения в пространстве, чтобы подгонять под него виртуальные декорации.
- Степень свободы. Возможность перемещения пользователя в пространстве. Устройства для *VR* со степенью свободы *3DoF* дают обзор на 360 градусов. Гаджеты *6DoF*, вдобавок к этому, позволяют смотреть вверх и вниз, перемещаться влево и вправо, вперед и назад.
- Угол обзора. Большинство популярных устройств для *VR* дают угол обзора в 100 градусов, но продвинутые модели уже стремятся к 200.
- Контроллеры. В случае простейших моделей используется одна кнопка для взаимодействия с объектом. Более продвинутые модели имеют много кнопок или трекпад, что расширяет возможности взаимодействия с виртуальной средой.
- Звук. Самые дешевые и простые устройства для *VR* имеют встроенные динамики, более продвинутые — встроенные наушники. Также часто встраивается микрофон, что позволяет общаться с другими пользователями в виртуальном мире.

Удобство ношения. Эта характеристика определяет, насколько комфортным и незаметным для пользователя будет ношение *VR*-устройства в течение длительного времени.

Современные массовые устройства для *VR* поддерживают следующие типы *подачи контента*.

- Интерактивный кинематограф. Виртуальная среда, с которой пользователь может взаимодействовать от первого лица.
- Виртуальный кинотеатр. Пользователь смотрит за действиями персонажей от третьего лица, находясь среди них.
- Виртуальный аквариум. Пользователь может перемещаться и рассматривать виртуальные объекты, но не взаимодействует с ними.
- Виртуальная социальная площадка. Среда, в которой пользователь имеет возможность общаться с другими пользователями.

Шлемы и очки / Head Mounted Display, HMD.

Такие устройства состоят из двух небольших экранов, расположенных напротив каждого глаза, шор, предотвращающих попадание внешнего света, и стереонаушников. Экраны показывают слегка смещенные друг относительно друга стереоскопические изображения, обеспечивая реалистичное 3D-восприятие. В шлемах также содержатся встроенные акселерометры и датчики положения. В большинстве своем продвинутые *VR*-шлемы довольно громоздкие, но в последнее время появилась тенденция к созданию упрощенных легковесных вариантов, которые обычно предназначены для смартфонов с *VR*-приложениями.

Шлемы для виртуальной реальности делятся на три типа. Для компьютера — работают в связке с ПК или консолями: *Oculus Rift*, *HTC Vive*, *Playstation VR*. Эти модели виртуальной реальности оснащены контроллерами движения и лазерными датчиками позиционирования, позволяющими отслеживать положение в пространстве в реальном времени. В данных *VR*-устройствах используются экраны высокого разрешения, обеспечивающие превосходное качество графики.

Для мобильных устройств — называются гарнитурами и работают в связке со смартфонами, представляют из себя держатель с линзами: *Google Cardboard*, *Samsung Gear VR*, *YesVR*. Положительной чертой данного типа устройств является их минимальная цена, максимальная доступность и максимальный охват аудитории. Многие модели изготавливаются из пластика, имеют широкие углы обзора и обладают хорошими линзами с регулировкой фокусного расстояния. Нередко производители в качестве дополнения к ним запускают собственный магазин приложений. Они позволяют смотреть фото и видео, играть в игры, но не способны в полной мере передать все красоты виртуального мира и дают лишь самое общее представление о технологии. Позиционирование производится по трем координатам (поворот вокруг трех осей), навигация в *VR*-пространстве затруднена и чаще всего у пользователей полностью отсутствуют возможности по взаимодействию с объектами в *VR* ввиду отсутствия *VR*-контроллеров.

Независимые очки виртуальной реальности (*Standalone*) — самостоятельные устройства, которые работают под управлением специальных или адаптированных ОС: *Sulon Q*, *DeePoon*, *AuraVisor*. Включают в себя все необходимые технические компоненты в едином корпусе ЦПУ (центрального процессорного устройства): графический процессор, дисплеи и, иногда, стереодинамики. Существенным преимуществом устройств данного типа является их полная автономность, низкая цена и отсутствие проводов в процессе использования. Устройства могут использоваться в любом пространстве, как в офисе, так и дома.

Комнаты / Cave Automatic Virtual Environment, CAVE

Изображения в данном случае транслируются не в шлем, а на стены помещения. Есть мнение, что *VR*-комнаты гораздо лучше *VR*-шлемов: более высокое разрешение, нет необходимости таскать на себе громоздкое устройство, в котором некоторых даже укачивает, и самоидентификация происходит проще благодаря тому, что пользователь имеет возможность постоянно себя видеть.

Информационные перчатки / Datagloves

Для удовлетворения инстинктивной потребности пользователя потрогать руками то, что он находит для себя интересным в процессе изучения среды, были созданы перчатки с сенсорами для захвата движений кистей и пальцев рук. Техническое обеспечение такого процесса варьируется — возможно использование оптоволоконных кабелей, тензометрических или пьезоэлектрических датчиков, а также электромеханических приспособлений (таких как потенциометры).

Джойстики (геймпады) / Wands

Специальные устройства для взаимодействия с виртуальной средой, содержащие встроенные датчики положения и движения, а также кнопки и колеса прокрутки, как у мыши. Сейчас их все чаще делают беспроводными, чтобы избежать неудобств и нагромождений при подсоединении к компьютеру.

В целом, *VR*-технологии могут быть использованы во многих отраслях, которые требуют взаимодействия с виртуальной средой, чтобы улучшить опыт пользователей и оптимизировать бизнес–процессы.

Благодаря *VR*-технологиям люди могут испытать ощущения, которых не могли бы получить в реальной жизни. Например, погрузиться в мир видеоигр или пройти виртуальный тест-драйв автомобиля. Также *VR*-технологии широко используются в медицине и образовании, чтобы симулировать реальные ситуации для обучения и тренировки.

VR-технологии применяются в таких сферах жизни, как образование, спорт, туризм, реклама, медицина, развлечения, архитектура и дизайн, промышленность и добыча полезных ископаемых, автомобилестроение и судостроение, авиация и космическая промышленность

Дополненная реальность — это среда, в реальном времени дополняющая физический мир цифровыми данными с помощью каких-либо устройств — планшетов, смартфонов и т. д. и их программной части. *AR*-системы основаны на трех принципах: объединении цифровой и физической реальности, взаимодействии в реальном времени и точном 3D-разграничении реальных и виртуальных объектов.

Дополненную реальность (*AR*) надо отличать от виртуальной (*VR*) и смешанной (MR). В дополненной реальности виртуальные объекты проецируются на реальное окружение.

Виртуальная реальность — это созданный техническими средствами мир, передаваемый человеку через (пока что) органы чувств.

Смешанная или гибридная реальность объединяет оба подхода.

Виртуальная реальность создает свой мир, куда может погрузиться человек, а дополненная добавляет виртуальные элементы в мир реальный. Выходит, что *VR* взаимодействует лишь с пользователями, а *AR* — со всем внешним миром.

Принципы работы дополненной реальности

Дополненная реальность создает иммерсивные возможности для пользователей. На данный момент функции *AR* реализуются с помощью очков и специальных приложений, но поскольку интерес к этим технологиям растет, на рынке появляется все больше новых видов оборудования. Современные *AR*-системы состоят из пяти основных элементов.

Искусственный интеллект. Для работы большинства решений с дополненной реальностью необходим искусственный интеллект (ИИ). Он позволяет выполнять действия с помощью голосовых команд и обрабатывать информацию в *AR*-приложениях.

Решения для работы с AR. Это инструменты и приложения для доступа к технологиям дополненной реальности, которые некоторые компании создают самостоятельно.

Обработка данных. Для работы *AR*-технологий необходима обработка данных, ресурсы для которой, как правило, предоставляет внутренняя операционная система устройства.

Фотоприложения. Для просмотра материалов и изображений нужны специальные фотоприложения или платформы изображений. Реалистичность объектов зависит от характеристик экрана.

Датчики. Для согласования виртуальных и физических объектов *AR*-системам необходимы данные о среде. Когда камера получает нужную информацию, она отправляет ее в программу для обработки.

Помимо этого, *AR* делится по принципу работы на два типа: маркерную или безмаркерную. Они отличаются способом отображения изображений и информации.

Маркерная *AR* создается с помощью технологии распознавания изображений. Эта технология определяет объекты, которые были предварительно добавлены в базу данных *AR*-устройства или приложения. Когда объекты появляются перед вами, маркеры помогают определять их положение и ориентацию камеры. Переключив режим камеры на градацию серого, вы можете сравнить маркер с другими маркерами в банке информации. Обнаружив соответствие, устройство автоматически определит позицию для размещения *AR*-изображения.

Безмаркерная *AR* — это более сложная технология, поскольку в ней нет якорного элемента. Устройство пользователя должно распознавать все объекты, которые появляются в поле зрения. Используя специ-

альный алгоритм, устройство анализирует цвета, текстуру и схожие черты, чтобы определять, какой объект перед ним находится. Для настройки ориентации используются данные о времени, а также информация, полученная с акселерометра, GPS и компаса. С помощью камеры нужное изображение накладывается на любой физический объект.

Внедрение дополненной реальности в бизнес–процессы помогает сотрудникам быстрее и эффективнее проходить обучение. *AR*-устройства и программы позволяют им приобретать нужные навыки в любое время и в любом месте при наличии нужного программного обеспечения.

Кроме того, специалисты могут использовать эти технологии на выезде, чтобы упрощать совместную работу и повышать безопасность. Этот инновационный метод обучения позволяет усваивать больше информации за меньшее время. Вот несколько примеров сфер, в которых *AR*-технологии пригодятся сотрудникам.

1. Поддержка производительности.
2. Модули для обучения и подготовки.
3. Адаптация новых сотрудников.
4. Записанное обучение.
5. Обслуживание клиентов и взаимодействие с ними.

Ниже приведены отрасли и сферы, в которых используется дополненная реальность.

Торговля. Сотрудники могут использовать *AR*-технологии для адаптации и обучения — сценарии продаж, экскурсия по торговому залу, а также другим помещениям магазина. Клиентам будут полезны возможности дополненной реальности, чтобы протестировать продукты перед покупкой или научиться их использовать в нужной среде. Благодаря практической информации в реальном контексте повышается качество обслуживания клиентов и эффективнее решаются их проблемы.

Производство. Пошаговые инструкции помогают преподавателям предоставлять обратную связь новичкам в процессе обучения. А функции смешанной реальности оставляют руки свободными, чтобы сотрудники могли учиться непосредственно на практике.

Образование. Технология может занять ту нишу, которая в научной фантастике отдана голограммам. Возможность увидеть в вузах и школах виртуальные интерактивные иллюстрации, которые можно рассмотреть со всех сторон, с которыми можно взаимодействовать и тут же видеть результат своих опытов.

Здравоохранение. Для медицинских специалистов важно получать практические знания, не подвергая пациентов риску. Благодаря *AR*-технологиям они могут безопасно применять знания об анатомии и хирургических операциях на практике.

Вооруженные силы. С помощью *AR* вы сможете моделировать ситуации и операции, в которых солдаты научатся ориентировать-

ся во времени и пространстве, а также использовать собственные силы.

Производство автомобилей. Дополненная реальность расширяет возможности для обучения и позволяет специалистам узнавать больше о текущих и новых моделях, а также изучать информацию о внутренних системах.

Компании используют *AR*-технологии не только для отраслевых задач, но и для выявления, отслеживания и устранения неполадок. Они также пригодятся для рекламы, развлечений и организации мероприятий, предоставляя пользователям всю необходимую информацию через их смартфоны.

Дополненная реальность (*AR*) — это не только игры и селфи с виртуальными масками. Это огромный потенциал для коммерческого использования, а также новые перспективы в таких сферах, как образование, промышленность, медицина, строительство, торговля и туризм. И будущее обещает быть еще более захватывающим.

AR демонстрирует впечатляющий коммерческий рост. В отличие от виртуальной реальности, она не требует специализированного оборудования или громоздких устройств. Технология прекрасно функционирует на самом распространенном носимом устройстве — смартфоне.

Следующий этап — широкое распространение *AR* за пределы развлечений и социальных сетей в области информационной поддержки. Автопроизводители начинают выпускать приложения, дополняющие пользовательские инструкции и помогающие владельцам лучше понимать свои автомобили. Все больше компаний разрабатывают приложения для ремонтных мастерских, которые помогают специалистам разобраться в устройстве сложных приборов.

Мы живем в эпоху активных исследований в области *AR*. Даже у крупнейших технологических компаний нет четкого понимания будущего развития этой технологии. Сейчас время для непрерывного появления новых идей, неожиданных способов применения и осознания всех возможностей этой некогда фантастической технологии — дополненной реальности..

Тема 4
ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ

4.1. Парадигма искусственного интеллекта

Искусственный интеллект (ИИ) реализуется в виде интеллектуальной системы (ИС). Интеллектуальная система — это техническая или программная система, способная решать задачи, традиционно считающиеся творческими, принадлежащие конкретной предметной области, знания о которой хранятся в памяти такой системы. Структура интеллектуальной системы включает в себя три основных блока — базу знаний, механизм вывода решений и интеллектуальный интерфейс [57].

Согласно [58] ИС может быть отнесена к классу интеллектуальных, если она обладает следующими свойствами:

— способны накапливать знания об окружающей среде, классифицировать и оценивать их, с точки зрения полезности и непротиворечивости, запускать процессы получения требуемых знаний, соотносить полученные знания с уже имеющимися;

— возможность пополнять поступившие знания с помощью процедур логического вывода, получать обобщенные знания на основе частных знаний, а также планировать свою деятельность.:

— возможность общаться с человеком на языке, максимально приближенным к естественному, в частности, профессиональному языку, учитывать обобщенности личности пользователя;

— возможность формировать для себя или по запросу пользователя объяснение собственной деятельности;

— возможность оказывать пользователю помощь за счет тех знаний, которые хранятся в памяти, и тех средств логического рассуждения, которые имеются в системе.

Очевидно, что перечисленные выше требования — это требования к идеальной системе. Практически все реальные системы обладают только отдельными из перечисленных выше свойств.

Получение знаний. Существуют различные и разнообразные источники знаний, которые могут выступать в качестве источников. К ним в первую очередь относятся эксперты. Важным источниками знаний являются данные, в особенности большие данные. Для получения знаний из перечисленных источников привлекаются

разнообразные методы, алгоритмы и программные инструменты, известные из математики и математической статистики. Исследователями в области ИИ созданы также свои специфические методы, алгоритмы и инструменты для решения задач извлечения знаний. Методы извлечения и структурирования знаний экспертов обычно относят к области инженерии знаний [59]. Для получения знаний из данных кроме стандартных методов математической статистики разработано огромное множество подходов, алгоритмов и инструментов, которые можно разделить на несколько групп, а именно:

— обнаружение связей и зависимостей в данных без акцента на их будущее использование (*Data Mining*);

— машинное обучение (*Machine Learning*), которое реализуется в контексте конкретного приложения;

— эволюционное машинное обучение с подкреплением — класс методов управляемого случайного поиска (*Reinforcement Learning*);

— эвристические методы оптимизации с использованием биоинспирированных метаэвристик, таких как генетические алгоритмы, муравьиные алгоритмы и т. п.

Если нейросетевые модели принятия решений относить к направлению ИИ, то в качестве еще одного класса методов, алгоритмов, технологий и инструментов извлечения знаний из данных следует назвать методы, структуры и технологии глубокого обучения (*Deep Learning*).

Однако главный тренд в этой области — это извлечение семантики знаний и технологии ее практического использования в рамках концепции семантических моделей знаний и семантических вычислений. Если говорить совсем кратко, то семантические модели, семантические технологии — это достаточно общие термины, применяемые к программному обеспечению, которое имеет некоторый, иногда даже совсем минимальный уровень «человеческого» понимания смысла информации, с которой программа имеет дело. По аналогии с семантикой естественного языка в ИИ термин понимание неявно трактуется как аналог понимания смысла информации, данных и знаний человеком. Можно сказать, что переход к использованию семантических моделей означает, что текст, представляющий, например, данные, становится не только машиночитаемым, но также и машинопонимаемым.

Представление и преобразование знаний и данных. Концептуальная модель знаний не может быть использована компьютером напрямую, поскольку машинный язык — это язык бинарных последовательностей. Описание знаний концептуального уровня на некотором формальном языке, которое может быть далее преобразовано в машиночитаемую форму — это и есть задача представления знаний. Другими словами, под представлением знаний понимается их спецификация в терминах конкретных структур, представленных

на формальном языке, тексты которого могут быть транслированы автоматически в машиночитаемую форму. Несколько упрощая, можно сказать, что задача данного этапа — это преобразование знаний из формального представления, понятного человеку, в форму, понятную компьютеру. На этом этапе знания и данные, описанные, например, текстами на естественных языках или представленные в другой форме, понятной человеку, преобразуются в машиночитаемую форму. В современной концепции семантических вычислений эта форма должна быть также машинопонимаемой. Сейчас ее принято называть онтологией.

Эта модель в настоящее время имеет два основных варианта реализации.

Первый вариант предполагает использовать для представления знаний и данных в хранилищах *RDF*-триплетов с моделью онтологии на метауровне и ее описанием на одном из языков группы *RDF*, *RDF-Schema* и *OWL* [60, 61]. Второй вариант представления — это структуры типа графов разметки свойств (*Labelled Property Graph*, *LPG*). Более подробно этот вопрос будет рассмотрен в следующем параграфе.

Использование знаний. Разработка технологий работы со знаниями не является самоцелью. Эти технологии должны использоваться для разработки парадигм построения ИС, информационных технологий и инструментальных средств их программной поддержки, например, мультиагентных систем, технологии семантического веба, технологии умных вещей, технологии КФС и т. д.

Эти технологии обычно соотносятся с некоторыми классами приложений. В частности, указанные в Национальной стратегии (НС) ИИ [62] технологии искусственного интеллекта, включающие компьютерное зрение, обработку естественного языка, распознавание и синтез речи, интеллектуальную поддержку принятия решений и перспективные методы искусственного интеллекта, относятся к использованию знаний. Можно выделить следующие основные виды ИИС:

— экспертные системы;
— советующие системы;
— информационные системы (базы данных и базы знаний);
— поисковые системы;
— системы виртуальной реальности;
— системы обработки естественного языка;
— системы интеллектуального анализа данных (бизнес-аналитики);
— системы распознавания, анализа и синтеза речи;
— системы распознавания, анализа и синтеза изображений;
— системы поддержки принятия решений;
— интеллектуальные КФС;

— многоагентные интеллектуальные системы;
— интеллектуальные системы проектирования;
— интеллектуальные системы реинжиниринга;
— обучающие системы;
— системы машинного творчества (игры, доказательство теорем);
— прочие.

Следует заметить, что это далеко не полный список, причем сфера применения ИИС постоянно расширяется.

4.2. Эволюция искусственного интеллекта

Современная информационная система — это набор информационных технологий, направленных на поддержку жизненного цикла информации и включающего три основных процесса: обработку данных, управление информацией и управление знаниями. В условиях резкого увеличения объемов информации переход к работе со знаниями на основе искусственного интеллекта является, по всей вероятности, единственной альтернативой информационного общества.

Воспользуемся определением «интеллектуальной системы» профессора Д. А. Поспелова [58]: «Система называется интеллектуальной, если в ней реализованы следующие основные функции, позволяющие:

- накапливать знания об окружающем систему мире, классифицировать их и оценивать их с точки зрения прагматической полезности и непротиворечивости, инициировать процессы получения новых знаний, осуществлять соотнесение новых знаний с ранее хранимыми;
- пополнять поступившие знания с помощью логического вывода, отражающего закономерности в окружающем систему мире или в накопленных ею ранее знаниях, получать обобщенные (знания на основе более частных знаниях) и логически планировать свою деятельность;
- общаться с человеком на языке, максимально приближенном к естественному человеческому языку, и получать информацию от каналов, аналогичных тем, которые использует человек при восприятии окружающего мира, уметь формировать для себя или по просьбе человека (пользователя) объяснение собственной деятельности, оказывать пользователю помощь за счет тех знаний, которые хранятся в памяти, и тех логических средств рассуждений, которые присущи системе».

Перечисленные функции можно назвать функциями представления и обработки знаний, рассуждения и общения. Наряду с обяза-

тельными компонентами, в зависимости от решаемых задач и области применения в конкретной системе эти функции могут быть реализованы в различной степени, что определяет индивидуальность архитектуры. На рис. 4.1 в наиболее общем виде представлена архитектура интеллектуальной системы (ИнС) в виде совокупности блоков и связей между ними [16].

База знаний представляет собой совокупность сред, хранящих знания различных типов. Рассмотрим кратко их назначение.

База фактов (данных) хранит конкретные данные, а *база правил* — элементарные выражения, называемые в теории искусственного интеллекта продукциями.

База процедур содержит прикладные программы, с помощью которых выполняются все необходимые преобразования и вычисления над данными. База закономерностей включает различные сведения, относящиеся к особенностям той среды, в которой действует система.

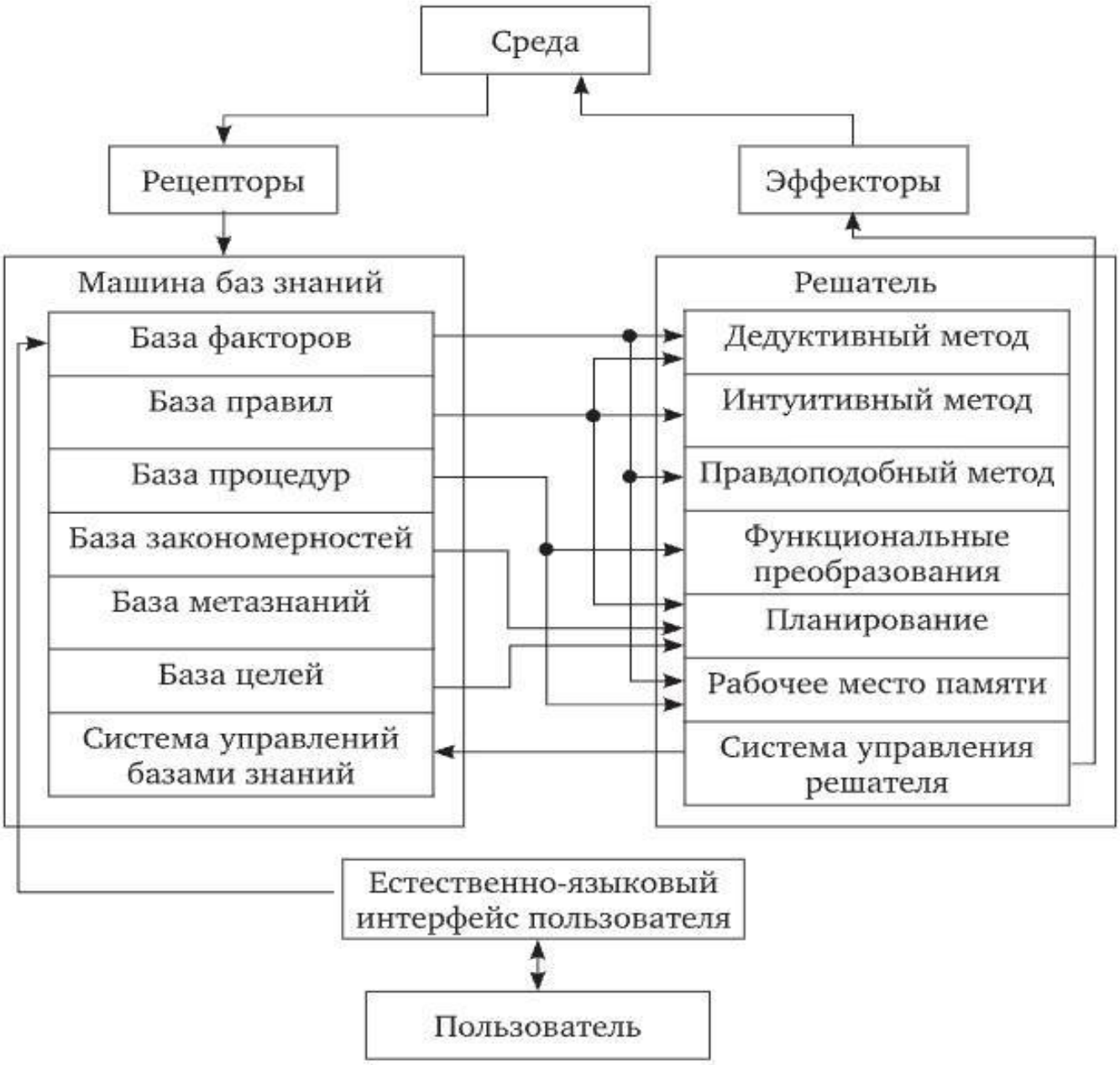


Рис. 4.1. **Общая структура интеллектуальной системы**

База метазнаний (база знаний о себе) содержит описание самой системы и способов ее функционирования: сведения о том, как внутри системы представляются единицы информации различного типа, как взаимодействуют различные компоненты системы, как было получено решение задачи.

База целей содержит целевые структуры, называемые сценариями, позволяющие организовать процессы движения от исходных фактов, правил, процедур к достижению той цели, которая поступила в систему от пользователя, либо была сформулирована самой системой в процессе ее деятельности в проблемной среде.

Управление всеми базами, входящими в базу знаний, и организацию их взаимодействия осуществляет система управления базами знаний. С ее же помощью реализуются связи баз знаний с внешней средой. Таким образом, машина базы знаний осуществляет первую функцию интеллектуальной системы.

Выполнение второй функции обеспечивает часть интеллектуальной системы, называемой решателем и состоящая из ряда блоков, управляемых системой управления решателя. Часть из блоков реализует логический вывод. Блок дедуктивного вывода осуществляет в решателе дедуктивные рассуждения, с помощью которых из закономерностей из базы знаний, фактов из базы фактов и правил из базы правил выводятся новые факты.

Кроме этого, данный блок реализует эвристические процедуры поиска решений задач, как поиск путей решения задачи по сценариям при заданной конечной цели. Для реализации рассуждений, которые не носят дедуктивный характер, т. е. поиск по аналогии, по прецеденту и пр., используются блоки индуктивного и правдоподобного выводов. Блок планирования используется в задачах планирования решений совместно с блоком дедуктивного вывода. Назначение блока функциональных преобразований состоит в решении задач расчетно-логического и алгоритмического типа.

Третья функция — функция общения — реализуется как с помощью компоненты естественно–языкового интерфейса, так и с помощью рецепторов и эффекторов, которые осуществляют так называемое невербальное общение и используются в интеллектуальных роботах.

Важное место в теории искусственного интеллекта (ИИ) занимает проблема представления знаний. В настоящее время выделяют следующие основные типы моделей представления знаний:

1) семантические сети, в том числе функциональные семантические сети;
2) фреймы и сети фреймов;
3) продукционные модели.

Семантические сети определяют как граф общего вида, в котором можно выделить множество вершин и ребер. Каждая вершина графа представляет некоторое понятие, а дуга — отношение между

парой понятий. Метка и направление дуги конкретизируют семантику. Метки вершин семантической нагрузки не несут, а используются как справочная информация.

Различные разновидности семантических сетей обладают различной семантической мощностью, следовательно, можно описать одну и ту же предметную область более компактно или громоздко.

Фреймом называют структуру данных для представления и описания стереотипных объектов, событий или ситуаций. Фреймовая модель представления знаний состоит из двух частей:

— набор фреймов, составляющих библиотеку внутри представляемых знаний;

— механизмы их преобразования, связывания и т. д.

Существует два типа фреймов:

— образец (прототип) — интенсиональное описание некоторого множества экземпляров;

— экземпляр (пример) — экстенсиональное представление фрейм- образца.

В общем виде фрейм может быть представлен следующим кортежем:

$$< \text{ИФ}, (\text{ИС}, \text{ЗС}, \text{ПП}), \ldots, (\text{ИС}, \text{ЗС}, \text{ПП}) >,$$

где ИФ — имя фрейма; ИС — имя слота; ЗС — значение слота; ПП — имя присоединенной процедуры (необязательный параметр).

Слоты — это некоторые незаполненные подструктуры фрейма, заполнение которых приводит к тому, что данный фрейм ставится в соответствие некоторой ситуации, явлению или объекту.

В качестве данных фрейм может содержать обращения к процедурам (так называемые присоединенные процедуры). Выделяют два вида процедур: процедуры-демоны и процедуры-слуги. Процедуры-демоны активизируются при каждой попытке добавления или удаления данных из слота. Процедуры-слуги активизируются только при выполнении условий, определенных пользователем при создании фрейма.

Продукционные модели — это набор правил вида «условия — действие», где условиями являются утверждения о содержимом базы данных, а действия представляют собой процедуры, которые могут изменять содержимое базы данных.

Формально продукция определяется следующим образом:

$$(i); Q; P; C; A \wedge B; N,$$

где (i) — имя продукции (правила); Q — сфера применения правила; P — предусловие (например, приоритетность); C — предикат (отношение); $A \wedge B$ — ядро; N — постусловия (изменения, вносимые в систему правил).

Практически продукции строятся по схеме «*если*» (причина или иначе посылка), «*то*» (следствие или иначе цель правила).

Полученные в результате срабатывания продукций новые знания могут использоваться в следующих целях:

- понимание и интерпретация фактов и правил с использованием продукций, фреймов, семантических цепей;
- решение задач с помощью моделирования;
- идентификация источника данных, причин несовпадений новых знаний со старыми, получение метазнаний;
- составление вопросов к системе;
- усвоение новых знаний, устранение противоречий, систематизация избыточных данных.

Процесс рассмотрения компьютером набора правил (выполнение программы) называют консультацией. Ее наиболее удобная для пользователя форма — дружественный диалог с компьютером. Интерфейс может быть в форме меню, на языке команд и на естественном языке.

Диалог может быть построен на системе вопросов, задаваемых пользователем, компьютером, или фактов — данных, хранящихся в базе данных. Возможен смешанный вариант, когда в базе данных недостаточно фактов.

При прямом поиске пользователь может задавать две группы вопросов, на которые компьютер дает объяснения.

1. Как получено решение? При этом компьютер должен выдать на экран трассу в виде ссылок на использованные правила;
2. Почему компьютер задал какой-то вопрос, при этом на экран выдается своеобразная трасса, которую компьютер хотел бы использовать для вывода после получения ответа на задаваемый вопрос? Вопрос «почему» может быть задан как в процессе консультации, так и после выполнения программы.

Специфичен алгоритм поиска, реализуемый логическими языками: он является фактически последовательным перебором по дереву сверху-вниз-слева-направо.

Сравнительные характеристики методов описания знаний представлены в табл. 4.1 [16, 63].

Из приведенных структур методов описания знаний нетрудно заметить, что построить табл. 4.1, позволяющую наглядно увидеть правила и обнаружить ошибки в их формировании, удобнее всего для продукций. К тому же продукции легко трансформируются при изменении правил. Перечисленные факторы определили более широкое использование продукций.

Помимо описания знаний важное значение имеет теория логического вывода. Выделяют следующие методы логики вывода:

1) традиционная логика;
2) алгебра логики (булева алгебра);
3) исчисление предикатов.

Таблица 4.1

Сравнительные характеристики метода описания знаний

Метод	Достоинства	Недостатки
Семантические сети	Возможность представления смысла фраз и ограничений. Наличие внутренней структуры. Определение операций над объектами (естественный язык)	Нет средств зависимости от времени. Произвольность структуры. Сложность сочетания с методами логики
Фреймы	Наличие внутренней структуры и связности	Жесткость структуры. Трудности построения модулей и модификации. Сложность сочетания с методами логики
Продукции	Модульность. Простота модификации правил. Отделение предметных правил от управления. Простота сочетания с методами логики	Отсутствие внутренней структуры. Зависимость шагов вывода от принятой стратегии вывода

Традиционная логика возникла еще во времена Аристотеля и была предназначена для уменьшения количества логических ошибок в различного рода диспутах и спорах. Отправной точкой этой логики служат следующие понятия.

Алгебра логики (булева алгебра) оперирует с высказываниями. Чаще всего используют одно $y = f(x)$ и двуместные $z = f(x, y)$ высказывания.

Предикаты первого порядка. Пусть имеется n исходных множеств X, что запишем как X_n.

Если осуществляется преобразование: $X_1 \hat{} X$, то говорят об алгебре.

Если осуществляется преобразование: $X_n \hat{} \{0, 1\}$, то говорят об отношениях.

Исчисление — совокупность правил оперирования с каким-либо символом (предикатами).

Предикат (сказуемое) — отношение между множествами элементов, соответствующих аргументов. Например:

$$Q(a_1, a_2, \ldots, a_n),$$

где a_n — аргументы; Q — предикат.

Предикаты первого порядка «наследуют» кванторы от традиционной логики и строгую логику алгебры логики. Предикат является предикатом первого порядка, если квантор не входит в сам предикат. Предикаты первого порядка охватывают большую часть отношений. Доказано, что в ряде частных случаев предикаты более высокого порядка могут быть сведены к предикатам первого порядка.

Хотя предикаты первого порядка не позволяют описывать возможность и необходимость, убеждения, намерения, цели, метазнания, они охватывают широкий спектр возможностей и получили широкое распространение в построении экспертных систем (ЭС).

Для любой системы выделяют процесс проектирования и эксплуатации. Процесс проектирования по своему назначению является интеллектуальным и включает формирование цели, технологию и инструментарий. В проектировании присутствует много ручных операций, хотя в последнее время все шире используются системы автоматизированного проектирования.

Когда говорят об интеллектуальных системах, то имеют в виду интеллектуальность, проявляющуюся в процессе эксплуатации системы. Общая схема ИнС представлена на рис. 4.2.

Блок «интерфейс пользователя» предназначен для связи компьютера с пользователем, для которого предпочтительным языком «разговора» является естественный язык или близкий к нему.

Следует отметить специфическое понятие «естественный язык», под которым понимается фактически искусственный язык, полученный на этапе концептуализации из естественного человеческого языка (русского, английского или любого другого) путем удаления неоднозначностей (синонимов, омонимов, идиоматических выражений). Следует заметить, что такой язык в теории автоматизированных систем носит название «искусственный язык» или «информационный язык».

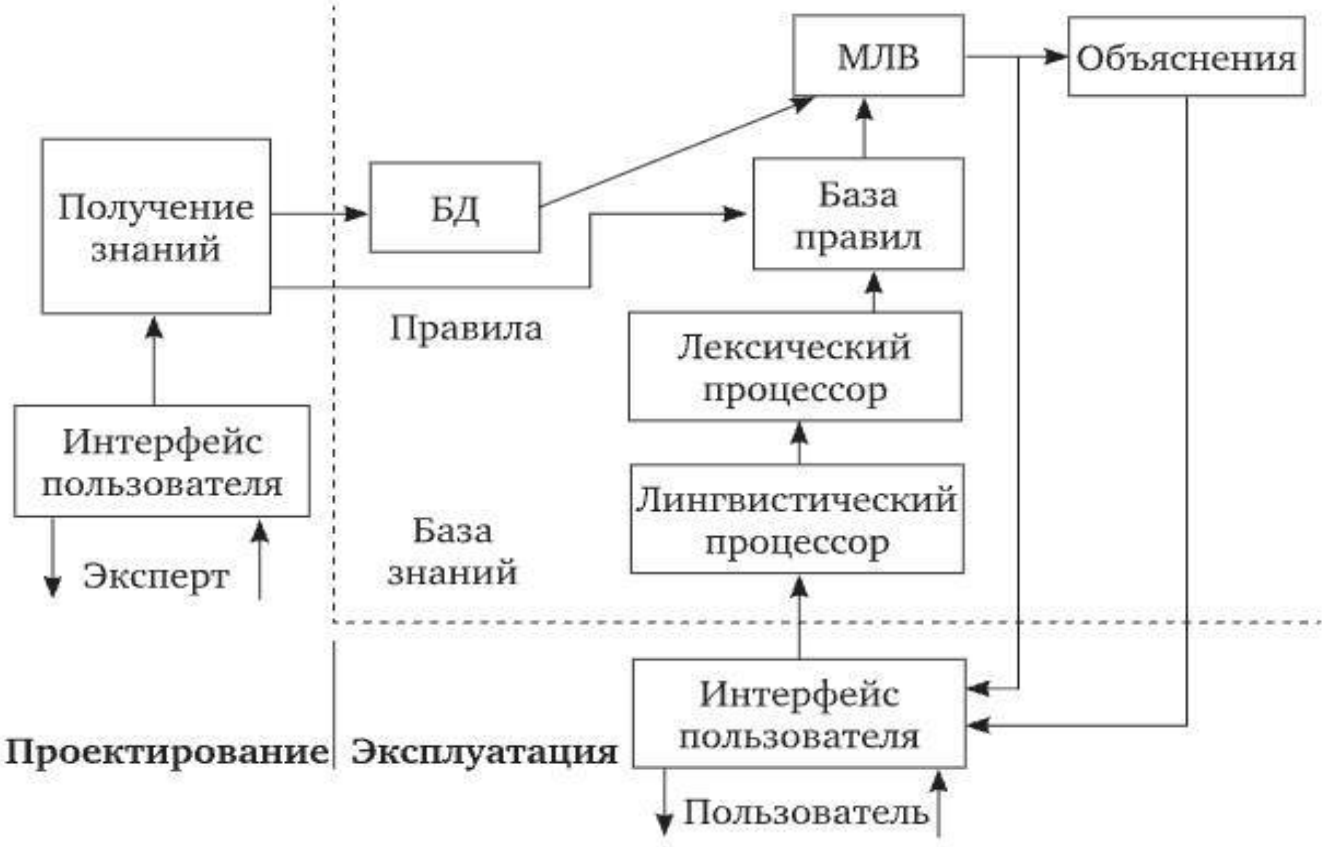


Рис. 4.2. **Схема интеллектуальной системы: МЛВ — машина логического вывода**

Основу ИС составляют блоки «база данных (БД)», «база правил (БП)», «машина логического вывода (МЛВ)», «объяснения». БД хранит исходные данные. В базе правил фиксируются знания и опыт эксперта. МЛВ выводит результат, взаимодействуя с БД и БП. Объяснения выводят систему правил, использованных для полученного конкретного результата, на естественном языке.

Все три «компьютерных» блока должны быть описаны математически. Правила записываются через интерфейс эксперта (см. рис. 4.2) в виде сложных правильно построенных формул (далее — ППФ) при посредничестве инженера по знаниям. Форма, в которой записываются ППФ, определяется синтаксисом, а истинностное значение алгоритмов логического вывода семантикой или смыслом.

По запросу пользователя компьютер с помощью блока «объяснения» может дать ответ на вопрос «как» (с помощью набора каких правил) получен результат и «почему» компьютер задает пользователю уточняющие вопросы, связанные, как правило, с данными.

Для стыковки интерфейса пользователя, работающего на естественном языке, с перечисленными блоками используют лингвистический и лексический процессоры.

Лингвистический процессор — непосредственно работает с естественным языком, преобразуя результат в «машинный вид».

Лексический процессор — словарный состав информационного языка.

Иногда включают и синтаксический процессор, при этом под синтаксисом понимают правила сочетания слов внутри предложения и построения предложений.

В блоке «Получение знаний» выделяют два понятия:

- если информация поступает из книг (документов), то говорят о выявлении знаний;
- если она получается на основе работы эксперта, то говорят об извлечении знаний.

Преобразование полученной информации инженером по знаниям называется получением знаний.

Сложность здесь заключается в том, что эксперт может не владеть языками программирования, в то время как инженер по знаниям может недостаточно ориентироваться в данной предметной области.

При этом в наиболее развитых моделях выделяют интенсиональную и экстенсиональную компоненты.

Экстенсиональная — различные сведения о предметной области (данные).

Интенсиональная — схемы связей между атрибутами, отражающие основные закономерности системы в предметной области решаемых функциональных задач. Для разработчиков моделей данные — это схема БД. Для представителей ИИ — это знания о проблемной области.

Понятие «интеллект» многогранно и потому существует значительное количество разновидностей интеллектуальных систем, которые целесообразно классифицировать.

Классификация — разделение множества на подмножества по принятому классификационному признаку.

Из множеств классификаций интеллектуальных систем воспользуемся классификацией, показанной на рис. 4.3.

Рис. 4.3. **Классификация интеллектуальных систем:**
ЭС — экспертные системы; ИНС — искусственные нейронные сети; РЛС — расчетно-логические системы; СГА — системы с генетическими алгоритмами; МАС — многоагентные системы; СЕЯ — системы на естественном языке; ИСУ — интеллектуальные системы управления; ЭСРВ — экспертные системы реального времени

Системы на естественном языке (СЕЯ) специфичны и предназначены преимущественно для таких целей, как машинный перевод, генерация документов, автоматическое аннотирование и реферирование.

Экспертные системы (ЭС) предполагают высокую степень формализации процессов на этапе концептуализации.

Разновидностью экспертных систем можно считать расчетно-логические системы (РЛС), оперирующими с функциями вместо правил.

Искусственные нейронные сети (ИНС) представляют собой фактически разновидность систем автоматического управления, использующие свойства нейрона.

Системы с генетическими алгоритмами (СГА) относятся к разновидности эволюционных эвристических методов.

Многоагентные системы (МАС) преследуют цель согласования теорий баз данных и баз знаний.

Интеллектуальные системы управления (ИСУ) используют режим адаптации к изменяющимся параметрам и целям эксплуатации (функционирования) систем. По характеру математического описания интеллектуальные системы возможно разделить на *непрерывные* и *дискретные* (рис. 4.4).

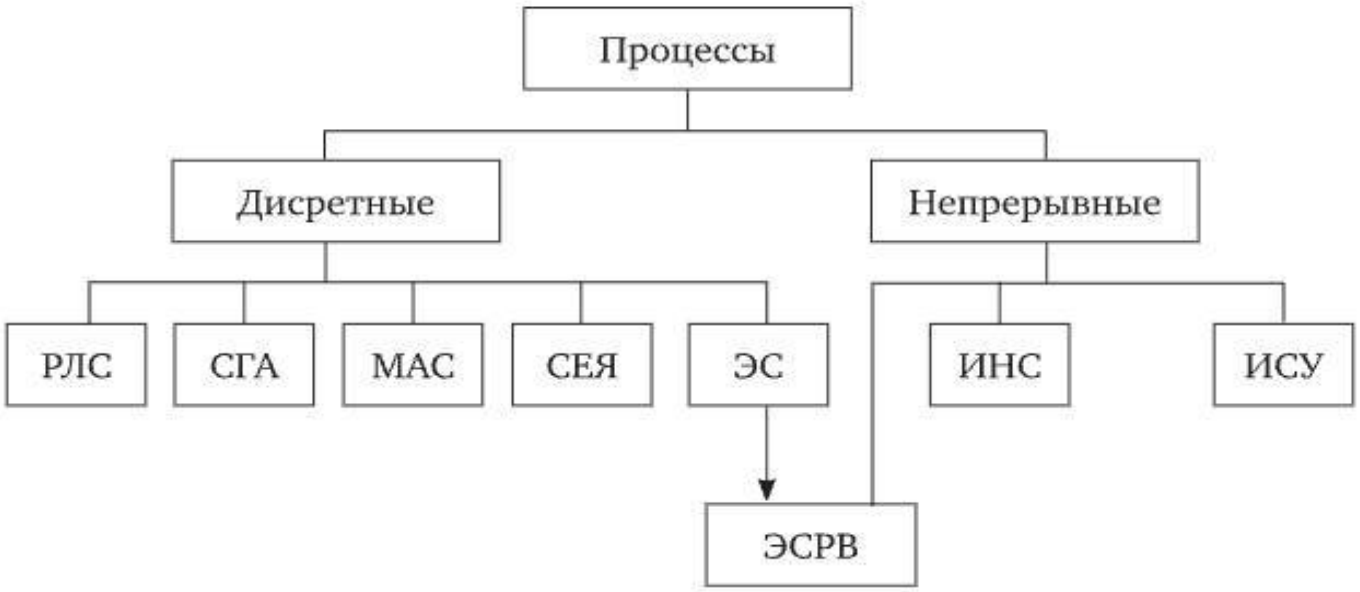


Рис. 4.4. **Методы математического описания**

Непрерывные процессы имеют место в ИнС, ИСУ, РЛС и описываются дифференциальными уравнениями с соответствующими критериями, функциональными (формульными) зависимостями. В этом случае используются преимущественно числовые данные, хотя они могут быть искусственно организованы в виде такой, например, структуры, как базы данных.

Интеллектуальные процессы человека чаще связаны с дискретными данными и знаниями, опирающимися на информационные языки.

В связи с этим ИНС оперируют с дискретными величинами (классы ЭС, СГА, МАС, СЕЯ). Однако и в этих классах не существует однородного математического описания. Здесь используются такие методы, как логическое описание, нечеткие множества, функциональные семантические сети, контекстно-зависимые грамматики.

К тому же имелась все большая необходимость связи дискретных и непрерывных величин, что привело к появлению гибридных моделей. Сюда относятся прежде всего экспертные системы реального времени (ЭСРВ), в которых имеется непрерывная составляющая, описываемая дифференциальными или разностными уравнениями.

Экспертная система (статическая) — вычислительная система, использующая знания эксперта и процедуры логического вывода и позволяющая дать объяснения полученным результатам. Синонимом ЭС является термин «система, базирующаяся на знаниях». *Знания* — совокупность данных и связывающих их правил.

На начальных этапах развития рассматривались статические ЭС. В таких системах время напрямую не фигурировало.

Изначально предполагалось, что схема ЭС будет иметь вид, показанный на рис. 4.2. Однако выяснилось, особенно после появления оболочки экспертной системы *GURU*, что построение «переводчика» с естественного языка на язык компьютера является сложной,

вполне самостоятельной проблемой. Появление декларативных логических языков типа «*Пролог*» (*Prolog*), а затем — языков, использующих Пролог–идеи, позволило упростить задачу «перевода» при условии использования текстовых элементов из заранее составленного естественного языка после выполнения этапа концептуализации. При этом структура интеллектуальной системы трансформировалась — исчезли блоки лингвистических и лексических процессоров (рис. 4.5).

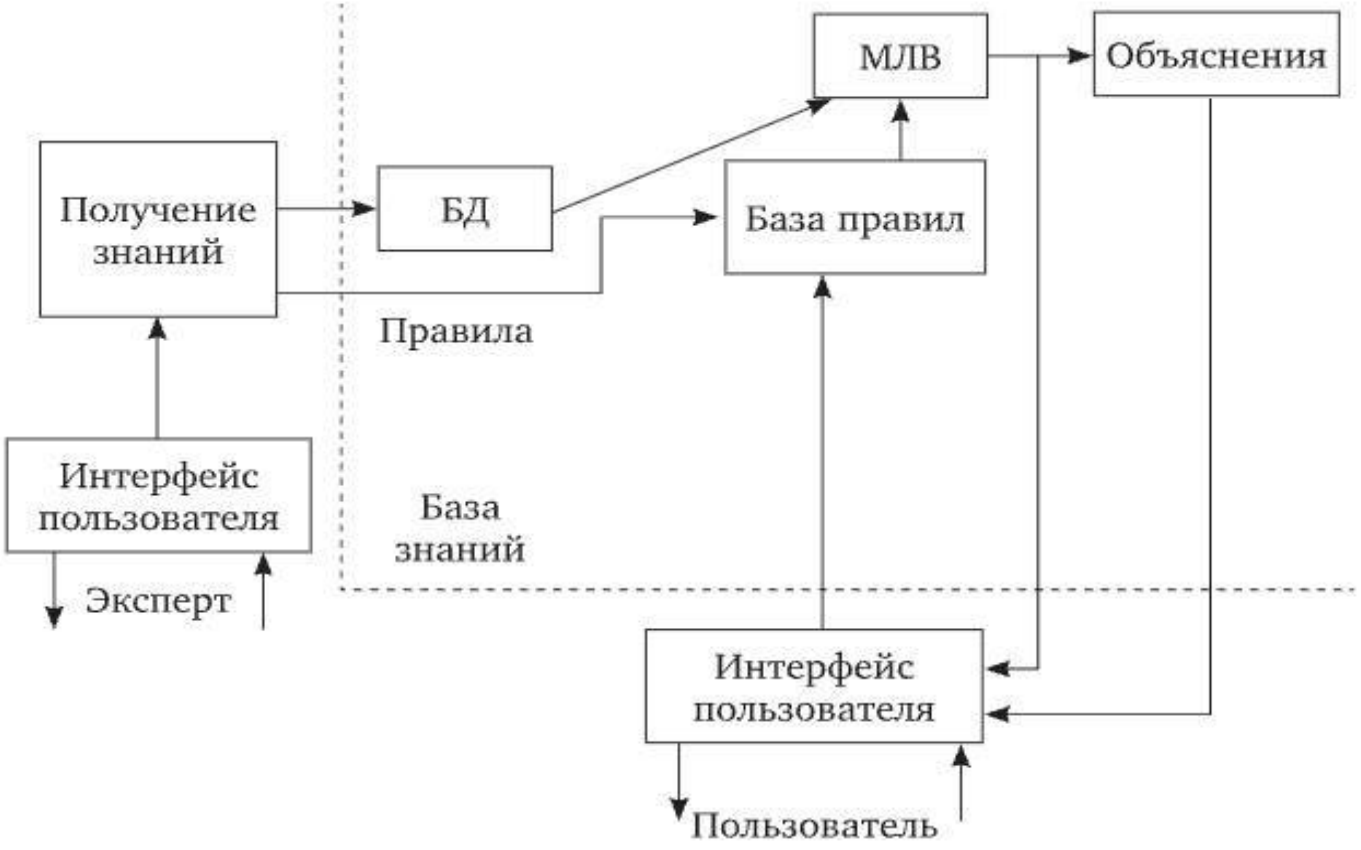


Рис. 4.5. **Схема экспертной системы**

Статическая ЭС без блока «базы правил» называется оболочкой ЭС (аналог СУБД). Наиболее известна оболочка *GURU*. В рамках языка Пролог также имеется оболочка ЭС.

В то же время системы управления имеют, как правило, замкнутую структуру.

Очень скоро выяснилось, что статические системы имеют ограниченную сферу применения.

Статические ЭС не позволяют учесть, сохранить и провести анализ изменяющихся во времени данных, поступающих от внешних источников; работать с асинхронными процессами; обеспечить механизм рассуждений при ограниченных ресурсах; обеспечить прогноз последствий принимаемых решений–советов; обеспечить создание и поддержку интерфейсов пользователей, уровень их защиты для различных категорий пользователей.

Реализация ЭС в области управления называется ЭС реального времени (ЭСРВ).

Следует отметить, что первоначально ЭСРВ работала с моделью объекта управления (см. рис. 4.5). В настоящее время в ЭСРВ включают реальный объект управления, информация о состоянии которого поставляется системой датчиков и отражается на табло.

Искусственные нейронные сети. Начиная с 1990-х гг. работы велись в плане поиска методов обучения и построения адаптивных систем. На определенном этапе становления искусственные нейронные сети (ИНС) развивались в противовес ЭС реального времени с целью устранения следующих недостатков:

1) сложное сочетание непрерывных и дискретных процессов;
2) сложность учета нелинейного характера процесса управления;
3) недостаточная адаптация ЭС реального времени к различного рода неопределенностям (параметров структуры).

В дальнейшем выяснилось, что сами ИНС имеют достаточно сложную структуру. Их применение: для малоразмерных систем управления проблематично.

Основой ИНС является базовый процессорный элемент, который построен как имитация естественному биологическому нейрону.

Системы с генетическими алгоритмами появились в процессе поиска новых методов математических расчетов, поскольку возможности как детерминированных, так и статистических методов оказались недостаточными для описания вновь появляющихся классов систем.

В методах эффективного поиска выделяются три группы.

1. Методы, основанные на математических вычислениях, — это все виды программирования. Серьезным ограничением здесь являются требования дифференцируемости функции. Если это требование не соблюдено, то используются специальные методы программирования. Сложным в общем случае может оказаться многоэкстремальный характер кривой выбора решения.

2. Перечислительные или переборные методы (целочисленное, динамическое программирование) характеризуются высокой размерностью задач, большим объемом вычислений.

3. Методы, использующие элемент случайности (случайный поиск), не дают гарантии оптимальности решений и получения более хорошего результата, чем в предыдущих методах.

В целом ряде случаев нужно не оптимальное решение, а более хорошее решение. Тогда оказалось полезным сочетать детерминированную составляющую алгоритмов со случайной составляющей. Это направление получило название *эволюционные вычисления*.

Методы эволюционных вычислений возможно классифицировать следующим образом:

- эволюционное программирование базируется на теории конечных автоматов, которые должны реагировать на изменения внешней среды;

- эволюционные стратегии по своей сути похожи на метод генетических алгоритмов (далее — ГА), однако целевая функция может меняться от поколения к поколению (основной признак изменения — мутация).

Общая цель названных методов — адаптация к изменяющимся условиям. В то же время общая цель генетических алгоритмов — описание процесса развития популяций.

В генетических алгоритмах используются приемы из генетики. Процесс заменяется некоторым кодом. Используется специфическая терминология.

Код и его структура называются генотипом. Конкретная интерпретация кода, т. е. переход от задачи к коду, называется *фенотипом*. Набор конкретных кодов — *популяцией*. Индивидуальный код называют *хромосомой* (особью, индивидом). Каждый шаг в алгоритме называют *поколением*. Особи предыдущего поколения — *родители*, последующего поколения — *потомки*.

Генетический алгоритм (ГА) является вариантом решения в виде битовой строки фиксированной длины, манипулирование с которой производится в отсутствии связей с ее смысловой интерпретацией.

Теория *мультиагентных* систем только начала складываться. Показанные на рис. 4.6 предметные области развивались практически автономно.

Область систем искусственного интеллекта (с позиции принятия решений) сильно ограничена. При этом рассматриваются дискретные процессы. В то же время появились адаптивные непрерывные системы.

Адаптация возможна к изменению параметров объекта управления (самонастраивающаяся система) к изменению структуры ОУ (самонастраивающаяся система).

Возникла задача интеграции предметных областей, характеризующихся кривой 1 на рис. 4.6. Напрямую решить такую задачу оказалось сложно. Поэтому пока изучают интеграцию предметных областей, ограниченных кривой 2.

Рис. 4.6. **Соотношение предметных областей**

Считается, что эта интеграция является началом решения первого понимания термина «искусственный интеллект». Одним из таких методов интеграции является метод мулътиагентных (многоагентных) систем (МАС) [12].

В мультиагентных системах используют модальную логику — логику достоверности. Достоверность может иметь, как отмечалось три разновидности:

- аподиктическая (наибольшая достоверность) — целое больше части;
- ассерторическая (утвердительная логика) — например, яблоко разделено на части;
- проблематичная (фактор уверенности, нечетные переменные).

Термин «агент» происходит от латинского глагола *agere*, что означает «действовать», «двигать», «править», «управлять».

С компьютерной точки зрения *агент* — вычислительный процесс, который реализует автономную коммуникационную функциональность приложений. С точки зрения систем искусственного интеллекта, целесообразно иметь интеллектуального агента (ИА).

Отталкиваясь от определений данных, будем считать, что ИА — это программный или аппаратный объект (сущность), автономно функционирующий для достижения целей, поставленных перед ним владельцем или пользователем, и обладающий определенными интеллектуальными способностями. Уровень этих способностей, необходимых для достижения поставленных перед ИА целей, можно определить, пользуясь классификацией Д. А. Поспелова.

Вид математического аппарата, позволяющего описать поведение агента в соответствующей среде, и является мерой его интеллектуальной сложности (или разумности).

Представим эту классификацию в виде табл. 4.2.

Две базовые характеристики — автономность и целенаправленность — позволяют отличать ИА от других программных и аппаратных объектов (модулей, подпрограмм, процедур). Наличие целесообразности поведения требует, чтобы ИА обладал свойством реактивности.

Такой уровень интеллекта соответствует рефлекторному поведению животного. Если же ИА обладает знаниями о среде, собственных целях и способах их достижения, то такой агент может быть назван разумным. Следовательно, может быть проведена граница между интеллектуальными и неинтеллектуальными агентами.

Значительные усилия по стандартизации агентных систем и технологий предпринимает организация *Foundation for Intelligent Physical Agents* (далее — FIPA), которая является международной организацией, выполняющей работу по разработке FIPA открытых спецификаций, поддерживающих интероперабельность агентов и агентных приложений.

Таблица 4.2

Мера интеллектуальной разумности

Тип среды функционирования	Метод математического описания	Вид ИА
Замкнутая детерминированная	Автоматные грамматики, конечные автоматы	Автоматные агенты
Замкнутая вероятностная	Вероятностные автоматы	Вероятностные автоматные агенты
Не трансформируемая открытая	Контекстно-свободные грамматики (КСГ), сценарии, магазинные автоматы	КСГ-агенты
Трансформируемая замкнутая	Контекстно-зависимые грамматики (линейные автоматы)	КЗГ-агенты
Трансформируемая открытая	Семиотические системы	Семиотические агенты

МАС может рассматриваться как сильно связанная сеть решателей, совместно работающих над проблемами, которые могут выходить за рамки возможностей индивидуальных агентов.

Исходя из изложенного, МАС — совокупность взаимосвязанных агентов как программных, так и аппаратных, способных взаимодействовать друг с другом и окружающей средой, обладающих определенными интеллектуальными способностями и возможностью индивидуальных и совместных действий.

Интеллектуальными МАС будем называть МАС, обладающие всеми признаками ИНС в соответствии с определением К. А. Пупкова.

Открытые МАС могут содержать переменные множества агентов, входящих в систему и выходящих из нее, причем возможно своекорыстное (столкновение интересов) поведение агентов.

Понятие «агентно–ориентированная система» (далее — АОС) представляется более широким, чем понятие «МАС». M. Wooldridge характеризует АОС как систему, в которой ключевой используемой абстракцией является агент.

АОС трактуем как гибридные системы, содержащие, наряду с МАС и другие системы (ЭС, обучающие и тестирующие системы, распределенные объектные приложения).

АОС в настоящее время являются глобально распределенными, взаимосвязанными совокупностями программных и аппаратных систем, содержащими сотни и тысячи компонентов, что и определяет возрастающую сложность их разработки, внедрения и эксплуатации.

Системы на естественном языке (СЕЯ) реализуют процедуры поиска, автоматического аннотирования и реферирования. В свою

очередь, в поисковой системе следует выделить ручной режим, характерный для библиотек, и автоматизированный режим для электронных документов. Правда, в современных библиотеках все шире используется автоматизированный поиск.

Понятие интеллектуальной системы управления (ИСУ) связано с современными адаптивными автоматизированными системами управления. Адаптивные системы отличаются от неадаптивных тем, что они в процессе функционирования получают дополнительную информацию о поведении среды или влиянии среды на параметры, структуру и цели системы и компенсируют это влияние.

4.3. Нейрокомпьютинг

Искусственные нейронные сети (ИНС) являются одним из наиболее быстро развивающихся направлений в области ИС. ИНС находят в таких областях как информатика, экономика, биология, биофизика, математика. В настоящее время существует много примеров, демонстрирующих возможности ИНС, а именно: превращение текста в фонетическое представление, распознавание рукописного текста, сжатие изображения и т. д.

ИНС представляют собой отдельный класс интеллектуальных систем. Применительно к ИНС под знаниями понимается хранимая информация или модели, используемые компьютером или человеком для интерпретации, предсказания или реакции на некоторые события во внешнем мире.

Имеются достаточно много определений понятия ИНС. Чаще всего ИНС определяют, как *совокупность моделей биологических нейронных сетей, которые представляют собой сеть элементов — искусственных нейронов, связанных между собой синаптическими соединениями*.

ИНС строятся из большого числа относительно простых нелинейных вычислительных элементов, называемых нейронами, которые организованы в виде сетей. Работа ИНС состоит в преобразовании входных сигналов, в результате чего меняется внутреннее состояние сети и формируются выходные воздействия. Обычно ИНС оперирует числовыми, а не символьными величинами.

С помощью ИНС делаются попытки имитировать организацию нервной системы биологических существ на уровни: память, обработка сенсорной информации, моторика.

Большинство моделей ИНС требуют обучения. Обучение можно определить как задачу многомерной оптимизации, для решения которой можно использовать множество разных алгоритмов.

Модель нейрона. Нейрон представляет собой элемент, из которых строятся ИНС. Структура нейрона показана на рис. 4.7. Нейрон

включает в себя три основные составляющие: множество синапсов (связей), каждому из которых ставится в соответствие вес, сумматор, который складывает входные сигналы, взвешенные относительно соответствующих синапсов и функцию активации, которая ограничивает амплитуду входного сигнала нейрона. Кроме того, имеется пороговый элемент.

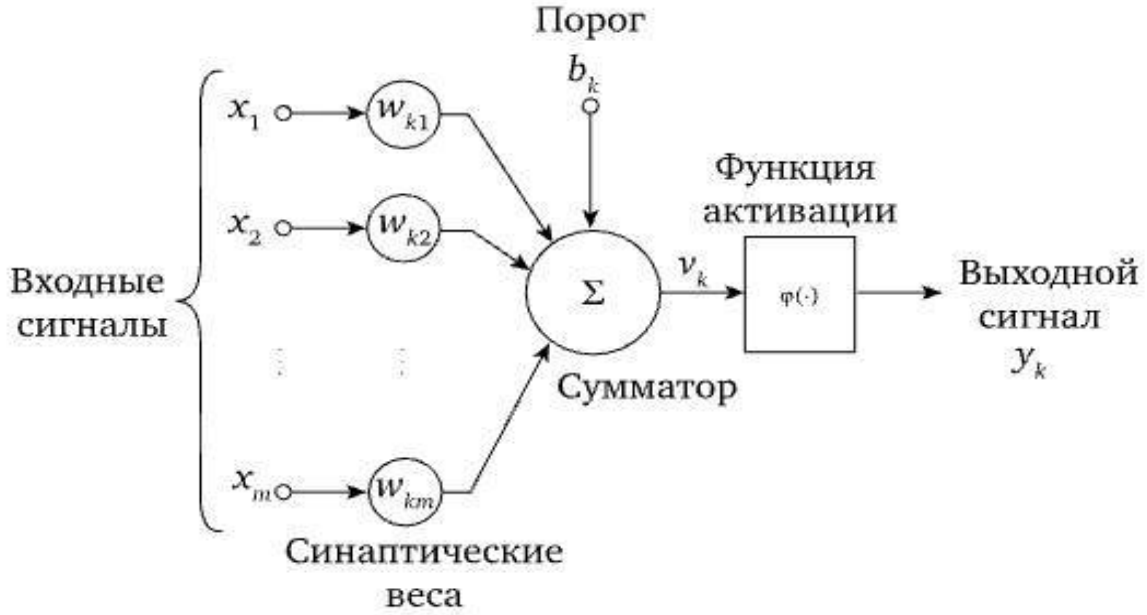


Рис. 4.7. **Структура нейрона**

Сигнал x_j, поступающий на вход j нейрона k умножается на вес w_{kj}. Вес может быть как положительным, так и отрицательным. Обычно диапазон выхода нейрона лежит в пределах [0,1] или [-1,1]. Порог задается величиной b_k. Это величина, на которую увеличивается или уменьшается входной сигнал, поступающий на вход функции активации.

Текущее состояние нейрона определяется, как взвешенная сумма его входов:

$$S = \sum_{i=1}^{n} x_i \cdot w_i + b, y = f(s).$$

Выход нейрона есть функция его состояния b — значение смещения, w_i — вес синапса, x_i — компонент входного сигнала X.

Будучи соединенными определенным образом, нейроны образуют сеть, состоящую из простых адаптивных элементов, которая взаимодействует с объектами реального мира в некотором смысле аналогично биологической нервной системе.

С инженерной точки зрения такая сеть представляет собой сильно распараллеленную динамическую систему с топологией направленного графа, которая может выполнять переработку информации посредством изменения своего состояния в ответ на постоянный или импульсный входной сигнал.

В зависимости от функций, выполняемых нейронами в сети, можно выделить три их типа:

— входные нейроны, на которые подается вектор, кодирующий входное воздействие или образ внешней среды; в них обычно не осуществляется вычислительных процедур, а информация передается с входа на выход путем изменения их активации;

— выходные нейроны, выходные значения которых представляют выходы нейронной сети;

— промежуточные нейроны, составляющие основу нейронных сетей.

Архитектура ИНС. Архитектура ИНС во многом определяется используемым алгоритмом обучения. В общем случае можно выделить три основных класса ИНС.

Однослойные сети прямого распространения (рис. 4.8). В ИНС нейроны располагаются по слоям. В простейшем случае в ИНС имеется входной слой (*input layer*) узлов источника, информация от которого поступает на выходной слой (*output layer*), на котором и производятся вычисления. Подобные сети называются однослойными сетями прямого распространения (*feed-forward*).

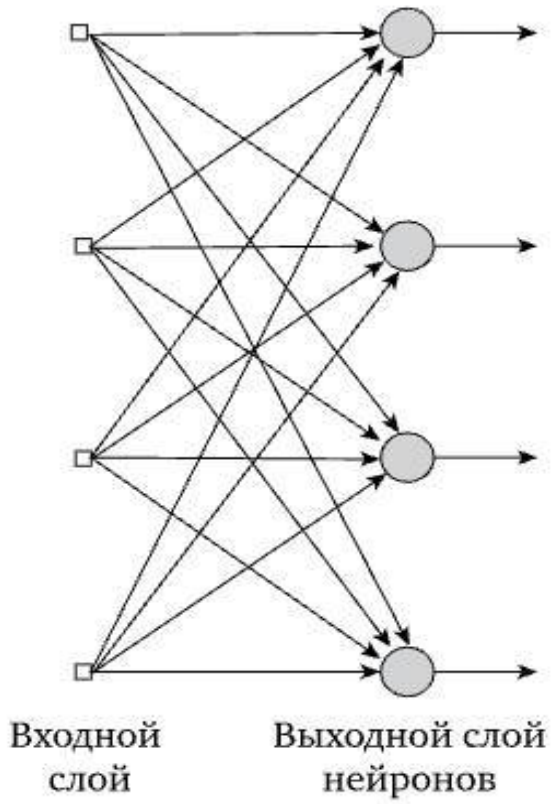


Рис. 4.8. **Однослойные сети прямого распространения**

Многослойные сети прямого распространения. Более сложными сетями прямого распространения являются многослойные сети прямого распространения, которые характеризуются наличием одного или нескольких скрытых слоев (*hidden layers*), наличие которых позволяет выделять глобальные свойства данных [64]. Это особенно важно, когда размер входного слоя большой.

ИНС считается полносвязанной (*fully connected*), если все узлы одного слоя связаны со всеми узлами следующего слоя. Например, ИНС по рис. 4.9 является полносвязанной. Если некоторые связи отсутствуют, то ИНС называется неполносвязанной (patyionally connected).

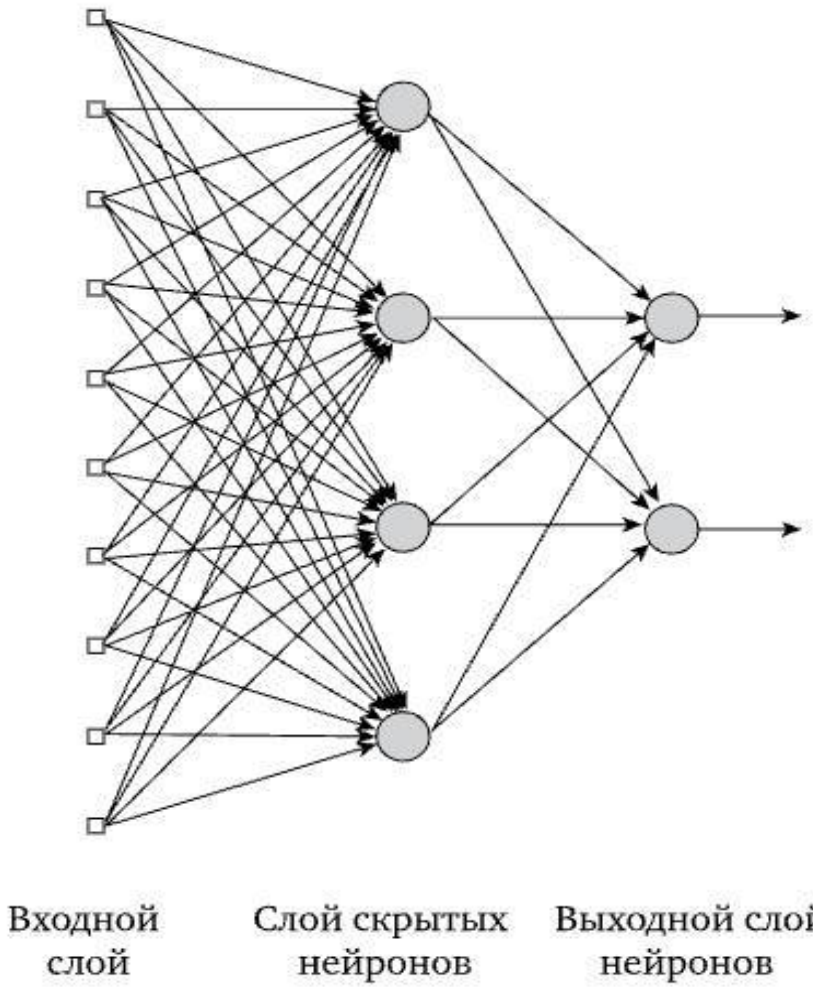


Рис. 4.9. **Многослойные сети прямого распространения**

Рекуррентные ИНС. Рекуррентные сети (recurrent networks) в отличие от ИНС прямого распространения имеют обратные связи, число которых не ограничено.

На рис. 4.10 в качестве примера показана ИНС, которая имеет один скрытый слой нейронов. В данном случае обратные связи исходят как из выходных, так и скрытых нейронов.

Обратная связь предполагает использование элементов задержки (*unity delay element*), которое на рис. 4.10 обозначено как $z - 1$, что приводит к нелинейному динамическому поведению. Наличие обратных связей оказывает существенное влияние на их способность к обучению.

Применительно к ИНС под знаниями понимается хранимая информация или модели, используемые компьютером или человеком для интерпретации, предсказания или реакции на некоторые события во внешнем мире.

ИНС представляют собой отдельный класс интеллектуальных систем, разработка приемлемых нейросетевых решений во многом яв-

ляется творческим процессом. При этом на первый план выступает задача ее наилучшего обучения.

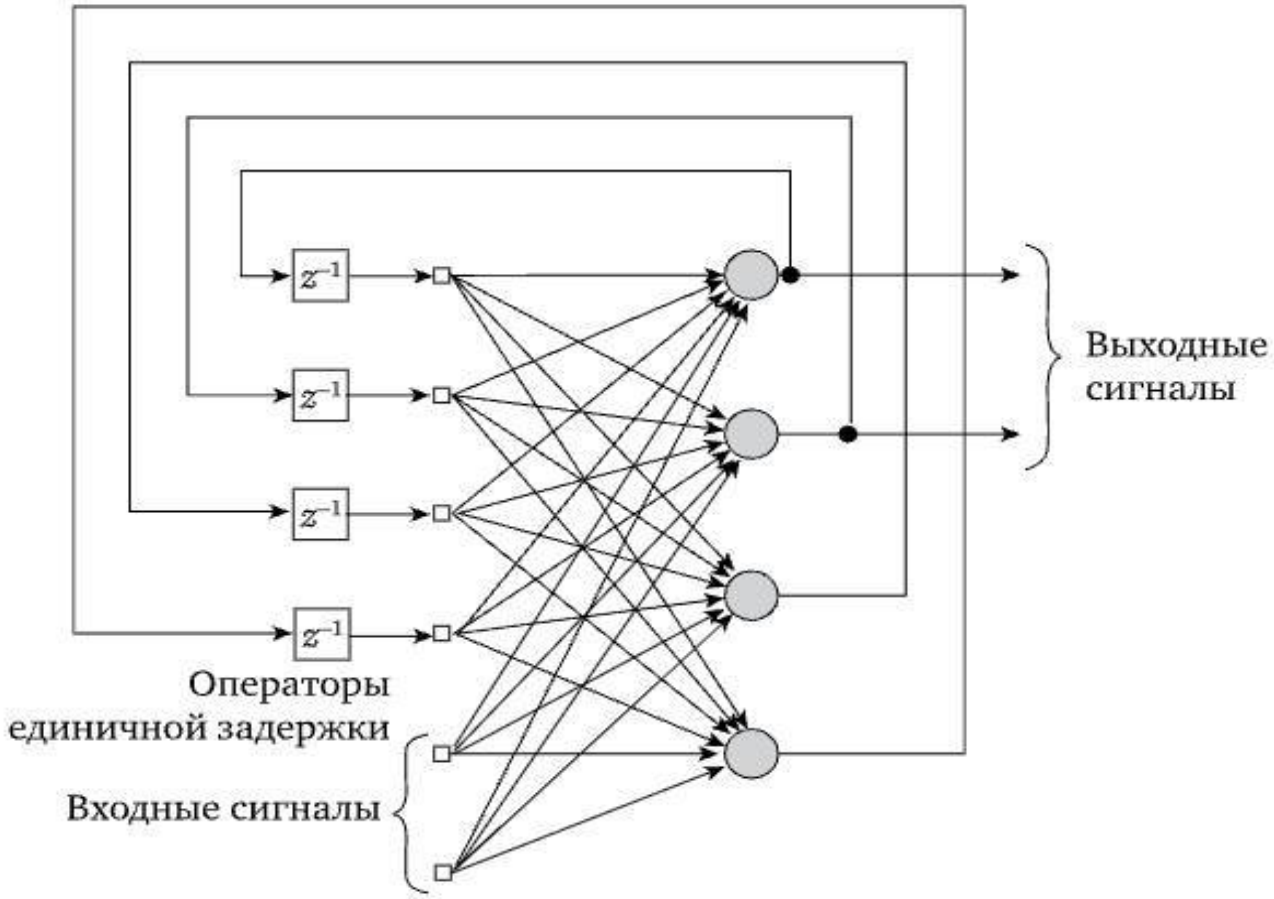


Рис. 4.10. **Рекуррентные ИНС**

Следует отметить, что разработка приемлемых нейросетевых решений во многом является творческим процессом. При этом на первый план выступает задача ее наилучшего обучения.

Процесс функционирования ИНС зависит от весов синаптических связей, поэтому, задавшись определенной структурой ИНС, отвечающей какой-либо задаче, разработчик сети должен найти оптимальные значения всех переменных весовых коэффициентов. Данный этап называется обучением ИНС, и от того, насколько качественно он будет выполнен, зависит способность сети решать поставленные перед ней задачи во время эксплуатации. На этапе обучения кроме параметра качества подбора весов важную роль играет также время обучения. Как правило, эти два параметра связаны обратной зависимостью и их приходится выбирать на основе компромисса.

Под обучением понимается целенаправленное изменение (подстройка) весовых коэффициентов синаптических связей из условий достижения требуемых характеристик сети, т. е. желаемая реакция на входные воздействия.

Существует много различных алгоритмов обучения, которые можно разделить на два больших класса: детерминистские и стохастические. В первом случае подстройка весов представляет собой

жесткую последовательность действий, во втором — она производится на основе действий, подчиняющихся некоторому случайному процессу.

По характеру обучения нейронные сети делят на нейронные сети, использующие обучение с учителем и нейронные сети, использующие обучение без учителя.

Обучение с учителем. Обучение с учителем предполагает, что для каждого входного вектора существует целевой вектор, представляющий собой требуемый выход. Вместе они называются обучающей парой. Обычно сеть обучается на некотором числе таких обучающих пар. Предъявляется выходной вектор, вычисляется выход сети и сравнивается с соответствующим целевым вектором. Далее, веса изменяются в соответствии с алгоритмом, стремящимся минимизировать ошибку. Векторы обучающего множества предъявляются последовательно, вычисляются ошибки и веса подстраиваются для каждого вектора до тех пор, пока ошибка по всему обучающему массиву не достигнет приемлемого уровня.

Таким образом, при обучении с учителем ИНС предъявляются значения как входных, так и выходных параметров, и она по некоторому внутреннему алгоритму подстраивает веса своих синаптических связей. Обучение с учителем предполагает, что для каждого входного вектора существует целевой вектор, представляющий собой требуемый выход.

Для обучения с учителем известно достаточно много разных подходов. В качестве классических можно назвать следующие: коррекция по ошибке, машина Больцмана, правило Хебба и обучение методом соревнования.

Коррекция по ошибке. Для каждого входного примера задан желаемый выход, однако реальный выход у сети может не совпадать с желаемым. Принцип коррекции по ошибке при обучении состоит в использовании разностного сигнала для модификации весов, обеспечивающего постепенное уменьшение ошибки. Такое обучение производится только в случае, когда ИНС ошибается. Следует заметить, что известно много разных вариантов реализации этого подхода.

Обучение Больцмана. Использует стохастическое правило обучения, которое следует из принципов теории информации и термодинамических принципов. Целью обучения в этом случае является такая настройка весовых коэффициентов, при которой состояния нейронов внешнего слоя удовлетворяют желаемому распределению вероятностей.

Обучение Больцмана часто рассматривают как частный случай коррекции по ошибке, в котором под ошибкой понимается расхождение корреляций состояний в двух режимах.

Правило Хебба. Наиболее старым обучающим правилом является постулат обучения Хебба. Этот подход опирается на нейрофи-

зиологические наблюдения. Если нейроны с обеих сторон синапса активизируются одновременно и регулярно, то сила синаптической связи возрастает. Спецификой этого подхода является то, что изменение синаптического веса в этом случае зависит только от активности нейронов, которые связаны данным синапсом.

Метод соревнования. Этот метод отличается от метода Хебба, тем, что при соревновательном обучении выходные нейроны соревнуются между собой за активизацию. Этот подход основывается на правиле, в соответствии с которым «победитель берет все». По утверждениям биологов, именно такое обучение имеет место в биологических нейронных сетях. При обучении методом соревнования изменяются только веса «победившего» нейрона. Эффект от этого правила достигается за счет такого изменения сохраненного в сети образца (вектора весов связей «победившего» нейрона), при котором он становится ближе ко входному примеру.

Обучение без учителя. Обучение без учителя является намного более правдоподобной моделью обучения с точки зрения биологических корней ИНС. В этом случае отсутствует необходимость в целевом векторе для выходов и, следовательно, не требуется сравнение с предопределенными идеальными ответами. Обучающее множество состоит лишь из входных векторов. Обучающий алгоритм подстраивает веса ИНС так, чтобы получались согласованные выходные векторы, т. е. чтобы предъявление достаточно близких входных векторов давало одинаковые выходы. Процесс обучения, следовательно, выделяет статистические свойства обучающего множества и группирует сходные векторы в классы.

Таким образом, при обучении без учителя ИНС предъявляются только входные сигналы, а выходы сети формируются самостоятельно с учетом только входных и производных от них сигналов. В процессе обучения выделяются статистические свойства обучающего множества и группируются сходные векторы в классы. Предъявление на вход вектора из данного класса даст определенный выходной вектор, но до обучения невозможно предсказать, какой выход будет производиться данным классом входных векторов.

Настройка весов. В зависимости от вида сети настройка весов производится двумя способами. В первом случае весовые коэффициенты нейронной сети выбираются сразу, исходя из условий задачи. Во втором случае в процессе обучения происходит настройка синаптических весов.

В любом случае целью обучения является минимизация вероятности ошибки между желаемым выходом сети и фактической реакцией ИНС.

Переобучение ИНС. При обучении ИНС достаточно часто возникает неприятная ситуация, называемая проблемой переобучения (*overfitting*).

Переобучение можно определить как излишне точное соответствие ИНС конкретному набору обучающих примеров, при котором ИНС теряет способность к обобщению. Переобучение возникает в случае слишком долгого обучения, недостаточного числа обучающих примеров или переусложненной структуры нейронной сети.

Переобучение обычно связано с тем, что выбор обучающего (тренировочного) множества является случайным. С первых шагов обучения происходит уменьшение ошибки. На последующих шагах с целью уменьшения ошибки (целевой функции) параметры подстраиваются под особенности обучающего множества. Однако при этом происходит «подстройка» не под общие закономерности ряда, а под особенности его части — обучающего подмножества. При этом точность прогноза уменьшается.

В качестве примера использования ИНС рассмотрим задачу распознавания цифр [65,66], для которой входной сигнал, соответствующий изображению цифры, представляет собой матрицы черно–белых точек. Каждое из изображений представляет собой одну из десяти рукописных цифр. На выходе требуется получить цифру. Обучающая выборка должна состоять из большого числа рукописных цифр, совокупность которых должна отображать ситуацию в реальном мире. Выборка должна содержать не только положительные, но и отрицательные примеры. Для рассматриваемого случая ИНС будет иметь число входов равное числу пикселей в изображении, а число выходов равное десяти. Внутренняя структура ИНС может быть разной. Сначала ИНС обучают, т. е. настраивают на имеющихся примерах, затем проверяют на реальных задачах. Если нейросеть неправильно распознает какие-то начертания цифр, то ее дообучают.

К важнейшим свойствам ИНС можно отнести:

1. *Параллельность обработки информации.* Каждый нейрон формирует свой выход только на основе своих входов и собственного внутреннего состояния под воздействием общих механизмов регуляции нервной системы.

2. *Способность к полной обработке информации.* Все известные человеку задачи решаются нейронными сетями. К этой группе свойств относятся ассоциативность (сеть может восстанавливать полный образ по его части), способность к классификации, обобщению, абстрагированию и множество других. Свойства до конца не систематизированы.

3. *Самоорганизация.* В процессе работы биологические нейросети самостоятельно, под воздействием внешней среды обучаются решению разнообразных задач. Не известно никаких принципиальных ограничений на сложность задач, решаемых биологическими нейронными сетями. Нервная система сама формирует алгоритмы своей деятельности, уточняя и усложняя их в течение жизни. Чело-

век пока не сумел создать искусственных систем, обладающих способностью к самоорганизации и самоусложнению.

4. *Аналоговость.* Биологические нейросети являются аналоговыми системами. Информация поступает в сеть по большому количеству каналов и кодируется по пространственному принципу: вид информации определяется номером нервного волокна, по которому она передается. Амплитуда входного воздействия кодируется плотностью нервных импульсов, передаваемых по волокну.

5. *Надежность.* Биологические нейросети обладают высокой надежностью: выход из строя даже 10 % нейронов в нервной системе не прерывает ее работы. Последовательные ЭВМ, основанные на принципах фон Неймана, выходят из строя, если происходит сбой ячейки памяти или одного узла в аппаратуре.

Современные ИНС обладают следующими свойствами:

1. *Обучаемость.* Выбрав одну из моделей нейросети, создав сеть и выполнив алгоритм обучения, возможно обучить сеть решению некоторой задачи.

2. *Способность к обобщению.* После обучения сеть становится нечувствительной к малым изменениям входных сигналов (шуму или вариациям входных образов) и дает правильный результат на выходе.

3. *Способность к абстрагированию.* Если предъявить нейросети несколько искаженных вариантов входного образа, то она сама сможет создать на выходе идеальный образ, с которым никогда не встречалась.

4. *Параллельность обработки и реализуемость.* Параллельность в работе нейросетей и ассоциативность при обработке информации приводит к тому, что разница в производительности между обычной ЭВМ и мозгом составляет 4–6 порядков.

Во многом этот выигрыш обусловлен параллельностью обработки информации в мозге. Следовательно, для повышения производительности ЭВМ необходимо перейти от принципов фон Неймана к параллельной обработке информации.

Несмотря на перспективность ИНС для их создания нет собственной элементной базы. Поэтому, вместо моделирования ИНС на параллельных машинах, большая часть исследований проводится двумя способами:

1. Моделирование ИНС на обычных последовательных ЭВМ.

2. Создание специализированных нейроплат и нейропроцессоров для ускорения работы ЭВМ с нейронными сетями.

Область применения ИНС. ИНС является средством обработки информации и может рассматриваться как:

— гибкая модель для нелинейной аппроксимации многомерных функций;

— средство прогнозирования во времени процессов, зависящих от многих переменных;

— классификатор по многим признакам, дающий разбиение входного пространства на области;

— средство распознавания образов;

— инструмент для поиска и классификации по ассоциациям;

— модель для поиска закономерностей в массивах данных.

Уникальное свойство нейросетей — универсальность. Для всех вышеуказанных задач существуют эффективные математические методы решения. И хотя нейросети проигрывают специализированным методам при решении конкретных задач, благодаря универсальности, к примеру, построении систем искусственного интеллекта и моделировании процессов мышления, они являются важным направлением исследования, требующим дальнейшего развития.

Самую подробную информацию по ИНС можно найти в учебниках [64,67].

4.4. Машинное обучение

Термин машинное обучение охватывает большой круг технологий, которые преследуют одну цель: извлечь имеющую смысл информацию из данных. Под данными здесь подразумевается все, что может быть записано и измерено (курс акций в удачные дни, масса планет в Солнечной системе, рост и вес людей, различные звуки (слова, произносимые кем-нибудь в его мобильный телефон), изображения (к примеру, фотографии кошек или других животных), тексты газетной статьи или романа), или все, что мы хотели бы изучить или исследовать. «Имеющая смысл информация» — это все, что мы можем извлечь из данных и что будет полезным для нас в некотором смысле. Мы решаем, что имеет смысл для нас и затем создаем алгоритм, чтобы найти настолько много смысла из наших данных, насколько это возможно.

Машинное обучение заключается в извлечении знаний из данных. Это научная область, находящаяся на пересечении статистики, искусственного интеллекта и компьютерных наук, также известная как прогнозная аналитика или статистическое обучение. В последние годы применение методов машинного обучения в повседневной жизни стало обыденным явлением. Многие современные веб–сайты и устройства используют алгоритмы машинного обучения, начиная с автоматических рекомендаций по просмотру фильмов, заказа еды или покупки продуктов, и заканчивая персонализированными онлайн–радиотрансляциями и распознаванием друзей на фотографиях. Когда вы видите сложный сайт типа *Amazon* или *Netflix*, то весьма вероятно, что каждый раздел сайта содержит несколько моделей машинного обучения.

На заре появления «интеллектуальных» приложений многие системы использовали жесткие правила «*if*» и «*else*» для обработки данных или корректировки информации, введенной пользовате-

лем. Вспомните о спам–фильтре, чья работа состоит в том, чтобы переместить соответствующие входящие сообщения электронной почты в папку «Спам». Вы можете составить черный список слов, которые будут идентифицировать письмо как спам. Это пример использования системы экспертных правил для разработки «интеллектуального» приложения. Разработка правил принятия решений в ручном режиме допустима в некоторых задачах, особенно в тех, где люди четко понимают процесс моделирования.

Однако, использование жестких решающих правил имеет два основных недостатка:

- логика, необходимая для принятия решения, относится исключительно к одной конкретной области и задаче. Даже несущественное изменение задачи может повлечь за собой переписывание всей системы.
- разработка правил требует глубокого понимания процесса принятия решения.

Один из примеров, где этот жесткий подход потерпит неудачу — это распознавание лиц на изображениях. На сегодняшний день каждый смартфон может распознать лицо на изображении. Тем не менее, распознавание лиц было нерешенной проблемой, по крайней мере, до 2001 года. Основная проблема заключается в том, что способ, с помощью которого компьютер «воспринимает» пиксели, формирующие изображение на компьютере, очень сильно отличается от человеческого восприятия лица. Эта разница в принципе не позволяет человеку сформулировать подходящий набор правил, описывающих лицо с точки зрения цифрового изображения.

Главная идея, лежащая в основе машинного обучения, — можно представить действительность, используя математические функции, которые алгоритму заранее неизвестны, но которые он может предположить на основании наблюдения некоторых данных. Вы можете выразить действительность и всю ее комплексную сложность в терминах неизвестных математических функций, которые алгоритмы машинного обучения способны выявить и сделать доступными. Эта концепция лежит в основе идей всех видов алгоритмов машинного обучения.

Обучение при машинном обучении является просто математической функцией, и она заканчивается ассоциацией определенных входных данных с определенными выходными. Она не имеет отношения к пониманию того, что алгоритм изучил (анализ данных вырабатывает понимание до некоторой степени), поэтому процесс обучения зачастую называют тренировкой, поскольку алгоритм учится находить правильный ответ (вывод) на каждый предоставленный вопрос (ввод).

Многие люди привыкли к идее, что приложения начинают работу с получения данных на входе, а затем предоставляют некий результат. Например, программист мог бы создать функцию *add( )*, которая получает как ввод два значения, например, 1 и 2, а затем возвращает

результат 3. Выводом этого процесса является значение. В прошлом написание программы означало понимание того, как функция должна манипулировать данными, чтобы получить заданный результат при определенных входных данных. Машинное обучение осуществляет переворот в этом процессе. В данном случае вы знаете входные данные, это 1 и 2. Вы также знаете, что желаемый результат — 3. Однако вы не знаете, какую функцию применить, чтобы получить желаемый результат. Обучение предоставляет алгоритм со всякого рода примерами входных данных и ожидаемыми результатами для этих входных данных. Затем алгоритм использует заданный ввод, чтобы создать функцию. Другими словами, обучение — это процесс, в ходе которого обучаемый алгоритм сопоставляет с данными гибкую функцию. Выводом обычно является вероятность определенного класса или числового значения. Задачи определения функции при заданном входе и выходе, а также определение входа выходным данным и функции являются некорректными задачами.

Корректно поставленная задача в математике — прикладная задача, математическое решение которой существует, единственно и устойчиво. Задача или алгоритм решения задачи называются вычислительно неустойчивыми, если малые изменения входных данных приводят к заметным изменениям решения.

При этом надо иметь в виду, что задача машинного обучения изначально не является корректно поставленной задачей.

Успешное решение задачи машинного обучения на 50 % состоит из знания «стандартных» методов и умения их применить и на 50 % из смекалки, умения правильно поставить задачу, внимательного анализа исходных данных и выделения признаков.

Выйдя за пределы коммерческих приложений, машинное обучение уже оказало огромное влияние на научные исследования, управляемые данными. Инструменты, представленные в этой книге, использовались для решения различных научных задач (исследование звезд, поиск далеких планет, открытие новых частиц, анализ последовательностей ДНК, а также разработка персонализированных методов лечения рака).

Дадим некоторые определения машинного обучения (единственное и всеобъемлющее определение практически дать невозможно, так как в зависимости от предметной области где оно используется возникает масса нюансов и дополнительных факторов, которые могут отсутствовать в других областях). Если использовать аналогию, то в теории систем существует до 300 определений понятия система, но ни одно из них не дает достаточно полного и не противоречивого определения «система». Также и с машинным обучением.

Для анализа огромных наборов данных машинное обучение полагается на алгоритмы. В настоящее время машинное обучение неспособно обеспечить такой вид искусственного интеллекта, как это

показывают в кино. Даже наилучшие алгоритмы не могут думать, чувствовать, обладать любой формой самосознания, морали, этики или иметь свободу выбора. На что машинное обучение способно, так это выполнять прогнозирующую аналитику намного быстрее, чем любой человек. В результате машинное обучение может помочь людям работать эффективнее. Да, искусственный интеллект в текущем состоянии способен выполнить выдающийся анализ, но смысл его результатов все еще должны осознавать люди, ведь они принимают необходимые моральные и этические решения на его основании. По существу, машинное обучение обеспечивает лишь часть обучения искусственного интеллекта, и эта часть ничуть не готова создать искусственный интеллект того вида, который вы видите в фильмах.

Существует несколько определений машинного обучения, отличающихся формулировками, но близких по смыслу.

Машинное обучение (*machine learning, ML*) — класс методов искусственного интеллекта, характерной чертой которых является не прямое решение задачи, а обучение в процессе применения решений множества сходных задач.

Машинное обучение — это обширный подраздел искусственного интеллекта, изучающий методы построения алгоритмов, способных обучаться.

Машинное обучение — это подобласть искусственного интеллекта. *Data-driven* подход, полагающийся на выучивание закономерностей из имеющихся размеченных или неразмеченных данных.

Машинное обучение — это наука, изучающая алгоритмы, автоматически улучшающиеся благодаря опыту.

С теоретической стороны *машинное обучение* — дисциплина, находящаяся на пересечении математической статистики, численных методов оптимизации, теории вероятностей, теории систем и системного, а также дискретного анализа. С помощью ее методов происходит решение задачи извлечения знаний из данных, которой занимается область «Интеллектуальный анализ данных» (*Data Mining*).

Data Mining (добыча данных, интеллектуальный анализ данных, глубинный анализ данных) — совокупность методов обнаружения в данных ранее неизвестных, нетривиальных, практически полезных и доступных интерпретации знаний, необходимых для принятия решений в различных сферах человеческой деятельности.

В теории искусственного интеллекта и экспертных систем, знание — это совокупность утверждений о мире, свойствах объектов, закономерностях процессов и явлений, а также правил логического вывода одних утверждений из других и правил использования их для принятия решений. Главное отличие знаний от данных состоит в их структурности и активности: появление в базе знаний новых фактов или установление новых связей между ними может стать источником изменений в принятии решений.

Если, опираясь на это определение знания, рассмотреть *Data Mining* как процесс, то его сутью будет нахождение в «сырых» данных знаний, обладающих свойствами нетривиальности, интерпретируемости и практической полезности для принятия решений в различных сферах деятельности человека. Причем, эти знания ранее неизвестны.

С практической же стороны машинное обучение нацелено на создание систем, способных адаптироваться к решению различных задач без явного кодирования алгоритма, т. е. систем, способных обучаться.

Машинное обучение (*ML*) содержательно включает в себя три компонента:

1. *Представление* (*Representation*). Что собой представляет модель, какие классы задач она способна (и не способна) решать, как именно представляются данные (включая этап выделения нужных признаков). Примеры: разделяющая гиперплоскость, деревья решений, нейросети.
2. *Оценка* (*Evaluation*). Как оценивать качество модели в контексте решения задачи, как выбирать лучшую модель из нескольких.
3. *Оптимизация* (*Optimization*). Как именно проводить обучение модели, каким именно образом осуществлять перебор пространства возможных моделей, чтобы найти лучшую.

Цель машинного обучения — предсказать результат по имеющимся данным. Чем качественнее и полнее входные данные, тем легче машине найти какие-либо закономерности и тем более точным является результат.

Машинное обучение нацелено на создание систем, способных получать знания из данных, способных с помощью обучения улучшать показатели своей работы. Таким образом, можно утверждать, что машинное обучение является одной из областей науки о данных (*Data Science*). На рис. 4.11 представлены ее составляющие и показано место машинного обучения среди них.

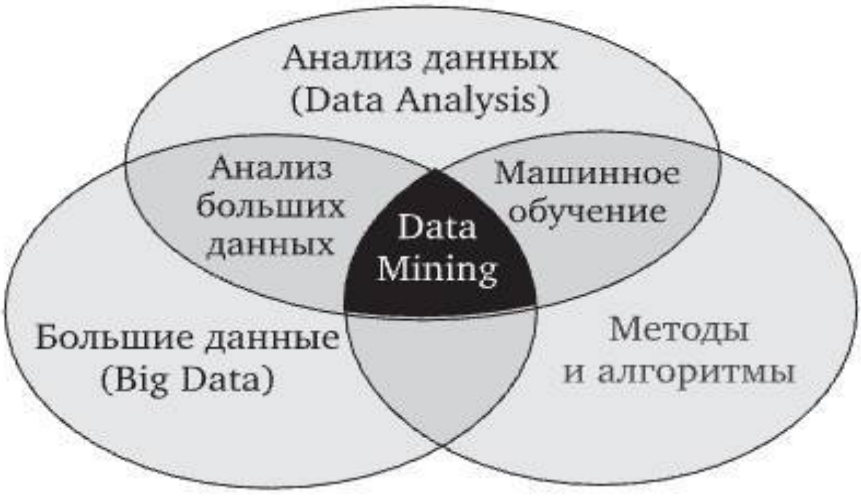


Рис.4.11. **Области науки о данных**

Сложность работы в сфере *Data Mining* обуславливается неточностью, противоречивостью, разнородностью данных, которые при этом могут иметь гигантские объемы обработки. Требуются специ-

альные программные средства, инструменты преобразования «сырых» данных в информацию, а информации в знания. Кроме того, алгоритмы обработки данных должны иметь возможность обучаться по прецедентам. Машинное обучение как раз и занимается разрешением подобных сложностей. Причем в настоящее время усилия сконцентрированы на повышении не столько качества решения, сколько скорости и оптимальности его получения.

Как видно из рис. 4.11, машинное обучение пересекается с анализом данных, т. е. имеет общие черты, но также и свою специфику. Анализ данных — это огромная и уже относительно не молодая область математики и информатики. Под нее попадает всяческая обработка данных. Например, обработка данных физического эксперимента статистическими методами для получения обобщенной информации (усредненных значений, точных значений, доверительных интервалов, оценок и т. п.), обработка сигнала методом Фурье, даже подсчет минимального и максимального значений или отношения каких-то величин могут быть анализом данных, если эти операции влекут за собой важные выводы. Знания о данных и позволяют решить поставленную задачу. Но получение дисперсии некой величины хоть и является анализом данных, но не является в чистом виде машинным обучением. При этом необходимо отметить, что методы статистики имеют большое значение и в машинном обучении.

Отличительной особенностью методов машинного обучения является то, что они способны решать широкий спектр задач без явного кодирования алгоритма решения.

Например, метод регрессии входит в машинное обучение. Регрессия давно применяется в различных областях человеческой деятельности: экономика, прикладная математика, технические и гуманитарные науки и т. д. Она используется для прогнозирования и оценки разнообразных параметров, потому что данная математическая модель в некотором роде универсальна. Однако стоит заметить, что регрессия появилась в математике гораздо раньше, чем ее стали использовать в прикладных областях.

Машинным обучением регрессия стала только тогда, когда все ее методы начали рассматриваться не как обособленные механизмы решения задач из специальных областей, а как методы преобразования информации вообще.

Для разработки решений на основе *ML* необходимы оцифрованные данные. Они являются ключевым аспектом, от которого зависят качество и полнота решения. Таким образом, очень важно понимать, что для применения методов машинного обучения требуются собранные и специально подготовленные данные.

Как правило, процесс машинного обучения проходит в несколько этапов. На рис. 4.12 представлен обобщенный алгоритм, выполняющийся при решении задач методами рассматриваемой области.

Рис.4.12. **Этапы машинного обучения**

Для реализации машинного обучения необходимы следующие вещи.

Данные. Хотим фильтровать спам — нужны экземпляры спам–писем, предугадывать курс валюты — нужна ретроспектива цен, определять предпочтения клиента — нужны персональные лайки или посты в социальных сетях. Необходимо собирать максимально возможное количество данных. Десятки тысяч кейсов — это минимум, не гарантирующий качественного обучения системы.

Сбор данных осуществляется разными способами. Часть данных собирается вручную — способ долгий, но снижается количество ошибок. Часть — автоматически — можно загрузить все, что есть в систему и ждать какого-то, возможно отрицательного результата. Хорошим способом, используемым, например, *Google*, является совмещение автоматического сбора данных с ручной разметкой.

Хорошие и проработанные наборы данных (датасеты) очень ценны в мире машинного обучения. Иногда компании, даже раскрывают алгоритмы своих систем, но датасеты практически никогда не публикуются в открытую.

Необходимо отметить, что данная иллюстрация отражает идеальный случай, когда после обучения мы сразу получили работающую модель. Однако в реальности, как правило, между вторым и третьим этапом есть промежуточное, но очень важное действие — оценка качества модели. И именно по ее результатам принимается решение о переходе на следующий этап или о возврате к одному из предыдущих. Этот вопрос мы рассмотрим подробнее далее, здесь

же введем определение пространства признаков, для лучшего понимания схемы на рис. 4.12.

Пространство признаков — это N–мерное пространство, где N — число измеряемых характеристик объектов, выделенное для конкретной задачи. При этом объекты в пространстве признаков задаются N–мерными векторами, каждая компонента которых представляет собой значение определенной характеристики.

Признаки иногда называют фичами (*features*). *Фичи* — это свойства, характеристики, признаки, которые описывают тот или иной объект или явление. К примеру, фичами являются пробег автомобиля, массогабариты человека, его пол, стоимость акций, частота появления букв в тексте и т. д. При обучении машина должна знать, что она должна учитывать. При структурированном хранении данных, когда информация хранится в таблицах названия столбцов можно считать признаком. Однако, чаще всего данные имеют совершенно другой формат. Если признаков слишком много, модель становится медленной и неэффективной. Может случиться так, что отбор корректных свойств занимает больше времени, чем все остальное обучение. С другой стороны, возможны и противоположные ситуации, когда человек самостоятельно решает использовать ли «правильные» на его взгляд признаки. Это вносит в модель субъективность, которая приводит к тому, что возрастает процент ошибок.

Алгоритм. Как правило, любая задача может быть решена большим количеством способов. От выбранного способа решения зависит размерность готовой математической модели, точность, вычислительные затраты, скорость работы, емкость. Однако следует учитывать, что если данные изначально некачественные, то даже самый лучший и точный алгоритм не поможет. Из лжи может следовать все, что угодно, и это правило Булевой алгебры никто не отменял. Поэтому еще раз стоит отметить, что необходимо собрать как можно больше данных.

Очень часто в различных изданиях в силу моды «искусственным интеллектом» называют любую линейную регрессию, которая применяется даже в средней школе. На самом деле, все немного сложнее. Место машинного обучения в системе искусственного интеллекта представлено на рис. 4.13.

Рис. 4.13. **Место машинного обучения в системе искусственного интеллекта**

Искусственный интеллект — это область науки и инжиниринга, занимающаяся созданием машин и компьютерных программ, обладающих интеллектом. Это название всей области, как физика или математика.

Нейросети — всего лишь вид машинного обучения. Это математическая модель, а также ее программное или аппаратное воплощение, построенная по принципу организации и функционирования биологических нейронных сетей. Есть и другие виды обучения, используемые для разных задач.

Глубокое обучение — архитектура нейросетей, один из подходов к их построению и обучению. Под этим термином понимают совокупность методов машинного обучения (с учителем, с частичным привлечением учителя, без учителя, с подкреплением), основанных на обучении представлениям (фичам), а не специализированным алгоритмам под конкретные задачи. В реальности определять глубину обучения сейчас не стараются, просто используют имеющиеся движки. Возможности машины представлены в табл 4.3.

Таблица 4.3

Возможности машины

МАШИНА МОЖЕТ	МАШИНА НЕ МОЖЕТ
Предсказывать	Создавать новое
Запоминать	Поумнеть самостоятельно
Воспроизводить	Выйти за рамки задачи
Выбирать лучшее	Брать ответственность

Классифицировать алгоритмы можно десятком способов. Ментальные карты сейчас — самый простой и наглядный способ. Однако не бывает так, чтобы какую-то задачу решал только один метод. Как правило, задачу можно решить несколькими способами, и явно есть «специалист», который захочет все решить нейросетями: лучше, быстрее, дешевле. На данный момент в машинном обучении есть четыре крупных основных направления (рис. 4.14).

1. Классическое обучение (простые данные, понятные признаки).
2. Обучение с подкреплением (данных нет, но есть среда, с которой можно взаимодействовать).
3. Ансамбли (сложные данные).
4. Нейросети и глубокое обучение (сложные данные, размытые признаки).

Задачи машинного обучения представлены на рис. 4.14.

Процесс решения задачи с помощью машинного обучения включает в себя:

1. Определиться со способом решения вашей задачи средствами ML.
2. Выбрать метрику качества.

3. Получить данные и выделить признаки.
4. Определиться с классами моделей.
5. Подготовить данные для обучения и оценки.
6. Провести обучение моделей, оценить результат, повторить шаги, если требуется.

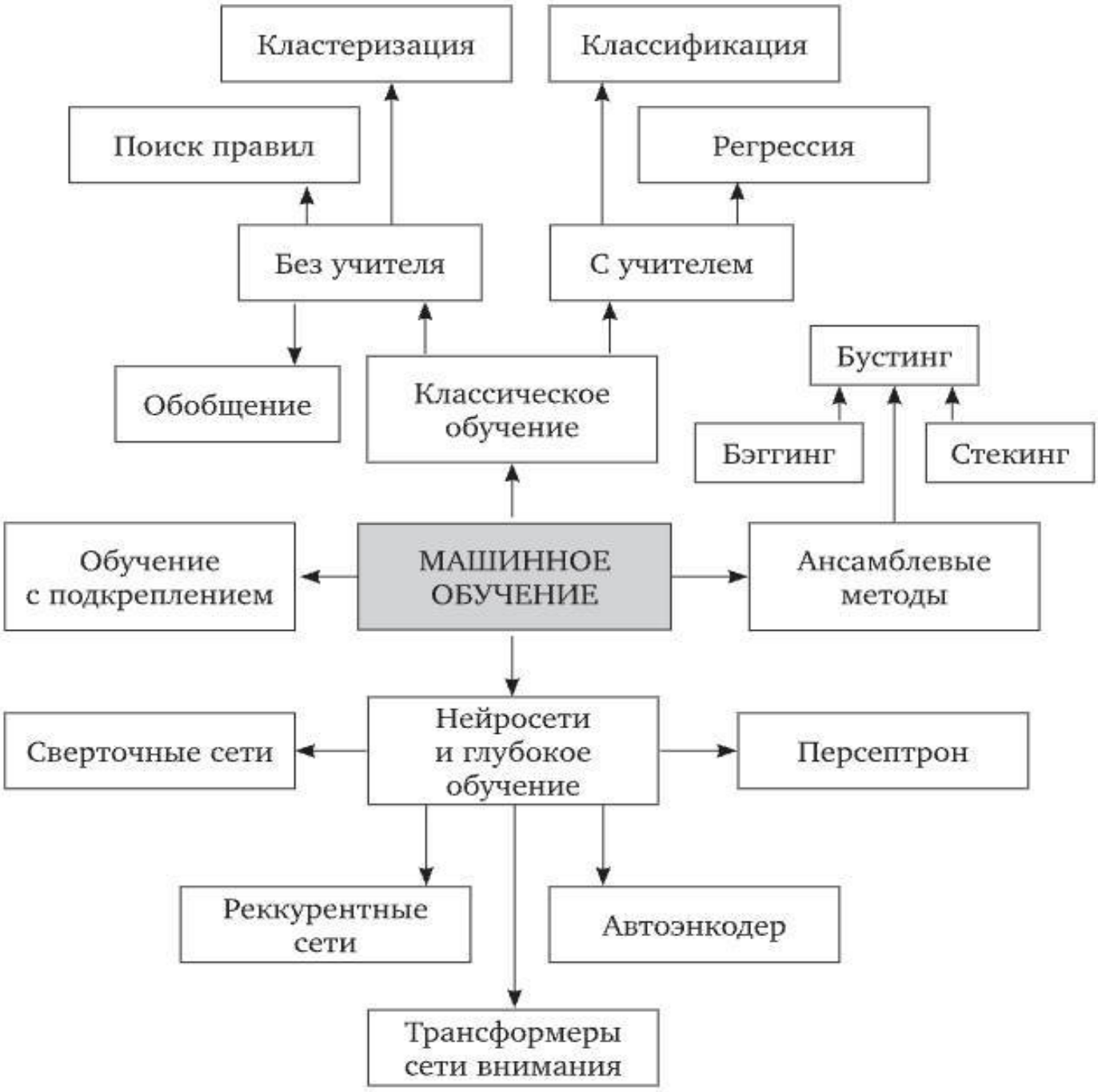


Рис.4.14. **Задачи машинного обучения**

Самую подробную информацию по ИНС можно найти в учебнике [68].

Тема 5
ПРИКЛАДНЫЕ ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ

5.1. Прикладной характер информационных технологий

Информационные технологии являются не только объектом исследований и разработки, но и средством создания информационных систем в различных предметных областях. Несмотря на специфику конкретных объектов, удалось разработать методологию, модели, методы и средства прикладных информационных технологий, что позволяет снизить затраты и сократить сроки информатизации. Спектр прикладных информационных технологий широк. Поскольку объем учебника ограничен, рассмотрим общий подход к реализации прикладных информационных технологий.

Если на первых порах информатизации в каждой предметной области решались отдельные локальные задачи, то в настоящее время наметилась тенденция к комплексному характеру использования ИТ. Общим для любой предметной области является управление материальными, финансовыми, трудовыми, информационными ресурсами, а специфика проявляется в решении технологических задач.

Прикладные ИТ, основываясь на стандартных моделях, методах и средствах, допускают формулировку, постановку и реализацию задач в терминах предметной области пользователя. Совершенствование данного класса технологий направлено на обеспечение автоматизированного формирования модели предметной области и погружения ее в стандартную инструментальную среду.

Таким образом, *корпоративное управление* является общим для использования ИТ в промышленности и экономике, в образовании, медицине и других предметных областях.

Корпоративное управление и создание корпоративных информационных систем в настоящее время опираются на различные ИТ, так как, к сожалению, не существует универсальной.

Можно выделить следующие три группы *методов управления*: ресурсами, процессами, корпоративными знаниями (коммуникациями). Среди ИТ в качестве наиболее используемых можно выделить следующие: СУБД, *Workflow*, стандарты ассоциации *Workflow*

Management Coalition, Intranet. На рис. 5.1 показаны место и назначение каждой из ИТ.

Рис. 5.1. **Место и назначение каждой из информационных технологий в корпоративном управлении**

На рис. 5.2 интенсивность цвета соответствует степени поддержки информационными технологиями методов управления.

	Ресурсы	Процессы	Коммуникации-знания
СУБД			
Workflow			
Интранет			

Рис. 5.2. **Степень поддержки информационными технологиями методов управления**

Задача управления ресурсами относится к числу классических методик управления и является первой, где широко стали использоваться информационные технологии. Это связано с наличием хорошо отработанных экономико-математических моделей, эффективно реализуемых средствами вычислительной техники. Рассмотрим эволюцию задач управления ресурсами.

Первоначально была разработана методология планирования материальных ресурсов предприятия *Material Requirements Planning* (далее — *MRP*), которая использовалась с методологией объемно-календарного планирования *Master Planning Shedule* (далее — *MPS*). Следующим шагом было создание методологии планирования

производственных ресурсов (мощностей) *Capacitiy Requirements Planning* (далее — *CRP*). Эта методология была принципиально похожа на *MRP*, но была ориентирована на расчет производственных мощностей, а не материалов и компонентов. Эта задача требует больших вычислительных ресурсов, даже на современном уровне.

Объединение указанных выше методологий привело к появлению задачи *MRP* «второго уровня» *MRP*II — интегрированной методологии планирования, включающей *MRP/CRP* и использующей *MPS* и *FRP* (*Finance Resource/Requirements Planning*) — планирование финансовых ресурсов. Далее была предложена концепция *Enterprise Resource Planning* (далее — *ERP*) — интегрированное планирование всех «бизнес»-ресурсов предприятия.

Эти методологии были поддержаны соответствующими инструментальными средствами. В большей степени к поддержке данных методологий применимы СУБД.

Следующим шагом было создание концепции управления производственными ресурсами — *Customer Synchronized Resource Planning* (далее — *CSPP*) — планирование ресурсов, синхронизированное с потреблением. Отличием данной концепции является учет вспомогательных ресурсов, связанных с маркетингом, продажей и послепродажным обслуживанием. На рис. 5.3 показано соотношение между понятиями *CSSP*, *ERP* и стадиями жизненного цикла товара.

В связи с тем, что в современном производстве задействовано множество поставщиков и покупателей, появилась новая концепция логистических цепочек (*Supply Chain*). Суть этой концепции состоит в учете при анализе хозяйственной деятельности всей цепочки (сети) превращения товара из сырья в готовое изделие (рис. 5.4).

Рис. 5.3. **Соотношение между понятиями CSSP, ERP и стадиями жизненного цикла товара**

При этом акцент сделан на следующих факторах:

- стоимость товара формируется на протяжении всей логистической цепочки, но определяющей является стадия продажи конечному потребителю;

- на стоимости товара критическим образом сказывается общая эффективность всех операций;
- наиболее управляемыми являются начальные стадии производства товара, а наиболее чувствительными — конечные (продажные).

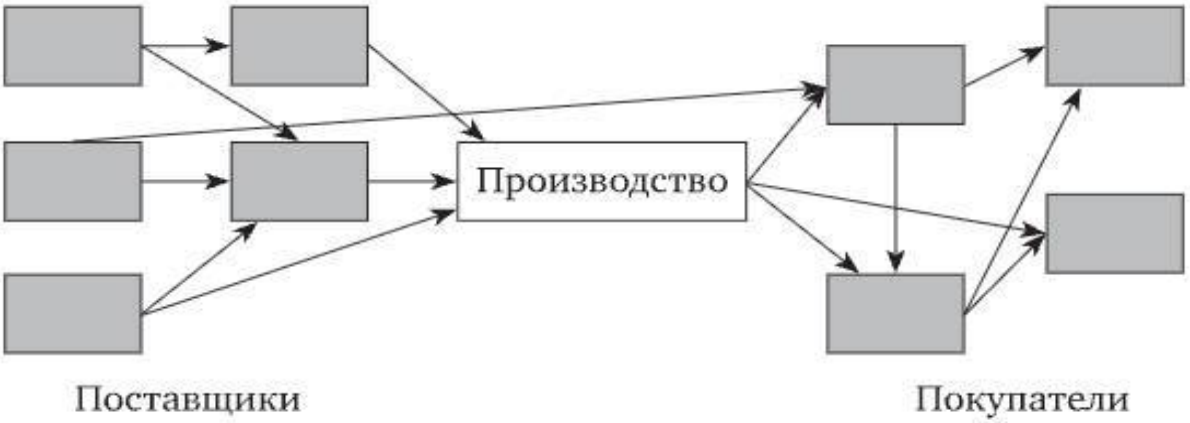


Рис. 5.4. **Концепция логистических цепочек**

Дальнейшим развитием концепции логистических цепочек является идея виртуального бизнеса (рис. 4.5), представляющего распределенную систему нескольких компаний и охватывающего полный жизненный цикл товара, или разделение одной компании на несколько «виртуальных бизнесов».

Рассмотренные выше методологии нашли проявление как в отдельных программных продуктах, так и в рамках интранета как инструмента корпоративного управления.

Универсальный язык представления корпоративных знаний не зависит от конкретной предметной области и определяет грамматику и синтаксис. На данном этапе не существует единого языка описания, и к этой категории может быть отнесен графический язык описания моделей данных, сетевых графиков, алгоритмов и др. Задачей универсального языка представления корпоративных знаний является: унификация представления знаний, однозначное толкование знаний, разбиение процессов обработки знаний на простые процедуры, допускающие автоматизацию.

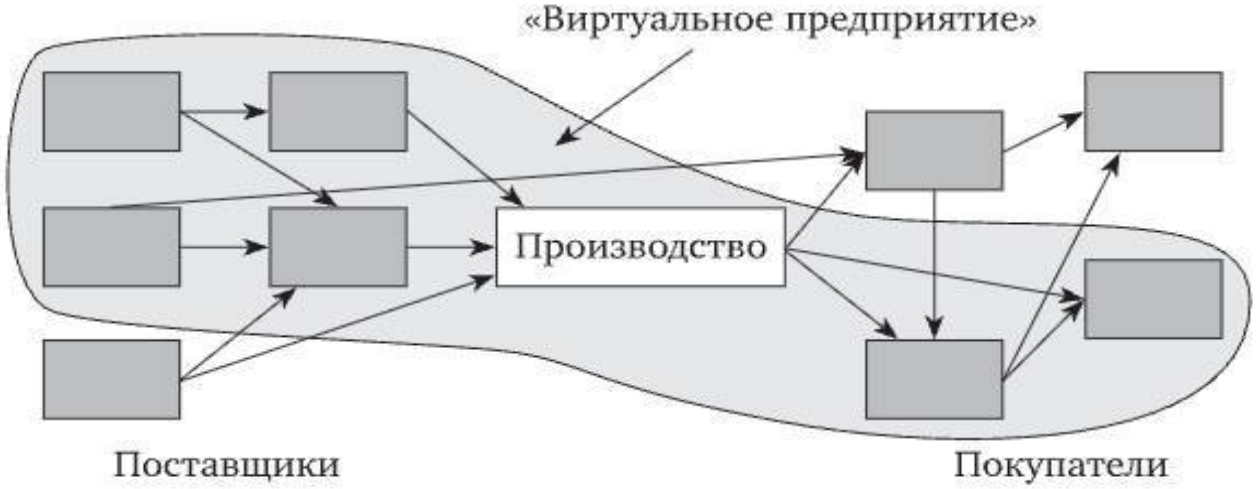


Рис. 5.5. **Идея виртуального бизнеса**

Модели представления определяют специфику деятельности организации. Знания этого уровня являются метаданными, описывающими первичные данные.

Фактические знания отображают конкретные предметные области и являются первичными данными.

Интранет дает ощутимый экономический эффект в деятельности организации, что связано в первую очередь с резким улучшением качества потребления информации и ее прямым влиянием на производственный процесс. Для информационной системы организации ключевыми становятся понятия: «публикация информации», «потребители информации», «представление информации». Дальнейшее развитие корпоративного управления связано с моделями *MRP/ERP* и управления жизненным циклом изделия.

5.2. Модели планирования материальных и финансовых ресурсов (MRP/ERP)

Модели *MRP/ERP* с момента возникновения развивались эволюционно. С целью оптимального управления производством в середине 1960-х гг. *APICS* сформулировало принципы управления материальными запасами предприятия. Эти принципы легли в основу концепции *MRP*, основными положениями которой являются [8]:

- описание производственной деятельности как потока взаимосвязанных заказов;
- учет ограничения ресурсов при выполнении заказов;
- обеспечение минимизация производственных циклов и запасов;
- формирование заказов снабжения и производства на основе реализации и производственных графиков;
- привязка движения заказов к экономическим показателям;
- завершение выполнения заказа к тому моменту, когда он необходим.

Развитие вычислительных средств и наличие концепции привело к тому, что в 1970-х гг. стали появляться первые автоматизированные системы, реализующие *MRP*-концепцию.

Методика *MRP* декларирует, какие процессы учета и управления должны быть реализованы на предприятии, в какой последовательности они должны выполняться, и содержит рекомендации о том, как они должны выполняться.

Дальнейшее развитие концепции *MRP* шло по пути расширения функциональных возможностей предприятия в сторону более полного удовлетворения потребностей клиентов и снижения производственных издержек. В результате в конце 1970-х гг. концепция *MRP* была дополнена положениями о формировании производственной

программы в масштабах всего предприятия и контроля ее выполнения на уровне подразделений (*Closed Loop MRP* или, другими словами, воспроизведение замкнутого цикла в *MRP*-системах). Следующим шагом стало появление концепции *MRP*II, основным содержанием которой стало прогнозирование, планирование и контроль производства по всему циклу, начиная от закупки сырья и заканчивая отгрузкой товара потребителю.

*MRP*II представляет собой методологию, направленную на эффективное управление всеми ресурсами производственного предприятия. В общем случае она обеспечивает решение задач планирования деятельности предприятия в натуральных единицах, финансовое планирование в денежном выражении. Эта методология представляет собой набор проверенных на практике принципов, моделей и процедур управления и контроля, использование которых способствует улучшению показателей экономической деятельности предприятия.

Стандарт *APICS* на системы класса *MRP*II содержит описание 16 групп функций системы:

1) *Sales and Operation Planning* (планирование продаж и производства);
2) *Demand Management* (управление спросом);
3) *Master Production Scheduling* (составление плана производства);
4) *Material Requirement Planning* (планирование материальных потребностей);
5) *Bill of Materials* (спецификации продуктов);
6) *Inventory Transaction Subsystem* (управление складом);
7) *Scheduled Receipts Subsystem* (плановые поставки);
8) *Shop Flow Control* (управление на уровне производственного цеха);
9) *Capacity Requirement Planning* (планирование потребностей в мощностях);
10) *Input/output control* (контроль входа/выхода);
11) *Purchasing* (материально-техническое снабжение);
12) *Distribution Resource Planning* (планирование ресурсов распределения);
13) *Tooling Planning and Control* (планирование и управление инструментальными средствами);
14) *Financial Planning* (управление финансами);
15) *Simulation* (моделирование);
16) *Performance Measurement* (оценка результатов деятельности).

По мере накопления опыта моделирования производственных и непроизводственных операций содержание этих групп уточняется, постепенно охватывая все больше функций. При этом следует иметь в виду, что перечисленный функциональный состав относится только к управлению производственными ресурсами предприятия.

Стандарт *MRP*II делит сферы отдельных функций (процедур) на два уровня: необходимый и опциональный. Для отнесения к классу *MRP*II автоматизированные системы управления должны выполнять определенный объем необходимых (основных) функций (процедур).

Состав функциональных модулей и их взаимосвязи имеют глубокое обоснование с позиции теории управления. Они обеспечивают интеграцию функций планирования и согласование различных процессов управления во времени и пространстве. Представленный набор модулей является не избыточным, что позволяет, в основном, сохранять его неизменным в системах следующих поколений. Более того, многие понятия, методы и алгоритмы, заложенные в функциональные модули *MRP*II, остаются неизменным в течение длительного времени и входят в качестве элементов в системы следующих поколений. По этой причине методологию *MRP*II можно считать базовой.

Для каждого уровня планирования *MRP*II характерны такие параметры, как степень детализации плана, горизонт планирования, вид условий и ограничений. Эти параметры для одного и того же уровня *MRP*II могут изменяться в широком диапазоне в зависимости от свойств производственного процесса на предприятии. С другой стороны, в зависимости от характера производственного процесса возможно применение на каждом отдельном предприятии определенного набора функциональных модулей *MRP*II. Таким образом, *MRP*II является гибкой и многофункциональной системой, применение которой возможно в широком спектре условий.

Принадлежность корпоративной информационной системы (КИС) к классу *MRP*II должна означать функциональную поддержку программным обеспечением выполнения следующего цикла: «планирование заказов → планирование потребности в сырье и материалах → планирование производственных ресурсов → контроль над исполнением производственной программы → обратная связь».

В общем виде система управления предприятием, построенная в соответствии со стандартом *MRP*II, имеет следующий вид (рис. 5.6). Дадим краткую характеристику вышеперечисленных функциональных блоков *MRP*II.

1. *Бизнес-планирование*: процесс формирования плана предприятия наиболее высокого уровня; планирование долгосрочное, план составляется в стоимостном выражении; наименее формализованный процесс выработки решений.
2. *Планирование спроса*: процесс прогнозирования (планирования) спроса на определенный период.
3. *Планирование продаж и производства*: бизнес-план и план спроса преобразуются в планы продаж основных видов продукции (как правило, от 5 до 10); при этом производственные мощности могут не учитываться или учитываться укрупненно; план носит среднесрочный характер.

Рис. 5.6. **Система управления предприятием, построенная в соответствии со стандартом MRPII**

План продаж по видам продукции преобразуется в объемный или объемно-календарный план производства видов продукции. Под видом здесь понимаются семейства однородной продукции. В этом плане впервые в качестве планово-учетных единиц выступают изделия, но представления о них носят усредненный характер. Например, речь может идти обо всех моделях телевизоров с одинаковым размером экрана, выпускаемых на заводе (без уточнения моделей). Часто этот модуль объединяется с предыдущим.

4. *План-график выпуска продукции*: план производства преобразуется в график выпуска продукции. Как правило, это среднесрочный объемно-календарный план, задающий количества конкретных изделий (или партий) со сроками их изготовления.

5. *Планирование потребностей в материальных ресурсах*: в ходе планирования на этом уровне определяются в количественном выражении и по срокам потребности в материальных ресурсах, необходимых для обеспечения графика выпуска продукции.

Входными данными для планирования потребностей в материалах являются спецификации изделий (состав и количественные характеристики комплектующих конкретного изделия) и размер текущих материальных запасов.

6. *Планирование производственных мощностей*: как правило, в этом модуле выполняются расчеты по определению и сравнению располагаемых и потребных производственных мощностей. С наибольшими изменениями этот модуль может применяться не только для производственных мощностей, но и для других видов производственных ресурсов, способных повлиять на пропускную способность предприятия. Подобные расчеты обычно производятся после формирования планов практически всех предыдущих уровней с целью повышения надежности системы планирования. Иногда решение данной задачи включают в модуль соответствующего уровня. Входными данными при планировании производственных мощностей являются также маршрутизация выпускаемых изделий.

7. *Управление заказами клиентов*: реальные потребности клиентов сопоставляются с планами выпуска продукции.

8. *Управление на уровне производственного цеха*: формируются оперативные планы-графики. В качестве планово-учетных единиц могут выступать детали (партии), сборочные единицы глубокого уровня, детале-(партие-) операции и т. п. Длительность планирования невелика (от нескольких дней до месяца).

9. *Оценка исполнения*: по сути, в данном модуле оценивается реальное исполнение всех вышеперечисленных планов, с тем чтобы внести корректировки во все предыдущие циклы планирования.

Связь между уровнями в *MRP*II обеспечивается универсальной формулой, на которой строится система. Задача планирования на каждом уровне реализуется как ответ на четыре вопроса:

1) что необходимо выполнить?
2) что необходимо для этого?
3) что есть в наличии?
4) что необходимо иметь?

В роли ответа на первый вопрос всегда выступает план более высокого уровня. Этим и обеспечивается связь между уровнями. Структура ответов на последующие вопросы зависит от решаемой задачи.

Дальнейшее развитие систем *MRP*II связано с их перерастанием в системы нового класса — «Планирование ресурсов предприятия» (*Enterprise Resource Planning, ERP*). Системы этого класса ориентированы на работу с финансовой информацией для решения задач управления большими корпорациями с разнесенными территориально ресурсами. Сюда включается все, что необходимо для получения ресурсов, изготовления продукции, ее транспортировки и расчеты по заказам клиентов. Помимо перечисленных функциональных требований, к системам *ERP* предъявляются и новые требования по применению графики, использованию реляционных баз данных, *CASE*-технологий для их развития, архитектуры вычислительных систем типа «клиент — сервер» и реализации их как открытых систем. Системы этого класса активно развиваются с конца 1980-х гг.

Следует отметить, что подход к решению задач планирования производства в системах *ERP* до недавнего времени оставался в основном неизменным, т. е. в том виде, в каком он утвердился в системах *MRP*II. Коротко его можно определить как подход, базирующийся на активном применении календарно-плановых нормативов на производственные циклы. Недостаток такого подхода состоит в том, что он вступает в противоречие с необходимостью оптимизации планирования. Элементы оптимизации планирования в традиционных *MRP*II/*ERP* системах встречаются только на нижнем уровне — при решении задач оперативного планирования с применением методов теории расписаний. С ростом мощностей вычислительных систем, внедрением *MRP*II/*ERP*, поиском новых более эффективных методов управления в условиях конкуренции с середины 1990-х гг. на базе систем *MRP*II/*ERP* появляются системы нового класса, которые получили название «Развитые системы планирования» (*Advanced Planning/Scheduling — APS*). Для этих систем характерно применение экономико-математических методов для решения задач планирования с постепенным снижением роли календарно-плановых нормативов на производственные циклы.

Рост производительности и снижение незавершенного производства за счет внедрения таких систем объясняется тем, что при определении длительности производственного цикла в него не закладывается заранее усредненное время пребывания сырья в очередях. Данный подход особенно эффективен для сложного многономенклатурного производства. В то же самое время он требует существенного повышения профессионального уровня управленческого персонала.

Следующее направление в развитии автоматизации управления предприятиями состоит в интеграции систем *MRP*II/*ERP* с другими автоматизированными системами, имеющимися на предприятиях. В их числе — системы *CAD/CAM*, управления технологическими

процессами и системами, системы финансовой отчетности и т. п. Системы такого класса получили название «Компьютерные интегрированные системы» (*Computer Integrated Manufacturing — CIM*). Эти системы используются, начиная с 1990-х гг.

Таким образом, система *MRP*II постоянно эволюционирует и совершенствуется. В каждый момент времени в концепциях *MRP*II/*ERP* можно выделить, условно, три слоя.

В первом слое находятся те методы и средства, которые проверены практикой и закреплены в виде стандартов.

Второй слой составляют достаточно устойчивые, часто применяемые методы и приемы, которые, однако, не носят обязательного характера. Эти методы и приемы можно обнаружить при более глубоком анализе функциональных структур. В качестве примеров можно привести методологию скользящего планирования в *MPS/MRP*, алгоритмы образования партий в *MRP*, правила приоритетов в *SFC* и многое другое.

Этот слой, жестко не регламентируемый, тем не менее, представляет собой довольно стройную систему взаимосвязанных идей и методов.

К третьему слою идей и методов *MRP*II/*ERP* следует отнести то новое, что вносят в свои базовые системы фирмы-производители программных продуктов. Реализованные на их основе новые информационные технологии представляют собой «*know-how*» фирм-разработчиков. Как правило, именно в этом слое можно обнаружить значительные отличия в продуктах различных фирм. Некоторые из новых технологий в состоянии оказывать серьезное влияние на эффективность построения крупных информационных систем.

Видное место среди идей и методов систем *MRP*II/*ERP* принадлежит специально разработанным методикам внедрения систем. Анализ литературы показывает, что на Западе сложилось устойчивое представление о том, в какой последовательности и какими методами следует внедрять системы типа *MRP*II/*ERP*. Тщательное планирование проектов по внедрению, организация деятельности коллективов, упор на переподготовку персонала всех уровней (особенно высшего уровня) — вот далеко не полный перечень условий достижения положительных результатов. Наличие мощной инфраструктуры и методологии построения систем способствовало в итоге достижению высокого уровня эффективности при внедрении систем управления типа *MRP*II/*ERP* на промышленных предприятиях. По некоторым оценкам внедрение подобных систем способно привести к сокращению запасов на 8—30 %, росту производительности труда на 8—27 %, возрастанию количества заказов, выполненных в срок на 7—20 %.

На рис. 5.7 представлены цели внедрения систем типа *MRP*II/*ERP*.

Рис. 5.7. **Цели внедрения систем типа MRPII/ERP**

Дальнейшим развитием моделей *MRP*II/*ERP* является архитектура, ориентированная на сервисы — *Service Oriental Architecture* (далее — *SOA*) и ориентированные на сервисы бизнес-приложения — *Service Oriental Business Application* (далее — *SOBA*). Развитие этих направлений связано с тем, что существенным недостатком *ERP* является отраслевая специфика [20]. *MRP*II, служащая основой для многих *ERP*, — это методология планирования дискретных, большей частью машиностроительных производств. Ряд факторов привел к многочисленным неудачным попыткам внедрения подобных систем на других производствах и к проблеме адаптации программного продукта под конкретное производство. Ситуацию усугубило распространение таких методологий планирования, как заказное, гибкое производство (*agile manufacturing*) и «экономичное производство» (*lean manufacturing*). Таким образом, возникла проблема гибкости и универсальности интерфейса управления производством.

В основе идеологии *SOA* — интеграция множественных, одновременно работающих разных предприятий, территориально разнесенных бизнес-приложений. Это могут быть как вновь внедряемые, так и унаследованные, в том числе традиционные *ERP*-системы. В определенном плане появление *SOA* — возврат к идеологии мейнфреймов, но на другом качественном уровне. Если мейнфреймы объ-

единяли вычислительные ресурсы, то *SOA* объединяют и позволяют использовать данные и приложения.

5.3. Модели управления жизненным циклом изделия (PLM)

В настоящее время остро стоит проблема перехода от использования отдельных информационных технологии к созданию единых интегрированных комплексов. Одним из направлений является модели *Product Lifecycle Management* (далее — *PLM*) (управление жизненным циклом изделия).

Появление *PLM* связано с развитием электронного бизнеса, классификация которого, предложенная компанией *Gartner Group*, представлен на (рис. 5.8) [8].

Рис. 5.8. **Этапы развития электронного бизнеса**

Выделено четыре фазы развития: присутствие, взаимодействие, передача данных (транзакции) и трансформация бизнеса. Начиная с 2000 г. активно развиваются третья и четвертая фазы. Другая классификация (рис. 5.9) основана на использовании данных, и развитие электронного бизнеса рассматривается как восхождение от простого обмена данными к обеспечению средствами технологий потоков работ и потоков принятия решений.

Бурное развитие *PLM* прослеживается в следующем. Современный бизнес решает триединую задачу: во-первых, необходимо устанавливать более тесные и доверительные отношения с поставщиками и заказчиками, во-вторых, повышать уровень собственной операционной эффективности и, в-третьих, повышать конкурентоспособность выпускаемой продукции. Первая составляющая обе-

спечивается системами поддержки отношений, получающими все большее распространение, — системой управления цепочками поставок *Supply Chain Management* (далее — *SCM*) и системой управления отношениями с заказчиками *Customer Relations Management* (далее — *CRM*), вторая — еще более популярными системами *ERP*, а вот третья пока не имеет достаточного комплексного информационного обеспечения. На то, чтобы занять это место, претендует подход, названный *new PLM* («*PLM* по-новому»). Он заметно отличается от традиционного представления о том, что такое управление жизненным циклом изделий. Раньше под *PLM* (в узком смысле) чаще всего понимали то, что имело отношение к жизненному циклу материальных изделий, начиная от запуска в производство, регулирования объемов выпуска, определения времени выпуска новых или обновленных изделий и, конечно же, сопровождение и сервис. Изменения в видении роли и места *PLM* произошли буквально в последние несколько лет. Теперь под этим термином понимают автоматизацию практически всех видов работ, которые составляет основу выпуска продукции — от проектирования до сбыта. Разные авторы расходятся в определениях: одни включают *SCM*, *CRM* и *ERP* в состав нового управления жизненным циклом изделий, другие считают эти системы взаимодополняющими (рис. 5.10).

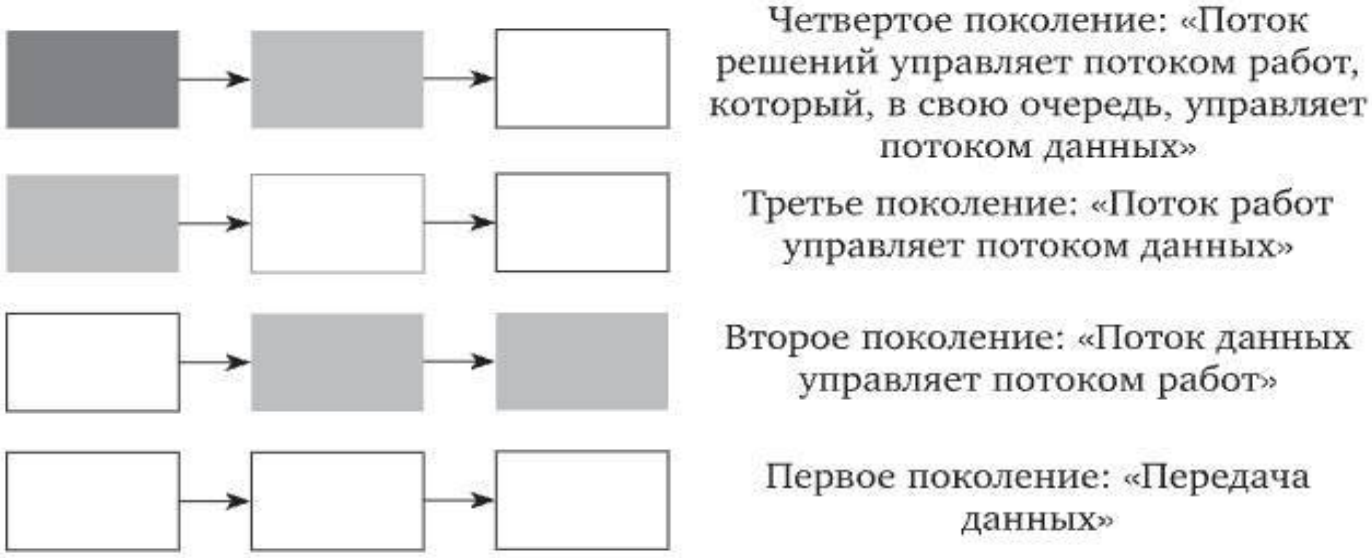


Рис 5.9. **Поколения электронного бизнеса:**
■ — поток решений (*Decision Flow*), ■ — поток работ (*Workflow*),
□ — поток данных (*Dataflow*)

В соответствии с триединой задачей «*PLM* по-новому» можно разделить на три взаимосвязанных составляющие управления жизненным циклом:

- жизненный цикл определения изделий (интеллектуальные активы предприятия);
- жизненный цикл производства (материальные активы предприятия);
- жизненный цикл операционной поддержки.

Первичным является жизненный цикл управления интеллектуальными активами; он начинается с оценки пользовательских требований, выработки концепции продукта, а завершается, когда предприятие полностью отказывается от продукта, в том числе и от его сервисной поддержки.

Рис. 5.10. **Взаимосвязь корпоративных приложений**

PLM-решения — один из самых быстрорастущих и перспективных сегментов рынка ИТ-услуг, что в значительной степени связано со способностью *PLM*-приложений существенно сокращать расходы на проектирование и ускорять темпы выпуска продуктов на рынок. В числе факторов, способствующих принятию решений об использовании услуг *PLM*, — высокий уровень использования ИТ в производстве, сложность производственных процессов, а также разобщенность подразделений предприятия, ответственных за конструирование, производство, сбыт и обслуживание продуктов.

Одно из наиболее полных определений *PLM* состоит из четырех пунктов:

1) стратегический подход к бизнесу, предлагающий непрерывный набор бизнес-решений, который поддерживает коллаборативный режим создания, управления, распределения и использования определения изделий (интеллектуальных активов предприятия);

2) поддержка «расширенного представления о предприятии» (*extended enterprise*), в том числе поддержка процессоров проектирования, пользователей и партнеров;

3) действие во времени от момента рождения концепции изделия до снятия его с производства и окончания сервисного периода;

4) интеграция людей, процессов, людей, систем и информации.

Важно подчеркнуть, что в этом определении *PLM* рассматривается не как часть или части технологий, а как бизнес-подход, цель

которого состоит в поиске ответов на вопросы: «Как работает бизнес?» и «Что создается?».

На основании этого определения выделяются три основные концепции *PLM*:

1) возможность универсального, безопасного и управляемого способа доступа и использования информации, определяющей изделия;

2) поддержание целостности информации, определяющей изделие, на протяжении всего его жизненного цикла изделия;

3) управление и поддержка бизнес-процессов, используемых при создании, распределении и использовании подобной информации.

Концепция *PLM* охватывает все этапы жизненного цикла изделия (рис. 5.11) и подразумевает управление данными, получаемыми от следующих компонент:

- *Product Data Management* (далее — *PDM*) — управление данными об изделиях;
- *Computer-Aided Manufacturing* (далее — *CAM*) — система автоматизированной подготовки производства;
- *Computer-Aided Engineering* (далее — *CAE*) — система автоматизированного инженерного анализа;
- *Computer-Aided Design* (далее — *CAD*) — система автоматизированного проектирования;
- *Manufacturing Process Management* (далее — *MPM*) — планирование и моделирование производства с использованием обрабатывающих центров с числовым программным управлением (ЧПУ), роботов и т. п.;
- *Bill Of material Management* (далее — *BOM*) — спецификация инженерная;
- *Customer Relationship Management* (далее — *CRM*) — управление отношениями с клиентами.

Рис. 5.11. **Этапы жизненного цикла изделия**

Одним из важнейших компонентов *PLM* является система управления данными об изделии (*PDM*), обеспечивающая обмен данными о составе изделия и вносимых в него изменениях и позволяющая создавать и поддерживать множество взаимосвязанных специфика-

ций изделия, благодаря чему пользователь получает согласованное представление о составе изделия по ходу работы над ним.

Система *PDM* должна реализовывать следующие функции.

1. *Функция управления составом изделия*, которая может быть представлена совокупностью следующих возможностей: ведение спецификаций; многоуровневые спецификации, отображающие как дерево сборки изделия, так и полный набор конструкторских, технологических и прочих атрибутов; динамический просмотр иерархически организованной информации; отслеживание принадлежности каждой детали, сборки, узла, изделия модельному ряду; определение условий применимости и отображение ограничений применимости; ведение протоколов изменения версий вплоть до версий каждой детали; отслеживание действия внесенных изменений и модификаций.

2. *Функция отслеживания ссылок на документы электронного архива*, соответствующих каждой детали, сборке, узлу, изделию. Получение данных непосредственно из электронного архива или непосредственно из САПР сборок.

3. *Функция сравнения структур изделий*, сопровождение и обслуживание информации об изделии с учетом специфики различных подразделений, включая предприятия-соисполнители (поставщики комплектующих, субподрядчики) и внешние торговые площадки.

4. *Дополнительные сервисные функции представления трехмерных данных* (геометрические электронные модели изделия, детали, сборки). Возможности визуализации в системе *PDM* не должны зависеть от типов и форматов исходных данных, что особенно актуально для предприятий, использующих разнотипные САПР. Сама визуализация должна поддерживать рендеринг, анимацию, построение сечений и разрезов, ведение комментариев на изображении и т. д.

Рассмотрим содержание каждого из этапов жизненного цикла изделия (ЖЦИ).

Концепция проекта. На основе анализа требований рынка формируется общая идея нового изделия или, что случается значительно чаще, концепция усовершенствований в проекте уже существующего продукта. Система *PLM* предоставляет информацию, которая может использоваться для анализа жизнеспособности полученной концепции.

Анализ требований рынка. Производитель должен понять, насколько востребован рынком новый продукт, и оценить выполнимость этих требований. На этом этапе система *PLM* используется для извлечения данных из различных информационных систем, которые могут способствовать получению более точной картины.

Проектирование. Конструкторы создают проект нового изделия — соответствующие САПР и *PDM*-решения являются интегральной частью *PLM*-решения. При проектировании используется вся

необходимая дополнительная информация, поставщиком которой являются *PLM*-модули, включая факторы, связанные с послепродажным обслуживанием изделия, информация о предпочтениях заказчика, данные о производственных возможностях и т. д.

Определение источников поставок (PLM-sourcing). Отдел закупок должен провести предварительную работу по поиску источников приобретения необходимых для производства изделия деталей, материалов, компонентов, оборудования и т. д. Задача систем *PLM* — предоставить достоверные данные о доступности тех или иных деталей/компонентов/материалов, их стоимости, потенциальных поставщиках и возможных альтернативных источниках.

Производство. В соответствии с определенными на этапе проектирования спецификациями и с использованием полученных на этапе поставок деталей и материалов производится продукт. Реализованные в *PLM* специальные методы контроля качества позволяют гарантировать соответствие производимого изделия заданным спецификациям.

Дистрибуция. Готовое изделие поставляется либо дистрибутору, который размещает его на своем складе до поступления соответствующего заказа, либо непосредственно заказчику. Полученные из системы *PLM* исторические данные о потребностях рынка помогают производителю свести к минимуму число уровней инвентаризации готовой продукции.

Техническая поддержка и обслуживание. На этом этапе выполняются техническое сопровождение, обслуживание и ремонт — в течение гарантийного срока или как дополнительно оплачиваемый сервис. *PLM* позволяет учесть различную информацию об изделии, поступающую на этом этапе жизненного цикла, при разработке последующих проектов и тем самым способствует повышению привлекательности продукции для клиентов.

В *PLM* доступ к данным организован на ролевой основе. Система позволяет предоставлять пользователю информацию в форме, соответствующей выполняемым им функциям в жизненном цикле изделия: трехмерные модели, схематические диаграммы, инженерные спецификации (*BOM*), календарные планы или прогнозы на основе анализа требований рынка. Этим обеспечивается работа каждого пользователя в привычной ему среде. Конструкторы и технологи в среде САПР (*CAD/CAM/CAE*), а сотрудник маркетингового подразделения сможет получить из системы представление трехмерной сборки, пригодное для размещения в рекламной брошюре. С помощью информации, которую интегрирует система *PLM*, даже не обладая специальными техническими знаниями, сотрудники отдела закупок смогут заниматься поиском нужных деталей и выбором оптимальных каналов поставки непосредственно по данным, поступающим из конструкторских подразделений.

Важным преимуществом системы *PLM*, объединяющей все структурные подразделения предприятия, является возможность решения многих проблем на этапе проектирования. В результате появляется возможность увязать проект со сложностями производственного цикла и задачами закупок, что позволит сохранить оптимальную цену и высокое качество продукции.

Другой характерной чертой *PLM* является эффективная работа с поставщиками, внешними контрагентами и смежниками за счет безбумажных форм обмена информацией.

Знания проблем технического сопровождения готовой продукции, ее гарантийного или платного обслуживания, оказывают влияние на последующие проекты. *PLM* предоставляет производителю возможность получения таких данных, их анализа и устранения выявленных проблем в следующих проектах. Это позволит удовлетворить запросы клиентов, повысить имидж и конкурентоспособность производителя.

В целом преимущества, которые дает *PLM*-решение, можно сформулировать следующим образом:

- ускорение вывода новой продукции на рынок благодаря привлечению к процессам проектирования в реальном времени всех заинтересованных участников, включая внешних поставщиков и заказчиков;
- совершенствование характеристик разрабатываемой продукции и повышение качества, обнаружение недостатков и ограничений проекта на самых ранних стадиях;
- увязка проектирования и производственных процессов: инженеры-технологи становятся интегральной частью команды проектировщиков, благодаря чему проект сразу создается с учетом специфики производственного процесса, включая тестирование, контроль качества и т. д.;
- учет и использование опыта других проектов;
- реализация новой бизнес-модели «виртуального предприятия» — к процессу проектирования и производства привлекаются поставщики, либо работы определенного этапа жизненного цикла продукции передаются на аутсорсинг внешним компаниям.

5.4. Интегрированная информационная среда управления ЖЦИ

На рис. 5.12 представлена диаграмма вариантов использования, описывающая состав и зависимости процессов ЖЦИ [37].

Общее количество объектов, используемых для описания ЖЦИ, составляет несколько тысяч. В соответствии с природой объектов условно первый уровень классификации можно представить в виде диаграммы классов, изображенной на рис. 5.13.

Рис. 5.12. **Зависимости процессов управления ЖЦИ**

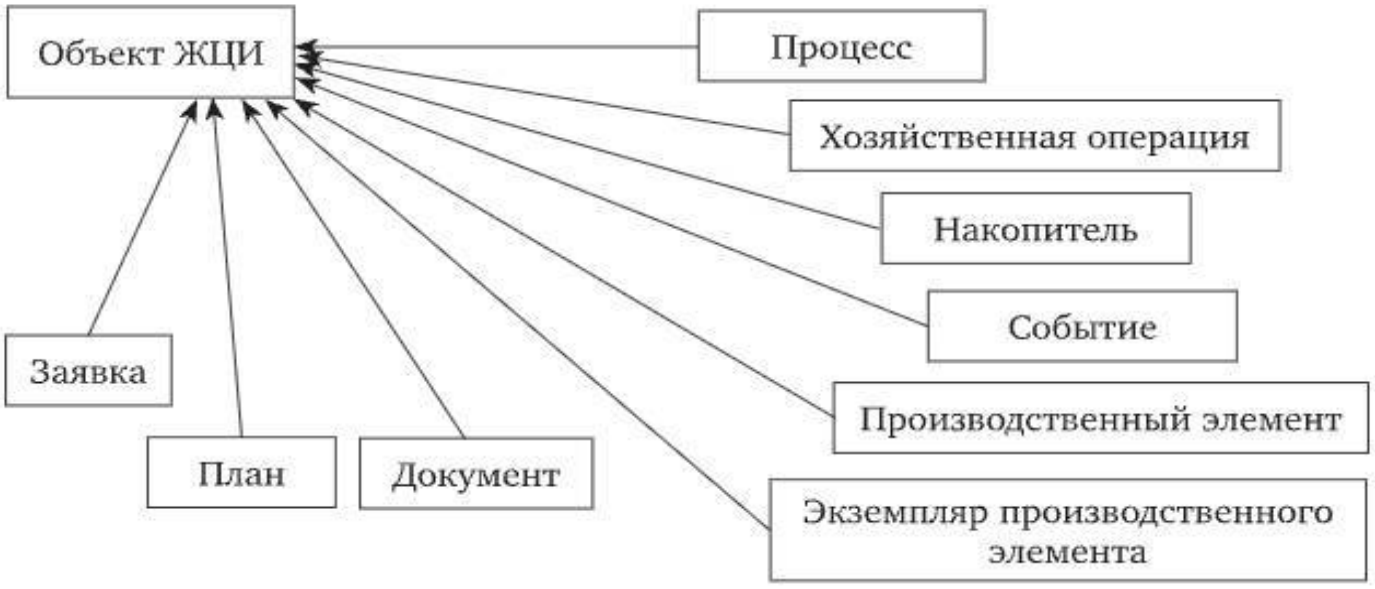


Рис. 5.13. **Классификации объектов ЖЦИ**

Сущность «*производственный элемент*» позволяет вести справочники предметов и средств труда, участвующих в ЖЦИ. Сущность «*процесс*» позволяет специфицировать процессы ЖЦИ. Сущность «*экземпляр производственного элемента*» позволяет специфицировать конкретные экземпляры производственных элементов в процессе производства и постпроизводственных стадиях ЖЦИ. Сущность «*хозяйственная операция*» позволяет описывать базовые операции с экземплярами производственных элементов. Сущность «*накопи-*

тель» позволяет решать задачи управления остатками и запасами. Сущность *«субъект хозяйственной деятельности»* позволяет специфицировать все организационные звенья, участвующие в ЖЦИ. Сущность *«событие»* специфицирует логику процессов ЖЦИ.

Сущность *«экономический элемент»* позволяет специфицировать экономику ЖЦИ. Сущность *«документ»* позволяет специфицировать документы, сопровождающие ЖЦИ. Сущность *«заявка»* позволяет специфицировать взаимоотношения субъектов хозяйственной деятельности. Сущность *«план»* позволяет специфицировать планируемые результаты по управлению ЖЦИ.

Управление жизненным циклом сложных изделий является одним из важнейших процессов деятельности современных компаний. Качество данного процесса во многом связано с созданием и поддержанием интегрированной информационной среды. Введем несколько необходимых определений. Базовым структурным компонентом интегрированной информационной среды является изделие. Построение интегрированной информационной среды основывается на построении и сопровождении информационной модели производственной деятельности, которая включает в себя, в частности, информационную модель изделия.

Информационная модель изделия (*Product Information Model*) (далее — ИМ) содержит абстрактное описание фактов и инструкций об изделии [24].

Интегрированная информационная среда (*Integrated Information Environment*) (далее — ИИС) — это совокупность распределенных баз данных, содержащих сведения об изделиях, производственной среде, ресурсах и процессах предприятия, обеспечивающая корректность, актуальность, сохранность и доступность данных тем субъектам производственно-хозяйственной деятельности (ПД), участвующим в осуществлении ЖЦИ (далее — субъекты ПД), кому это необходимо и разрешено. Все сведения (данные) хранятся в виде информационных объектов, которые составляют основу предметной области производственной деятельности информационной модели.

Информационный объект (*Information Object*) (далее — ИО) — это совокупность данных и программного кода, обладающая свойствами (атрибутами) и методами, позволяющими определенным образом обрабатывать данные. ИО является самостоятельной единицей применения и хранения в ИИС. Описание информационных объектов основывается на понятии класса и экземпляра класса.

Класс информационных объектов (*Class Of Information Objects/ Entity*) (далее — КИО) определяет свойства объектов до тех пор, пока им не присвоены конкретные значения. Класс может порождать экземпляры, наследующие его свойства и методы.

Экземпляр класса (*Instance*) — ИО, получающийся из КИО присвоением свойствам конкретных значений.

Интегрированная информационная среда должна обеспечить информационное взаимодействие между всеми участниками ПД.

Информационное взаимодействие (*Information Interaction*) — это совместное использование данных, находящихся в ИИС, и обмен данными субъектами ПД в соответствии с установленными правилами.

Главным компонентом ИИС является база интегрированных данных, включающая в себя следующие части: общая база данных об изделиях, общая база данных о предприятии.

Общая база данных об изделиях (*Common Product Database*) (далее — ОБДИ) — это часть ИИС — хранилище ИО, содержащих в произвольном формате информацию, требуемую для выпуска и поддержки технической документации, необходимой на всех стадиях ЖЦИ, для всех изделий, выпускаемых предприятием. Каждый ИО в ОБДИ идентифицируется уникальным кодом и может быть извлечен из ОБДИ для выполнения действий с ним. ОБДИ обеспечивает информационное обслуживание и поддержку деятельности:

- заказчиков (владельцев) изделия;
- разработчиков (конструкторов), технологов, управленческого и производственного персонала предприятия-изготовителя;
- эксплуатационного и ремонтного персонала заказчика.

ОБДИ может состоять из нескольких разделов: нормативно-справочного, долговременного и актуального.

Нормативно-справочный раздел ОБДИ (*Standard Andreference Section*) — это раздел ОБДИ, хранящий ИО, содержащие данные:

- о конструкционных материалах;
- нормализованных деталях (нормалях);
- нормативных документах на покупные комплектующие изделия;
- нормативных документах на детали собственного изготовления;
- нормативных документах на расчетные методы;
- государственных, международных и внутренних нормативных документов (НД);
- прочих нормативных документах.

Долговременный раздел ОБДИ (*Permanent Section*) — это раздел ОБДИ, хранящий ИО, содержащие данные, аккумулирующие собственный опыт предприятия, в том числе данные:

- о ранее выполненных готовых проектах (архив);
- типовых узлах и агрегатах собственного производства;
- типовых деталях собственного производства;
- типовых конструктивно-технологических элементах деталей;
- типовых и групповых технологических процессах;
- типовой технологической оснастке и инструменте;
- готовых и типовых расчетных методиках и математических моделях изделий собственной разработки;
- прочих готовых и типовых решениях.

Долговременный раздел ОБДИ обновляется по мере создания новых технических решений, признанных типовыми и пригодными для дальнейшего использования.

Актуальный раздел ОБДИ (*Actual Section*) — это раздел ОБДИ, хранящий ИО, содержащие данные об изделиях, находящихся на различных стадиях ЖЦИ:

- о конструкции и технологии изготовления изделий;
- конкретных экземплярах изделий в производстве;
- конкретных экземплярах изделий, находящихся на постпроизводственных стадиях ЖЦИ.

Общая база данных о предприятии (*Common Enterprise Database*) (далее — ОБДП) — это часть ИИС — хранилище ИО, содержащих в произвольном формате данные о финансово-экономическом состоянии предприятия, его внешних связях, производственно-технологической среде, действующей на предприятии системе качества и т. д.

База данных по экономике и финансам (*Economic and Finance Database*) — это раздел ОБДП, хранящий ИО, содержащие сведения:

- о конъюнктуре рынка изделий, включая цены и их динамику;
- состоянии финансовых ресурсов предприятия;
- ситуации на фондовом и финансовом рынках (курсы акций предприятия, биржевые индексы, процентные ставки, валютные курсы и т. д.);
- реальном и прогнозируемом портфеле заказов;
- финансово-экономического и бухгалтерского характера.

База данных о внешних связях предприятия (*Enterprise External Relationship Database*) — это раздел ОБДП, хранящий ИО, содержащие сведения о фактических и возможных поставщиках и заказчиках. Формируется и используется в процессе маркетинговых исследований.

База данных о производственно-технологической среде предприятия (*Enterprise Manufacturing and Technological Calenvironment Database*) — это раздел ОБДП, хранящий ИО, содержащие сведения:

- о производственной структуре предприятия;
- технологическом и контрольно-измерительном оборудовании;
- транспортно-складской системе предприятия;
- об энерговооруженности предприятия;
- о кадрах;
- прочие данные о предприятии.

База данных о системе качества (*Quality Management System Database*) — это раздел ОБДП, хранящий ИО, содержащие сведения:

- о структуре действующей на предприятии системы качества;
- действующих на предприятии стандартах по качеству;
- международных и российских НД по качеству;
- должностных инструкциях в области качества;
- прочая информация по системе качества.

Часть данных об изделии определяется как технические данные.

Данные технические (*Technical Data*) — это информация о свойствах и характеристиках технической продукции в электронной форме.

Особо следует выделить такой класс информационных объектов, как *электронный документ* (*Electronic Document*) — это информационный объект, состоящий из двух частей: реквизитной части, содержащей идентифицирующие атрибуты (имя, время и место создания, данные об авторе и т. д.) и электронную цифровую подпись; содержательной части, включающей текстовую, числовую и (или) графическую информацию, которая обрабатывается в качестве единого целого.

Технический электронный документ (*Technical Electronic Document*) — электронный документ, содержательная часть которого включает технические данные.

Тема 6
ИНСТРУМЕНТАЛЬНАЯ СРЕДА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

6.1. Программные средства информационных технологий

Программные средства информационных технологий можно разделить на две большие группы: базовые и прикладные [25].

Базовые программные средства относятся к инструментальной страте информационных технологий и включают в себя:

- ОС;
- языки программирования;
- программные среды;
- СУБД.

Прикладные программные средства предназначены для решения комплекса задач или отдельных задач в различных предметных областях.

6.1.1. Операционные системы

ОС предназначены для управления ресурсами ЭВМ и процессами, использующими эти ресурсы. В настоящее время существуют две основные линии развития ОС: *Windows* и *Unix*. Генеалогические линии данных ОС развивались следующим образом:

CP/M QDOS 86-DOS MS-DOS Windows

Multics UNIX Minix Linux

В свою очередь, каждый элемент линии имеет свое развитие, например, *Windows* развивался в такой последовательности: *Windows* 95, 98, *Me*, *NT*, 2000. Соответственно, *Linux* развивался следующим образом: версии 0.01, 0.96, 0.99, 1.0, 1.2, 2.0, 2.1, 2.1.10. Каждая версия может отличаться добавлением новых функциональных возможностей (сетевые средства, ориентация на разные процессоры, многопроцессорные конфигурации и др.).

Основные функции ОС:

- выполнение по запросу программ тех достаточно элементарных (низкоуровневых) действий, которые являются общими для большинства программ и часто встречаются почти во всех програм-

мах (ввод и вывод данных, запуск и остановка других программ, выделение и освобождение дополнительной памяти и др.);

- загрузка программ в оперативную память и их выполнение;
- стандартизованный доступ к периферийным устройствам;
- управление оперативной памятью (распределение между процессами, организация виртуальной памяти);
- управление доступом к данным на энергонезависимых носителях (таких как жесткий диск, оптические диски и др.), организованным в той или иной файловой системе;
- обеспечение пользовательского интерфейса;
- сетевые операции, поддержка стека сетевых протоколов.

Дополнительные функции ОС:

- параллельное или псевдопараллельное выполнение задач (многозадачность);
- эффективное распределение ресурсов вычислительной системы между процессами;
- разграничение доступа различных процессов к ресурсам;
- организация надежных вычислений (невозможности одного вычислительного процесса намеренно или по ошибке повлиять на вычисления в другом процессе), основана на разграничении доступа к ресурсам;
- взаимодействие между процессами: обмен данными, взаимная синхронизация;
- защита самой системы, а также пользовательских данных и программ от действий пользователей (злонамеренных или по незнанию) или приложений;
- многопользовательский режим работы и разграничение прав доступа (аутентификация, авторизация).

Таким образом, современные универсальные ОС можно охарактеризовать прежде всего как:

- использующие файловые системы (с универсальным механизмом доступа к данным);
- многопользовательские (с разделением полномочий);
- многозадачные (с разделением времени).

Многозадачность и распределение полномочий требуют определенной иерархии привилегий компонентов самой ОС. В составе ОС различают три группы компонентов:

- ядро, содержащее планировщик; драйверы устройств, непосредственно управляющие оборудованием; сетевая подсистема, файловая система;
- системные библиотеки;
- оболочка с утилитами.

Большинство программ, как системных (входящих в ОС), так и прикладных, исполняются в непривилегированном («пользовательском») режиме работы процессора и получают доступ к обо-

рудованию (и при необходимости к другим ресурсам ядра, а также ресурсам иных программ) только посредством системных вызовов. Ядро исполняется в привилегированном режиме: именно в этом смысле говорят, что ОС (точнее, ее ядро) управляет оборудованием.

В определении состава ОС значение имеет критерий операционной целостности (замкнутости): система должна позволять полноценно использовать (включая модификацию) свои компоненты. Поэтому в полный состав ОС включают и набор инструментальных средств (от текстовых редакторов до компиляторов, отладчиков и компоновщиков).

Ядро ОС — центральная часть ОС, обеспечивающая приложениям координированный доступ к ресурсам компьютера, таким как процессорное время, память, внешнее аппаратное обеспечение, внешнее устройство ввода и вывода информации, переводя команды языка приложений на язык двоичных кодов, которые понимает компьютер. Также обычно ядро предоставляет сервисы файловой системы и сетевых протоколов.

Объекты ядра ОС:

- процессы;
- файлы;
- события;
- потоки;
- семафоры;
- мьютексы;
- каналы.

Рассмотрим типы архитектур ядер ОС [38].

Монолитное ядро предоставляет богатый набор абстракций оборудования. Все части монолитного ядра работают в одном адресном пространстве. Это такая схема операционной системы, при которой все компоненты ее ядра являются составными частями одной программы, используют общие структуры данных и взаимодействуют друг с другом путем непосредственного вызова процедур.

Достоинства: скорость работы, упрощенная разработка модулей [1].

Недостатки: нарушение работоспособности всей системы от сбоя в одном из компонентов, повышенные требования к объему оперативной памяти, необходимость полной перекомпиляции ядра при изменении состава аппаратного обеспечения компьютера.

Примеры ОС с монолитными ядрами: Традиционные ядра *UNIX* (такие как *BSD*), *Linux*; ядро *MS-DOS*, ядро *KolibriOS*.

Альтернативой монолитным ядрам считаются архитектуры, основанные на микроядрах.

Модульное ядро — современная, усовершенствованная модификация архитектуры монолитных ядер операционных систем компьютеров.

В отличие от «классических» монолитных ядер, считающихся ныне устаревшими, модульные ядра, как правило, не требуют полной перекомпиляции ядра при изменении состава аппаратного обеспечения компьютера. Вместо этого модульные ядра предоставляют тот или иной механизм подгрузки модулей ядра, поддерживающих то или иное аппаратное обеспечение (например, драйверов). Все модули ядра работают в адресном пространстве ядра и могут пользоваться всеми функциями, предоставляемыми ядром. Поэтому модульные ядра продолжают оставаться монолитными. Модульность ядра осуществляется на уровне бинарного образа, а не на архитектурном уровне ядра, так как динамически подгружаемые модули загружаются в адресное пространство ядра и в дальнейшем работают как интегральная часть ядра.

Модульные ядра предоставляют особый программный интерфейс (*API*) для связывания модулей с ядром для обеспечения динамической подгрузки и выгрузки модулей. В свою очередь, не любая программа может быть сделана модулем ядра: на модули ядра накладываются определенные ограничения в части используемых функций (например, невозможность использования какой-либо стандартной библиотеки программ). Не все части ядра могут быть сделаны модулями. Некоторые части ядра всегда обязаны присутствовать в оперативной памяти и должны быть жестко «вшиты» в ядро. Также не все модули допускают динамическую подгрузку (без перезагрузки ОС). Примером может служить *Virtual File System* (*VFS* — виртуальная файловая система), совместно используемая многими модулями файловых систем в ядре *Linux*.

Микроядро — это минимальная реализация функций ядра операционной системы.

Классические микроядра предоставляют лишь очень небольшой набор низкоуровневых примитивов, или системных вызовов, реализующих базовые сервисы операционной системы.

К ним относятся:

- управление адресным пространством оперативной памяти;
- управление адресным пространством виртуальной памяти;
- управление процессами и потоками (нитями);
- средства межпроцессной коммуникации.

Классическим примером микроядерной системы является *Symbian OS*. Это пример распространенной и отработанной микроядерной (а начиная с версии *Symbian OS v*8.1, и наноядерной) операционной системы.

В отличие от *Windows NT*, создателям *Symbian OS* удалось совместить эффективность и концептуальную стройность, несмотря на то, что современные версии этой системы предоставляют обширные возможности, в том числе средства для работы с потоковыми данными, стеками протоколов, критичными к латентности ядра, графикой и видео высокого разрешения.

Разработчики *Symbian* вынесли практически все прикладные (т.е. выходящие за пределы компетенции ядра) задачи в модули-серверы, функционирующие в пользовательском адресном пространстве.

Экзоядро — ядро операционной системы компьютеров, предоставляющее лишь функции для взаимодействия между процессами и безопасного выделения и освобождения ресурсов.

В традиционных операционных системах ядро предоставляет не только минимальный набор сервисов, обеспечивающих выполнение программ, но и большое количество высокоуровневых абстракций для использования разнородных ресурсов компьютера: оперативной памяти, жестких дисков, сетевых подключений. В отличие от них, ОС на основе экзоядра предоставляет лишь набор сервисов для взаимодействия между приложениями, а также необходимый минимум функций, связанных с защитой: выделение и высвобождение ресурсов, контроль прав доступа и т. д. Экзоядро не занимается предоставлением абстракций для физических ресурсов — эти функции выносятся в библиотеку пользовательского уровня (так называемую *libOS*).

Основная идея операционной системы на основе экзоядра состоит в том, что ядро должно выполнять лишь функции координатора для небольших процессов, связанных только одним ограничением, — экзоядро должно иметь возможность гарантировать безопасное выделение и освобождение ресурсов оборудования. В отличие от ОС на основе микроядра, ОС, базирующиеся на экзоядре, обеспечивают гораздо большую эффективность за счет отсутствия необходимости в переключении между процессами при каждом обращении к оборудованию.

Архитектуры на основе экзоядер являются дальнейшим развитием и усовершенствованием микроядерных архитектур и одновременно ужесточают требования к минималистичности и простоте кода ядра.

LibOS может обеспечивать произвольный набор абстракций, совместимый с той или иной уже существующей операционной системой, например *Linux* или *Windows*.

Гибридное ядро — модифицированные микроядра (минимальная реализация основных функций ядра операционной системы компьютера), позволяющие для ускорения работы запускать «несущественные» части в пространстве ядра.

Все рассмотренные подходы к построению операционных систем имеют свои достоинства и недостатки. В большинстве случаев современные операционные системы используют различные комбинации этих подходов. Так, например сейчас, ядро «Linux» представляет собой монолитную систему с отдельными элементами модульного ядра.

Наиболее тесно элементы микроядерной архитектуры и элементы монолитного ядра переплетены в ядре *Windows NT*. Хотя *Windows NT* часто называют микроядерной операционной системой, это не совсем так. Микроядро *NT* слишком велико (более 1 Мбайт, кроме того, в ядре системы находится, например, еще и модуль графического интерфейса), чтобы носить приставку «микро». Компоненты ядра *Windows NT* располагаются в вытесняемой памяти и взаимодействуют друг с другом путем передачи сообщений, как и положено в микроядерных операционных системах. В то же время все компоненты ядра работают в одном адресном пространстве и активно используют общие структуры данных, что свойственно операционным системам с монолитным ядром. Причина проста: чисто микроядерный дизайн коммерчески невыгоден, поскольку неэффективен.

Таким образом, *Windows NT* можно с полным правом назвать гибридной операционной системой.

В процессе эволюции сформировались и были реализованы основные идеи, определяющие функциональность ОС: пакетный режим, разделение времени и многозадачность, разделение полномочий, реальный масштаб времени, файловые структуры и файловые системы.

6.1.2. Языки программирования

Большинство алгоритмических языков программирования (Си, Паскаль) созданы на рубеже 1960 и 1970-х гг. (за исключением *Java*). За прошедший период времени периодически появлялись новые языки программирования, однако на практике они не получили широкого и длительного по времени распространения. Другим направлением в эволюции современных языков программирования были попытки создания универсальных языков программирования (Алгол, *PL/1*, Ада), объединявших в себе достоинства ранее разработанных языков.

Появление ПК и ОС с графическим интерфейсом (*Mac OS*, *Windows*) привело к смещению внимания разработчиков ПО в сферу

визуального или объектно-ориентированного программирования, сетевых протоколов, баз данных. В результате в настоящее время в качестве инструментальной среды используется конкретная среда программирования (*Delphi*, *Access* и др.) и знания базового языка программирования не требуется. Поэтому можно считать, что круг используемых языков программирования стабилизировался.

Анализ синтаксиса и семантики языков программирования показывает, что их родственные конструкции различаются главным образом «внешним видом» (набором ключевых слов или порядком следования компонентов). Содержимое практически идентично, за исключением небольших различий, не имеющих существенного значения. Таким образом, конструкции современных языков имеют общее содержание (семантику), различный порядок следования компонент (синтаксис) и разные ключевые слова (лексику). Следовательно, различные языки предоставляют пользователю одинаковые возможности при различном внешнем виде программ.

Стандартизацию языков программирования в настоящее время осуществляют комитеты *ISO/ANSI*, однако их деятельность направлена в основном на неоправданное синтаксическое расширение языков. Для исключения существующих недостатков предложены способы задания семантического и синтаксического стандарта языков программирования.

Семантическое описание любой конструкции языка (оператора, типа данных, процедуры и т. д.) должно содержать не менее трех обязательных частей:

- список компонент (в Тип Указатель это компоненты Имя Типа и Базовый Тип);
- описание каждой компоненты;
- описание конструкции в целом.

Для синтаксического описания обычно используется формальное описание конструкции, например, в виде БНФ (Бэкуса — Наура форма, формальная система описания синтаксиса языка программирования). Синтаксическое описание присутствует в любом языке, начиная с Алгола.

Со времени создания первых программируемых машин человечество придумало более 8500 языков программирования. Каждый год их число пополняется новыми. Ввиду их большого количества классификация затруднена. В общем плане можно выделить две группы: языки высокого уровня и языки низкого уровня.

Языки высокого уровня — максимально приближены к задаче. Наиболее выражено в предметно-ориентированных языках. Приоритет: что?

Языки низкого уровня — в центре внимания не задача, а технология ее реализации, связанная с языком/машиной. Привлекаются дополнительные понятия, не связанные с задачей. Приоритет: как?

Некоторыми языками умеет пользоваться небольшое количество их собственных разработчиков, другие становятся известны миллионам людей.

Профессиональные программисты иногда применяют в своей работе более десятка разнообразных языков программирования.

Среди большого количества языков программирования самую заметную роль в развитии программирования сыграли три пары: Алгол-60 и Фортран, Паскаль и СИ, *Java* и *C++*. Эти языки не случайно объединены в пары, так как в противостоянии заложенных в них идей происходило прогрессивное развитие.

В табл. 6.1 приведены основные сведения о наиболее распространенных языках, а в табл. 6.2 — о языках специального назначения (экспериментальных и промышленных) [39]. Виды (парадигмы) языков по областям применения:

A — процедурное программирование;
B — объектно-ориентированное программирование;
C — структурное программирование;
D — модульное (компонентное программирование);
E — логическое (реляционное) программирование;
F — функциональное программирование;
G — параллельное программирование;
H — гибрид (смесь парадигм *B+C+D+G*);
I — специализированный язык.

Таблица 6.1

Сведения о наиболее распространенных языках

Название	Год создания	Вид	Фирма	Стандарт
Фортран (*Fortran*)	1954	A	IBM	ISO 1539:1997
Лисп (*Lisp*)	1958	F	MIT	—
Алгол-60 (*Algol* 60)	1960	A	IFIP	—
Кобол (*COBOL*)	1960	A	COLASYL Commitete	ISO 1989:1985
Симула (*Sumula*)	1962	B	—	—
Бейсик (*Basic*)	1963	A	Darmouth College	ISO 10279:1991
ПЛ/1 (*PL/1*)	1964	A	IBM	ISO 6160:1979
Алгол-68 (*Algol* 68)	1968	A	IFIP	—
Паскаль (*Pascal*)	1970	C	ETH	ISO 7185:1990
Форт (*Forth*)	1970	A*	Mohasco Industrie	ISO 15145:1997
Си (*C*)	1972	C*	AT&T Bell Labs	ISO 9899:1999

Окончание табл. 6.1

Название	Год создания	Вид	Фирма	Стандарт
Smalltalk	1972	B	Xerox PARC	—
Пролог (*Prolog*)	1973	E	Univ. of Aix-Marseille	ISO 13211:1995
Ада (*Ada*)	1980	H*	Cil Honewell	ISO 8652:1995
C++	1984	H*	AT&T Bell Labs	ISO 14882:1998
Java	1995	H	Sun Labs	—

Таблица 6.2

Сведения о языках специального назначения

Наименование	Год создания	Вид	Фирма	Стандарт
АПЛ (*APL*)	1957	*I*	*Harvard Univ.*	*ISO* 8485:1989
Снобол (*SNOBOL*)	1962	*I*	*AT&T Bell Labs*	—
Сетл (*SETL*)	1969	*I*	*IBM*	—
Параллельный Паскаль (*Concurrent Pascal*)	1974	*G*	*CIT*	—
CLU	1974	*D*	*MIT*	—
Schema	1975	*F*	*MIT*	—
Mesa	1976	*D**	*Xerox PARC*	—
Icon	1977	*I*	*AT&T Bell Labs*	—
Модула-2 (Modula-2)	1979	*D**	*ETH*	*ISO* 10514:1996
Оккам (*Occam*)	1982	*G**	*Inmos*	—
Cedar	1983	*H**	*Xerox PARC*	—
Common Lisp	1984	*F*	*MIT*	—
Objective C	1984	*H**	*Productivity Products*	—
Эйфель (*Eiffel*)	1986	*D**	*ISE*	—
Оберон (*Oberon*)	1988	*D**	*ETH*	—
Модула-3 (*Modula*-3)	1988	*H**	*DEC SRC*	—
Оберон 2 (*Oberon*-2))	1991	*D**	*ETH*	—
Limbo	1996	*D**	*Bell Labs (Lucent)*	—

Окончание табл. 6.2

Наименование	Год создания	Вид	Фирма	Стандарт
Component Pascal	1997	*D**	Оберон *Microsystems*	—
C#	2000	*H**		—

Сокращения: MIT — Massachusetts Institute of Technology; *PARC* — Palo Alto Research Center; *ETH* — Swiss Federal Institute of Technology; *SRC* — Systems Research Center; *ISE* — Interactive Software Engineering; *ISO* — International Standard Organization; *CIT* — California Institute of Technology; * — Поддержка системного программирования

Важно различать язык программирования и его реализацию. *Сам язык* — это система записи, набор правил, определяющих синтаксис и семантику программы. *Реализация языка* — это программа, которая преобразует запись высокого уровня в последовательность машинных команд. Существует два способа реализации языка: компиляция (рис. 6.1) и интерпретация (рис. 6.2).

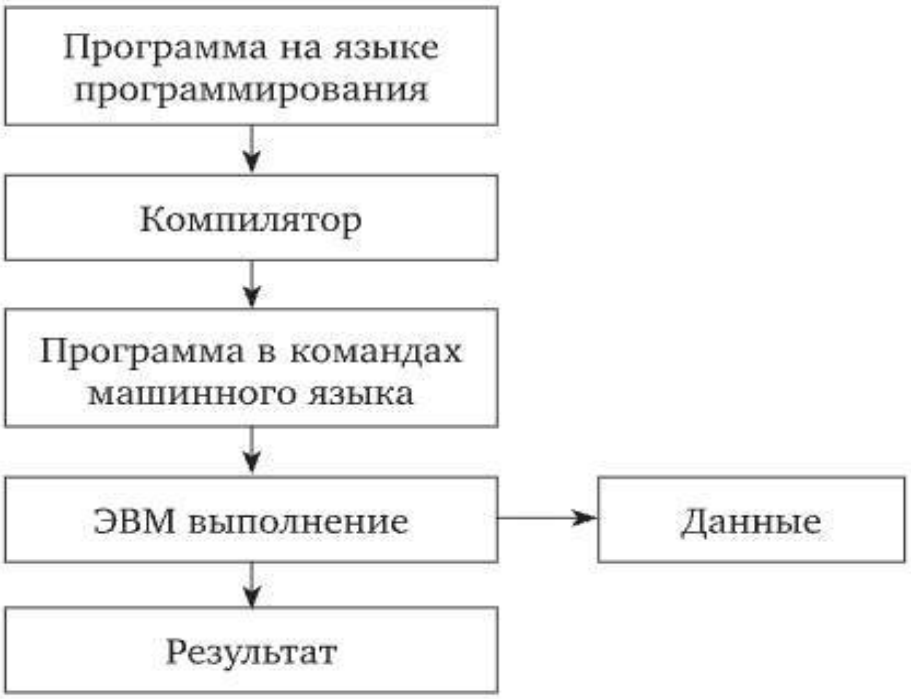


Рис. 6.1. **Схема компиляции**

При компиляции специальная рабочая программа (компилятор) осуществляет перевод рабочей программы в эквивалентную на машинном коде и в дальнейшем ее выполнение совместно с данными. В методе интерпретации специальная программа (интерпретатор) устанавливает соответствие между языком и машинными кодами, применяя команды к данным. В принципе, любой язык программирования может быть как интерпретируемым, так и компилируемым, но в большинстве случаев есть свой предпочтительный способ реализации. К сожалению, в настоящее время не существу-

ет «универсального» компилятора, который мог бы работать с любым существующим языком. Это объясняется отсутствием единой семантической базы. Хотя современные языки программирования похожи друг на друга, идентичность их далеко не полная.

Рис. 6.2. **Схема интерпретации**

На рис. 6.3 представлены области пересечения и объединения языков программирования.

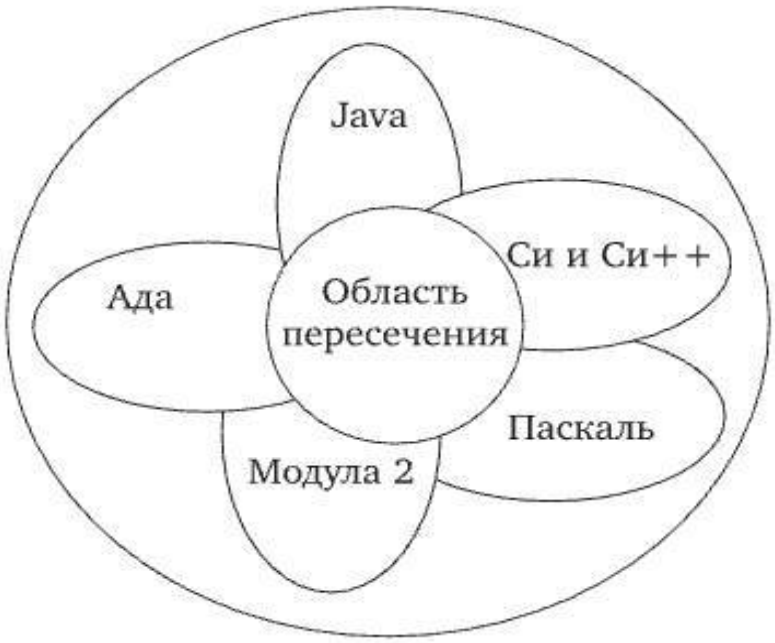


Рис. 6.3. **Области пересечения и объединения языков программирования**

Таким образом, существует общая семантическая зона, в которую входят конструкции, принадлежащие всем языкам программирования (или большинству из них), и область объединения, содержащая конструкции специфические для данного языка. Поэтому создание «универсального» компилятора возможно двумя путями:

1. Использование общих конструкций (область пересечения), отбрасывая специфические конструкции языков (область объединения). Это приведет к «обеднению» всех языков программирования.
2. Использование всех имеющихся конструкций (область объединения + область пересечения). Такой подход приведет к значи-

тельному расширению семантической базы использованию дополнительных ресурсов.

Многие годы идет спор о том, что такое программирование — наука, искусство или производственный процесс. Надо признать, что право на существование имеют все три версии. Однако в связи с появлением информационных технологий на первый план выходит промышленный характер программирования, который соответствует традиционным стадиям жизненного цикла программного продукта, изложенным в параграфе 3.7.

Однако наряду с этим направлением развивается так называемое исследовательское программирование. Например, предложенное Э. Раймондом самоорганизующееся, анархичное программирование, получившему название «базар». Отличительными чертами его являются отсутствие четкого плана, минимальное управление проектом, большое число сторонних территориально удаленных разработчиков, свободный обмен идеями и кодами.

Программные среды реализуют отдельные задачи и операции информационных технологий. К их числу относятся:

1) текстовые процессоры: *Microsoft Word*, Лексикон, *Lotus Word Perfect*, *Corel Word Pro*, *Sun Star Office Writer* и др.;

2) электронные таблицы: *Microsoft Excel*, *Corel Quattro Pro*, *Lotus* 1-2-3, *Sun Star Office Calc* и др.;

3) личные информационные системы: Microsoft Outlook, Lotus Organizer, Lotus Notes, Sun Star Office Schedule и др.;

4) программы презентационной графики: *Microsoft Power Point*, *Lotus Freelance Graphics*, *Corel Presentations*, *Sun Star Office Impress* и др.;

5) браузеры: *Microsoft Internet Explorer*, *Netscape Navigator*, *Opera* и др.;

6) редакторы веб-страниц: *Microsoft Front Page*, *Netscape Composer*, *Macromedia Free Hand* и др.;

7) почтовые клиенты: *Microsoft Outlook*, *Microsoft Outlook Express*, *Netscape Messenger*, *The Bat* и др.;

8) редакторы растровой графики: *Adobe Photoshop*, *Corel Photo-Paint* и др.;

9) редакторы векторной графики: *Corel Draw*, *Adobe Illustrator* и др.

10) настольные издательские системы: *Adobe Page Maker*, *Quark Xpress*, *Corel Ventura*, *Microsoft Publisher* и др.;

11) средства разработки: *Borland Delphi*, *Microsoft Visual Basic*, *Borland C++ Builder*, *Microsoft Visual C++* и др.

6.2. Технические средства информационных технологий

Основу технического обеспечения информационных технологий составляют компьютеры, являющиеся ядром любой информацион-

ной системы. Первоначально компьютеры были созданы для реализации большого объема вычислений, представляющих длинные цепочки итераций. Главным требованием при этом были высокая точность и минимальное время вычислений. Такие процессы характерны для числовой обработки.

По мере внедрения ЭВМ, их эволюционного развития, в частности создания персональных компьютеров, широко стали возникать другие области применения, отличные от вычислений, например, обработка экономической информации, создание информационно-справочных систем, автоматизация учрежденческой деятельности и т. п. В данном случае не требовались высокая точность и большой объем вычислений, однако объем обрабатываемой информации мог достигать миллионов и миллиардов записей. При этом требовалось не только обработать информацию, а предварительно ее найти и организовать соответствующую процедуру вывода. Указанные процессы характерны для нечисловой обработки, требующей в большинстве случаев больших затрат машинного времени. Рассмотренные аспекты оказали решающее влияние на развитие архитектуры ЭВМ.

ЭВМ классической (*фоннеймановской*) архитектуры состоит из пяти основных функциональных блоков (рис. 6.4):

1) запоминающего устройства (далее — ЗУ);
2) устройства управления;
3) устройств управления и арифметически-логического, рассматриваемых вместе и называемых центральным процессором;
4) устройства ввода;
5) устройства вывода.

Рис. 6.4. **Фоннеймановская архитектура ЭВМ**
—— — управление; ----- — данные.

В фоннеймановской архитектуре для обработки огромного объема информации (миллиарды байтов) используется один процессор.

Связь с данными осуществляется через канал обмена. Ограничения пропускной способности канала и возможностей обработки в центральном процессоре приводят к тупиковой ситуации при нечисловой обработке в случае увеличении объемов информации. Для выхода из тупика было предложено два основных изменения в архитектуру ЭВМ:

- использование параллельных процессоров и организация параллельной обработки;
- распределенная логика, приближающая процессор к данным и устраняющая их постоянную передачу.

Другой недостаток фоннеймановской архитектуры связан с организацией процесса обращения к ЗУ, осуществляемого путем указания адреса для выборки требуемого объекта из памяти. Это приемлемо для числовой обработки, но при нечисловой обработке обращение должно осуществляться по содержанию (ассоциативная адресация). Поскольку для нечисловой обработки в основном используется та же архитектура, необходимо было найти способ организации ассоциативного доступа. Он осуществляется путем создания специальных таблиц (справочников) для перевода ассоциативного запроса в соответствующий адрес. При такой организации обращения к ЗУ, называемом эмуляцией ассоциативной адресации, в случае работы с большими объемами информации резко падает производительность ЭВМ. Это связано с тем, что нечисловая обработка — это не только просмотр, но и обновление данных.

Для преодоления ограничений организации памяти были предложены ассоциативные запоминающие устройства.

Таким образом, ЭВМ для нечисловой обработки должна удовлетворять следующим требованиям: ассоциативность, параллелизм, обработка в памяти. Кроме этого, на более высоком уровне к архитектуре предъявляются следующие требования:

- перестраиваемость параллельных процессоров и запоминающих устройств;
- сложные топологии соединений между процессорами;
- мультипроцессорная организация, направленная на распределение функций.

Перечисленные выше ограничения и требования были реализованы в МБД.

Подытоживая вышесказанное, приведем *классификацию архитектур ЭВМ*:

- архитектура с одиночным потоком команд и одиночным потоком данных (*SISD*);
- архитектура с одиночным потоком команд и множественным потоком данных (*SIMD*);
- архитектура с множественным потоком команд и одиночным потоком данных (*MISD*);

- архитектура с множественным потоком команд и множественным потоком данных (*MIMD*).

К разряду *SISD* относятся современные фоннеймановские однопроцессорные системы. В этой архитектуре центральный процессор работает с парами атрибут-значение. Атрибут (метка) используется для локализации соответствующего значения в памяти, а одиночная команда, обрабатывающая содержимое накопителя (регистра) и значение, выдает результат. В каждой итерации из входного потока данных используется только одно значение.

К классу *SIMD* относят большой класс архитектур, основная структура которых состоит из одного контроллера, управляющего комплексом одинаковых процессоров. В зависимости от возможностей контроллера и процессорных элементов, числа процессоров, организации поиска и характеристик маршрутных и выравнивающих сетей выделяют четыре типа *SIMD*.

Матричные процессоры, организованные так, что при выполнении заданных вычислений, инициированных контроллером, они работают параллельно. Предназначены для решения векторных и матричных задач, относящихся к числовой обработке.

Ассоциативные процессоры, обеспечивающие работу в режиме поиска по всему массиву, за счет соединения каждого процессора непосредственно с его памятью. Используются для решения нечисловых задач.

Процессорные ансамбли, представляющие совокупность процессоров, объединенных определенным образом для решения заданного класса задач, ориентированных на числовую и нечисловую обработку.

Конвейерные процессоры (последовательные и векторные), осуществляющие выполнение команд и обработку потоков данных по принципу, аналогичному транспортному конвейеру. В этом случае каждый запрос использует одни и те же ресурсы. Как только некоторый ресурс освобождается, он может быть использован следующим запросом, не ожидая окончания выполнения предыдущего. Если процессоры выполняют аналогичные, но не тождественные задания, то это последовательный конвейер, если все задания одинаковы — векторный конвейер.

К классу *MISD* может быть отнесена единственная архитектура, конвейер, но при условии, что каждый этап выполнения запроса является отдельной командой.

К разряду *MIMD*, хотя и не всегда однозначно, относят следующие конфигурации:

- мультипроцессорные системы;
- системы с мультиобработкой;
- вычислительные системы из многих машин;
- вычислительные сети.

Общим для данного класса является наличие ряда процессоров и мультиобработки. В отличие от параллельных матричных систем, число процессоров невелико, а термин «мультиобработка» понимается в широком смысле для обозначения функционально распределенной обработки (сортировки, слияния, ввода-вывода и др.).

Другим направлением развития вычислительной техники является *нейрокомпьютеринг*, основанный на нейронных сетях. Разработки проводятся в двух направлениях: аппаратном и программном. Нейрокомпьютеры обладают сверхвысокой производительностью, но благодаря сложным технологиям имеют очень высокую стоимость. Поэтому они используются узким кругом пользователей для решения суперзадач.

В последние годы ведутся работы по созданию биокомпьютера на основе молекулярных технологий. Идея молекулярного вычислителя состоит в представлении «машинного» слова в виде состояний молекул.

На данное время существуют следующие классы технических средств информационных технологий:

- персональные компьютеры;
- мобильные (носимые) ПК;
- нестандартные конструкции ПК;
- мейнфреймы;
- нейрокомпьютеры;
- системы для облачных вычислений;
- суперкомпьютеры;
- вычислительный кластер.

6.2.1. Персональные компьютеры

Подавляющее большинство людей используют именно ПК в качестве средства обработки и хранения информации. К ПК на данное время можно отнести стационарные, планшетные, карманные компьютеры и ноутбуки.

Несмотря на то что ЭВМ изначально были рассчитаны на различные вычисления, сейчас бóльшая часть людей использует их для доступа к информационным сетям или играм. Вообще, сам термин подразумевает собой любую ЭВМ, используемую в личных целях, будь то суперкомпьютер или калькулятор.

Стационарные ПК. Первые персональные компьютеры являлись стационарными в виду их объема и веса, определяемыми конструктивными особенностями и элементной базой. Существует два направления развития стационарных компьютеров: раздельная схема и моноблок.

Раздельная схема предполагает, что ПК состоит из отдельных, конструктивно завершенных и самостоятельных частей, к которым относятся системный блок и разнообразные внешние устройства,

подключаемые к системному блоку извне через стандартные интерфейсы (например, *USB*, *D-Sub*, *DVI*, *FireWire*) стандартными интерфейсными кабелями. Состав внешних устройств зависит от функциональных требований и может включать мониторы, клавиатуру, мышь, микрофоны, звуковые колонки, веб-камеры, принтеры, сканеры, различные внешние модемы и игровые устройства.

Несмотря на прогресс в области миниатюризации, раздельная схема остается самой распространенной формой стационарных ПК. Например, профессиональные рабочие станции практически всегда строятся по такой схеме.

Функциональным ядром в раздельной схеме стационарного ПК является системный блок, имеющий два вида конструктивной компоновки:

1) *desktop* — горизонтальная конструктивная компоновка системного блока с возможностью размещения монитора на таком системном блоке;

2) *tower* — вертикальная конструктивная компоновка, называемая «башенной».

Достоинствами раздельной схемы являются:

- сравнительно легкая масштабируемость, определяемая простотой замены и дополнительного подключения любой компоненты;
- простота модернизации.

Недостаток определяется трудностями транспортирования и относительной громоздкости ПК. Отсюда область применения раздельной схемы — стационарное размещение для решения задач, привязанных к месту расположения ПК, например, диспетчерская служба.

Моноблок. Появление данного вида стационарного компьютера связано с требованиями эргономики и минимизации используемого пространства. Конструктивно схема моноблока объединяет системный блок, монитор, микрофон, звуковые колонки и веб-камеру в одно устройство.

Достоинства моноблока:

- унификация используемых инструментальных средств;
- более привлекательные эргономические и эстетические решения;
- более высокая транспортабельность.

К числу недостатков относятся сложности самостоятельной технической модернизации и масштабируемости.

Область применения моноблоков — офисные организации с большим количеством пользователей, выполняющих однотипную работу.

Мобильные (носимые) ПК

Достижения в области миниатюризации элементной базы позволили решить задачу мобильности ПК и сделать его в полной мере

индивидуальным. Среди мобильных ПК выделяют следующие разновидности: ноутбуки, планшетные ПК, карманные ПК, смартфоны.

Ноутбук (англ. *notebook* — блокнот, портативный ПК) — переносной персональный компьютер, в корпусе которого, складывающемся в виде книжки, объединены типичные компоненты ПК: дисплей, клавиатура, устройство указания (сенсорная панель, или тачпад), обеспечивающая автономность, аккумуляторная батарея, развитые средства подключения к проводным и беспроводным сетям, встроенное мультимедийное оборудование (динамики, микрофон и веб-камера).

Другой широко применяемый к ноутбукам термин — «лэптоп» (англ. *laptop*: *lap* — колени сидящего человека, *top* — верх).

Реализация ноутбуков стала возможной, благодаря внедрению специальных технологий: специализированные микросхемы *ASIC* (англ. *application-specific integrated circuit* — интегральная схема специального назначения), минимизированные ОЗУ и жесткие диски, компактная клавиатура, не содержащая цифрового поля.

Достоинствами ноутбуков являются:

- мобильность и автономность;
- возможность подключения к беспроводным сетям.

К недостаткам можно отнести:

- недостаточно высокую производительность;
- ограниченность модернизации;
- проблемы совместимости с различными операционными системами.

Планшетный ПК (англ. *tablet PC*) — являются следующим этапом развития ноутбуков, направленным на повышение мобильности и миниатюризации. Для этого класса ПК характерны следующие главные особенности:

- аппаратная совместимость с IBM PC-компьютерами и установленными на них операционными системами (*Microsoft Windows NT* (*Windows XP Tablet PC Edition*, *Windows* 7, *Windows* 8, *Windows* 10), *Apple Mac OS X*, *Linux*));
- ввод текста и управление через экранный интерфейс, реализуемый на основе электромагнитного принципа (с помощью стилуса (палочки) или сенсора (с помощью пальца)).

Достоинствами планшетных ПК являются:

- высокая мобильность;
- распознавание рукописного текста, написанного на экране.

Основная функция применения планшетных компьютеров — оперативный доступ к информационным ресурсам вне рабочего места.

Карманные персональные компьютеры (КПК) (англ. *Personal Digital Assistant*, *PDA* — личный цифровой секретарь) построены на основе сверхэкономичных процессоров и флеш-накопителей небольшого объема. Несмотря на ограниченные вычислительные

мощности содержат все признаки ПК: процессор, накопитель, оперативную память, монитор, операционную систему и прикладное ПО. Управление происходит с помощью небольшого по размерам и разрешению экрана, чувствительного к нажатию пальца, или специальной палочки-указки — стилуса.

Основные функциональные возможности КПК определяют область использования — электронный органайзер. К ним относятся:

- чтение текстов;
- электронная почта и Интернет;
- карты местности;
- звуковой проигрыватель, диктофон;
- записи от руки, создание текстов и изображений;
- дистанционное управление бытовой техникой;
- офисные приложения;
- управленческий учет;
- программирование.

Основным недостатком традиционных КПК является отсутствие мобильной связи, что приводит к их вытеснению коммуникаторами со встроенным коммуникационным модулем, обеспечивающим мобильную связь и подключение к Интернету в любой точке, где есть сотовая связь совместимого стандарта (GSM/GPRS/3G/4G, CDMA, LTE). Наибольшее распространение получили смартфоны.

Коммуникатор (англ. *communicator*, *PDA phone*) — КПК, дополненный функциональностью мобильного телефона

Смартфон (англ. *smartphone* — умный телефон) — мобильный телефон, дополненный функциональностью КПК. Отличительной особенностью смартфонов является наличие развитой операционной системы, открытой для разработки программного обеспечения пользователями (операционная система обычных мобильных телефонов закрыта для сторонних разработчиков). Это позволяет устанавливать дополнительные приложения, улучшающие функциональные возможности смартфона.

В настоящее время разделение на коммуникаторы и смартфоны условное, оба термина обозначают одно и то же.

6.2.2. Нестандартные конструкции ПК

Barebone (англ. *bare* — голый и *bone* — кость) — компьютер, строящийся пользователем для выполнения определенных задач (обычно в качестве мультимедийной станции) и собираемый на основе каркасной (скелетной) системы, называемой баребон-основой.

Материнская плата, как правило, оснащена встроенными звуковым и видеоконтроллерами. Выбор конфигурации и, соответственно, комплектующих в виде дисковых накопителей, памяти и периферии, а также других устройств (ТВ-тюнера, дополнительной видеокарты и т. п.) остается на усмотрение пользователя. Как пра-

вило, «баребоны» имеют меньшую высоту корпуса и, как следствие, уменьшенный внутренний объем, а также усовершенствованную систему охлаждения, отличающуюся низкой шумностью.

Защищенные ПК — компьютеры, обладающие устойчивостью к агрессивным средам: сильной вибрации, ударам, большой запыленности, влажности, вандализму и другим воздействиям окружающей среды. Основная сфера применения связана с тяжелыми условиями эксплуатации: военное дело, геология, космос и др. Устойчивые ПК, как правило, выпускаются в формате ноутбуков, конструктивно более тяжелых и больших по размерам, чем обычные.

Промышленные ПК (англ. *Industrial PC, IPC*) предназначены для решения задач промышленной автоматизации. Как и защищенные ПК, обладают стойкостью к неблагоприятным внешним условиям, а также увеличенным жизненным циклом изделия, возможностью подключения к промышленным сетям. Отличительными особенностями являются совместное использование с ПО, автоматизация управляемого процесса, возможность встраивания в промышленное оборудование. Выпускаются серийно в виде коммерчески доступного изделия.

Тихий (бесшумный) ПК предназначен для профессиональной деятельности (работа со звуком или видео) и для личного использования (музыкальные центры, офисные компьютеры), когда необходимо соблюдать тишину. Чтобы сделать ПК тихим, используется специальные бесшумные технологии; в таких ПК вентилятор полностью отсутствует.

Одним из направлений использования тихих ПК является режим постоянного включенного ПК, когда он может выполнять ряд функций:

- мультимедийная станция (воспроизведение и запись видео- и аудиоинформации, интернет-радио);
- P2P-клиент (англ. *peer-to-peer, P2P* — один на один, с глазу на глаз) нужен для обмена файлами с другими компьютерами в пиринговых сетях в автоматическом режиме. В таких сетях отсутствуют выделенные серверы, а каждый узел (*peer*) является как клиентом, так и сервером;

Компактные ПК. Технологии, уменьшающие габариты ПК:

- материнская плата уменьшенного формата (*mini*-ITX и др.);
- малогабаритный корпус;
- встроенные *DVD*- и *CD*-дисководы со щелевой загрузкой или отсутствие встроенного *DVD*- и *CD*-дисковода;
- меньшее количество отсеков для жестких дисков и *DVD*-, *CD*-дисководов; зачастую всего один;
- меньше гнезд *USB*, аудио- и т. д.;
- внешние блоки питания;
- использование внешних (как, например, *DVD*- и *CD*-дисководы) устройств вместо встроенных.

6.2.3. Мейнфреймы

Термин «мейнфрейм» (также «мейнфрейм», от англ. *mainframe*) имеет три основных значения [40]:

- большая универсальная ЭВМ — высокопроизводительный компьютер со значительным объемом оперативной и внешней памяти, предназначенный для организации централизованных хранилищ данных большой емкости и выполнения интенсивных вычислительных работ;
- компьютер с архитектурой *IBM System/360, 370, 390, zSeries*;
- наиболее мощный компьютер, используемый в качестве главного или центрального компьютера (например, в качестве главного сервера).

Рассмотрим особенности и характеристики современных мейнфреймов.

Среднее время наработки на отказ. Время наработки на отказ современных мейнфреймов оценивается в 12—15 лет. Надежность мейнфреймов — это результат их почти 60-летнего совершенствования. Группа разработки операционной системы *VM/ESA* затратила 20 лет на удаление ошибок, и в результате была создана система, которую можно использовать в самых ответственных случаях.

Повышенная устойчивость систем. Мейнфреймы могут изолировать и исправлять большинство аппаратных и программных ошибок за счет использования следующих принципов.

Дублирование: два резервных процессора, резервные модули памяти, альтернативные пути доступа к периферийным устройствам.

Горячая замена всех элементов вплоть до каналов, плат памяти и центральных процессоров.

Целостность данных. В мейнфреймах используется память с коррекцией ошибок. Ошибки не приводят к разрушению данных в памяти, или данных, ожидающих вывода на внешние устройства. Дисковые подсистемы, построенные на основе *RAID*-массивов с горячей заменой и встроенных средств резервного копирования, защищают от потерь данных.

Рабочая нагрузка. Рабочая нагрузка мейнфреймов может составлять 80—95 % от их пиковой производительности. Операционная система мейнфрейма будет обрабатывать все сразу, причем все приложения будут тесно сотрудничать и использовать общие компоненты ПО.

Пропускная способность. Подсистемы ввода-вывода мейнфреймов разработаны так, чтобы работать в среде с высочайшей рабочей нагрузкой на ввод-вывод данных.

Масштабирование. Масштабирование мейнфреймов может быть как вертикальным, так и горизонтальным. *Вертикальное масштабирование* обеспечивается линейкой процессоров с производительностью от 5 до 200 *MIPS* и наращиванием до 12 центральных процессоров

в одном компьютере. *Горизонтальное масштабирование* реализуется объединением ЭВМ в *Sysplex* (*System Complex*) — многомашинный кластер, выглядящий с точки зрения пользователя единым компьютером. Всего в *Sysplex* можно объединить до 32 машин. Географически распределенный *Sysplex* называют *GDPS*. В случае использования операционной системы *VM* для совместной работы можно объединить любое количество компьютеров. *Программное масштабирование* — на одном мейнфрейме может быть сконфигурировано фактически бесконечное число различных серверов. Причем все серверы могут быть изолированы друг от друга так, как будто они выполняются на отдельных выделенных компьютерах и в то же время совместно использовать аппаратные и программные ресурсы и данные.

Доступ к данным. Поскольку данные хранятся на одном сервере, прикладные программы не нуждаются в сборе исходной информации из множества источников, не требуется дополнительного дискового пространства для их временного хранения, не возникают сомнения в их актуальности. Требуется небольшое количество физических серверов и значительно более простое программное обеспечение. Все это, в совокупности, ведет к повышению скорости и эффективности обработки.

Защита. Встроенные в аппаратуру возможности защиты, такие как криптографические устройства, и *Logical Partition*, и средства защиты операционных систем, дополненные программными продуктами *RACF* или *VM:SECURE*, обеспечивают надежную защиту.

Пользовательский интерфейс. Пользовательский интерфейс у мейнфреймов всегда оставался наиболее слабым местом. Сейчас же стало возможно для прикладных программ мейнфреймов в кратчайшие сроки и при минимальных затратах обеспечить современный веб-интерфейс.

Сохранение инвестиций — использование данных и существующих прикладных программ не влечет дополнительных расходов по приобретению нового программного обеспечения для другой платформы, переучиванию персонала, переноса данных и т. д.

Основным производителем мейнфреймов была и остается компания *IBM*. Более 500 крупнейших мировых корпораций имеют вычислительные комплексы, оснащенные мейнфреймами. Везде, где высоки требования к быстродействию, надежности, безопасности, используются именно они. Такие предприятия, как банки, страховые компании, транспортные корпорации, активно используют сейчас технологии электронной коммерции (электронный обмен информацией, электронные транзакции и др.), обслуживание пользователей с использованием современных информационных технологий.

В 2000—2005 гг. была разработана новая архитектура мейнфреймов, так называемая *z*-Архитектура с 64-разрядной адресаций памяти: семейство мейнфреймов *System z* под управлением опера-

ционных систем *z/OS*. Новая операционная система большой вычислительной машины предоставила возможность параллельного использования и других операционных систем, например, *Linux*. В настоящее время более тысячи *Linux* машин используют специальную операционную систему *z/VM*. Что следует особо отметить, *Z*-платформа много дешевле, чем предыдущие архитектуры мейнфреймов.

Мейнфреймы вбирали в себя все новые технологии и модели обработки информации по мере их появления, начиная с модели централизованных вычислений и заканчивая веб-моделью. Они предоставляют пользователям широкий набор языков программирования — от языка *COBOL* до *Java*.

Мейнфреймы могут выполнять два вида работ, которые, соответственно, представляют собой обслуживание двух абсолютно различных типов рабочих нагрузок:

- пакетная обработка заданий (*Batch Job*), когда компьютер выполняет работу без участия человека. Используется в случае значительных объемов данных на входе;
- обработка заданий в реальном времени (*On-line*), например, транзакционные системы, такие как система приобретения железнодорожных билетов, система оплаты по кредитной карте и т. п.

Разделение ресурсов — это еще одно ключевое различие между моделями. В случае централизованной обработки информации каждый ресурс (дисковое пространство, оперативная память, процессоры, каналы ввода-вывода и т. д.) используется совместно различными приложениями. Это позволяет добиться более эффективного использования ресурсов и снижает время простоя. В распределенных системах с выделенными ресурсами эффективность значительно меньше, а время простоя — больше.

В отличие от серверов архитектуры *Intel* или машин, работающих под управлением *UNIX*, для которых характерна загрузка от 5 до 15 %, мейнфреймы, как правило, обеспечивают уровень загрузки от 80 до 100 %. В мейнфрейме нагрузка может быть распределена между системами, находящимися на расстоянии до 100 км друг от друга, благодаря чему восстановление после катастрофы осуществляется быстро и удобно. А такие операции, как наращивание мощности или замена вышедших из строя узлов, не прерывают процесса работы. Многие мейнфреймы работают без перерывов на протяжении лет, а их адаптация к меняющимся задачам и приложениям осуществляется на ходу.

6.2.4. Нейрокомпьютеры

Нейрокомпьютер — это устройство переработки информации на основе принципов работы естественных нейронных систем.

Эти принципы были формализованы, что позволило говорить о теории искусственных нейронных сетей. Проблематика же нейрокомпьютеров заключается в построении реальных физических устройств, что позволит не просто моделировать искусственные нейронные сети на обычном компьютере, но так изменить принципы работы компьютера, что станет возможным говорить о том, что они работают в соответствии с теорией искусственных нейронных сетей [41].

Его отличительные черты:

- ассоциативный характер обработки информации — в памяти хранится набор эталонов, и обработка состоит в том, чтобы отнести поступающий сигнал к тому или иному эталону по заданной мере близости, которая может быть разной и задается архитектурой обрабатывающей нейронной сети, не всегда известной и выражаемой в аналитическом виде;
- способность к мэппингу — способность нейронной сети автоматически построить топологически корректное отображение одного пространства в другое, когда размерности этих пространств разные;
- обучаемость (первые две черты проявляются в результате одной универсальной процедуры обучения на примерах).

Согласно принятой в нейрокомпьютинге модели искусственный нейрон, как и настоящий, имеет несколько входов и один выход. Все входные сигналы, поступающие в нейрон, сначала умножаются на определенные коэффициенты, называемые весами, затем суммируются, сумма преобразуется с помощью несложной функции и передается на выход. Ключевой элемент такой модели — веса. Именно они придают системе гибкость и позволяют настроиться на решение определенной задачи. Сигнал, который умножается на большой вес, дает большой вклад в общую сумму, а сигнал с нулевым весом не учитывается вовсе

В отличие от цифровых систем, представляющих собой комбинации процессорных и запоминающих блоков, нейропроцессоры содержат память, распределенную в связях между очень простыми процессорами, которые часто могут быть описаны как формальные нейроны или блоки из однотипных формальных нейронов. Тем самым основная нагрузка на выполнение конкретных функций процессорами ложится на архитектуру системы, детали которой в свою очередь определяются межнейронными связями. Подход, основанный на представлении как памяти данных, так и алгоритмов системой связей (и их весами), называется коннекционизмом.

Три основных преимущества нейрокомпьютеров:

- все алгоритмы нейроинформатики высокопараллельны, а это уже залог высокого быстродействия;
- нейросистемы можно легко сделать очень устойчивыми к помехам и разрушениям;

• устойчивые и надежные нейросистемы могут создаваться и из ненадежных элементов, имеющих значительный разброс параметров.

Основные правила выделения функциональных компонентов идеального нейрокомпьютера (по Миркесу):

• относительная функциональная обособленность: каждый компонент имеет четкий набор функций. Его взаимодействие с другими компонентами может быть описано в виде небольшого числа запросов;

• возможность взаимозамены различных реализаций любого компонента без изменения других компонентов.

Постепенно складывается рынок нейрокомпьютеров. В настоящее время широко распространены различные высокопараллельные нейроускорители (сопроцессоры) для различных задач. Моделей универсальных нейрокомпьютеров на рынке мало отчасти потому, что большинство из них реализованы для спецприменений. Примерами нейрокомпьютеров являются нейрокомпьютер *Synapse* (*Siemens*, Германия), процессор *NeuroMatrix*. Издается специализированный научно-технический журнал «Нейрокомпьютеры: разработка, применение». Проводятся ежегодные конференции по нейрокомпьютерам. С технической точки зрения, сегодняшние нейрокомпьютеры — это вычислительные системы с параллельными потоками одинаковых команд и множественным потоком данных (*MSIMD*-архитектура). Это одно из основных направлений развития вычислительных систем с массовым параллелизмом.

Основным требованием к аппаратному нейрокомпьютеру является каскадирование, т. е. возможность наращивания вычислительной мощности без особых технических требований.

Все фирмы, выпускающие специализированные нейронные вычислительные системы, положили в основу их построения принцип модульности, согласно которому вычислительная система строится как набор модулей, устанавливаемых на материнскую плату.

Применяя принцип модульности, можно в одной системе объединять модули различного назначения: вычислительные, периферийные, преобразования сигналов, ввода/вывода и т. д.

В нейрокомпьютинге постепенно созревает новое направление, основанное на соединении биологических нейронов с электронными элементами. По аналогии с *Software* (программное обеспечение — «мягкий продукт») и *Hardware* (электронное аппаратное обеспечение — «твердый продукт»), эти разработки получили наименование *Wetware* (англ.) — «влажный продукт».

В настоящее время уже существует технология соединения биологических нейронов со сверхминиатюрными полевыми транзисторами с помощью нановолокон (*Nanowire*). В разработках используется современная нанотехнология, в том числе для создания

соединений между нейронами и электронными устройствами используются углеродные нанотрубки.

Распространено также и другое определение термина «*Wetware*» — человеческий компонент в системах «человек-компьютер».

Применения.

1. Управление в реальном времени, в том числе:
   - самолетами и ракетами;
   - технологическими процессами непрерывного производства (в энергетике, металлургии и др.);
   - гибридным двигателем автомобиля;
   - пневмоцилиндром;
   - сварочным аппаратом;
   - электропечью;
   - турбогенератором.
2. Распознавание образов:
   - изображений, человеческих лиц, букв и иероглифов, отпечатков пальцев в криминалистике, речи, сигналов радара и сонара, элементарных частиц и происходящих с ними физических процессов (эксперименты на ускорителях или наблюдение за космическими лучами);
   - заболеваний по симптомам (в медицине).

6.2.5. Системы для облачных вычислений

Внешне *Cloud Computing* выглядит как перенос компьютеров и систем хранения из предприятия в отдельную общую группу, или «облако». Конечный пользователь выставляет определенные требования к ресурсам (например, ему требуются вычисления и доступ в глобальную сеть с определенной скоростью), а «облако» собирает из своих внутренних компонентов нужные мощности и предоставляет их [42].

Главные причины переноса в «облако» — это экономия средств и масштабируемость. Одна из целей, которую преследует *Cloud Computing*, — предоставить более дешевые ресурсы, чем те, которые есть у вас и которые вы сами обслуживаете. Помимо экономии, мы также получаем небывалую гибкость и масштабируемость, так как поставщик «облачных» ресурсов имеет возможность легко расширить виртуальную среду потребителя за счет своей виртуальной инфраструктуры, предоставив ему более высокую пропускную способность или более мощные вычислительные ресурсы.

Начнем с самого нижнего уровня, который отвечает за инфраструктуру (*Infrastructure-as-a-Service — IaaS*, инфраструктура как сервис). *IaaS* представляет собой сервис по аренде инфраструктуры, т. е. вычислительных ресурсов и систем хранения. К этим ресурсам относятся не только виртуальные серверы с гарантированной вычислительной мощностью, но и каналы связи требуемой пропуск-

ной способности для доступа к хранилищам данных и Интернет. На этом уровне предоставляется возможность временного использования компьютеров или дата-центров при требуемом качестве обслуживания с возможностью исполнения произвольной операционной системы и программ.

Следующий уровень сервиса при движении на диаграмме вверх — уровень платформы (*Platform-as-a-Service* или *PaaS*, платформа как сервис). *PaaS* похож на уровень *IaaS*, но включает в себя операционные системы и сопутствующие службы, ориентированные на определенные приложения. Например, *PaaS* совместно с виртуальными серверами и системами хранения предоставляет определенную операционную систему и набор приложений (обычно в виде образа виртуальной машины, например, файла формата *.vmdk* для *VMWare*), а также доступ к различным специализированным локальным сервисам (например, базе данных *MySQL*). Другими словами, *PaaS* — это *IaaS* вместе со стеком приложений, выполняющим конкретную задачу.

И наконец, на самом верху схемы располагается наиболее простой предоставляемый уровень — уровень приложений (*Software-as-a-Service* или *SaaS*, программное обеспечение как сервис), который предполагает использование приложения из централизованной (и, возможно, удаленной — из «облака») системы для работы на локальном компьютере. *SaaS* является измеряемой услугой и позволяет как бы арендовать приложение и оплачивать только время работы с ним.

Виртуализация предоставляет уникальные преимущества, хотя и не является обязательной для создания динамически масштабируемых архитектур. Помимо масштабируемости, виртуализация предоставляет возможность переносить виртуальные машины (*Virtual Machine — VM*) между физическими серверами в целях выравнивания нагрузки. Виртуализация обеспечивается программным уровнем, который называется гипервизором (другое название — монитор виртуальных машин (*Virtual Machine Monitor — VMM*). Этот уровень обеспечивает возможность одновременной работы множества операционных систем (и их приложений) на одной физической машине. В гипервизоре имеется объект, называемый виртуальной машиной, который инкапсулирует в себе операционную систему, ее приложения и конфигурацию. Также гипервизор или виртуальная машина могут эмулировать работу различных устройств.

Базовые платформы облачных вычислений предлагают минимум — только виртуальное аппаратное обеспечение и, возможно, операционную систему. Они обычно более гибкие, поскольку у них меньше ограничений. Можно указать некоторые аппаратные требования, например, тип процессора с определенной тактовой частотой, с определенным объемом памяти и т. д. Среди существующих

базовых платформ можно отметить *Amazon Elastic Compute Cloud*, *IBM Blue Cloud*, *Joyent Accelerator*, *Mosso*.

Специализированные платформы облачных вычислений предоставляют какую-либо среду для разработки приложений и собственные сервисы в добавление к базовой платформе. Специализированные платформы, как правило, проще и зачастую предлагают ряд уникальных услуг. Можно выделить следующие специализированные платформы: *Microsoft Azure*, *Google App Engine*, *Aptana Cloud*, *Heroku*, *Ning*, *Salesforce*.

6.2.6. Суперкомпьютеры

Современный суперкомпьютер — это мощный компьютер с производительностью несколько миллиардов операций с плавающей точкой в секунду. Суперкомпьютер представляет собой многопроцессорный и (или) многомашинный комплекс, работающий на общую память и общее поле внешних устройств [1].

Термин «суперкомпьютер» вошел в общеупотребительный лексикон благодаря распространенности компьютерных систем американца Сеймура Крея — *Control Data* 6600, *Control Data* 7600, *Cray*-1, *Cray*-2, *Cray*-3 и *Cray*-4. На сегодняшний день суперкомпьютеры являются уникальными системами, создаваемыми «традиционными» лидерами компьютерного рынка, такими как *IBM*, *Hewlett-Packard*, *NEC* и др., которые приобрели множество ранних компаний, вместе с их опытом и технологиями. Компания *Cray Inc.* по прежнему занимает достойное место в ряду производителей суперкомпьютерной техники.

Большинство суперкомпьютеров 1970-х гг. оснащались векторными процессорами. К началу и середине 1980-х гг. небольшое число (от 4 до 16) параллельно работающих векторных процессоров практически стало стандартным суперкомпьютерным решением. Типичный векторный компьютер включает в себя скалярный процессор целочисленной арифметики, функциональные блоки сложения и умножения чисел с плавающей точкой, векторный процессор и общую память. Это компьютеры, построенные по технологии «разделяемая память — один поток управления — много потоков данных» («*Shared Memory — Single Instruction — Multi Data*»).

Конец 1980-х и начало 1990-х гг. охарактеризовались сменой магистрального направления развития суперкомпьютеров от векторно-конвейерной обработки данных к большому и сверхбольшому числу параллельно соединенных скалярных процессоров.

Массивно-параллельные системы стали объединять в себе сотни и даже тысячи отдельных процессорных элементов, причем ими могли служить не только специально разработанные, но и общеизвестные и доступные в свободной продаже процессоры. Большинство

массивно-параллельных компьютеров создавалось на основе мощных процессоров с архитектурой *RISC* (*Reduced Instruction Set Computer*), наподобие *Power PC* или *PA-RISC*. Использование серийных микропроцессоров позволило не только гибко менять мощность установки в зависимости от потребностей и возможностей, но и значительно удешевить производство. Примерами суперкомпьютеров этого класса могут служить *Intel Paragon*, *IBM SP*, *Cray T3D/T3E* и ряд других.

Петафлопсный рубеж (тысяча триллионов операций с плавающей запятой в секунду) компания *Cray Inc.* преодолела в 2007 г.

Однако уникальные решения с рекордными характеристиками обычно недешевы, поэтому и стоимость подобных систем никак не могла быть сравнима со стоимостью систем, находящихся в массовом производстве и широко используемых в бизнесе. Прогресс в области сетевых технологий сделал свое дело: появились недорогие, но эффективные решения, основанные на коммуникационных технологиях. Это и предопределило появление ***кластерных вычислительных систем***, фактически являющихся одним из направлений развития компьютеров с ***массовым параллелизмом вычислительного процесса*** (*Massively Parallel Processing — MPP*).

6.2.7. Вычислительный кластер

Это совокупность компьютеров, объединенных в рамках некоторой сети для решения крупной вычислительной задачи. В качестве узлов обычно используются доступные однопроцессорные компьютеры, двух- или четырехпроцессорные *SMP*-серверы (*Symmetric Multi Processor*). Каждый узел работает под управлением своей копии операционной системы, в качестве которой чаще всего используются стандартные операционные системы: *Linux*, *NT*, *Solaris* и т. п. Рассматривая крайние точки зрения, кластером можно считать как пару персональных компьютеров, связанных локальной 10-мегабитной сетью *Ethernet*, так и обширную вычислительную систему, создаваемую в рамках крупного проекта. Такой проект объединяет тысячи рабочих станций на базе процессоров *Alpha*, связанных высокоскоростной сетью *Myrinet*, которая используется для поддержки параллельных приложений, а также сетями *Gigabit Ethernet* и *Fast Ethernet* для управляющих и служебных целей.

Состав и мощность узлов может меняться даже в рамках одного кластера, давая возможность создавать обширные гетерогенные (неоднородные) системы с задаваемой мощностью. Выбор конкретной коммуникационной среды определяется многими факторами: особенностями класса решаемых задач, доступным финансированием, необходимостью последующего расширения кластера и т. п. Возможно включение в конфигурацию специализированных компьютеров, например файл-сервера, и, как правило, предоставлена возможность удаленного доступа на кластер через Интернет.

На современном рынке представлено не так много поставщиков готовых кластерных решений. Это связано, прежде всего, с доступностью комплектующих, легкостью построения самих систем, значительной ориентацией на свободно распространяемое программное обеспечение, а также с уникальностью задач, решаемых с помощью кластерных технологий. Среди наиболее известных поставщиков стоит отметить *SGI*, *VALinux* и *Scali Computer*.

Технологии суперкомпьютеров и кластеров первоначально «выросли» в основном из научных потребностей — для решения фундаментальных и прикладных задач физики, механики, астрономии, метеорологии, сопротивления материалов и т. д., где требовались огромные вычислительные мощности. В каких рыночных нишах будет востребована подобная производительность? Прежде всего, это проектирование сложных управляемых систем (самолетов, ракет, космических станций), создание синтетических лекарств с заданными свойствами, генная инженерия, предсказание погоды и природных катаклизмов, повышение эффективности и надежности атомных электростанций, прогнозирование макроэкономических эффектов и многое другое.

6.2.8. Компьютеры следующего поколения

Размеры вычислительных устройств постоянно уменьшаются. Когда-то предполагалось, что более мощные машины будут требовать больше места для периферийных устройств, памяти и т. д. Это предположение оказалось неверным. В 1965 г. Гордон Мур сформулировал действующее и сейчас правило (названное законом Г. Мура), согласно которому производительность вычислительных систем удваивается каждые 18 месяцев [45]. Г. Мур вывел свой эмпирический закон, построив зависимость числа транзисторов в интегральной микросхеме от времени. Как следствие из этого закона можно вывести темпы миниатюризации отдельного транзистора.

Ежегодное уменьшение на 10—30 % размеров элементарных вычислительных модулей приведет в ближайшие 5—10 лет к практическому применению устройств с элементарными модулями размером примерно в 100—200 ангстрем (0,01—0,02 мк). Другими словами, быстрое развитие цифровых электронных технологий приводит к тому, что размер элементарного вычислительного устройства приближается к размеру молекулы или даже атома.

На таком уровне законы классической физики перестают работать и начинают действовать квантовые законы, которые для многих важных динамических задач еще не описаны теоретически. Для описания работы таких устройств неприменимы классические объекты и методы информатики. В частности, в силу квантового принципа неопределенности Гейзенберга, в таких микроскопических системах нет аналога понятию «*bit*».

Вместо двоичных цифр новые устройства будут оперировать с «волновыми функциями» («квантовыми битами»). В некотором смысле информатика в своем развитии в недалеком будущем должна будет перейти от «арифметики» к «функциональному анализу». С одной стороны, это обусловливает переосмысление и замену основных классических (неквантовых) алгоритмов, а с другой — дает возможность вплотную подступиться к решению проблем искусственного интеллекта.

В научно-исследовательских лабораториях крупнейших университетов и транснациональных ИТ-компаний рассматриваются несколько возможных основных направлений создания элементной базы нового поколения вычислительных устройств [43]:

- на принципах ядерного магнитного или электронного парамагнитного резонанса;
- на атомных ионах, помещенных в ловушки Паули или Пеннинга;
- с использованием явления сверхпроводимости;
- на квантовых точках в полупроводниковых неорганических системах;
- на основе оптической симуляции квантовой логики или на металло-биологической гибридной основе.

Многие из указанных направлений имеют существенные недостатки, которые в некоторых случаях приводят к принципиальной невозможности создания конкурентоспособного вычислительного устройства. Характерным примером является проект корпорации *IBM*, которая в 1999 г. только на первый этап разработки молекулярной элементной базы нового поколения выделила 17 млрд долл. на пять лет. В результате был создан макет, оперирующий с 5 или 7 квантовыми битами и весом около 7 т, способный решать только примитивные задачи типа разложения числа 15 на два множителя 5 и 3.

В настоящее время наиболее перспективным направлением разработки элементной базы компьютеров нового поколения представляется использование самоорганизующихся квантовых точек в твердотельных системах, которые могут выполнять функции квантовых битов и быть связанными в квантовый регистр на основе, например, электростатического.

6.3. Методические средства информационных технологий

Для большинства технологий характерной чертой их развития является стандартизация и унификация.

Стандартизация — нахождение решений для повторяющихся задач и достижение оптимальной степени упорядоченности.

Унификация — относительное сокращение разнообразия элементов по сравнению с разнообразием систем, в которых они используются.

Если в области традиционного «материального производства» уже давно сложилась система формирования и сопровождения стандартов, то в области информационных технологий многое предстоит сделать.

Главная задача стандартизации в рассматриваемой области — создание системы нормативно-справочной документации, определяющей требования к разработке, внедрению и использованию всех компонентов информационных технологий. На сегодняшний день в области информационных технологий наблюдается неоднородная картина уровня стандартизации. Для ряда технологических процессов характерен высокий уровень стандартизации, например для транспортирования информации, для других, уровень стандартизации находится в зачаточном состоянии.

Многообразные стандарты и подобные им методические материалы упорядочим по следующим признакам.

1. По утверждающему органу:
- официальные международные стандарты;
- официальные национальные стандарты;
- национальные ведомственные стандарты;
- стандарты международных комитетов и объединений;
- стандарты фирм-разработчиков;
- стандарты «де-факто».

2. По предметной области стандартизации:
- функциональные стандарты (стандарты на языки программирования, интерфейсы, протоколы, кодирование, шифрование и др.);
- стандарты на фазы развития (жизненного цикла) информационных систем (стандарты на проектирование, материализацию, эксплуатацию, сопровождение и др.).

В зависимости от методического источника в качестве стандартов могут выступать «метод», «модель», «методология», «подход». Следует отметить, что указанные материалы обладают разной степенью обязательности, конкретности, детализации, открытости, гибкости и адаптируемости.

Исходя из важности проблемы большое внимание уделяется ее решению на международной практике [46].

Организационная структура, поддерживающая процесс стандартизации ИТ, включает три основные группы организаций:

1. Международные организации, входящие в структуру ООН:
- *International Organization for Standardization* (далее — *ISO*) — Международная организация по стандартизации;

- *International Electrotechnical Commision* (далее — *IEC*) — Международная электротехническая комиссия;
- *International Telecommunication Union-Telecommunications* (далее — *ITU-T*) — Международный союз по телекоммуникации — телекоммуникация. До 1993 г. эта организация имела другое название — *CCITT* (*International Telegraph and Telephone Consultative Committee* — Международный консультативный комитет по телефонии и телеграфии (МККТТ)). X.200, X-400, X-500, X-600.

2. Промышленные профессиональные или административные организации:

- *Institute of Electrical and Electronic Engineers* (*IEEE*) — Институт инженеров по электротехнике и электронике), *LAN IEEE*802, *POSIX*;
- *Internet Activities Board* (*Internet* и *IAB*) (Совет управления деятельностью *Internet*), *TCP/IP*;
- *Regional WOS* (*Workshops on Open Systems*) (Рабочие группы по открытым системам), *OSE*-profiles;
- промышленные консорциумы;
- *European Computer Manufacturers Association* (далее — *ECMA*) — Европейская ассоциация производителей вычислительных машин) *OSI*, безопасность, управление, *Office Document Architecture* (далее — *ODE*);
- *OMG* (*Object Management Group*) (Группа управления объектами) *RM*: *Common Object Request Broker Architecture* (*CORBA*);
- *X/Open* (Организована группой поставщиков компьютерной техники) *X/Open Portability Guide* (*XPG*4) *Common Application Environment*;
- *Network Management Forum* (*NMF*) — Форум управления сетями.
- *Open Software Foundation* (*OSF*) — Фонд открытого программного обеспечения). Имеет следующие предложения: *OSF*/1 (Соответствует стандарту *POSIX* и *XPG*4). *MOTIF* — графический пользовательский интерфейс (*Distributed Computer Environment* — *DCE* — технология интеграции *ECMA*.

В 1987 г. *ISO* и *IEC* объединили свою деятельность в области стандартизации ИТ, создав единый орган *Joint Technical Committee* 1 (далее — *JTC*1) (Объединенный технический комитет 1), предназначенный для формирования всеобъемлющей системы базовых стандартов в области ИТ и их расширений для конкретных сфер деятельности.

Работа над стандартами ИТ в *JTC*1 тематически распределена по подкомитетам (*Subcommittees* — *SC*).

В дополнение создана специальная группа по функциональным стандартам (*Special Group on Functional Standards* — *SGFS*) для обработки предложений по Международным стандартизованным профилям (*International Standardized Profiles* — *ISPs*), представляющим пределения профилей ИТ.

Среди отечественных стандартов в области ИТ можно отметить следующие:

- ГОСТ 34.003—1990 «Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Термины и определения»;
- ГОСТ 34.602—89 «Техническое задание на создание автоматизированной системы (АС)»;
- ГОСТ Р 34.10—2012 «Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи»;
- РД 50—34.698—90 «Требования к содержанию документов» и др.;
- ГОСТ Р 51954—2002 «Информационная технология. Профиль прикладной среды организации вычислений на суперЭВМ»;
- ГОСТ РВ 51987 «Информационная технология. Комплекс стандартов на автоматизированные системы. Типовые требования и показатели качества функционирования информационных систем. Общие положения»;
- Р 50.1.041—2002 «Информационные технологии. Руководство по проектированию профилей среды открытой системы (СОС) организации-пользователя»;
- Р 50.1.027—2001 Информационные технологии поддержки жизненного цикла продукции. Автоматизированный обмен технической информацией. Основные положения и общие требования;
- Р 50.1.028—2001 Информационные технологии поддержки жизненного цикла продукции. Методология функционального моделирования.

В качестве примера рассмотрим ряд стандартов различного уровня.

Международный стандарт *ISO/OSI* разработан международной организацией по стандартизации *ISO*, предназначен для использования в области сетевого информационного обмена, представляет эталонную семиуровневую модель, известную как модель *OSI*. Первоначально усилия были направлены на разработку структуры (модели) протоколов связи цифровых устройств. Основная идея была связана с разбиением функций протокола на семь различных категорий (уровней), каждый из которых связан с одним более высоким и с одним более низким уровнем (за исключением самого верхнего самого нижнего). Идея семиуровневого открытого соединения состоит не попытке создания универсального множества протоколов связи, а в реализации «модели», в рамках которой могут быть использованы уже имеющиеся различные протоколы. В последнее время достигнут значительный прогресс в реализации различных типов протоколов, о чем говорит успешное функционирование многих сетей передачи данных, например, Интернет. Более подробно данный стандарт изложен в параграфе 2.2.

Международный стандарт *ISO/IEC* 12207:1995-08-01 — базовый стандарт процессов жизненного цикла программного обеспечения, ориентированный на различные его виды, а также типы информационных систем, куда программное обеспечение входит как составная часть. Разработан в 1995 г. объединенным техническим комитетом *ISO/IEC JTC*1 «Информационные технологии, подкомитет *SC7*, проектирование программного обеспечения». Включает описание основных, вспомогательных и организационных процессов.

Основные процессы программного обеспечения:

- процесс приобретения, определяющий действия покупателя, приобретающего информационную систему, программный продукт или его сервис;
- процесс поставки, регламентирующий действия поставщика, снабжающего указанными выше компонентами;
- процесс разработки, определяющий действия разработчика принципов построения программного изделия;
- процесс функционирования, определяющий действия оператора, обслуживающего информационную систему в интересах пользователей и включающий помимо требований инструкции по эксплуатации консультирование пользователей и организацию обратной связи с ним;
- процесс сопровождения, регламентирующий действия персонала по модификации программного продукта, поддержки его текущего состояния и функциональной работоспособности.

Вспомогательные процессы регламентируют документирование, управление конфигурацией, обеспечение качества, верификацию, аттестацию, совместную оценку, аудит.

Степень обязательности для организации, принявшей решение о применении *ISO/IEC* 12207, обусловливает ответственность в условиях торговых отношений за указание минимального набора требуемых процессов и задач, требующих согласования с данным стандартом.

Стандарт содержит мало описаний, направленных на проектирование баз данных, что объясняется наличием отдельных стандартов по данной тематике.

ГОСТ 34 в качестве объекта стандартизации рассматривает автоматизированные системы различных видов и все виды их компонентов, в том числе программное обеспечение и базы данных. Стандарт в основном рассматривает проектные документы, что отличает его от стандарта *ISO/IEC* 12207. В структуре стандарта выделяют стадии и этапы разработки автоматизированных систем (далее — АС).

Рассмотрим краткую характеристику стадий и этапов.

1. Формирование требований к АС:

- обследование объекта и обоснование необходимости создания АС;

- формирование требований пользователя к АС;
- оформление отчета о выполненной работе и заявки на разработку АС (тактико-технического задания).

2. Разработка концепции АС:
- изучение объекта;
- проведение необходимых научно-исследовательских работ;
- разработка вариантов концепции АС, удовлетворяющей требованиям пользователя;
- оформление отчета о выполненной работе.

3. Техническое задание:
- разработка и утверждение технического задания.

4. Эскизный проект:
- разработка предварительных проектных решений по системе и ее частям;
- разработка документации на АС и ее части.

5. Технический проект:
- разработка проектных решений по системе и ее частям;
- разработка документации на АС и ее части;
- разработка и оформление документации на поставку изделий для комплектования АС и (или) технических требований (технических заданий) на их разработку;
- разработка заданий на проектирование в смежных частях проекта объекта автоматизации.

6. Рабочая документация:
- разработка рабочей документации на систему и ее части;
- разработка или адаптация программ.

7. Ввод в действие:
- подготовка объекта автоматизации к вводу АС в действие;
- подготовка персонала;
- комплектация АС поставляемыми изделиями (программными, техническими и информационными средствами);
- строительно-монтажные работы;
- пусконаладочные работы;
- проведение предварительных испытаний;
- проведение опытной эксплуатации;
- проведение приемочных испытаний.

8. Сопровождение АС:
- выполнение работ в соответствии с гарантийными обязательствами;
- послегарантийное обслуживание.

ГОСТ 34 содержит обобщенную понятийную и терминологическую систему, общую схему разработки, общий набор документов. С точки зрения обязательности, в настоящее время таковая отсутствует, что сделало ГОСТ 34 методической поддержкой.

Методика *Oracle CDM* (*Custom Development Method*) является развитием ранее разработанной версии *Oracle CASE-Method*, известной по использованию *Designer/2000*. Она ориентирована на разработку прикладных информационных систем под заказ. Структурно построена как иерархическая совокупность этапов, процессов и последовательностей задач, краткая характеристика которых дана ниже.

Этапы:

- стратегия (определение требований);
- анализ (формирование детальных требований);
- проектирование (преобразований требований в спецификации);
- реализация (разработка и тестирование приложений);
- внедрение (установка, отладка и ввод в эксплуатацию);
- эксплуатация (поддержка, сопровождение, расширение).

Процессы:

- *RD* — определение производственных требований;
- *ES* — исследование и анализ существующих систем;
- *TA* — определение технической архитектуры;
- *DB* — проектирование и построение базы данных;
- *MD* — проектирование и реализация модулей;
- *CV* — конвертирование данных;
- *DO* — документирование;
- *TE* — тестирование;
- *TR* — обучение;
- *TS* — переход к новой системе;
- *PS* — поддержка и сопровождение.

Процессы состоят из последовательностей задач, причем задачи разных процессов взаимосвязаны ссылками.

Методика не предусматривает включения новых задач, удаления старых задач, изменения последовательности выполнения задач. Методика необязательна, может считаться фирменным стандартом.

В связи с широким использованием в настоящее время объектной технологии большой интерес представляет *CORBA* — стандарт в виде набора спецификаций для промежуточного программного обеспечения (*middleware*) объектного типа. Его автором является международный консорциум *OMG*, объединяющий более 800 компаний (*IBM*, *Siements*, *Microsoft*, *Sun*, *Oracle* и др.). *OMG* разработала семантический стандарт, включающий четыре основных типа:

1) объекты, моделирующие мир (студент, преподаватель, экзамен);

2) операции, относящиеся к объекту и характеризующие его свойства (дата рождения студента, пол и др.);

3) типы, описывающие конкретные значения операций;

4) подтипы, уточняющие типы.

На основе этих понятий *OMG* определила объектную модель, спецификацию для развития стандарта *CORBA*, постоянно развива-

емую. В настоящее время *CORBA* состоит из четырех основных частей:

1) *Object Request Broker* (посредник объектных запросов);
2) *Object Services* (объектные сервисы);
3) *Common Facilities* (общие средства);
4) *Application and Domain Interfaces* (прикладные и отраслевые интерфейсы).

Параллельно с *CORBA* корпорацией *Microsoft* был разработан стандарт *COM/DCOM* (*Component Object Model/Distributed COM*), предназначенный для объединения мелких офисных программ. Основным недостатком данного стандарта была ориентация на *Windows* и *Microsoft*. Корпорация *Microsoft* долгое время не присоединялась к *OMG* и развивала собственный стандарт. Однако жизнь заставила приступить к мирным переговорам. *OMG* взаимодействует с другими центрами стандартизации: *ISO*, *Open Group*, *WWW* консорциум, *IEEE* и многими другими. *CORBA* стал неотъемлемой частью распределенных объектных компьютерных систем.

Приведенные примеры стандартов дают представление о подходах к решению проблем стандартизации.

Естественно, затраты на стандартизацию могут сделать проектные работы по внедрению информационных технологий более дорогостоящими, однако эти затраты с лихвой окупаются в процессе эксплуатации и развития системы, например при замене оборудования или программной среды.

Тема 7
ТЕХНОЛОГИИ ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННЫХ СИСТЕМ

7.1. Методология проектирования информационных систем

Согласно определению [8] **система** есть сущность, которая в результате взаимодействия ее частей (компонентов) может поддерживать свое существование и функционировать как единое целое. В этом определении важно отметить, что поведение системы зависит не от природы свойств ее образующих частей, а от того, как эти части соединены между собой.

Из определения системы следует, что системы функционируют как целое, что порождает у них свойства, отличающиеся от свойств составляющих ее частей. Эти свойства известны как *эмерджентные* (*возникающие*). Разделив систему на компоненты, мы никогда не обнаружим ее существенных свойств, они проявляются только в результате действия целостной системы. Эмерджентные свойства являются важными в понимании сущности анализа и синтеза. Разбиение системы на составляющие части для познания принципов ее функционирования называется *анализом*. С помощью анализа мы получаем знания, однако теряем возможность понимания свойств. Дополнением анализа является *синтез* — создание целого из частей. С помощью синтеза мы обретаем *понимание* [8].

Принято выделять ***простые*** и ***сложные системы***. Сложность может проявляться двумя различными путями: статическим и динамическим. *Статическая сложность* (детальная) определяется в процессе детализации как количество рассматриваемых элементов. *Динамическая сложность* зависит от отношений между элементами. Информационные системы относятся к классу динамически сложных. Таким образом, информационная система состоит из множества элементов или подсистем, которые находятся в разных состояниях и могут изменяться в зависимости от изменения других частей.

По виду элементов информационная система относится к системам типа «процесс» (элементами являются информационные процессы).

Отметим другие важные определения.

Структура — совокупность элементов и их связей.

Цель — состояние, к которому стремится система.

Среда — метасистема, в которой рассматриваемая система входит составной частью.

Функционирование системы — работа системы в рамках заданной структуры.

Развитие системы — работа системы в условии острых противоречий, которые могут вызвать изменение структуры.

Управление — целенаправленный перевод системы из одного состояния в другое желаемое.

Проектирование — процесс, который дает начало изменениям в искусственной среде.

Такое определение акцентирует внимание на последствиях внедрения. Проектировщик должен предвидеть конечный результат осуществления своего проекта и определить меры, необходимые для достижения этого результата. Проектирование представляется как итерационный цикл (рис. 7.1). Каждая итерация отличается большей детализацией и меньшей общностью.

Рис. 7.1. **Процесс проектирования**

В процессе проектирования выделяют следующие *фазы*:

1) ***дивергенция*** — расширение границ проектной ситуации с целью обеспечения более обширного пространства поиска решения;

2) ***трансформация*** — стадия создания принципов и концепций (исследование структуры проблемы);

3) ***конвергенция*** — охватывает традиционное проектирование (кодирование, отладка, проработка деталей).

Трудности проектирования:

- предположение о конечном результате проектирования приходится делать еще до того, как исследованы средства его достижения;
- часто случается, что в ходе исследования событий в обратном порядке (от конечного результата) обнаруживаются непредвиденные трудности или открываются новые, более благоприятные возможности.

Под **проектированием ИС** понимается процесс преобразования входной информации об объекте, методах и опыте проектирования объектов аналогичного назначения в соответствии с ГОСТом (34.601—90 «Автоматизированные системы. Стадии создания») в проект ИС.

С этой точки зрения, проектирование ИС сводится к последовательной формализации проектных решений на различных стадиях жизненного цикла ИС: планирования и анализа требований, технического и рабочего проектирования, внедрения и эксплуатации ИС.

Основными особенностями исходных данных для проектирования ИС являются следующие:

- большое количество действий, подлежащих реализации (многофункциональность);
- значительный объем и сложность ограничений на взаимосвязи проектируемой системы с окружением и трудности их формального описания;
- распределенный и асинхронный режим обработки данных;
- многообразие используемых информационных объектов и их свойств;
- нечеткость требований, их субъективный характер;
- неполнота требований, их расширение в процессе проектирования, необходимость учета развития системы.

Отличительная черта проектирования ИС — коллективное проектирование.

Технология проектирования ИС — это совокупность методологии и средств проектирования ИС, а также методов и средств его организации (управление процессом создания и модернизации проекта ИС).

Технологический процесс проектирования ИС в целом делится на совокупность последовательно-параллельных, связанных и соподчиненных цепочек действий, каждое из которых может иметь свой предмет. Таким образом, технология проектирования задается регламентированной последовательностью технологических операций, выполняемых на основе того или иного метода, в результате чего становится ясным, не только что должно быть сделано для создания проекта, но и как, кем и в какой последовательности.

Методология проектирования предполагает наличие некоторой концепции, принципов проектирования, реализуемых набором методов, которые, в свою очередь, должны поддерживаться некоторыми средствами.

Назначение методологии создания ИС заключается в организации процесса проектирования и обеспечении управления этим процессом гарантированного выполнения требований как к самой ИС,

так и к характеристикам процесса разработки. *Основными задачами методологии* являются следующие:

- обеспечение создания ИС, отвечающих предъявляемым к ним требованиям;
- создание системы с заданным качеством в заданные сроки и в рамках бюджета;
- поддержка сопровождения, модификации и наращивания ИС;
- создание ИС, отвечающих требованиям открытости, переносимости и масштабируемости.

Методология должна обеспечивать снижение сложности процесса создания ИС за счет полного и точного описания этого процесса и применения современных методов и технологий создания ИС на всем жизненном цикле ИС — от замысла до реализации.

Организация проектирования предполагает определение методов взаимодействия проектировщиков между собой и с заказчиком в процессе создания проекта ИС, которые могут также поддерживаться набором специфических средств.

1. **Классическое проектирование ИС** берет свое начало в 1970-х гг. Одно из первых направлений получило название ***каскадной схемы*** проектирования (рис. 7.2). Она широко использовалась при проектировании ИС и включала следующие стадии проекта: разработка требований, проектирование, реализация, тестирование, ввод в действие (внедрение). Основной особенностью данной методики является последовательная организация работ при разбиении структуры ИС на заранее определенный ряд подсистем: организационное, методическое, информационное, программное и аппаратное обеспечения. В западной литературе такая схема организации работ получила название «водопадной модели» (*waterfall model*) и включала дополнительно итерационные процедуры уточнения требований к системе и рассмотрения вариантов проектных решений. Каскадная модель предусматривает последовательное выполнение всех этапов проекта в строго фиксированном порядке. Переход на следующий этап означает полное завершение работ на предыдущем этапе.

Рис. 7.2. **Каскадная модель проектирования ИС**

Основными недостатками каскадной схемы проектирования являются запаздывание получения конечных результатов и низкая эффективность.

2. Дальнейшим развитием каскадной модели явилась ***итерационная*** (***поэтапная модель***) с промежуточным контролем (рис. 7.3). Разработка ИС ведется итерациями с циклами обратной связи между этапами. Межэтапные корректировки позволяют учитывать реально существующее взаимовлияние результатов разработки на различных этапах. Время жизни каждого из этапов растягивается на весь период разработки.

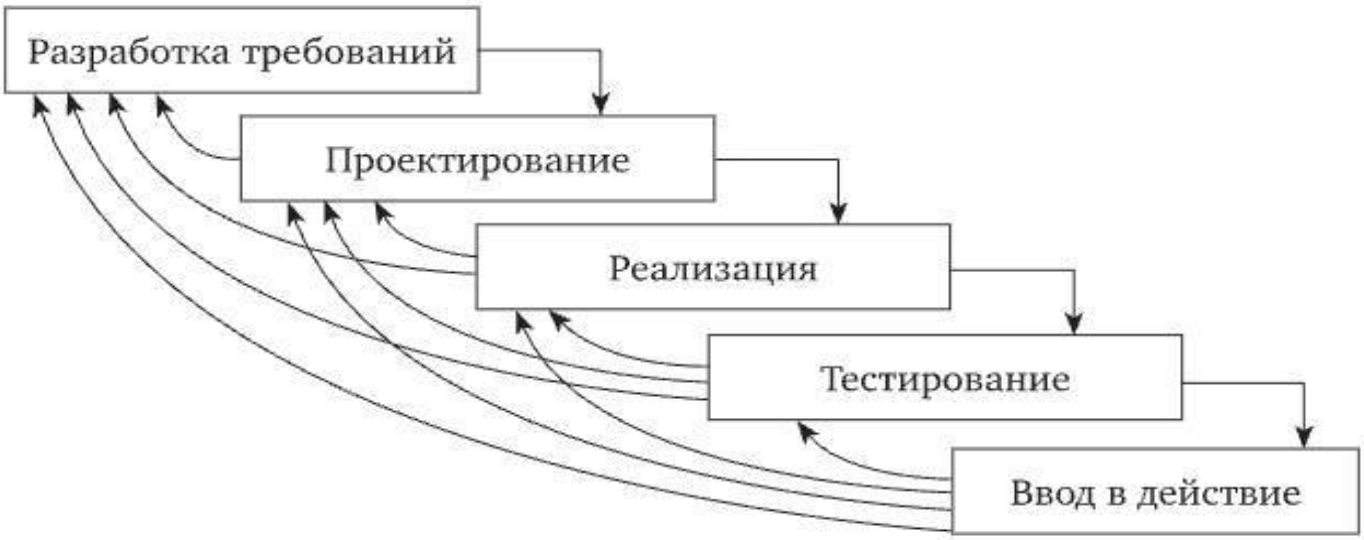


Рис. 7.3. **Поэтапная модель проектирования ИС с промежуточным контролем**

3. В процессе совершенствования разработки ИС появилась ***схема непрерывной*** (***спиральной***) ***разработки*** (рис. 7.4), использовавшаяся при реализации больших проектов фирмы *IBM* в 1970—1980-х гг.

Характерной особенностью данной методики стал непрерывный спиральный процесс разработки ИС с планируемыми точками передачи в эксплуатацию новых версий и новых функциональных подсистем. На каждом витке спирали выполняется создание очередной версии продукта, уточняются требования проекта, определяется его качество и планируются работы следующего витка. Особое внимание уделяется начальным этапам разработки — анализу и проектированию, где реализуемость тех или иных технических решений проверяется и обосновывается посредством создания прототипов (макетирования).

Методы проектирования ИС можно классифицировать по степени использования средств автоматизации, типовых проектных решений, адаптивности к предполагаемым изменениям.

Так, по *степени автоматизации* методы проектирования разделяются:

- на ***ручное***, при котором проектирование компонентов ИС осуществляется без использования специальных инструментальных

программных средств, а программирование — на алгоритмических языках;

- ***компьютерное***, при котором производится генерация или конфигурирование (настройка) проектных решений на основе использования специальных инструментальных программных средств.

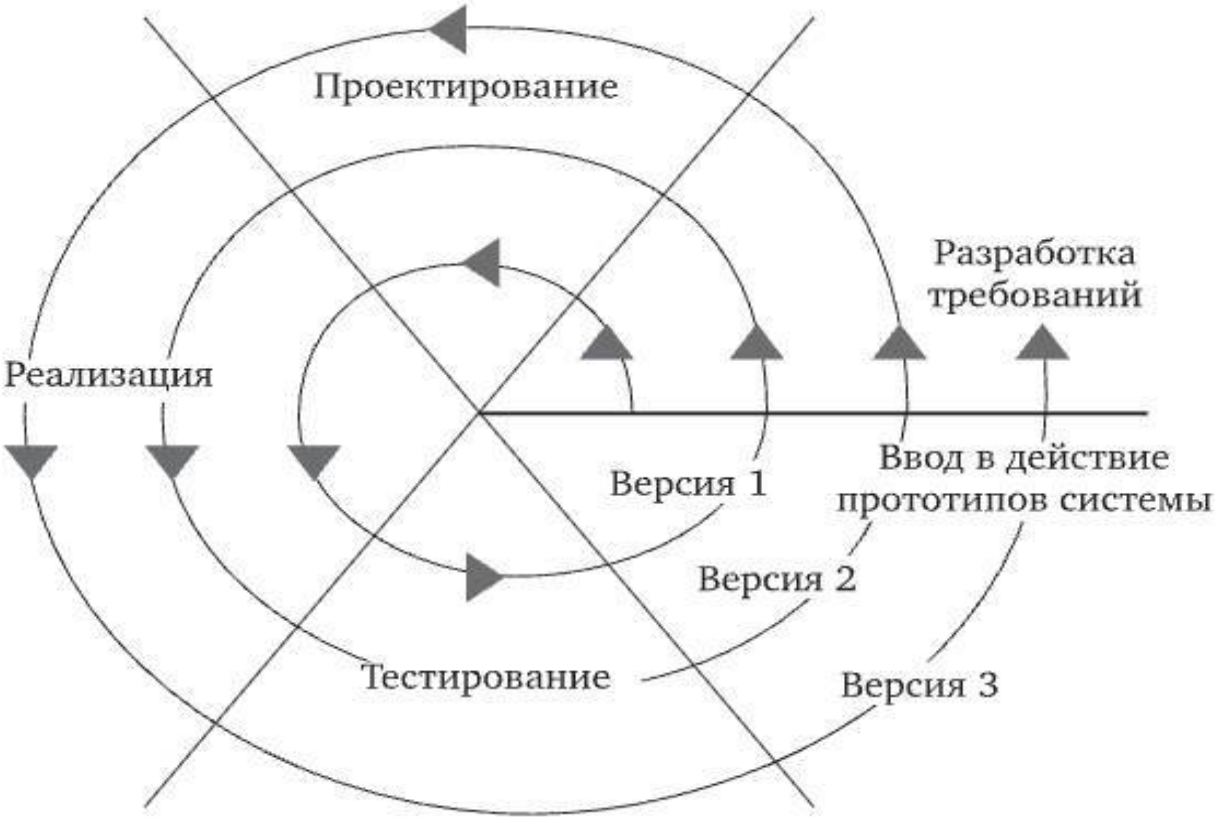


Рис. 7.4. **Спиральная модель проектирования ИС**

По *степени использования типовых проектных решений* различают следующие методы проектирования:

- ***оригинальное*** (индивидуальное), когда проектные решения разрабатываются «с нуля» в соответствии с требованиями к АИС. Характеризуется тем, что все виды проектных работ ориентированы на создание индивидуальных для каждого объекта проектов, которые в максимальной степени отражают все его особенности;
- ***типовое***, предполагающее конфигурирование ИС из готовых типовых проектных решений (программных модулей). Выполняется на основе опыта, полученного при разработке индивидуальных проектов. Типовые проекты как обобщение опыта для некоторых групп организационно-экономических систем или видов работ в каждом конкретном случае связаны со множеством специфических особенностей и различаются по степени охвата функций управления, выполняемым работам и разрабатываемой проектной документации.

По *степени адаптивности проектных решений* выделяют методы:

- ***реконструкции***, когда адаптация проектных решений выполняется путем переработки соответствующих компонентов (перепрограммирования программных модулей);

- ***параметризации***, когда проектные решения настраиваются (генерируются) в соответствии с изменяемыми параметрами;
- ***реструктуризации модели***, когда изменяется модель проблемной области, на основе которой автоматически заново генерируются проектные решения.

Сочетание различных признаков классификации методов обусловливает характер используемых технологий проектирования ИС, среди которых выделяют два основных класса: *каноническую* и *индустриальную* технологии. Индустриальная технология проектирования, в свою очередь, разбивается на два подкласса: *автоматизированное* (использование *CASE*-технологий) и *типовое* (параметрически-ориентированное или модельно-ориентированное) проектирование. Использование индустриальных технологий не исключает использования в отдельных случаях канонических.

В основе ***канонического проектирования*** лежит каскадная модель жизненного цикла ИС. Процесс каскадного проектирования в жизненном цикле ИС в соответствии с применяемым в нашей стране ГОСТ 34.601-90 «Автоматизированные системы. Стадии создания» делится на следующие семь стадий:

1) исследование и обоснование создания системы;
2) разработка технического задания;
3) создание эскизного проекта;
4) техническое проектирование;
5) рабочее проектирование;
6) ввод в действие;
7) функционирование, сопровождение, модернизация.

В зависимости от сложности объекта автоматизации и набора задач, требующих решения при создании конкретной ИС, стадии и этапы работ могут иметь различную трудоемкость. Допускается объединять последовательные этапы и даже исключать некоторые из них на любой стадии проекта. Допускается также начинать выполнение работ следующей стадии до окончания предыдущей.

Стадии и этапы создания ИС, выполняемые организациями-участниками, прописываются в договорах и технических заданиях на выполнение работ.

Стадия 1. Формирование требований к ИС. На начальной стадии проектирования выделяют следующие этапы работ:

- обследование объекта и обоснование необходимости создания ИС;
- формирование требований пользователей к ИС;
- оформление отчета о выполненной работе и тактико-технического задания на разработку.

Стадия 2. Разработка концепции ИС.

- изучение объекта автоматизации;
- проведение необходимых научно-исследовательских работ;

- разработка вариантов концепции ИС, удовлетворяющих требованиям пользователей;
- оформление отчета и утверждение концепции.

Стадия 3. Техническое задание:

- разработка и утверждение технического задания на создание ИС.

Стадия 4. Эскизный проект:

- разработка предварительных проектных решений по системе и ее частям;
- разработка эскизной документации на ИС и ее части.

Стадия 5. Технический проект:

- разработка проектных решений по системе и ее частям;
- разработка документации на ИС и ее части;
- разработка и оформление документации на поставку комплектующих изделий;
- разработка заданий на проектирование в смежных частях проекта.

Стадия 6. Рабочая документация:

- разработка рабочей документации на ИС и ее части;
- разработка и адаптация программ.

Стадия 7. Ввод в действие:

- подготовка объекта автоматизации;
- подготовка персонала;
- комплектация ИС поставляемыми изделиями (программными и техническими средствами, программно-техническими комплексами, информационными изделиями);
- строительно-монтажные работы;
- пусконаладочные работы;
- проведение предварительных испытаний;
- проведение опытной эксплуатации;
- проведение приемочных испытаний.

Стадия 8. Сопровождение ИС:

- выполнение работ в соответствии с гарантийными обязательствами;
- послегарантийное обслуживание.

Типовое проектирование ИС предполагает создание системы из готовых типовых элементов. Основополагающим требованием для применения методов типового проектирования является возможность декомпозиции проектируемой ИС на множество составляющих компонентов (подсистем, комплексов задач, программных модулей и т. д.). Для реализации выделенных компонентов выбираются имеющиеся на рынке типовые проектные решения, которые настраиваются на особенности конкретного предприятия.

Типовое проектное решение (далее — *ТПР*) — это тиражируемое (пригодное к многократному использованию) проектное реше-

ние. Принятая классификация ТПР основана на уровне декомпозиции системы. Выделяются следующие классы ТПР:

- *элементные ТПР* — типовые решения по задаче или по отдельному виду обеспечения задачи (информационному, программному, техническому, математическому, организационному);
- *подсистемные ТПР* — в качестве элементов типизации выступают отдельные подсистемы, разработанные с учетом функциональной полноты и минимизации внешних информационных связей;
- *объектные ТПР* — типовые отраслевые проекты, которые включают полный набор функциональных и обеспечивающих подсистем ИС.

Для реализации типового проектирования используются два подхода: параметрически-ориентированное и модельно-ориентированное проектирование.

Параметрически-ориентированное проектирование включает следующие этапы: определение критериев оценки пригодности пакетов прикладных программ (далее — ППП) для решения поставленных задач, анализ и оценка доступных ППП по сформулированным критериям, выбор и закупка наиболее подходящего пакета, настройка параметров (доработка) закупленного ППП.

Критерии оценки ППП делятся на следующие группы:

- назначение и возможности пакета;
- отличительные признаки и свойства пакета;
- требования к техническим и программным средствам;
- документация пакета;
- факторы финансового порядка;
- особенности установки пакета;
- особенности эксплуатации пакета;
- помощь поставщика по внедрению и поддержанию пакета;
- оценка качества пакета и опыт его использования;
- перспективы развития пакета.

Модельно-ориентированное проектирование заключается в адаптации состава и характеристик типовой ИС в соответствии с моделью объекта автоматизации.

Технология проектирования в этом случае должна обеспечивать единые средства для работы как с моделью типовой ИС, так и с моделью конкретного предприятия.

Типовая ИС в специальной базе метаинформации — репозитории — содержит модель объекта автоматизации, на основе которой осуществляется конфигурирование ПО. Таким образом, модельно-ориентированное проектирование ИС предполагает, прежде всего, построение модели объекта автоматизации с использованием специального программного инструментария. Возможно также создание системы на базе типовой модели ИС из репозитория, который поставляется вместе с программным продуктом и расширяется

по мере накопления опыта проектирования ИС для различных отраслей и типов производства.

Реализация типового проекта предусматривает выполнение следующих операций:

- установку глобальных параметров системы;
- задание структуры объекта автоматизации;
- определение структуры основных данных;
- задание перечня реализуемых функций и процессов;
- описание интерфейсов;
- описание отчетов;
- настройку авторизации доступа;
- настройку системы архивирования.

Следствием недостатков классических методов проектирования явился переход к системному проектированию.

Системный подход оперирует с рядом категориальных понятий. Фундаментальным понятием системного подхода является понятие системы, определяя которую необходимо преследовать определенную цель. Если целью является познание уже существующей системы, то вполне пригодным оказывается дескриптивное определение системы, которая заключается в следующем: система — это совокупность объектов, свойства которой определяются отношением между этими объектами. Объекты называют подсистемами или элементами системы. Каждый объект при самостоятельном исследовании может рассматриваться как система. Функции объекта определяются внутренним устройством объекта. Таким образом, дескриптивное определение системы играет познавательную роль для объяснения функций, реализуемых системой. Функции системы проявляются в процессе взаимодействия ее с внешней средой. При этом важно определить границу между внешней средой и создаваемой системой. Это можно осуществить на основе конструктивного определения системы для технических систем особое значение имеет конструктивный подход.

Любая техническая система создается под заранее известную цель. Цель такой системы обычно является субъективной, поскольку она предлагается разработчиком, но эта цель должна исходить из объективных потребностей общества. Таким образом, можно считать, что цель формируется в процессе взаимодействия между явлениями окружающей нас действительности. При этом возникает ситуация, которая заставляет строить новую систему. Ситуация может стать проблемной, если она не разрешается имеющимися средствами. Могут создаваться новые недостающие средства, и в этом смысле ярким примером является информационная технология.

Архитектурное описание самым тесным образом связано с процессом *проектирования* ИС, причем в ряде определений термина «архитектура» на этот факт указывается в явном виде.

Обычно выделяются пять различных подходов в проектировании, которые называют также стилями проектирования и, по существу, определяют группы методологий разработки ПО:

1) стиль, основанный на календарном планировании (*Calendar-driven*);

2) стиль, основанный на управлении требованиями (*Requirements-driven*);

3) стиль, в основу которого положен процесс разработки документации (*Documentation-driven*);

4) стиль, основанный на управлении качеством (*Quality-driven*);

5) архитектурный стиль (*Architecture-driven*).

Основой ***календарного*** стиля является график работ. Команда разработчиков выполняет работы поэтапно. Проектные решения принимаются из целей и задач конкретного этапа. Слабыми местами данного стиля являются то, что основные решения принимаются исходя из локальных целей, при этом мало внимания уделяется процессу разработки, разработке документации, созданию стабильных архитектур и процессу внесения изменений. Все это приводит к высокой суммарной стоимости владением в долгосрочном плане. Данный стиль считается морально устаревшим, однако многие организации продолжают его использовать.

Стиль, основанный на управлении требованиями или чертами (*features*), предполагает, что основное внимание уделяется функциональным характеристикам системы, при этом часто недостаточно внимания уделяется нефункциональным характеристикам, таким как масштабируемость, портабельность, поддерживаемость и другим, определенным в *ISO* 9126. Проектные решения принимаются преимущественно исходя из локальных целей, связанных с реализацией тех или иных функций. Данный подход достаточно эффективен, в случае если требования определены и не изменяются в процессе проектирования. Основными недостатками данного подхода являются следующие: недостаточное внимание уделяется требованиям стандарта *ISO* 9126 (управление качеством); разрабатываемые архитектуры, как правило, не являются стабильными, так как каждая из реализуемых функций отображается на один или несколько функциональных компонентов. Это обстоятельство усложняет добавление к системе новых требований. Данный подход позволяет успешно отслеживать требования в плане реализации требуемой функциональности, однако использование его для долгосрочных проектов является неэффективным.

Стиль, в основу которого положен процесс разработки документации, до настоящего времени продолжает использоваться в правительственных организациях и крупных компаниях. Данный стиль может рассматриваться как вырожденный вариант стиля, основанного на управлении качеством, и ориентирован на разработ-

ку документации. В качестве основных недостатков данного подхода выделяются следующие: неоправданно много времени и сил затрачивается на разработку документации в ущерб качеству разрабатываемого кода. При этом создаваемая документация часто не используется ни пользователем, ни заказчиком.

Стиль, основанный на управлении качеством, предполагает самое широкое использование различных мер для отслеживания критичных, с точки зрения функционирования, параметров. Например, требуется получить время реакции системы на запрос менее одной секунды, или обеспечить среднее время между отказами не менее 10 лет. В этом случае данные параметры отслеживаются на всех этапах проектирования систем, и часто это делается в ущерб другим характеристикам, таким как масштабируемость, простота сопровождения и т. п. Типовыми проблемами, возникающими при использовании данного стиля, являются следующие: часто оптимизируются не те параметры, которые должны быть в действительности, когда появляются новые требования, бывает очень трудно изменить функциональность системы, созданные таким образом системы обычно не отличаются высоким качеством архитектурных решений. В целом данный стиль считается консервативным. Его использование может быть оправдано в случае, когда необходимо создать системы с экстремальными характеристиками.

Архитектурный стиль представляет собой наиболее зрелый подход к проектированию. В рамках данного стиля во главу угла ставится создание фреймворков (набора типовых решений, в том числе программных), которые могут быть легко адаптированы ко всем потенциальным требованиям всех потенциальных заказчиком. Отличительная особенность данного стиля состоит в том, что задача проектирования разбивается на две отдельные подзадачи: создание многократно используемого фреймворка и создание конкретного приложения (системы) на его основе. При этом эти две задачи могут решать разные специалисты. Основная цель создания данного стиля — это устранение недостатков стиля, основанного на управлении требованиями. Использование архитектурного стиля позволяет реализовать инкрементное и итеративное проектирование, т. е. оперативно изменять существующую и добавлять новую функциональность.

В качестве примера рассмотрим методологию создания информационных систем, основанную на использовании средств быстрой разработки приложений (*Rapid Application Development* (далее — *RAD*) и методологию создания корпоративных информационных систем.

7.1.1. Методология *RAD*

Данная методология охватывает все этапы жизненного цикла современных информационных систем.

Методология *RAD* — это комплекс специальных инструментальных средств, позволяющих оперировать с определенным набором графических объектов, функционально отображающих отдельные информационные компоненты приложений [18].

Основные принципы методологии RAD можно свести к следующим:

- используется итерационная (спиральная) модель разработки;
- полное завершение работ на каждом из этапов жизненного цикла не обязательно;
- в процессе разработки информационной системы обеспечивается тесное взаимодействие с заказчиком и будущими пользователями;
- *CASE*-средства и средства быстрой разработки приложений;
- применяются средства управления конфигурацией, облегчающие внесение изменений в проект и сопровождение готовой системы;
- используются прототипы, позволяющие полнее выяснить и реализовать потребности конечного пользователя;
- тестирование и развитие проекта осуществляются одновременно с разработкой;
- разработка ведется немногочисленной и хорошо управляемой командой профессионалов;
- обеспечиваются грамотное руководство разработкой системы, четкое планирование и контроль выполнения работ.

Объектно-ориентированный подход. Средства *RAD* позволили реализовать совершенно иную по сравнению с традиционной технологию создания приложений: информационные объекты формируются как некие действующие модели (прототипы), чье функционирование согласуется с пользователем, а затем разработчик может переходить непосредственно к формированию законченных приложений, не теряя из виду общей картины проектируемой системы.

Использование объектно-ориентированных принципов позволяет создать описание (модель) предметной области в виде совокупности объектов — сущностей, объединяющих данные и методы обработки этих данных (процедуры). Каждый объект обладает собственным поведением и моделирует некоторый объект реального мира. С этой точки зрения объект является вполне осязаемым и демонстрирует определенное поведение.

Применение принципов объектно-ориентированного программирования позволило создать средства проектирования приложений, называемые средствами *визуального программирования*. Визуальные инструменты *RAD* позволяют создавать сложные графические интерфейсы пользователя вообще без написания кода программы. При этом разработчик может на любом этапе наблюдать то, что закладывается в основу принимаемых решений. Визуальные

средства разработки оперируют в первую очередь со стандартными интерфейсными объектами — окнами, списками, текстами, которые легко можно связать с данными из базы данных и отобразить на экране монитора. Другая группа объектов представляет собой стандартные элементы управления — кнопки, переключатели, флажки, меню и т. п., с помощью которых осуществляется управление отображаемыми данными.

Логика приложения, построенного средствами *RAD*, является событийно-ориентированной. Это означает, что каждый объект, входящий в состав приложения, может генерировать события и реагировать на события, генерируемые другими объектами. Примерами событий могут быть открытие и закрытие окон, щелчок на кнопке, нажатие клавиши клавиатуры, движение мыши, изменение данных в базе данных и т. п.

Фазы жизненного цикла в рамках методологии RAD. При использовании методологии быстрой разработки приложений жизненный цикл информационной системы состоит из четырех фаз:

1) анализа и планирования требований;
2) проектирования;
3) построения;
4) внедрения.

Рассмотрим каждую из них более подробно.

1. *Фаза анализа и планирования требований* содержит определение следующих характеристик:

- функции, которые должна выполнять разрабатываемая информационная система;
- наиболее приоритетные функции, требующие разработки в первую очередь;
- информационные потребности;
- масштаб проекта;
- временные рамки для каждой из последующих фаз;
- сама возможность реализации данного проекта в установленных рамках финансирования на имеющихся аппаратных и программных средствах.

Если реализация проекта принципиально возможна, то результатом фазы анализа и планирования требований будет список функций разрабатываемой информационной системы с указанием их приоритетов, а также предварительные функциональные и информационные модели системы.

2. На *фазе проектирования* необходимым инструментом являются *CASE*-средства, используемые для быстрого получения работающих прототипов приложений. Прототипы, созданные с помощью *CASE*-средств, анализируются пользователями, которые уточняют и дополняют те требования к системе, которые не были выявлены на предыдущей фазе. Таким образом, на данной фазе также необхо-

димо участие будущих пользователей в техническом проектировании системы.

Далее на этой фазе проводится анализ и, если требуется, корректировка функциональной модели системы. Детально рассматривается каждый процесс системы. При необходимости для каждого элементарного процесса создается частичный прототип: экран, диалоговое окно или отчет (это позволяет устранить неясности или неоднозначности). Затем определяются требования разграничения доступа к данным.

После детального рассмотрения процессов определяется количество функциональных элементов разрабатываемой системы. Это позволяет разделить информационную систему на ряд подсистем, каждая из которых реализуется одной командой разработчиков за приемлемое для *RAD*-проектов время (порядка полутора месяцев). С использованием *CASE*-средств проект распределяется между различными командами — делится функциональная модель.

На этой же фазе происходит определение набора необходимой документации. Результатами данной фазы являются:

- общая информационная модель системы;
- функциональные модели системы в целом и подсистем, реализуемых отдельными командами разработчиков;
- точно определенные с помощью *CASE*-средства интерфейсы между автономно разрабатываемыми подсистемами;
- построенные прототипы экранов, диалоговых окон и отчетов.

3. На *фазе построения* выполняется собственно быстрая разработка приложения. На данной фазе разработчики производят итеративное построение реальной системы на основе полученных ранее моделей, а также требований нефункционального характера. Разработка приложения ведется средствами визуального программирования. Формирование программного кода частично выполняется с помощью автоматических генераторов кода, входящих в состав *CASE*-средств. Код генерируется на основе разработанных моделей.

После окончания работ каждой отдельной команды разработчиков производится постепенная интеграция данной части системы с остальными, формируется полный программный код, выполняется тестирование совместной работы данной части приложения с остальными, а затем тестирование системы в целом.

Завершается физическое проектирование системы, а именно:

- определяется необходимость распределения данных;
- производится анализ использования данных;
- производится физическое проектирование базы данных;
- определяются требования к аппаратным ресурсам;
- определяются способы повышения производительности;
- завершается разработка документации проекта.

Результатом реализации данной фазы является готовая информационная система, удовлетворяющая всем требованиям пользователей.

4. *Фаза внедрения* в основном сводится к обучению пользователей разработанной информационной системы.

Ограничения методологии RAD. Ее применение наиболее эффективно при создании сравнительно небольших систем, разрабатываемых для конкретного заказчика.

При разработке же типовых систем, не являющихся законченным продуктом, а представляющих собой совокупность типовых элементов информационной системы, большое значение имеют такие показатели проекта, как управляемость и качество, которые могут войти в противоречие с простотой и скоростью разработки. Это связано с тем, что типовые системы обычно централизованно сопровождаются и могут адаптироваться к различным программно-аппаратным платформам, системам управления базами данных, коммуникационным средствам, а также интегрироваться с существующими разработками. Поэтому для такого рода проектов необходимы высокий уровень планирования и жесткая дисциплина проектирования, строгое следование заранее разработанным протоколам и интерфейсам, что снижает скорость разработки.

Методология *RAD* не подходит для создания сложных расчетных программ, операционных систем и программ управления сложными инженерно-техническими объектами; для разработки приложений, в которых интерфейс пользователя является вторичным, т. е. отсутствует наглядное определение логики работы системы (приложения реального времени, драйверы или службы); для разработки систем, от которых зависит безопасность людей, например систем управления транспортом или атомными электростанциями.

7.1.2. Профили открытых ИС

Открытые ИС создаются в процессе информатизации всех основных сфер современного общества: органов государственного управления, финансово-кредитной сферы, информационного обслуживания предпринимательской деятельности, производственной сферы, науки, образования. Развитие и использование открытых информационных систем неразрывно связаны с применением стандартов на основе методологии функциональной стандартизации информационных технологий.

Профиль — это совокупность нескольких (или подмножество одного) базовых стандартов с четко определенными и гармонизированными подмножествами обязательных и факультативных возможностей, предназначенная для реализации заданной функции или группы функций.

Базовые стандарты и профили в зависимости от проблемно-ориентированной области применения информационных систем могут использоваться как непосредственные директивные, руководящие или рекомендательные документы, а также как нормативная база, необходимая при выборе или разработке средств автоматизации технологических этапов или процессов создания, сопровождения и развития информационных систем.

Обычно рассматривают две группы профилей, регламентирующих:

1) архитектуру и структуру ИС;

2) процессы проектирования, разработки, применения, сопровождения и развития системы.

В международной функциональной стандартизации информационных технологий принято довольно жесткое понятие профиля. Считается, что его основой могут быть только утвержденные международные и национальные стандарты. Использование стандартов де-факто и нормативных фирменных документов не допускается.

Другой подход к разработке и применению профилей ИС состоит в использовании совокупности адаптированных и параметризованных базовых международных и национальных стандартов и открытых спецификаций, отвечающих стандартам де-факто и рекомендациям международных консорциумов.

Эталонная модель среды открытых систем определяет разделение любой информационной системы на две составляющие: *приложения* (прикладные программы и программные комплексы) и *среду*, в которой эти приложения функционируют.

Профили ИС с иерархической структурой могут включать в себя:

- стандартизованные описания функций, выполняемых данной системой;
- функции взаимодействия системы с внешней для нее средой;
- стандартизованные интерфейсы между приложениями и средой информационной системы;
- профили отдельных функциональных компонентов, входящих в систему.

Структура профилей ИС. Профили характеризуют каждую конкретную ИС на всех стадиях ее жизненного цикла, задавая согласованный набор базовых стандартов, которым должны соответствовать система и ее компоненты.

Стандарты, важные с точки зрения заказчика, должны задаваться в техническом задании на проектирование системы и составлять ее первичный профиль. То, что не задано в техническом задании, первоначально остается на усмотрение разработчика системы. Профиль конкретной системы не является статичным.

В профиль конкретной системы включаются спецификации компонентов, разработанных в составе данного проекта, и специфика-

ции использованных готовых программных и аппаратных средств, если эти средства не специфицированы соответствующими стандартами. После завершения проектирования и испытаний системы, в ходе которых проверяется ее соответствие профилю, профиль применяется как основной инструмент сопровождения системы при эксплуатации, модернизации и развитии.

На стадиях жизненного цикла информационной системы выбираются и затем применяются следующие основные функциональные профили:

- прикладного программного обеспечения;
- среды информационной системы;
- защиты информации в информационной системе;
- инструментальных средств, встроенных в информационную систему.

7.1.3. Методология создания корпоративных ИС

Рассматриваемая методология создания корпоративных ИС состоит из двух основных взаимосвязанных частей:

1) методологии анализа ИС, включающей описание деятельности организации и формирование требований к ИС на основе бизнес-процессов;

2) методологии проектирования от данных, предназначенной для проектирования и быстрой разработки программного и информационного обеспечения ИС.

Методология является развитием одного из классических направлений с включением новых разработок ИТ и содержит:

- итерационную спиральную модель жизненного цикла ИС;
- комплекс развивающихся систем согласованных моделей;
- технологию анализа ИС на основе бизнес-процессов;
- технологию проектирования от данных;
- комплекс согласованных инструментальных средств.

Итерационная спиральная модель жизненного цикла ИС. Процесс создания и сопровождения информационных систем в виде жизненного цикла (ЖЦ) ИС представляется в виде последовательности стадий, каждая из которых разбита на этапы, и выполняемых на них процессов. Для каждого этапа определяются последовательность выполняемых работ, получаемые результаты, методы и средства, необходимые для выполнения работ, роли и ответственность участников и т. д. Такое формальное описание ЖЦ ИС позволяет спланировать и организовать процесс коллективной разработки и обеспечить управление этим процессом.

Жизненный цикл ИС можно представить как ряд событий, происходящих с системой в процессе ее создания и использования. Модель жизненного цикла отражает различные состояния системы начиная с момента возникновения необходимости в данной ИС и заканчивая

моментом ее полного выхода из употребления. Модель жизненного цикла — структура, содержащая процессы, действия и задачи, которые осуществляются в ходе разработки, функционирования и сопровождения программного продукта в течение всей жизни системы, от определения требований до завершения ее использования.

Жизненный цикл ИС, определяемый методологией, приведен в табл. 7.1. Он включает стадии анализа, проектирования, разработки, тестирования и интеграции, внедрения, сопровождения и развития ИС. В табл. 7.1 приведены также перечень основных этапов для каждой стадии ЖЦ и процессы, выполняемые на протяжении всего ЖЦ — процессы управления и интегральные процессы. Эти процессы в той или иной степени присутствуют на каждом из этапов.

Таблица 7.1

Жизненный цикл ИС

Процессы организации и управления проектом: планирование, управление, контроль					
Анализ	**Проектирование**	**Разработка**	**Интеграция и тестирование**	**Внедрение**	**Сопровождение**
Обследование и создание моделей деятельности организации. Анализ (моделей) существующих ИС. Анализ моделей и формирование требований к ИС. Разработка плана создания ИС	Концептуальное проектирование. Разработка архитектуры ИС. Проектирование общей модели данных. Формирование требований к приложениям	Разработка, прототипирование и тестирование приложений. Разработка интеграционных тестов. Разработка пользовательской документации	Интеграция и тестирование приложений в составе системы. Оптимизация приложений и баз данных. Подготовка эксплуатационной документации. Тестирование системы	Обучение пользователей. Развертывание системы на месте эксплуатации. Инсталляция баз данных Эксплуатация. Проведение приемосдаточных испытаний (ПСИ)	Регистрация, диагностика и локализация ошибок. Внесение изменений и тестирование. Управление режимами работы ИС
Интегральные процессы: управление конфигурацией, документирование, проверки, интеграция					

Процесс создания ИС представляет собой процесс построения и последовательного преобразования согласованных моделей на всех этапах ЖЦ. Эти модели сохраняются и накапливаются в репозитории проекта. С помощью *CASE*-средств модели создаются, преобразуются и контролируются. Основными результатами на каждом этапе

ЖЦ являются модели определяемых на данном этапе объектов (организации, требований к ИС, проекта ИС, требований к приложениям и т. д.). Характер выполняемых процессов и организация работ в представленной модели ЖЦ основаны на подходе информационного инжиниринга и отличаются от классической каскадной модели ЖЦ, несмотря на внешнюю схожесть. При традиционной обработке данных разработка велась строго последовательно. Требования ТЗ утверждались в начале разработки, а их выполнение проверялось в конце. Переход от стадии к стадии, от этапа к этапу допускался только после полного выполнения всего перечня работ и получения всех запланированных результатов ЖЦ ИС, предлагаемый в новой методологии определяется следующими особенностями:

- современные средства *CASE*, *4GL*, СУБД и др. предоставляют возможности быстрого проектирования, прототипирования, разработки и тестирования приложений и баз данных на основе построенных моделей;
- методология предполагает активное участие заказчиков на всех этапах создания ИС, поскольку модели, создаваемые на каждом этапе, понятны и разработчику и заказчику.

Предусмотрены возможности оперативного и быстрого пересмотра требований и разработанных решений на основе современных средств, возможности неравномерной, параллельной разработки различных частей проекта, возможности возврата на предыдущие этапы по отдельным частям проекта при необходимости внесения изменений, версионный характер изменения проекта или его частей при поддержке *CASE*-средств. Это определяет итерационный, спиральный характер предлагаемой модели жизненного цикла.

Комплекс развивающихся систем согласованных моделей. Методология предусматривает создание корпоративных информационных систем как процесс построения и последовательного развития систем согласованных моделей, начиная от системы моделей, описывающих деятельность организации, и заканчивая готовой информационной системой. Модели должны создаваться, преобразовываться и контролироваться с помощью соответствующих *CASE*-средств и сохраняться в репозитории. Исходными являются модели бизнес-процессов, на основе которых строится компьютерная модель организации, описанная в терминах бизнес-процессов и бизнес-функций. Из этой модели может быть получено большинство важнейших требований к ИС. Создается система моделей описания требований к ИС, которая затем преобразуется в систему моделей, описывающих проект ИС. Формируются модели архитектуры ИС, требований к ПО и информационному обеспечению (ИОБ). Затем формируется архитектура ПО и ИО, выделяются корпоративные БД и отдельные приложения, формируются модели требований к приложениям и проводится их разработка, тестирование и интеграция.

Технология анализа ИС на основе бизнес-процессов. Основу деятельности любой организации составляют ее бизнес-процессы, которые определяются целями и задачами организации. Процессы обеспечивают реализацию всех видов деятельности организации, связанных с производством товаров и (или) услуг, которые корпорация либо производит, либо продает и поставляет, либо делает все это в совокупности. Каждый бизнес-процесс характеризуется четко определенными во времени началом и концом, внешними интерфейсами, которые либо связывают его с другими бизнес-процессами внутри организации, либо описывают выход во внешнее окружение, последовательностью выполняемых работ и правилами их выполнения (бизнес-правилами). Для каждой работы, входящей в бизнес-процесс, установлены временны́е характеристики, определяющие ее место в общей последовательности работ, условия инициации и время выполнения. В отличие от описания организации на основе иерархической функциональной структуры, которую невозможно объективно оценить, описание на основе процессов позволяет точно представить цели, характеристики (в том числе динамические) и конечный результат каждого вида деятельности организации. Исходя из того, что основные бизнес-процессы реализуют по своей природе цели и задачи организации, методология предлагает строить описание деятельности организации как процесс создания и развития систем согласованных моделей, основанных на моделях бизнес-процессов. В процессе детализации моделей и их последующей интеграции должно обеспечиваться сохранение всех функциональных свойств, отражающих цели и задачи организации, и согласованности моделей. Такая согласованность обеспечивается методологией и поддерживающими ее современными *CASE*-средствами.

В процессе описания организации и ее деятельности формируются три основных системы моделей организации: *стратегическая*, *укрупненная* и *детальная*. Все эти системы моделей, описывая основные аспекты организации и ее деятельности, базируются на бизнес-процессах. В систему моделей описания организации добавлена также дополнительная система моделей, для того чтобы можно было учесть аспекты, не связанные с бизнес-процессами, но необходимые при создании ИС.

Назначение ***стратегической системы моделей*** заключается, во-первых, в определении основных целей и задач организации и, во-вторых, в формировании моделей бизнес-процессов, описывающих основные виды деятельности организации и реализующих ее стратегические цели и задачи.

В ***укрупненной системе моделей*** отображаются основные бизнес-процессы, которые описаны на стратегическом уровне, исходя из основных целей и задач организации (без привязки к ее струк-

туре), на реальную иерархически-функциональную структуру организации, а также выделяются основные функции подразделений и уточняются состав и характеристики бизнес-процессов.

Все создаваемые далее модели организации строятся на базе построенных бизнес-процессов по результатам обследования деятельности организации, проводимого на уровне подразделений. Модели строятся с помощью *CASE*-средств и сохраняются в репозитории проекта. Построение этих моделей допускает распараллеливание работ при проведении обследования и при построении моделей.

Главными целями создания ***детальной системы моделей*** является построение концептуальной модели данных и функциональной модели организации. Для достижения этих целей проводится детализация описания деятельности организации от уровня описания реализации общих бизнес-процессов в организации и списковых моделей в подразделениях до уровня детальных моделей подразделений, позволяющих выделить все функции подразделений, обрабатываемые документы, основные данные, описать регламент работы персонала и создать в итоге функциональную модель организации и концептуальную модель данных.

Дальнейшее развитие систем согласованных моделей происходит на базе схемы преобразования моделей, представленной в табл. 7.2, которая описывает развитие согласованных моделей по основным аспектам, определяющим деятельность организации (данные, функции, люди, сеть, время, правила). Каждая строка таблицы соответствует системе согласованных моделей данного уровня и раскрывается в виде частных моделей по каждой группе характеристик. Переход от строки к строке показывает развитие системы моделей в процессе описания организации, а правила перехода определяются методологией и обеспечивают полноту и согласованность при построении моделей.

Технология проектирования от данных. Поскольку данные составляют основу деятельности любой организации и являются наиболее стабильной ее составляющей (функции и структура организации меняются гораздо чаще), то при построении корпоративной ИС наиболее адекватным решаемым задачам является подход к проектированию, основанный на данных. Такой подход обеспечивает наилучшее архитектурное решение при разбиении системы на приложения, а также простоту и согласованность при интеграции приложений.

Комплекс согласованных инструментальных средств. Предлагаемая методология создания ИС поддерживается комплексом согласованных между собой инструментальных средств, который обеспечивает непрерывный цикл автоматизации процессов, выполняемых на всех этапах ЖЦ ИС. Согласованность этих средств обеспечивается наличием интерфейсов для прямого взаимодействия и поддержкой общепринятых стандартов открытых систем.

Таблица 7.2

Схема преобразования моделей

Уровень описания/группы характеристик — моделей	Функции	Данные	Люди	Сеть	Время	Правила (мотивация)
Укрупненная система моделей организации	Список бизнес-процессов и бизнес-функций организации	Списки документов Списки объектов	Структурная модель организации	Перечень структурных подразделений и внешних организаций	Список работ во времени	Список целей и задач организации критерии и правила выполнения бизнес-процессов
Детальная система моделей организации	Функциональные модели подразделений: диаграммы потоков данных	Информационная модель организации (концептуальная модель)	Структурные модели подразделений, роли персонала	Логическая модель сетей подразделений организации и внешних связей	Временная модель выполнения бизнес-процессов и бизнес-функций	Критерии выполнения бизнес-функций; бизнес-правила
Система моделей требований к информационным управляющим системам (ИУС)	Требования к функциям; диаграмма потоков данных	Требования к данным; структуры данных концептуальная модель данных	Требования к регламенту и интерфейсу пользователей	Требования к сетевой архитектуре системы	Требования к временным характеристикам функции	Требования к регламенту работы ис
Детализация проекта	↓	↓	↓	↓	↓	
Функционирующая система	Функции	Данные	Пользователи	Коммуникации	Зависимость	Правила

Методология и поддерживающий ее набор инструментальных средств обеспечивают полный контроль и гибкое управление ходом разработки, включая:

- поддержку коллективной разработки с возможностью параллельного и распределенного выполнения различных работ;
- возможность перехода к следующему этапу (шагу), не дожидаясь полного завершения предыдущего;
- применение методов контроля качества и постоянный контроль полученных результатов;
- поддержку итеративного характера разработки (возможность пересмотра полученных результатов и возврата на любой из предыдущих этапов);
- возможность быстрого внесения изменений в требования в процессе разработки;
- управление конфигурацией.

7.2. Технологии реализации информационных систем

Термин «*компонент*» в ИТ-отрасли используется для обозначения различных понятий. Можно выделить аппаратные (*hardware components*) и программные компоненты (*software components*).

Термин «**программный компонент**» (далее — ПрК) используется для обозначения двух связанных, но разных понятий. Если речь идет о программной архитектуре, то обычно под *компонентом* подразумевается программный модуль, реализующий некоторую функцию или набор функций, и решает определенные подзадачи в рамках общих задач, решаемых системой. Это те компоненты, которые могут быть изображены на диаграммах компонентов (*component diagram*) в языке *UML*. Если речь идет о компонентных технологиях программирования или компонентно-ориентированной (*component based*) разработке программного обеспечения (ПО), то под ПрК понимают объекты со специальными свойствами. В дальнейшем термин «компонент» будет употребляться во втором значении.

В этом случае понятие ПрК выступает в качестве в качестве ключевого для определения понятия таких понятий, как «компонентно-ориентированное программирование» и «компонентно-ориентированный подход к проектированию ИС».

Согласно [18] **программный компонент** представляет собой структурную единицу программной системы, обладающую четко определенным интерфейсом, который полностью описывает ее зависимости от контекста.

ПК представляет собой откомпилированный автономный программный модуль, который можно объединять с другими модуля-

ми или кодом, организованным другим способом с целью создания приложений. ПрК могут быть как простым, таким как кнопка, или сложным, таким как компонент, реализующий управление сетью.

Чаще всего под ПрК понимают откомпилированный «двоичный компонент», который можно интегрировать в приложение «на лету». При работе с компонентами исходный код обычно недоступен, и поэтому компонент нельзя изменять. Под данное определение подпадают, в частности, динамические библиотеки.

Компонентно-ориентированное приложение обычно состоит из множества компонентов. Само приложение представляет собой некоторую среду, включающую средства «склеивания» или скелетный код, в который можно встраивать ПрК, которые могут взаимодействовать со средой и операционной системой и друг с другом. Изменение интерфейса ПрК приводит к изменению его кода, но изменение способа реализации не обязательно приводит к изменению интерфейса.

ПрК можно рассматривать и как пакет, ориентированный на повторное использование кода, который можно приобрести у независимого производителя.

Типовой ПрК обладает следующими свойствами:

- представляет собой фрагмент самодостаточного кода, т. е. для его функционирования не требуется наличия дополнительные библиотек;
- является самоустановливаемым модулем, который может быть включен в состав системы, исключен из ее состава или заменен на другой модуль, например, принадлежащий другому производителю, при минимальном участии пользователя;
- может повторно использоваться в различных контекстах;
- при работе с ПК используются механизмы динамического связывания;
- можно объединять с другими ПК с целью создания более крупных ПК;
- пользователи используют ПК преимущественно по принципу черного ящика, т. е. пользователю известны только интерфейсы, но не внутренняя структура системы.

ПрК может быть независимо поставлен или не поставлен, добавлен в состав некоторой системы или удален из нее, в том числе может включаться в состав систем других поставщиков.

ПрК можно определить как фрагмент самодостаточного, самоустанавливаемого кода, обладающий хорошо определенной функциональностью и имеющий четко определенные интерфейсы. ПрК могут агрегатироваться с другими компонентами в более сложные системы. Компоненты и системы компонентов могут повторно использоваться в различных контекстах. Пользователи используют компоненты преимущественно по принципу черного ящика, т. е.

пользователю известны только интерфейсы, но не внутренняя структура ПрК.

Для функционирования ПрК, как правило, необходимо наличие соответствующей инфраструктуры, которая позволяет компонентам находить друг друга и взаимодействовать по определенным правилам. Набор правил, определяющих интерфейсы ПК и их реализаций, а также правил, по которым ПрК работают в системе и взаимодействуют друг с другом, называют *компонентной моделью* (*component model*) [18]. В компонентную модель входят также правила, регламентирующие жизненный цикл ПрК. Взаимодействовать друг с другом могут только ПК, построенные в рамках одной модели.

Для работы ПрК необходим некоторый набор *базовых служб* (*basic services*), которые обеспечивают, например, нахождение компонентов в распределенной среде, обеспечение обмена данными через сеть.

Набор таких базовых, необходимых для функционирования большинства компонентов служб, вместе с поддерживаемой с их помощью компонентной моделью называется *компонентной средой*, или *компонентным фрейворком* (*component framework*).

Компонентные технологии можно рассматривать как одну из фаз развития технологий разработки распределенных систем, при этом можно выделить следующие ***основные фазы*** (рис. 7.5):

— сокеты;
— вызов удаленных процедур;
— системы распределенных объектов;
— компонентные технологии;
— сервисно-ориентированные системы.

Перечисленные технологии появлялись именно в таком порядке. Переход к следующей фазе можно рассматривать как достижение некоторого уровня зрелости, поскольку технологии нижележащего уровня используются в качестве сервисов более низкого уровня. В частности, вызов удаленных процедур основывается на использовании сокетов, а системы распределенных объектов, в свою очередь, базируются на вызове удаленных процедур, компонентные технологии могут использовать такие механизмы, как *RMI*, а вебсервисы могут быть реализованы как компоненты.

Известно достаточно много различных компонентных технологий. Некоторые из них остались на уровне теоретических исследования, однако ряд технологий активно используется на практике в течение уже многих лет. К последней группе можно отнести такие компонентно-ориентированные (*component based*) технологии, как *JavaBeans*, *EJB*, *CORBA*, *ActiveX*, *VBA*, *COM*, *DCOM*, *.Net* компоненты. Принципиально мультиагентные технологии также можно рассматривать как разновидность компонентных технологий.

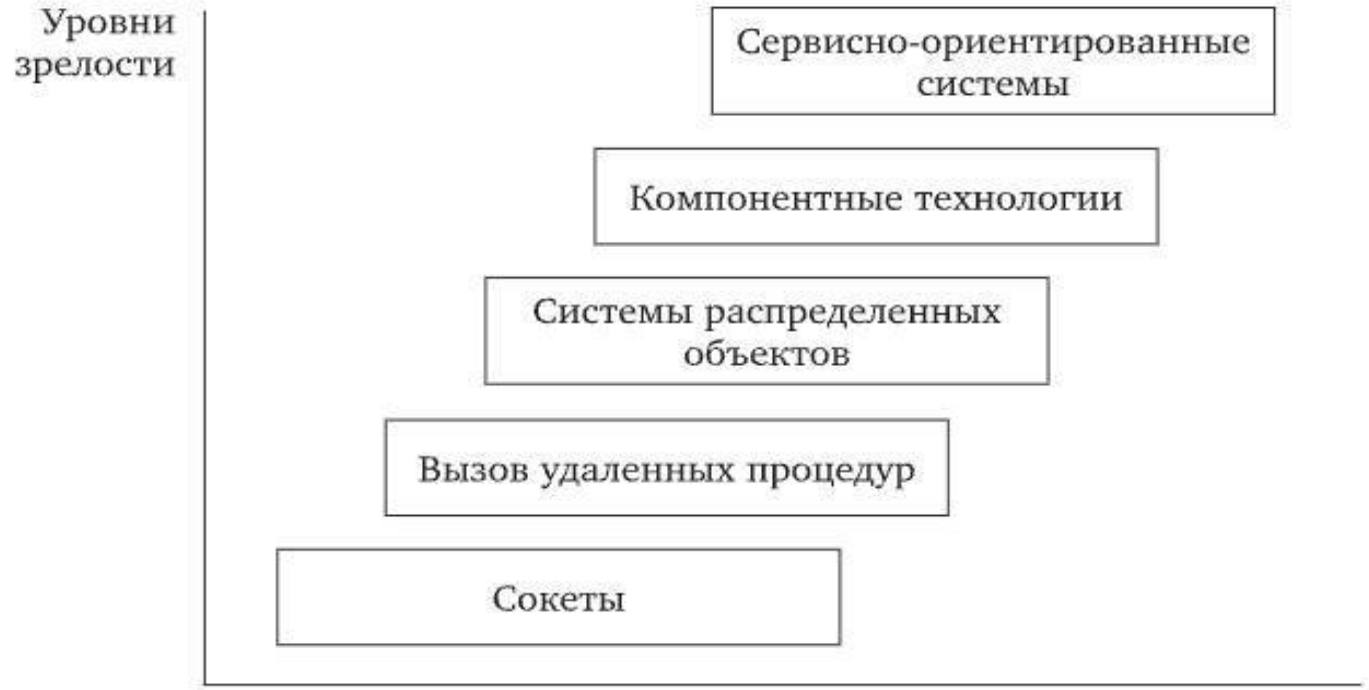


Рис. 7.5. **Фазы развития технологий разработки распределенных систем**

7.2.1. Сокеты

Самым старым способом взаимодействия между элементами распределенных приложений являются сокеты, которые позволяют связывать приложения напрямую, записывая данные в сокет и читая данные из сокета. *API* для сокетов является низкоуровневым и предоставляет максимальный контроль над передаваемыми данными.

Однако сокеты плохо подходят для работы со сложными структурами данных, особенно если это программные компоненты, поэтому разработка сложных приложений с использованием сокетов достаточно проблематична.

Работа с сокетами в самом общем виде выглядит следующим образом. Любой процесс может создать серверный сокет и привязать его к какому-нибудь порту. После этого сервер переходит в режим ожидания и ожидает запрос со стороны клиента.

Клиент также создает сокет, через который он может взаимодействовать с сервером. Взаимодействие может реализовываться как с установлением соединения, так и без установления соединения. В первом случае в качестве транспорта используется протокол *TCP*, а во втором — *UDP*.

Механизм работы с сокетами поддерживается практически во всех современных языках программирования.

Самую подробную информацию по программированию сокетов можно найти в [18].

7.2.2. Вызов удаленных процедур

Основная идея механизма сокетов состоит в использовании операций *send* и *receive* для обмена данными между процессами, кото-

рые выполняются на разных хостах. Основной недостаток сокетов состоит в сложности программирования, поскольку программирование осуществляется на низком уровне.

Идея вызова удаленных процедур состоит в обеспечении возможности программам вызывать процедуры, находящиеся на других машинах. Когда процесс, запущенный на хосте А, вызывает процедуру с хоста В, вызывающий процесс на хосте А приостанавливается, а выполнение вызванной процедуры происходит на хосте В. Информация может быть передана от вызывающего процесса к вызываемой процедуре через параметры и возвращена процессу в виде результата выполнения процедуры. Для программиста вызов удаленной процедуры ничем не отличается от вызова локальной процедуры. Данный метод известен под названием удаленный вызов процедур (*RPC*).

Использование подпрограмм (процедур) в программе — традиционный способ структурировать задачу и сделать ее более ясной. Обычно подпрограммы собираются в библиотеки. Применительно к локальным вызовам данный подход используется давно и повсеместно.

Удаленный вызов процедуры существенным образом отличается от традиционного локального с точки зрения реализации, однако с точки зрения программиста такие отличия практически отсутствуют.

7.2.3. Среда распределенных вычислений

Механизм удаленных вызовов процедур был тщательно адаптирован для использования в качестве основы систем промежуточного уровня и вообще распределенных систем и достаточно хорошо работал при использовании таких языков программирования, как *C++*. Дальнейшим развитием данного подхода можно считать *среду распределенных вычислений* (*Distributed Computing Environment — DCE*), разработанную организацией *Open Software Foundation* (*OSF*), которая позже была переименована в *Open Group*. *DCE* изначально была разработана под *UNIX*, однако позже были созданы версии для других ОС.

DCE включает ряд сервисов, таких как поддержка работы с нитями (*Threads*), вызов удаленных процедур и аутентификация, служба времени, служба каталогов, служба безопасности и файловый сервис. Архитектура *DCE* показана на рис. 7.6.

Служба распределенных файлов (*Distributed File Service*) представляет собой всемирную файловую систему, предоставляющую прозрачные методы доступа к любому файлу системы одинаковым образом.

Служба каталогов (*Directory Service*) используется для отслеживания местонахождения любого из ресурсов системы. Служба каталогов позволяет процессу запрашивать ресурсы, не зная, где они

находятся, если это не необходимо для процесса. *Служба защиты* (*Security Sewice*) позволяет защищать ресурсы любого типа, кроме того, получение некоторых данных может быть открыто только тем, кому это разрешено.

Рис. 7.6. **Архитектура DCE**

Служба распределенного времени (*Distributed Time Service*) *предоставляет механизмы* синхронизации часов различных хостов.

Модель программирования, лежащая в основе всей системы *DCE*, — это модель «клиент — сервер», в которой связь между клиентами и серверами осуществляется посредством использования *RPC*.

Более подробную информацию по *DCE* можно найти в [18].

В *DCE* распределенные объекты были добавлены достаточно искусственно в качестве расширения вызовов удаленных процедур. При этом клиент работает со ссылкой на удаленную процедуру для объекта. Фактически ссылки на объекты отсутствуют.

В качестве полноценной системы работы с распределенными объектами можно рассматривать *Java RMI*. В *Java* распределенные объекты интегрированы с языком.

Java поддерживает распределенные объекты в форме удаленных объектов, т. е. объектов, тело которого постоянно находится на одном и том же хосте, а интерфейсы доступны удаленным процессам. Интерфейсы реализованы обычным образом через заглушки, которые предоставляют интерфейсы, идентичные интерфейсам удаленных объектов, при этом заглушка для клиента представляется в виде локального объекта, находящегося в адресном пространстве клиента.

7.2.4. Объектная модель компонентов (*COM*)

В самом общем виде идея *COM* состоит в том, что одна часть ПО должна получать доступ к сервисам, предоставляемым другой частью, причем используется доступ ко всем видам программных сервисов независимо от способа их реализации. *COM* используется стандартный механизм.

В *COM* любая часть программного обеспечения реализует свои сервисы как один или несколько объектов *COM*.

Каждый такой объект поддерживает один или несколько интерфейсов, состоящих из методов.

Каждый метод — это процедура или функция, которая выполняет требуемое действие и может быть вызвана программным обеспечением, использующим данный объект (*клиентом* объекта).

Клиенты получают доступ к сервисам объекта *COM* только через вызовы методов интерфейсов объекта — у них нет непосредственного доступа к данным объекта.

Объект *COM* может поддерживать более одного интерфейса, например, объект *COM*, показанный на рис. 7.7, имеет три интерфейса.

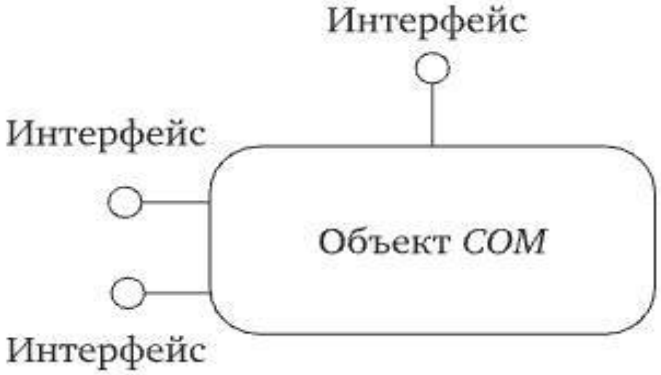


Рис. 7.7. **Пример объекта *COM***

Каждый интерфейс включает множество методов. В число этих методов входит три стандартных метода, которые будут рассмотрены далее, и произвольное число методов, определяемых пользователем. Однажды определенный интерфейс нельзя изменять и дополнять. Если появляется необходимость изменить интерфейс, то создается новый интерфейс.

Рассмотрим пример. Допустим, разработчиками был создан объект *COM*, для реализации проверки правописания, единственный интерфейс *ISpeller*, который кроме стандартных содержит три метода: *FindWord* ( ); *AddWord* ( ), *RemoveWord* ( ). которые позволяют находить слово в словаре, добавлять слово в словарь и удалять слово из словаря, соответственно.

Если позднее появляется идея расширить функциональные возможности объект COM за счет включения функции поиска синонимов, то, поскольку интерфейс нельзя изменять, то можно создать новый интерфейс, который можно назвать, например, *IThesaur*, который помимо трех стандартных методов содержит единственный метод *GetSynonym* ( ).

COM с самого начала разрабатывалась как распределенная система, в которой компоненты можно создавать на другом хосте и их методы по сети (реально распределенная *COM* (*Distributed COM — DCOM*) появилась в 1996 г.). *DCOM* можно рассматривать

как незначительное расширение оригинальной *COM*. По существу, *DCOM* добавляет к *COM* всего три основных элемента: способ создания удаленного объекта, протокол вызова методов удаленного компонента и механизмы обеспечения безопасного доступа к удаленному компоненту.

Технология *COM+* появилась в рамках *Windows* 2000 и основывается, с одной стороны, на *DCOM*, а с другой — на сервере транзакций (*Microsoft Transaction Server*). Основной целью создания *COM+* следует считать разработку компонентной модели, которая могла эффективно использоваться в ИС уровня крупного предприятия.

Отличительными особенностями среды *COM+* являются следующие:

- наличие эффективных механизмов работы с транзакциями;
- возможность реализации асинхронного взаимодействия с помощью очередей сообщений;
- наличие механизмов работы с событиями;
- улучшенные показатели безопасности;
- возможность работы с пулом объектов.

Все это позволяет создавать достаточно хорошо масштабируемые, стабильно работающие приложения уровня крупного предприятия.

Объекты *COM+*, являющиеся экземплярами компонент *COM+*, работают под управлением среды *COM+*. Набор связанных между собой компонент *COM+*, находящихся в одной динамической библиотеке, называется приложением *COM+*. Приложение *COM+* включает множество компонент и ролей для доступа к ним. Информация обо всех зарегистрированных приложениях находится в каталоге *COM+*.

В 2002 г., т. е. всего через два года после появления *COM+*, была официально выпущена платформа *Microsoft .NET*, которая была объявлена *Microsoft* рекомендуемой основой для создания приложений и компонентов под *Windows*, в рамках которой используется собственная компонентная модель, радикально отличающаяся *COM*.

7.2.5. *NET Framework*

NET Framework программная платформа компании *Microsoft*, предназначенная как для создания обычных программ и веб-приложений. *.NET* является патентованной технологией.

В основу данной платформы положена амбициозная идея сделать платформо независимую универсальную виртуальную машину, которая могла бы выполнять код, написанный на произвольном языке программирования в различных ОС без перекомпиляции кода. Однако со временем *Microsoft* ограничилась поддержкой только собственных ОС, предоставив независимым разработчикам заниматься поддержкой других платформ.

Основными составными частями *.NET Framework* являются инвариантная к языку программирования среда исполнения — *Common*

Language Runtime (далее — *CLR*) и библиотека классов *Framework* (*Framework Class Library — FCL*). *CLR* — это некоторая обертка для *API* ОС, которая служит средой для исполнения управляемых приложений (*Managed Applications*). *FCL* предоставляет объектно-ориентированный *API*, к которому обращаются управляемые приложения.

При работе с управляемыми приложениями программист теряет возможность работать с *Windows API*, *MFC*, *ATL*, *COM* и другими знакомыми инструментами и технологиями и должен работать с *FCL*.

Если программист не хочет отказываться от «старых» технологий, то ему предоставляется возможность создавать приложения, основанные на использовании неуправляемому коду, который исполняется без использования *CLR*. Совмещение в рамках одного приложения управляемого и неуправляемого кода возможно, но не приветствуется разработчиками платформы.

Инвариантная к языку программирования достигается за счет того, что среда разработки создает байт-код, который интерпретируется виртуальной машиной. Встроенные (поставляются вместе с *.NET Framework*). В качестве основных языков, поддерживаемых платформой *.NET*, выступают: *C#*, *VB.NET*, *JScript .NET*, *C++/CLI*, *IronPython*, *IronRuby* и *F#* (функциональный язык общего назначения).

В качестве входного языка виртуальной машины в *.NET* используется *Common Intermediate Language* (далее — *CIL*) (в более ранних версиях он назывался *Microsoft Intermediate Language* (далее — *MSIL*)).

7.2.6. Компонентная модель *.NET*

Компонент *.NET* представляет собой прекомпилированный *MSIL*, который может быть помещен в любой другой *MSIL* компонент или клиентскую программу без выполнения компиляции.

Компонент *.NET* представляет собой прекомпилированный самоописываемый *MSIL*-модуль, построенный на базе одного или более классов или модулей, расположенных в *DLL* файле сборки (*DLL assembly file*) [18]. Сборки представляют собой базовые строительные блоки, которые обеспечивают развертывание и выполнение приложений *.NET*.

Исполняемый файл, библиотека или файл ресурсов, созданный на платформе *.NET*, представляет собой сборку. Кроме того, сборки обеспечивают реализацию механизма работы с версиями, область действия типов и ресурсов, зависимости и управление правами доступа.

7.2.7. Технология *CORBA*

Основы стандарта *CORBA* ранее рассмотрены в параграфе 3.5. Данный стандарт весьма важен при проектировании ИС, так как позволяет на основе спецификаций связать воедино компоненты,

написанные на разных языках программирования и функционирующих в распределенной системе на различных машинах.

Ключевыми компонентами данного стандарта являются объектный брокер запросов и язык определения интерфейсов *IDL*. Следующими по важности элементами являются сервис динамического формирования запросов, репозиторий интерфейсов, сервис динамической обработки запросов и сервис, обеспечивающий взаимодействие различных брокеров запросов. Основная идея состоит в том, что приложение представляется в виде множества объектов. Каждый из объектов содержит информацию о возможностях кода и способах его вызова. Объект может быть вызван локально или через сеть из других приложений или объектов *CORBA*.

Обобщенная структура системы, построенной с использованием *CORBA*, показана на рис. 7.8. Основу *CORBA*-системы составляет «системная шина», которая позволяет компонентам, реализованным на разных языках и работающих на разных платформах находить друг друга и взаимодействовать. Эту шину называют брокером объектных запросов (*Common Object Request Broker Architecture*) или просто *ORB*.

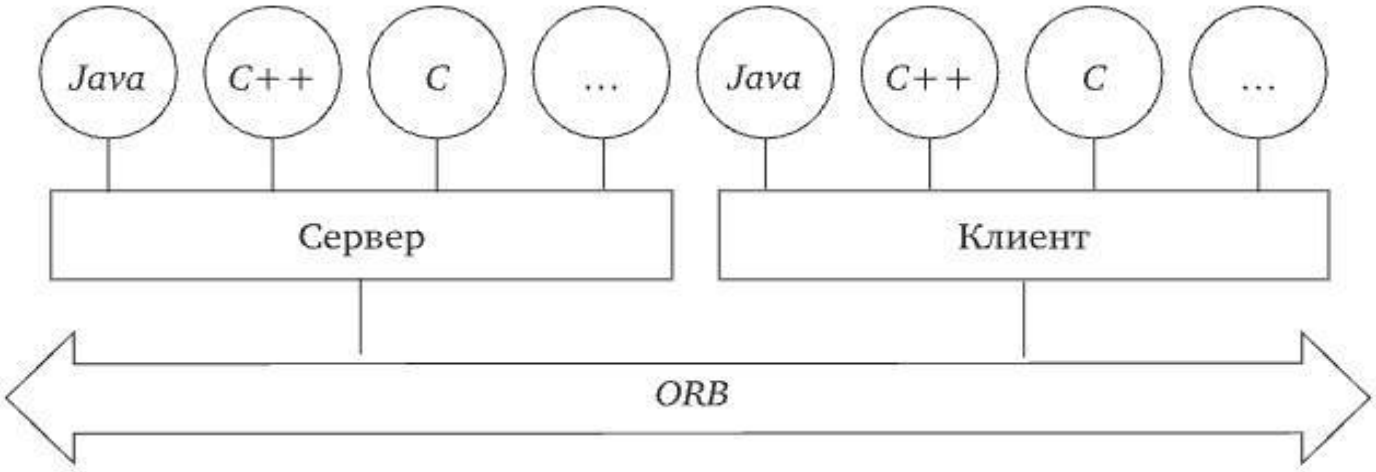


Рис. 7.8. **Обобщенная структура системы, построенной с использованием *CORBA***

Интерфейсы в *CORBA* определяются средствами языка *IDL* (*Interface Definition Language* — язык описания интерфейса). *IDL* описание используется для генерации кода заглушек для поддерживаемых языков программирования (*Java*, *C++*, *C*, *COBOL*, *Lisp*, *PL/I*, *Smalltalk*, *Python*). Сгенерированный код используется как основа для доступа к объекту в определенной программной среде.

Наибольшая популярность технологии *CORBA* пришлась на последние годы прошлого века и первые годы текущего века. *CORBA* всегда рассматривалась как «тяжелая» технология, ориентированная на решение сложных задач, преимущественно в сфере крупных корпоративных систем. Однако *CORBA* далеко не всегда используется исключительно в крупных приложениях. Специализированные

версии *CORBA* до сих пор работают в системах реального времени и малых встраиваемых системах.

С точки зрения реализации *CORBA* является спецификацией; она предоставляет описание для реализации конкретных продуктов. Существует множество компаний и сообществ, поставляющих продукты *CORBA* для различных языков программирования. Список поставщиков можно найти на сайте *CORBA*. Глобальная архитектура *CORBA* восходит к модели *OMG*, в рамках которой выделяется четыре группы архитектурных элементов (рис. 7.9):

1) брокер объектных запросов;
2) сервисы *CORBA*;
3) средства *CORBA*;
4) прикладные объекты и приложения.

Рис. 7.9. **Группы архитектурных элементов *CORBA***

Брокер объектных запросов (*ORB*) определяет объектную шину *CORBA*, Сервисы *CORBA* (*CORBA Services*) определяют системный уровень объектной структуры и могут рассматриваться как расширение объектной шины, Общие Средства *CORBA* (*CORBA Facilities*) определяют сервисы, которые непосредственно используются приложениями (бизнес-объектами), *Application Objects* представляют прикладные объекты и приложения, т. е. конечных потребителей инфраструктуры *CORBA*.

Каждый нижележащий уровень выступает как сервер для вышележащего.

В основе *CORBA* лежит понятие *объектной модели*. *Объектная модель* описывает объектные концепции и терминологию, применимые к клиентскому коду и реализации объектов. Объектная мо-

дель *CORBA* являет собой пример классической объектной модели, в которой клиенты посылают сообщения объектам, а объекты интерпретируют сообщения и выполняют соответствующие им действия.

В основе объектной системы лежит понятие *объекта* (*object*). *Объект* — идентифицируемая, инкапсулированная сущность, предоставляющая один или более сервисов, которые могут быть запрошены клиентами.

7.2.8. Технология *Enterprise JavaBeans* (*EJB*)

EJB представляет собой высокоуровневый подход к построению распределенных приложений масштаба предприятия, основанный на модели серверных компонентов (*server component model*) для *Java*.

Данный подход позволяет изменять архитектуру сервера, расширяя или ограничивая его функциональные возможности в зависимости от решаемых задач путем добавления или удаления компонент.

Краткая история. Фирма *Sun Microsystems Inc.* анонсировала технологию *EJB* в 1996 г., а в 1998 г. была представлена первая реализация (версия 1.0). В 2006 г. появилась версия *EJB* 3.0, которая трансформировалась в настоящее время в версии *EJB* 3.1, *EJB* 3.2. Новые версии имеют большее число расширений и более тесно интегрируется с другими технологиями *Java*.

Версии. *EJB 1.0.* Это начальная версия, в рамках которой поддерживались *stateful* и *stateless*-компонент, с которыми можно было работать через удаленный интерфейс. В качестве желательной опции рассматривались компоненты, имеющие состояния.

EJB 1.1. В данном релизе в качестве обязательных появились компоненты с состоянием и *XML*-дескриптор (*deployment descriptor*).

EJB 2.0. В данном релизе в дополнение к удаленному интерфейсу появился локальный интерфейс, который могли использовать компоненты, работающие в том же контейнере. Появился новый тип компонентов — компоненты, управляемые сообщениями The *Message-Driven Bean* (*MDB*), которые могли принимать асинхронные сообщения. Для компонентов с сохранением была определена возможность сохранения состояния средствами контейнера. Появился язык *Enterprise JavaBeans Query Language* (далее — *EJB QL*), похожий по структуре на *SQL* и предназначенный для поиска компонентов.

EJB 2.1. В рамках данного релиза появилась поддержка работы с веб-сервисами, расширились возможности *EJB QL* и в дескрипторе развертывания вместо *DTD* стала использоваться *XML schema*.

Можно заметить, что основные различия между версиями состоит в наращивании возможностей.

EJB 3.0. Данный релиз радикально отличается от предшествующих и определяет фактически новую компонентную модель. Основное отличие состоит в том, что пользователь не должен ниче-

го знать о домашнем интерфейсе (*Home Interface*). В рамках данной спецификации используется идея *Plain Old Java Objects* (*POJO*). Идея состоит в том, что программист пишет только ту часть кода, которая реализует бизнес-логику. Для того чтобы определить, каким именно способом реализуется бизнес-код, используется механизм аннотаций. Например, если требуется определить интерфейс как локальный, то достаточно описать интерфейс обычным способом и вставить строку аннотацию *@Local*. Механизм аннотаций позволяет очень существенно упростить процесс разработки.

Таким образом, речь идет о двух разных компонентных моделях и разных философиях разработки компонентом.

Использование *EJB* позволяет упростить процесс разработки КИС за счет того, что многие функции реализуются контейнерами. Кроме того, *EJB* можно использовать многократно, причем для повторного использования не требуется перекомпиляция всего проекта.

EJB компоненты хорошо сопрягаются с веб-сервисами и *CORBA*.

Архитектурная модель EJB включает в себя следующие элементы (рис. 7.10):

- *JEE*-контейнер;
- *EJB*-контейнер;
- веб-контейнер;
- клиент;
- БД.

Рис. 7.10. **Архитектурная модель EJB**

Одним из побудительных мотивов создания *EJB*-технологии были сложности *CORBA*. Однако эволюция технологии *EJB* привело к тому, что эта технология стала казаться разработчикам чрезмерно сложной, в частности:

- чрезмерно тяжеловесная модель программирования, требующая работы с несколькими интерфейсами;
- необходимость непосредственного взаимодействовать с *Java Naming Directory Interface* (*JNDI*);
- необходимость работать с непомерно сложным *XML DD*.

Эти недостатки устранены в *EJB*3.0, где используется только один бизнес-интерфейс вместо *home* и *remote* интерфейсов, минимизируется использование *DD* (англ. *dynamic document* — динамический

документ) за счет использования аннотаций. Кроме того, используется новый механизм для сохранения состояния для компонентов-сущностей (*Entity Beans*).

Одной из новых черт *EJB* 3.0 является использование *Java Persistence API* (*JPA*) для реализации сохранения состояния.

7.2.9. Сервис-ориентированная архитектура (SOA)

Сервис-ориентированная архитектура (***Service-Oriented Architecture — SOA***) — это подход к созданию ИС, основанный на использовании *сервисов* или *служб* (*service*). Ниже термины сервис и служба рассматриваются как синонимы. SOA — это, в первую очередь, интеграционная архитектура, использование которой позволяет обеспечить гибкую интеграцию ИС. При использовании SOA приложения взаимодействуют, вызывая сервисы, входящие в состав других приложений. Сервисы объединяются в более крупные последовательности, реализуя бизнес-процессы, которые могут быть доступны, как сервисы.

SOA можно рассматривать также как подход к построению слабосвязанных (*loosely coupled*) систем, реализующих механизмы асинхронного взаимодействия. К слабосвязанным системам обычно относят такие системы, как электронная почта и системы очередей сообщений.

Переход на SOA архитектуры позволяет решить следующие задачи:

- сократить сроки освоения и внедрения новых ИТ-систем, быстро создавать новые ИТ-системы на базе уже существующих;
- уменьшить суммарную стоимость владения ИТ-продуктом и стоимость их интеграции;
- увеличить срок жизни ИТ-систем за счет возможности их оперативной модернизации;
- использовать гибкие модели ценообразования путем передачи разработки детализированных бизнес-модулей сторонним производителям (аутсорсинг);
- уменьшить стоимость работ по интеграции, необходимой при слиянии и поглощении компаний;
- осуществить реализацию бизнес-процессов на уровне, не зависящем от приложений и платформ, используемых для поддержки процессов.

SOA — это интеграционная архитектура, основанная на концепции сервисов (служб).

Бизнес-функции и инфраструктурные функции, которые необходимы для построения распределенных систем, реализуются как сервисы, которые в сочетании или по отдельности обеспечивают прикладную функциональность либо другим сервисам, либо приложениям, взаимодействующим с конечным пользователем.

Концепция SOA предполагает использование единого механизма взаимодействия служб. Этот механизм строится на основе концеп-

ции свободных связей и должен поддерживать использование формальных интерфейсов.

СОА привносит преимущества, которые на основе слабосвязанности и инкапсуляции дают возможность объединить различные архитектурные стили и модели организации.

Сервисы можно рассматривать как строительные блоки, которые могут использоваться как для построения сервисов более высокого уровня, так и для построения законченных распределенных ИТ-систем.

Сервисы могут вызываться независимо внешними или внутренними потребителями для выполнения элементарных функций либо могут объединяться в цепочки для формирования более сложных функций и для быстрого создания новых функций.

Используя СОА, организации могут создавать гибкие КИС, которые позволяют оперативно реализовывать быстро изменяющиеся бизнес-процессы и многократно использовать одни и те же компоненты в рамках одной ИТ-системы, в рамках семейств продуктов и в независимых ИТ-системах.

7.3. Оценка качества информационных систем

Понятие *качества* ИС соответствует понятию о том, что система успешно справляется со всеми возлагаемыми на нее задачами, имеет хорошие показатели надежности, имеет приемлемую стоимость, удобна в эксплуатации и обслуживании, легко сопрягается с другими системами и, в случае необходимости, может быть модифицирована.

Разные группы пользователей имеют различные точки зрения на характеристики качества ИС. Например, если задать вопрос о том, какой должна быть хорошая ИС, то от пользователя можно, скорее всего, получить следующие варианты ответов:

- должна иметь хорошую производительность;
- должна иметь широкие функциональные возможности;
- должна быть удобна в эксплуатации;
- должна быть надежна.

Менеджер даст, скорее всего, другие варианты ответов:

- не должна быть очень дорогой;
- не должна быть очень дорогой в эксплуатации;
- не должна морально устаревать в течение возможно более длительного промежутка времени, а, в случае необходимости, может быть легко модифицирована.

Для системного администратора наиболее важными могут оказаться такие характеристики системы, как:

- должна быть надежна и стабильна в работе;
- должна быть проста в администрировании;

- должна иметь хорошую эксплуатационную документацию;
- должна иметь хорошую поддержку изготовителем.

Другие заинтересованные лица могут иметь свои точки зрения на то, какой должна быть качественная система.

Качество ИС связано с *дефектами*, заложенными на этапе проектирования и проявляющиеся в процессе эксплуатации. Любые свойства ИС, в том числе и дефектологические, могут проявляться лишь во взаимодействии с внешней средой, включающей технические средства, персонал, информационное и программное окружение.

В зависимости от целей исследования и этапов жизненного цикла ИС дефектологические свойства разделяют на дефектогенность, дефектабельность и дефектоскопичность.

Дефектогенность определяется влиянием следующих факторов:

- количество разработчиков ИС, их профессиональные и психофизиологические характеристики;
- условия и организация процесса разработки ИС;
- характеристики инструментальных средств и компонент ИС;
- сложность задач, решаемых ИС;
- степень агрессивности внешней среды (потенциальная возможность внешней среды вносить преднамеренные дефекты, например, воздействие вирусов).

Дефектабельность характеризует наличие дефектов ИС и определяется их количеством, местонахождением и др. факторами, влияющими на дефектабельность, являются:

- структурно-конструктивные особенности ИС;
- интенсивность и характеристики ошибок, приводящих к дефектам.

Дефектоскопичность характеризует возможность проявления дефектов в виде отказов и сбоев в процессе отладки, испытаний или эксплуатации. На дефектоскопичность влияют:

- количество, типы и характер распределения дефектов в ИС;
- устойчивость ИС к проявлению дефектов;
- характеристики средств контроля и диагностики дефектов;
- квалификация обслуживающего персонала.

Оценка качества ИС является крайне сложной задачей ввиду многообразия интересов пользователей. Поэтому невозможно предложить одну универсальную меру качества и приходится использовать ряд характеристик, охватывающих весь спектр предъявляемых требований. Наиболее близки к задачам оценки качества ИС модели качества программного обеспечения, являющегося одной из важных составных частей ИС. В настоящее время используется несколько абстрактных моделей качества программного обеспечения, основанных на определениях характеристики качества, показателя качества, критерия и метрики.

Критерий может быть определен как независимый атрибут ИС или процесса ее создания. С помощью такого критерия может быть измерена характеристика качества ИС на основе той или иной метрики. Совокупность нескольких критериев определяет показатель качества, формируемый исходя из требований, предъявляемых к ИС. В настоящее время наибольшее распространение получила иерархическая модель взаимосвязи компонент качества ИС. Вначале определяется характеристика качества, в числе которых могут быть, например, общая полезность, исходная полезность, удобство эксплуатации. Далее формируются показатели, к числу которых могут быть отнесены: практичность, целостность, корректность, удобство обслуживания, оцениваемость, гибкость, адаптируемость, мобильность, возможность взаимодействия. Каждому показателю качества ставится в соотвествие группа критериев. Для указанных выше показателей ниже приведены возможные критерии. Надо отметить, что один и тот же критерий может характеризовать несколько показателей.

С помощью метрик можно дать количественную или качественную оценку качества ИС. Различают следующие виды метрик и шкал для измерения критериев.

Первый тип — метрики, которые используют интервальную шкалу, характеризуемую относительными величинами или реально измеряемыми физическими показателями, например, временем наработки на отказ, вероятностью ошибки, объемом информации и др.

Второй тип — метрики, которым соответствует порядковая шкала, позволяющая ранжировать характеристики путем сравнения с опорными значениями.

Третий тип — метрики, которым соответствуют номинальная или категорированная шкала, определяющая наличие рассматриваемого свойства или признака у рассматриваемого объекта без учета градаций по этому признаку. Так, например, интерфейс может быть «простым для понимания», «умеренно простым», «сложным для понимания».

Развитием иерархического подхода является представленная на рис. 7.11 модель классификации критериев качества информационных систем. С помощью функциональных критериев оценивается степень выполнения ИС основных целей или задач. Конструктивные критерии предназначены для оценки компонент ИС, не зависящих от целевого назначения.

Поскольку в современных ИС ключевой компонентой является программная компонента, а пользователи, работающие с системой, в подавляющем большинстве случаев взаимодействуют непосредственно с программной компонентой, поэтому показатели качества информационных и программных систем в значительной степени совпадают.

Рис. 7.11. **Модель классификации критериев качества информационных систем**

Качество программного обеспечения определяется в стандарте *ISO* 9126 как вся совокупность его характеристик, относящихся к возможности удовлетворять высказанные или подразумеваемые потребности всех заинтересованных лиц.

Различаются понятия внутреннего качества, связанного с характеристиками ПО самого по себе, без учета его поведения; внешнего качества, характеризующего ПО с точки зрения его поведения; и *качества* ПО при использовании в различных контекстах — того качества, которое ощущается пользователями при конкретных сценариях работы ПО. Для всех этих аспектов качества введены метрики, позволяющие оценить их. Кроме того, для создания добротного ПО существенно качество технологических процессов его разработки.

ISO 9126 — это международный стандарт, определяющий оценочные характеристики качества программного обеспечения. Российский аналог стандарта ГОСТ 28195. Стандарт разделяется на четыре части, описывающие следующие вопросы: модель качества; внешние метрики качества; внутренние метрики качества; метрики качества в использовании.

Вторая и третья части стандарта *ISO* 9126-2,3 посвящены формализации соответственно внешних и внутренних метрик характеристик качества сложных программных систем. В ней изложены

содержание и общие рекомендации по использованию соответствующих метрик и взаимосвязей между типами метрик.

Четвертая часть стандарта *ISO* 9126-4 предназначена для покупателей, поставщиков, разработчиков, сопровождающих, пользователей и менеджеров качества программные системы (ПС). В ней повторена концепция трех типов метрик, а также аннотированы рекомендуемые виды измерений характеристик. Модель качества, установленная в первой части стандарта *ISO* 9126-1, классифицирует качество ПО в шести структурных наборах характеристик:

- функциональность;
- надежность;
- эффективность (производительность);
- удобство использования;
- удобство сопровождения;
- переносимость.

Перечисленные характеристики, в свою очередь, детализированы подхарактеристиками (субхарактеристиками) (рис. 7.12). Ниже приведены определения этих характеристик и атрибутов по стандарту *ISO* 9126:2001.

Функциональность (*functionality*) определяется как способность ПО в определенных условиях решать задачи, нужные пользователям.

Для данной характеристики выделяются следующие субхарактеристики:

- функциональная пригодность;
- точность;
- способность к взаимодействию;
- защищенность;
- соответствие стандартам и правилам.

Функциональная пригодность (*suitability*) — способность решать нужный набор задач.

Точность (*accuracy*) — способность выдавать нужные результаты.

Способность к взаимодействию (*interoperability*) — это способность взаимодействовать с нужным набором других систем.

Соответствие стандартам и правилам (*compliance*) — соответствие ПО имеющимся индустриальным стандартам, нормативным и законодательным актам, другим регулирующим нормам.

Защищенность (*security*) — способность предотвращать неавторизированный, т. е. без указания лица, пытающегося его осуществить, и неразрешенный доступ к данным и программам.

Надежность (*reliability*) — это способность ПО поддерживать определенную работоспособность в заданных условиях.

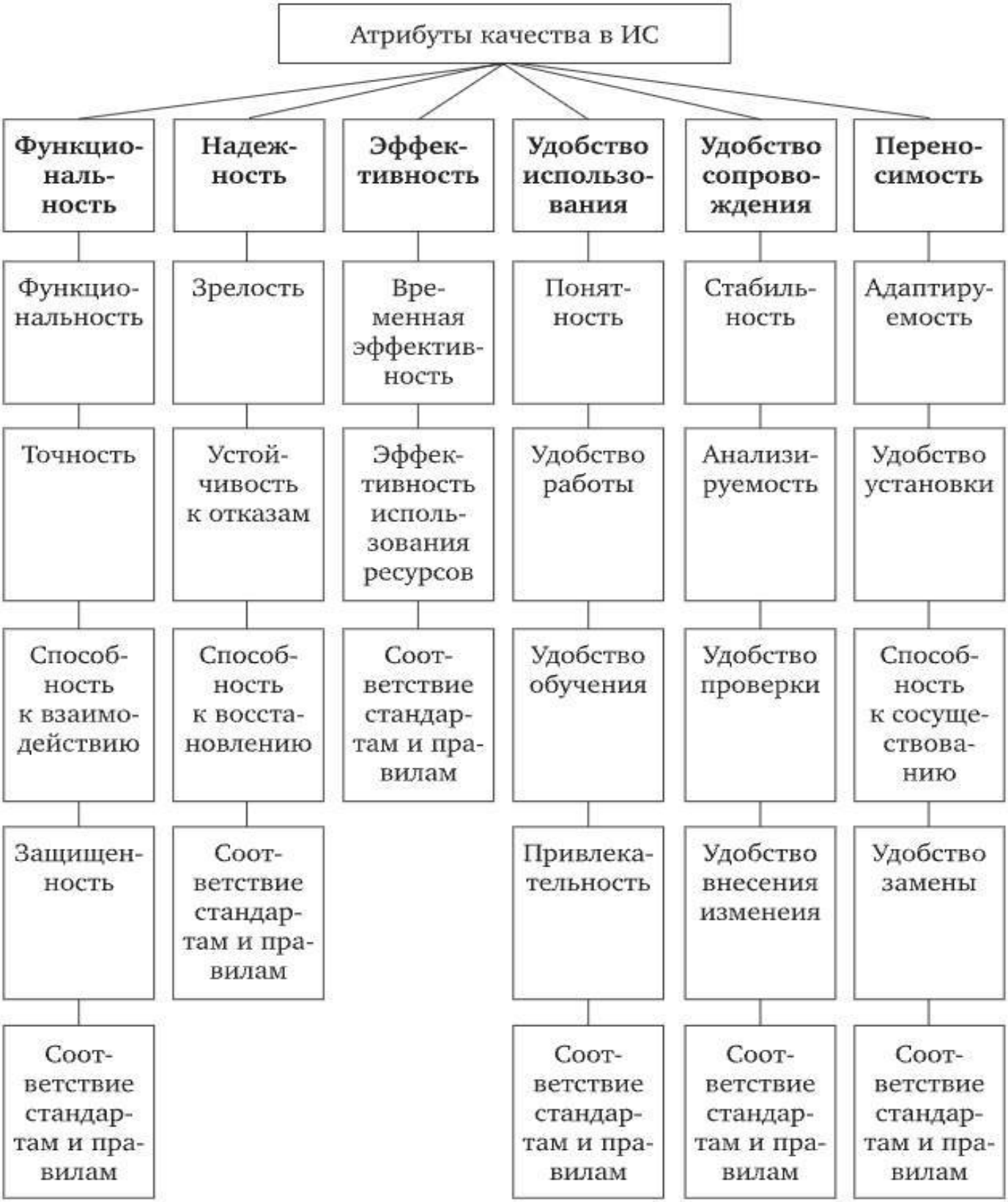


Рис. 7.12. **Характеристики качества ИС**

Для данной характеристики выделяются следующие субхарактеристики:

- зрелость;
- устойчивость к отказам;
- способность к восстановлению;
- соответствие стандартам.

Зрелость, завершенность (*maturity*) — величина, обратная частоте отказов ПО. Обычно измеряется средним временем работы

без сбоев и величиной, обратной вероятности возникновения отказа за данный период времени.

Устойчивость к отказам (*fault tolerance*) — способность поддерживать заданный уровень работоспособности при отказах и нарушениях правил взаимодействия с окружением.

Способность к восстановлению (*recoverability*) — способность восстанавливать определенный уровень работоспособности и целостность данных после отказа, необходимые для этого время и ресурсы.

Соответствие стандартам надежности (*reliability compliance*).

Удобство использования (*usability*), или практичность, определяется как способность ПО быть удобным в обучении и использовании, а также привлекательным для пользователей.

Для данной характеристики выделяются следующие субхарактеристики:

- понятность;
- удобство обучения;
- удобство работы;
- привлекательность;
- соответствие стандартам.

Понятность (*understandability*) — это показатель, обратный к усилиям, которые затрачиваются пользователями на восприятие основных понятий ПО и осознание их применимости для решения своих задач.

Удобство обучения (*learnability*) — показатель, обратный усилиям, затрачиваемым пользователями на обучение работе с ПО.

Удобство работы (*operability*) — это показатель, обратный усилиям, предпринимаемым пользователями для решения своих задач с помощью ПО.

Привлекательность (*attractiveness*) — это способность ПО быть привлекательным для пользователей. Этот атрибут добавлен в 2001 г.

Соответствие стандартам определяется как удобства использования (*usability compliance*).

Производительность (*efficiency*), или эффективность, — это способность ПО при заданных условиях обеспечивать необходимую работоспособность по отношению к выделяемым для этого ресурсам. Можно определить ее и как отношение получаемых с помощью ПО результатов к затрачиваемым на это ресурсам всех типов.

Для данной характеристики выделяются следующие субхарактеристики:

- временная эффективность;
- эффективность использования ресурсов;
- соответствие стандартам.

Временная эффективность (*time behaviour*) — способность ПО выдавать ожидаемые результаты, а также обеспечивать передачу необходимого объема данных за отведенное время.

Эффективность использования ресурсов (*resource utilisation*) — способность решать нужные задачи с использованием определенных объемов ресурсов определенных видов. Имеются в виду такие ресурсы, как оперативная и долговременная память, сетевые соединения, устройства ввода и вывода и пр.

Соответствие стандартам производительности (*efficiency compliance*).

Удобство сопровождения (*maintainability*) определяется как удобство проведения всех видов деятельности, связанных с сопровождение программ.

Для данной характеристики выделяются следующие субхарактеристики:

- анализируемость;
- удобство внесения изменений;
- стабильность;
- удобство проверки;
- соответствие стандартам.

Анализируемость (*analyzability*), или удобство проведения анализа, определяется как удобство проведения анализа ошибок, дефектов и недостатков, а также удобство анализа необходимости изменений и их возможных последствий.

Удобство внесения изменений (*changeability*) — показатель, обратный трудозатратам на выполнение необходимых изменений.

Стабильность (*stability*) — это показатель, обратный риску возникновения неожиданных эффектов при внесении необходимых изменений.

Удобство проверки (*testability*) — это показатель, обратный трудозатратам на проведение *тестирования* и других видов проверки того, что внесенные изменения привели к нужным результатам.

Соответствие стандартам удобства сопровождения (*maintainability compliance*).

Переносимость (*portability*) определяется как способность ПО сохранять работоспособность при переносе из одного окружения в другое, включая организационные, аппаратные и программные аспекты окружения.

Иногда эта характеристика называется в русскоязычной литературе мобильностью. Однако термин «мобильность» стоит зарезерви-

ровать для перевода «*mobility*» — способности ПО и компьютерной системы в целом сохранять работоспособность при ее физическом перемещении в пространстве.

Для данной характеристики выделяются следующие субхарактеристики:

- адаптируемость;
- удобство установки;
- способность к сосуществованию;
- удобство замены;
- соответствие стандартам;

Адаптируемость (*adaptability*) — способность ПО приспосабливаться различным окружениям без проведения для этого действий, помимо заранее предусмотренных.

Удобство установки (*installability*) — это способность ПО быть установленным или развернутым в определенном окружении.

Способность к сосуществованию (*coexistence*) — это способность ПО сосуществовать с другими программами в общем окружении, деля с ними ресурсы.

Удобство замены (*replaceability*) другого ПО данным определяется как возможность применения данного ПО вместо других программных систем для решения тех же задач в определенном окружении.

Соответствие стандартам *переносимости* (*portability compliance*).

Перечисленные атрибуты относятся к внутреннему и внешнему *качеству ПО* согласно *ISO* 9126. Для описания *качества ПО* при использовании стандарт *ISO* 9126-4 предлагает другой, более узкий набор характеристик:

- эффективность;
- продуктивность;
- безопасность;
- удовлетворение пользователей;
- эффективность (*effectiveness*).

Способность ПО предоставлять пользователям возможность решать их задачи с необходимой точностью при использовании в заданном контексте.

Продуктивность (*productivity*) — это способность ПО предоставлять пользователям определенные результаты в рамках ожидаемых затрат ресурсов.

Безопасность (*safety*) — это способность ПО обеспечивать необходимо низкий уровень риска нанесения ущерба жизни и здоровью людей, бизнесу, собственности или окружающей среде.

Удовлетворение пользователей (*satisfaction*) — это способность ПО приносить удовлетворение пользователям при использовании в заданном контексте.

Одним из путей обеспечения качества ИС является сертификация. В США Радиотехническая комиссия по аэронавтике в своем

руководящем документе определяет процесс сертификации следующим образом: «Сертификация — процесс официального утверждения государственным полномочным органом выполняемой функции системы путем удостоверения, что функция удовлетворяет всем требованиям заказчика, а также государственным нормативным документам». К сожалению, в настоящее время не существует стандартов, полностью удовлетворяющим оценке качества ИС. В западно-европейских странах имеется ряд стандартов, определяющих основы сертификации программных систем. Стандарт Великобритании (*BS*750) описывает структурные построения программных систем, при соблюдении которых может быть документ — гарантия качества на государственном уровне. Имеется международный аналог указанного стандарта (*ISO*9000) и аналог для стран членов НАТО (*AQAP*1). Существующая в нашей стране система нормативно-технических документов относит программное обеспечение к «продукции производственно-технического назначения», которая рассматривается как материальный объект. Однако программное обеспечение является, скорее, абстрактной нематериальной сферой. Существующие ГОСТы, например ГОСТ 28195—89. «Оценка качества программных средств. Общие положения», явно устарели и являются неполными.

Список литературы

1. *Граничин, О.* Информационные технологии в управлении / О. Граничин, В. Кияев. — URL: http://intuit.ru/studies/courses/1055/271/info (дата обращения: 09.08.2024).
2. *Цехановский В. В.* Проектирование информационных систем: архитектуры и платформы : учебное пособие / В. В. Цехановский, А. И. Водяхо — Москва : Ай Пи Ар Медиа, 2023. — 240 с.
3. *Кутузов О. И.* Инфокоммуникационные системы и сети : учебник для вузов / О. И. Кутузов, Т. М. Татарникова, В. В. Цехановский — Санкт-Петербург : Лань, 2020. — 244 с.
4. *Дмитриев, В. И.* Прикладная теория информации / В. И. Дмитриев. — Москва : Высшая школа, 1989.
5. *Казиев, В. М.* Информация: понятия, виды, получение, измерение и проблема обучения / В. М. Казиев // Информатика и образование. — 2000. — № 4. — С. 12—22.
6. *Макарова, Н. В.* Информатика / Н. В. Макарова. — Москва : Финансы и статистика, 2000.
7. *Дрот, В. Л.* Толковый словарь современной компьютерной лексики / В. Л. Дрот, Ф. А. Новиков. — Санкт-Петербург.: БХВ-Санкт-Петербург, 2004.
8. *Борзенко, А.* Новые технологии в центрах обработки данных / А. Борзенко. — URL: https://www.bytemag.ru/artides/detail.php?ro=12174 (дата обращения: 09.08.2024).
9. *Грей, Дж.* Управление данными: Прошлое, Настоящее и Будущее / Дж. Грей // Системы управления базами данных — 1998. — № 3. — С. 71—80.
10. *Городецкий В. И.* Искусственный интеллект — наука и информационная технология: настоящее и будущее / В. И. Городецкий, Р. М. Юсупов // сборник научных статей — Санкт-Петербург, 2020 — С. 10—12.
11. *Юсупов Р. М.* К семидесятилетию отечественной информатики / Р. М. Юсупов // Труды СПИИРАН. — 2019. — № 1. — С. 5—29.
12. *Советов, Б. Я.* Теория информационных процессов и систем / Б. Я. Советов [и др.] — Москва : Академия, 2010.
13. *Головин, Ю. А.* Информационные сети / Ю. А. Головин, А. А. Суконщиков, С. А. Яковлев. — Москва : Академия, 2013.
14. *Назаров, С. В.* Компьютерные технологии обработки информации / С. В. Назаров. — Москва : Финансы и статистика, 1999.

15. *Петровский, А. Б.* Теория принятия решений / А. Б. Петровский. — М. : Академия, 2009.

16. *Советов, Б. Я.* Интеллектуальные системы и технологии / Б. Я. Советов, В. В. Цехановский, В. Д. Чертовской. — Москва : Академия, 2013.

17. *Советов, Б. Я.* Базы данных : учебник для вузов / Б. Я. Советов, В. В. Цехановский, В. Д. Чертовской. — 4-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2024.

18. *Цехановский В. В.* Управление данными : учебник для вузов / В. В. Цехановский, В. Д. Чертовской. — Санкт-Петербург : Лань, 2020. 432 с.

19. *Саймон А.* Стратегия баз данных: менеджмент на 2000 год / А. Саймон. — Финансы и статистика, 1999.

20. *Падерно, П. И.* Введение в проектирование интеллектуальных интерфейсов / П. И. Падерно, С. Ф. Сергеев, Н. А. Назаренко. — Санкт-Петербург : СПбГУ ИТМО, 2011.

21. *Рудиметов, Е.* Аппаратные средства и мультимедиа : справочник / Е. Рудиметов. — Санкт-Петербург : Питер, 1999.

22. *Кошкарев,* А. В. Геоинформатика / А. В. Кошкарев, В. С. Тикунов. — Москва : Картоцентр-Геоиздат, 1998.

23. *Ведров, А. М. CASE*-технологии. Современные методы и средства проектирования информационных систем / А. М. Ведров — URL: http://citforum.ru/ database/case/index.shtml (дата обращения: 09.08.2024).

24. *Новоженов, Ю. В.* Объектно-ориентированные *CASE*-средства / Ю. В. Новоженов, М. З. Звонкин, Н. Н. Тимонин // Системы управления базами данных. — 1996. — № 5—6. — С. 119—125.

25. *Советов, Б. Я.* Архитектура информационных систем / Б. Я. Советов [и др.] — Москва : Академия, 2010.

26. Интернет вещей. —Электрон. дан. — Режим доступа: http://ru.wikipedia.org/wiki/ Интернет вещей (дата обращения: 09.08.2024).

27. *Кутузов, О. И.* Инфокоммуникационные системы и сети: учебник для вузов / О. И. Кутузов, Т. М. Татарникова, В. В. Цехановский. — Санкт-Петербург : Лань, 2020. — 244 с.

28. *Ли, П.* Архитектура интернета вещей / П. Ли. — Москва : ДМК Пресс, 2019. — 454 с.

29. *Srikanta, P.* New Paradigm of Industry 4.0 Internet of Things, Big Data & Cyber Physical Systems / P. Srikanta. — Springer, 2020 — 187 p.

30. *Grieves, M.* Digital twin: manufacturing excellence through virtual factory replication / M. Grieves. 2014. URL: https://web.archive.org/web/20170517031855/http://innovate.fit.edu/plm/documents/doc_mgr/912/1411.0_Digital_Twin_White_Paper_Dr_Grieves.pdf (дата обращения: 09.08.2024).

31. *Grieves, M.* Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems, in Trans-Disciplinary Perspectives on System Complexity / M. Grieves, J. Vickers. — Springer, 2016. — P. 85—114.

32. *Gould, L. S.* What Are Digital Twins and Digital Threads? / L. S. Gould. URL: https://www.gardnerweb.com/articles/what-are-digital-twins-and-digital-threads (дата обращения: 09.08.2024).

33. *Коварцев, А. Н.* Современные технологии разработки и тестирования программного обеспечения (ПО). Ч. 1: Разработка ПО средствами технологии графосимволического программирования : методические указания /А. Н. Коварцев. — Самара : Самар. гос. аэрокосмический ун-т, 2000.

34. *Орлик, С.* Основы программной инженерии. / С. Орлик. URL: http//swebok.sorlik.ru

35. *Соловьев, С. В.* Технология разработки прикладного программного обеспечения / С. В. Соловьев, Р. И. Цой, Л. С. Гринкруг. — Пенза : Академия Естествознания, 2011.

36. ГОСТ Р ИСО/МЭК 15910—2002 — Процесс создания документации пользователя программного средства.

37. ГОСТ 19781—90 — Единая система программной документации. Обеспечение систем обработки информации программное.

38. *Морозов, А. В.* Тестирование в современном образовательном процессе. проблемы проектирования тестов / А. В. Морозов, И. А. Кочанов. — URL: window.edu.ru/resource/432/62432/files/book425.pdf (дата обращения: 09.08.2024).

39. *Терехов, А. Н.* Введение в технологию программирования / А. Н. Терехов. — URL: http://intuit.ru/goods_store/ebooks/8300 (дата обращения: 09.08.2024).

40. *Gasevic, D.* Model Driven Architecture and Ontology Development / D. Gasevic, D. Djuric, V. Devedzi. — Berlin; Heidelberg: Springer-Verlag, 2006.

41. *Zheng, Q.* Software Architecture / Q. Zheng, X. Jiankuan, Z. Xiang. — Berlin, Heidelberg: Co-published by Zhejiang University Press, Hangzhou and Springer-Verlag GmbH, 2008. — 365 p.

42. *Taylor, R.* Software Architecture Foundations, Theory and Practice. / R. Taylor, N. Medvidovic, E. Dashofy. — Wiley, 2009 — 12 p.

43. Стандарт ISO/IEC 10746 Reference Model for Open Distributed Processing (RM-ODP) [Электронный ресурс]. — Электрон. дан. — Режим доступа: http://www.iso.org/iso/support/faqs.html (дата обращения: 09.08.2024).

44. *Foster, I.* Grid Services for Distributed System Integration / I. Foster, J. M. Nick, S. Tuecke // Computer. — 2002. — № 6. — P. 37—46

45. *Ван Стин, М.* Распределенные системы / М. ван Стин., Э. С. Таненбаум. — Москва : ДМК Пресс, 2021. — 584 с.

46. Linux Virtualization — Электрон. дан. — Режим доступа: http://www.ibm.com/developerworks/ru/library/l-linuxvirt/index.html (дата обращения: 09.08.2024).

47. Облачные_вычисления. — Электрон. дан. — Режим доступа: http://ru.wikipedia.org/wiki/Облачные_вычисления (дата обращения: 09.08.2024).

48. *Lea, P.* Internet of Things for Architects / P. Lea. — Birmingham, UK : Packt Publishing, 2018 – 499 p.

49. Open Fog Reference Architecture for Fog Computing — Электрон. дан. — Режим доступа: https://iiconsortium.org/pdf/Open Fog_Reference_Architecture_2_09_17.pdf (дата обращения: 09.08.2024).

50. Edge computing — Электрон. дан. — Режим доступа: https://www.gartner.com/en/information-technology/glossary/edge-computing (дата обращения: 09.08.2024).

51. *Buyya, R.* Fog and Edge Computing Principles and Paradigms / R. Buyya, S. N. Srirama — NJ, USA : John Wiley & Sons, 2019 — 490 p.

52. *Макаров, С. Л.* Arduino Uno и Raspberry Pi 3: от схемотехники к интернету вещей. / С. Л. Макаров. — Москва : ДМКПресс, 2018. — 204 с.

53. Точное земледелие. — Электрон. дан. — Режим доступа: http://ru.wikipedia.org/wiki/ Точное земледелие (дата обращения: 09.08.2024).

54. *Zhou F.* Mobile Edge Computing in Unmanned Aerial Vehicle Networks / F. Zhou [et al.] — URL: https://www.researchgate.net/publication/338673494_Mobile_Edge_Computing_in_Unmanned_Aerial_Vehicle_Networks (дата обращения: 09.08.2024).

55. *Wu, W.* Unmanned Aerial Vehicle Swarm-Enabled Edge Computing: Potentials, Promising Technologies, and Challenges / W. Wu [et al.] — URL: https://arxiv.org/pdf/2201.08517.pdf (дата обращения: 09.08.2024).

56. *Таненбаум, Э.* Компьютерные сети / Э. Таненбаум, Д. Уэзеролл — 5-е изд. — Санкт-Петербург : Питер, 2012. — 960 с.

57. *Рассел, С.* Искусственный интеллект: современный подход / С. Рассел, П. Норвиг — 2-е изд. — Москва : Вильямс, 2006. — 1408 с.

58. *Поспелов, Д. А.* Ситуационное управление. Теория и практика / Д. А. Поспелов. — Москва : Наука, 1986. — 145 с.

59. *Гаврилова Т. А.* Инженерия знаний. Модели и методы : учебник / Т. А. Гаврилова, Д. В. Кудрявцев, Д. И. Муромцев — Санкт-Петербург : Издательство «Лань», 2016. — 324 с.

60. Среда описания ресурса (RDF): понятия и абстрактный синтаксис. W3C Рекомендация от 10 Февраля 2004. — Электрон. дан. — Режим доступа: https://www.w3.org/2007/03/rdf_concepts_ru/ (дата обращения: 09.08.2024).

61. OWL, язык веб-онтологий. Руководство. Рекомендация W3C от 10 февраля 2004.

62. Указ Президента РФ «О развитии искусственного интеллекта в Российской Федерации» от 10 октября 2019 г. № 490. — Электрон. дан. — Режим доступа: http://www.kremlin.ru/acts/news/61785 (дата обращения: 09.08.2024).

63. *Советов, Б. Я.* Представление знаний в информационных системах / Б. Я. Советов, В. В. Цехановский, В. Д. Чертовской. — Москва : Академия, 2012.

64. Искусственные нейронные сети : учебник / под. ред. В. В. Цехановского. — Москва : КноРус, 2023. — 352 с.

65. *Хайкин, С.* Нейронные сети: полный курс. / С. Хайкин. — Москва : Издательский дом «Вильямс», 2006. — 1104 с.

66. *Аггарвал, Ч.* Нейронные сети и глубокое обучение : учебный курс / Ч. Аггарвал. — Санкт-Петербург : Диалектика, 2020. — 752 с.

67. Искусственные нейронные сети. Практикум : учебное пособие / под ред. В. В. Цехановского. — Москва : КноРус, 2024. — 381 с.

68. *Бутырский, Е. Ю.* Машинное обучение: учебник: [16+] / Е. Ю. Бутырский, В. В. Цехановский, Н. А. Жуков. — Москва : Директ-Медиа, 2023. – 368 с.

69. *Дубенецкий, В. А.* Проектирование корпоративных информационных систем / В. А. Дубенецкий, Б. Я. Советов, В. В. Цехановский. — Санкт-Петербург : Изд-во СПбГЭТУ «ЛЭТИ», 2013.

70. *Лав, Р.* Разработка ядра Linux / Р. Лав. — 2-е изд. — Москва : Вильямс, 2006 — 448 с.

71. *Мюллер, С.* Модернизация и ремонт ПК / С. Мюллер. — 17-е изд. — Москва : Вильямс, 2007.

72. Мейнфрейм. — Электрон. дан. — Режим доступа: https:// www.Hvelib.ru/book/1001025572- mejnfrejm (дата обращения: 09.08.2024).

73. *Королев, Л. Н.* Нейрокомпьютинг, нейросети и нейрокомпьютеры / Л. Н. Королев. — URL: http://podelise.ru/docs/index-614137. html (дата обращения: 09.08.2024).

74. Cloud Computing and Linux. — Электрон. дан. — Режим доступа: http://www.ibm.com/develo- perworks/ru/library/l-cloud-computing/index.html (дата обращения: 09.08.2024).

75. *Граничин, О. Н.* Создание гибридных сверхбыстрых компьютеров и системное программирование / О. Н. Граничин, С. Л. Молодцов. — Санкт-Петербург : Изд-во СПбГУ, 2006.

76. *Moore, H.* Cramming more components onto integrated circuits / H. Moore. // Electronics — 1965. — № 38. — P. 114—119.

77. *Сухомлин, В.* Итология — наука об инфлормационных технологиях. — URL: http://citforum.ru/ programming/prg96/sukhomlin. shtml (дата обращения: 09.08.2024).

78. *Цехановский, В. В.* Архитектор информационных систем: как проектировать большие системы / В. В. Цехановский, А. И. Водя-

хо. — Москва, Алма-Аты. : Ай Пи Ар Медиа, EDP Hub (Идипи Хаб), 2024. — 284 с.

79. *Советов, Б. Я.* Метолы и средства проектирования информационных систем и технологий : учебник для студентов вузов / Б. Я Советов, В. А. Дубенецкий, В. В. Цехановский. — Москва : Издательский центр «Академия», 2018. — 352 с.

80. *Hwang, K.* Distributed and Cloud Computing. From Parallel Processing to the Internet of Things / K. Hwang, G. Fox, J. Dongarra. — Waltham, MA, USA : Morgan Kaufmann, 2012. 648 p.